hushuqi 算法竞赛模板 (上)

ICPC/CCPC Algorithm Templates

目录

0. 比赛注意 8
0.1 赛前与比赛周 . . . 8
0.2 赛时心态与分工 . . . 8
0.3 开题与选题 . . . 8
0.4 上机与打印 . . . 8
0.5 卡题 . . . 9
0.6 提交前 . . . 9
0.7 AC . . . 9
0.8 WA . . . 9
0.9 TLE . . . 10
0.10 RE . . . 10

1. 基础与工程 11
1.1 基础代码框架 . . . 11
1.1.001 C++ 通用头文件与类型别名 . . . 11
1.1.002 标准 `main` / 多测框架 . . . 11
1.1.003 常量、无穷大、浮点误差与符号函数 . . . 12
1.1.004 常用函数重载与容器输出 . . . 12
1.1.005 Lambda 递归写法 . . . 13
1.1.006 C 语言基础模板与常用 C 库 . . . 14
1.1.007 Python 基础模板 . . . 14
1.1.008 静态取模类 . . . 15
1.2 输入输出 . . . 18
1.2.001 快速输入输出 . . . 18
1.2.002 `__int128` 输入输出 . . . 20
1.2.003 泛型读写封装 . . . 22
1.2.004 浮点数与输出精度控制 . . . 22
1.2.005 未知长度的一行输入 . . . 22
1.2.006 文件输入输出 . . . 23
1.2.007 手动扩大栈空间 . . . 23
1.2.008 无原生 `__int128` 时的双字 `i128` . . . 23
1.3 编译、调试与测试 . . . 27
1.3.001 编译参数与调试构建 . . . 27
1.3.002 Makefile（Windows / Linux） . . . 30
1.3.003 bashrc / vimrc . . . 30
1.3.004 对拍器（Windows / Linux） . . . 31
1.3.005 随机数据生成器 . . . 35
1.3.006 GNU C++ 环境、本地计时与评测机测速 . . . 35
1.3.007 调试输出与断言 . . . 37
1.3.008 常见未定义行为和易错点清单 . . . 38
1.4 STL、扩展库与防卡 . . . 38
1.4.001 `vector`、`array`、`string`、`tuple`、`pair` . . . 38

1.4.002 stack、queue、deque、priority_queue . . . 38
1.4.003 set、multiset、map、multimap . . . 39
1.4.004 unordered_map / unordered_set 防 hack 哈希 . . . 39
1.4.005 bitset 与手写 bitset . . . 40
1.4.006 GNU PBDS . . . 42
1.4.007 lower_bound、upper_bound、unique、iota . . . 43
1.4.008 next_permutation、prev_permutation、shuffle . . . 44
1.4.009 accumulate、is_sorted、迭代器 . . . 44
1.4.010 __builtin_* 与 C++20 位运算函数 . . . 44
1.4.011 sort killer 与哈希 hack . . . 44
1.4.012 何时才需要卡常 . . . 45
1.4.013 字符串分割：getline / Split . . . 45
1.4.014 optional . . . 46
1.5 平台优化与极限卡常 . . . 46
1.5.001 卡常使用边界与专题索引 . . . 47

2. 基础算法与离线算法 49
2.1 搜索与枚举 . . . 49
2.1.001 整数二分、实数二分 . . . 49
2.1.002 整数三分、实数三分、黄金分割三分 . . . 51
2.1.003 全排列与不重复排列 . . . 53
2.1.004 子集枚举、固定大小子集枚举 . . . 54
2.1.005 Gray Code . . . 54
2.1.006 DFS / BFS 基础搜索 . . . 55
2.1.007 启发式搜索与 A*（第 K 短路） . . . 56
2.1.008 模拟退火 . . . 58
2.1.009 DLX 精确覆盖 . . . 59
2.1.010 IDDFS、IDA* 与双向 BFS . . . 63
2.2 排序、分治与序列处理 . . . 67
2.2.001 快速排序、归并排序、基数排序 . . . 67
2.2.002 离散化与坐标压缩 . . . 69
2.2.003 逆序对及其扩展 . . . 70
2.2.004 整体二分 . . . 72
2.2.005 CDQ 分治与三维偏序 . . . 75
2.2.006 分数级联（Fractional Cascading） . . . 80
2.2.007 扫描线 . . . 82
2.2.008 单调栈、单调队列 . . . 84
2.2.009 滑动窗口与区间最值 . . . 86
2.2.010 最长上升子序列及输出方案 . . . 86
2.2.011 二维 LIS / LCIS . . . 87
2.2.012 最大全 1 子矩形 . . . 88
2.2.013 约瑟夫问题 . . . 89
2.2.014 快速选择 / 线性期望第 k 元素 . . . 90
2.2.015 折半搜索 . . . 91
2.3 莫队与离线查询 . . . 94

2.3.001 普通莫队 . . . 94
2.3.002 带修改莫队 . . . 97
2.3.003 回滚莫队 . . . 99
2.3.004 二次离线莫队 . . . 100
2.3.005 区间不同数统计 . . . 104
2.3.006 区间第 K 小整体二分 . . . 105
2.3.007 树上莫队 . . . 105
2.4 差分与前缀技巧 . . . 110
2.4.001 一维/二维前缀和 . . . 110
2.4.002 一维/二维差分 . . . 111
2.4.003 K 阶差分 . . . 113
2.4.004 子段和、子矩阵和 . . . 114
2.4.005 所有区间 GCD . . . 115
2.4.006 连续区间按位异或 . . . 116
2.4.007 根号分治：区间加与区间和 . . . 117

3. 数据结构 119
3.1 并查集 . . . 119
3.1.001 普通并查集 . . . 119
3.1.002 按大小/按秩合并 . . . 120
3.1.003 带权并查集 . . . 120
3.1.004 可撤销并查集 . . . 124
3.1.005 可持久化并查集 . . . 126
3.1.006 区间并行并查集 . . . 129
3.1.007 可删除并查集 . . . 131
3.1.008 非交换群势能并查集 . . . 133
3.2 树状数组 . . . 136
3.2.001 单点修改、前缀和 . . . 136
3.2.002 单点赋值、区间和 . . . 139
3.2.003 前缀和上的二分 . . . 140
3.2.004 区间修改、区间求和 . . . 141
3.2.005 前缀/后缀最大值 . . . 142
3.2.006 后缀和 . . . 143
3.2.007 取模树状数组 . . . 144
3.2.008 二维树状数组 . . . 145
3.2.009 矩形修改、矩形求和 . . . 147
3.2.010 静态点集矩形和 . . . 150
3.2.011 矩形加、单点求值 . . . 153
3.3 线段树家族 . . . 153
3.3.001 单点修改线段树与摘要二分 . . . 153
3.3.002 懒标记线段树 . . . 157
3.3.003 区间加、乘、赋值（仿射懒标记） . . . 163
3.3.004 区间加、区间最小值 . . . 169
3.3.005 ZKW 线段树 . . . 172
3.3.006 非递归线段树 . . . 173

3.3.007 动态开点线段树 . . . 174
3.3.008 可持久化线段树 / 主席树 . . . 178
3.3.009 线段树合并 . . . 181
3.3.010 线段树分裂 . . . 184
3.3.011 Segment Tree Beats . . . 188
3.3.012 扫描线线段树 . . . 194
3.3.013 矩形面积并、周长并 . . . 198
3.3.014 静态矩形加、矩形和 . . . 198
3.3.015 扫描线求矩形至少二重覆盖面积 . . . 198
3.3.016 线段树分治 . . . 201
3.3.017 动态区间严格众数 . . . 203
3.3.018 可排序线段树 . . . 207
3.3.019 区间一次函数加、区间最小值 . . . 213
3.3.020 可持久化仿射区间和 . . . 218
3.3.021 稀疏仿射区间和 . . . 224
3.3.022 区间赋同函数、区间复合 . . . 227
3.4 RMQ 与静态区间结构 . . . 231
3.4.001 ST 表 . . . 231
3.4.002 基于状压的 RMQ . . . 234
3.4.003 静态区间半群 . . . 238
3.4.004 笛卡尔树 . . . 239
3.4.005 小波矩阵 . . . 241
3.4.006 区间第 K 小、排名、前驱后继 . . . 246
3.4.007 离线静态区间频次 . . . 246
3.4.008 静态区间众数（秩优化） . . . 247
3.4.009 静态区间 LIS 查询 . . . 250
3.4.010 静态区间不同值个数 . . . 257
3.4.011 静态区间逆序对查询 . . . 257
3.4.012 静态区间和 . . . 260
3.4.013 Fischer-Heun 线性预处理 RMQ . . . 260
3.5 平衡树与堆 . . . 264
3.5.001 Treap . . . 264
3.5.002 FHQ Treap . . . 269
3.5.003 可持久化 Treap . . . 273
3.5.004 Splay . . . 277
3.5.005 Splay 重构 . . . 283
3.5.006 珂朵莉树（ODT） . . . 284
3.5.007 PBDS 平衡树 . . . 287
3.5.008 左偏树 / 可并堆 . . . 287
3.5.009 带删堆 . . . 289
3.5.010 Top-K 和维护 . . . 291
3.5.011 隐式 Treap（动态序列、翻转、仿射与区间和） . . . 293
3.5.012 可持久化队列 . . . 299
3.5.013 Radix Heap / 基数堆 . . . 302

3.6 Trie 与集合结构 . . . 304
3.6.001 字符 Trie . . . 304
3.6.002 01 Trie / 压位 Trie . . . 307
3.6.003 可删二进制 Trie . . . 312
3.6.004 Trie Graph . . . 313
3.6.005 线段树优化 Trie Graph . . . 315
3.6.006 哈希表 . . . 318
3.6.007 可持久化数组 . . . 322
3.6.008 手写 bitset . . . 324
3.6.009 动态区间频率索引 . . . 328
3.6.010 可持久化 01 Trie . . . 330
3.6.011 静态区间 MEX . . . 333
3.6.012 按标签维护的 01 Trie . . . 335
3.6.013 左儿子右兄弟 Trie . . . 337
3.6.014 分层位集前驱/后继集合 . . . 339
3.7 函数与空间数据结构 . . . 343
3.7.001 李超线段树 . . . 343
3.7.002 动态开点李超树 . . . 345
3.7.003 可持久化李超树 . . . 348
3.7.004 K-D Tree . . . 351
3.7.005 K-D Tree 二进制分组 . . . 354
3.7.006 分散层叠 . . . 357
3.7.007 双端队列全局查询 . . . 357
3.7.008 析合树（$O(n \log n)$ 构造） . . . 360
3.7.009 动态点集矩形仿射修改与矩形和 . . . 365
3.7.010 离散坐标线段李超树（线段插入、点查询） . . . 369
3.8 线性基 . . . 372
3.8.001 异或线性基 . . . 372
3.8.002 第 K 大线性基 . . . 375
3.8.003 区间线性基 . . . 377
3.8.004 线性基合并 . . . 379
3.9 GNU PBDS . . . 380
3.9.001 PBDS 头文件、命名空间、模板策略与迭代器体系 . . . 380
3.9.002 PBDS `priority_queue` 策略 . . . 381
3.9.003 PBDS 堆的 `modify`、`erase`、`join` 与点迭代器 . . . 382
3.9.004 PBDS 堆和 STL 堆、二叉堆、线段树的性能与适用范围 . . . 382
3.9.005 `tree`：`rb_tree_tag`、`splay_tree_tag`、`ov_tree_tag` . . . 383
3.9.006 PBDS `tree` 的 set/map 用法、`split` 与 `join` . . . 383
3.9.007 `tree_order_statistics_node_update` . . . 384
3.9.008 `find_by_order` 与 `order_of_key` . . . 384
3.9.009 自定义 `Node_Update`、`metadata_type` 与节点迭代器 . . . 384
3.9.010 自定义子树信息：区间和、前缀和及其他可合并信息 . . . 385
3.9.011 `cc_hash_table` 与 `gp_hash_table` . . . 387
3.9.012 哈希策略、探测策略、扩容策略与性能测试 . . . 387

3.9.013 编译器版本兼容性、常数、失效规则与不适用场景 . . . 387
3.10 根号分块 . . . 388
3.10.001 基础数列分块 . . . 388
3.10.002 有序数列分块 . . . 391
3.10.003 王室联邦树分块 . . . 395
3.10.004 图上按度数分块 . . . 396

4. 字符串 398
4.1 基础匹配与哈希 . . . 398
4.1.001 字符串哈希（Mersenne 单模、随机底数） . . . 398
4.1.002 KMP / 前缀函数 . . . 403
4.1.003 扩展 KMP / Z 函数 . . . 406
4.1.004 Manacher . . . 407
4.1.005 最小表示法 . . . 409
4.1.006 Lyndon 分解 . . . 410
4.1.007 Runs . . . 411
4.1.008 Karp-Rabin . . . 414
4.1.009 Sunday Algorithm . . . 414
4.1.010 `strstr` 类匹配 . . . 415
4.1.011 通配符模式匹配 . . . 416
4.1.012 随机单卷积通配符匹配 . . . 417
4.1.013 编辑距离与 Myers 位并行 . . . 419
4.2 Trie 与多模式匹配 . . . 421
4.2.001 Trie . . . 421
4.2.002 AC 自动机 . . . 423
4.2.003 AC 自动机 fail 树应用 . . . 425
4.2.004 子序列自动机 . . . 426
4.3 后缀结构 . . . 427
4.3.001 后缀数组：倍增法 . . . 427
4.3.002 后缀数组：SA-IS . . . 428
4.3.003 LCP / Height / RMQ 应用 . . . 432
4.3.004 后缀自动机 SAM . . . 435
4.3.005 广义 SAM / ExSAM . . . 439
4.3.006 在线广义 SAM . . . 443
4.3.007 SAMSA / CDAWG . . . 445
4.3.008 隐式后缀树 Ukkonen . . . 447
4.3.009 后缀平衡树（前插、回滚与动态排名） . . . 450
4.3.010 区间本质不同子串 . . . 456
4.3.011 Dictionary of Basic Factors . . . 458
4.3.012 BWT / FM-index . . . 463
4.4 回文结构 . . . 466
4.4.001 回文自动机 PAM / 回文树 . . . 466
4.4.002 双端可撤销回文树 . . . 468
4.4.003 Palindrome Series . . . 474
4.4.004 SqAM . . . 476

4.5 子序列与高级问题 477
4.5.001 最长公共子序列 LCS 477
4.5.002 Bitset 优化 LCS 478
4.5.003 区间 LCS / 区间 LIS 479
4.5.004 多串最长公共子串/子序列 480
4.5.005 最短公共超序列 484
4.5.006 字符串周期、循环节与前后缀 485
4.5.007 前缀-子串 LCS 批量查询 485

0. 比赛注意

0.1 赛前与比赛周

到赛地后记住自己是来比赛的，状态比临时多学一点更重要。

1. 赛前一天睡够 10 小时以上；前一天晚饭和当天早饭要吃好。
2. 到新环境注意防病。感冒、肠炎不算大病，却最影响成绩。
3. 比赛周不学新算法，也不连续几天高强度训练。可以看看 M67 等博客、翻小学奥数题，保持思维活跃。
4. 平时每场练习赛都按正式比赛打：所有题都要看过，赛后尽量补完没做出的题。
5. 有条件就看看往届比赛的总结、照片和录像，熟悉正式赛场，避免临场紧张到做不出题。
6. 队内最好三人都能做各类题，但各有侧重。要保证每出现一道题，都有人会做、敢做，至少知道做法。

0.2 赛时心态与分工

可以紧张，但不能慌张，慌张最影响出题。任何时候发现自己或队友慌了，就提醒对方深呼吸。

1. 第一道签到题交给队里最稳的人。遇到麻烦也别慌，很多队都过了就更不用急：它一定是简单题，晚几分钟也比罚时 20 分钟好。
2. 除签到题外尽量每人都看，并写下自己一眼能看出的观察。无论有人过没过，所有题都必须读完，最好每题都有两人以上读过，尽量避免靠讲题传题意。
3. 完全弄清题意，正确判断难易，不要想当然。觉得某题更适合队友做就转手；非万不得已不要中途换人写同一题。
4. 英语看不懂就查词典，偷懒一次可能就少一道题。
5. 比赛发的餐食留到赛后吃。
6. 最后一小时是出题高峰，谁松懈谁落后。最后一小时出一道是正常，出两道更好。
7. 没思路或头晕时，可以去洗手间洗把脸，或在座位旁站一会儿。

0.3 开题与选题

1. 时空效率危险且代码量大的题优先级放低。
2. 除非很多队都过了，复杂几何题留到最后考虑，精度带来的问题太多。
3. 非签到的构造题往往要想很久，过的队不多时不要开。
4. 尽量多想题目能用上的数学性质。
5. 常规方法不行时换思路：不常见的数据结构、加预处理、限时搜索等。效率第一，觉得 DP 难写就用记忆化搜索，想清楚后用最短时间出题。
6. 卡在一道全场通过率很高的题（按本队实力，例如超过 60% 的队伍通过）超过 1 小时，可以请求整队重做这题，不沿用任何旧代码。

0.4 上机与打印

1. 先在纸上把程序写得尽量完整再上机。每道题上机时间（输入、测试、调试合计）不超过 1 小时。

2. 有队友等着写题时，当前 coder 对着键盘发呆超过 3 分钟就立即下机，说明这题还没想好；下机时打印代码。
3. 程序出错一时排除不了，就打印出来读，把机器让给队友。
4. 每道题交完先打印，不等评测结果，直接写下一题。
5. 少复制粘贴，重复逻辑写成函数便于修改。必须块复制时，逐处检查对应部分是否已经改对。
6. 照纸敲代码和样例时不要敲错，尤其是字符串、提示文字等文本信息；不要自作主张改动模板。

0.5 卡题

不要急。通过队伍多的题不一定难，不要一开始就把题目想得过于复杂。

1. 一个人想 5 分钟没有思路，立即找队友讨论；再过 10 分钟还不会，再找第三个队友。队友在忙就先放下这题去看别的，并留下观察。
2. 把观察到的任何有用的点都记下来，不论看起来多显然。
3. 重新仔细阅读题面，思考每个可能有用的条件，尤其是数据范围。
4. 尝试倒着思考，从目标状态或答案反推必要条件。
5. 在题单上写下已有想法，整理已经确定的结论和仍待验证的分支。
6. 寻求队友帮助时直接说明卡住的具体位置，不要只转述题意。
7. 猜到结论但证明不了时，多造几组数据验证结论。
8. 要否定队友的结论，给出一组反例，而不是只说” 显然不对”。
9. 觉得思路正确时多测试几个样例，优先补充边界情况。

0.6 提交前

恭喜你马上要通过这一题了，但不要因为样例正确就放松检查。比赛中要比平时更稳，尽量 1Y。

1. 手算至少 3 组样例，必须包含边界情况；这通常花费不到 5 分钟，不要省略。
2. 检查程序中的关键位置、需要初始化的变量和数组大小。
3. 重新估算时间复杂度和空间复杂度，确认还有没有可优化的部分。
4. 删除调试输出和临时断言。
5. 提交代码。
6. 打印代码前再确认提交的是最后修改并验证过的版本。

0.7 AC

恭喜，通过后去看下一题；若这份代码以后要写入模板，再补充边界测试和接口说明。

0.8 WA

能通过样例只说明代码距离正确不远，不能说明实现已经覆盖所有情况。压力下也要仔细：不要改一处就交，很可能还有别处错误。

1. 手算至少 5 组样例，必须包含边界情况。测试要完整，每个分支都测到，数据不必特别复杂，例如矩阵为 1×1、$1 \times n$、$m \times 1$。
2. 检查是否会溢出 `long long` 或 `__int128` 的取值范围。
3. 检查所有需要初始化的变量。
4. 删除调试信息，确认调试分支不会影响提交代码。
5. 检查数组的行列含义是否写反。
6. 检查排序方向是从小到大还是从大到小，严格与非严格关系是否一致。
7. 检查输出格式是否匹配题面，包括前导零、空格、换行和精度。
8. 使用随机数据进行多组对拍，缩小到最小反例。
9. 重新读一遍题意，逐句确认自己的理解没有遗漏。
10. 让队友根据题面独立构造几组样例。
11. 使用小黄鸭调试法，逐行解释代码并重点检查变量名是否写错。

0.9 TLE

程序已经接近正确时，先排除实现死循环，再确认复杂度和数据规模。

1. 优先检查是否存在死循环、递归不收敛或队列没有及时清空。
2. 重新核对复杂度，确认最坏情况而不是平均情况能够通过。
3. 检查读入数据的格式和题目约定是否一致，避免把错误输入当成算法瓶颈。
4. 评估能否用线性数据结构替代 `map`、`set` 等对数结构。
5. 删除调试输出。
6. 识别可以预处理的量，避免在每次查询或每个状态中重复计算。
7. 使用小黄鸭调试法，逐行检查循环边界、状态转移和变量名。

0.10 RE

运行时错误通常来自越界、非法数学运算或资源使用不当。先根据错误类型缩小范围。

1. 检查数组是否开小，所有下标是否满足题面和实现的边界。
2. 检查是否存在除以零、对负数开平方等非法运算。
3. 检查输入格式是否和题目匹配，读入失败后是否继续使用了无效数据。
4. 检查数组是否开得过大；内存超限有时会以 RE 的形式返回。
5. 检查循环边界和递归终止条件。
6. 使用小黄鸭调试法，逐行检查变量名、引用和指针是否写错。

1. 基础与工程

1.1 基础代码框架

1.1.001 C++ 通用头文件与类型别名

统一头文件、整数宽度、无穷大与浮点容差是全书其余代码的公共前导，后续章节不再重复。`#define int long long` 必须写在系统头之后；若先于 `<bits/stdc++.h>` 展开，标准库中的 `int` 也会被替换，代码无法编译。宏展开后 `INF` 的类型和值分别是 `long long` 与 `numeric_limits<long long>::max()/4`。它只适合在已经证明数值范围受控时作哨兵；可能接近完整整数范围的算法必须把 “不可达” 与数值分开表示。

```cpp
#include <bits/stdc++.h>

using namespace std;

#define int long long

constexpr long long INF = numeric_limits<long long>::max() / 4;
constexpr long double EPS = 1E-12L;
```

一般整数按题目范围使用 `int`；外部二进制格式或系统接口要求固定位宽时使用 `int`、`unsigned long long` 等明确类型。乘积可能超 9×10^{18} 时在运算前转成 `__int128`，见 1.2.002 `__int128` 输入输出。

1.1.002 标准 `main` / 多测框架

`signed main()` 是规避 `int` 宏的常用写法，因为单独的 `signed` 等价于未被宏替换的 `signed int`。它不是唯一写法；也可以不用该宏，或在入口前 `#undef int` 后使用标准的 `int main()`。单测逻辑放进 `solve()`，多测只在 `main` 中循环调用，便于对拍时复用同一份 `solve()`。

```cpp
void solve()
{
    // 处理一组数据，n 为本组规模
    int n;
    cin >> n;
}

signed main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t = 1;
    cin >> t;          // 单测题删除本行，保留 t = 1
    while (t--)
    {
        solve();
    }
}
```

交互题不要解绑 cin/cout，且每次询问后 cout << flush，否则输出滞留缓冲区会导致死等。框架额外开销 $O(t)$，总复杂度为各组 solve() 之和。

1.1.003 常量、无穷大、浮点误差与符号函数

不能用浮点字面量表达要求精确的整数，例如 100000000000000001LL 转成 double 后会舍入；纯整数常量应写整数后缀。sgn 只处理 “按给定绝对容差与零比较”，返回 -1、0 或 1。

```cpp
// 依赖 1.1.001 C++ 通用头文件与类型别名
int sgn(long double x, long double eps = EPS)
{
    // 返回 x 相对 0 的符号：-1、0 或 1
    return (x > eps) - (x < -eps);
}
```

用法如 if (sgn(cross, eps) == 0) 判共线。eps 必须按数据尺度和量纲选择；比较两个一般浮点数时还要考虑相对误差或 ULP，不能把固定绝对 EPS 当成通用判据。整数几何优先使用精确叉积。

1.1.004 常用函数重载与容器输出

chmin/chmax 把 “比较、赋值、返回是否更新” 三步并成一次调用，返回值还能顺带记录转移来源（如最短路的前驱）。vector 的流重载只为本地对拍省样板，正式提交若与他人代码合并需留意命名冲突。

```cpp
template <class T> bool chmin(T &x, const T &y)
{
    // y < x 时把 x 降到 y 并返回 true
    if (y < x)
    {
        x = y;
        return true;
    }
    return false;
}

template <class T> bool chmax(T &x, const T &y)
{
    // x < y 时把 x 升到 y 并返回 true
    if (x < y)
    {
        x = y;
        return true;
    }
    return false;
}

template <class T> istream &operator>>(istream &in, vector<T> &a)
{
    // 按 a 的既有长度依次读入
    for (auto &x : a)
```

```
    {
        in >> x;
    }
    return in;
}

template <class T> ostream &operator<<(ostream &out, const vector<T> &a)
{
    // 空格分隔输出，不带行尾空格
    for (int i = 0; i < (int)a.size(); i++)
    {
        if (i)
        {
            out << ' ';
        }
        out << a[i];
    }
    return out;
}
```

`vector<int> a(n); cin >> a;` 依赖预设长度，重载不改变容器大小。`chmin/chmax` 要求两个实参类型相同，避免混合有符号比较或隐式窄化；需要转换时由调用处显式完成。更新为 $O(1)$，容器读写为 $O(n)$。

1.1.005 Lambda 递归写法

只服务单个算法的 DFS 留在函数内部，避免全局状态与冗长函数名。C++ 里 lambda 无法直接引用自身，用 `auto &&self` 显式传入（Y 组合子写法）；有返回值时必须写出 `→ T`，否则编译器推不出递归返回类型。

```
auto dfs = [&](auto &&self, int x, int fa) -> void
{
    // self 传自身用于递归，fa 为父结点，dep 记录深度
    for (auto y : g[x])
    {
        if (y == fa)
        {
            continue;
        }
        dep[y] = dep[x] + 1;
        self(self, y, x);
    }
};
dfs(dfs, 0, -1);
```

递归深度可能达 $O(n)$（链状图）时须确认系统栈足够，必要时参见 1.2.007 手动扩大栈空间；本模板仍直接使用递归函数或递归 lambda，不用容器模拟调用帧。复杂度与普通递归一致，每层仅多传一个引用。

1.1.006 C 语言基础模板与常用 C 库

仅用于强制 C、嵌入式接口或评测明确要求 `.c` 的场景，不与本模板的 C++ 宏混用。64 位整数的读写用 `<inttypes.h>` 的 `SCNd64`/`PRId64` 宏，跨平台格式串才正确。

```c
#include <assert.h>
#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void)
{
    int64_t n;
    if (scanf("%" SCNd64, &n) != 1)
    {
        return 0;
    }
    printf("%" PRId64 "\n", n);
    return 0;
}
```

`malloc/calloc` 后必须 `free`；`memset` 等库函数复杂度取决于参数（$O(n)$），框架本身 $O(1)$。

1.1.007 Python 基础模板

适合大整数、字符串处理和对拍暴力。`input = sys.stdin.buffer.readline` 绕开 Python 慢速行读取；正式解题前先估常数与递归深度，Python 递归默认上限约 1000 层。

```python
import sys

input = sys.stdin.buffer.readline

def solve() -> None:
    n = int(input())
    a = list(map(int, input().split()))
    print(sum(a[:n]))

def main() -> None:
    t = int(input())
    for _ in range(t):
        solve()

if __name__ == "__main__":
    main()
```

未知组数时遍历 `sys.stdin.buffer`；深递归优先改迭代，确需时再 `sys.setrecursionlimit`（并注意实际系统栈限制）。

1.1.008 静态取模类

模数在编译期固定且不超过 10^9+7 时，把它做成模板参数 `MInt<P>`，使有限域上的加、减、乘、除、幂与逆元统一使用值语义接口。代表元固定用 32 位无符号整数保存，乘法显式提升到 64 位；这一布局不受公共 `#define int long long` 影响，适合组合数、矩阵、多项式与 NTT 的大数组。构造时把整数规范到 $0,P)$。`inv()` 与除法使用费马小定理，只在 P 为素数且除数非零时有效；只需环上的加减乘时不要求 P 为素数。运行期模数使用 [7.1.008 动态取模类，接近 64 位上界或合数模的专用算法使用 7.1.001 快速幂的宽乘法原语，不应复制一份私有取模类。

```
template <uint32_t P> struct MInt
{
    static_assert(1 < P && P <= 1000000007);
    uint32_t x = 0; // 固定 32 位以保证对象为 4 字节，提高大数组的缓存密度。

    MInt(long long v = 0)
    {
        // v 是要转入模 P 剩余类的整数；构造其最小非负代表元。
        v %= (long long)P;
        if (v < 0)
        {
            v += P;
        }
        x = (uint32_t)v;
    }

    static constexpr uint32_t mod()
    {
        // 无参数；返回编译期模数 P。
        return P;
    }

    uint32_t val() const
    {
        // 无参数；返回当前剩余类的最小非负代表元。
        return x;
    }

    MInt operator-() const
    {
        // 无参数；返回当前剩余类的加法逆元。
        MInt ans;
        ans.x = x ? P - x : 0;
        return ans;
    }

    MInt &operator+=(const MInt &o)
    {
        // o 是要加到当前值上的同模剩余类；原地完成模加并返回当前对象引用。
        x += o.x;
        if (x >= P)
        {
```

```
            x -= P;
        }
        return *this;
    }

    MInt &operator-=(const MInt &o)
    {
        // o 是要从当前值减去的同模剩余类；原地完成模减并返回当前对象引用。
        x += P - o.x;
        if (x >= P)
        {
            x -= P;
        }
        return *this;
    }

    MInt &operator*=(const MInt &o)
    {
        // o 是要乘到当前值上的同模剩余类；原地完成模乘并返回当前对象引用。
        // 乘积可能超过 32 位，显式提升到无符号 64 位。
        x = (uint64_t)x * o.x % P;
        return *this;
    }

    MInt &operator/=(const MInt &o)
    {
        // o 是非零同模剩余类且 P 必须为素数；原地乘以 o 的逆元并返回当前对象引用。
        return *this *= o.inv();
    }

    friend MInt operator+(MInt a, const MInt &b)
    {
        // a、b 是同模剩余类；返回 a+b。
        return a += b;
    }

    friend MInt operator-(MInt a, const MInt &b)
    {
        // a、b 是同模剩余类；返回 a-b。
        return a -= b;
    }

    friend MInt operator*(MInt a, const MInt &b)
    {
        // a、b 是同模剩余类；返回 a*b。
        return a *= b;
    }

    friend MInt operator/(MInt a, const MInt &b)
    {
        // a、b 是同模剩余类且 b 非零；返回 a/b。
        return a /= b;
```

```cpp
    }

    friend bool operator==(const MInt &, const MInt &) = default;

    MInt pow(uint64_t b) const
    {
        // b 是非负指数；返回当前剩余类的 b 次幂。
        MInt a = *this;
        MInt ans = 1;
        while (b)
        {
            if (b & 1)
            {
                ans *= a;
            }
            a *= a;
            b >>= 1;
        }
        return ans;
    }

    MInt inv() const
    {
        // 当前值必须非零且 P 必须为素数；返回乘法逆元。
        assert(x != 0); // 调试检查，可删
        return pow(P - 2);
    }
};

template <uint32_t P> const vector<MInt<P>> &invTable(int n)
{
    // n 是需要的最大下标且 0<=n<P；返回至少覆盖 0..n 的共享模逆元表，其中位置 0 为 0。
    assert(0 <= n && (uint64_t)n < P); // 调试检查，可删
    static vector<MInt<P>> iv{0, 1};
    int old = iv.size();
    if (old <= n)
    {
        iv.resize(n + 1);
        for (int i = old; i <= n; i++)
        {
            iv[i] = MInt<P>(0) - MInt<P>(P / i) * iv[P % i];
        }
    }
    return iv;
}

constexpr uint32_t mod = 998244353;
using Z = MInt<mod>;
```

`MInt<P>` 的单次加减乘为 $O(1)$，模幂和单个逆元为 $O(\log P)$。`invTable<P>(n)` 要求 $0 \le n < P$，首次扩到 n 为 $O(n)$，之后读取为 $O(1)$。下面的固定验证同时覆盖常用模数 998244353 与 1000000007，检查负数构造、跨模加减、乘法、除法、逆元和共享逆元表；复制实现后可直接运行。

```cpp
template <uint32_t P> void checkMInt(uint32_t divAns)
{
    // divAns 是 (P-2)/5 的已知最小非负余数；核对模 P 的基础四则运算与逆元。
    using M = MInt<P>;
    assert(M(-2).val() == P - 2);
    assert((M(P - 2) + M(5)).val() == 3);
    assert((M(2) - M(5)).val() == P - 3);
    assert((M(P - 1) * M(P - 1)).val() == 1);
    assert((M(P - 2) / M(5)).val() == divAns);
    assert((M(5) * M(5).inv()).val() == 1);
    assert(invTable<P>(5)[5] == M(5).inv());
}

signed main()
{
    checkMInt<998244353>(798595482);
    checkMInt<1000000007>(200000001);
}
```

1.2 输入输出

1.2.001 快速输入输出

普通题关闭同步的 `cin/cout` 已足够；输入达数百万整数且 I/O 成瓶颈时才用 `FastInput`。核心是一次 `fread` 灌满 64 KiB 缓冲、逐字节手动解析，省掉 `cin` 的格式化开销。读整数时先跳空白、再处理可选负号、逐位累加；EOF 或格式错误返回 `false`。为缩短热路径，整数越界不再逐位检查，使用时必须保证题面输入能由目标类型表示；无符号目标不能读负数。构造时可传入 `FILE *` 作为读取流，比赛中保持默认的 `stdin`，本地测试可传临时流而不依赖外部标准输入。

```cpp
struct FastInput
{
    static constexpr int S = 1 << 16; // 缓冲区容量（字节）
    array<char, S> buf{};             // fread 批量读入的原始字节
    FILE *src = stdin;                // 当前读取的 C 输入流
    int p = 0;                        // 下一个未读位置
    int n = 0;                        // 当前有效字节数

    // 参数 src 为读取来源，默认从标准输入读取。
    FastInput(FILE *src = stdin) : src(src) {}

    char get()
    {
        // 返回下一个输入字节，EOF 返回 0
        if (p == n)
        {
            n = fread(buf.data(), 1, S, src);
            p = 0;
            if (!n)
            {
```

```
                return 0;
            }
        }
        return buf[p++];
    }

    template <class T> bool read(T &x)
    {
        // 题面保证整数能由 T 表示；EOF 或格式错误返回 false
        static_assert(is_integral_v<T> && !is_same_v<remove_cv_t<T>, bool>);
        using U = make_unsigned_t<T>;
        auto c = get();
        while (c && c <= ' ')
        {
            c = get();
        }
        if (!c)
        {
            return false;
        }
        bool neg = c == '-';
        if (neg)
        {
            c = get();
        }
        if (c < '0' || c > '9')
        {
            return false;
        }
        U v = 0;
        while ('0' <= c && c <= '9')
        {
            v = v * 10 + (U)(c - '0');
            c = get();
        }
        if constexpr (is_unsigned_v<T>)
        {
            if (neg)
            {
                return false;
            }
            x = (T)v;
        }
        else if (neg)
        {
            x = (T)(U(0) - v);
        }
        else
        {
            x = (T)v;
        }
        return true;
```

```
    }

    bool read(string &s)
    {
        // 读一个非空白字符串（以空白或 EOF 结束）
        auto c = get();
        while (c && c <= ' ')
        {
            c = get();
        }
        if (!c)
        {
            return false;
        }
        s.clear();
        while (c > ' ')
        {
            s.push_back(c);
            c = get();
        }
        return true;
    }
};
```

`FastInput in; in.read(n);` 直接取代 `cin >> n`。整数目标可以是有符号或无符号整型；负号不能写入无符号目标，调用者须保证数值在 `T` 范围内。同一程序内不要混用 `FastInput`、`cin` 与 `scanf`，它们各自维护缓冲。读长度 L 的输入为 $O(L)$，缓冲固定为 $O(2^{16})$。

1.2.002 `__int128` 输入输出

`__int128` 没有流重载，读写需要经字符串中转。`toString` 用无符号幅值逐位取模，避开 -2^{127} 取负溢出；`parseInt128` 严格检查符号、数字和范围，失败时返回空值。计算 `a*b%mod` 时写 `(__int128)a*b%mod`，让乘法在 128 位中完成。

```
string toString(__int128 x)
{
    // 128 位整数转十进制字符串
    if (!x)
    {
        return "0";
    }
    auto neg = x < 0;
    auto y = neg ? 0 - (unsigned __int128)x : (unsigned __int128)x;
    string s;
    while (y)
    {
        s.push_back(char('0' + y % 10));
        y /= 10;
    }
    if (neg)
```

```
    {
        s.push_back('-');
    }
    reverse(s.begin(), s.end());
    return s;
}

optional<__int128> parseInt128(const string &s)
{
    // 严格解析十进制串，非法或越界时返回空
    if (s.empty())
    {
        return nullopt;
    }
    bool neg = s[0] == '-';
    int p = neg || s[0] == '+';
    if (p == (int)s.size())
    {
        return nullopt;
    }
    unsigned __int128 lim = (unsigned __int128)1 << 127;
    if (!neg)
    {
        lim--;
    }
    unsigned __int128 x = 0;
    for (; p < (int)s.size(); p++)
    {
        if (s[p] < '0' || s[p] > '9')
        {
            return nullopt;
        }
        unsigned d = (unsigned)(s[p] - '0');
        if (x > (lim - d) / 10)
        {
            return nullopt;
        }
        x = x * 10 + d;
    }
    if (!neg)
    {
        return (__int128)x;
    }
    if (x == ((unsigned __int128)1 << 127))
    {
        return -(__int128)(x - 1) - 1;
    }
    return -(__int128)x;
}
```

解析与输出均为 $O(d)$，其中 d 是十进制位数。下面用最小值往返、带正号输入和一次乘法快速检查主路径，并保留一条非法字符检查：

```
signed main()
{
    string s = "-170141183460469231731687303715884105728";
    auto x = parseInt128(s);
    assert(x && toString(*x) == s);
    assert(parseInt128("+42") == optional<__int128>(42));
    assert(!parseInt128("12x"));
    auto a = parseInt128("123456789");
    auto b = parseInt128("987654321");
    assert(a && b && toString(*a * *b) == "121932631112635269");
}
```

1.2.003 泛型读写封装

容器长度已知时，1.1.004 常用函数重载与容器输出 已提供 `vector` 流重载。本条只展示调用，不重复定义同一组函数；复制模板时保留 1.1.004 的唯一实现即可。

```
vector<int> a(n);
cin >> a;
cout << a << '\n';
```

必须先 `vector<int> a(n)` 定好长度再 `cin >> a`——重载按既有 `size()` 读取，不会自动扩容。长度 n 时 $O(n)$ 时间、$O(1)$ 额外空间。

1.2.004 浮点数与输出精度控制

`fixed` 下 `setprecision(k)` 是小数位数，`defaultfloat` 下是有效数字，二者含义不同，误用会少输出几位导致精度不足。判题给绝对/相对误差时，多输出 2 到 4 位保险。

```
cout << fixed << setprecision(12) << x << '\n';
cout << scientific << setprecision(6) << x << '\n';
cout << defaultfloat;   // 恢复默认模式
```

单次格式化与位数线性相关，竞赛中视为 $O(1)$。

1.2.005 未知长度的一行输入

一行元素个数未知但行边界有语义时，不能用 `while (cin >> x)`——它跨行读取会吞掉下一行数据。正确做法是整行 `getline` 后用 `stringstream` 切分。

```
string line;
getline(cin >> ws, line);
stringstream ss(line);
vector<int> a;
for (int x; ss >> x;)
{
    a.push_back(x);
```

```
}
```

`cin >> ws` 跳过前导空白（含上一行残留换行）。若空行本身有效，则不能吃空白，应先单独消费前一个换行再 `getline(cin, line)`。行长 L 时 $O(L)$。

1.2.006 文件输入输出

本地固定样例、传统文件题与对拍时重定向标准流。用 `#ifndef ONLINE_JUDGE` 包裹，保证提交到评测机时不生效——否则普通标准输入题会因重定向读不到数据。

```
#ifndef ONLINE_JUDGE
freopen("input.txt", "r", stdin);
freopen("output.txt", "w", stdout);
#endif
```

若题面要求固定文件名（如 `data.in`），按题面改。只改变流来源，算法复杂度不变。

1.2.007 手动扩大栈空间

仅在递归写法已经清楚、且确认栈溢出时使用。C++ 的 DFS、分治和回溯仍直接写普通递归函数或递归 lambda；256 MiB 栈不等于可安全递归任意层——栈帧里的大对象、局部容器仍会吃掉配额。

Linux 运行前抬高软限制：

```
ulimit -s 262144
./main
```

Windows MinGW 在链接期设定栈大小：

```
g++ main.cpp -Wl,--stack,268435456 -o main.exe
```

不改变时间复杂度，只提高进程栈上限。

1.2.008 无原生 `__int128` 时的双字 `i128`

仅当编译器没有原生 `__int128` 时才使用本实现；它是兼容性兜底方案，加减乘除和十进制转换均明显慢于原生 `__int128`。用两个 `unsigned long long` 保存高低机器字比搬入任意精度整数更短。位型采用二进制补码：加减显式传递进位与借位，乘法把 64 位数拆成两个 32 位块求高半积，除法按位恢复商和余数。机器字必须是无符号型；若改成两个有符号整数，中间溢出会产生未定义行为。

```
struct i128
{
    unsigned long long hi = 0; // 补码的高 64 位
    unsigned long long lo = 0; // 补码的低 64 位

    i128(int x = 0) : hi(x < 0 ? ~0ULL : 0), lo((unsigned long long)x)
    {
```

```
        // 把 64 位有符号整数按补码符号扩展到 128 位
    }

    explicit operator bool() const
    {
        // 返回当前 128 位整数是否非零
        return hi || lo;
    }

private:
    i128(unsigned long long h, unsigned long long l) : hi(h), lo(l)
    {
        // h、l 是已经计算好的高、低 64 位补码
    }

    static bool lessU(i128 a, i128 b)
    {
        // 把 a、b 当作无符号 128 位整数，返回 a 是否小于 b
        return a.hi != b.hi ? a.hi < b.hi : a.lo < b.lo;
    }

    static unsigned long long mulHi(unsigned long long a, unsigned long long b)
    {
        // 返回 64 位无符号乘积的高 64 位
        constexpr unsigned long long MASK = 0xffffffffULL;
        unsigned long long a0 = a & MASK;
        unsigned long long a1 = a >> 32;
        unsigned long long b0 = b & MASK;
        unsigned long long b1 = b >> 32;
        unsigned long long p = a0 * b0;
        unsigned long long x = a0 * b1;
        unsigned long long y = a1 * b0;
        unsigned long long mid = (p >> 32) + (x & MASK) + (y & MASK);
        return a1 * b1 + (x >> 32) + (y >> 32) + (mid >> 32);
    }

public:
    friend i128 operator-(i128 x)
    {
        // 返回 x 的 128 位补码相反数
        auto lo = ~x.lo + 1;
        return i128(~x.hi + (lo == 0), lo);
    }

    friend i128 operator+(i128 a, i128 b)
    {
        // 返回 a+b 的低 128 位，低位溢出时向高位进位
        auto lo = a.lo + b.lo;
        return i128(a.hi + b.hi + (lo < a.lo), lo);
    }

    friend i128 operator-(i128 a, i128 b)
```

```
60      {
61          // 返回 a-b 的低 128 位，低位借位计入高位
62          auto lo = a.lo - b.lo;
63          return i128(a.hi - b.hi - (a.lo < b.lo), lo);
64      }
65
66      friend i128 operator*(i128 a, i128 b)
67      {
68          // 返回 a*b 的低 128 位
69          return i128(mulHi(a.lo, b.lo) + a.hi * b.lo + a.lo * b.hi,
70                      a.lo * b.lo);
71      }
72
73  private:
74      static pair<i128, i128> divU(i128 n, i128 d)
75      {
76          // n、d 按无符号解释且 d 非零；返回商与余数
77          i128 q;
78          i128 r;
79          for (int i = 127; i >= 0; i--)
80          {
81              auto carry = r.hi >> 63;
82              auto bit = i < 64 ? n.lo >> i & 1ULL : n.hi >> (i - 64) & 1ULL;
83              r = i128(r.hi << 1 | r.lo >> 63, r.lo << 1 | bit);
84              if (carry || !lessU(r, d))
85              {
86                  r = r - d;
87                  if (i < 64)
88                  {
89                      q.lo |= 1ULL << i;
90                  }
91                  else
92                  {
93                      q.hi |= 1ULL << (i - 64);
94                  }
95              }
96          }
97          return {q, r};
98      }
99
100     static pair<i128, i128> divS(i128 a, i128 b)
101     {
102         // b 非零且商可表示；返回向零取整的商和与 a 同号的余数
103         auto na = a.hi >> 63;
104         auto nb = b.hi >> 63;
105         auto [q, r] = divU(na ? -a : a, nb ? -b : b);
106         if (na != nb)
107         {
108             q = -q;
109         }
110         if (na)
111         {
```

```
            r = -r;
        }
        return {q, r};
    }

public:
    friend i128 operator/(i128 a, i128 b)
    {
        // 返回 a/b 向零取整的商
        return divS(a, b).first;
    }

    friend i128 operator%(i128 a, i128 b)
    {
        // 返回 a%b，非零余数与 a 同号
        return divS(a, b).second;
    }

    string toString() const
    {
        // 返回当前值的十进制表示
        auto neg = hi >> 63;
        i128 x = neg ? -*this : *this;
        string s;
        do
        {
            auto [q, r] = divU(x, i128(10));
            s.push_back(char('0' + r.lo));
            x = q;
        } while (x);
        if (neg)
        {
            s.push_back('-');
        }
        reverse(s.begin(), s.end());
        return s;
    }

    friend ostream &operator<<(ostream &out, const i128 &x)
    {
        // 把 x 的十进制表示写入 out，并返回 out
        return out << x.toString();
    }
};
```

用 `i128 x = (long long)v` 从 64 位整数构造，`cout << x` 直接输出。加、减、乘保留补码乘积的低 128 位；若按有符号整数使用，应保证数学结果落在 $[-2^{127}, 2^{127}-1]$。除数必须非零，且不能计算结果无法表示的 $\frac{-2^{127}}{-1}$；商向零取整，余数与被除数同号。加减乘为 $O(1)$，除法固定做 128 轮，转十进制为 $O(128d)$（$d \le 39$），额外空间均为 $O(1)$。编译器支持原生类型时优先用 1.2.002 `__int128` 输入输出；范围超过 128 位时改用 14.1.001 高精度整数（BigInt）。

下面用一组负数运算和一个跨机器字的数检查五则运算、符号规则与除法：

```cpp
signed main()
{
    i128 x(-123456789), y(12345);
    assert((x + y).toString() == "-123444444");
    assert((x - y).toString() == "-123469134");
    assert((x * y).toString() == "-1524074060205");
    assert((x / y).toString() == "-10000");
    assert((x % y).toString() == "-6789");

    i128 cross;
    cross.hi = 1;
    cross.lo = 1;
    assert((cross / i128((1LL << 32) + 1)).toString() == "4294967295");
    assert((cross % i128((1LL << 32) + 1)).toString() == "2");
}
```

1.3 编译、调试与测试

1.3.001 编译参数与调试构建

CLion 中为同一个 CMake 项目建立 `Release` 和 `Debug` 两个 Profile。Release 用于本地性能和提交前测试，Debug 用于定位越界、未定义行为和 STL 迭代器错误。下面的缓存变量应填入 CLion 的 CMake options；命令行构建时另加对应的 `-DCMAKE_BUILD_TYPE`。

最小 `CMakeLists.txt`：

```cmake
cmake_minimum_required(VERSION 3.20)
project(acm)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
add_executable(main main.cpp)
```

Release Profile 的 CMake options：

```
-DCMAKE_CXX_FLAGS_RELEASE="-g -O2" -DCMAKE_EXE_LINKER_FLAGS_RELEASE="-static"
```

Debug Profile 的 CMake options：

```
-DCMAKE_CXX_FLAGS_DEBUG="-O0 -g3 -gdwarf-4 -Wall -Wextra -Wshadow -Wconversion
↪ -Wsign-conversion -Wformat=2 -Warray-bounds=2 -Wstringop-overflow=4 -Wnull-dereference
↪ -Wduplicated-cond -Wlogical-op -Wvla -Wstack-usage=1048576 -Werror=return-type
↪ -fsanitize=undefined,bounds-strict -fsanitize-undefined-trap-on-error
↪ -fno-omit-frame-pointer -fno-optimize-sibling-calls -fstack-protector-all -DLOCAL
↪ -D_GLIBCXX_DEBUG -D_GLIBCXX_DEBUG_PEDANTIC -D_GLIBCXX_ASSERTIONS"
↪ -DCMAKE_EXE_LINKER_FLAGS_DEBUG="-static"
```

参数作用如下：`-DCMAKE_CXX_FLAGS_RELEASE` 和 `-DCMAKE_CXX_FLAGS_DEBUG` 分别设置两种构建的 C++ 编译参数；`-DCMAKE_EXE_LINKER_FLAGS_RELEASE` 和 `-DCMAKE_EXE_LINKER_FLAGS_DEBUG` 分别设置可执行文

件的链接参数。`-g` 保留 Release 调试信息，`-O2` 开启常用安全优化；`-O0` 关闭优化便于单步，`-g3` 生成包含宏的完整调试信息，`-gdwarf-4` 使用 DWARF v4。

`-Wall` 开启常见警告，`-Wextra` 开启额外警告，`-Wshadow` 检查变量遮蔽，`-Wconversion` 检查可能改变数值的隐式转换，`-Wsign-conversion` 检查有符号和无符号转换，`-Wformat=2` 加强 `printf` / `scanf` 格式检查，`-Warray-bounds=2` 加强数组越界分析，`-Wstringop-overflow=4` 加强字符串和内存写入越界分析，`-Wnull-dereference` 检查可证明的空指针解引用，`-Wduplicated-cond` 检查重复条件，`-Wlogical-op` 检查可疑逻辑运算，`-Wvla` 警告运行时长度数组，`-Wstack-usage=1048576` 在函数栈使用量可能超过 1 MiB 时警告，`-Werror=return-type` 把缺失返回值等返回类型错误视为编译错误。

`-fsanitize=undefined` 检查常见未定义行为，`-fsanitize=bounds-strict` 加强边界检查，`-fsanitize-undefined-trap-on-error` 在发现 UBSan 错误后直接触发 trap；需要可读的诊断文本时可以暂时删掉最后一项。`-fno-omit-frame-pointer` 保留帧指针，`-fno-optimize-sibling-calls` 禁止尾调用优化，二者便于回溯；`-fstack-protector-all` 为函数增加栈破坏保护。`-DLOCAL` 开启本地代码，`-D_GLIBCXX_DEBUG` 开启 libstdc++ 调试迭代器，`-D_GLIBCXX_DEBUG_PEDANTIC` 开启更严格的 STL 检查，`-D_GLIBCXX_ASSERTIONS` 开启标准库断言。`-static` 要求静态链接可执行文件，缺少对应静态库时会链接失败；`_GLIBCXX_DEBUG` 改变 STL ABI，Debug 和 Release 对象不能混合链接。

临时不用 CLion 时，可在 PowerShell 用下面的精简命令；完整告警、STL 检查和静态链接仍以上面的两个 Profile 为准。

```
g++ -std=c++20 -O2 -Wall -Wextra -Wshadow main.cpp -o main.exe
g++ -std=c++20 -O0 -g3 -gdwarf-4 -Wall -Wextra -Wshadow '-fsanitize=undefined,bounds-strict'
↪ -fsanitize-undefined-trap-on-error -fno-omit-frame-pointer -fno-optimize-sibling-calls
↪ -DLOCAL main.cpp -o main_debug.exe
```

下面是 GCC 优化选项清单。它是卡常查询表，不是默认提交前导；必须先用普通 `-O2` 版本和固定输入确认瓶颈，再逐项打开并对拍。`#pragma GCC optimize` 只影响其后的函数或翻译单元，`#pragma GCC target` 还会改变生成的指令集。

```
#if defined(__GNUC__)
#pragma GCC optimize("O3,unroll-loops") // O3 增强优化；展开循环，减少循环控制开销。
#if defined(__x86_64__) || defined(__i386__)
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt,fma") // 开启下文逐项说明的六组 x86 指令。
#pragma GCC target("avx", "sse2") // 再开启 AVX 与 SSE2；x86-64 上 SSE2 通常已默认支持。
#endif
#pragma GCC optimize("Ofast") // 在 O3 上放宽浮点语义，可能破坏严格 IEEE 结果。
#pragma GCC optimize("inline") // 鼓励内联函数，减少调用开销。
#pragma GCC optimize("unroll-loops") // 展开循环，可能增大代码体积。
#pragma GCC optimize("-fgcse") // 消除全局公共子表达式。
#pragma GCC optimize("-fgcse-lm") // 加强循环中的加载移动和公共子表达式消除。
#pragma GCC optimize("-fipa-sra") // 跨函数拆分聚合参数，减少无用字段传递。
#pragma GCC optimize("-ftree-pre") // 对树形中间表示做部分冗余消除。
#pragma GCC optimize("-ftree-vrp") // 传播变量取值范围，删除不可能分支。
#pragma GCC optimize("-fpeephole2") // 执行目标相关的局部指令改写。
#pragma GCC optimize("-ffast-math") // 允许不严格 IEEE 规则的浮点优化。
#pragma GCC optimize("-fsched-spec") // 允许更激进地提前调度指令。
#pragma GCC optimize("-falign-jumps") // 对齐跳转目标，可能改善取指。
#pragma GCC optimize("-falign-loops") // 对齐循环入口。
#pragma GCC optimize("-falign-labels") // 对齐普通代码标签。
```

```
#pragma GCC optimize("-fdevirtualize") // 将可确定目标的虚调用改为直接调用。
#pragma GCC optimize("-fcaller-saves") // 在调用附近保存寄存器，减少溢出到内存。
#pragma GCC optimize("-fcrossjumping") // 合并不同分支末尾的相同代码。
#pragma GCC optimize("-fthread-jumps") // 重排条件跳转，减少连续跳转。
#pragma GCC optimize("-funroll-loops") // 再次显式指定循环展开。
#pragma GCC optimize("-freorder-blocks") // 重排基本块，让常走路径顺序执行。
#pragma GCC optimize("-fschedule-insns") // 在寄存器分配前调度指令。
#pragma GCC optimize("inline-functions") // 鼓励普通函数内联。
#pragma GCC optimize("-ftree-tail-merge") // 合并不同分支的相同尾部。
#pragma GCC optimize("-fschedule-insns2") // 在寄存器分配后再次调度指令。
#pragma GCC optimize("-fstrict-aliasing") // 假设不同类型指针不别名；违反规则会出错。
#pragma GCC optimize("-falign-functions") // 对齐函数入口。
#pragma GCC optimize("-fcse-follow-jumps") // 沿条件跳转路径继续消除公共子表达式。
#pragma GCC optimize("-fsched-interblock") // 允许跨基本块调度指令。
#pragma GCC optimize("-fpartial-inlining") // 只内联函数中实际需要的部分。
#pragma GCC optimize("no-stack-protector") // 关闭栈保护，只适合确认安全的性能实验。
#pragma GCC optimize("-freorder-functions") // 重排函数布局，改善代码局部性。
#pragma GCC optimize("-findirect-inlining") // 间接目标可确定时尝试内联。
#pragma GCC optimize("-fhoist-adjacent-loads") // 提前条件两侧相邻的加载操作。
#pragma GCC optimize("-frerun-cse-after-loop") // 循环优化后再次消除公共子表达式。
#pragma GCC optimize("inline-small-functions") // 鼓励内联小函数。
#pragma GCC optimize("-finline-small-functions") // 与上一项同类，鼓励内联小函数。
#pragma GCC optimize("-ftree-switch-conversion") // 将部分 switch 转为查表或位判断。
#pragma GCC optimize("-foptimize-sibling-calls") // 做尾调用优化，减少栈使用。
#pragma GCC optimize("-fexpensive-optimizations") // 开启耗时但可能有效的优化。
#pragma GCC optimize("inline-functions-called-once") // 内联只调用一次的函数。
#pragma GCC optimize("-fdelete-null-pointer-checks") // 按未定义行为假设删除无效空指针检查。
// 当前 GCC 不接受下面四个名称作为有效 optimize pragma，仅保留作历史索引：
// -fwhole-program：假设整个程序一次性编译。
// -fstrict-overflow：基于没有有符号溢出的假设优化。
// -fcse-skip-blocks：旧版 CSE 跳块选项。
// -funsafe-loop-optimizations：基于循环满足额外约束优化。
#endif
```

目标参数中，`avx2` 把多数整数 SIMD 扩展到 256 位，`bmi` 提供 `andn`、`blsi` 等基础位操作，`bmi2` 提供 `pext`、`pdep` 等扩展位操作，`lzcnt` 提供前导零计数，`popcnt` 提供置位计数，`fma` 提供单次舍入的融合乘加，`avx` 提供 256 位浮点 SIMD，`sse2` 提供 128 位整数与双精度浮点 SIMD。这些指令只能在评测机明确支持时开启，否则程序可能直接收到非法指令；本机可用 `-march=native` 让 GCC 按当前 CPU 选择，但不能据此假定评测机也支持。

选项边界必须记住：`-O3` 通常已经包含大多数普通 `-f*` 优化，重复列出不等于再次加速；`-Ofast` 与 `-ffast-math` 允许破坏严格 IEEE 浮点语义；`-fstrict-aliasing` 依赖程序遵守别名规则；`no-stack-protector` 会降低栈保护；展开、对齐、内联和调度可能增大代码并使指令缓存变差。`#pragma once` 只是头文件保护，与性能无关。

这些参数只影响常数，不改变渐进复杂度。`-fsanitize` 用于调试，不能与面向提交的极限优化混用；Linux 提栈用 1.2.007 的 `ulimit`，MinGW 才使用 `-Wl,--stack,` 字节数。卡常前先用 1.5.001 卡常使用边界与专题索引 的基准步骤确认瓶颈。

1.3.002 Makefile（Windows / Linux）

固定编译参数，减少比赛中漏写标准版本或调试开关。配方行开头必须是真实 Tab（不是空格），否则 `make` 报错。

```
CXX := g++
CXXFLAGS := -std=c++20 -O2 -Wall -Wextra -Wshadow

main: main.cpp
	$(CXX) $(CXXFLAGS) $< -o $@

debug: main.cpp
	$(CXX) -std=c++20 -O1 -g -fsanitize=address,undefined $< -o main_debug

clean:
	rm -f main main.exe main_debug main_debug.exe
```

Linux/WSL 用 `make`，Windows 用 `mingw32-make`。

1.3.003 bashrc / vimrc

只保留不改变提交语义的快捷命令与编辑器设置。比赛环境不确定时不要依赖只存在于个人 shell 的别名。

```
alias c20='g++ -std=c++20 -O2 -Wall -Wextra -Wshadow'
alias asan='g++ -std=c++20 -O1 -g -fsanitize=address,undefined'
ulimit -c unlimited
```

```
set number
set expandtab
set shiftwidth=4
set tabstop=4
set autoindent
```

若比赛环境允许改自己的 shell，可把编译、解压样例、批量跑样例封成函数（`ulimit -s` 提栈上限，避免深递归段错误）：

```
ulimit -s 262144

build() {
    # $1: 源文件名，不带 .cpp
    if [ "$#" -ne 1 ]; then
        echo "usage: build <source>" >&2
        return 2
    fi
    if ! g++ -g -gdwarf-4 -O0 -Wall -Wextra -Wshadow -std=gnu++23 \
        -fsanitize=address,undefined -fno-omit-frame-pointer \
        -fno-optimize-sibling-calls "$1.cpp" -o "$1"; then
        echo "compile failed" >&2
        return 1
    fi
```

```
    echo "compile finished"
}

testcase() {
    # $1: 样例压缩包前缀
    if [ "$#" -ne 1 ]; then
        echo "usage: testcase <prefix>" >&2
        return 2
    fi
    unzip "samples-$1.zip" -d "./samples-$1/"
}

run() {
    # $1: 源文件名, $2: 样例前缀, $3: 起始编号, $4: 结束编号
    if [ "$#" -ne 4 ]; then
        echo "usage: run <source> <prefix> <first> <last>" >&2
        return 2
    fi
    build "$1" || return 1
    for ((i=$3;i<=$4;i++)); do
        echo "===== input: ./samples-$2/$i.in ====="
        if ! "./$1" < "./samples-$2/$i.in"; then
            echo "runtime error on sample $i" >&2
            return 1
        fi
    done
}
```

这些函数只包装本地命令，不改变被测程序复杂度。运行当前目录中的程序时保留 `./main`，不要把可写的当前目录加入 `PATH`。

1.3.004 对拍器（Windows / Linux）

对拍 = 生成器反复造随机数据，喂给待测程序和一个信得过的参考程序，比对结果，首个反例立即停并保留那组输入。生成器种子取自命令行参数（`gen 1`、`gen 2`……），同一种子永远产生同一组数据，反例可复现。按判定方式分两类：逐字比对（答案唯一）与 special judge（答案多解，用 checker 程序判）。

固定命名：`gen` 生成数据，`my` 是待测程序，`brute` 是可信的暴力或参考程序。生成器从 `argv[1]` 读取种子。每轮只输出 `AC i` 或 `WA i`，不打印程序输出、比较器说明或其他进度；首个 `WA i` 后停止，反例保存在 `in.txt`。

常规题用 `fc /b` 或 `diff -q` 严格比较；按空白分隔 token 或允许浮点误差时，改用题目对应的 checker。`dp.bat`：

```
@echo off
for %%x in (gen my brute) do (g++ -std=c++20 -O2 %%x.cpp -o %%x.exe || exit /b)
set i=0
:loop
set /a i+=1
gen.exe %i% > in.txt 2> nul
```

```
if errorlevel 1 goto wa
my.exe < in.txt > my.out 2> nul
if errorlevel 1 goto wa
brute.exe < in.txt > bf.out 2> nul
if errorlevel 1 goto wa
fc /b my.out bf.out > nul
if errorlevel 1 goto wa
echo AC %i%
goto loop
:wa
echo WA %i%
```

`dp.sh`:

```
#!/bin/bash
for x in gen my brute; do g++ -std=c++20 -O2 "$x.cpp" -o "$x" || exit; done
for ((i=1;;i++)); do
    if ! ./gen "$i" > in.txt 2>/dev/null ||
       ! ./my < in.txt > my.out 2>/dev/null ||
       ! ./brute < in.txt > bf.out 2>/dev/null; then
        echo "WA $i"
        break
    elif diff -q my.out bf.out > /dev/null 2>&1; then
        echo "AC $i"
    else
        echo "WA $i"
        break
    fi
done
```

只有浮点答案且答案唯一时，不需要编写题目专用 SPJ。把下面的 `f_cmp.py` 与 `dp.bat` 或 `dp.sh` 放在同一目录后直接调用。它接收待测输出和正确输出两个文件，要求两边浮点 token 数量相同，逐项使用题面给出的 $|x-y| \le 10^{-6}\max(1,y)$；它自己不输出内容，只用退出码表示是否全部通过：

```
import sys

x = [float(v) for v in open(sys.argv[1]).read().split()]
y = [float(v) for v in open(sys.argv[2]).read().split()]
ok = len(x) == len(y) and all(
    abs(a - b) <= 1e-6 * max(1.0, b)
    for a, b in zip(x, y)
)
raise SystemExit(not ok)
```

把逐字版脚本中的 `fc /b` 判定换成以下 Windows 片段：

```
python f_cmp.py my.out bf.out > nul 2> nul
if errorlevel 1 goto wa
echo AC %i%
goto loop
:wa
```

```
echo WA %i%
```

Linux / WSL 中把 `diff` 判定换成：

```
if python3 f_cmp.py my.out bf.out > /dev/null 2>&1; then
    echo "AC $i"
else
    echo "WA $i"
    break
fi
```

SPJ 题有多组合法输出，checker 直接验证待测输出，退出码 `0` 表示 AC。下面示例检查输出正整数 x, y 且 $x + y = n$；比赛时只替换中间的题目约束：

```
#include <bits/stdc++.h>
using namespace std;
signed main(signed argc, char *argv[])
{
    if (argc != 3) return 1;
    ifstream in(argv[1]), out(argv[2]);
    int n, x, y;
    if (!(in >> n) || !(out >> x >> y)) return 1;
    out >> ws;
    return !(x > 0 && y > 0 && x + y == n && out.eof());
}
```

最优化题另把标程答案作为第三个文件传给 checker。`spj.bat`：

```
@echo off
for %%x in (gen my spj) do (g++ -std=c++20 -O2 %%x.cpp -o %%x.exe || exit /b)
set i=0
:loop
set /a i+=1
gen.exe %i% > in.txt 2> nul
if errorlevel 1 goto wa
my.exe < in.txt > my.out 2> nul
if errorlevel 1 goto wa
spj.exe in.txt my.out > nul 2> nul
if errorlevel 1 goto wa
echo AC %i%
goto loop
:wa
echo WA %i%
```

`spj.sh`：

```
#!/bin/bash
for x in gen my spj; do g++ -std=c++20 -O2 "$x.cpp" -o "$x" || exit; done
for ((i=1;;i++)); do
    if ! ./gen "$i" > in.txt 2>/dev/null ||
       ! ./my < in.txt > my.out 2>/dev/null; then
```

```
        echo "WA $i"
        break
    elif ./spj in.txt my.out > /dev/null 2>&1; then
        echo "AC $i"
    else
        echo "WA $i"
        break
    fi
done
```

把脚本与源码放在同一目录，直接运行 `dp.bat`、`spj.bat`、`bash dp.sh` 或 `bash spj.sh`。每轮恰好输出一行 `AC i` 或 `WA i`；出现 `WA i` 时用留下的 `in.txt` 调试。对拍器是开发工具，允许用独立暴力程序校验待测程序；各算法条目的比赛内快速查验仍不得附第二套实现。

交互题由 Python 把解答与交互器的标准流相连。`interact.py` 的三个参数依次是解答程序、交互器和隐藏数据文件：

```
import subprocess, sys

sol = subprocess.Popen([sys.argv[1]], stdin=subprocess.PIPE,
                       stdout=subprocess.PIPE, stderr=subprocess.DEVNULL)
itr = subprocess.Popen([sys.argv[2], sys.argv[3]], stdin=sol.stdout,
                       stdout=sol.stdin, stderr=subprocess.DEVNULL)
sol.stdout.close()
sol.stdin.close()
a = itr.wait()
b = sol.wait()
raise SystemExit(a or b)
```

Windows 的 `itr.bat`：

```
@echo off
for %%x in (gen my interactor) do (g++ -std=c++20 -O2 %%x.cpp -o %%x.exe || exit /b)
set i=0
:loop
set /a i+=1
gen.exe %i% > in.txt 2> nul
if errorlevel 1 goto wa
python interact.py my.exe interactor.exe in.txt > nul 2> nul
if errorlevel 1 goto wa
echo AC %i%
goto loop
:wa
echo WA %i%
```

Linux / WSL 的 `itr.sh`：

```
#!/bin/bash
for x in gen my interactor; do g++ -std=c++20 -O2 "$x.cpp" -o "$x" || exit; done
for ((i=1;;i++)); do
    if ! ./gen "$i" > in.txt 2>/dev/null; then
```

```
        echo "WA $i"
        break
    elif python3 interact.py ./my ./interactor in.txt > /dev/null 2>&1; then
        echo "AC $i"
    else
        echo "WA $i"
        break
    fi
done
```

交互器从隐藏数据文件读取测试并在答错时返回非零；解答和交互器每次输出后都必须刷新。直接运行 `itr.bat` 或 `bash itr.sh`，每轮也只输出 `AC i` 或 `WA i`。单轮耗时为本轮所有进程之和，脚本不负责时限。

1.3.005 随机数据生成器

生成可复现随机数据，覆盖边界（最小、最大、重复、有序）与随机情形。种子取自 `argv`，同一种子永远产生同一组数据。绝不用 `rand()`——它的低位周期短、分布差。

```cpp
signed main(signed argc, char **argv)
{
    // 种子来自命令行第一个参数，缺省为 1
    auto seed = argc > 1 ? stoull(argv[1]) : 1ULL;
    mt19937_64 rng(seed);
    int n = uniform_int_distribution<int>(1, 50)(rng);
    cout << n << '\n';
    for (int i = 0; i < n; i++)
    {
        cout << uniform_int_distribution<int>(-100, 100)(rng)
             << " \n"[i + 1 == n];
    }
}
```

图数据生成后要额外检查连通、无重边等题目约束。生成 n 个元素为 $O(n)$。

1.3.006 GNU C++ 环境、本地计时与评测机测速

先输出标准版本、编译器版本、指针位数和优化状态；`__cplusplus >= 202002L` 才能依赖 C++20。实际程序耗时用 `steady_clock` 包住 `solve()`，只在 `LOCAL` 下输出到 `cerr`。

```cpp
cout << __cplusplus << '\n';
cout << __VERSION__ << '\n';
cout << sizeof(void *) * 8 << '\n';
#ifdef __OPTIMIZE__
cout << "optimized\n";
#endif

#ifdef LOCAL
auto beg = chrono::steady_clock::now();
```

```
#endif
solve();
#ifdef LOCAL
auto end = chrono::steady_clock::now();
cerr << chrono::duration<double, milli>(end - beg).count() << " ms\n";
#endif
```

热身赛比较本机与评测机速度时，使用固定的 `mt19937` 工作块。每块生成 10^7 个随机数并滚动异或；`speedCheck` 是该固定过程的真实结果。轮数和块数各做一次 `volatile` 读取，循环内部仍是普通局部变量；返回值必须进入最终输出或错误分支，因此 `-O2` 不能把整段计算当作无用代码删除。

```
volatile int speedRounds = 10000000; // 每块生成次数；修改后必须重算 speedCheck
volatile int speedBlocks = 1;        // 重复块数；测速时只调整这一项
constexpr long long speedCheck = 3894306128LL;

long long rot32(long long x, int k)
{
    // x 是低 32 位有效的无符号字；k 是 1..31 的左旋位数；返回低 32 位结果。
    return ((x << k) | (x >> (32 - k))) & 0xffffffffLL;
}

#if defined(__GNUC__)
__attribute__((noinline))
#endif
long long speedBlock()
{
    // 返回固定随机序列的滚动异或摘要
    int n = speedRounds;
    mt19937 rng(712367821);
    long long h = 0;
    for (int i = 0; i < n; i++)
    {
        h = rot32(h, 7) ^ (unsigned long long)rng();
        h &= 0xffffffffLL;
    }
    return h;
}

long long speedTest()
{
    // 重复 speedBlocks 块；全部摘要正确时返回 0
    int k = speedBlocks;
    long long bad = 0;
    for (int i = 0; i < k; i++)
    {
        bad |= speedBlock() ^ speedCheck;
    }
    return bad;
}
```

本地计时调用如下；`bad` 必须打印或判断，不能只测一个未使用的返回值。

```cpp
auto beg = chrono::steady_clock::now();
long long bad = speedTest();
auto end = chrono::steady_clock::now();
cerr << chrono::duration<double, milli>(end - beg).count()
     << " ms, check = " << bad << '\n';
```

评测机测速先令 `speedBlocks=0` 提交一次，记基线时间为 t_0；再令 `speedBlocks=k` 提交，记时间为 t_k。以下时间均换算为秒。一个块约耗时 $(t_k - t_0)/k$，该评测机每秒约执行

$$\frac{10^7 k}{t_k - t_0}$$

次本测试循环，一秒对应的块数约为 $k/(t_k - t_0)$。时间显示较粗时取 $k = 8$ 或 16，直到 $t_k > t_0$；只返回 AC/TLE 时，提交一个只运行 `speedTest()` 并输出检查值的程序，按 $1, 2, 4, 8, \ldots$ 增大块数，首次 TLE 后在最后两个值间二分。若时限是 L 秒、边界附近可通过 k 块，则一秒约能运行 k/L 块，即约 $10^7 k/L$ 次本测试循环；该值受时间取整和机器负载影响，只取近似。整数答案可写 `ans ^= speedTest()`，正确时异或零而不改变答案；字符串或多行输出在正式输出前写 `if (speedTest()) return 1;`。测速通常只调整 `speedBlocks`；修改 `speedRounds` 后必须本地输出一次 `speedBlock()`，用新摘要替换 `speedCheck`。测速只代表当前编译器下的 `mt19937` 和整数运算速度，不能替代图、内存、I/O 或具体算法的最坏数据测试。环境信息为 $O(1)$；测速为 $O(\mathit{speedRounds} \cdot \mathit{speedBlocks})$ 时间和 $O(1)$ 额外空间。

使用上述固定工作块实测时，各平台每秒完成的块数约为：

平台	每秒块数	每秒本测试循环次数
Codeforces	42	4.2×10^8
QOJ	51	5.1×10^8
洛谷	47	4.7×10^8
牛客	$30 \sim 31$	$3.0 \times 10^8 \sim 3.1 \times 10^8$
AtCoder	72	7.2×10^8
PTA	27	2.7×10^8

本次实测未记录具体评测机型号、编译器版本与重复轮数，以上数据只用于赛前粗略估算，不能视为平台的固定性能。

1.3.007 调试输出与断言

`debug(x)` 只在本地定义 `LOCAL` 时打印到 `cerr`，提交时展开为空语句，不影响 `cout` 结果。`assert` 内不要放有副作用的语句——`-DNDEBUG` 会整体删除表达式，副作用随之消失。

```cpp
#ifdef LOCAL
#define debug(x) cerr << #x << " = " << (x) << '\n'
#else
#define debug(x) ((void)0)
#endif

assert(0 <= l && l <= r && r <= n);
```

关闭时 $O(1)$，开启时取决于输出量。

1.3.008 常见未定义行为和易错点清单

提交前快速排查 “本地能跑、评测漂移” 的问题：

- 有符号整数溢出是未定义行为；只有乘积可能超出当前类型时，才对该表达式局部转成 `__int128`，普通加减优先用边界比较改写。
- 不对负数或超位宽做位移；`1ULL << k` 要求 $0 \le k < 64$。
- 不解引用 `end()`，不在容器扩容后沿用旧迭代器、指针或引用。
- `sort` 比较器必须严格弱序，不能写成 $a \le b$（自反会崩溃或死循环）。
- 不同时使用关闭同步的 iostream 与 stdio。
- 递归深度、数组下标、空容器 `back()`、除零、负数取模都要单独检查。

只在 `-O2` 下出错时，第一步跑 ASan/UBSan（1.3.001），不要先加 `volatile` 或关优化掩盖问题。清单本身无运行时复杂度。

1.4 STL、扩展库与防卡

1.4.001 `vector`、`array`、`string`、`tuple`、`pair`

`vector` 管运行时动态连续空间，`array` 管编译期定长，`string` 管字节串，`pair/tuple` 管小型聚合。已知最终长度用 `resize` 一次到位，只知上界用 `reserve` 预留避免反复搬迁。

```
vector<int> a(n);
array<int, 4> d{1, 0, -1, 0};     // 常用于四方向偏移
pair<int, int> e{x, y};
auto [u, v] = e;                  // 结构化绑定
```

`string` 下标按字节，不按 Unicode 字符。随机访问 $O(1)$，`push_back` 摊还 $O(1)$，中间插删 $O(n)$。

1.4.002 `stack`、`queue`、`deque`、`priority_queue`

分别对应后进先出、先进先出、双端、堆。`priority_queue` 默认大根堆，小根堆传 `greater<>`；它不支持删除任意元素，需要动态删除时用延迟删除双堆或带删堆（3.5.009）。

```
priority_queue<int> mx;                                 // 大根堆
priority_queue<int, vector<int>, greater<>> mn;         // 小根堆
deque<int> q;
```

需要遍历全部元素时直接用底层 `vector/deque`。栈、队列、双端两端操作 $O(1)$；堆插入、弹出 $O(\log n)$，取顶 $O(1)$。

1.4.003 `set`、`multiset`、`map`、`multimap`

需要有序键、前驱后继或稳定最坏复杂度时用红黑树容器。`multiset::erase(x)` 按值删除全部等于 `x` 的元素；只删一个时先用 `find` 精确查找，再删除该迭代器。

```
multiset<int> s;
auto it = s.find(x);
if (it ≠ s.end())
{
    s.erase(it);
}
```

不要用 `distance(s.begin(), it)` 求排名——它对树迭代器是 $O(n)$；要排名用 PBDS 有序树（1.4.006）。查找、插删均 $O(\log n)$。

1.4.004 `unordered_map` / `unordered_set` 防 hack 哈希

标准哈希对整数近乎恒等，出题人能构造大量同桶键把它卡成 $O(n)$ 每次。用 SplitMix64 充分扰动、再混入进程启动时间做随机种子，构造者无法预测桶分布。

```
struct CustomHash
{
    static unsigned long long mix(unsigned long long x)
    {
        // SplitMix64 finalizer: 雪崩扰动
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

    size_t operator()(unsigned long long x) const
    {
        // 混入随机种子，抵御针对固定哈希的构造
        static const unsigned long long seed =
            (unsigned long long)chrono::steady_clock::now().time_since_epoch().count();
        return mix(x + seed);
    }

    size_t operator()(const pair<int, int> &x) const
    {
        // 只接受两个整数分量的 pair
        static const unsigned long long seed =
            (unsigned long long)chrono::steady_clock::now().time_since_epoch().count();
        unsigned long long a = mix((unsigned long long)x.first + seed);
       unsigned long long b = mix((unsigned long long)x.second + seed + 0x9e3779b97f4a7c15ULL);
        return mix(a ^ rotl(b, 32));
    }
};
```

使用时把它作为哈希容器的第三个模板参数，并预留容量：

```
unordered_map<int, int, CustomHash> cnt;
cnt.reserve(n * 2);          // 预留减少重哈希
cnt.max_load_factor(.7);
```

单个整数键和由两个整数组成的 `pair` 可以直接使用该哈希；字符串或复合键应另写与键类型对应的哈希器。操作期望为 $O(1)$，最坏仍为 $O(n)$；要求稳定最坏复杂度时改用 `map`。

1.4.005 `bitset` 与手写 bitset

编译期定长首选 `bitset<N>`；长度运行时才定用 `DynBitset`——按 64 位机器字打包，位运算天然 $\frac{1}{64}$ 加速。下面这份提供置位、清空、查询、计数、按位与/或，够覆盖 “布尔可达性并行转移” 一类用途；需要移位再按题补充，避免做成难核对的完整容器。

```
struct DynBitset
{
    int n = 0;                      // 有效位数
    vector<unsigned long long> a; // 按 64 位机器字打包

    DynBitset(int m = 0)
    {
        // 构造 m 位的全零集合，要求 m>=0
        init(m);
    }

    void init(int m)
    {
        // 把集合重设为 m 位并清零
        assert(m >= 0); // 调试检查，可删
        n = m;
        a.assign((n + 63) / 64, 0);
    }

    void set(int p, bool v = true)
    {
        // 第 p 位写成 v
        assert(0 <= p && p < n); // 调试检查，可删
        auto b = 1ULL << (p & 63);
        if (v)
        {
            a[p >> 6] |= b;
        }
        else
        {
            a[p >> 6] &= ~b;
        }
    }

    void reset()
```

```
    {
        // 全部有效位清零
        fill(a.begin(), a.end(), 0);
    }

    bool test(int p) const
    {
        // 返回第 p 位
        assert(0 <= p && p < n); // 调试检查，可删
        return a[p >> 6] >> (p & 63) & 1ULL;
    }

    int count() const
    {
        // 1 的总数
        int ans = 0;
        for (auto x : a)
        {
            ans += popcount(x);
        }
        return ans;
    }

    DynBitset &operator|=(const DynBitset &o)
    {
        // 逐位或入 o，返回当前集合
        assert(n == o.n); // 调试检查，可删
        for (int i = 0; i < (int)a.size(); i++)
        {
            a[i] |= o.a[i];
        }
        return *this;
    }

    DynBitset &operator&=(const DynBitset &o)
    {
        // 逐位与入 o，返回当前集合
        assert(n == o.n); // 调试检查，可删
        for (int i = 0; i < (int)a.size(); i++)
        {
            a[i] &= o.a[i];
        }
        return *this;
    }
};
```

`init(m)` 要求 $m \geq 0$，单点接口要求 $0 \leq p < n$，两个集合做按位运算时位数必须相同。单点操作为 $O(1)$，计数与按位运算为 $O\left(\frac{n}{64}\right)$。以下固定例覆盖机器字边界 $63/64$、末位、清位与按位或：

```
signed main()
{
```

```
    DynBitset a(70);
    a.set(0);
    a.set(63);
    a.set(69);
    DynBitset b(70);
    b.set(64);
    a |= b;
    assert(a.count() == 4 && a.test(63) && a.test(64));
    a.set(69, false);
    assert(a.count() == 3 && !a.test(69));
    a.reset();
    assert(a.count() == 0);
}
```

1.4.006 GNU PBDS

PBDS 是 libstdc++ 的非标准扩展，只能在提供相应头文件的 GNU 环境中使用。常用接口包括支持排名的 `tree`、可改键与合并的 `priority_queue`，以及 `cc_hash_table`、`gp_hash_table`。哈希表的默认哈希仍然可预测；需要抵御构造碰撞时，应显式使用 1.4.004 的随机化哈希器。三个扩展头必须写在 `#define int long long` 之前（放在文件最开头），否则宏会改写库内部的 `int` 声明而编译失败。下面给出四类接口的最小调用方式：

```
// 依赖 1.4.004 unordered_map / unordered_set 防 hack 哈希
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/priority_queue.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace __gnu_pbds;

template <class T> using OrderedSet = tree<
    T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

using MinHeap = __gnu_pbds::priority_queue<
    int, greater<int>, pairing_heap_tag>;

using ResizePolicy = hash_standard_resize_policy<
    hash_exponential_size_policy<>,
    hash_load_check_resize_trigger<>,
    true>;

using SizedHashMap = gp_hash_table<
    unsigned long long, // 64 位无符号键，保留完整机器字范围
    int,
    CustomHash,
    equal_to<unsigned long long>,
    direct_mask_range_hashing<>,
    linear_probe_fn<>,
    ResizePolicy>;
```

```cpp

void pbdsExample(int x, int id, int k)
{
    // x、id 是插入有序树的值与编号，k 是 0 起排名；演示四类 PBDS 接口，输出 x 的排名与第 k 小的值。
    OrderedSet<pair<int, int>> s;
    s.insert({x, id});
    auto rank = s.order_of_key({x, numeric_limits<int>::min()});
    assert(0 <= k && k < (int)s.size()); // 调试检查，可删
    auto value = *s.find_by_order(k);

    MinHeap a;
    MinHeap b;
    auto it = a.push(5);
    a.modify(it, 2);
    a.erase(it);
    a.join(b);

    cc_hash_table<int, int, CustomHash> chain;
    gp_hash_table<int, int, CustomHash> probe;
    chain[x]++;
    probe[x]++;

    SizedHashMap mp1;
    mp1.resize(1ULL << 20); // 批量插入前把桶数调到 2 的幂
    mp1[(unsigned long long)x]++;
    cout << rank << ' ' << value.first << '\n';
}
```

有序树的插入、删除、排名和第 k 小为 $O(\log n)$；解引用 `find_by_order(k)` 前要求 $0 \le k < |s|$，比较器必须是严格弱序，不能用 `less_equal` 模拟多重集合。`pairing_heap_tag` 的 `modify` 按摊还 $O(\log n)$ 使用，`join` 为 $O(1)$；哈希表操作只有期望 $O(1)$。

`gp_hash_table` 的 `resize` 只有当扩容策略 `hash_standard_resize_policy` 的第三个模板参数 `External_Size_Access` 为 `true` 时才能从外部调用；默认策略取 `false`，直接调用会编译失败。它接收扩容策略的表容量目标，不等同于按元素数预留的 `unordered_map::reserve`，也不保证能容纳同样数量的元素而不再扩容。这里的 `direct_mask_range_hashing` 用掩码定位桶，容量应取 2 的幂；`resize` 会重散列已有元素，故应在批量插入前调用。`CustomHash` 仍负责抵御构造碰撞。具体策略与兼容性见 3.9 PBDS 专题。

1.4.007 `lower_bound`、`upper_bound`、`unique`、`iota`

二分有序区间、去重、生成连续编号。`unique` 只把重复项移到尾部并返回新逻辑末尾，必须配 `erase` 才真正删除。二分前必须满足对应有序/单调条件。

```cpp
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());               // 排序去重
auto l = lower_bound(a.begin(), a.end(), x) - a.begin();    // 首个 >= x
auto r = upper_bound(a.begin(), a.end(), x) - a.begin();    // 首个 > x
iota(id.begin(), id.end(), 0);                              // 0,1,2,...
```

随机访问迭代器二分 $O(\log n)$；`unique`、`iota` 为 $O(n)$。

1.4.008 next_permutation、prev_permutation、shuffle

按字典序枚举排列或随机打乱。从排序后的序列起循环 next_permutation，可无重复地枚举多重集合的全部排列。random_shuffle 已被移除，一律用 shuffle 配 mt19937。

```
sort(a.begin(), a.end());
do
{
    use(a);
} while (next_permutation(a.begin(), a.end()));

shuffle(a.begin(), a.end(), rng);
```

每次排列推进 $O(n)$，枚举 P 个排列共 $O(Pn)$；打乱 $O(n)$。

1.4.009 accumulate、is_sorted、迭代器

求和、验有序、写泛型区间代码。累加初值决定结果类型：#define int long long 改不了字面量 0 的 int 类型，大整数求和仍须显式写 0LL，否则中途溢出。

```
auto sum = accumulate(a.begin(), a.end(), 0LL);   // 初值 0LL 定死 long long
assert(is_sorted(a.begin(), a.end()));
for (auto it = a.begin(); it ≠ a.end(); it++)
{
    use(*it);
}
```

均为 $O(n)$ 时间、$O(1)$ 额外空间。

1.4.010 __builtin_* 与 C++20 位运算函数

统计位数、找最高位、二进制上取整。C++20 的 popcount/bit_width/bit_ceil/countr_zero 要求无符号实参，比 __builtin_* 更安全：countr_zero(0) 有定义（返回位宽），而 __builtin_ctzll(0) 未定义。

```
auto u = (unsigned long long)x;
auto ones = popcount(u);                                  // 1 的个数
int hi = u ? (int)bit_width(u) - 1 : -1;                 // 0 的最高位约定为 -1
auto p2 = bit_ceil((unsigned long long)max<int>(1, n));   // >= n 的最小 2 的幂
auto z = countr_zero(u);                                  // 末尾 0 的个数
```

这些操作通常编译为单条或少量指令，视为 $O(1)$。调用 bit_ceil 时还要保证结果能由实参的无符号类型表示。

1.4.011 sort killer 与哈希 hack

理解旧式快排和固定哈希会被构造数据卡退化，据此做防御（非鼓励攻击评测）：

1. 排序直接用标准库 sort——它是 introsort，最坏退化时自动转堆排序，保证 $O(n \log n)$ 次比较。

2. 哈希表用 `CustomHash`、`reserve` 与较低装载因子（1.4.004）。
3. 需要稳定最坏复杂度时用 `map/set` 或排序离线处理。

哈希仍只有期望 $O(1)$。

1.4.012 何时才需要卡常

只在复杂度正确且剖析已经定位瓶颈后降低常数。卡常不能替代算法改进，具体入口见 1.5.001 卡常使用边界与专题索引。常规提交先检查以下事项：

- 连续数据优先 `vector` 并提前 `reserve`，避开链式容器与频繁分配。
- 热循环避免 `endl`（每次 flush）、虚函数、重复取模、`std::function` 类型擦除。
- 用 `bitset`/机器字并行处理布尔转移。
- 快读只在 I/O 确占主要时间时启用。
- 编译器、评测机指令集和内存限制必须满足所用优化的条件。

只改常数，不改渐进复杂度。

1.4.013 字符串分割：getline / Split

按单字符分隔符切分整行，并保留连续分隔符产生的空字段——这是它与 `stringstream` 按空白切分的关键区别：后者会吞掉空字段。

```
vector<string> split(const string &s, char sep = ' ')
{
    // 按 sep 切分，保留空字段
    vector<string> ans;
    string cur;
    for (auto c : s)
    {
        if (c == sep)
        {
            ans.push_back(cur);
            cur.clear();
        }
        else
        {
            cur.push_back(c);
        }
    }
    ans.push_back(cur);
    return ans;
}
```

`split("a,,bc,", ',')` 得到 `{"a", "", "bc", ""}`。按任意空白切数字用 `stringstream`，按固定字符且空字段有意义时用 `split`。长度 n 时 $O(n)$。

1.4.014 optional

optional<T> 表示 “一个 T 或没有值”，适合表达查找失败、无解等状态，避免拿 -1、INF 等可能与合法答案冲突的哨兵。默认构造和 nullopt 都表示空；装入一个 T 后才可以解引用。

```
optional<int> firstPos(const vector<int> &a, int x)
{
    // 返回 x 在 a 中首次出现的下标，不存在时返回空
    for (int i = 0; i < (int)a.size(); i++)
    {
        if (a[i] == x)
        {
            return i;
        }
    }
    return nullopt;
}

void optExample(const vector<int> &a, int x)
{
    // 输出 x 的首次出现位置，并演示置值与置空
    auto pos = firstPos(a, x);
    if (pos)
    {
        cout << *pos << '\n';
    }
    else
    {
        cout << "not found\n";
    }

    optional<int> ans;
    ans = 42;                   // 现在含有值 42
    ans.reset();                // 重新置空
    int v = ans.value_or(-1);   // 空时取得默认值 -1
    cout << v << '\n';
}
```

用 if (x) 或 x.has_value() 判空，确认非空后用 *x、x→member 访问；value() 在空值上会抛出 bad_optional_access，竞赛代码通常先判空。value_or(defaultValue) 只提供一个替代值，不会改变原对象；optional 只区分成功与无值，需要区分多种失败原因时改用状态码或 variant。除所存对象自身的构造、析构成本外，判空、访问、reset 均为 $O(1)$；设 a 的长度为 n，示例 firstPos 的时间复杂度为 $O(n)$、额外空间为 $O(1)$。

1.5 平台优化与极限卡常

本节只保留比赛现场能直接使用的卡常入口。先确认复杂度正确，再用 1.3.006 GNU C++ 环境、本地计时与评测机测速 定位常数；依赖具体测试答案的硬编码、越界访问和有符号溢出不属于优化。

1.5.001 卡常使用边界与专题索引

普通提交先用 `-O2`、连续容器、预留容量和 1.2.001 快速输入输出。确实接近时限时再从 1.3.001 编译参数与调试构建 的 GCC 清单逐项试验，并对同一固定输入核对答案和时间。下表保留各类平台优化的有效结论；具体快慢依赖 CPU、编译器、数据分布和问题规模，不把一次本机结果写成普遍顺序。

技术	适用条件	复杂度与边界
AVX2 自动向量化或专用循环	x86 评测机明确支持 AVX2，数据连续且循环无依赖	扫描 n 项仍为 $O(n)$ 时间、$O(1)$ 额外空间；一次处理更多元素只改变常数，不同 CPU 上收益可能反转
PCLMUL / NEON PMULL	固定宽度二进制多项式乘法，且运行平台支持对应指令	固定 64 位硬件乘法视为 $O(1)$；标量回退需逐位处理，推广到 w 位时为 $O(w^2)$，均为 $O(1)$ 额外空间
延迟约减	已证明一个分块内全部乘加不会超过累计类型	长度 n 的点积为 $O(n)$ 时间、$O(1)$ 额外空间；块长必须由模数和累计类型上界推出，不能只靠经验值
radix-4 NTT、根表复用与频域驻留	固定友好模数、长度满足单位根条件，并有大量同规模卷积	10.1.003 NTT 已给 `FastNTT`、radix-2 回退、固定校验和与本机基准；两版均为 $O(N \log N)$ 时间、$O(N)$ 空间，基准差异只作当前环境参考
`mmap` / `fread` / 手写输出缓冲	输入确实占主要时间；`mmap` 还要求普通文件，管道输入必须回退	读写 B 字节均为 $O(B)$；固定缓冲额外空间为 $O(S)$，映射额外占用 $O(B)$ 虚拟地址空间，是否更快依赖系统与输入来源
arena / 空闲表	大量等生命周期小对象，析构可统一处理	预留 C 字节后单次顺序分配为 $O(1)$，清空为 $O(1)$，占用 $O(C)$；需要逐对象析构或未知峰值时不适合
CSR / SoA / 分块与预取	图或记录静态，遍历受缓存而非运算限制	CSR 建表 $O(n+m)$ 时间和空间，完整扫描 $O(m)$；预取和字段拆分不改变复杂度，收益必须按目标平台实测
位表、分支提示与经验阈值	状态域固定且表可由算法定义生成	固定 64 位 `popcount` 或查表视为 $O(1)$；阈值选择为 $O(1)$，被选择算法仍按自身复杂度计算，阈值须记录基准环境

固定模乘可选 7.1.009 Barrett 约减或 7.1.010 Montgomery 模乘。AVX2、PCLMUL、`mmap`、通用内存池、手写预取和经验阈值没有一份能直接适用于所有题目的短接口，因此这里只保留选择条件与严格界；需要时在当前题目代码旁写专用版本和可移植回退。依赖具体测试答案的硬编码、越界访问和有符号溢出不属

于优化。

2. 基础算法与离线算法

2.1 搜索与枚举

2.1.001 整数二分、实数二分

在单调谓词上定位分界点。整数版维护「答案仍在闭区间 $[l, r]$ 中」的不变量，每轮用 `midpoint` 安全折半，`firstTrue` 返回第一个为真的位置、`lastTrue` 返回最后一个为真的位置；`lastTrue` 里 `m == l` 时上取到 `r`，是为了偶数长度不陷入死循环。两者都不设 $l-1$ / $r+1$ 哨兵，调用者须保证答案确在给定闭区间内。实数版不比较浮点端点是否相等，改用固定迭代轮数把误差压到初始长度的 2^{-it}。二分只对单调判定成立，不能对任意函数盲套。

```
template <class F> int firstTrue(int l, int r, F f)
{
    // l、r 是答案所在闭区间，f 是由假变真的谓词；返回第一个为真的位置。
    while (l < r)
    {
        auto m = midpoint(l, r);
        if (f(m))
        {
            r = m;
        }
        else
        {
            l = m + 1;
        }
    }
    return l;
}

template <class F> int lastTrue(int l, int r, F f)
{
    // l、r 是答案所在闭区间，f 是由真变假的谓词；返回最后一个为真的位置。
    while (l < r)
    {
        auto m = midpoint(l, r);
        if (m == l)
        {
            m = r;
        }
        if (f(m))
        {
            l = m;
        }
        else
        {
            r = m - 1;
        }
    }
```

```
    return l;
}

template <class F> long double firstReal(long double l, long double r, F f, int it = 100)
{
    // l、r 是初始实数区间，f 是单调谓词，it 是迭代轮数；返回分界点近似值。
    for (int t = 0; t < it; t++)
    {
        auto m = midpoint(l, r);
        if (f(m))
        {
            r = m;
        }
        else
        {
            l = m;
        }
    }
    return midpoint(l, r);
}
```

若要在已有随机访问有序区间上反复做下界/上界查询，分支结果通常难以预测，可以使用无分支步长收缩版：

```
template <class It, class T, class F = less<>> It fastLower(It first, It last, const T &x, F f
↪ = {})
{
    // [first,last) 必须支持随机访问；返回第一个不小于 x 的位置。
    assert(first <= last);
    int n = last - first;
    for (n++; n > 1;)
    {
        int h = n / 2;
        n -= h;
        first += f(first[h - 1], x) * h;
    }
    return first;
}

template <class It, class T, class F = less<>> It fastUpper(It first, It last, const T &x, F f
↪ = {})
{
    // [first,last) 必须支持随机访问；返回第一个大于 x 的位置。
    assert(first <= last);
    int n = last - first;
    for (n++; n > 1;)
    {
        int h = n / 2;
        n -= h;
        first += !f(x, first[h - 1]) * h;
    }
```

```
    return first;
}
```

谓词方向决定用哪支：`firstTrue` 的谓词须由假变真，`lastTrue` 的谓词须由真变假；无解时先在调用前验证边界谓词，或把返回区间外扩一格作显式哨兵。三个谓词接口的最小调用如下：

```
signed main()
{
    int x = firstTrue(0, 10, [](int p)
    {
        return p * p >= 20;
    });
    int y = lastTrue(0, 10, [](int p)
    {
        return p * p <= 20;
    });
    long double z = firstReal(0, 2, [](long double p)
    {
        return p * p >= 2;
    });
    assert(x == 5 && y == 4 && fabsl(z * z - 2) < 1E-12L);
}
```

整数版调用谓词 $\lceil \log_2(r-l+1) \rceil$ 次、额外空间 $O(1)$；实数版固定调用 it 次。`fastLower/fastUpper` 与标准库语义相同，允许空区间，但只适合随机访问迭代器；它们仍是 $O(\log n)$，优势只在不可预测比较下减少分支。本机一百万次随机下界查询的 7 轮中位数约为 `66ms`，标准 `lower_bound` 约为 `112ms`；小数组或比较器很贵时差距可能消失，默认仍可用标准库版本。

2.1.002 整数三分、实数三分、黄金分割三分

求单峰函数的极值，模板求最大值；求最小值时应反转比较，只有确认值域取负不溢出时才传 `-f(x)`。比较两个三等分点后能证明峰值不在较差点外侧那一段，于是每轮丢掉约三分之一。整数版收窄到常数宽度后逐点枚举剩余的至多 5 个位置，专门兜住平台峰与整除取整；`ternaryReal` 每轮独立取两点，`goldenMax` 用黄金比 $\varphi - 1 \approx 0.618$ 布点，使收缩后有一个内点可复用、每轮只新增一次求值，因此函数求值昂贵时更省。三分要求真单峰，函数有多个相隔局部峰时没有正确性保证。

```
template <class F> int ternaryInt(int l, int r, F f)
{
    // l、r 是整数闭区间，f 是单峰函数；返回一个最大值位置。
    // 求最小值要把收缩循环和末尾扫描中的 f 比较同时反向；只改一处会得到错误区间。
    while (r - l > 4)
    {
        auto d = r - l;
        auto x = l + d / 3;
        auto y = r - d / 3;
        if (f(x) < f(y))
        {
            l = x + 1;
        }
```

```
        else
        {
            r = y - 1;
        }
    }
    auto p = l;
    auto x = l;
    while (x < r)
    {
        x++;
        if (f(p) < f(x))
        {
            p = x;
        }
    }
    return p;
}

template <class F> long double ternaryReal(long double l, long double r, F f, int it = 120)
{
    // l、r 是实数区间，f 是单峰函数，it 是迭代轮数；返回最大值位置近似值。
    // 求最小值时反向循环内的 f(x)、f(y) 比较；若改传 -f，先确认取负不会溢出。
    for (int t = 0; t < it; t++)
    {
        auto x = lerp(l, r, 1.0L / 3);
        auto y = lerp(l, r, 2.0L / 3);
        if (f(x) < f(y))
        {
            l = x;
        }
        else
        {
            r = y;
        }
    }
    return midpoint(l, r);
}

template <class F> long double goldenMax(long double l, long double r, F f, int it = 120)
{
    // l、r 是实数区间，f 是单峰函数，it 是迭代轮数；返回黄金分割搜索结果。
    // 求最小值时反向 fx、fy 的比较；其余复用点和区间更新方向不变。
    constexpr long double ratio = 0.61803398874989484820L;
    auto x = lerp(l, r, 1 - ratio);
    auto y = lerp(l, r, ratio);
    auto fx = f(x);
    auto fy = f(y);
    for (int t = 0; t < it; t++)
    {
        if (fx < fy)
        {
            l = x;
```

```
            x = y;
            fx = fy;
            y = lerp(l, r, ratio);
            fy = f(y);
        }
        else
        {
            r = y;
            y = x;
            fy = fx;
            x = lerp(l, r, 1 - ratio);
            fx = f(x);
        }
    }
    return midpoint(l, r);
}
```

只在能证明单峰时使用。三个函数都可直接接收 Lambda：

```
signed main()
{
    auto fi = [](int x)
    {
        return -(x - 3) * (x - 3);
    };
    auto fr = [](long double x)
    {
        return -(x - 1.5L) * (x - 1.5L);
    };
    assert(ternaryInt(-10, 10, fi) == 3);
    assert(fabsl(ternaryReal(0, 3, fr) - 1.5L) < 1E-12L);
    assert(fabsl(goldenMax(0, 3, fr) - 1.5L) < 1E-12L);
}
```

整数版要求调用前保证 `r-l` 能由 64 位有符号整数表示，循环中的差值和三分点因此不会溢出；实数版要求两个端点有限，`midpoint` 与 `lerp` 避免先把同号大端点相加或先计算跨极值的 `r-l`。整数版进行 $O(\log(r-l+1))$ 轮、每轮两次求值，末尾至多检查 5 点；`ternaryReal` 固定 $2\cdot it$ 次求值，`goldenMax` 初始化 2 次、之后每轮新增 1 次共 $it+2$ 次；额外空间均为 $O(1)$。

2.1.003 全排列与不重复排列

按字典序枚举多重集合的全部不同排列。先排序得到最小字典序，再反复 `next_permutation`：标准库按「下一个严格更大的序列」推进，遇到相同元素时自动跳过，因此不会产生重复排列。

```
vector<int> a{1, 1, 2}; // a 是待枚举的多重集合。
sort(a.begin(), a.end());
do
{
    use(a); // use 处理当前排列。
} while (next_permutation(a.begin(), a.end()));
```

必须先排序再进入 `do while`，故 `{1,1,2}` 恰好产生 3 个排列；`use(a)` 在循环体内直接处理当前排列，需要留存时自行复制一份 `vector`。不同排列数为 P 时，排序 $O(n \log n)$、枚举 $O(Pn)$；`next_permutation` 本身为 $O(1)$ 额外空间，常用 libstdc++/libc++ 的 `sort` 栈空间为 $O(\log n)$，若换用带缓冲的排序实现则以该实现为准，回调自行复制才额外需要 $O(n)$。

2.1.004 子集枚举、固定大小子集枚举

用位掩码枚举子集：普通枚举把第 i 位是否为 1 解释为是否选第 i 个元素，直接从 0 递增到 $2^n - 1$。Gosper's hack 由当前掩码直接构造下一个 popcount 相同的掩码——`low = s & -s` 取最低置位，`s + low` 进位后再把低位补齐——因此只走恰有 k 位的状态，不扫描其余掩码。两段都限定 $n < 63$，避免 `1ULL << n` 越过无符号 64 位边界。

```
// n 是元素数，s 是当前子集掩码，use 处理当前子集。
assert(0 <= n && n < 63); // 调试检查，可删
for (auto s = 0ULL; s < (1ULL << n); s++)
{
    use(s);
}
```

只枚举恰含 k 个元素的子集时，用 Gosper's hack 直接跳到下一个同位数掩码：

```
// n 是元素数，k 是子集大小，s 是当前掩码，use 处理当前子集。
assert(0 <= k && k <= n && n < 63); // 调试检查，可删
if (k == 0)
{
    use(0ULL);
}
else
{
    for (auto s = (1ULL << k) - 1; s < (1ULL << n);)
    {
        use(s);
        auto low = s & -s;
        auto nxt = s + low;
        s = nxt | (s ^ nxt) / low >> 2;
    }
}
```

两段都要求 $0 \le n < 63$，固定大小版本还要求 $0 \le k \le n$；$k = 0$ 时直接处理唯一的空集。若只要枚举某个掩码 `s` 的非空子集，用 `for (t = s; t; t = (t - 1) & s)`，不要遍历整个状态空间。全部子集共 2^n 个、固定大小子集恰 $\binom{n}{k}$ 个，每次推进 $O(1)$、额外空间 $O(1)$。

2.1.005 Gray Code

Gray Code 把 $0, 1, \ldots, 2^n - 1$ 重编码为 n 位序列，使相邻编码的汉明距离恒为 1，适合枚举子集时每步只增删一个元素（增量维护子集和、行列式符号、状态代价等），它只改变访问顺序、不减少 2^n 个状态。二进制反射编码为 $g = x \oplus (x \gg 1)$：相邻整数加一会翻转从最低位起的一段连续位，与右移一位的自身异或后这段翻转两两抵消，只剩一位变化；当 $n > 0$ 时，反射构造还保证完整序列首尾只差一位，形成

哈密顿环。逆变换满足 $x_i = g_i \oplus x_{i+1}$，即从最高位向低位做前缀异或，代码靠不断右移并累积异或一次完成。

```
// x 是普通二进制编号，g 是对应的 Gray Code。
auto g = x ^ (x >> 1);
```

从 Gray Code 还原普通编号时，把 g 的各次右移逐一异或：

```
// g 是 Gray Code，x 是还原后的普通二进制编号。
auto x = g;
for (auto y = g >> 1; y; y >>= 1)
{
    x ^= y;
}
```

对普通编号 `x` 执行第一段得 `g`，对 `g` 执行第二段还原 `x`；想知道第 x 个状态相对前一个翻转了哪一位，算 `g ^ preG` 取其唯一置位即可。当 $n > 0$ 时，连续枚举完整的 2^n 个 n 位 Gray Code，其相邻项与首尾都恰差一位；截取任意前缀后不再保证首尾相邻。逆变换仅适用于此处的二进制反射编码。编码在 64 位机器字上 $O(1)$，解码执行 g 的二进制位数减一次那么多轮，即 $O(\log(g+1))$ 时间、$O(1)$ 空间。

2.1.006 DFS / BFS 基础搜索

BFS 求无权图最短边数：队列按距离层弹出，节点首次被访问时的距离即最短。DFS 沿一条分支深入到底再回溯，访问顺序由邻接表决定；本模板按约定直接写递归 lambda，深链场景需确认系统栈足够。两者都靠 `d` / `vis` 保证每个点只正式处理一次。

```
vector<int> bfs(const vector<vector<int>> &g, int s)
{
    // g 是邻接表，s 是起点；返回各顶点到 s 的最少边数，-1 表示不可达。
    vector<int> d(g.size(), -1);
    queue<int> q;
    d[s] = 0;
    q.push(s);
    while (!q.empty())
    {
        auto x = q.front();
        q.pop();
        for (auto y : g[x])
        {
            if (d[y] != -1)
            {
                continue;
            }
            d[y] = d[x] + 1;
            q.push(y);
        }
    }
    return d;
}

```

```cpp
vector<int> dfsOrder(const vector<vector<int>> &g, int s)
{
    // g 是邻接表，s 是起点；返回从 s 出发的递归 DFS 首次访问顺序。
    vector<int> ans, vis(g.size());
    auto dfs = [&](auto &&self, int x) -> void
    {
        vis[x] = 1;
        ans.push_back(x);
        for (auto y : g[x])
        {
            if (!vis[y])
            {
                self(self, y);
            }
        }
    };
    dfs(dfs, s);
    return ans;
}
```

图用 `vector<vector<int>>` 邻接表、顶点编号 $[0, n)$，`bfs(g, s)` 返回各点到 `s` 的最少边数（`-1` 表示不可达），`dfsOrder(g, s)` 返回首次访问序（具体顺序由邻接表顺序决定）。邻接表图上均为 $O(n+m)$ 时间、$O(n)$ 额外空间；递归 DFS 另占最坏 $O(n)$ 调用栈。

2.1.007 启发式搜索与 A*（第 K 短路）

求第 k 短游走（允许重复经过顶点和边，不是第 k 条简单路径）。先在反图上跑 Dijkstra，得到每点到终点的真实最短距离 $h(x)$，它既不高估又满足一致性，是理想的 A* 启发函数；随后正图 A* 每次弹出全局最小的 $g+h$ 状态，同一顶点第 c 次有效弹出对应到它的第 c 条候选游走，终点达到所需弹出次数时即得答案。每点只扩展至多 `need` 次；通常 `need == k`，仅在起终点相同且不计空路时为 `k + 1`。负边会同时破坏反图 Dijkstra 与这一弹出顺序，故要求边权非负；可达状态用 `optional` 与距离分开，调用者必须保证实际距离、估价与二者之和可由 64 位有符号整数表示。

```cpp
// WEdge[0] 表示终点，WEdge[1] 表示边权。
using WEdge = array<int, 2>;

optional<int> kthShortest(const vector<vector<WEdge>> &g, int s, int t, int k, bool allowEmpty
    = false)
{
    // g 是非负权图，s、t 是起终点，k 是排名，allowEmpty 表示是否计入空路；不存在第 k 条游走时返回空。
    auto n = (int)g.size();
    assert(0 <= s && s < n && 0 <= t && t < n); // 调试检查，可删
    assert(k >= 1);                             // 调试检查，可删
    vector<vector<WEdge>> rg(n);
    for (int x = 0; x < n; x++)
    {
        for (auto [y, w] : g[x])
        {
            assert(0 <= y && y < n && w >= 0); // 调试检查，可删
```

```
            rg[y].push_back({x, w});
        }
    }
    vector<optional<int>> h(n);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> dq;
    h[t] = 0;
    dq.push({0, t});
    while (!dq.empty())
    {
        auto [dx, x] = dq.top();
        dq.pop();
        if (!h[x] || dx != *h[x])
        {
            continue;
        }
        for (auto [y, w] : rg[x])
        {
            auto d = dx + w;
            if (!h[y] || d < *h[y])
            {
                h[y] = d;
                dq.push({d, y});
            }
        }
    }
    if (!h[s])
    {
        return nullopt;
    }
    auto need = (unsigned long long)k + (s == t && !allowEmpty);
    // State 依次保存估价距离、已走距离和当前顶点。
    using State = tuple<int, int, int>;
    priority_queue<State, vector<State>, greater<>> q;
    vector<unsigned long long> cnt(n);
    q.push({*h[s], 0, s});
    while (!q.empty())
    {
        auto cur = q.top();
        q.pop();
        auto d = get<1>(cur);
        auto x = get<2>(cur);
        cnt[x]++;
        if (cnt[x] > need)
        {
            continue;
        }
        if (x == t && cnt[x] == need)
        {
            return d;
        }
        for (auto [y, w] : g[x])
        {
```

```
            if (!h[y])
            {
                continue;
            }
            auto nd = d + w;
            q.push({nd + *h[y], nd, y});
        }
    }
    return nullopt;
}
```

边表 `WEdge = {to, weight}`（`[0]` 终点、`[1]` 非负边权），建图后调用 `kthShortest(g,s,t,k)`，`nullopt` 只表示没有第 k 条游走；调用者须保证距离与估价和能由 `int` 表示。默认 `s == t` 时不把空路径计入，传 `allowEmpty = true` 才计入长度 0。以边集 $0\to1(1),0\to2(2),0\to3(10),1\to2(1),1\to3(4),2\to3(1)$ 为例，$0\to3$ 的前三短游走长度为 `3,3,5`。反图 Dijkstra 为 $O((n+m)\log n)$；A* 中每点至多有效弹出 $k+1$ 次、扩边至多 $O(km)$ 次，堆复杂度 $O(km\log(kn+km))$、最坏空间 $O(kn+km)$。改成简单路径则问题性质不同，不能套此模板。

2.1.008 模拟退火

无可靠精确算法、但状态可局部扰动且评分连续时求近似最优解，它没有正确性保证，不能替代可证明的凸优化、DP 或搜索。更优状态必然接受，更差状态以 $\exp(-\Delta/t)$ 的概率接受：高温放行较差跳转以逃出局部最优，降温后逐渐收敛稳定。结果质量高度依赖邻域设计、初温、降温率 `rate`、每层尝试次数 `each` 和多起点策略，因此接口把这些旋钮全部显式交给调用者。

```
template <class S, class Score, class Next, class Rng>
pair<S, long double> anneal(S cur,
                            Score score,
                            Next next,
                            Rng &rng,
                            long double t = 1,
                            long double stop = 1E-8L,
                            long double rate = .995L,
                            int each = 1)
{
    // cur 是初态，score/next 是评分与邻域函数，rng 是随机源；其余参数控制降温，返回最佳状态及分数。
    assert(t > stop && stop > 0 && 0 < rate && rate < 1 && each > 0); // 调试检查，可删
    long double val = (long double)score(cur);
    auto best = cur;
    long double ans = val;
    uniform_real_distribution<long double> dis(0, 1);
    while (t > stop)
    {
        for (int z = 0; z < each; z++)
        {
            auto p = next(cur, rng, t);
            long double v = (long double)score(p);
            long double d = v - val;
            if (d <= 0 || dis(rng) < expl(-d / t))
```

```
            {
                cur = move(p);
                val = v;
            }
            if (val < ans)
            {
                best = cur;
                ans = val;
            }
        }
        t *= rate;
    }
    return {best, ans};
}
```

`score` 越小越好且结果须可转为 `long double`，`next(cur,rng,t)` 生成邻居。下面从已知最优点出发，固定种子并核对最优状态不会丢失：

```
signed main()
{
    mt19937_64 rng(1);
    auto score = [](int x)
    {
        return (long double)(x - 3) * (x - 3);
    };
    auto next = [](int x, auto &rng, long double t)
    {
        (void)t;
        return x + uniform_int_distribution<int>(-1, 1)(rng);
    };
    auto [state, value] = anneal(3LL, score, next, rng, 1.L, 1E-4L, .9L, 4);
    assert(state == 3 && value == 0);
}
```

参数要求 $t_0 > stop > 0$、$0 < rate < 1$、`each > 0`，固定随机种子便于复现，正式提交宜多起点运行取最好值。温度层数为 $\left\lceil \frac{\log(stop/t_0)}{\log(rate)} \right\rceil$，候选总数为该值乘 `each`，总时间再乘单次 `score` 与 `next` 的代价，空间只两个状态；正确率无渐进保证。

2.1.009 DLX 精确覆盖

精确覆盖要求从 $0/1$ 矩阵中选若干行，使每一列恰被一个所选行覆盖；数独和棋盘精确铺放都能把候选决策写成行、把必须唯一满足的约束写成列。DLX 用循环双向十字链表实现 Algorithm X：递归时选择当前节点最少的列 c，尝试该列中的每一行；选行后删除它覆盖的全部列以及与这些列冲突的其他行，回溯时严格按相反顺序恢复，链表就回到进入递归前的状态。这里所有列都是必须覆盖一次的主列，列编号为 $1 \dots m$；同一行的列不得重复，只有 “至多一次” 的次要列需要另行扩展表头规则。

```
struct DLX
{
    // m 是必须恰好覆盖的列数。
```

```
    int m;
    // l、r、u、d 是十字链表节点向左、右、上、下的指针。
    vector<int> l, r, u, d;
    // row、col 分别是数据节点对应的行编号和列编号。
    vector<int> row, col;
    // sz[c] 是当前列 c 中仍未删除的数据节点数。
    vector<int> sz;
    // cur 是当前递归路径选中的行编号，ans 是找到的完整答案。
    vector<int> cur, ans;

    explicit DLX(int cols, int reserveNodes = 0)
        : m(cols), l(m + 1), r(m + 1), u(m + 1), d(m + 1), row(m + 1), col(m + 1), sz(m + 1)
    {
        // cols 是列数，reserveNodes 是预估的数据节点数；建立带根节点的列链表。
        assert(m >= 0 && reserveNodes >= 0); // 调试检查，可删。
        for (int i = 0; i <= m; i++)
        {
            l[i] = i - 1;
            r[i] = i + 1;
            u[i] = d[i] = col[i] = i;
        }
        l[0] = m;
        r[m] = 0;
        if (reserveNodes > 0)
        {
            auto cap = m + 1 + reserveNodes;
            l.reserve(cap);
            r.reserve(cap);
            u.reserve(cap);
            d.reserve(cap);
            row.reserve(cap);
            col.reserve(cap);
        }
    }

    void addRow(int id, const vector<int> &columns)
    {
        // id 是返回答案时使用的行编号，columns 是该行覆盖的列编号；把一行加入十字链表。
        int head = -1;
        for (int k = 0; k < (int)columns.size(); k++)
        {
            int c = columns[k];
            assert(1 <= c && c <= m); // 调试检查，可删。
            assert(find(columns.begin(), columns.begin() + k, c) == columns.begin() + k); // 调
            ↪ 试检查，可删。
            int x = (int)l.size();
            l.push_back(x);
            r.push_back(x);
            u.push_back(u[c]);
            d.push_back(c);
            row.push_back(id);
            col.push_back(c);
```

```
            d[u[c]] = x;
            u[c] = x;
            sz[c]++;
            if (head == -1)
            {
                head = x;
            }
            else
            {
                l[x] = l[head];
                r[x] = head;
                r[l[head]] = x;
                l[head] = x;
            }
        }
    }

    optional<vector<int>> solve()
    {
        // 无参数；返回一组覆盖全部列的行编号，无解时返回空。
        cur.clear();
        ans.clear();
        if (!dfs())
        {
            return nullopt;
        }
        return ans;
    }

private:
    void cover(int c)
    {
        // c 是当前仍在列链表中的列编号；从结构中删除该列及与其冲突的行。
        r[l[c]] = r[c];
        l[r[c]] = l[c];
        for (int i = d[c]; i != c; i = d[i])
        {
            for (int j = r[i]; j != i; j = r[j])
            {
                d[u[j]] = d[j];
                u[d[j]] = u[j];
                sz[col[j]]--;
            }
        }
    }

    void uncover(int c)
    {
        // c 是最近一次删除的列编号；按相反顺序恢复列和冲突行。
        for (int i = u[c]; i != c; i = u[i])
        {
            for (int j = l[i]; j != i; j = l[j])
```

```
            {
                sz[col[j]]++;
                d[u[j]] = j;
                u[d[j]] = j;
            }
        }
        r[l[c]] = c;
        l[r[c]] = c;
    }

    bool dfs()
    {
        // 无参数；返回当前子问题是否有解，并在成功时把行编号写入 ans。
        if (r[0] == 0)
        {
            ans = cur;
            return true;
        }
        int c = r[0];
        for (int j = r[c]; j != 0; j = r[j])
        {
            if (sz[j] < sz[c])
            {
                c = j;
            }
        }
        if (sz[c] == 0)
        {
            return false;
        }
        cover(c);
        for (int i = d[c]; i != c; i = d[i])
        {
            cur.push_back(row[i]);
            for (int j = r[i]; j != i; j = r[j])
            {
                cover(col[j]);
            }
            bool found = dfs();
            for (int j = l[i]; j != i; j = l[j])
            {
                uncover(col[j]);
            }
            cur.pop_back();
            if (found)
            {
                uncover(c);
                return true;
            }
        }
        uncover(c);
        return false;
```

```
    }
};
```

先构造 `DLX dlx(m,z)`，其中 `z` 只是用于预留数据节点容量的可选估计，再以 `addRow(id,columns)` 加入候选行并调用 `solve()`；返回值中的行编号按搜索选择顺序排列，空值表示无解，$m = 0$ 时空行集就是合法答案。设矩阵中 1 的总数为 z，建表时间与空间均为 $O(m + z)$；搜索最坏为指数级，单次删除或恢复的代价等于实际遍历的链表节点数，最小列启发式只改善搜索树而不改变最坏上界。若只需一般集合覆盖的最少行数，问题不再是精确覆盖，应使用状态压缩 DP、搜索剪枝或近似算法。

下面用七列标准实例核对唯一解由第 $2, 4, 6$ 行组成，再检查无列实例和无解实例：

```cpp
signed main()
{
    DLX dlx(7, 18);
    dlx.addRow(1, {1, 4, 7});
    dlx.addRow(2, {1, 4});
    dlx.addRow(3, {4, 5, 7});
    dlx.addRow(4, {3, 5, 6});
    dlx.addRow(5, {2, 3, 6, 7});
    dlx.addRow(6, {2, 7});
    auto ans = dlx.solve();
    assert(ans);
    sort(ans->begin(), ans->end());
    assert(*ans == vector<int>({2, 4, 6}));
    auto empty = DLX(0).solve();
    assert(empty && empty->empty());
    DLX bad(2);
    bad.addRow(1, {1});
    assert(!bad.solve());
    cout << (*ans)[0] << ' ' << (*ans)[1] << ' ' << (*ans)[2] << '\n'; // 期望输出：2 4 6
    return 0;
}
```

2.1.010 IDDFS、IDA* 与双向 BFS

三种方法都用于求起点到终点的最少步数或最小代价，适合状态空间很大、无法先建出完整图的搜索题。代码按显式邻接表写成，套到状态搜索时只需把 `g[u]` 换成状态 u 的后继生成，把 `used` 换成路径上状态的判重。

- IDDFS：依次以边数上限 $k = 0, 1, 2, \ldots$ 做限深 DFS，第一次找到终点时的 k 就是最少边数。任何时刻只保存当前路径，内存 $O(d)$（d 为答案深度）；浅层会被重复搜索，但分支因子 $b \geq 2$ 时最后一层占主导，总量仍是 $O(b^d)$。
- IDA*：把限深换成限制 $f(u) = g(u) + h(u)$，其中 $g(u)$ 是已走代价，$h(u)$ 是 u 到终点代价的下界（可采纳：$h(u) \leq \mathrm{dist}(u, t)$，且 $h(t) = 0$）。一轮中超过阈值的结点被剪掉，下一轮阈值取本轮被剪掉的最小 f。由于阈值单调增加且每轮只放行 f 不超过阈值的路径，第一次到达终点时的代价就是最小代价。
- 双向 BFS：从 s 沿原图、从 t 沿反图同时按层扩展，每次扩展规模较小一侧的整层。一个点同时被两侧到达时得到长度 $d_s(v) + d_t(v)$ 的路径，记最好值为 $best$。设两侧待扩展层号为 L_s, L_t，尚未发

现的路径长度至少 $L_s + L_t + 1$，所以 $L_s + L_t \geq best$ 时停止。

```cpp
namespace iter_search
{
optional<vector<int>> iddfs(const vector<vector<int>> &g, int s, int t, int lim)
{
    // g 是有向图邻接表，s、t 是起点和终点，lim 是允许的最多边数；
    // 返回边数最少的简单路径（含两端点的顶点序列），lim 内不可达时返回空。
    vector<char> used(g.size());
    vector<int> path;
    auto dfs = [&](auto &&self, int u, int d) -> bool
    {
        // u 是当前点，d 是还能走的边数；找到 t 时保留 path 并返回 true。
        path.push_back(u);
        used[u] = true;
        if (u == t)
        {
            return true;
        }
        if (d > 0)
        {
            for (int v : g[u])
            {
                if (!used[v] && self(self, v, d - 1))
                {
                    return true;
                }
            }
        }
        used[u] = false;
        path.pop_back();
        return false;
    };
    for (int k = 0; k <= min(lim, (int)g.size() - 1); k++)
    {
        if (dfs(dfs, s, k))
        {
            return path;
        }
    }
    return nullopt;
}

struct Path
{
    __int128 cost;        // cost 是路径边权和。
    vector<int> vertices; // vertices 是从起点到终点的顶点序列。
};

optional<Path> ida(const vector<vector<pair<int, int>>> &g, int s, int t, const vector<int> &h)
{
    // g[u] 存 {v,w}，w 为非负边权；h[u] 是 u 到 t 的可采纳下界，且 h[t]=0。
```

```
    // 返回 s 到 t 的一条最短简单路径，不可达时返回空。
    assert(h.size() == g.size() && h[t] == 0); // 调试检查，可删。
    const __int128 INF = (__int128)1 << 120;
    __int128 lim = h[s], next = INF, found = 0;
    vector<int> path;
    vector<char> used(g.size());
    auto dfs = [&](auto &&self, int u, __int128 cost) -> bool
    {
        // u 是当前点，cost 是已走代价；f=cost+h[u] 超过阈值时记录下一轮阈值并剪枝。
        __int128 f = cost + h[u];
        if (f > lim)
        {
            next = min(next, f);
            return false;
        }
        path.push_back(u);
        used[u] = true;
        if (u == t)
        {
            found = cost;
            return true;
        }
        for (auto [v, w] : g[u])
        {
            if (!used[v] && self(self, v, cost + w))
            {
                return true;
            }
        }
        used[u] = false;
        path.pop_back();
        return false;
    };
    while (true)
    {
        next = INF;
        if (dfs(dfs, s, 0))
        {
            return Path{found, path};
        }
        if (next == INF)
        {
            return nullopt;
        }
        lim = next;
    }
}

optional<vector<int>> biBfs(const vector<vector<int>> &g, int s, int t)
{
    // g 是有向无权图邻接表；返回 s 到 t 的一条最短路径顶点序列，不可达时返回空。
    int n = g.size();
```

```
    vector<vector<int>> rev(n);
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            rev[v].push_back(u);
        }
    }
    // da、db 是正向、反向 BFS 距离，pa、pb 是对应的前驱，a、b 是两侧当前层。
    vector<int> da(n, -1), db(n, -1), pa(n, -1), pb(n, -1), a{s}, b{t};
    da[s] = db[t] = 0;
    int best = n + 1, meet = -1;
    auto relax = [&](int v)
    {
        // v 是刚被访问的点；若两侧都到过 v，就用 v 更新当前最短拼接路径。
        if (da[v] >= 0 && db[v] >= 0 && da[v] + db[v] < best)
        {
            best = da[v] + db[v];
            meet = v;
        }
    };
    relax(s);
    while (!a.empty() && !b.empty() && da[a[0]] + db[b[0]] < best)
    {
        // 每次扩展较小的一侧整层，保证两侧层数之和单调增加。
        bool fw = a.size() <= b.size();
        auto &cur = fw ? a : b;
        auto &d = fw ? da : db;
        auto &p = fw ? pa : pb;
        const auto &adj = fw ? g : rev;
        vector<int> nxt;
        for (int u : cur)
        {
            for (int v : adj[u])
            {
                if (d[v] < 0)
                {
                    d[v] = d[u] + 1;
                    p[v] = u;
                    nxt.push_back(v);
                }
                relax(v);
            }
        }
        cur.swap(nxt);
    }
    if (meet < 0)
    {
        return nullopt;
    }
    vector<int> path;
    for (int u = meet; u >= 0; u = pa[u])
```

```
    {
        path.push_back(u);
    }
    reverse(path.begin(), path.end());
    for (int u = pb[meet]; u >= 0; u = pb[u])
    {
        path.push_back(u);
    }
    return path;
}
}
```

`iddfs(g,s,t,lim)` 只找边数不超过 $\min(lim, n-1)$ 的简单路径，`ida(g,s,t,h)` 要求非负边权、可采纳的 `h` 且 `h[t]==0`，`biBfs(g,s,t)` 用于无权有向图；三者都返回含两端点的顶点序列（`ida` 另带代价），不可达时返回空。IDDFS 与 IDA* 是指数时间：IDDFS 至多枚举长度不超过答案深度的全部简单路径，IDA* 的轮数等于不同阈值个数，整数边权时不超过最优代价加一；二者都只用 $O(n)$ 额外空间（n 为点数，即路径与判重数组）。双向 BFS 时间与空间都是 $O(n+m)$，比单向 BFS 少搜索的部分只在状态图分支很多时才明显。

显式图上的普通最短路直接用 5.1.002 DFS / BFS 或 5.2.001 Dijkstra：稀疏图与稠密图；内存足够时带启发函数的单轮搜索用 2.1.007 启发式搜索与 A*（第 K 短路）。下面的图里 $0 \to 4$ 的直连边只有 1 条边但代价为 9，用来区分最少边数与最小代价：

```
signed main()
{
    vector<vector<int>> g{{1, 2, 4}, {3}, {3}, {4}, {}};
    vector<vector<pair<int, int>>> wg{{{1, 1}, {2, 2}, {4, 9}}, {{3, 5}}, {{3, 1}}, {{4, 1}},
↪   {}};
    vector<int> h{2, 1, 1, 1, 0}; // 每条边代价至少 1，取到 4 的最少边数，可采纳。
    auto p = iter_search::iddfs(g, 0, 3, 5);
    auto q = iter_search::biBfs(g, 0, 4);
    auto r = iter_search::ida(wg, 0, 4, h);
    assert(p && *p == vector<int>({0, 1, 3}));
    assert(q && *q == vector<int>({0, 4}));
    assert(r && r->cost == 4 && r->vertices == vector<int>({0, 2, 3, 4}));
    assert(!iter_search::iddfs(g, 4, 0, 5) && !iter_search::biBfs(g, 4, 0));
}
```

2.2 排序、分治与序列处理

2.2.001 快速排序、归并排序、基数排序

三种排序不是同一功能的快慢版本，接口和内存约束各异。通用比较排序 `sort` 只依赖严格弱序，能处理任意对象和自定义比较器，是默认首选；`mergeSort` 在合并两个有序段时保持相等元素原顺序，需要稳定性或想在归并过程中顺带统计（如逆序对）时用它；`radixSort` 把有符号 64 位整数翻转符号位（`^ (1ULL << 63)`）后按 16 位一段稳定分桶、四趟完成，数据是大量纯整数且内存充裕时最快。

```
sort(a.begin(), a.end());
```

```cpp
template <class T, class Cmp = less<T>> void mergeSort(vector<T> &a, Cmp cmp = {})
{
    // a 是待排序数组，cmp 是严格弱序比较器；排序结果直接写回 a。
    vector<T> b;
    b.reserve(a.size());
    auto dfs = [&](auto &&self, int l, int r) -> void
    {
        if (r - l <= 1)
        {
            return;
        }
        auto m = midpoint(l, r);
        self(self, l, m);
        self(self, m, r);
        auto i = l;
        auto j = m;
        b.clear();
        while (i < m || j < r)
        {
            if (j == r || (i < m && !cmp(a[j], a[i])))
            {
                b.push_back(move(a[i++]));
            }
            else
            {
                b.push_back(move(a[j++]));
            }
        }
        move(b.begin(), b.end(), a.begin() + l);
    };
    dfs(dfs, 0, a.size());
}

void radixSort(vector<int> &a)
{
    // a 是待排序的有符号 64 位整数数组；排序结果直接写回 a。
    static_assert(numeric_limits<int>::digits == 63);
    vector<unsigned long long> x(a.size()), b(a.size());
    for (int i = 0; i < (int)a.size(); i++)
    {
        x[i] = (unsigned long long)a[i] ^ (1ULL << 63);
    }
    constexpr int B = 1 << 16;
    vector<int> cnt(B);
    for (int sh = 0; sh < 64; sh += 16)
    {
        fill(cnt.begin(), cnt.end(), 0);
        for (auto v : x)
        {
            cnt[v >> sh & (B - 1)]++;
        }
        partial_sum(cnt.begin(), cnt.end(), cnt.begin());
```

```
        for (auto it = x.rbegin(); it != x.rend(); it++)
        {
            auto p = *it >> sh & (B - 1);
            b[--cnt[p]] = *it;
        }
        x.swap(b);
    }
    for (int i = 0; i < (int)a.size(); i++)
    {
        a[i] = bit_cast<int>(x[i] ^ (1ULL << 63));
    }
}
```

`sort(a.begin(), a.end())` 通用首选，`mergeSort(a)` 稳定备用，`radixSort(a)` 专供纯整数大数据。归并缓冲通过 `reserve` 后逐个构造，不要求 `T` 可默认构造；基数版要求 64 位二补码有符号整数，末步用 `bit_cast<int>` 恢复位模式，不依赖超范围无符号数到有符号数的实现定义转换。本机基准（MinGW g++ 14.2、`-O2`、一百万随机 64 位整数、7 轮中位数）为 `radixSort=16ms`、`sort=41ms`、`mergeSort=67ms`，基数排序最快，但比较器、自定义对象和低内存场景仍用 `sort`。`sort` 与 `mergeSort` 均 $O(n \log n)$、归并额外空间 $O(n)$；基数排序固定 4 趟、每趟 $O(n + 2^{16})$，总空间 $O(n + 2^{16})$。

传自定义比较器时，例如按降序稳定归并：

```
signed main()
{
    vector<int> a{2, 1, 3};
    auto cmp = [](int x, int y)
    {
        return x > y;
    };
    mergeSort(a, cmp);
    assert((a == vector<int>{3, 2, 1}));
}
```

2.2.002 离散化与坐标压缩

把任意可比较值映射到 $[0, V)$，保留大小关系和相等关系，是本章多处（整体二分、区间不同数等）共用的底层结构，故在此首次完整给出。做法是复制全部候选值，排序去重后用 `lower_bound` 求排名：`id(x)` 要求 `x` 已存在并返回其编号，`lower(x)` 返回第一个不小于 `x` 的位置，`value(p)` 反查原值。注意编号只能用于比较顺序、频率和树状数组下标；原值差、几何距离等运算必须回到 `value(id)`，否则会把不等距坐标误当成等距。

```
template <class T> struct Discrete
{
    vector<T> a; // a 保存升序且去重后的原值。

    Discrete() = default;

    explicit Discrete(vector<T> v) : a(move(v))
    {
```

```
        // v 收集所有可能出现的值。
        sort(a.begin(), a.end());
        a.erase(unique(a.begin(), a.end()), a.end());
    }

    int id(const T &x) const
    {
        // x 必须已存在；返回 x 从 0 开始的离散编号。
        auto it = lower_bound(a.begin(), a.end(), x);
        assert(it != a.end() && *it == x); // 调试检查，可删
        return it - a.begin();
    }

    int lower(const T &x) const
    {
        // x 是查询值；返回第一个不小于 x 的离散位置。
        return lower_bound(a.begin(), a.end(), x) - a.begin();
    }

    const T &value(int p) const
    {
        // p 是合法离散编号；返回对应原值的常量引用。
        return a[p];
    }
};
```

以 `Discrete<int> dis(a)` 构造后，`dis.id(x)`、`dis.lower(x)`、`dis.value(p)` 分别取编号、下界位置和原值。这是静态离散化：在线出现新值时会失效，须提前收集所有可能的更新值，或改用平衡树；也不要把离散编号当原值做距离运算。构造 $O(n \log n)$ 时间、$O(n)$ 空间，单次映射 $O(\log V)$。

2.2.003 逆序对及其扩展

统计 $i < j$ 且 $a_i > a_j$ 的二元组。两种同为 $O(n \log n)$ 但扩展方向不同的做法都完整保留，不合并成黑盒：归并版在合并时，每当右半元素先被取出，就一次性加上左半尚未取出的元素数 `m - i`，无需离散化且便于顺带累计其他归并贡献；BIT 版先离散化，再从右向左扫描、查询严格小于当前值的已加入数量，便于改成按值排名、动态频率或别的前缀统计。相等元素在两版中都不计入逆序对。

```
// 依赖 2.2.002 离散化与坐标压缩
template <class T> int invMerge(vector<T> a)
{
    // a 是按值复制的序列；返回严格逆序对数量。
    vector<T> b;
    b.reserve(a.size());
    int ans = 0;
    auto dfs = [&](auto &&self, int l, int r) -> void
    {
        if (r - l <= 1)
        {
            return;
```

```
        }
        auto m = midpoint(l, r);
        self(self, l, m);
        self(self, m, r);
        auto i = l;
        auto j = m;
        b.clear();
        while (i < m || j < r)
        {
            if (j == r || (i < m && !(a[j] < a[i])))
            {
                b.push_back(move(a[i++]));
            }
            else
            {
                ans += m - i;
                b.push_back(move(a[j++]));
            }
        }
        move(b.begin(), b.end(), a.begin() + l);
    };
    dfs(dfs, 0, a.size());
    return ans;
}

template <class T> int invBit(const vector<T> &a)
{
    // a 是原序列；返回用离散化和 Fenwick 统计的严格逆序对数量。
    Discrete<T> dis(a);
    vector<int> bit(dis.a.size() + 1);
    auto add = [&](int p)
    {
        for (p++; p < (int)bit.size(); p += p & -p)
        {
            bit[p]++;
        }
    };
    auto sum = [&](int p)
    {
        int ans = 0;
        for (; p > 0; p -= p & -p)
        {
            ans += bit[p];
        }
        return ans;
    };
    int ans = 0;
    for (auto it = a.rbegin(); it != a.rend(); it++)
    {
        auto p = dis.id(*it);
        ans += sum(p);
        add(p);
```

```
    }
    return ans;
}
```

`invMerge(a)` 与 `invBit(a)` 结果相同；返回值为 `int`，在本模板即 64 位。只做一次静态统计选归并版更直接，还要按值维护频率或排名选 BIT 版；扩展到其他偏序时必须重新确认严格/非严格比较和重复值处理。本机基准（同一组一百万随机 64 位整数、7 轮中位数）归并版 `69ms`、BIT 版 `177ms`，故静态统计默认归并版，BIT 版作值域统计扩展保留。两版均 $O(n \log n)$ 时间、$O(n)$ 额外空间。

2.2.004 整体二分

当一批询问都在同一有序答案域上二分、且一次判定能用可撤销的数据结构批量完成时，把「每个询问各自二分」改成递归地一起划分询问与元素：在值域中点 `vm` 处把离散排名 $< vm$ 的元素加入 Fenwick，每个询问看自己区间内已加入的元素数 `c` 决定落入左半还是右半答案域（落右半时把 `k` 扣掉 `c`），本层递归前务必回滚 Fenwick 修改。它要求判定关于答案单调，不支持在线强制查询。本条实例 `rangeKth` 求静态数组区间第 k 小；其中的离散化依赖 2.2.002 离散化与坐标压缩 的 `Discrete`，此处不再重复，而 `Fenwick` 树状数组在本章首次完整给出，后续 2.2.005 CDQ 分治与三维偏序、2.3.005 区间不同数统计 均按编号引用本条。

```
struct Fenwick
{
    int n = 0;      // n 是下标范围长度。
    vector<int> a; // a 保存从 1 开始的树状数组节点和。

    Fenwick(int m = 0)
    {
        // m 是初始下标范围长度。
        init(m);
    }

    void init(int m)
    {
        // m 是新的下标范围长度；调用后全部频率清零。
        assert(m >= 0); // 调试检查，可删
        n = m;
        a.assign(n + 1, 0);
    }

    void add(int p, int v)
    {
        // p 是从 0 开始的位置，v 是增量。
        assert(0 <= p && p < n); // 调试检查，可删
        for (p++; p <= n; p += p & -p)
        {
            a[p] += v;
        }
    }

    int sum(int r) const
```

```
    {
        // r 是右端点；返回半开前缀 [0,r) 的和。
        assert(0 <= r && r <= n); // 调试检查，可删
        int ans = 0;
        for (; r > 0; r -= r & -r)
        {
            ans += a[r];
        }
        return ans;
    }

    int sum(int l, int r) const
    {
        // l、r 是半开区间端点；返回 [l,r) 的和。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return sum(r) - sum(l);
    }
};

// KthQuery[0..2] 依次表示左端点、右端点、从 1 开始的排名。
using KthQuery = array<int, 3>;

vector<int> rangeKth(const vector<int> &a, vector<KthQuery> qs)
{
    // a 是静态数组，qs 中每项是 {l,r,k}；返回各半开区间第 k 小值。
    if (qs.empty())
    {
        return {};
    }
    assert(!a.empty()); // 调试检查，可删
    for (auto q : qs)
    {
        assert(0 <= q[0] && q[0] < q[1] && q[1] <= (int)a.size()); // 调试检查，可删
        assert(1 <= q[2] && q[2] <= q[1] - q[0]);                  // 调试检查，可删
    }
    Discrete<int> dis(a);
    vector<int> rk(a.size());
    for (int i = 0; i < (int)a.size(); i++)
    {
        rk[i] = dis.id(a[i]);
    }
    vector<int> es(a.size()), ids(qs.size()), ans(qs.size());
    iota(es.begin(), es.end(), 0);
    iota(ids.begin(), ids.end(), 0);
    Fenwick bit(a.size());
    auto solve = [&](auto &&self, int vl, int vr, vector<int> ps, vector<int> qi) -> void
    {
        if (qi.empty())
        {
            return;
        }
        if (vr - vl == 1)
```

```
        {
            for (auto id : qi)
            {
                ans[id] = dis.value(vl);
            }
            return;
        }
        auto vm = midpoint(vl, vr);
        vector<int> pl, pr, ql, qr;
        for (auto p : ps)
        {
            if (rk[p] < vm)
            {
                bit.add(p, 1);
                pl.push_back(p);
            }
            else
            {
                pr.push_back(p);
            }
        }
        for (auto id : qi)
        {
            auto &q = qs[id];
            auto c = bit.sum(q[0], q[1]);
            if (q[2] <= c)
            {
                ql.push_back(id);
            }
            else
            {
                q[2] -= c;
                qr.push_back(id);
            }
        }
        for (auto p : pl)
        {
            bit.add(p, -1);
        }
        self(self, vl, vm, move(pl), move(ql));
        self(self, vm, vr, move(pr), move(qr));
    };
    solve(solve, 0, dis.a.size(), move(es), move(ids));
    return ans;
}
```

`KthQuery = {l, r, k}`（半开区间加从 1 开始的排名，须 $1 \le k \le r-l$），`rangeKth(a, qs)` 返回各区间第 k 小值；空询问集直接返回空数组，非空询问要求原数组非空。函数内部只改询问副本里的 `k`、不动调用者数据；上面的代码需与本章前导及 2.2.002 的 `Discrete` 拼接编译。整体二分须满足答案单调、修改可按答案域分组、判定状态可回滚，若只是一次静态 `nth_element` 就别套它。答案域离散后大小为 V 时，每层处理全部元素与询问并做 Fenwick 操作，时间 $O((n+q)\log V \log n)$、空间 $O(n+q+V)$。

固定数组同时覆盖半开子区间、重复值和排名两端：

```cpp
signed main()
{
    vector<int> a{5, 1, 4, 1, 3};
    vector<KthQuery> qs{{0, 5, 1}, {0, 5, 3}, {0, 5, 5},
                        {1, 4, 1}, {1, 4, 2}, {1, 4, 3}};
    auto ans = rangeKth(a, qs);
    assert((ans == vector<int>{1, 3, 5, 1, 1, 4}));
    cout << ans[0] << ' ' << ans[1] << ' ' << ans[2] << ' '
         << ans[3] << ' ' << ans[4] << ' ' << ans[5] << '\n'; // 1 3 5 1 1 4
    return 0;
}
```

2.2.005 CDQ 分治与三维偏序

统计每个三维点被多少点在三维上同时不大于。先按 `x`，`y`，`z` 排序，第一维借排序天然处理，再对完全相同的点合组（否则递归顺序会让相同点得到不同答案）；CDQ 分治把第二维当作「时间维」，只统计左半对右半的贡献，合并时按 `y` 归并、用 Fenwick 查询 `z` 前缀。所用 `Fenwick` 与 2.2.004 整体二分 完全相同，此处依赖该条的 `Fenwick`，不再重复粘贴。

```cpp
// Point3[0..3] 依次是 x、y、z 坐标与原输入编号。
using Point3 = array<int, 4>;

// answer[i] 是三维坐标均不大于第 i 个点的点数。
inline vector<int> dom3(vector<Point3> points)
{
    // points 的每项为 {x,y,z,id}；返回每个原编号的三维非严格支配计数。
    int n = points.size();
    vector<int> coord;
    for (auto point : points)
    {
        coord.push_back(point[2]);
    }
    ranges::sort(coord);
    coord.erase(unique(coord.begin(), coord.end()), coord.end());
    ranges::sort(points,
                 [](const Point3 &a, const Point3 &b)
                 {
                     return tie(a[0], a[1], a[2]) < tie(b[0], b[1], b[2]);
                 });
    struct Node
    {
        // x、y 是原坐标，z 是离散编号，weight 是该坐标的重复点数。
        int x, y, z, weight;
        // ids 保存该组重复点的原编号。
        vector<int> ids;
        // answer 保存当前累计的非严格支配计数。
        long long answer;
    };
```

```
    vector<Node> nodes;
    for (auto point : points)
    {
        int z = lower_bound(coord.begin(), coord.end(), point[2]) - coord.begin();
        if (!nodes.empty() && tie(nodes.back().x, nodes.back().y, nodes.back().z) ==
        ↪ tie(point[0], point[1], z))
        {
            ++nodes.back().weight;
            ++nodes.back().answer;
            nodes.back().ids.push_back(point[3]);
        }
        else
        {
            nodes.push_back({point[0], point[1], z, 1, {point[3]}, 1});
        }
    }
    vector<int> bit(coord.size() + 1), order(nodes.size()), buffer(nodes.size()), answer(n);
    iota(order.begin(), order.end(), 0);
    auto add = [&](int x, int value)
    {
        for (++x; x < bit.size(); x += x & -x)
        {
            bit[x] += value;
        }
    };
    auto sum = [&](int x)
    {
        int result = 0;
        for (++x; x > 0; x -= x & -x)
        {
            result += bit[x];
        }
        return result;
    };
    function<void(int, int)> cdq = [&](int left, int right)
    {
        if (right - left <= 1)
        {
            return;
        }
        int middle = midpoint(left, right);
        cdq(left, middle);
        cdq(middle, right);
        int i = left, j = middle, output = left;
        while (j < right)
        {
            while (i < middle && nodes[order[i]].y <= nodes[order[j]].y)
            {
                add(nodes[order[i]].z, nodes[order[i]].weight);
                buffer[output++] = order[i++];
            }
            nodes[order[j]].answer += sum(nodes[order[j]].z);
```

```
            buffer[output++] = order[j++];
        }
        auto used = i;
        while (i < middle)
        {
            buffer[output++] = order[i++];
        }
        for (int k = left; k < used; ++k)
        {
            add(nodes[order[k]].z, -nodes[order[k]].weight);
        }
        copy(buffer.begin() + left, buffer.begin() + right, order.begin() + left);
    };
    cdq(0, nodes.size());
    for (auto &node : nodes)
    {
        for (int id : node.ids)
        {
            answer[id] = node.answer;
        }
    }
    return answer;
}

struct Point4
{
    // x、y、z、w 是四维坐标。
    int x, y, z, w;
};

struct Dom4Event
{
    // y、z、w 是后三维坐标；src 表示该点来自较小 x 的左半部。
    int y, z, w;
    bool src;
};

inline long long dom4Cross(vector<Dom4Event> a, int m)
{
    // a 是一次跨段统计的事件，m 是离散后第四维大小；返回后三维均严格递增的跨段点对数。
    sort(a.begin(), a.end(), [](const auto &u, const auto &v)
    {
        return tie(u.y, u.src) < tie(v.y, v.src);
    });
    vector<Dom4Event> buf(a.size());
    vector<int> bit(m + 1);
    auto add = [&](int p, int v)
    {
        for (++p; p <= m; p += p & -p)
        {
            bit[p] += v;
        }
```

```
    };
    auto sum = [&](int p)
    {
        int ans = 0;
        for (; p; p -= p & -p)
        {
            ans += bit[p];
        }
        return ans;
    };
    long long ans = 0;
    auto cdq = [&](auto &&self, int l, int r) -> void
    {
        if (r - l <= 1) return;
        int mid = (l + r) / 2;
        self(self, l, mid);
        self(self, mid, r);
        int i = l;
        for (int j = mid; j < r; j++)
        {
            while (i < mid && a[i].z < a[j].z)
            {
                if (a[i].src) add(a[i].w, 1);
                i++;
            }
            if (!a[j].src) ans += sum(a[j].w);
        }
        for (int k = l; k < i; k++)
        {
            if (a[k].src) add(a[k].w, -1);
        }
        merge(a.begin() + l, a.begin() + mid, a.begin() + mid, a.begin() + r,
              buf.begin() + l, [](const auto &u, const auto &v)
        {
            return u.z < v.z;
        });
        copy(buf.begin() + l, buf.begin() + r, a.begin() + l);
    };
    cdq(cdq, 0, a.size());
    return ans;
}

inline long long countFourDimensional(vector<Point4> a)
{
    // a 是四维点集；返回两点四维坐标均严格递增的有序点对数量。
    vector<int> ws;
    for (auto p : a)
    {
        ws.push_back(p.w);
    }
    sort(ws.begin(), ws.end());
    ws.erase(unique(ws.begin(), ws.end()), ws.end());
```

```
    for (auto &p : a)
    {
        p.w = lower_bound(ws.begin(), ws.end(), p.w) - ws.begin();
    }
    sort(a.begin(), a.end(), [](const auto &u, const auto &v)
    {
        return tie(u.x, u.y, u.z, u.w) < tie(v.x, v.y, v.z, v.w);
    });
    long long ans = 0;
    auto solve = [&](auto &&self, int l, int r) -> void
    {
        if (r - l <= 1) return;
        int mid = (l + r) / 2;
        while (mid < r && a[mid - 1].x == a[mid].x)
        {
            mid++;
        }
        if (mid == r)
        {
            mid = (l + r) / 2;
            while (mid > l && a[mid - 1].x == a[mid].x)
            {
                mid--;
            }
            if (mid == l) return;
        }
        self(self, l, mid);
        self(self, mid, r);
        vector<Dom4Event> e;
        e.reserve(r - l);
        for (int i = l; i < mid; i++)
        {
            e.push_back({a[i].y, a[i].z, a[i].w, true});
        }
        for (int i = mid; i < r; i++)
        {
            e.push_back({a[i].y, a[i].z, a[i].w, false});
        }
        ans += dom4Cross(move(e), ws.size());
    };
    solve(solve, 0, a.size());
    return ans;
}
```

`Point3 = {x, y, z, id}`，`id` 须覆盖 $[0,n)$ 且唯一，`dom3(p)` 返回每个原下标的支配计数；例如 `{{1,1,1,0},{1,1,1,1},{2,0,2,2},{2,2,2,3}}` 得 `{2,2,1,4}`。计数含点自身和完全相同的其他点，严格偏序需调整排序、分组和查询边界。排序与 CDQ 共 $O(n\log^2 n)$ 时间，Fenwick、归并缓冲和节点数组共 $O(n)$ 空间。同一条目的 `countFourDimensional` 统计四维坐标均严格递增的有序点对；外层按第一维分治，内层 CDQ 处理后三维，复杂度 $O(n\log^3 n)$，坐标任一维相等均不计入。

固定例覆盖重复点、不可比点、被全部点支配的点和全同点（`Fenwick` 取自 2.2.004）：

```cpp
signed main()
{
    vector<Point3> p{{1, 1, 1, 0}, {1, 1, 1, 1},
                     {2, 0, 2, 2}, {2, 2, 2, 3}};
    auto ans = dom3(p);
    assert((ans == vector<int>{2, 2, 1, 4}));
    assert((dom3({{7, 7, 7, 0}, {7, 7, 7, 1}, {7, 7, 7, 2}}) ==
            vector<int>{3, 3, 3}));
    cout << ans[0] << ' ' << ans[1] << ' ' << ans[2] << ' ' << ans[3] << '\n'; // 期望输出：2 2
    ↪ 1 4
    return 0;
}
```

2.2.006 分数级联（Fractional Cascading）

给定 k 个静态有序表，需要很多次把同一个值 x 分别送进每张表求 `lower_bound`。逐表二分单次是 $O(\sum_i \log n_i)$；分数级联预处理后只对第一张增强表二分，随后逐层沿「桥」转移，把单次查询降到 $O(\log N + k)$，其中 $N = \sum_i n_i$。构造从最后一层向前：增强表 B_i 把原表 A_i 与下一层 B_{i+1} 的约一半元素（隔一个采样一个）合并，每个增强元素记录它在 A_i 中的下界 `own` 与在 B_{i+1} 中的下界 `down`。因为下层每隔一个就被采样，沿 `down` 落到的位置与真实下界至多差一步（查询里的 `p--` 修正这一步），所以后续每层只需常数次比较；采样保证所有增强表总长仍为 $O(N)$。典型用于线段树或树链分解后同时查多份有序目录，以及计算几何中沿相邻层级定位同一坐标。

```cpp
template <class T> struct FracCascade
{
    struct Item
    {
        T x;           // x 是增强表中的键值。
        int own, down; // own 指向本层原表下界，down 指向下一层增强表下界。
    };

    vector<vector<T>> a;    // a 保存各层原始有序表。
    vector<vector<Item>> b; // b 保存采样合并后的增强表和桥指针。

    explicit FracCascade(vector<vector<T>> src) : a(move(src)), b(a.size())
    {
        // src 是若干个按同一规则升序排列的静态表。
        for (auto &x : a)
        {
            assert(is_sorted(x.begin(), x.end())); // 调试检查，可删
        }
        for (int i = (int)a.size() - 1; i >= 0; i--)
        {
            auto v = a[i];
            if (i + 1 < (int)b.size())
            {
                for (int j = 1; j < (int)b[i + 1].size(); j += 2)
                {
                    v.push_back(b[i + 1][j].x);
```

```
                }
            }
            sort(v.begin(), v.end());
            v.erase(unique(v.begin(), v.end()), v.end());
            for (auto x : v)
            {
                auto p = lower_bound(a[i].begin(), a[i].end(), x) - a[i].begin();
                int q = 0;
                if (i + 1 < (int)b.size())
                {
                    q = lower_bound(b[i + 1].begin(),
                                    b[i + 1].end(),
                                    x,
                                    [](const Item &u, const T &rhs)
                                    {
                                        return u.x < rhs;
                                    }) -
                        b[i + 1].begin();
                }
                b[i].push_back({x, p, q});
            }
        }
    }

    vector<int> lowerBound(const T &x) const
    {
        // x 是所有表共用的查询值；返回每个原表中的 lower_bound 位置。
        vector<int> ans(a.size());
        int p = 0;
        if (!b.empty())
        {
            p = lower_bound(b[0].begin(),
                            b[0].end(),
                            x,
                            [](const Item &u, const T &rhs)
                            {
                                return u.x < rhs;
                            }) -
                b[0].begin();
        }
        for (int i = 0; i < (int)a.size(); i++)
        {
            ans[i] = p == (int)b[i].size() ? (int)a[i].size() : b[i][p].own;
            if (i + 1 == (int)a.size())
            {
                break;
            }
            p = p == (int)b[i].size() ? (int)b[i + 1].size() : b[i][p].down;
            if (p && b[i + 1][p - 1].x >= x)
            {
                p--;
            }
```

```
        }
        return ans;
    }
};
```

每个输入表须按同一规则升序排列，构造后不支持插入、删除或改值。`fc.lowerBound(x)` 返回长度 k 的位置数组，第 i 项是 A_i 中第一个不小于 x 的下标、无则等于 A_i 长度；例如三张表 `{{-3,1,5,9},{0,2,8},{-5,4,4,10}}` 查 4 得 `{2,2,1}`。只有一次查询或表数很少时预处理不划算，有动态修改时改用平衡树、线段树套有序结构或重建分块。当前实现逐层排序建桥，构造 $O(N \log N)$ 时间、$O(N)$ 空间，单次查询 $O(\log N + k)$；改用线性归并建每层可把预处理降到 $O(N)$。

固定查询覆盖小于全部元素、重复值、空表、命中元素和大于全部元素：

```
signed main()
{
    vector<vector<int>> a{{-3, 1, 5, 9}, {}, {0, 2, 8}, {-5, 4, 4, 10}};
    FracCascade<int> fc(a);
    assert((fc.lowerBound(-10) == vector<int>{0, 0, 0, 0}));
    assert((fc.lowerBound(4) == vector<int>{2, 0, 2, 1}));
    assert((fc.lowerBound(5) == vector<int>{2, 0, 2, 3}));
    assert((fc.lowerBound(11) == vector<int>{4, 0, 3, 4}));
    auto ans = fc.lowerBound(4);
    cout << ans[0] << ' ' << ans[1] << ' ' << ans[2] << ' ' << ans[3] << '\n'; // 期望输出：2 0
 ↪  2 1
    return 0;
}
```

2.2.007 扫描线

矩形面积并是扫描线最常用、也最便于完整测试的实例：把每个矩形拆成左右两条竖边，按 x 排序成进入（+1）/离开（−1）事件，线段树维护当前 y 轴被覆盖的总长度。相邻两事件横坐标之间活跃矩形集合不变，故面积增量为「当前 y 覆盖长度 × 横向宽度」。线段树每个节点记覆盖次数 `cov` 和实际覆盖长度 `len`：`cov` 非零时整段有效，为零时才向下合并子节点的 `len`——正因如此这棵覆盖线段树不需要 lazy 下传。相同 x 的事件必须成组处理，避免出现零宽度的中间状态。

```
// Rect[0..3] 依次表示左横坐标、下纵坐标、右横坐标、上纵坐标。
using Rect = array<int, 4>;

int rectArea(const vector<Rect> &rs)
{
    // rs 中每项是 {左、下、右、上}；返回所有半开矩形的面积并。
    // Event[0..3] 依次表示横坐标、下纵坐标、上纵坐标、覆盖次数变化量。
    using Event = array<int, 4>;
    vector<Event> es;
    vector<int> ys;
    for (auto [xl, yl, xr, yr] : rs)
    {
        if (xl >= xr || yl >= yr)
        {
```

```
            continue;
        }
        es.push_back({xl, yl, yr, 1});
        es.push_back({xr, yl, yr, -1});
        ys.push_back(yl);
        ys.push_back(yr);
    }
    if (es.empty())
    {
        return 0;
    }
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());
    sort(es.begin(),
         es.end(),
         [](const auto &a, const auto &b)
         {
             return a[0] < b[0];
         });
    vector<int> cov(ys.size() * 4);
    vector<int> len(ys.size() * 4);
    auto add = [&](auto &&self, int p, int l, int r, int ql, int qr, int d) -> void
    {
        if (ql <= l && r <= qr)
        {
            cov[p] += d;
        }
        else
        {
            auto m = midpoint(l, r);
            if (ql < m)
            {
                self(self, p * 2, l, m, ql, qr, d);
            }
            if (m < qr)
            {
                self(self, p * 2 + 1, m, r, ql, qr, d);
            }
        }
        if (cov[p])
        {
            len[p] = ys[r] - ys[l];
        }
        else if (r - l == 1)
        {
            len[p] = 0;
        }
        else
        {
            len[p] = len[p * 2] + len[p * 2 + 1];
        }
    };
```

```
    int ans = 0;
    auto pre = es[0][0];
    for (int i = 0; i < (int)es.size();)
    {
        auto width = es[i][0] - pre;
        ans += len[1] * width;
        auto j = i;
        while (j < (int)es.size() && es[j][0] == es[i][0])
        {
            auto [x, yl, yr, d] = es[j];
            auto l = lower_bound(ys.begin(), ys.end(), yl) - ys.begin();
            auto r = lower_bound(ys.begin(), ys.end(), yr) - ys.begin();
            add(add, 1, 0, ys.size() - 1, l, r, d);
            j++;
        }
        pre = es[i][0];
        i = j;
    }
    return ans;
}
```

`Rect = {xl, yl, xr, yr}`，区域为 $[x_l, x_r) \times [y_l, y_r)$，退化矩形自动忽略；`rectArea(rs)` 返回面积并，例如 `{{0,0,2,2},{1,1,3,2}}` 得 5。调用者须保证坐标差、扫描线覆盖长度及最终面积都能由 `int` 表示；确实需要更宽面积时把这份短代码中的坐标和面积类型一起改宽。$2n$ 个事件、$2n$ 个纵坐标，排序与更新共 $O(n \log n)$ 时间、$O(n)$ 空间。

2.2.008 单调栈、单调队列

共同直觉是：新元素到来时，所有既不优于它、又比它更早的候选都可永久删除，因为新元素值更优且失效更晚。`prevLess` 用栈求每个位置左侧最近严格更小元素；`windowMin` / `windowMax` 用双端队列维护固定窗口的最值，比栈多一步从队首删除滑出窗口（`q.front() <= i - k`）的下标。比较符号决定「严格」还是「允许相等」——这里弹出条件用 `>=`（最小值）/ `<=`（最大值），留下的是严格意义的最近更小/更大，照抄前务必按题意核对。

```
template <class T> vector<int> prevLess(const vector<T> &a)
{
    // a 是原序列；返回每个位置左侧最近严格更小元素的下标，不存在时为 -1。
    // 改求最近严格更大时，把弹栈条件 >= 改为 <=；若允许相等，则分别用 > 或 <。
    vector<int> stk, ans(a.size(), -1);
    for (int i = 0; i < (int)a.size(); i++)
    {
        while (!stk.empty() && a[stk.back()] >= a[i])
        {
            stk.pop_back();
        }
        if (!stk.empty())
        {
            ans[i] = stk.back();
        }
```

```
        stk.push_back(i);
    }
    return ans;
}

template <class T> vector<T> windowMin(const vector<T> &a, int k)
{
    // a 是原序列，k 是固定窗口长度；返回每个连续窗口的最小值。
    // 改最大值时把维护队尾的 >= 改为 <=；过期下标的队首删除条件不变。
    assert(1 <= k && k <= (int)a.size()); // 调试检查，可删
    deque<int> q;
    vector<T> ans;
    for (int i = 0; i < (int)a.size(); i++)
    {
        while (!q.empty() && q.front() <= i - k)
        {
            q.pop_front();
        }
        while (!q.empty() && a[q.back()] >= a[i])
        {
            q.pop_back();
        }
        q.push_back(i);
        if (i + 1 >= k)
        {
            ans.push_back(a[q.front()]);
        }
    }
    return ans;
}

template <class T> vector<T> windowMax(const vector<T> &a, int k)
{
    // a 是原序列，k 是固定窗口长度；返回每个连续窗口的最大值。
    // 改最小值时把维护队尾的 <= 改为 >=；过期下标的队首删除条件不变。
    assert(1 <= k && k <= (int)a.size()); // 调试检查，可删
    deque<int> q;
    vector<T> ans;
    for (int i = 0; i < (int)a.size(); i++)
    {
        while (!q.empty() && q.front() <= i - k)
        {
            q.pop_front();
        }
        while (!q.empty() && a[q.back()] <= a[i])
        {
            q.pop_back();
        }
        q.push_back(i);
        if (i + 1 >= k)
        {
            ans.push_back(a[q.front()]);
```

```
        }
    }
    return ans;
}
```

`prevLess(a)` 返回各位置左侧最近严格更小元素的下标（无则 -1），`windowMin(a, k)` / `windowMax(a, k)` 返回每个连续窗口的最值。模板弹出满足 $a_{stk.back()} \geq a_i$ 的元素，故留下的是严格更小；要「更小或相等」就改成只在严格大于时弹出。窗口接口要求 $1 \leq k \leq n$。每个下标至多进出一次，时间 $O(n)$、空间 $O(n)$。

2.2.009 滑动窗口与区间最值

连续定长窗口的最值直接复用 2.2.008 单调栈、单调队列 的 `windowMin` / `windowMax`：队列保存值单调且下标仍在窗口内的候选，队首恒为当前答案，固定窗口下每个下标只进出一次，$O(n)$ 时间；除返回数组外额外空间为 $O(k)$，代码不再重复。真正的分界在于查询是否按右端点顺序滑动——一旦不是（窗口长度可变或任意区间查询），单调队列的不变量就失效，应换成 `multiset`（$O(n \log k)$）、对顶双堆、ST 表或线段树。切忌用 `priority_queue` 后再线性删除过期元素；若用堆，必须给元素带上下标并在堆顶循环清理过期项。

2.2.010 最长上升子序列及输出方案

求严格上升子序列并恢复一组原数组下标。核心不变量是 `val[len]` 保存长度为 $len+1$ 的上升子序列所能取得的最小结尾——结尾越小越有利于将来扩展，因此用 `lower_bound` 找到第一个不小于 `a[i]` 的位置替换即可，不损失最优性。`pos[len]` 记录该最小结尾来自哪个原下标，`pre[i]` 记录位置 i 在方案中的前驱，最后从最长长度的结尾沿 `pre` 反向恢复一条合法方案。

```
vector<int> lis(const vector<int> &a)
{
    // a 是原序列；返回一组严格上升子序列的原数组下标。
    vector<int> val, pos, pre(a.size(), -1);
    for (int i = 0; i < (int)a.size(); i++)
    {
        auto p = lower_bound(val.begin(), val.end(), a[i]) - val.begin();
        if (p == (int)val.size())
        {
            val.push_back(a[i]);
            pos.push_back(i);
        }
        else
        {
            val[p] = a[i];
            pos[p] = i;
        }
        if (p)
        {
            pre[i] = pos[p - 1];
        }
```

```
    }
    vector<int> ans;
    for (auto x = pos.empty() ? -1 : pos.back(); x >= 0; x = pre[x])
    {
        ans.push_back(x);
    }
    reverse(ans.begin(), ans.end());
    return ans;
}
```

`lis(a)` 返回一条最长严格上升子序列的原数组下标（不是值），需要打印子序列元素时用 `a[p]`。严格上升用 `lower_bound`，改求最长非降子序列则换成 `upper_bound`。时间 $O(n \log n)$，尾值、结尾下标和前驱数组共 $O(n)$ 空间。

2.2.011 二维 LIS / LCIS

本条是两个只有名字相近、不能互相替代的问题。二维 LIS 在点集里找两维都严格递增的最长链（信封嵌套、二维偏序、宽高同增等）：先按第一维升序、同第一维时按第二维降序排序，再对第二维做一维严格 LIS。同第一维降序是保证严格性的关键——它让同组元素在第二维上不构成上升关系，一维化后就不会误选两个第一维相等的点；若某维允许相等，排序并列规则与 `lower_bound` / `upper_bound` 须一并调整。LCIS 在两个序列中找既是公共子序列、值又严格递增的最长序列，无法用「先 LCS 再 LIS」拼出：令 f_j 为已扫描 a 的前缀中以 b_j 结尾的 LCIS 长度，处理 $x = a_i$ 时从左到右维护所有满足 $b_j < x$ 的 f_j 最大值 `best`，遇到 $b_j = x$ 就令 $f_j \leftarrow \max(f_j, best + 1)$；因为 `best` 只从严格小于 x 的状态更新，同一轮不会重复用到 x 自身。

```
int lis2D(vector<pair<int, int>> a)
{
    // a 保存二维点并按值复制；返回两维都严格增加的最长链长度。
    sort(a.begin(),
         a.end(),
         [](const auto &x, const auto &y)
         {
             if (x.first != y.first)
             {
                 return x.first < y.first;
             }
             return x.second > y.second;
         });
    vector<int> d;
    for (auto [x, y] : a)
    {
        auto it = lower_bound(d.begin(), d.end(), y);
        if (it == d.end())
        {
            d.push_back(y);
        }
        else
        {
            *it = y;
```

```
        }
    }
    return d.size();
}

int lcis(const vector<int> &a, const vector<int> &b)
{
    // a、b 是两个原序列；返回最长公共严格上升子序列长度。
    vector<int> f(b.size());
    for (auto x : a)
    {
        int best = 0;
        for (int j = 0; j < (int)b.size(); j++)
        {
            if (b[j] < x)
            {
                chmax(best, f[j]);
            }
            else if (b[j] == x)
            {
                chmax(f[j], best + 1);
            }
        }
    }
    return f.empty() ? 0 : *max_element(f.begin(), f.end());
}
```

`lis2D(points)` 接收二元组副本、只返回链长而不恢复点链，重复点最多选一个；`lcis(a, b)` 只返回长度，要恢复方案得为每个 f_j 额外记录前驱位置和来自 a 的层次。二维 LIS 为 $O(n \log n)$ 时间、$O(n)$ 空间；两序列长 n, m 时 LCIS 为 $O(nm)$ 时间、$O(m)$ 空间。

2.2.012 最大全 1 子矩形

把 01 矩阵逐行压成直方图：以当前行为底的全 1 矩形，其每列可用高度就是该列往上连续 1 的数量，于是问题化为「直方图中最大矩形」并对每行调用一次。直方图里每根柱子的最大贡献由它左右第一个更矮的柱子界定，单调栈在柱子被弹出时一次性确定其右边界，末尾补一个高度 0 的哨兵负责清空栈中残余柱子。

```
int maxRect(const vector<int> &h)
{
    // h 是非负直方图柱高；返回由 Acc 可表示的最大矩形面积。
    vector<int> stk;
    int ans = 0;
    for (int i = 0; i <= (int)h.size(); i++)
    {
        auto x = i == (int)h.size() ? 0 : h[i];
        while (!stk.empty() && h[stk.back()] >= x)
        {
            auto p = stk.back();
```

```
            stk.pop_back();
            auto l = stk.empty() ? 0 : stk.back() + 1;
            chmax(ans, h[p] * (i - l));
        }
        stk.push_back(i);
    }
    return ans;
}

int maxRect(const vector<vector<int>> &a)
{
    // a 是规则矩阵，非零元素视为 1；返回由 Acc 可表示的最大全 1 子矩形面积。
    if (a.empty() || a[0].empty())
    {
        return 0;
    }
    vector<int> h(a[0].size());
    int ans = 0;
    for (auto &row : a)
    {
        assert(row.size() == h.size()); // 调试检查，可删
        for (int j = 0; j < (int)h.size(); j++)
        {
            h[j] = row[j] ? h[j] + 1 : 0;
        }
        chmax(ans, maxRect(h));
    }
    return ans;
}
```

`maxRect(grid)` 接收规则矩阵、非零元素视为 1，返回最大全 1 子矩形面积；重载 `maxRect(h)` 直接对非负柱高的直方图求最大矩形。调用者须保证柱高、宽度乘积和最终面积能由 `int` 表示。若还要矩形的坐标，须在弹栈时一并记录左右边界和当前行。$n \times m$ 矩阵总时间 $O(nm)$、额外空间 $O(m)$；直方图长度为 m 时为 $O(m)$ 时间和 $O(m)$ 额外空间。

2.2.013 约瑟夫问题

n 人编号 $0 \dots n-1$、每次数到第 k 人出局，求最后幸存者。把规模 i 的答案 $f(i)$ 由 $f(i-1)$ 反推：删掉第一个出局者后余下 $i-1$ 人重新编号，相当于整体偏移 k，于是 $f(i) = (f(i-1)+k) \bmod i$，$f(1) = 0$。只有一个状态和一段短循环，直接内联进解题代码即可，无需封装。

```
// n 是人数，k 是报数步长，ans 是从 0 开始的幸存者编号。
assert(n > 0 && k > 0); // 调试检查，可删
int ans = 0;
if (n >= 2)
{
    int i = 2;
    while (true)
    {
```

```
        int x = k % i;
        ans = ans >= i - x ? ans - (i - x) : ans + x;
        if (i == n)
        {
            break;
        }
        i++;
    }
}
```

循环结束后的 `ans` 是从 0 开始的幸存者编号，题目若从 1 编号则答案加 1。每轮先取 `k % i`，再用比较和减法实现模加，避免 `ans + k % i` 溢出；循环在处理 `i == n` 后先退出再递增，因此 `n`、`k` 取完整正 64 位范围时不会在取模前或末次自增时溢出。该递推只求最后一人，不输出完整删除顺序。$O(n)$ 时间、$O(1)$ 空间。

2.2.014 快速选择 / 线性期望第 k 元素

只要第 k 小而不需完整排序时，划分后只递归含第 k 位的那一侧、丢弃另一侧，期望线性。`nthValue` 直接包了标准库 `nth_element`，经过充分优化，是默认实现；`quickSelect` 手写随机枢轴 + 三路划分，把等于枢轴的元素聚在中间，对高重复数据更稳，供不能用 STL 或需定制状态时备用。

```
template <class T> T nthValue(vector<T> a, int k)
{
    // a 是按值复制的数组，k 是从 0 开始的排名；返回第 k 小值。
    assert(0 <= k && k < (int)a.size()); // 调试检查，可删
    nth_element(a.begin(), a.begin() + k, a.end());
    return a[k];
}

template <class T, class Rng> T quickSelect(vector<T> a, int k, Rng &rng)
{
    // a 是按值复制的数组，k 是从 0 开始的排名，rng 是随机源；返回第 k 小值。
    assert(0 <= k && k < (int)a.size()); // 调试检查，可删
    int l = 0, r = a.size();
    while (r - l > 1)
    {
        auto p = a[uniform_int_distribution<int>(l, r - 1)(rng)];
        auto m1 = partition(a.begin() + l,
                            a.begin() + r,
                            [&](const T &x)
                            {
                                return x < p;
                            });
        auto m2 = partition(m1,
                            a.begin() + r,
                            [&](const T &x)
                            {
                                return !(p < x);
                            });
```

```
        auto x = m1 - a.begin();
        auto y = m2 - a.begin();
        if (k < x)
        {
            r = x;
        }
        else if (k >= y)
        {
            l = y;
        }
        else
        {
            return p;
        }
    }
    return a[l];
}
```

两者的 `k` 都从 0 开始：`nthValue(a, k)` 是主版本，`quickSelect(a, k, rng)` 是随机备用。本机基准（100 组、每组十万元素、7 轮中位数）`nth_element=50ms`、手写三路 `61ms`，故默认 `nth_element`。标准库对 `nth_element` 保证线性期望比较次数，手写版期望 $O(n)$、最坏 $O(n^2)$，两者都按值复制输入需 $O(n)$ 空间。

2.2.015 折半搜索

折半搜索把 n 个元素拆成前后两半，各自枚举 $2^{\lceil n/2 \rceil}$ 个子集和，再对其中一半排序，用另一半逐个二分配对，从而把 $O(2^n)$ 的子集枚举降到 $O(2^{n/2}n)$。适用于 $n \le 40$ 左右、值域太大不能做背包的子集和问题；元素可以为负。

`bound` 求不超过 `target` 的最大子集和（`ceil=true` 时求不小于 `target` 的最小子集和）：对前半的每个和 x，在排好序的后半中二分找最接近 $target - x$ 的一侧。`closest` 取两者中离 `target` 更近的一个。`fourSum` 是同一思想的四数组版本：把 $a_0 + a_1$ 的 N^2 个和排序，再对 $a_2 + a_3$ 的每个和二分查找差值。

```
namespace mitm
{
struct State
{
    __int128 sum;            // sum 是该子集的元素和。
    unsigned long long mask; // mask 的第 i 位表示半边内第 i 个元素是否被选。
};

vector<State> sums(const vector<int> &a, int l, int r)
{
    // a 是原数组，[l,r) 是一半的下标范围；返回这一半全部 2^(r-l) 个子集的和与选择掩码。
    vector<State> v{{0, 0}};
    for (int i = l; i < r; i++)
    {
        int n = v.size();
        for (int j = 0; j < n; j++)
```

```
        {
            v.push_back({v[j].sum + a[i], v[j].mask | (1ULL << (i - l))});
        }
    }
    return v;
}

struct Result
{
    __int128 sum;         // sum 是所选子集的和。
    vector<int> indices; // indices 是所选元素的原下标，升序。
};

optional<Result> bound(const vector<int> &a, __int128 target, bool ceil = false)
{
    // a 至多 46 个元素且可为负；ceil=false 求不超过 target 的最大子集和，
    // ceil=true 求不小于 target 的最小子集和；返回和与下标，不存在时返回空。
    assert(a.size() <= 46); // 调试检查，可删。
    int k = a.size() / 2;
    auto x = sums(a, 0, k), y = sums(a, k, a.size());
    sort(y.begin(), y.end(), [](const State &u, const State &v) { return u.sum < v.sum; });
    optional<Result> ans;
    unsigned long long mx = 0, my = 0;
    for (auto u : x)
    {
        // 在后半找最接近 target-u.sum 的和：it 是第一个不小于它的位置。
        auto it = lower_bound(y.begin(), y.end(), target - u.sum,
                              [](const State &v, __int128 s) { return v.sum < s; });
        if (!ceil)
        {
            if (it == y.end() || it->sum ≠ target - u.sum)
            {
                if (it == y.begin())
                {
                    continue;
                }
                --it;
            }
        }
        else if (it == y.end())
        {
            continue;
        }
        __int128 s = u.sum + it->sum;
        if (!ans || (ceil ? s < ans->sum : s > ans->sum))
        {
            ans = Result{s, {}};
            mx = u.mask;
            my = it->mask;
        }
    }
    if (ans)
```

```
    {
        for (int i = 0; i < (int)a.size(); i++)
        {
            if (i < k ? (mx >> i & 1) : (my >> (i - k) & 1))
            {
                ans->indices.push_back(i);
            }
        }
    }
    return ans;
}

Result closest(const vector<int> &a, __int128 target)
{
    // a 至多 46 个元素；返回与 target 绝对差最小的子集和及下标，两侧等距时取较小的和。
    auto lo = bound(a, target), hi = bound(a, target, true);
    if (!lo)
    {
        return *hi;
    }
    if (!hi || target - lo->sum <= hi->sum - target)
    {
        return *lo;
    }
    return *hi;
}

optional<array<int, 4>> fourSum(const array<vector<int>, 4> &a, int target)
{
    // a 是四个独立数组；返回 {i,j,k,l} 使 a[0][i]+a[1][j]+a[2][k]+a[3][l]=target，不存在时返回空。
    // v 的三个分量依次是 a[0][i]+a[1][j]、i、j。
    vector<tuple<__int128, int, int>> v;
    for (int i = 0; i < (int)a[0].size(); i++)
    {
        for (int j = 0; j < (int)a[1].size(); j++)
        {
            v.emplace_back((__int128)a[0][i] + a[1][j], i, j);
        }
    }
    sort(v.begin(), v.end());
    for (int i = 0; i < (int)a[2].size(); i++)
    {
        for (int j = 0; j < (int)a[3].size(); j++)
        {
            __int128 need = (__int128)target - a[2][i] - a[3][j];
            auto it = lower_bound(v.begin(), v.end(), tuple<__int128, int, int>{need, -1, -1});
            if (it != v.end() && get<0>(*it) == need)
            {
                return array<int, 4>{get<1>(*it), get<2>(*it), i, j};
            }
        }
    }
```

```
    return nullopt;
}
}
```

`bound`、`closest` 要求 $n \le 46$（掩码装入 64 位），返回的 `indices` 是所选元素的原下标、升序；空集也是合法子集，所以 `closest` 总有解，`bound` 只在所有子集和都在 `target` 另一侧时返回空。时间 $O(2^{n/2}n)$；每一半保存 $2^{\lceil n/2 \rceil}$ 个 32 字节的 `State`，$n = 40$ 时两半约 64 MiB，$n = 46$ 时超过 500 MiB，要先核对内存限制。`fourSum` 的时间 $O(N^2 \log N)$、空间 $O(N^2)$，N 是单个数组长度。

只需判定某个和能否取到且值域不大时用 12.1.001 01 背包或 12.1.010 布尔母函数 / 有界子集和可达性；计数版见 10.2.022 计数版子集和生成函数。下面核对固定数组上的三种查询：

```
signed main()
{
    vector<int> a{8, -3, 5, 12, -7, 4};
    assert(mitm::bound(a, 10)->sum == 10);        // 取 8-3+5。
    assert(mitm::bound(a, 11, true)->sum == 11);
    auto c = mitm::closest(a, 100);               // 所有正数之和 29 最接近 100。
    assert(c.sum == 29 && c.indices == vector<int>({0, 2, 3, 5}));
    auto f = mitm::fourSum({vector<int>{1, 5}, {2, 7}, {3}, {4, 9}}, 20);
    assert(f && *f == (array<int, 4>{0, 1, 0, 1})); // 取 1+7+3+9。
}
```

2.3 莫队与离线查询

2.3.001 普通莫队

当单个端点的加入/删除都能 $O(1)$ 维护、且询问允许离线重排时，把大量区间查询降到约 $O(n\sqrt{q})$ 次状态更新。按左端点分块排序，使同块内右端点大体单调移动，奇偶块内右端点反向排序进一步减少折返。调度函数本身只负责移动当前 $[l, r)$，真正的答案状态交由四个端点回调（`addL` / `addR` / `delL` / `delR`）维护，因此保持为单函数而非封装成类。

```
// MoQuery[0..2] 依次表示左端点、右端点、答案编号。
using MoQuery = array<int, 3>;

template <class Ans, class AddL, class AddR, class DelL, class DelR, class Get>
vector<Ans> mo(int n, vector<MoQuery> qs, AddL addL, AddR addR, DelL delL, DelR delR, Get get)
{
    // n 是数组长度，qs 是询问；四个端点回调维护状态，get 返回当前答案。
    auto z = max<int>(1, (int)(n / sqrtl(max<int>(1, qs.size()))));
    sort(qs.begin(),
         qs.end(),
         [&](const auto &a, const auto &b)
         {
             auto x = a[0] / z, y = b[0] / z;
             if (x != y)
             {
                 return x < y;
```

```
            }
            return x & 1 ? a[1] > b[1] : a[1] < b[1];
        });
    vector<Ans> ans(qs.size());
    int l = 0, r = 0;
    for (auto q : qs)
    {
        while (l > q[0])
        {
            addL(--l);
        }
        while (r < q[1])
        {
            addR(r++);
        }
        while (l < q[0])
        {
            delL(l++);
        }
        while (r > q[1])
        {
            delR(--r);
        }
        ans[q[2]] = get();
    }
    return ans;
}
```

用 `auto ans = mo<int>(n, qs, addL, addR, delL, delR, get)` 调用，`MoQuery={l,r,id}`。询问使用 `[l, r)`，`id` 必须覆盖 $[0, q)$。左右端操作不对称时分别实现四个回调，对称时可以对 `add/del` 传同一个函数。排序为 $O(q \log q)$；块长为 B 时端点移动上界为 $O\left(\frac{n^2}{B} + qB\right)$，代码取 $B \approx \frac{n}{\sqrt{q}}$，得到 $O(n\sqrt{q} + q \log q)$ 时间；除维护状态外，调度额外空间为 $O(q)$。

当区间端点位于较大坐标范围、又不想为块长调参时，可以按二维端点的 Hilbert 曲线排序。递归把每个 $2^k \times 2^k$ 正方形分成四块，并在相邻块之间旋转方向，使相邻询问的端点通常同时较近；这只改变访问顺序，不改变莫队状态。`moHilbert` 仍使用半开区间和单个 `add/del` 回调，空区间也允许；实现要求数组长度不超过 2^{31}，以便把二维键压进无符号 64 位。

```
unsigned long long hilbertOrder(int x, int y, int power, int rotate = 0)
{
    // x、y 是 [0,2^power) 内的坐标，power 不超过 31；返回 Hilbert 序键。
    if (!power)
    {
        return 0;
    }
    int h = 1LL << (power - 1);
    int seg = x < h ? (y < h ? 0 : 3) : (y < h ? 1 : 2);
    seg = (seg + rotate) & 3;
    static constexpr int delta[4] = {3, 0, 0, 1};
    int nextRot = (rotate + delta[seg]) & 3;
    unsigned long long z = 1ULL << (2 * power - 2);
```

```
    unsigned long long sub = hilbertOrder(x & (h - 1), y & (h - 1), power - 1, nextRot);
    return (unsigned long long)seg * z + (seg == 1 || seg == 2 ? sub : z - sub - 1);
}

template <class Ans, class Add, class Del, class Get>
vector<Ans> moHilbert(int n, vector<MoQuery> qs, Add add, Del del, Get get)
{
    // n 是数组长度，qs 是半开区间询问；add/del 接收下标，get 返回当前答案。
    int power = 0;
    while ((1LL << power) < max<int>(1, n))
    {
        power++;
    }
    assert(power <= 31);
    sort(qs.begin(), qs.end(), [&](const MoQuery &a, const MoQuery &b)
    {
        int ax = min<int>(a[0], max<int>(0, n - 1));
        int ay = min<int>(max<int>(0, a[1] - 1), max<int>(0, n - 1));
        int bx = min<int>(b[0], max<int>(0, n - 1));
        int by = min<int>(max<int>(0, b[1] - 1), max<int>(0, n - 1));
        return hilbertOrder(ax, ay, power) < hilbertOrder(bx, by, power);
    });
    vector<Ans> ans(qs.size());
    int l = 0, r = 0;
    for (auto q : qs)
    {
        while (l > q[0])
        {
            add(--l);
        }
        while (r < q[1])
        {
            add(r++);
        }
        while (l < q[0])
        {
            del(l++);
        }
        while (r > q[1])
        {
            del(--r);
        }
        ans[q[2]] = get();
    }
    return ans;
}
```

`moHilbert(n,qs,add,del,get)` 与普通 `mo` 的答案接口相同，`MoQuery={l,r,id}` 且 `id` 从 0 开始。排序为 $O(q \log q)$，端点移动次数取决于 Hilbert 曲线的局部性，通常不需要显式块长；它不适合带修改时间轴，带修改仍使用 2.3.002 带修改莫队。

下面是普通莫队、Hilbert 莫队的最小可运行调用：

```cpp
signed main()
{
    vector<int> a{1, 2, 1, 3};
    vector<MoQuery> qs{{0, 3, 0}, {1, 4, 1}};
    int cur = 0;
    auto add = [&](int p) { cur += a[p]; };
    auto del = [&](int p) { cur -= a[p]; };
    auto get = [&]() { return cur; };
    assert((mo<int>(4, qs, add, add, del, del, get) == vector<int>{4, 6}));
    cur = 0;
    assert((moHilbert<int>(4, qs, add, del, get) == vector<int>{4, 6}));
}
```

2.3.002 带修改莫队

当询问之间夹有单点修改时，给莫队再加一维时间轴 `t`，让修改既能向前应用又能向后撤销。每个询问记录它之前已发生的修改数，块长取 $n^{2/3}$，排序时同时控制左端、右端和时间指针三者的移动。应用或撤销一次修改时，如果修改位置正落在当前区间内，必须先移除旧贡献、再改值、再加入新贡献，漏掉任一步都会让状态与时间轴不一致。

```cpp
// ModifyQuery[0..3] 依次表示左端点、右端点、答案编号、修改时间。
using ModifyQuery = array<int, 4>;

template <class Ans, class Add, class Del, class Change, class Get>
vector<Ans> moModify(int n, vector<ModifyQuery> qs, Add add, Del del, Change change, Get get)
{
    // n 是数组长度，qs 含修改时间；add/del 维护端点，change(p,forward,l,r) 切换第 p 次修改，get 返回答案。
    auto z = max<int>(1, (int)powl(max<int>(1, n), 2.0L / 3));
    sort(qs.begin(),
         qs.end(),
         [&](const auto &a, const auto &b)
         {
             auto al = a[0] / z, bl = b[0] / z;
             if (al != bl)
             {
                 return al < bl;
             }
             auto ar = a[1] / z, br = b[1] / z;
             if (ar != br)
             {
                 return al & 1 ? ar > br : ar < br;
             }
             return ar & 1 ? a[3] > b[3] : a[3] < b[3];
         });
    vector<Ans> ans(qs.size());
    int l = 0, r = 0, t = 0;
    for (auto q : qs)
    {
        while (t < q[3])
```

```
        {
            change(t++, true, l, r);
        }
        while (t > q[3])
        {
            change(--t, false, l, r);
        }
        while (l > q[0])
        {
            add(--l);
        }
        while (r < q[1])
        {
            add(r++);
        }
        while (l < q[0])
        {
            del(l++);
        }
        while (r > q[1])
        {
            del(--r);
        }
        ans[q[2]] = get();
    }
    return ans;
}
```

调用 `moModify<int>(n, qs, add, del, change, get)`，`ModifyQuery={l,r,id,t}` 中 `id` 须覆盖 $[0,q)$，`t` 是该询问前的修改次数且满足 $0 \le t \le c$。`change(p, forward, l, r)` 中 `p` 是从 0 开始的修改编号；修改位置当前位于 $[l,r)$ 时，回调必须同步删除旧贡献、写入新值、加入新贡献，反向时恢复旧值。块长取 $n^{2/3}$，在 n、q 和修改数同阶时状态移动为 $O\left(n^{5/3}\right)$，排序为 $O(q\log q)$，空间为 $O(n+q+c)$（c 是修改数）。

调用示例（区间和、一次修改）：

```
signed main()
{
    vector<int> b{1, 2, 3};
    vector<ModifyQuery> mq{{0, 3, 0, 1}};
    int sum = 0;
    auto addM = [&](int p) { sum += b[p]; };
    auto delM = [&](int p) { sum -= b[p]; };
    auto change = [&](int p, bool forward, int l, int r)
    {
        int pos = 1, old = 2, now = 5;
        if (p != 0) return;
        if (l <= pos && pos < r) sum += forward ? now - old : old - now;
        b[pos] = forward ? now : old;
    };
    auto getM = [&]() { return sum; };
```

```
    assert(moModify<int>(3, mq, addM, delM, change, getM)[0] == 9);
}
```

2.3.003 回滚莫队

当状态只能廉价加入、却难以删除时（如维护区间最大值、可撤销并查集），改用回滚莫队：右端在同一左端块内按升序单调扩展并跨询问保留，左端只对当前询问临时补入、回答后立刻撤销到快照；询问区间完全落在一个块内时直接临时加入再撤销。算法本身仍只是一次离线调度，历史栈等可回滚状态由调用者维护。

```
// 依赖 2.3.001 普通莫队
template <class Ans, class Add, class Snap, class Undo, class Get>
vector<Ans> rollbackMo(int n, vector<MoQuery> qs, Add add, Snap snap, Undo undo, Get get)
{
    // n 是数组长度，qs 是询问；add 加入位置，snap/undo 管理快照，get 返回当前答案。
    auto z = max<int>(1, (int)sqrtl(max<int>(1, n)));
    sort(qs.begin(),
         qs.end(),
         [&](const auto &a, const auto &b)
         {
             auto x = a[0] / z, y = b[0] / z;
             return x != y ? x < y : a[1] < b[1];
         });
    vector<Ans> ans(qs.size());
    auto empty = snap();
    for (int i = 0; i < (int)qs.size();)
    {
        auto k = qs[i][0] / z;
        auto j = i;
        while (j < (int)qs.size() && qs[j][0] / z == k)
        {
            j++;
        }
        auto mid = min<int>(n, (k + 1) * z);
        auto r = mid;
        for (auto p = i; p < j; p++)
        {
            auto q = qs[p];
            if (q[1] <= mid)
            {
                auto s = snap();
                for (auto x = q[0]; x < q[1]; x++)
                {
                    add(x);
                }
                ans[q[2]] = get();
                undo(s);
                continue;
            }
            while (r < q[1])
```

```
            {
                add(r++);
            }
            auto s = snap();
            for (auto x = mid - 1; x >= q[0]; x--)
            {
                add(x);
            }
            ans[q[2]] = get();
            undo(s);
        }
        undo(empty);
        i = j;
    }
    return ans;
}
```

调用 `rollbackMo<int>(n, qs, add, snap, undo, get)`，`MoQuery={l,r,id}`；`snap()` 返回历史栈大小等快照，`undo(s)` 必须精确恢复到该状态，块内短询问在临时加入后回滚。块长为 B 时右端扩展为 $O\left(\frac{n^2}{B}\right)$，短询问与左端临时扩展为 $O(qB)$，取 $B \approx \sqrt{n}$ 得 $O((n+q)\sqrt{n})$，排序为 $O(q \log q)$；除调用者保存的回滚历史外，调度额外空间为 $O(q)$。

回调可以用一个历史栈实现可撤销的区间和：

```
signed main()
{
    vector<int> c{1, 2, 3};
    vector<MoQuery> rq{{0, 2, 0}};
    int val = 0;
    vector<int> hist;
    auto addR = [&](int p) { hist.push_back(val); val += c[p]; };
    auto snap = [&]() { return (int)hist.size(); };
    auto undo = [&](int s)
    {
        while ((int)hist.size() > s)
        {
            val = hist.back();
            hist.pop_back();
        }
    };
    auto getR = [&]() { return val; };
    assert(rollbackMo<int>(3, rq, addR, snap, undo, getR)[0] == 3);
}
```

2.3.004 二次离线莫队

当区间答案是点对贡献之和、普通莫队每移动一个端点仍要扫描整段才能算出新增贡献时，用二次离线把这份代价均摊掉。前提是答案能写成

$$F(l,r)=\sum_{l\le i<j<r} w(a_i,a_j),$$

并且能预处理或离线回答前缀贡献 $G(p,x)=\sum_{0\le i<x} w(a_i,a_p)$——此时加入右端点 p 的增量是 $G(p,p)-G(p,l)$，加入左端点同理。于是第一次莫队并不立即计算昂贵的端点贡献，而是把它拆成若干带正负号的「端点对某段前缀」的 G 事件；第二次沿前缀位置从左到右扫描，用值域计数一次性回答整批事件。下面给出的异或点对是一个完整问题实例（统计区间内 $\mathrm{popcount}(a_i\oplus a_j)=k$ 的无序点对数），其中的值域计数只对这一贡献函数成立，不能当成对任意 w 都通用的类。

```cpp
vector<int> xorPairs(const vector<int> &a, const vector<pair<int, int>> &qs, int k, int bits)
{
    // a 是 bits 位非负整数，qs 是半开区间，k 是目标异或位数；返回各区间点对数。
    assert(0 <= bits && bits <= 20); // 调试检查，可删
    auto n = (int)a.size();
    auto lim = 1LL << bits;
    for (auto x : a)
    {
        assert(0 <= x && x < lim); // 调试检查，可删
        (void)x;
    }
    vector<int> ans(qs.size());
    if (k < 0 || k > bits)
    {
        return ans;
    }

    // Query[0..2] 依次表示从 1 开始的左端点、右端点、原询问编号。
    using Query = array<int, 3>;
    vector<Query> ord;
    for (int id = 0; id < (int)qs.size(); id++)
    {
        auto [l, r] = qs[id];
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l < r)
        {
            ord.push_back({l + 1, r, id});
        }
    }
    if (ord.empty())
    {
        return ans;
    }

    auto z = max<int>(1, (int)(n / sqrtl((long double)ord.size())));
    sort(ord.begin(),
         ord.end(),
         [&](const auto &x, const auto &y)
         {
             auto bx = (x[0] - 1) / z, by = (y[0] - 1) / z;
             if (bx != by)
```

```
            {
                return bx < by;
            }
            return bx & 1 ? x[1] > y[1] : x[1] < y[1];
        });

    vector<int> masks;
    for (int s = 0; s < lim; s++)
    {
        if (popcount((unsigned long long)s) == k)
        {
            masks.push_back(s);
        }
    }
    vector<int> v(n + 1), cnt(lim), pre(n + 1);
    copy(a.begin(), a.end(), v.begin() + 1);
    for (int i = 1; i <= n; i++)
    {
        pre[i] = pre[i - 1];
        for (auto s : masks)
        {
            pre[i] += cnt[v[i] ^ s];
        }
        cnt[v[i]]++;
    }

    // Event[0..3] 依次表示左端点、右端点、贡献符号、排序后询问编号。
    using Event = array<int, 4>;
    vector<vector<Event>> ev(n + 1);
    vector<int> cur(ord.size());
    int l = 1, r = 0;
    for (int id = 0; id < (int)ord.size(); id++)
    {
        auto q = ord[id];
        cur[id] = pre[q[1]] - pre[r] + pre[q[0] - 1] - pre[l - 1];
        if (!k)
        {
            cur[id] += q[0] - l;
        }
        if (r < q[1])
        {
            ev[l - 1].push_back({r + 1, q[1], -1, id});
            r = q[1];
        }
        if (l > q[0])
        {
            ev[r].push_back({q[0], l - 1, 1, id});
            l = q[0];
        }
        if (r > q[1])
        {
            ev[l - 1].push_back({q[1] + 1, r, 1, id});
```

```
            r = q[1];
        }
        if (l < q[0])
        {
            ev[r].push_back({l, q[0] - 1, -1, id});
            l = q[0];
        }
    }

    fill(cnt.begin(), cnt.end(), 0);
    for (int i = 1; i <= n; i++)
    {
        for (auto s : masks)
        {
            cnt[v[i] ^ s]++;
        }
        for (auto e : ev[i])
        {
            int val = 0;
            for (int j = e[0]; j <= e[1]; j++)
            {
                val += cnt[v[j]];
            }
            cur[e[3]] += e[2] * val;
        }
    }
    for (int i = 1; i < (int)cur.size(); i++)
    {
        cur[i] += cur[i - 1];
    }
    for (int i = 0; i < (int)ord.size(); i++)
    {
        ans[ord[i][2]] = cur[i];
    }
    return ans;
}
```

用 `auto ans = xorPairs(a, qs, k, bits)` 调用，`qs` 是若干 $[l,r)$，返回每个区间内满足 $\mathrm{popcount}(a_i \oplus a_j) = k$ 的无序点对数，要求 $0 \le a_i < 2^{bits}$ 且 $bits \le 20$。令 $C = \binom{bits}{k}$、块长为 B，本实例时间为 $O\left(2^{bits} + nC + \frac{n^2}{B} + qB + q\log q\right)$，取 $B \approx \frac{n}{\sqrt{q}}$ 时莫队部分为 $O(n\sqrt{q})$，空间为 $O(2^{bits} + n + q + C)$。二次离线只在前缀贡献 G 能显著快于逐段扫描时才划算；区间不同数、区间和这类端点本就 $O(1)$ 的问题应继续用普通莫队，而换一个贡献函数 w 都必须重新推导并对拍。下面用四个两位数同时检查整段、子段与空区间：

```
signed main()
{
    vector<int> a{0, 1, 2, 3};
    vector<pair<int, int>> q{{0, 4}, {1, 4}, {2, 2}};
    assert((xorPairs(a, q, 1, 2) == vector<int>{4, 2, 0}));
}
```

2.3.005 区间不同数统计

静态数组上的区间不同数，比起普通莫队有更快且确定性更强的做法：把询问按右端点离线，扫描右端点 `r` 时用树状数组只保留每个值的最后一次出现——遇到新的 `a[r-1]` 就把它旧的最后出现位置减 1、当前位置加 1，于是 Fenwick 中每个不同值恰好留下一个标记，区间 $[l, r)$ 的标记和即不同值数量。这依赖查询静态，一旦夹杂修改，「最后出现位置」会整体漂移而失效。这里先离散化值域，最后位置、Fenwick 和查询桶都只存下标与频次，打印版按普通 `int` 处理；`Fenwick` 的查询接口不直接复用是因为它的节点类型必须保留一般的 64 位和，而此处计数上界为数组长度。此处只给新增的 `distinct`：

```
vector<int> distinct(const vector<int> &a, const vector<pair<int, int>> &qs)
{
    // a 是静态数组，qs 是半开区间；返回每个区间中的不同值数量。
    // 只有下标、排名和频次的热点数据使用普通 int，避免无意义地扩大工作集。
    vector<int> val = a;
    sort(val.begin(), val.end());
    val.erase(unique(val.begin(), val.end()), val.end());
    vector<int> rank(a.size());
    for (int i = 0; i < (int)a.size(); i++)
    {
        rank[i] = (int)(lower_bound(val.begin(), val.end(), a[i]) - val.begin());
    }
    // Ask[0..1] 依次表示左端点与答案编号，按右端点分桶。
    using Ask = array<int, 2>;
    vector<vector<Ask>> ask(a.size() + 1);
    for (int i = 0; i < (int)qs.size(); i++)
    {
        auto [l, r] = qs[i];
        assert(0 <= l && l <= r && r <= (int)a.size()); // 调试检查，可删
        assert(r <= numeric_limits<int>::max());    // 调试检查，可删
        ask[r].push_back({(int)l, (int)i});
    }
    vector<int> bit(a.size() + 1), last(val.size(), -1);
    auto add = [&](int p, int delta)
    {
        // p 是从 0 开始的位置，delta 是出现位置标记的变化。
        for (p++; p < (int)bit.size(); p += p & -p)
        {
            bit[p] += (int)delta;
        }
    };
    auto sum = [&](int p)
    {
        // p 是半开前缀右端点；返回 [0,p) 的最后出现位置标记数。
        int ans = 0;
        for (; p; p -= p & -p)
        {
            ans += bit[p];
        }
        return ans;
    };
    vector<int> ans(qs.size());
```

```
    for (int r = 0; r <= (int)a.size(); r++)
    {
        if (r)
        {
            auto x = rank[r - 1];
            if (last[x] >= 0)
            {
                add(last[x], -1);
            }
            add(r - 1, 1);
            last[x] = r - 1;
        }
        for (auto [l, id] : ask[r])
        {
            ans[id] = sum(r) - sum(l);
        }
    }
    return ans;
}
```

用 `auto ans = distinct(a, qs)` 调用，`qs` 里每个区间为 $[l,r)$，允许空区间。数组值经防卡哈希记录最后位置，需要稳定最坏复杂度时可先离散化再把 `last` 换成 `vector<int>`。每个位置至多执行两次 Fenwick 修改、每个询问执行一次区间和，时间为 $O((n+q)\log n)$，空间为 $O(n+q)$。

2.3.006 区间第 K 小整体二分

在静态数组上批量回答 $[l,r)$ 第 k 小，用整体二分套树状数组即可，无需建可持久化线段树：把元素按离散值域中点分到左右两组，Fenwick 只暂存左半值在各位置的出现次数，每个询问用区间计数 `c` 判断第 k 小落在哪组——落右组时把 k 扣掉 `c`，每层递归前务必撤销本层的 Fenwick 修改。这套代码与 2.2.004 整体二分 给出的 `rangeKth` 完全一致（那里连同离散化 `Discrete`、`Fenwick` 一并给出并附固定样例），此处不再重复。`KthQuery={l,r,k}` 三维为半开区间左右端点与从 1 开始的排名，须满足 $1 \le k \le r-l$，函数只改询问副本中的 k、不动调用者数据；时间 $O((n+q)\log V \log n)$、空间 $O(n+q+V)$。若有在线强制查询，改用主席树或小波矩阵。

2.3.007 树上莫队

欧拉括号序把树上路径查询降到序列区间查询：DFS 时进入结点 u 追加一次记录 st_u、离开时再追加一次记录 ed_u，得到长 $2n$ 的序列 seq，每个结点恰出现两次。对路径 (u,v)（先令 $st_u \le st_v$），设 $w=\mathrm{lca}(u,v)$：

- 若 $w=u$（u 是 v 的祖先），路径点集恰为 $seq[st_u..st_v]$ 中「出现奇数次」的那些结点；
- 否则取区间 $seq[ed_u..st_v]$，其奇数次结点为路径去掉 w 后的部分，需额外单独加入 w。

因此把每个询问转成一段区间（外加一个可选的 w），用普通莫队（2.3.001 普通莫队）离线跑，`work(u)` 按 u 当前出现次数的奇偶切换其贡献（偶 → 奇计入、奇 → 偶移除）。LCA 用倍增（6.1.002 最近公共祖先：倍增）。下例统计路径上不同权值个数，权值需先离散化到 $[0,n)$。

```cpp
struct TreeMo
{
    int n = 0;                      // n 是树的顶点数。
    int lg = 1;                     // lg 是倍增祖先表的层数。
    int blk = 1;                    // blk 是当前询问排序的块长。
    vector<vector<int>> g;          // g 是树的邻接表。
    vector<int> col;                // col[u] 是顶点 u 离散到 [0,n) 的权值。
    vector<int> st;                 // st[u] 是顶点 u 进入欧拉序的位置。
    vector<int> ed;                 // ed[u] 是顶点 u 离开欧拉序的位置。
    vector<int> seq;                // seq 是每个顶点进出各出现一次的欧拉序。
    vector<int> dep;                // dep[u] 是顶点 u 到当前根的边数。
    vector<vector<int>> up;         // up[k][u] 是顶点 u 的第 2^k 级祖先。
    vector<array<int, 4>> qs;       // 每项是 {欧拉左端、欧拉右端、额外 LCA、答案编号}。

    explicit TreeMo(int m) : n(m), g(m), col(m), st(m), ed(m), dep(m)
    {
        // m 是非空树的顶点数。
        assert(m > 0); // 调试检查，可删
    }

    void addTreeEdge(int u, int v)
    {
        // u、v 是无向树边的两个端点；调用后把边加入邻接表。
        assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void build(int root = 0)
    {
        // root 是本次选定的根；重建欧拉序和祖先表，并清空依赖旧序的询问。
        assert(0 <= root && root < n); // 调试检查，可删
        lg = 1;
        while ((1ULL << lg) < (unsigned long long)n)
        {
            lg++;
        }
        up.assign(lg, vector<int>(n, root));
        fill(dep.begin(), dep.end(), 0);
        seq.clear();
        seq.reserve((size_t)n * 2);
        qs.clear();

        st[root] = 0;
        seq.push_back(root);
        auto dfs = [&](auto &&self, int u, int p) -> void
        {
            for (auto v : g[u])
            {
                if (v == p)
                {
                    continue;
```

```
                }
                up[0][v] = u;
                dep[v] = dep[u] + 1;
                st[v] = seq.size();
                seq.push_back(v);
                self(self, v, u);
            }
            ed[u] = seq.size();
            seq.push_back(u);
        };
        dfs(dfs, root, -1);
        for (int k = 1; k < lg; k++)
        {
            for (int u = 0; u < n; u++)
            {
                up[k][u] = up[k - 1][up[k - 1][u]];
            }
        }
    }

    int lca(int u, int v) const
    {
        // u、v 是树上顶点；返回它们在当前根下的最近公共祖先。
        assert(!up.empty() && 0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删
        if (dep[u] < dep[v])
        {
            swap(u, v);
        }
        auto d = dep[u] - dep[v];
        for (int k = 0; k < lg; k++)
        {
            if (d >> k & 1)
            {
                u = up[k][u];
            }
        }
        if (u == v)
        {
            return u;
        }
        for (int k = lg - 1; k >= 0; k--)
        {
            if (up[k][u] != up[k][v])
            {
                u = up[k][u];
                v = up[k][v];
            }
        }
        return up[0][u];
    }

    void addQuery(int u, int v)
```

```
    {
        // u、v 是路径两端；把该路径转成半开欧拉区间并按加入顺序编号。
        assert(!up.empty() && 0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删
        if (st[u] > st[v])
        {
            swap(u, v);
        }
        auto w = lca(u, v);
        auto id = (int)qs.size();
        if (w == u)
        {
            qs.push_back({st[u], st[v] + 1, -1, id});
        }
        else
        {
            qs.push_back({ed[u], st[v] + 1, w, id});
        }
    }

    vector<int> solve()
    {
        // 返回每条已加入路径上的不同权值数，顺序与 addQuery 一致。
        if (qs.empty())
        {
            return {};
        }
        for (auto x : col)
        {
            assert(0 <= x && x < n); // 调试检查，可删
        }
        auto m = (int)qs.size();
        blk = max<int>(1, (int)((long double)seq.size() / sqrtl(m)));
        sort(qs.begin(),
             qs.end(),
             [&](const auto &a, const auto &b)
             {
                 auto x = a[0] / blk;
                 auto y = b[0] / blk;
                 if (x != y)
                 {
                     return x < y;
                 }
                 return x & 1 ? a[1] > b[1] : a[1] < b[1];
             });
        vector<int> cnt(n);
        vector<int> used(n);
        vector<int> ans(m);
        int cur = 0;
        auto work = [&](int u)
        {
            // u 是本次切换的顶点；奇数次出现计入路径，偶数次出现移出路径。
            used[u] ^= 1;
```

```
157             if (used[u])
158             {
159                 if (cnt[col[u]]++ == 0)
160                 {
161                     cur++;
162                 }
163             }
164             else
165             {
166                 if (--cnt[col[u]] == 0)
167                 {
168                     cur--;
169                 }
170             }
171         };
172         int l = 0;
173         int r = 0;
174         for (auto q : qs)
175         {
176             auto ql = q[0];
177             auto qr = q[1];
178             auto w = q[2];
179             auto id = q[3];
180             while (l > ql)
181             {
182                 work(seq[--l]);
183             }
184             while (r < qr)
185             {
186                 work(seq[r++]);
187             }
188             while (l < ql)
189             {
190                 work(seq[l++]);
191             }
192             while (r > qr)
193             {
194                 work(seq[--r]);
195             }
196             if (w >= 0)
197             {
198                 work(w);
199             }
200             ans[id] = cur;
201             if (w >= 0)
202             {
203                 work(w);
204             }
205         }
206         return ans;
207     }
208 };
```

先用 addTreeEdge 连成一棵树，填好 col 后调用 build(root)，再用 addQuery(u,v) 挂路径询问；solve() 返回按加入顺序排列的不同权值数。再次 build 会重置深度、欧拉序、祖先表，并清空基于旧根编码的询问。带修改版再增加时间轴，思路同 2.3.002 带修改莫队。欧拉序长 $2n$，代码取块长约 $2n/\sqrt{q}$，排序和状态切换共 $O(n\sqrt{q}+q\log q)$ 时间，祖先表与其余状态共 $O(n\log n+q)$ 空间。

下面的四点树覆盖跨分支路径、祖孙路径、单点路径与根路径，并检查换根重建会清空旧询问：

```
signed main()
{
    TreeMo tr(4);
    tr.addTreeEdge(0, 1);
    tr.addTreeEdge(0, 2);
    tr.addTreeEdge(1, 3);
    tr.col = {1, 2, 1, 3};
    tr.build(0);
    for (auto [u, v] : vector<pair<int, int>>{{3, 2}, {3, 1}, {2, 2}, {3, 0}})
    {
        tr.addQuery(u, v);
    }
    assert((tr.solve() == vector<int>{3, 2, 1, 3}));
    tr.build(3);
    assert(tr.qs.empty());
}
```

2.4 差分与前缀技巧

2.4.001 一维/二维前缀和

静态数组区间和与静态矩阵子矩形和的基础工具。一维前缀定义 $s_i=\sum_{0\le j<i}a_j$，于是 $[l,r)$ 的区间和为 s_r-s_l；二维前缀沿两个轴累计，查询时用四个角做容斥。全程半开区间，让空区间、边界与长度计算保持一致。一维只需 partial_sum：

```
// a 是原数组，s 是前缀和，l、r 是半开区间端点，sum 是区间和。
vector<int> s(a.size() + 1);
partial_sum(a.begin(), a.end(), s.begin() + 1);
auto sum = s[r] - s[l];
```

二维把它封装成 Prefix2，构造时预处理带零行零列的前缀矩阵，sum 用四角容斥回答子矩形：

```
template <class T> struct Prefix2
{
    int n = 0, m = 0;     // n、m 是原矩阵行数与列数。
    vector<vector<T>> s; // s 是带额外零行零列的二维前缀和。

    explicit Prefix2(const vector<vector<T>> &a)
    {
        // a 是规则原矩阵。
        n = a.size();
        m = n ? a[0].size() : 0;
```

```
        s.assign(n + 1, vector<T>(m + 1));
        for (int i = 0; i < n; i++)
        {
            assert((int)a[i].size() == m); // 调试检查，可删
            for (int j = 0; j < m; j++)
            {
                s[i + 1][j + 1] = a[i][j] + s[i][j + 1] + s[i + 1][j] - s[i][j];
            }
        }
    }

    T sum(int x1, int y1, int x2, int y2) const
    {
        // x1、y1、x2、y2 是半开子矩形的上、左、下、右边界；返回子矩形和。
        return s[x2][y2] - s[x1][y2] - s[x2][y1] + s[x1][y1];
    }
};
```

调用形如 `Prefix2<int> ps(mat); auto rect = ps.sum(x1, y1, x2, y2)`，所有接口都用半开区间，二维矩阵必须规则矩形，空矩阵只能构造不能查询。模板参数 `T` 必须能表示每个前缀、四角容斥和查询结果；能由范围证明 64 位安全时使用 `int`，否则把累计类型实例化为更宽类型，不要只看单个元素范围。一维构造 $O(n)$、查询 $O(1)$、空间 $O(n)$；二维构造 $O(nm)$、查询 $O(1)$、空间 $O(nm)$。

2.4.002 一维/二维差分

前缀和的逆运算：离线执行大量区间加、矩形加，最后一次性恢复全部值。差分只在区间边界记录「从这里开始增加、到那里停止」，最终做一次前缀和即恢复所有位置；二维矩形修改在四个角写入容斥符号，再沿两个轴分别做前缀和。若修改后还要在线查询，则应改用 Fenwick 或线段树。一维封装为 `Diff`：

```
template <class T> struct Diff
{
    int n = 0;   // n 是目标数组长度。
    vector<T> d; // d 保存长度为 n+1 的边界差分。

    Diff(int m = 0) : n(m), d(m + 1)
    {
        // m 是目标数组长度。
    }

    void add(int l, int r, T x)
    {
        // l、r 是半开区间端点，x 是区间增量。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        d[l] += x;
        d[r] -= x;
    }

    vector<T> build() const
    {
```

```
        // 返回执行全部区间修改后的长度 n 数组。
        vector<T> a(n);
        T cur{};
        for (int i = 0; i < n; i++)
        {
            cur += d[i];
            a[i] = cur;
        }
        return a;
    }
};
```

二维版 `Diff2` 在矩形四角写差分、`build()` 时沿两轴前缀和还原：

```
template <class T> struct Diff2
{
    int n = 0, m = 0;     // n、m 是目标矩阵行数与列数。
    vector<vector<T>> d; // d 保存带额外边界的二维差分。

    Diff2(int x = 0, int y = 0) : n(x), m(y), d(x + 1, vector<T>(y + 1))
    {
        // x、y 是目标矩阵行数与列数。
    }

    void add(int x1, int y1, int x2, int y2, T v)
    {
        // 四个坐标描述半开子矩形，v 是矩形增量。
        assert(0 <= x1 && x1 <= x2 && x2 <= n); // 调试检查，可删
        assert(0 <= y1 && y1 <= y2 && y2 <= m); // 调试检查，可删
        d[x1][y1] += v;
        d[x1][y2] -= v;
        d[x2][y1] -= v;
        d[x2][y2] += v;
    }

    vector<vector<T>> build() const
    {
        // 返回执行全部矩形修改后的 n 行 m 列矩阵。
        auto a = d;
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < m; j++)
            {
                if (i)
                {
                    a[i][j] += a[i - 1][j];
                }
                if (j)
                {
                    a[i][j] += a[i][j - 1];
                }
                if (i && j)
```

```
                {
                    a[i][j] -= a[i - 1][j - 1];
                }
            }
        }
        a.resize(n);
        for (auto &row : a)
        {
            row.resize(m);
        }
        return a;
    }
};
```

调用形如 `Diff<int> d(n); d.add(l, r, x); auto a = d.build();` 与 `Diff2<int> d2(n, m); d2.add(x1, y1, x2, y2, v); auto mat = d2.build();`，修改同样是半开区间，`build()` 不清空差分故可重复恢复同一结果。模板参数 `T` 必须能表示边界差分、前缀恢复中的所有中间量和最终数组；若单次修改或多次叠加可能越过 64 位，应显式选择更宽累计类型。每次修改 $O(1)$；一维恢复 $O(n)$、空间 $O(n)$；二维恢复 $O(nm)$、空间 $O(nm)$。

2.4.003 K 阶差分

定义一阶差分 $\Delta a_i = a_{i+1} - a_i$，它把多项式次数降低 1，其逆过程即前缀和。于是普通差分重复 k 次后，次数不超过 k 的多项式序列会变成常量——这可用来识别多项式序列、从有限差分恢复值，以及设计以组合数为基的多项式区间更新。后者的依据是组合数基满足

$$\binom{x}{k} - \binom{x-1}{k} = \binom{x-1}{k-1},$$

对它做一次差分会自然降阶，区间加一个多项式可转成少量高阶差分端点修改。但各题的端点补偿取决于所选基底、下标起点和模数，不宜塞进一个没有明确语义的通用类，因此本节的短函数只负责对一个静态序列求 k 阶差分；

```
template <class T> vector<T> kthDiff(vector<T> a, int k)
{
    // a 是按值复制的原序列，k 是差分阶数；返回 k 阶差分序列。
    assert(k >= 0); // 调试检查，可删
    auto t = min<int>(k, a.size());
    for (int z = 0; z < t; z++)
    {
        for (int i = 1; i < (int)a.size(); i++)
        {
            a[i - 1] = a[i] - a[i - 1];
        }
        a.pop_back();
    }
    return a;
}
```

调用 `kthDiff(a, k)` 返回长度为 $\max(0, n-k)$ 的新序列、不修改传入数组，例如 `kthDiff(vector<int>{0,1,4,9,16}, 2)` 得 `{2,2,2}`（平方序列的二阶差分为常量）。序列在第 n 阶后已经为空，`k>n` 时不再执行无意义循环。若题目要多次区间加后统一输出，应先把所加多项式写成 $\binom{i}{j}$ 的线性组合，再据上式推导左右端点补偿；需在线查询则把各阶差分放入 Fenwick 或线段树。令 $h=\min(k,n)$，当前实现执行 $\sum_{j=0}^{h-1}(n-j-1)$ 次减法，时间 $O(hn)$、额外空间 $O(n)$；所有差分值须能由 `T` 表示。组合数区间更新的题目特化通常单次修改 $O(k)$、最终恢复 $O(kn)$。

2.4.004 子段和、子矩阵和

Kadane 求非空最大子段和：在每个位置只比较「接到前一段后面」与「从当前位置重新开始」，全负数时也能返回最大单元素而不误判为空：

```cpp
template <class T> T maxSub(const vector<T> &a)
{
    // a 是非空序列；返回非空最大子段和。
    // 若要最小子段和，保留首元素初始化，把 max 改成 min、chmax 改成 chmin；不能只改最终答案比较。
    assert(!a.empty()); // 调试检查，可删
    T ans = a[0];
    T cur = a[0];
    for (int i = 1; i < (int)a.size(); i++)
    {
        T x = a[i];
        cur = max(x, cur + x);
        chmax(ans, cur);
    }
    return ans;
}
```

最大子矩阵和枚举上下边界，把夹在两行间的每列和压成一维数组再复用同一个 Kadane；调用前让较小维度承担平方枚举，可明显降低常数：

```cpp
template <class T> T maxSubMat(const vector<vector<T>> &a)
{
    // a 是非空规则矩阵；返回非空最大子矩阵和。
    // 若要最小子矩阵和，同步使用最小子段版本并把 chmax 改成 chmin；边界仍以 a[0][0] 初始化。
    assert(!a.empty() && !a[0].empty()); // 调试检查，可删
    auto n = (int)a.size();
    auto m = (int)a[0].size();
    for (auto &row : a)
    {
        assert((int)row.size() == m); // 调试检查，可删
    }
    T ans = a[0][0];
    for (int u = 0; u < n; u++)
    {
        vector<T> s(m);
        for (int d = u; d < n; d++)
        {
            for (int j = 0; j < m; j++)
```

```
            {
                s[j] += a[d][j];
            }
            chmax(ans, maxSub(s));
        }
    }
    return ans;
}
```

直接 `maxSub(a)`、`maxSubMat(mat)` 调用，两者都要求非空输入并已正确处理全负数，返回类型与元素类型 `T` 相同。调用者须保证全部累计量能由 `T` 表示；确实需要 `__int128` 时先把输入元素转成 `__int128`，不为这两个短函数再加一层累计类型参数。当前实现平方枚举行数，故当行数大于列数时应先转置。最大子段 $O(n)$ 时间、$O(1)$ 空间；$n \times m$ 矩阵当前实现 $O(n^2m)$ 时间、$O(m)$ 空间，令较小维度作为行数后为 $O(\min(n,m)^2 \max(n,m))$。

2.4.005 所有区间 GCD

令 $V = \max_i |a_i|$。固定右端点 r、让左端点从大到小移动时，$\gcd(a_l, \dots, a_r)$ 除了从 0 变为首个非零值外，只会变成前一个正值的真因数，因此不同取值至多 $O(\log(V+2))$ 个。据此对每个右端点维护一张压缩列表：从 `r-1` 的列表转移到 `r` 时，把每个旧 gcd 与 `a[r]` 再取 gcd 并合并相邻相同结果，同时记录每个 gcd 生效的最小左端点，查询固定右端点后按左端点二分即可。因为要保存所有右端点的列表并支持多次查询，这里封装成结构体是必要的：

```
template <class T> struct GcdRanges
{
    using U = make_unsigned_t<T>;

    vector<vector<pair<int, U>>> g; // g[r] 保存以 r 结尾的不同 GCD 及最小左端点。

    static U mag(T x)
    {
        // x 是原数组元素；返回不对最小负数取正值的无符号幅值。
        auto u = (U)x;
        if constexpr (is_signed_v<T>)
        {
            if (x < 0)
            {
                return (U)0 - u;
            }
        }
        return u;
    }

    explicit GcdRanges(const vector<T> &a) : g(a.size())
    {
        // a 是用于预处理所有右端点 GCD 分段的原数组。
        static_assert(is_integral_v<T>);
        vector<pair<int, U>> pre;
        for (int r = 0; r < (int)a.size(); r++)
```

```
        {
            auto v = mag(a[r]);
            vector<pair<int, U>> cur{{r, v}};
            for (auto [l, x] : pre)
            {
                x = gcd(x, v);
                if (cur.back().second == x)
                {
                    cur.back().first = l;
                }
                else
                {
                    cur.push_back({l, x});
                }
            }
            reverse(cur.begin(), cur.end());
            g[r] = cur;
            pre = cur;
            reverse(pre.begin(), pre.end());
        }
    }

    U query(int l, int r) const
    {
        // l、r 是闭区间端点；返回 a[l..r] 的绝对值 GCD。
        assert(0 <= l && l <= r && r < (int)g.size()); // 调试检查，可删
        auto it = upper_bound(g[r].begin(),
                              g[r].end(),
                              l,
                              [](int x, const auto &p)
                              {
                                  return x < p.first;
                              });
        return prev(it)->second;
    }
};
```

调用形如 `GcdRanges<int> gr(a); auto g = gr.query(l, r);`，其中 `query` 是本章唯一使用闭区间 $[l, r]$ 的接口——因为结构按「以 r 结尾」存储，这里特意保留差异以防与半开区间混用。GCD 以 `make_unsigned_t<T>` 保存，负数先按位转换后在无符号环中取幅值，因此最小有符号整数也得到可表示的 2^{63}，返回值同样是无符号类型。每个右端点至多保留 $O(\log(V+2))$ 个不同 GCD；按 64 位 gcd 为常数操作的竞赛 RAM 模型，构造为 $O(n\log(V+2))$，若计入 Euclid 迭代则保守上界为 $O(n\log^2(V+2))$；空间 $O(n\log(V+2))$，单次查询 $O(\log(1+\log(V+2)))$。

2.4.006 连续区间按位异或

连续四个整数的低两位模式，使前缀异或 $0\oplus 1\oplus\cdots\oplus n$ 只由 $n \bmod 4$ 决定（依次为 n、1、$n+1$、0），于是闭区间 $[l, r]$ 的异或可用两个前缀异或相消得到，常数时间完成。这是纯公式，没有状态，也不需要结构体：

```cpp
// l、r 是闭区间端点，ans 是区间内所有整数的异或和。
auto ans = r;
if ((r & 3) == 1) ans = 1;
else if ((r & 3) == 2) ans = r + 1;
else if ((r & 3) == 3) ans = 0;

if (l)
{
    auto x = l - 1;
    auto pre = x;
    if ((x & 3) == 1) pre = 1;
    else if ((x & 3) == 2) pre = x + 1;
    else if ((x & 3) == 3) pre = 0;
    ans ^= pre;
}
```

代码结束后的 ans 即闭区间 $[l, r]$ 的异或和，要求 $0 \le l \le r$，且 l、r 使用无符号 64 位整数；若要对数组下标区间异或仍应改用普通前缀异或数组。$O(1)$ 时间、$O(1)$ 空间。

2.4.007 根号分治：区间加与区间和

根号分治把下标划成长度约为 $\sqrt{n}$ 的块。整块更新只记录懒标记，边界块逐元素修改；查询同样只扫描两个边界块并累加中间块和。它适合操作可按块聚合、数据规模允许根号复杂度的题目；复杂区间合并应使用 3.3.002 懒标记线段树。接口使用左闭右开区间 $[l, r)$。

```cpp
struct SqrtRangeAdd
{
    int n; // 元素个数。
    int bs; // 块长。
    vector<int> a; // 扣除整块懒标记后的元素值。
    vector<int> lazy; // 各块所有元素共同增加的值。
    vector<int> sum; // 各块包含懒标记的真实元素和。

    SqrtRangeAdd(const vector<int> &v) : n(v.size()), a(v)
    {
        // v 是初始数组；构造支持半开区间加与区间和的分块结构。
        bs = max<int>(1, sqrtl(n));
        lazy.resize((n + bs - 1) / bs);
        sum.resize(lazy.size());
        for (int i = 0; i < n; i++)
        {
            sum[i / bs] += a[i];
        }
    }

    void add(int l, int r, int v)
    {
        // l、r 指定 [l,r)，v 是每个元素的增量；原地修改，无返回值。
        assert(0 <= l && l <= r && r <= n);
        while (l < r)
```

```
        {
            int b = l / bs, end = min(n, (b + 1) * bs);
            if (l % bs == 0 && end <= r)
            {
                lazy[b] += v;
                sum[b] += (end - l) * v;
                l = end;
            }
            else
            {
                a[l++] += v;
                sum[b] += v;
            }
        }
    }

    int query(int l, int r) const
    {
        // l、r 指定 [l,r)；返回区间和，空区间返回 0。
        assert(0 <= l && l <= r && r <= n);
        int ans = 0;
        while (l < r)
        {
            int b = l / bs, end = min(n, (b + 1) * bs);
            if (l % bs == 0 && end <= r)
            {
                ans += sum[b];
                l = end;
            }
            else
            {
                ans += a[l++] + lazy[b];
            }
        }
        return ans;
    }
};
```

`add(l,r,v)` 将 $[l,r)$ 每项加上 `v`，`query(l,r)` 返回区间和。单次操作时间为 $O(\sqrt{n})$，额外空间为 $O(n)$；`sum[b]` 始终包含第 `b` 块的懒标记贡献。示例：

```
signed main()
{
    // 无参数；验证半开区间加与区间和，返回 0。
    SqrtRangeAdd a({1, 2, 3, 4, 5, 6, 7});
    a.add(1, 6, 10);
    assert(a.query(0, 7) == 78 && a.query(2, 5) == 42);
}
```

3. 数据结构

3.1 并查集

3.1.001 普通并查集

并查集维护若干互不相交的集合，支持合并、判连通、查询集合大小，是动态加边连通性、Kruskal 求最小生成树、等价类划分的基础工具。每个集合是一棵父指针树，`find` 用路径压缩把访问路径上的点直接挂到根，`merge` 按集合大小把小树接到大树下；两者配合后单次操作均摊到近似常数。要留意路径压缩会原地改写父链，因此这份实现无法撤销、也保存不了历史版本，需要那些能力时见下文的可撤销与可持久化变体。

```
struct DSU
{
    // f[x] 是 x 的父亲；s[x] 仅在 x 为根时表示集合大小。
    vector<int> f, s;

    DSU(int n = 0)
    {
        // n 是元素个数；初始时每个元素自成集合。
        init(n);
    }

    void init(int n)
    {
        // n 是新的元素个数；清空旧状态并重建单点集合。
        assert(n >= 0); // 调试检查，可删
        f.resize(n);
        iota(f.begin(), f.end(), 0);
        s.assign(n, 1);
    }

    int find(int x)
    {
        // x 是元素编号；压缩父链并返回集合根。
        return x == f[x] ? x : f[x] = find(f[x]);
    }

    bool merge(int x, int y)
    {
        // x、y 是元素编号；合并两集合并返回是否真正发生合并。
        x = find(x);
        y = find(y);
        if (x == y)
        {
            return false;
        }
        if (s[x] < s[y])
        {
```

```
            swap(x, y);
        }
        f[y] = x;
        s[x] += s[y];
        return true;
    }

    bool same(int x, int y)
    {
        // x、y 是元素编号；返回二者是否同属一个集合。
        return find(x) == find(y);
    }

    int size(int x)
    {
        // x 是元素编号；返回其集合大小。
        return s[find(x)];
    }
};
```

公开接口为 `DSU`：`DSU(n)` 或 `init(n)` 建立 n 个单点集合（下标 $[0,n)$），`merge(x,y)` 把两点所在块并起来并返回是否真正合并，`same(x,y)` 判同集，`size(x)` 取 x 所在块的元素数。构造 $O(n)$；路径压缩加按大小合并后 m 次操作总代价 $O(m\,\alpha(n))$，单次最坏 $O(\log n)$，空间 $O(n)$。只需加边连通性用本条；维护点间差值或异或约束改用 3.1.003 带权并查集，离线撤销最近操作用 3.1.004 可撤销并查集，在线访问任意历史版本用 3.1.005 可持久化并查集，一次合并两段等长区间用 3.1.006 区间并行并查集。

3.1.002 按大小/按秩合并

这不是一份新代码，而是对合并策略的说明：本章并查集统一采用按大小合并，直接依赖 3.1.001 普通并查集 的 `DSU`。`merge(x,y)` 里 `if (s[x] < s[y]) swap(x, y)` 保证较小的树挂到较大的树下，树高因此不超过 $O(\log n)$；再叠加路径压缩，m 次操作均摊 $O(m\,\alpha(n))$。按秩合并是等价的替代口径——把 `s` 换成树高上界，仅当两根秩相等时把新根的秩加一，其余只比较不更新；两种口径都能压树高，但「大小」和「秩」语义不同，不能混用同一个数组。单独用路径压缩而不按大小/秩合并时，可以先形成长度为 n 的父链，第一次查询链尾的单次代价为 $O(n)$；标准的反阿克曼均摊界要求路径压缩与按秩或按大小合并配合。带权、可撤销、可持久化等变体分别见 3.1.003 带权并查集、3.1.004 可撤销并查集、3.1.005 可持久化并查集。

3.1.003 带权并查集

带权并查集在连通性之外，还沿父边维护每个点到其父亲的势能，从而回答同一集合内两点的差值，并检验新约束是否与已有约束矛盾。加法势能版把约束 $val_y - val_x = d$ 视作一条带权边：`find` 沿路径压缩时把 `w` 累加成到根的势能，`merge` 先各自找根，再由 $v = d + w_x - w_y$ 算出两根间应有的差；若两点已同根，只需检查 v 是否为零，否则把小树挂到大树并令新根边权等于 $\pm v$。势能类型 `T` 必须支持零元、加减、相反数和相等比较。

```cpp
template <class T> struct WDSU
{
    // f[x] 是 x 的父亲；s[x] 仅在 x 为根时表示集合大小。
    vector<int> f, s;
    // w[x] 是 x 到父亲的势能差 val[x]-val[f[x]]。
    vector<T> w;

    WDSU(int n = 0)
    {
        // n 是元素个数；初始势能均为零。
        init(n);
    }

    void init(int n)
    {
        // n 是新的元素个数；清空旧状态并重建单点集合。
        assert(n >= 0); // 调试检查，可删
        f.resize(n);
        iota(f.begin(), f.end(), 0);
        s.assign(n, 1);
        w.assign(n, T{});
    }

    int find(int x)
    {
        // x 是元素编号；压缩父链、累加到根的势能并返回根。
        if (x == f[x])
        {
            return x;
        }
        int p = f[x];
        f[x] = find(p);
        w[x] += w[p];
        return f[x];
    }

    // 加入 val[y]-val[x]=d；已连通时返回该约束是否一致。
    bool merge(int x, int y, T d)
    {
        // 施加 val[y]-val[x]=d；已连通时返回该约束是否一致。
        int a = find(x), b = find(y);
        T v = d + w[x] - w[y];
        if (a == b)
        {
            return v == T{};
        }
        if (s[a] < s[b])
        {
            f[a] = b;
            w[a] = -v;
            s[b] += s[a];
        }
```

```
        else
        {
            f[b] = a;
            w[b] = v;
            s[a] += s[b];
        }
        return true;
    }

    bool same(int x, int y)
    {
        // x、y 是元素编号；返回二者是否同属一个集合。
        return find(x) == find(y);
    }

    // 要求 x,y 已连通，返回 val[y]-val[x]。
    T diff(int x, int y)
    {
        // x、y 必须连通；返回 val[y]-val[x]。
        assert(same(x, y)); // 调试检查，可删
        return w[y] - w[x];
    }
};
```

`WDSU<T>(n)` 建 n 个孤立点。`merge(x,y,d)` 施加约束 $val_y - val_x = d$：两点原本不连通时返回 `true` 并合并，已连通时返回该约束是否与现状一致（不修改结构）。`diff(x,y)` 要求 x, y 已同集（内部 `assert`），返回 $val_y - val_x$。

异或势能不能直接套用加法版的模板参数。下面令 `xr[x]` 表示 $val_x \oplus val_{f_x}$；路径压缩时沿父链累积异或值，加入约束 $val_x \oplus val_y = w$ 时，两根之间的异或值为 `xr[x]^xr[y]^w`。

```
struct XorDSU
{
    // f[x] 是 x 的父亲；s[x] 仅在 x 为根时表示集合大小。
    vector<int> f, s, xr;

    XorDSU(int n = 0)
    {
        // n 是元素个数；初始异或势能均为零。
        init(n);
    }

    void init(int n)
    {
        // n 是新的元素个数；清空旧状态并重建单点集合。
        assert(n >= 0); // 调试检查，可删
        f.resize(n);
        iota(f.begin(), f.end(), 0);
        s.assign(n, 1);
        xr.assign(n, 0);
    }
```

```
21
22      int find(int x)
23      {
24          // x 是元素编号；压缩父链、累积到根的异或值并返回根。
25          if (x == f[x])
26          {
27              return x;
28          }
29          int p = f[x];
30          f[x] = find(p);
31          xr[x] ^= xr[p];
32          return f[x];
33      }
34
35      bool merge(int x, int y, int w)
36      {
37          // 施加 val[x]^val[y]=w；已连通时返回该约束是否一致。
38          int a = find(x), b = find(y);
39          int v = xr[x] ^ xr[y] ^ w;
40          if (a == b)
41          {
42              return v == 0;
43          }
44          if (s[a] < s[b])
45          {
46              f[a] = b;
47              xr[a] = v;
48              s[b] += s[a];
49          }
50          else
51          {
52              f[b] = a;
53              xr[b] = v;
54              s[a] += s[b];
55          }
56          return true;
57      }
58
59      bool same(int x, int y)
60      {
61          // x、y 是元素编号；返回二者是否同属一个集合。
62          return find(x) == find(y);
63      }
64
65      int diff(int x, int y)
66      {
67          // x、y 必须连通；返回 val[x]^val[y]。
68          assert(same(x, y)); // 调试检查，可删
69          return xr[x] ^ xr[y];
70      }
71
72      int value(int x)
```

```
    {
        // x 是元素编号；把当前根的值视为零，返回 val[x]。
        find(x);
        return xr[x];
    }
};
```

`XorDSU(n)` 建 n 个孤立点。`merge(x,y,w)` 施加 $val_x \oplus val_y = w$，返回该约束是否与现状一致；`diff(x,y)` 返回同一集合内两点的异或值。`value(x)` 把当前集合根的值视为零，只给出这一约定下的点值；未固定根的真实值时，只有两点异或值是确定的。两版构造均为 $O(n)$，单次操作最坏 $O(\log n)$、均摊 $O(\alpha(n))$，空间 $O(n)$。仅需连通性用 3.1.001 普通并查集；若势能属于矩阵、置换等非交换乘法群，不能改写 `WDSU` 的加减顺序，应使用 3.1.008 非交换群势能并查集；若约束需要撤销或回退到历史版本，则分别改用 3.1.004 可撤销并查集 与 3.1.005 可持久化并查集，并相应维护势能（为保历史不能使用路径压缩）。

3.1.004 可撤销并查集

可撤销并查集支持回退最近若干次合并，用于线段树分治、时间线回退和离线分治：处理完一段时间/一个分治区间后，把期间做过的合并原样撤销，恢复到进入前的状态。路径压缩会不可控地改写大量父指针、无法逐条回滚，因此这里彻底放弃压缩，只保留按大小合并（用负数 `p[root]` 直接编码集合大小）。每次 `merge` 都往栈里留痕：真正合并时压两条记录（两个被改根的旧父值），两点已同集时压一条哨兵 `{-1,0}`；`snap` 取当前栈高作为快照编号，`rollback(s)` 弹栈直到高度回到 `s` 并逐一还原父指针。因为没有压缩，`find` 单次是 $O(\log n)$ 而非 $\alpha(n)$。

```
struct RollbackDSU
{
    // p[x]<0 表示 x 是根且集合大小为 -p[x]；否则 p[x] 是父亲。
    vector<int> p;
    // st 依次保存每个被改位置及其修改前的 p 值；{-1,0} 表示一次空合并。
    vector<pair<int, int>> st;

    RollbackDSU(int n = 0) : p(n, -1)
    {
        // n 是元素个数；初始时每个元素自成集合。
        assert(n >= 0); // 调试检查，可删
    }

    int find(int x) const
    {
        // x 是元素编号；沿父链返回根，不做路径压缩。
        while (p[x] >= 0)
        {
            x = p[x];
        }
        return x;
    }

    int snap() const
    {
```

```
        // 返回当前回滚栈高度，作为之后回滚所用的快照编号。
        return (int)st.size();
    }

    bool merge(int a, int b)
    {
        // a、b 是元素编号；按大小合并并返回是否真正发生合并。
        a = find(a);
        b = find(b);
        if (a == b)
        {
            st.push_back({-1, 0});
            return false;
        }
        if (p[a] > p[b])
        {
            swap(a, b);
        }
        st.push_back({b, p[b]});
        st.push_back({a, p[a]});
        p[a] += p[b];
        p[b] = a;
        return true;
    }

    void rollback(int s)
    {
        // s 是此前由 snap 返回的栈高；撤销其后的所有合并。
        assert(0 <= s && s <= (int)st.size()); // 调试检查，可删
        while ((int)st.size() > s)
        {
            auto [u, x] = st.back();
            st.pop_back();
            if (u >= 0)
            {
                p[u] = x;
            }
        }
    }

    bool same(int a, int b) const
    {
        // a、b 是元素编号；返回二者是否同属一个集合。
        return find(a) == find(b);
    }

    int size(int x) const
    {
        // x 是元素编号；返回其集合大小。
        return -p[find(x)];
    }
};
```

典型用法：`snap()` 记下当前进度，做若干 `merge`，事后 `rollback(snap)` 撤回。`merge(a,b)` 返回是否真正合并，`same/size` 只读查询。`find`、`merge` 单次 $O(\log n)$，`rollback` 与被撤销的合并数成正比，栈空间 $O(n+q)$。它正是 3.3.016 线段树分治 与整体二分里维护连通性的标准件；只要求普通连通性、无需回退时用 3.1.001 普通并查集（能用路径压缩更快），需要在线跳到任意历史版本而非仅撤销栈顶时改用 3.1.005 可持久化并查集。

3.1.005 可持久化并查集

可持久化并查集为每次合并保留一个不可变的历史版本，允许从任意旧版本继续分叉、并查询当时的连通关系，适合强制在线的历史状态题。底层是一棵稀疏可持久化线段树（当作可持久化数组用）：未写叶子隐含为单点集合；写入的根以负数编码集合大小，非根写父亲编号。这样初始版本只用根 0，每次合并才按两条修改路径复制 $O(\log n)$ 个结点，旧版本原封不动。因为版本要共享结构，绝不能用路径压缩（那会原地改链、污染旧版本），只能按大小合并压树高——于是 `find` 里每跳一步父亲都要走一次 $O(\log n)$ 的线段树查询，单次 `find` 是 $O(\log^2 n)$。

```
struct PerDSU
{
    struct N
    {
        // l、r 是持久化数组的左右儿子；value 为叶子覆盖值，负数编码根的集合大小，非负数是父亲。
        int l = 0, r = 0, value = -1;
    };

    // n 是元素个数；t[0] 是全未写的初始版本，隐含每个元素自成集合。
    int n;
    vector<N> t{{}};

    PerDSU(int n) : n(n)
    {
        // n 是元素个数；允许 n=0，此时 build 返回空根 0。
        assert(n >= 0); // 调试检查，可删
    }

    int build(int l, int r) const
    {
        // [l,r) 是初始数组范围；初始值全是隐含单点集合，返回空根 0。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return 0;
    }

    int build() const
    {
        // 返回 n 个单点集合组成的初始版本根；空结构也返回 0。
        return 0;
    }

    int value(int x, int l, int r, int p) const
    {
        // x 是版本树节点，[l,r) 是其区间，p 是元素下标；返回覆盖值，未写位置返回 -1。
```

```
        if (!x)
        {
            return -1;
        }
        if (r - l == 1)
        {
            return t[x].value;
        }
        int m = (l + r) / 2;
        return p < m ? value(t[x].l, l, m, p) : value(t[x].r, m, r, p);
    }

    pair<int, int> get(int rt, int p) const
    {
        // rt 是版本根，p 是元素下标；返回 {父亲，根大小}，非根的第二项不使用。
        assert(0 <= p && p < n); // 调试检查，可删
        int v = value(rt, 0, n, p);
        return v < 0 ? pair<int, int>{p, -v} : pair<int, int>{v, 0};
    }

    int set(int x, int l, int r, int p, int v)
    {
        // x 是旧版本节点，[l,r) 是其区间，p 是元素下标；写入覆盖值 v 并返回新根。
        assert(t.size() < numeric_limits<int>::max()); // 调试检查，可删
        int y = (int)t.size();
        t.push_back(t[x]);
        if (r - l == 1)
        {
            t[y].value = v;
            return y;
        }
        int m = (l + r) / 2;
        if (p < m)
        {
            t[y].l = set(t[y].l, l, m, p, v);
        }
        else
        {
            t[y].r = set(t[y].r, m, r, p, v);
        }
        return y;
    }

    int find(int rt, int x) const
    {
        // rt 是版本根，x 是元素编号；沿父链返回根，不做路径压缩。
        while (true)
        {
            int p = get(rt, x).first;
            if (p == x)
            {
                return x;
```

```
            }
            x = p;
        }
    }

    int merge(int rt, int x, int y)
    {
        // rt 是旧版本根，x、y 是元素编号；返回按大小合并后的新版本根。
        x = find(rt, x);
        y = find(rt, y);
        if (x == y)
        {
            return rt;
        }
        int sx = get(rt, x).second;
        int sy = get(rt, y).second;
        if (sx < sy)
        {
            swap(x, y);
            swap(sx, sy);
        }
        rt = set(rt, 0, n, y, x);
        return set(rt, 0, n, x, -(sx + sy));
    }

    bool same(int rt, int x, int y) const
    {
        // rt 是版本根，x、y 是元素编号；返回二者是否同属一个集合。
        return find(rt, x) == find(rt, y);
    }

    int size(int rt, int x) const
    {
        // rt 是版本根，x 是元素编号；返回其集合大小。
        return get(rt, find(rt, x)).second;
    }
};
```

用法：`PerDSU pd(n)` 后 `int root = pd.build()` 取初始版本根，之后所有操作都显式带上版本根——`merge(rt,x,y)` 返回新版本根（无合并时返回原 `rt`），`find(rt,x)`、`same(rt,x,y)`、`size(rt,x)` 在指定版本上只读查询。`n=0` 时 `build()` 返回空根 0，此时没有合法元素可供查询或合并。可把每个版本根存进数组以随时回访。`find`/`same`/`size` 最坏 $O(\log^2 n)$，`merge` 最坏 $O(\log^2 n)$ 且新增 $O(\log n)$ 个结点；初建 $O(1)$，总空间 $O(q \log n)$。打印版的结点下标和叶子值使用普通 `int`；性能版为缩小持久化节点固定成 32 位，因此使用性能头时还要求 n、节点总数和集合大小不超过 $2^{31}-1$。它比 3.1.004 可撤销并查集 强在能访问任意历史版本而非只能撤销栈顶，代价是多一个 $\log$；若只需撤销最近操作，可撤销版更省。底层是按下标单点持久化的线段树，与 3.3.008 可持久化线段树 / 主席树 同源。

3.1.006 区间并行并查集

区间并行并查集一次把两个等长区间 $[x, x+len)$ 与 $[y, y+len)$ 的对应位置分别合并，用于「区间 A 与区间 B 逐位相等」这类约束（如强制某两段子串相等）。朴素做法逐位 `merge` 是 $O(len\,\alpha)$，这里借鉴 ST 表倍增：对每个层级 k 维护一层「长度 2^k 的块」的并查集 `d[k]`，先在合适层级把两个长块整体合并，仅当该层块是首次被合并时才递归下拆到两个半块，已合并过的块直接剪枝。真正的单点等价通过回调 `f(a,b)` 落到第 0 层，再由调用方交给 3.1.001 普通并查集 的 `DSU` 记录最终连通性；下面只打印新增的 `RangeDSU`。

```
// 依赖 3.1.001 普通并查集
struct RangeDSU
{
    struct D
    {
        // f[x]<0 表示 x 是块集合根且大小为 -f[x]；否则 f[x] 是父亲。
        vector<int> f;
        D(int n = 0) : f(n, -1)
        {
            // n 是本层可作为块起点的位置数。
        }
        int find(int x)
        {
            // x 是本层块起点；压缩父链并返回块集合根。
            return f[x] < 0 ? x : f[x] = find(f[x]);
        }
        pair<int, int> merge(int x, int y)
        {
            // x、y 是本层块起点；返回 {新根, 被吸收根}，已经同集时返回 {-1,-1}。
            x = find(x);
            y = find(y);
            if (x == y)
            {
                return {-1, -1};
            }
            if (f[x] > f[y])
            {
                swap(x, y);
            }
            f[x] += f[y];
            f[y] = x;
            return {x, y};
        }
    };

    // n 是原序列长度；d[k] 维护所有长度为 2^k 的块之间的等价关系。
    int n;
    vector<D> d;

    RangeDSU(int n) : n(n)
    {
        // n 是原序列长度；预建每个倍增层的块并查集。
        assert(n >= 0); // 调试检查，可删
```

```
        int m = max<int>(1, bit_width((unsigned long long)n));
        for (int k = 0; k < m; k++)
        {
            d.emplace_back(n - (1LL << k) + 1);
        }
    }

    template <class F> void mergeBlock(int k, int x, int y, F &f)
    {
        // k 是块层级，x、y 是两个块起点；首次连通单点块时调用 f(新根, 被吸收根)。
        auto [a, b] = d[k].merge(x, y);
        if (a < 0)
        {
            return;
        }
        if (!k)
        {
            return f(a, b);
        }
        int h = 1LL << (k - 1);
        mergeBlock(k - 1, x, y, f);
        mergeBlock(k - 1, x + h, y + h, f);
    }

    template <class F> void merge(int x, int y, int len, F f)
    {
        // 合并 [x,x+len) 与 [y,y+len)；每个新单点合并调用一次 f(新根, 被吸收根)。
        assert(0 <= x && x <= n && 0 <= y && y <= n && 0 <= len && len <= n - x && len <= n -
        ↪ y); // 调试检查，可删
        if (!len)
        {
            return;
        }
        int k = bit_width((unsigned long long)len) - 1;
        int z = 1LL << k;
        mergeBlock(k, x, y, f);
        mergeBlock(k, x + len - z, y + len - z, f);
    }
};
```

用法：`RangeDSU rd(n)` 建好各倍增层；对每条「两段相等」约束调用 `rd.merge(x, y, len, [&](int root, int absorbed){ … })`。lambda 只会在第 0 层真正合并时调用，两个参数分别是此结构第 0 层并查集的新根和被吸收根，因此既可只把它们交给外部 `DSU base(n)` 做 `base.merge(root, absorbed)`，也可把连通块和、计数等聚合状态直接维护在 `root` 上后再吸收 `absorbed`。要求回调返回 `void`，且 $[x, x+len)$、$[y, y+len)$ 都落在 $[0, n)$ 内；`len=0` 不产生操作。每层每对块至多真正合并一次，全部 q 次区间操作的总时间为 $O((n\log n + q)\alpha(n))$，总空间 $O(n\log n)$。它是逐位 `merge` 的批量加速版：区间约束很多、单点朴素合并成为瓶颈时才值得用，约束零散时直接 3.1.001 普通并查集 更简单。

外部 `DSU` 只记录第 0 层真实连通关系时，最小调用如下：

```
signed main()
{
    RangeDSU rd(6);
    DSU base(6);
    rd.merge(0, 3, 3, [&](int root, int absorbed)
    {
        base.merge(root, absorbed);
    });
    assert(base.same(0, 3) && base.same(1, 4) && base.same(2, 5));
}
```

3.1.007 可删除并查集

普通并查集的父指针一旦压缩，就无法从一棵集合树中直接摘掉任意元素；可删除并查集给每个原元素 x 保存一个当前替身 `pos[x]`，删除时不改旧父链，只把旧集合的有效元素数减一，再为 x 新建一个单点替身。`p` 记录只增不减的物理节点数并据此按大小合并，`cnt` 单独记录当前有效元素数；不能拿会随删除减小的 `cnt` 决定合并方向，否则一棵物理大树反复失去元素后可能被当成小树悬挂，破坏并查集的高度界。这里的删除语义是“把元素移到新的单点集合”，不是让元素从编号域中消失。

```
struct ErasableDSU
{
    // p[x]<0 表示 x 是内部树根，-p[x] 是只增不减的物理节点数；否则 p[x] 是父亲。
    vector<int> p;
    // cnt[x] 仅在 x 是根时表示集合中的当前元素数。
    vector<int> cnt;
    // pos[x] 是当前代表原元素 x 的内部节点。
    vector<int> pos;

    ErasableDSU(int n = 0)
    {
        // n 是原元素个数；初始时每个元素自成集合。
        init(n);
    }

    void init(int n)
    {
        // n 是新的原元素个数；清空全部合并与删除状态，无返回值。
        assert(n >= 0); // 调试检查，可删
        p.assign(n, -1);
        cnt.assign(n, 1);
        pos.resize(n);
        iota(pos.begin(), pos.end(), 0);
    }

    int findNode(int x)
    {
        // x 是内部节点编号；压缩父链并返回内部树根。
        return p[x] < 0 ? x : p[x] = findNode(p[x]);
    }
```

```
31
32      bool merge(int x, int y)
33      {
34          // x、y 是原元素编号；合并所在集合并返回是否真正发生合并。
35          assert(0 <= x && x < (int)pos.size() && 0 <= y && y < (int)pos.size()); // 调试检查，可删
36          int a = findNode(pos[x]);
37          int b = findNode(pos[y]);
38          if (a == b)
39          {
40              return false;
41          }
42          if (p[a] > p[b])
43          {
44              swap(a, b);
45          }
46          p[a] += p[b];
47          p[b] = a;
48          cnt[a] += cnt[b];
49          return true;
50      }
51
52      bool erase(int x)
53      {
54          // x 是原元素编号；把它移到新的单点集合，原本已是单点时返回 false。
55          assert(0 <= x && x < (int)pos.size()); // 调试检查，可删
56          int r = findNode(pos[x]);
57          if (cnt[r] == 1)
58          {
59              return false;
60          }
61          cnt[r]--;
62          pos[x] = p.size();
63          p.push_back(-1);
64          cnt.push_back(1);
65          return true;
66      }
67
68      bool same(int x, int y)
69      {
70          // x、y 是原元素编号；返回二者当前是否同属一个集合。
71          assert(0 <= x && x < (int)pos.size() && 0 <= y && y < (int)pos.size()); // 调试检查，可删
72          return findNode(pos[x]) == findNode(pos[y]);
73      }
74
75      int size(int x)
76      {
77          // x 是原元素编号；返回其当前集合中的原元素数。
78          assert(0 <= x && x < (int)pos.size()); // 调试检查，可删
79          return cnt[findNode(pos[x])];
80      }
81  };
```

`ErasableDSU(n)` 的原元素编号固定为 $[0,n)$；`merge`、`same`、`size` 与普通并查集同义，`erase(x)` 仅当 x 原来不是单点时新建替身并返回 `true`。若共有 e 次成功删除，构造为 $O(n)$，其余操作均摊 $O(\alpha(n+e))$、单次最坏 $O(\log(n+e))$，空间 $O(n+e)$。没有删除时直接用 3.1.001 普通并查集；按时间撤销合并用 3.1.004 可撤销并查集，访问任意历史版本用 3.1.005 可持久化并查集，这两种历史语义都不能由替身删除代替。

```
signed main()
{
    ErasableDSU d(4);
    d.merge(0, 1);
    d.merge(1, 2);
    assert(d.size(0) == 3 && d.erase(1));
    assert(!d.same(0, 1) && d.same(0, 2));
    assert(d.size(0) == 2 && d.size(1) == 1);
    assert(!d.erase(1));
    d.merge(1, 3);
    assert(d.same(1, 3) && d.size(3) == 2);
}
```

3.1.008 非交换群势能并查集

矩阵、置换、刚体变换等势能约束的乘法次序不能交换；把加法版 `WDSU` 里的 `+`、`-` 机械替换为乘法和逆元会在路径压缩或按大小换根时反转因子，常常只在链形数据上才暴露。这里把未知量记为群元素 $value_x$，群运算按书写顺序记为 $a\circ b$，并在父边保存右势能

$$value_x = value_{f_x}\circ pot_x.$$

模板参数 `Group` 必须提供 `using T`、静态函数 `id()`、`op(a,b)`、`inv(a)`，分别返回单位元、$a\circ b$ 与 a^{-1}；`T` 还须支持 `==`。`find` 把一条两边路径压成根边时严格按 $pot_x \leftarrow pot_{f_x}\circ pot_x$ 更新。`merge(x,y,d)` 的语义固定为 $value_x\circ d = value_y$，因此同根校验是 $pot_x^{-1}\circ pot_y = d$。若根 a 挂到根 b，应令

$$pot_a = pot_y\circ d^{-1}\circ pot_x^{-1};$$

反方向则令 $pot_b = pot_x\circ d\circ pot_y^{-1}$。两条式子都不能重排。对于 Library Checker 的 $a_u = a_v x$，直接调用 `merge(v,u,x)`；查询 $a_v^{-1}a_u$ 则调用 `diff(v,u)`。

```
template <class Group> struct PotentialDSU
{
    using T = typename Group::T;

    // f[x] 是 x 的父亲；s[x] 仅在 x 为根时表示集合大小。
    vector<int> f, s;
    // pot[x] 满足 value[x]=value[f[x]]*pot[x]；根的 pot 是单位元。
    vector<T> pot;

    PotentialDSU(int n = 0)
    {
```

```
        // n 是元素个数；初始时每个元素自成集合且势能为群单位元。
        init(n);
    }

    void init(int n)
    {
        // n 是新的元素个数；清空旧状态并重建单点集合。
        assert(n >= 0); // 调试检查，可删
        f.resize(n);
        iota(f.begin(), f.end(), 0);
        s.assign(n, 1);
        pot.assign(n, Group::id());
    }

    int find(int x)
    {
        // x 是元素编号；压缩父链、更新到根的右势能并返回集合根。
        if (x == f[x])
        {
            return x;
        }
        int p = f[x];
        f[x] = find(p);
        pot[x] = Group::op(pot[p], pot[x]);
        return f[x];
    }

    bool merge(int x, int y, const T &d)
    {
        // 施加 value[x]*d=value[y]；返回约束是否与已有信息相容。
        int a = find(x), b = find(y);
        if (a == b)
        {
            return Group::op(Group::inv(pot[x]), pot[y]) == d;
        }
        if (s[a] < s[b])
        {
            f[a] = b;
            pot[a] = Group::op(Group::op(pot[y], Group::inv(d)), Group::inv(pot[x]));
            s[b] += s[a];
        }
        else
        {
            f[b] = a;
            pot[b] = Group::op(Group::op(pot[x], d), Group::inv(pot[y]));
            s[a] += s[b];
        }
        return true;
    }

    bool same(int x, int y)
    {
```

```
        // x、y 是元素编号；返回二者是否同属一个集合。
        return find(x) == find(y);
    }

    T diff(int x, int y)
    {
        // x、y 必须连通；返回 value[x] 的逆乘 value[y]，即 value[x]^{-1}*value[y]。
        assert(same(x, y)); // 调试检查，可删
        return Group::op(Group::inv(pot[x]), pot[y]);
    }

    int size(int x)
    {
        // x 是元素编号；返回 x 所在集合的元素个数。
        return s[find(x)];
    }
};
```

`PotentialDSU<Group>(n)` 建立下标 $[0,n)$ 的单点集合；`merge(x,y,d)` 返回新约束是否相容、相容且原来不连通时合并；`same(x,y)` 判连通；`diff(x,y)` 要求二者连通，返回 $value_x^{-1}\circ value_y$；`size(x)` 返回集合大小。设一次 `Group` 运算为 $O(1)$，构造 $O(n)$，路径压缩配合按大小合并后 m 次操作均摊 $O(m\alpha(n))$、单次最坏 $O(\log n)$，额外空间 $O(n)$。加法差分用 3.1.003 带权并查集 更短，异或约束也用该条的 `XorDSU`；需要回滚或持久版本时现有 3.1.004 可撤销并查集、3.1.005 可持久化并查集 只维护连通性，不能直接替代这里的群势能。

下面以置换群固定验证非交换顺序、反向查询与把小集合挂到大集合的分支；`op(a,b)` 定义为先作用 `b` 再作用 `a`：

```
struct PermGroup
{
    using T = array<int, 3>; // T[i] 是置换把 i 映到的像。

    static T id()
    {
        // 返回三元置换群的恒等置换。
        return {0, 1, 2};
    }

    static T op(const T &a, const T &b)
    {
        // a、b 是置换；返回先作用 b 再作用 a 的复合 a*b。
        T c{};
        for (int i = 0; i < 3; i++)
        {
            c[i] = a[b[i]];
        }
        return c;
    }

    static T inv(const T &a)
    {
```

```
        // a 是置换；返回其逆置换。
        T b{};
        for (int i = 0; i < 3; i++)
        {
            b[a[i]] = i;
        }
        return b;
    }
};

signed main()
{
    array<int, 3> a{1, 2, 0}; // a[i] 是循环置换 a 对 i 的像。
    array<int, 3> b{1, 0, 2}; // b[i] 是交换 0、1 的置换 b 对 i 的像。
    assert(PermGroup::op(a, b) != PermGroup::op(b, a));
    PotentialDSU<PermGroup> d(4);
    assert(d.merge(0, 1, a));
    assert(d.merge(1, 2, b));
    assert(d.merge(3, 0, a));
    assert(d.diff(3, 2) == PermGroup::op(a, PermGroup::op(a, b)));
    assert(!d.merge(3, 2, PermGroup::op(PermGroup::op(a, b), a)));
    cout << "OK\n";
}
```

3.2 树状数组

3.2.001 单点修改、前缀和

树状数组（Fenwick / BIT）以 $O(\log n)$ 支持单点增量与前缀和，区间和由两个前缀相减得到，是本章最轻量的可修改前缀结构，也是主席树套 BIT、二维数点等的底座。核心是 `a[x]` 保存以 x 结尾、长度为 $\mathrm{lowbit}(x) = x \,\&\, (-x)$ 的块和：`add` 沿 `x += x & -x` 逐级更新覆盖该点的祖先块，`sum` 沿 `x -= x & -x` 把前缀拆成 $O(\log n)$ 个不相交块累加。内部下标从 1 开始（接口里先 `++x`），对外一律 0 下标、区间用左闭右开 $[l, r)$。它只支持有逆元的可加聚合（求和、异或），最大值这类不可逆运算要另用 3.2.005 前缀/后缀最大值 或线段树。附带的 `kth` 借二进制提升，在各点权非负、前缀和单调时按前缀和二分位置，避免「外层二分 + 每步 `sum`」的双 log。这份 `BIT<T>` 是全章树状数组的公共底座，后续条目按编号引用它、不再重复粘贴。

```
template <class T> struct BIT
{
    // n 是数组长度；a[i] 是以内部下标 i 结尾的 lowbit 块和。
    int n;
    vector<T> a; // a[pos(i)] 是 Fenwick 节点 i 管辖区间之和，i 从 1 开始。

    BIT(int n = 0)
    {
        // n 是数组长度；初值全为零。
        init(n);
    }
```

```
12
13      void init(int n_)
14      {
15          // n_ 是新的数组长度；清空旧状态并置零。
16          assert(n_ >= 0); // 调试检查，可删
17          n = n_;
18          a.assign(n + 1, T{});
19      }
20
21      void add(int x, T v)
22      {
23          // x 是 0 下标位置，v 是增量；把 a[x] 增加 v。
24          assert(0 <= x && x < n); // 调试检查，可删
25          for (++x; x <= n; x += x & -x)
26          {
27              a[x] += v;
28          }
29      }
30
31      T sum(int x) const
32      {
33          // x 是前缀右端点；返回 [0,x) 的和。
34          assert(0 <= x && x <= n); // 调试检查，可删
35          T ans{};
36          for (; x; x -= x & -x)
37          {
38              ans += a[x];
39          }
40          return ans;
41      }
42
43      T sum(int l, int r) const
44      {
45          // l、r 是半开区间端点；返回 [l,r) 的和。
46          assert(0 <= l && l <= r && r <= n); // 调试检查，可删
47          return sum(r) - sum(l);
48      }
49
50      int kth(T k) const
51      {
52          // 各点权必须非负且 k>=0；返回满足 sum(x)<=k 的最大前缀长度 x。
53          assert(k >= T{}); // 调试检查，可删
54          int x = 0;
55          T cur{};
56          for (int d = bit_floor((unsigned long long)n); d; d >>= 1)
57          {
58              if (x + d <= n && cur + a[x + d] <= k)
59              {
60                  x += d;
61                  cur += a[x];
62              }
63          }
```

```
        return x;
    }
};
```

当 n 很大且更新、查询密集时，Fenwick 的 2 的幂位置会集中映射到少数缓存组。下面的 `FastBIT` 每隔 1024 个逻辑槽插入一个空洞，只增加约 0.1% 空间，却能打散这类冲突；接口与 `BIT` 相同，并补充了 $O(n)$ 建树：

```
template <class T> struct FastBIT
{
    int n = 0; // n 是逻辑长度（元素个数）。
    vector<T> a;

    static constexpr int pos(int x)
    {
        // x 是 Fenwick 下标；返回每 1024 个槽插入一个空洞后的物理下标。
        return x + (x >> 10);
    }

    FastBIT(int n = 0) { init(n); }

    explicit FastBIT(const vector<T> &v)
    {
        // v 是初值数组；用前缀和 O(n) 建树，a[pos(i)] = pre[i] - pre[i & (i-1)]。
        init(v.size());
        vector<T> pre(n + 1);
        for (int i = 1; i <= n; i++)
        {
            pre[i] = pre[i - 1] + v[i - 1];
            a[pos(i)] = pre[i] - pre[i & (i - 1)];
        }
    }

    void init(int m)
    {
        // m 是元素个数；清空为 m 个零元素。
        n = m;
        a.assign(pos(n) + 1, T{});
    }

    void add(int x, T v)
    {
        // x 是 0 下标位置，v 是增量；把第 x 个元素增加 v。
        assert(0 <= x && x < n); // 调试检查，可删
        for (x++; x <= n; x += x & -x)
        {
            a[pos(x)] += v;
        }
    }

    T sum(int x) const
```

```
    {
        // x 是前缀长度；返回 [0,x) 的和。
        assert(0 <= x && x <= n); // 调试检查，可删
        T ans{};
        for (; x; x &= x - 1)
        {
            ans += a[pos(x)];
        }
        return ans;
    }

    T sum(int l, int r) const
    {
        // l、r 是半开区间端点；返回 [l,r) 的和。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return sum(r) - sum(l);
    }

    int kth(T k) const
    {
        // 各元素必须非负且 k>=0；返回满足 sum(x)<=k 的最大前缀长度 x。
        assert(k >= T{}); // 调试检查，可删
        int x = 0;
        T cur{};
        for (int d = bit_floor((unsigned long long)n); d; d >>= 1)
        {
            if (x + d <= n && cur + a[pos(x + d)] <= k)
            {
                x += d;
                cur += a[pos(x)];
            }
        }
        return x;
    }
};
```

`BIT<T> b(n)`：`add(x,v)` 给 0 下标位置 x 加 v，`sum(x)` 返回前缀 $[0,x)$ 之和，`sum(l,r)` 返回 $[l,r)$，`kth(k)` 返回满足 `sum(x)<=k` 的最大前缀长度 $x\in[0,n]$，要求各点权非负且 $k\geq 0$；若总和也不超过 k，结果就是 n。全部操作 $O(\log n)$，构造与空间 $O(n)$。`FastBIT<T>` 语义完全相同，由数组构造时是 $O(n)$；本机两百万次随机更新/前缀查询的 7 轮中位数约为 `29ms`，普通连续布局约为 `69ms`。小数据优先用更短的 `BIT`，大数据热路径再换 `FastBIT`。需要单点赋值而非增量见 3.2.002 单点赋值、区间和，区间加与区间和改用 3.2.004 区间修改、区间求和，二维点加矩形和用 3.2.008 二维树状数组、矩形加矩形和升级为 3.2.009 矩形修改、矩形求和；只有不可逆的单调最大值才选 3.2.005 前缀/后缀最大值。

3.2.002 单点赋值、区间和

树状数组天生做增量，要支持「把某点赋成新值」需要一层薄封装：额外用 `a[x]` 记住每个位置的当前值，赋值时把它翻译成增量 `v - a[x]` 喂给底层 BIT，再更新 `a[x]`。关键坑是绝不能直接覆盖 `bit[x]`——那样会破坏所有包含该点的 lowbit 块；必须走「差值 + `add`」这条路。底层直接复用 3.2.001 单点修改、前缀

和 的 BIT<T>。

```cpp
// 依赖 3.2.001 单点修改、前缀和
template <class T> struct AssignBIT
{
    // n 保存数组长度；a 保存每个位置的当前值；bit 维护增量的前缀和。
    int n;
    vector<T> a;
    BIT<T> bit;

    AssignBIT(int n = 0) : n(n), a(n), bit(n)
    {
        // n 表示数组长度；初始值全部为零。
        assert(n >= 0); // 调试检查，可删
    }

    void set(int x, T v)
    {
        // x 表示 0 下标位置，v 表示新值；把赋值转成增量 v-a[x]。
        assert(0 <= x && x < n); // 调试检查，可删
        bit.add(x, v - a[x]);
        a[x] = v;
    }

    T sum(int l, int r) const
    {
        // l、r 表示半开区间 [l,r)；返回区间和。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return bit.sum(l, r);
    }
};
```

`AssignBIT<T> b(n)`（初值全零）：`set(x,v)` 把 0 下标位置 x 赋成 v，`sum(l,r)` 查 $[l,r)$。两者都是 $O(\log n)$，额外 $O(n)$ 存当前值。若本就只做增量、不需要赋值语义，直接用 3.2.001 单点修改、前缀和 更省一份数组。

3.2.003 前缀和上的二分

给定目标前缀和，在树状数组上直接沿二进制层级定位位置，比「外层二分 + 每步 `sum`」少一个 log。这一功能已经内建为 3.2.001 单点修改、前缀和 的 `BIT<T>::kth`，不另立代码：从最高位 $d = \text{bit_floor}(n)$ 向下试探，每步判断把当前下标右移 d 后累计的前缀和是否仍不超过目标 k，是则接受该跳跃。原理依赖前缀和单调，因此各点权必须非负，出现负权时前缀和不单调、二进制提升失效。`kth(k)` 返回使前 x 个元素之和不超过 k 的最大下标 x，单次 $O(\log n)$。典型用途是权值树状数组上求第 k 小、区间排名，功能更全的静态区间第 k 小见 3.4.006 区间第 K 小、排名、前驱后继。

3.2.004 区间修改、区间求和

要让区间加与区间求和都只需对数时间，对差分序列 $d_i = a_i - a_{i-1}$ 下手：区间加 $[l, r)$ $+= v$ 在差分上只改两个端点 d_l $+= v$、d_r $-= v$。前缀和 $\sum_{i<x} a_i = \sum_{i<x}(x-i)d_i = x\sum_{i<x} d_i - \sum_{i<x} i\,d_i$，于是维护两个 BIT——`bit0` 存 d_i、`bit1` 存 $i\,d_i$——即可由 $x \cdot \mathrm{sum}(bit0, x) - \mathrm{sum}(bit1, x)$ 还原任意前缀。区间加只在两端各更新两棵树，区间和是两个前缀相减。它比单纯用一棵 BIT 多存一倍空间，换来了区间修改能力。

```
template <class T> struct RangeFenwick
{
    // n 是数组长度；bit0 维护 d[i]，bit1 维护 i*d[i]。
    int n;
    vector<T> bit0, bit1;

    void add(vector<T> &bit, int x, T value)
    {
        // bit 是一棵内部 BIT，x 是差分下标，value 是增量。
        for (++x; x <= n; x += x & -x)
        {
            bit[x] += value;
        }
    }

    T sum(const vector<T> &bit, int x) const
    {
        // bit 是一棵内部 BIT；返回其前 x 个差分量之和。
        T result{};
        for (; x > 0; x -= x & -x)
        {
            result += bit[x];
        }
        return result;
    }

    explicit RangeFenwick(int n = 0) : n(n), bit0(n + 1), bit1(n + 1)
    {
        // n 是数组长度；初值全为零。
        assert(n >= 0); // 调试检查，可删
    }

    explicit RangeFenwick(const vector<T> &a) : RangeFenwick((int)a.size())
    {
        // a 是初始数组；逐点加入其初值。
        for (int i = 0; i < n; ++i)
        {
            rangeAdd(i, i + 1, a[i]);
        }
    }

    void rangeAdd(int l, int r, T value)
    {
```

```
        // l、r 是半开区间端点，value 是增量；给 [l,r) 整段增加 value。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        add(bit0, l, value);
        add(bit0, r, -value);
        add(bit1, l, value * l);
        add(bit1, r, -value * r);
    }

    T prefixSum(int r) const
    {
        // r 是前缀右端点；返回 [0,r) 的元素和。
        assert(0 <= r && r <= n); // 调试检查，可删
        return sum(bit0, r) * r - sum(bit1, r);
    }

    T rangeSum(int l, int r) const
    {
        // l、r 是半开区间端点；返回 [l,r) 的元素和。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return prefixSum(r) - prefixSum(l);
    }
};
```

`RangeFenwick<T> b(n)` 或由 `vector<T>` 构造（逐点 `rangeAdd`）：`rangeAdd(l,r,v)` 给 $[l,r)$ 每个元素加 v，`rangeSum(l,r)` 查区间和，`prefixSum(r)` 查前缀 $[0,r)$；端点断言 $0 \le l \le r \le n$。`T` 必须能表示负差分以及所有 id_i、前缀和中间量和答案，需要宽和时直接选用更宽的 `T`；由于实现会保存负差分，不应使用无符号整数。从 `vector` 构造为 $O(n \log n)$，其余操作 $O(\log n)$，空间 $O(n)$。只需单点改、区间查时退回 3.2.001 单点修改、前缀和 省一半常数；要在区间加之上再做区间最值、区间历史最值等复杂标记时，树状数组无能为力，应换 3.3.002 懒标记线段树 或 3.3.004 区间最值与历史最值。

3.2.005 前缀/后缀最大值

最大值没有逆元，所以树状数组维护前缀最大值时只能「单点抬高」而不能「任意改小」：`chmax` 沿 `x += x & -x` 把途经块与新值取 max，`maxPrefix` 沿 `x -= x & -x` 合并覆盖前缀的块最大值。若某次要把某点值调小，旧的最大值无法从块里减掉，必须重建受影响块或直接换线段树——因此本结构仅适用于点值单调不减的场景（如按位置维护 LIS 的 dp 值）。构造时的幺元 `e` 取类型最小值，作为空查询返回。改成前缀最小值时，成对替换 `max/lowest/chmax/maxPrefix` 为 `min/numeric_limits<T>::max()/chmin/minPrefix`，并把适用条件改为点值单调不增；不能只替换查询函数。

```
template <class T> struct MaxBIT
{
    // 当前是“单点只增大、前缀取最大值”版。改成最小值版时，
    // 将 max/lowest/chmax/maxPrefix 分别换成 min/numeric_limits<T>::max()/chmin/minPrefix，且更
    ↪ 新方向改为单点只减小。
    // n 是数组长度，e 是空前缀的幺元，a[i] 是 lowbit 块最大值。
    int n;
    T e;
    vector<T> a;
```

```

    MaxBIT(int n = 0, T e = numeric_limits<T>::lowest()) : n(n), e(e), a(n + 1, e)
    {
        // n 是数组长度，e 是无元素贡献时的返回值。
        assert(n >= 0); // 调试检查，可删
    }

    // 只支持把单点值增大；若需要任意修改应改用线段树。
    void chmax(int x, T v)
    {
        // x 是 0 下标位置，v 是候选新值；令该点不低于 v。
        assert(0 <= x && x < n); // 调试检查，可删
        for (++x; x <= n; x += x & -x)
        {
            a[x] = max(a[x], v);
        }
    }

    T maxPrefix(int r) const
    {
        // r 是前缀右端点；返回 [0,r) 的最大值。
        assert(0 <= r && r <= n); // 调试检查，可删
        T ans = e;
        for (; r; r -= r & -r)
        {
            ans = max(ans, a[r]);
        }
        return ans;
    }
};
```

`MaxBIT<T> b(n, e)`（`e` 默认取 `numeric_limits<T>::lowest()`）：`chmax(x,v)` 把 0 下标位置 x 的值上抬到不低于 v，`maxPrefix(r)` 返回前缀 $[0,r)$ 的最大值（注意接口按前缀最大值实现，只能查前缀而非任意区间）。两操作均 $O(\log n)$，空间 $O(n)$。一旦需要任意方向的单点修改或任意区间最值，就必须换 3.3.001 单点修改线段树与摘要二分；求和类需求回到 3.2.001 单点修改、前缀和。

3.2.006 后缀和

有时更自然的查询是「从某位置到末尾」的后缀和。与其重写一套反向循环，不如把 lowbit 的推进方向对调：`add` 沿 `x -= x & -x` 更新，`sumSuffix` 沿 `x += x & -x` 累加，等价于在反转下标上跑标准 BIT，原数组后缀恰好对应 BIT 前缀。这样修改与查询共用同一套 lowbit 逻辑，不必同时维护两套相反循环。

```
template <class T> struct SuffixBIT
{
    // n 是数组长度；a[i] 是反向 lowbit 块和。
    int n;
    vector<T> a;

    SuffixBIT(int n = 0) : n(n), a(n + 1)
```

```
    {
        // n 是数组长度；初值全为零。
        assert(n >= 0); // 调试检查，可删
    }

    void add(int x, T v)
    {
        // x 是 0 下标位置，v 是增量；把该点增加 v。
        assert(0 <= x && x < n); // 调试检查，可删
        for (++x; x; x -= x & -x)
        {
            a[x] += v;
        }
    }

    T sumSuffix(int l) const
    {
        // l 是后缀左端点；返回 [l,n) 的和，允许 l=n。
        assert(0 <= l && l <= n); // 调试检查，可删
        T ans{};
        for (++l; l <= n; l += l & -l)
        {
            ans += a[l];
        }
        return ans;
    }

    T sum(int l, int r) const
    {
        // l、r 是半开区间端点；返回 [l,r) 的和。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return sumSuffix(l) - sumSuffix(r);
    }
};
```

`SuffixBIT<T> b(n)`：`add(x,v)` 给 0 下标位置 x 加 v，`sumSuffix(l)` 返回后缀 $[l,n)$ 之和，`sum(l,r)` 返回 $[l,r)$（由两个后缀相减）。均 $O(\log n)$，空间 $O(n)$。本质与 3.2.001 单点修改、前缀和 完全对称，仅方向相反；同时要前缀又要后缀时，直接用一棵普通 BIT 加「总和减前缀」即可，无需两棵。

3.2.007 取模树状数组

这是一个配置型条目，不引入新结构：树状数组只要求值类型带零元、加法、减法，于是普通求和与模意义求和共用同一份 3.2.001 单点修改、前缀和 的 `BIT<T>`——`BIT<int>` 存普通整数，`BIT<MInt>`（模整数类型自动把每次加减归一到 $[0,\mathrm{mod})$）存模意义前缀和，二者的结构、lowbit 推进与复杂度完全一致，不该复制第二份 BIT，只用一行 `using` 起个语义别名。

```
// 依赖 3.2.001 单点修改、前缀和
template <class MInt> using ModBIT = BIT<MInt>;
```

`ModBIT<MInt> b(n)` 即 `BIT<MInt>`，接口与 3.2.001 单点修改、前缀和 相同，各操作 $O(\log n)$、空间

$O(n)$；模板参数换成 `int` 便退回普通版。举例：模 7、依次在下标 $0,2,2$ 加 $5,6,4$，则 `sum(0,3)` 得 $(5+0+10) \bmod 7 = 1$、区间 $[1,3)$ 之和为 $10 \bmod 7 = 3$。模整数类型 `MInt` 的完整实现见数论 / 组合数学章。

3.2.008 二维树状数组

二维点集上的单点加权、矩形和：把一维 lowbit 套两层，节点 `tr[i][j]` 保存两段 lowbit 张成的小矩形和，修改在两维都沿 `+= lowbit` 上行、前缀查询在两维都沿 `-= lowbit` 下行，任意矩形和由四个二维前缀容斥。坐标范围大时不能开满二维数组，故这里采用「离线离散 + 树套树」：先由所有可能的更新点确定外层横坐标离散表，再对每个外层 BIT 节点 i 收集它会访问到的纵坐标，各自离散成一条内层 BIT；因此更新点必须在构造时全部给出，之后 `add` 的 (x, y) 只能落在预登记的坐标上。

```cpp
class BIT2
{
    // xs 是可更新横坐标；ys[i] 是外层节点 i 收集的纵坐标。
    vector<int> xs;
    vector<vector<int>> ys;
    // tr[i] 是外层节点 i 对应的纵坐标 BIT。
    vector<vector<long long>> tr;

  public:
    explicit BIT2(const vector<pair<int, int>> &pts = {})
    {
        // pts 列出所有可能发生更新的二维坐标。
        prepare(pts);
    }

    void prepare(const vector<pair<int, int>> &pts)
    {
        // pts 列出所有可能更新点；清空旧权值并重建两层离散索引。
        xs.clear();
        for (auto [x, y] : pts)
        {
            xs.push_back(x);
        }
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
        int n = xs.size();
        ys.assign(n + 1, {});
        for (auto [x, y] : pts)
        {
            int p = lower_bound(xs.begin(), xs.end(), x) - xs.begin() + 1;
            for (int i = p; i <= n; i += i & -i)
            {
                ys[i].push_back(y);
            }
        }
        tr.resize(n + 1);
        for (int i = 1; i <= n; ++i)
        {
```

```
            auto &v = ys[i];
            sort(v.begin(), v.end());
            v.erase(unique(v.begin(), v.end()), v.end());
            tr[i].assign(v.size() + 1, 0);
        }
    }

    void add(int x, int y, long long v)
    {
        // (x,y) 必须在构造时登记，v 是该点的权值增量。
        int n = xs.size();
        auto it = lower_bound(xs.begin(), xs.end(), x);
        assert(it != xs.end() && *it == x); // 调试检查，可删
        int p = it - xs.begin() + 1;
        for (int i = p; i <= n; i += i & -i)
        {
            auto jt = lower_bound(ys[i].begin(), ys[i].end(), y);
            assert(jt != ys[i].end() && *jt == y); // 调试检查，可删
            int q = jt - ys[i].begin() + 1;
            for (int j = q; j < (int)tr[i].size(); j += j & -j)
            {
                tr[i][j] += v;
            }
        }
    }

    long long prefixSum(int x, int y) const
    {
        // x、y 是开前缀上界；返回横坐标小于 x 且纵坐标小于 y 的点权和。
        int p = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        long long ans = 0;
        for (int i = p; i > 0; i -= i & -i)
        {
            int q = lower_bound(ys[i].begin(), ys[i].end(), y) - ys[i].begin();
            for (int j = q; j > 0; j -= j & -j)
            {
                ans += tr[i][j];
            }
        }
        return ans;
    }

    void addRectangle(int l, int d, int r, int u, long long w)
    {
        // [l,r) x [d,u) 是半开加权矩形，w 是增量；四角必须已在构造时预登记。
        assert(l < r && d < u); // 调试检查，可删
        add(l, d, w);
        add(l, u, -w);
        add(r, d, -w);
        add(r, u, w);
    }

```

```
    long long pointGet(int x, int y) const
    {
        // (x,y) 是查询点；返回所有包含该点的半开加权矩形的权值和。
        return prefixSum(x + 1, y + 1);
    }
};
```

`BIT2 b(updatePoints)` 传入所有将被 `add` 的坐标（可含重复）；`add(x,y,v)` 在预登记点 (x, y) 加 v，`prefixSum(X,Y)` 返回 $x < X \wedge y < Y$ 的点权和，矩形 $[x_1, x_2) \times [y_1, y_2)$ 用四个 `prefixSum` 容斥。若反复需要“矩形加、单点值”这一固定组合，`addRectangle(l,d,r,u,w)` 会将严格非空半开矩形拆成四个差分角，`pointGet(x,y)` 返回其在整数点 (x, y) 的差分前缀；四个角同样必须预登记，查询点不必登记。它们不增加第二棵树或额外状态，只避免四角符号、端点开闭写错。设更新点数 q，离线建表 $O(q \log^2 q)$，`add`、`prefixSum`、`addRectangle` 与 `pointGet` 均为 $O(\log^2 q)$，空间 $O(q \log q)$。它原生支持单点加、矩形和；矩形加、单点值可用上述差分入口，矩形加、矩形和需要四棵二维 BIT 做二维差分，见 3.2.009 矩形修改、矩形求和。坐标可完全在线、还要带修矩形最值时改用 3.3.013 矩形面积并、周长并 一类扫描线或 KD-Tree 3.7.004 K-D Tree。

3.2.009 矩形修改、矩形求和

矩形加、矩形和：对二维差分 $d_{x,y}$ 下手，一次矩形加 $[x_1, x_2) \times [y_1, y_2)$ $+= w$ 拆成四个角上的差分事件。二维前缀和 $\sum_{x<X,\, y<Y} a = \sum (X-x)(Y-y) d_{x,y}$ 展开为 $XY \sum d - X \sum yd - Y \sum xd + \sum xyd$ 四项，于是需维护四棵二维 BIT 分别累计 d、yd、xd、xyd，查询时按上式组合再对矩形做四角容斥。本实现走完全离线：把所有矩形加与查询各拆成四个角点，按 x 排成扫描线、纵坐标离散后只用四棵一维 BIT 边扫边累加，每个查询结果按 $\pm$ 系数汇总。

```
template <class T> struct RectAddSum
{
    struct R
    {
        // l、d、r、u 依次是加权矩形的左、下、右、上边界。
        long long l, d, r, u;
        // w 是矩形内每个位置增加的权值。
        T w;
    };

    struct Q
    {
        // l、d、r、u 依次是询问矩形的左、下、右、上边界。
        long long l, d, r, u;
    };

    // a 保存全部矩形加操作，q 保存全部矩形和询问。
    vector<R> a;
    vector<Q> q;

    void add(long long l, long long d, long long r, long long u, T w)
    {
        // 给半开矩形 [l,r) x [d,u) 内每个位置增加 w；退化矩形无操作。
```

```
        if (l < r && d < u)
        {
            a.push_back({l, d, r, u, w});
        }
    }

    int query(long long l, long long d, long long r, long long u)
    {
        // 登记半开矩形 [l,r) x [d,u) 的权值和询问并返回询问编号。
        q.push_back({l, d, r, u});
        return q.size() - 1;
    }

    vector<T> work() const
    {
        // 统一处理全部操作；返回值按 query 的登记编号排列。
        struct E
        {
            // x、y 是二维差分事件坐标，w 是该点的带符号权值。
            long long x, y;
            T w;
        };
        struct P
        {
            // x、y 是前缀询问上界；id 是原询问编号，s 是容斥符号。
            long long x, y;
            int id, s;
        };

        vector<E> e;
        vector<P> p;
        vector<long long> ys;
        auto addE = [&](long long x, long long y, T w)
        {
            e.push_back({x, y, w});
            ys.push_back(y);
        };
        for (auto [l, d, r, u, w] : a)
        {
            addE(l, d, w);
            addE(l, u, -w);
            addE(r, d, -w);
            addE(r, u, w);
        }
        auto addP = [&](long long x, long long y, int id, int s)
        {
            p.push_back({x, y, id, s});
            ys.push_back(y);
        };
        for (int i = 0; i < (int)q.size(); i++)
        {
            auto [l, d, r, u] = q[i];
```

```
            addP(r, u, i, 1);
            addP(l, u, i, -1);
            addP(r, d, i, -1);
            addP(l, d, i, 1);
        }
        sort(e.begin(),
             e.end(),
             [](const E &x, const E &y)
             {
                 return x.x < y.x;
             });
        sort(p.begin(),
             p.end(),
             [](const P &x, const P &y)
             {
                 return x.x < y.x;
             });
        sort(ys.begin(), ys.end());
        ys.erase(unique(ys.begin(), ys.end()), ys.end());

        struct B
        {
            // a 保存一棵扫描线纵轴 BIT 的 lowbit 块和。
            vector<T> a;

            B(int n) : a(n + 1)
            {
                // n 是离散纵坐标数；初值全为零。
            }
            void add(int x, T v)
            {
                // x 是离散纵坐标，v 是增量；更新该点。
                for (++x; x < (int)a.size(); x += x & -x)
                {
                    a[x] += v;
                }
            }
            T sum(int x) const
            {
                // x 是前缀长度；返回前 x 个离散位置的和。
                T s{};
                for (; x; x -= x & -x)
                {
                    s += a[x];
                }
                return s;
            }
        };

        array<B, 4> b{B(ys.size()), B(ys.size()), B(ys.size()), B(ys.size())};
        vector<T> ans(q.size());
        int k = 0;
```

```
        for (auto z : p)
        {
            while (k < (int)e.size() && e[k].x < z.x)
            {
                auto [x, y, w] = e[k++];
                int v = lower_bound(ys.begin(), ys.end(), y) - ys.begin();
                b[0].add(v, w);
                b[1].add(v, w * T(y));
                b[2].add(v, w * T(x));
                b[3].add(v, w * T(x) * T(y));
            }
            int y = lower_bound(ys.begin(), ys.end(), z.y) - ys.begin();
            T v = T(z.x) * T(z.y) * b[0].sum(y) - T(z.x) * b[1].sum(y) - T(z.y) * b[2].sum(y) +
 ↪  b[3].sum(y);
            ans[z.id] += T(z.s) * v;
        }
        return ans;
    }
};
```

用法分两阶段：先用 `add(x1,y1,x2,y2,w)` 登记矩形加、`query(x1,y1,x2,y2)` 登记查询（返回该查询的编号），登记完调用 `work()` 一次性离线算出 `vector<T>`，其下标即 `query` 的返回值。区间左闭右开，退化的加法矩形被忽略，退化询问自然得到零；坐标与权值的乘积、中间和及答案都必须能由 `T` 表示。设矩形数 n、查询数 q，时间 $O((n+q)\log(n+q))$、空间 $O(n+q)$。举例：先对 $[0,2)\times[0,3)$ 加 4、再对 $[1,4)\times[2,5)$ 加 -2，查询 $[0,4)\times[0,5)$、$[1,2)\times[2,3)$、空矩形，`work()` 依次返回 $6,2,0$。只做单点加、矩形和时用离线树套树 3.2.008 二维树状数组 更省；强制在线的矩形加矩形和需要二维树套树或其他在线二维结构，本条不能处理依赖前一次答案的操作。

3.2.010 静态点集矩形和

静态带权点集的矩形和只需一次离线扫描，不该用查询最坏 $O(n)$ 的 KD-Tree。把每个半开矩形 $[l,r)\times[d,u)$ 拆为横向前缀差：横坐标小于 r 的纵向区间和减去横坐标小于 l 的纵向区间和。点和前缀事件都按横坐标排序，扫描到事件 X 时加入所有 $x<X$ 的点；点的纵坐标离散后，一棵 Fenwick 维护已加入点的权值，两个 `lower_bound` 得到 $[d,u)$。因此端点相同的点不会被错误计入左闭右开边界。

```
template <class T> struct StaticRect
{
    struct P
    {
        // x、y 是静态点坐标，w 是该点权值。
        long long x, y;
        T w;
    };

    struct Q
    {
        // l、d、r、u 依次是半开查询矩形的左、下、右、上边界。
        long long l, d, r, u;
    };
```

```
    // p 保存全部静态带权点，q 保存全部待回答的矩形询问。
    vector<P> p;
    vector<Q> q;

    void add(long long x, long long y, T w)
    {
        // 把坐标为 (x,y)、权值为 w 的静态点加入点集。
        p.push_back({x, y, w});
    }

    int query(long long l, long long d, long long r, long long u)
    {
        // 登记半开矩形 [l,r) x [d,u) 的点权和询问，返回其编号。
        q.push_back({l, d, r, u});
        return (int)q.size() - 1;
    }

    vector<T> work() const
    {
        // 离线回答全部登记询问，返回值按 query 的返回编号排列。
        struct E
        {
            // x 是扫描线横坐标；d、u 是纵向半开范围；id 是询问编号，sgn 是容斥符号。
            long long x, d, u;
            int id, sgn;
        };

        vector<P> a = p;
        vector<E> e;
        e.reserve(2 * q.size());
        for (int i = 0; i < (int)q.size(); ++i)
        {
            auto [l, d, r, u] = q[i];
            e.push_back({l, d, u, i, -1});
            e.push_back({r, d, u, i, 1});
        }
        sort(a.begin(), a.end(), [](const P &x, const P &y)
             {
                 return x.x < y.x;
             });
        sort(e.begin(), e.end(), [](const E &x, const E &y)
             {
                 return x.x < y.x;
             });

        vector<long long> ys;
        ys.reserve(a.size());
        for (auto [x, y, w] : a)
        {
            ys.push_back(y);
        }
```

```
        sort(ys.begin(), ys.end());
        ys.erase(unique(ys.begin(), ys.end()), ys.end());
        vector<T> bit(ys.size() + 1);
        auto addBit = [&](int x, T w)
        {
            // x 是 0 下标离散纵坐标，w 是加入该点的权值。
            for (++x; x < (int)bit.size(); x += x & -x)
            {
                bit[x] += w;
            }
        };
        auto sumBit = [&](int x)
        {
            // x 是离散前缀长度，返回所有离散下标小于 x 的权值和。
            T ans{};
            for (; x; x -= x & -x)
            {
                ans += bit[x];
            }
            return ans;
        };

        vector<T> ans(q.size());
        int at = 0;
        for (auto [x, d, u, id, sgn] : e)
        {
            while (at < (int)a.size() && a[at].x < x)
            {
                int y = lower_bound(ys.begin(), ys.end(), a[at].y) - ys.begin();
                addBit(y, a[at].w);
                ++at;
            }
            int l = lower_bound(ys.begin(), ys.end(), d) - ys.begin();
            int r = lower_bound(ys.begin(), ys.end(), u) - ys.begin();
            T cur = sumBit(r);
            cur -= sumBit(l);
            if (sgn == 1)
            {
                ans[id] += cur;
            }
            else
            {
                ans[id] -= cur;
            }
        }
        return ans;
    }
};
```

先用 `add(x,y,w)` 登记全部静态点，再用 `query(l,d,r,u)` 登记半开矩形并取得编号，最后调用一次 `work()`；答案向量按编号排列。点可重合、权值可为负，`T` 必须默认构造且支持 `+=`、`-=`；坐标、累计权值和答案均须能由相应类型表示。设点数为 n、询问数为 q，排序和扫描总时间为 $O((n+q)\log(n+q))$，额外空

间为 $O(n+q)$。需要在线单点加时用 3.2.008 二维树状数组，需要矩形加则用 3.2.009 矩形修改、矩形求和，静态点集但允许较松平均复杂度的多种几何查询才考虑 3.7.004 K-D Tree。

3.2.011 矩形加、单点求值

这是 3.2.008 二维树状数组 的四角差分入口，不另造第二棵结构。一次半开矩形 $[l,r)\times[d,u)$ 加 w，在二维差分上写 $(l,d):+w$、$(l,u):-w$、$(r,d):-w$、$(r,u):+w$；因此任意整数点 (x,y) 的当前值正是所有横坐标不超过 x、纵坐标不超过 y 的差分点权和。离线先把初始矩形及每条 0 型更新的四个角都收集进 `updatePoints`，再构造 `BIT2`；查询点无需登记。

```
// updatePoints 已收集每个初始和未来矩形的四个角。
BIT2 bit(updatePoints);
bit.addRectangle(l, d, r, u, w); // 给 [l,r) x [d,u) 加 w。
auto ans = bit.pointGet(x, y);   // 取整数点 (x,y) 的当前值。
```

`addRectangle(l,d,r,u,w)` 要求 $l<r,d<u$，四个角必须在构造 `BIT2` 时预登记；`pointGet(x,y)` 不要求查询点预登记，按题面半开边界返回恰好包含该点的所有矩形权值和。设 U 是去重前的全部矩形角数，预处理 $O(U\log^2 U)$，每次矩形加与点查询均为 $O(\log^2 U)$，空间 $O(U\log U)$；答案及中间差分和必须可由 `long long` 表示。要查询整个矩形的加权和而不只是单点，改用 3.2.009 矩形修改、矩形求和；若矩形或查询坐标不能预读，当前离线树套树不适用。

3.3 线段树家族

3.3.001 单点修改线段树与摘要二分

维护满足结合律的区间信息，支持单点修改与半开区间查询：叶子挂在不小于 n 的最小二次幂基址下，父节点存左右儿子合并的结果，`set` 只沿根到该叶的链自底向上重算，`query` 把 $[l,r)$ 拆成 $O(\log n)$ 段。查询里用 `x`、`y` 分别累计左边界与右边界扫上来的贡献，最后 `x + y`，这样即使合并不满足交换律（最大子段和、区间赋值后的仿射信息等）也能保持左右顺序正确。信息类型 `Info` 只需给出合并单位元（默认构造）与 `operator+`，示例 `Sum` 维护区间和，`MaxInfo` 可维护区间最大值。`findFirst` / `findLast` 是在同一棵树上的摘要搜索：谓词返回假时，必须保证该节点内没有答案；例如 `MaxInfo` 查第一个不小于 `x` 的位置时传入 `info.val >= x`。搜索遇到完全落入 $[l,r)$ 的节点时先用摘要剪枝，随后分别左优先或右优先递归，和 jiangly 模板的剪枝语义一致。这是自底向上的迭代维护配递归搜索，空间 $O(2\,\mathrm{pow2}(n))=O(n)$，代价是不带懒标记、仅支持单点改。

```
// 区间和实例：Sum{} 是合并单位元，Sum{x} 表示一个值为 x 的叶子。
struct Sum
{
    // val 是当前区间的元素和。
    long long val = 0;

    Sum() = default;
    Sum(long long x) : val(x)
    {
        // x 是单个叶子的值。
    }
```

```
12
13      friend Sum operator+(Sum a, Sum b)
14      {
15          // a、b 是相邻区间信息；返回合并后的区间和。
16          return a.val + b.val;
17      }
18  };
19
20  struct MaxInfo
21  {
22      // 当前信息取区间最大值。改成最小值信息时，把 lowest 改为 numeric_limits<long long>::max()、max
        ↪ 改为 min，
23      // 并同步把类型名、查询谓词中的 >= 改成 <=。
24      // val 是区间最大值；空区间以 long long 最小值作为 max 的单位元。
25      long long val = numeric_limits<long long>::lowest();
26
27      MaxInfo() = default;
28      MaxInfo(long long x) : val(x)
29      {
30          // x 是单个叶子的值。
31      }
32
33      friend MaxInfo operator+(MaxInfo a, MaxInfo b)
34      {
35          // a、b 是按顺序相邻的区间信息；返回两段的最大值。
36          return max(a.val, b.val);
37      }
38  };
39
40  template <class Info> struct Seg
41  {
42      // n 是真实叶子数，size 是不小于 n 的二次幂；tr 保存区间信息。
43      int n;
44      int size;
45      vector<Info> tr;
46
47      Seg(int n = 0)
48      {
49          // n 是叶子数；初始信息均为合并单位元。
50          init(n);
51      }
52      Seg(const vector<Info> &a)
53      {
54          // a 是各叶子的初始信息。
55          init(a);
56      }
57
58      void init(int n_)
59      {
60          // n_ 是新的叶子数；幂次二基址让每个内部节点对应连续区间。
61          assert(n_ >= 0); // 调试检查，可删
62          n = n_;
```

```
        size = 1;
        while (size < n)
        {
            size <<= 1;
        }
        tr.assign(2 * size, Info{});
    }

    void init(const vector<Info> &a)
    {
        // a 是新的叶子信息；线性重建整棵树。
        init(a.size());
        copy(a.begin(), a.end(), tr.begin() + size);
        for (int i = size - 1; i > 0; --i)
        {
            pull(i);
        }
    }

    void pull(int p)
    {
        // p 是内部节点编号；按左右顺序合并两个儿子。
        tr[p] = tr[2 * p] + tr[2 * p + 1];
    }

    void set(int p, Info v)
    {
        // p 是 0 下标位置，v 是新叶子信息；修改后重算祖先。
        assert(0 <= p && p < n); // 调试检查，可删
        for (tr[p += size] = v; p >>= 1;)
        {
            pull(p);
        }
    }

    Info query(int l, int r) const
    {
        // l、r 是半开区间端点；按原顺序返回 [l,r) 的合并结果。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        Info x{}, y{};
        for (l += size, r += size; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1)
            {
                x = x + tr[l++];
            }
            if (r & 1)
            {
                y = tr[--r] + y;
            }
        }
        return x + y;
```

```
    }

    template <class F>
    int findFirst(int p, int l, int r, int ql, int qr, F &ok) const
    {
        // p 管辖 [l,r)，查询区间 [ql,qr)，ok 对区间信息单调（父真则至少一个儿子真）；
        // 返回 [ql,qr) 内使 ok(tr[叶]) 为真的最小下标，不存在返回 -1。
        if (r <= ql || qr <= l)
        {
            return -1;
        }
        if (ql <= l && r <= qr && !ok(tr[p]))
        {
            return -1;
        }
        if (r - l == 1)
        {
            return l < n ? l : -1;
        }
        int m = l + (r - l) / 2;
        int x = findFirst(2 * p, l, m, ql, qr, ok);
        return x == -1 ? findFirst(2 * p + 1, m, r, ql, qr, ok) : x;
    }

    template <class F>
    int findFirst(int l, int r, F ok) const
    {
        // l、r 是半开区间，ok(Info) 判断区间是否含目标；返回最左满足的叶子下标，不存在返回 -1。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return l == r ? -1 : findFirst(1, 0, size, l, r, ok);
    }

    template <class F>
    int findLast(int p, int l, int r, int ql, int qr, F &ok) const
    {
        // 参数同 findFirst 的递归版；返回 [ql,qr) 内使 ok 为真的最大下标，不存在返回 -1。
        if (r <= ql || qr <= l)
        {
            return -1;
        }
        if (ql <= l && r <= qr && !ok(tr[p]))
        {
            return -1;
        }
        if (r - l == 1)
        {
            return l < n ? l : -1;
        }
        int m = l + (r - l) / 2;
        int x = findLast(2 * p + 1, m, r, ql, qr, ok);
        return x == -1 ? findLast(2 * p, l, m, ql, qr, ok) : x;
    }
```

```
167
168     template <class F>
169     int findLast(int l, int r, F ok) const
170     {
171         // l、r 是半开区间，ok(Info) 判断区间是否含目标；返回最右满足的叶子下标，不存在返回 -1。
172         assert(0 <= l && l <= r && r <= n); // 调试检查，可删
173         return l == r ? -1 : findLast(1, 0, size, l, r, ok);
174     }
175 };
```

用 `Seg<Sum> seg(a)` 由 `vector<Sum>` 建树，构造 $O(n)$；`set(p, v)` 把下标 p 改成 `Info v`，`query(l, r)` 返回 $[l, r)$ 的合并结果，`set` 和 `query` 为 $O(\log n)$，空间 $O(n)$。`findFirst/findLast` 只要求“谓词为假就一定无解”时正确，但允许摘要假阳性会使最坏时间退化为 $O(n)$；若 `ok(info)` 准确等价于该节点内存在答案，例如区间最大值能准确判断是否存在不小于 x 的元素，则搜索只走一条成功路径及其被剪掉的兄弟，时间为 $O(\log n)$。懒标记版本需要在搜索递归前 `push`，不能直接套本条函数。它与 3.3.005 ZKW 线段树 是同一类固定规模、单点修改、结合律查询的结构，ZKW 常数更小；一旦需要区间整体修改就得上懒标记，见 3.3.002 懒标记线段树；值域巨大且访问稀疏改 3.3.007 动态开点线段树，要查历史版本改 3.3.008 可持久化线段树 / 主席树。区间 chmin/chmax 不是普通懒标记，必须用 3.3.011 Segment Tree Beats。

`ok` 是调用者传入的摘要谓词，例如查第一个不小于 4 的位置：

```
1 signed main()
2 {
3     Seg<MaxInfo> seg(vector<MaxInfo>{{2}, {5}, {1}, {4}});
4     int p = seg.findFirst(0, 4, [](MaxInfo x)
5     {
6         return x.val >= 4;
7     });
8     assert(p == 1);
9 }
```

3.3.002 懒标记线段树

要支持区间整体修改，需在递归线段树上挂懒标记：一次落到完整整段的修改先把效果 `apply` 到该节点的聚合信息、并把标记复合进 `tag`，直到后续要递归进儿子前才 `push` 下传。正确性系于两条：`Info::apply(Tag)` 必须能仅凭区间长度 `len` 把标记作用到聚合值（所以 `Info` 存了 `len`），且标记复合 `Tag::apply(Tag)` 的方向要与真实执行时间一致——「先有旧标记、再叠新标记」。这里以区间加、区间和为例，换题一般只改 `Tag` 与 `Info` 两个结构（例如加入乘法、赋值时复合规则更复杂，见下一条）。用 $4n$ 结点的堆式存储。

```
1 // 区间加、区间和实例。换题时通常只需要修改这两个结构。
2 struct Tag
3 {
4     // add 是尚未下传的区间增量。
5     long long add = 0;
6
7     // 先前标记后再执行 t：区间加直接累加。
8     void apply(const Tag &t)
```

```
    {
        // t 是后执行的区间加标记；与当前标记复合。
        add += t.add;
    }
};

struct Info
{
    // sum 是区间和，len 是区间叶子数。
    long long sum = 0;
    int len = 1;

    Info() = default; // 默认构造一个值为 0 的叶子信息
    Info(long long x) : sum(x), len(1)
    {
        // x 是叶子的初始值。
    }

    static Info identity()
    {
        // 无参数；返回长度为 0 的空区间信息，作为合并单位元。
        Info c;
        c.len = 0;
        return c;
    }

    // 把区间加标记作用到当前节点。
    void apply(const Tag &t)
    {
        // t 是区间加标记；更新当前区间和。
        sum += t.add * len;
    }

    friend Info operator+(Info a, Info b)
    {
        // a、b 是相邻区间信息；返回合并结果。
        Info c;
        c.sum = a.sum + b.sum;
        c.len = a.len + b.len;
        return c;
    }
};

template <class Info, class Tag> struct LazySeg
{
    // n 是叶子数；tr 保存已计入本节点修改的区间信息，tag 和 has 保存尚未下传给儿子的修改。
    int n;
    vector<Info> tr;
    vector<Tag> tag;
    vector<unsigned char> has;

    static Info identity()
```

```
61      {
62          // 无参数；Info 自带 identity() 时用它，否则用默认构造值作为合并单位元。
63          if constexpr (requires { Info::identity(); })
64              return Info::identity();
65          else
66              return Info{};
67      }
68
69      LazySeg(int n = 0)
70      {
71          // n 是叶子数；每个叶子由 Info{} 默认构造成初始信息。
72          init(n);
73      }
74      LazySeg(const vector<Info> &a)
75      {
76          // a 是各叶子的初始信息。
77          init(a);
78      }
79
80      void init(int n_)
81      {
82          // n_ 是新的叶子数；用 n_ 个 Info{} 重建默认数组。
83          assert(n_ >= 0); // 调试检查，可删
84          n = n_;
85          tr.assign(4 * max<int>(n, 1), Info{});
86          tag.assign(4 * max<int>(n, 1), Tag{});
87          has.assign(4 * max<int>(n, 1), 0);
88          if (n)
89          {
90              vector<Info> a(n);
91              build(1, 0, n, a);
92          }
93      }
94
95      void init(const vector<Info> &a)
96      {
97          // a 是新的叶子信息；清空旧状态并重建。
98          n = a.size();
99          tr.assign(4 * max<int>(n, 1), Info{});
100         tag.assign(4 * max<int>(n, 1), Tag{});
101         has.assign(4 * max<int>(n, 1), 0);
102         if (n)
103         {
104             build(1, 0, n, a);
105         }
106     }
107
108     void build(int p, int l, int r, const vector<Info> &a)
109     {
110         // p 是当前节点，[l,r) 是其区间，a 是叶子信息。
111         if (r - l == 1)
112         {
```

```
            tr[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(2 * p, l, m, a);
        build(2 * p + 1, m, r, a);
        pull(p);
    }

    void pull(int p)
    {
        // p 是内部节点编号；按左右顺序合并两个儿子。
        tr[p] = tr[2 * p] + tr[2 * p + 1];
    }

    void apply(int p, const Tag &v)
    {
        // p 是整段命中的节点，v 是本次区间修改标记。
        tr[p].apply(v);
        tag[p].apply(v);
        has[p] = 1;
    }

    void push(int p)
    {
        // p 是当前节点；仅在有待执行标记时把它下传给两个儿子。
        if (!has[p])
        {
            return;
        }
        apply(2 * p, tag[p]);
        apply(2 * p + 1, tag[p]);
        tag[p] = Tag{};
        has[p] = 0;
    }

    void apply(int p, int l, int r, int ql, int qr, const Tag &v)
    {
        // p、[l,r) 是当前节点，  [ql,qr) 是修改区间，v 是本次标记。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            apply(p, v);
            return;
        }
        push(p);
        int m = (l + r) / 2;
        apply(2 * p, l, m, ql, qr, v);
        apply(2 * p + 1, m, r, ql, qr, v);
```

```
        pull(p);
    }

    Info query(int p, int l, int r, int ql, int qr)
    {
        // p、[l,r) 是当前节点，  [ql,qr) 是询问区间；返回相交部分的信息。
        if (qr <= l || r <= ql)
        {
            return identity();
        }
        if (ql <= l && r <= qr)
        {
            return tr[p];
        }
        push(p);
        int m = (l + r) / 2;
        return query(2 * p, l, m, ql, qr) + query(2 * p + 1, m, r, ql, qr);
    }

    Info get(int p, int l, int r, int i) const
    {
        // p、[l,r) 是当前节点，i 是目标叶子下标；只读返回该叶子的最新信息。
        if (r - l == 1)
        {
            return tr[p];
        }
        int m = (l + r) / 2;
        Info ans;
        if (i < m)
        {
            ans = get(2 * p, l, m, i);
        }
        else
        {
            ans = get(2 * p + 1, m, r, i);
        }
        if (has[p])
        {
            ans.apply(tag[p]);
        }
        return ans;
    }

    template <class F>
    int findFirst(int p, int l, int r, int ql, int qr, F &ok)
    {
        // p、[l,r) 是当前节点，  [ql,qr) 是搜索区间；返回其中第一个满足条件的下标。
        if (r <= ql || qr <= l)
        {
            return -1;
        }
        if (ql <= l && r <= qr && !ok(tr[p]))
```

```
        {
            return -1;
        }
        if (r - l == 1)
        {
            return l < n ? l : -1;
        }
        push(p);
        int m = l + (r - l) / 2;
        int x = findFirst(2 * p, l, m, ql, qr, ok);
        return x == -1 ? findFirst(2 * p + 1, m, r, ql, qr, ok) : x;
    }

    template <class F>
    int findLast(int p, int l, int r, int ql, int qr, F &ok)
    {
        // p、[l,r) 是当前节点，  [ql,qr) 是搜索区间；返回其中最后一个满足条件的下标。
        if (r <= ql || qr <= l)
        {
            return -1;
        }
        if (ql <= l && r <= qr && !ok(tr[p]))
        {
            return -1;
        }
        if (r - l == 1)
        {
            return l < n ? l : -1;
        }
        push(p);
        int m = l + (r - l) / 2;
        int x = findLast(2 * p + 1, m, r, ql, qr, ok);
        return x == -1 ? findLast(2 * p, l, m, ql, qr, ok) : x;
    }

    void apply(int l, int r, const Tag &v)
    {
        // l、r 是半开区间端点，v 是区间修改标记。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        apply(1, 0, n, l, r, v);
    }
    Info query(int l, int r)
    {
        // l、r 是半开区间端点；返回 [l,r) 的信息。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return query(1, 0, n, l, r);
    }
    Info get(int i) const
    {
        // i 是叶子下标；只读返回该位置在所有待下传标记生效后的信息。
        assert(0 <= i && i < n); // 调试检查，可删
        return get(1, 0, n, i);
```

```
    }

    template <class F>
    int findFirst(int l, int r, F ok)
    {
        // l、r 是半开搜索区间，ok 判断节点摘要是否可能包含答案；返回第一个满足条件的下标。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return l == r ? -1 : findFirst(1, 0, n, l, r, ok);
    }

    template <class F>
    int findLast(int l, int r, F ok)
    {
        // l、r 是半开搜索区间，ok 判断节点摘要是否可能包含答案；返回最后一个满足条件的下标。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return l == r ? -1 : findLast(1, 0, n, l, r, ok);
    }
};
```

`LazySeg<Info, Tag> seg(a)` 由 `vector<Info>` 建树（$O(n)$），`apply(l, r, Tag{ ... })` 对 $[l,r)$ 施加修改，`query(l, r)` 返回区间聚合。节点摘要 `tr[p]` 已计入本节点标记，`has[p]` 为真时 `tag[p]` 尚未作用到儿子；部分区间操作和上二分才调用 `push`。`get(i) const` 不修改树，而是在从叶子回溯时按由内到外的顺序施加尚未下传的祖先标记，返回单个叶子。`findFirst` / `findLast` 返回指定区间内第一个或最后一个可能满足条件的位置；谓词为假时必须保证该节点内没有答案，搜索下传懒标记后再递归，若谓词能由节点摘要准确剪枝，搜索为 $O(\log n)$；若只保证假值时整段无解，接口仍正确但最坏可能退化到 $O(n)$，空间为 $O(n)$。上二分接口会修改懒标记的存放位置，所以不能通过 `const LazySeg` 调用。只做单点修改时用不带标记的 3.3.001 单点修改线段树与摘要二分 更省；需要同时支持加、乘、赋值等仿射修改见 3.3.003 区间加、乘、赋值（仿射懒标记），区间取 $\min$ / $\max$ 这类无法直接复合的标记必须用 3.3.011 Segment Tree Beats，历史版本见 3.3.008 可持久化线段树 / 主席树。

下面对 $[1,2,3,4]$ 做区间加并检查区间和、单点值与上二分：

```
signed main()
{
    LazySeg<Info, Tag> seg(vector<Info>{1, 2, 3, 4});
    seg.apply(1, 4, Tag{2}); // 序列变为 [1,4,5,6]。
    assert(seg.query(0, 4).sum == 16 && seg.query(1, 3).sum == 9);
    assert(seg.get(2).sum == 5);
}
```

3.3.003 区间加、乘、赋值（仿射懒标记）

区间加、乘、赋值这三种修改可以统一成一个仿射变换 $x \mapsto mul \cdot x + add$：区间加是 $(mul, add) = (1, x)$，区间乘是 $(x, 0)$，区间赋值是 $(0, x)$。仿射变换在复合下封闭，「先做当前标记、再做 t」的复合为 $mul \leftarrow mul \cdot t.mul$、$add \leftarrow add \cdot t.mul + t.add$（`AffineTag<T>::apply` 正是这条，方向不能反），作用到区间和则是 $sum \leftarrow sum \cdot mul + len \cdot add$。`LazySeg<AffineInfo<T>, AffineTag<T>>` 因而同时支持区间仿射修改和区间和；若题目只有区间仿射修改与单点查询，则 `AffPointSeg<T>` 不维护区间摘要，省去每次部分修改的 `pull`。两者都使用同一 `AffineTag<T>`，`T` 必须支持 $0, 1, +, \times$；整数可

取 `long long`，模题直接取本书的 `Z`，不再把模数逻辑散落到适配代码。异或、取模不落在同一仿射族里：异或需按位维护，取模要靠「区间最大值不超过模数就剪枝」的势能分析，不能硬塞进仿射公式。

```cpp
// 依赖 3.3.002 懒标记线段树
template <class T> struct AffineTag
{
    // mul、add 表示待执行变换 x -> mul*x+add。
    T mul = T(1), add{};

    // 已有变换后再执行 t。
    void apply(const AffineTag<T> &t)
    {
        // t 是后执行的仿射变换；把它复合到当前标记之后。
        mul *= t.mul;
        add = add * t.mul + t.add;
    }
};

template <class T> struct AffineInfo
{
    // sum 是区间和，len 是区间叶子数。
    T sum{};
    int len = 0;

    AffineInfo() = default;
    AffineInfo(const T &x) : sum(x), len(1)
    {
        // x 是单个叶子的初值。
    }

    AffineInfo(const T &s, int n) : sum(s), len(n)
    {
        // s 是当前区间元素和，n 是该区间叶子数。
    }

    void apply(const AffineTag<T> &t)
    {
        // t 是本次仿射标记；更新当前区间和。
        sum = sum * t.mul + T(len) * t.add;
    }

    friend AffineInfo operator+(AffineInfo a, AffineInfo b)
    {
        // a、b 是相邻区间信息；返回合并结果。
        AffineInfo c;
        c.sum = a.sum + b.sum;
        c.len = a.len + b.len;
        return c;
    }
};

template <class T> class AffPointSeg
```

```
{
    // n 是叶子数；a 保存初始点值；tag 和 has 保存尚未下传的仿射修改。
    int n; // n 是叶子数。
    vector<T> a;
    vector<AffineTag<T>> tag;
    vector<unsigned char> has;

    void apply(int p, const AffineTag<T> &v)
    {
        // p 是完整命中节点，v 是后执行的仿射变换；只复合该段延迟标记。
        tag[p].apply(v);
        has[p] = 1;
    }

    void push(int p)
    {
        // p 是内部节点；把较早的整段修改下传后，才允许对子段施加较晚修改。
        if (!has[p])
        {
            return;
        }
        apply(2 * p, tag[p]);
        apply(2 * p + 1, tag[p]);
        tag[p] = AffineTag<T>{};
        has[p] = 0;
    }

    void affine(int p, int l, int r, int ql, int qr, const AffineTag<T> &v)
    {
        // p、[l,r) 是当前节点，[ql,qr) 是修改区间，v 是后执行的仿射变换。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            apply(p, v);
            return;
        }
        push(p);
        int m = (l + r) / 2;
        affine(2 * p, l, m, ql, qr, v);
        affine(2 * p + 1, m, r, ql, qr, v);
    }

    T get(int p, int l, int r, int i) const
    {
        // p、[l,r) 是当前节点，i 是目标叶子下标；只读返回所有仿射修改后的点值。
        T ans;
        if (r - l == 1)
        {
            ans = a[i];
```

```
        }
        else
        {
            int m = (l + r) / 2;
            ans = i < m ? get(2 * p, l, m, i) : get(2 * p + 1, m, r, i);
        }
        if (has[p])
        {
            ans = ans * tag[p].mul + tag[p].add;
        }
        return ans;
    }

  public:
    AffPointSeg(vector<T> a) : n(a.size()), a(move(a))
    {
        // a 是初始点值；构造仅分配延迟标记，不维护区间摘要。
        tag.assign(4 * max<int>(n, 1), AffineTag<T>{});
        has.assign(4 * max<int>(n, 1), 0);
    }

    void affine(int l, int r, const T &mul, const T &add)
    {
        // l、r 是半开端点；把 [l,r) 内每个值变为 value*mul+add。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return;
        }
        affine(1, 0, n, l, r, {mul, add});
    }

    T get(int i) const
    {
        // i 是叶子下标；只读返回该点当前值。
        assert(0 <= i && i < n); // 调试检查，可删
        return get(1, 0, n, i);
    }
};

class HistMaxSeg
{
  public:
    struct Info
    {
        // cur、hist 分别是区间当前最大值和历次修改后的历史最大值。
        long long cur, hist;
    };

  private:
    static constexpr long long neg = numeric_limits<long long>::lowest() / 2; // neg 表示
    ↪ max-plus 意义下的负无穷，取一半防止相加溢出。
```

```
153     struct Tag
154     {
155         // cc、ch、hh、vc、vh 描述 max-plus 变换 (cur,hist,1) 的五个有限系数。
156         long long cc = 0, ch = neg, hh = 0, vc = neg, vh = neg;
157     };
158     int n; // n 是叶子数。
159     vector<Info> tr; // tr[p] 是节点 p 区间的当前最大值与历史最大值。
160     vector<Tag> lz; // lz[p] 是节点 p 尚未下传给儿子的复合标记。
161 
162     static long long plus(long long a, long long b)
163     {
164         // a、b 是普通值或负无穷；返回 max-plus 乘法。
165         return a == neg || b == neg ? neg : a + b;
166     }
167 
168     static Tag compose(const Tag &a, const Tag &b)
169     {
170         // a、b 是先后执行的标记；返回先 a 后 b 的复合标记。
171         return {plus(a.cc, b.cc),
172                 max(plus(a.cc, b.ch), plus(a.ch, b.hh)),
173                 plus(a.hh, b.hh),
174                 max(plus(a.vc, b.cc), b.vc),
175                 max({plus(a.vh, b.hh), plus(a.vc, b.ch), b.vh})};
176     }
177 
178     static Info apply(Info a, const Tag &t)
179     {
180         // a 是节点摘要，t 是区间标记；返回变换后的摘要。
181         return {max(plus(a.cur, t.cc), t.vc),
182                 max({plus(a.cur, t.ch), plus(a.hist, t.hh), t.vh})};
183     }
184 
185     void pull(int p)
186     {
187         // p 是内部节点；由两个儿子重算摘要。
188         tr[p] = {max(tr[2 * p].cur, tr[2 * p + 1].cur),
189                  max(tr[2 * p].hist, tr[2 * p + 1].hist)};
190     }
191 
192     void build(int p, int l, int r, const vector<long long> &a)
193     {
194         // p、[l,r) 是当前节点，a 是初始数组。
195         if (r - l == 1) return void(tr[p] = {a[l], a[l]});
196         int m = (l + r) / 2;
197         build(2 * p, l, m, a), build(2 * p + 1, m, r, a), pull(p);
198     }
199 
200     void apply(int p, const Tag &t)
201     {
202         // p 是整段命中节点，t 是后执行的标记。
203         tr[p] = apply(tr[p], t);
204         lz[p] = compose(lz[p], t);
```

```
205     }
206
207     void push(int p)
208     {
209         // p 是内部节点；把累计标记下传。
210         apply(2 * p, lz[p]), apply(2 * p + 1, lz[p]), lz[p] = {};
211     }
212
213     void update(int p, int l, int r, int ql, int qr, const Tag &t)
214     {
215         // p、[l,r) 是当前节点，[ql,qr) 是修改区间，t 是修改标记。
216         if (qr <= l || r <= ql) return;
217         if (ql <= l && r <= qr) return apply(p, t);
218         push(p);
219         int m = (l + r) / 2;
220         update(2 * p, l, m, ql, qr, t), update(2 * p + 1, m, r, ql, qr, t), pull(p);
221     }
222
223     Info query(int p, int l, int r, int ql, int qr)
224     {
225         // p、[l,r) 是当前节点，[ql,qr) 是询问区间；返回相交部分摘要。
226         if (qr <= l || r <= ql) return {neg, neg};
227         if (ql <= l && r <= qr) return tr[p];
228         push(p);
229         int m = (l + r) / 2;
230         Info a = query(2 * p, l, m, ql, qr), b = query(2 * p + 1, m, r, ql, qr);
231         return {max(a.cur, b.cur), max(a.hist, b.hist)};
232     }
233
234   public:
235     explicit HistMaxSeg(const vector<long long> &a) : n(a.size()), tr(4 * max<int>(n, 1),
      ↪  {neg, neg}), lz(4 * max<int>(n, 1))
236     {
237         // a 是非空初始数组；建立当前最大值与历史最大值线段树。
238         assert(n > 0); // 调试检查，可删
239         build(1, 0, n, a);
240     }
241
242     void add(int l, int r, long long v)
243     {
244         // l、r 是半开端点，v 是增量；给区间加 v 并记录修改后的历史峰值。
245         assert(0 <= l && l <= r && r <= n); // 调试检查，可删
246         update(1, 0, n, l, r, {v, v, 0, neg, neg});
247     }
248
249     void assign(int l, int r, long long v)
250     {
251         // l、r 是半开端点，v 是新值；给区间赋值并记录修改后的历史峰值。
252         assert(0 <= l && l <= r && r <= n); // 调试检查，可删
253         update(1, 0, n, l, r, {neg, neg, 0, v, v});
254     }
255
```

```
    Info query(int l, int r)
    {
        // l、r 是非空半开端点；返回区间当前最大值和历史最大值。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        return query(1, 0, n, l, r);
    }
};
```

用 `LazySeg<AffineInfo<T>, AffineTag<T>> seg(a)` 后，区间加 `seg.apply(l, r, {T(1), x})`、区间乘 `seg.apply(l, r, {x, T(0)})`、区间赋值 `seg.apply(l, r, {T(0), x})`，`seg.query(l, r).sum` 取区间和。仅有区间仿射和单点查询时，把初值存为 `vector<T>` 后构造 `AffPointSeg<T> seg(a)`，调用 `seg.affine(l,r,mul,add)` 和 `seg.get(i)`；它没有 `query`，不能替代通用 `LazySeg` 的区间和接口。完整命中的更新只复合节点标记，部分更新先 `push` 较早标记，保证后来子段更新的时间顺序；`get(i) const` 从叶到根施加尚未下传标记。显式 `has` 既不要求 `T` 支持相等比较，也防止 $(mul, add) = (0, 0)$ 的赋零操作被误判为没有待下传标记。两种结构构造 $O(n)$，每次修改与各自查询 $O(\log n)$，空间 $O(n)$。混用加、乘时务必让所有系数与 `sum` 在同一类型中；例如 Library Checker 的模题取 `T=Z`，有界整数题才取 `long long` 并自行保证不溢出。只有单一区间加时用 3.3.002 懒标记线段树 的最简 `Tag` 更清爽；值域很大、初值全零且访问稀疏时用 3.3.021 稀疏仿射区间和。区间 chmin/chmax 见 3.3.011 Segment Tree Beats。3.3.004 区间加、区间最小值 只维护当前最小值，不提供历史最值。

下面依次检查通用仿射区间和、仿射单点查询与历史最大值：

```
signed main()
{
    LazySeg<AffineInfo<int>, AffineTag<int>> seg(vector<AffineInfo<int>>{1, 2, 3, 4});
    seg.apply(0, 2, {1, 5}); // 加 5: [6,7,3,4]。
    seg.apply(1, 3, {2, 0}); // 乘 2: [6,14,6,4]。
    seg.apply(3, 4, {0, 7}); // 赋 7: [6,14,6,7]。
    assert(seg.query(0, 4).sum == 33 && seg.query(1, 3).sum == 20);

    AffPointSeg<int> ps(vector<int>{1, 2, 3, 4});
    ps.affine(0, 3, 2, 1);  // [3,5,7,4]。
    ps.affine(1, 4, 1, -2); // [3,3,5,2]。
    assert(ps.get(1) == 3 && ps.get(3) == 2);

    HistMaxSeg h(vector<int>{1, 5, 2});
    h.add(0, 3, -4); // [-3,1,-2]，历史最大值仍是 5。
    auto q = h.query(0, 3);
    assert(q.cur == 1 && q.hist == 5);
    h.assign(0, 1, 10);
    h.add(0, 1, -20); // a[0] 先变 10 再变 -10。
    q = h.query(0, 1);
    assert(q.cur == -10 && q.hist == 10);
}
```

3.3.004 区间加、区间最小值

`AddMinSeg` 维护当前数组的区间最小值：每个节点存最小值 `minimum` 与待下传增量 `lazy`，区间加时把二者同步增加，`push` 把父标记加到两个儿子，`pull` 取两儿子最小值。本条不维护历史时刻达到过的最小值；

历史最值还需要额外保存历史极值和懒标记执行过程中的最小前缀增量，不能把当前最值接口当作历史查询使用。若改成区间最大值，`mn/rangeMin/min/`正无穷单位元必须一起换成 `mx/rangeMax/max/`负无穷，区间加和懒标记不变。

```
class AddMinSeg
{
    // 当前是区间加、区间最小值版。改成最大值只需把 mn/rangeMin/min/正无穷单位元
    // 成对换成 mx/rangeMax/max/负无穷；区间加和懒标记逻辑不变。
    // n 是数组长度；mn、lz 分别保存区间最小值与待下传增量。
    int n;
    vector<long long> mn, lz;

    void build(int p, int l, int r, const vector<long long> &a)
    {
        // p 是当前节点，[l,r) 是其区间，a 是初始数组。
        if (r - l == 1)
        {
            mn[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(2 * p, l, m, a);
        build(2 * p + 1, m, r, a);
        pull(p);
    }

    void apply(int p, long long v)
    {
        // p 是整段命中的节点，v 是区间增量。
        mn[p] += v;
        lz[p] += v;
    }

    void push(int p)
    {
        // p 是当前节点；把待执行增量下传给两个儿子。
        if (!lz[p])
        {
            return;
        }
        apply(2 * p, lz[p]);
        apply(2 * p + 1, lz[p]);
        lz[p] = 0;
    }

    void pull(int p)
    {
        // p 是内部节点；由两个儿子的最小值重算 mn[p]。
        mn[p] = min(mn[2 * p], mn[2 * p + 1]);
    }

    void add(int p, int l, int r, int ql, int qr, long long v)
```

```
    {
        // p、[l,r) 是当前节点，[ql,qr) 是修改区间，v 是增量。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            apply(p, v);
            return;
        }
        push(p);
        int m = (l + r) / 2;
        add(2 * p, l, m, ql, qr, v);
        add(2 * p + 1, m, r, ql, qr, v);
        pull(p);
    }

    long long rangeMin(int p, int l, int r, int ql, int qr)
    {
        // p、[l,r) 是当前节点；返回 [ql,qr) 内的最小值。
        if (qr <= l || r <= ql)
        {
            return numeric_limits<long long>::max();
        }
        if (ql <= l && r <= qr)
        {
            return mn[p];
        }
        push(p);
        int m = (l + r) / 2;
        return min(rangeMin(2 * p, l, m, ql, qr), rangeMin(2 * p + 1, m, r, ql, qr));
    }

  public:
    explicit AddMinSeg(const vector<long long> &a)
        : n(a.size()), mn(4 * max<int>(n, 1)), lz(4 * max<int>(n, 1))
    {
        // a 是初始数组；空数组建立空结构。
        if (n)
        {
            build(1, 0, n, a);
        }
    }

    void add(int l, int r, long long v)
    {
        // 给半开区间 [l,r) 增加 v；空区间无操作。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return;
```

```
        }
        add(1, 0, n, l, r, v);
    }

    long long rangeMin(int l, int r)
    {
        // 返回非空半开区间 [l,r) 的最小值。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        return rangeMin(1, 0, n, l, r);
    }
};
```

`AddMinSeg seg(values)` 由 `vector<long long>` 建树，空数组也能构造；`add(l, r, v)` 对 $[l, r)$ 区间加并允许空区间，`rangeMin(l, r)` 查询时要求区间非空。两种操作均为 $O(\log n)$，空间 $O(n)$。求和类换 3.3.002 懒标记线段树；区间对某值取 $\min$ / $\max$（不是加）这类操作要靠势能分析，见 3.3.011 Segment Tree Beats。

3.3.005 ZKW 线段树

ZKW 线段树把叶子铺在连续数组 `tr[n..2n)` 上（`n` 取不小于原长的 2 的幂），全程自底向上迭代、无递归、无懒标记，因而缓存友好、常数小于递归版。`set` 改叶子后沿父链逐级重算；`prod(l,r)` 从左右端点同时上跳，用 `a`、`b` 分别累计左边界与右边界扫上来的贡献、最后 `op(a,b)`，从而对非交换的 `op` 也保持顺序正确。它要求 `op` 满足结合律并带幺元 `e`，只支持单点修改与区间查询，且规模在构造时固定、之后不能扩容。

```
template <class T, class F> struct ZkwSeg
{
    // n 保存补到二次幂后的叶子数；e 保存幺元；op 保存满足结合律的合并运算；tr 保存连续线段树节点。
    int n = 1;
    T e;
    F op;
    vector<T> tr;

    ZkwSeg(const vector<T> &a, T e, F op) : e(e), op(op)
    {
        // a 表示初始序列，e 表示幺元，op 表示区间合并运算；构造固定规模的迭代线段树。
        while (n < (int)a.size())
        {
            n *= 2;
        }
        tr.assign(2 * n, e);
        copy(a.begin(), a.end(), tr.begin() + n);
        for (int p = n - 1; p; p--)
        {
            tr[p] = op(tr[2 * p], tr[2 * p + 1]);
        }
    }

    void set(int x, T v)
    {
```

```
        // x 表示 0 下标位置，v 表示新值；修改叶子并沿父链重算。
        for (tr[x += n] = v; x >>= 1;)
        {
            tr[x] = op(tr[2 * x], tr[2 * x + 1]);
        }
    }

    T prod(int l, int r) const
    {
        // l、r 表示半开区间 [l,r)；返回保持原顺序合并后的区间信息。
        T a = e, b = e;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1)
            {
                a = op(a, tr[l++]);
            }
            if (r & 1)
            {
                b = op(tr[--r], b);
            }
        }
        return op(a, b);
    }
};
```

构造 `ZkwSeg<T, F> seg(a, e, op)`：`e` 是幺元，`op` 为满足结合律的二元合并；`set(x, v)` 单点赋值，`prod(l, r)` 查 $[l, r)$。调用时必须显式给出合并函数，例如：

```
signed main()
{
    vector<int> a{2, -1, 4, 3};
    auto op = [](int x, int y)
    {
        return x + y;
    };
    ZkwSeg<int, decltype(op)> seg(a, 0, op);
    assert(seg.prod(1, 4) == 6);
    seg.set(2, 7);
    assert(seg.prod(0, 3) == 8);
}
```

build $O(n)$，`set`/`prod` 为 $O(\log n)$，空间 $O(n)$。需要区间整体修改时它无能为力，改用 3.3.002 懒标记线段树；这份 ZKW 就是下一条「非递归线段树」的同一实现。

3.3.006 非递归线段树

「非递归线段树」在本书里与 3.3.005 ZKW 线段树 是同一个实现模型——固定规模、单点修改、结合律区间查询、自底向上迭代无递归——因此不再重复打印代码，直接使用那里的 `ZkwSeg`。唯一要反复强调的坑是非交换运算：`prod` 里左边界用 `op(a, tr[l])`、右边界用 `op(tr[--r], b)`，两侧顺序不能颠倒，否则

字符串拼接、矩阵乘法等会出错（例如取 op 为字符串拼接、a = {"a","b","c","d"}，prod(1, 4) 应得 "bcd"）。需要区间整体懒修改、动态开点或复杂下传时都要换回递归模板：分别见 3.3.002 懒标记线段树 与 3.3.007 动态开点线段树。

3.3.007 动态开点线段树

当值域跨度极大（甚至含负数、达 10^9）而真正被赋值的位置很少时，普通动态开点线段树会为每个新点开出整条根到叶路径，空间达到 $O(q \log U)$。这里改为每个出现的位置仅存一个节点：节点保存该位置的值、左右子树和按下标顺序合并的积；插入时按当前值域二分方向下探，必要时交换当前节点与待插点，使一个节点仍只代表一个实际位置。于是树高至多 $\lceil \log_2 U \rceil$，而节点数恰为出现过的位置数（即使后来赋回幺元也保留该节点）。未出现的位置统一取幺元 e，故它适合 “初始全是幺元、单点赋值、区间结合积”，尤其是非交换的函数复合；查询累积时必须保持左、中、右的顺序。

泛型 SparsePtSeg<T,F> 的 op(left,right) 必须返回左段在前、右段在后的结合结果，e 是该运算幺元。旧的 DynamicSeg 只是其加法包装，保留单点加、区间和接口；新题优先直接使用泛型版本，不能把非交换函数复合伪装成可交换求和。

```cpp
template <class T, class F> class SparsePtSeg
{
    struct Node
    {
        // l、r 是孩子编号，pos 是本节点实际保存的位置，x、prod 分别是该点值与子树按下标顺序的积。
        int l = 0, r = 0;
        long long pos = 0;
        T x{}, prod{};
    };

    // lo、hi 是总半开值域，rt 是根编号，e 是空位置值，op 是按左到右合并的结合运算，t 是节点池。
    long long lo, hi;
    int rt = 0;
    T e;
    F op;
    vector<Node> t{Node{}};

    int newNode(long long x, const T &v)
    {
        // x 是新节点位置，v 是该点值；新建只保存此点的节点并返回编号。
        Node q;
        q.pos = x;
        q.x = q.prod = v;
        t.push_back(move(q));
        assert(t.size() <= (size_t)numeric_limits<int>::max()); // 调试检查，可删
        return (int)t.size() - 1;
    }

    void pull(int p)
    {
        // p 是非空节点；按下标顺序合并左右子树与本点，重算子树积。
        T cur = t[p].x;
```

```
        if (t[p].l)
        {
            cur = op(t[t[p].l].prod, cur);
        }
        if (t[p].r)
        {
            cur = op(cur, t[t[p].r].prod);
        }
        t[p].prod = cur;
    }

    int set(int p, long long l, long long r, long long x, T v)
    {
        // p、[l,r) 是当前稀疏子树，x、v 是待写位置和值；返回更新后的子树根。
        if (!p)
        {
            return newNode(x, v);
        }
        if (t[p].pos == x)
        {
            t[p].x = v;
            pull(p);
            return p;
        }
        long long m = l + (r - l) / 2;
        if (x < m)
        {
            if (t[p].pos < x)
            {
                swap(t[p].pos, x);
                swap(t[p].x, v);
            }
            int a = set(t[p].l, l, m, x, move(v));
            t[p].l = a;
        }
        else
        {
            if (x < t[p].pos)
            {
                swap(t[p].pos, x);
                swap(t[p].x, v);
            }
            int b = set(t[p].r, m, r, x, move(v));
            t[p].r = b;
        }
        pull(p);
        return p;
    }

    void prod(int p, long long l, long long r, long long ql, long long qr, T &ans) const
    {
        // p、[l,r) 是当前子树，[ql,qr) 是查询区间，ans 按从左到右顺序累积答案。
```

```
85          ql = max(ql, l);
86          qr = min(qr, r);
87          if (ql >= qr || !p)
88          {
89              return;
90          }
91          if (ql == l && qr == r)
92          {
93              ans = op(ans, t[p].prod);
94              return;
95          }
96          long long m = l + (r - l) / 2;
97          prod(t[p].l, l, m, ql, qr, ans);
98          if (ql <= t[p].pos && t[p].pos < qr)
99          {
100             ans = op(ans, t[p].x);
101         }
102         prod(t[p].r, m, r, ql, qr, ans);
103     }
104
105 public:
106     SparsePtSeg(long long lo, long long hi, T e, F op) : lo(lo), hi(hi), e(move(e)),
    ↪ op(move(op))
107     {
108         // lo、hi 是非空半开整数值域，e 是未写位置的值，op 是保持左右顺序的结合运算。
109         assert(lo < hi); // 调试检查，可删
110     }
111
112     void reserveNodes(int cap)
113     {
114         // cap 是预计出现的不同位置数；预留节点池容量，不改变结构语义。
115         assert(cap >= 0); // 调试检查，可删
116         t.reserve(cap + 1);
117     }
118
119     void set(long long x, const T &v)
120     {
121         // x 是值域内位置，v 是新值；把该点赋为 v，未写过的位置默认取 e。
122         assert(lo <= x && x < hi); // 调试检查，可删
123         rt = set(rt, lo, hi, x, v);
124     }
125
126     T get(long long x) const
127     {
128         // x 是值域内位置；返回该点当前值，未出现过时返回 e。
129         assert(lo <= x && x < hi); // 调试检查，可删
130         int p = rt;
131         while (p)
132         {
133             if (t[p].pos == x)
134             {
135                 return t[p].x;
```

```
            }
            p = x < t[p].pos ? t[p].l : t[p].r;
        }
        return e;
    }

    T prod(long long l, long long r) const
    {
        // l、r 是值域内半开端点；返回 [l,r) 按下标从左到右合并的积，空区间返回 e。
        assert(lo <= l && l <= r && r <= hi); // 调试检查，可删
        T ans = e;
        prod(rt, lo, hi, l, r, ans);
        return ans;
    }
};

class DynamicSeg
{
    struct Add
    {
        long long operator()(long long a, long long b) const
        {
            // a、b 是相邻两段的元素和；返回两段合并后的和。
            return a + b;
        }
    };

    // tr 是泛型稀疏线段树的加法实例，保留旧的单点加、区间和接口。
    SparsePtSeg<long long, Add> tr;

  public:
    DynamicSeg(long long lo, long long hi) : tr(lo, hi, 0, Add{})
    {
        // lo、hi 是非空半开值域边界；初始所有点均为零。
    }

    void reserveNodes(int cap)
    {
        // cap 是预计被修改过的不同位置数；转交给内部节点池预留容量。
        tr.reserveNodes(cap);
    }

    void add(long long x, long long v)
    {
        // x 是值域内位置，v 是有符号增量；把该点增加 v。
        tr.set(x, tr.get(x) + v);
    }

    long long sum(long long l, long long r) const
    {
        // l、r 是查询半开区间；返回其中元素和。
        return tr.prod(l, r);
```

```
    }
};
```

构造 `SparsePtSeg<T,F> tr(lo, hi, e, op)`，值域为半开 $[lo, hi)$（可为负）；`set(x,v)` 单点赋值，`get(x)` 取点值，`prod(l,r)` 查半开区间的有序积。调用时例如用加法维护稀疏位置：

```
signed main()
{
    auto op = [](int x, int y)
    {
        return x + y;
    };
    SparsePtSeg<int, decltype(op)> tr(0, 1000000000, 0, op);
    tr.set(5, 7);
    tr.set(900000000, 3);
    assert(tr.get(5) == 7);
    assert(tr.prod(0, 1000000000) == 10);
}
```

设 $U = hi - lo$，出现过的不同位置数为 s，`set`、`get`、`prod` 最坏均为 $O(\log U)$，节点数 $O(s)$；`reserveNodes(s)` 只优化扩容，不能把位置数预留成 U。例如取 `T=array<Z,2>`，令 `e={1,0}`、`op(f,g)=g\circ f`，即可处理长度至多 10^9、初始全为恒等函数的单点赋值与区间函数复合。仍需要旧的加法接口时构造 `DynamicSeg tr(lo,hi)` 并调用 `add(x,v)`、`sum(l,r)`；其复杂度相同。若操作是大值域上的区间仿射、区间和，节点不能只表示单点，应改用 3.3.021 稀疏仿射区间和；要把整段函数统一赋成一个值并查询有序复合，改用 3.3.022 区间赋同函数、区间复合。它不提供两棵树合并——需要合并权值树时用 3.3.009 线段树合并，值域不大时直接离散化后用 3.3.002 懒标记线段树 常数更小，要保历史版本用 3.3.008 可持久化线段树 / 主席树。

3.3.008 可持久化线段树 / 主席树

可持久化线段树（主席树）每次单点修改只新建根到叶的一条链、其余结点与旧版本共享，于是每个版本根都是一份完整快照，仅用 $O(\log n)$ 额外结点保存历史。结点同时保存出现次数 `sum` 与可独立指定的附加权值和 `weight`，版本差便能在同一值域区间返回计数和总权值。静态区间第 k 小仍只依赖非负计数：第 r 版减第 l 版得到下标区间 $[l, r)$ 的值域计数，再按左子树计数二分；`kth` 的 k 从 0 开始，附加权值允许为负但不参与排名。

```
struct PerSeg
{
    struct Node
    {
        // l、r 是左右儿子编号，sum、weight 是当前值域的计数和与附加权值和。
        int l = 0, r = 0;
        long long sum = 0;
        long long weight = 0;
    };

    // n 是离散值域大小，t 是所有版本共享的节点池。
    int n;
```

```
    vector<Node> t{{}};

    PerSeg(int n) : n(n)
    {
        // n 是非空离散值域大小；0 号根表示空版本。
        assert(n > 0); // 调试检查，可删
    }

    int add(int root, int x, long long v)
    {
        // root 是旧版本根，x 是值域下标，v 是计数增量；返回新版本根。
        return add(root, x, v, 0);
    }

    int add(int root, int x, long long dc, long long dw)
    {
        // root 是旧版本根，x 是值域下标；dc、dw 是计数与附加权值增量，返回新版本根。
        assert(0 <= x && x < n); // 调试检查，可删
        return add(root, 0, n, x, dc, dw);
    }

    long long query(int root, int l, int r) const
    {
        // root 是版本根；返回值域 [l,r) 的计数和。
        return queryInfo(root, l, r)[0];
    }

    array<long long, 2> queryInfo(int root, int l, int r) const
    {
        // root 是版本根，l、r 是值域半开区间；返回 {计数和，附加权值和}。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return queryInfo(root, 0, n, l, r);
    }

    // 返回右版本减左版本后的第 k 小下标，k 从 0 开始；sum 必须表示非负计数。
    int kth(int rootL, int rootR, long long k) const
    {
        // rootL、rootR 是两个前缀版本根，k 是 0 基排名；返回版本差中的第 k 小值域下标。
        assert(0 <= k && k < t[rootR].sum - t[rootL].sum); // 调试检查，可删
        int l = 0, r = n;
        while (r - l > 1)
        {
            int m = (l + r) / 2;
            long long z = t[t[rootR].l].sum - t[t[rootL].l].sum;
            if (k < z)
            {
                rootL = t[rootL].l;
                rootR = t[rootR].l;
                r = m;
            }
            else
            {
```

```
                k -= z;
                rootL = t[rootL].r;
                rootR = t[rootR].r;
                l = m;
            }
        }
        return l;
    }

  private:
    int clone(int p)
    {
        // p 是旧节点编号；复制它并返回新节点编号。
        t.push_back(t[p]);
        return (int)t.size() - 1;
    }

    int add(int p, int l, int r, int x, long long dc, long long dw)
    {
        // p 是旧节点，[l,r) 是其值域，x 是更新位置；两个增量分别修改计数与附加权值，返回新节点。
        int q = clone(p);
        t[q].sum += dc;
        t[q].weight += dw;
        if (r - l == 1)
        {
            return q;
        }
        int m = (l + r) / 2;
        if (x < m)
        {
            t[q].l = add(t[q].l, l, m, x, dc, dw);
        }
        else
        {
            t[q].r = add(t[q].r, m, r, x, dc, dw);
        }
        return q;
    }

    array<long long, 2> queryInfo(int p, int l, int r, int ql, int qr) const
    {
        // p 是当前节点，[l,r) 是其值域；返回 [ql,qr) 的计数和与附加权值和。
        if (!p || qr <= l || r <= ql)
        {
            return {0, 0};
        }
        if (ql <= l && r <= qr)
        {
            return {t[p].sum, t[p].weight};
        }
        int m = (l + r) / 2;
        auto a = queryInfo(t[p].l, l, m, ql, qr);
```

```
        auto b = queryInfo(t[p].r, m, r, ql, qr);
        return {a[0] + b[0], a[1] + b[1]};
    }
};
```

用法：`PerSeg ps(n)`，其中 n 为非空离散值域大小。维护版本根数组，令 `root[0]=0` 为空树；只维护计数时调用三参数 `add(root,x,dc)`，同时维护权值时调用四参数 `add(root,x,dc,dw)`。`queryInfo(root,l,r)` 返回 `{计数和，附加权值和}`，原 `query` 仍只返回计数。逐点构造前缀版本可写成

```
root[i + 1] = ps.add(root[i], a[i], 1);
```

查询下标半开区间 $[l,r)$ 的第 k 小写 `ps.kth(root[l], root[r], k)`，返回离散值域下标。`add`、`queryInfo`、`kth` 均 $O(\log n)$，每次 `add` 新增 $O(\log n)$ 结点，总空间 $O(n\log n)$。同源的可持久化数组 / 并查集见 3.1.005 可持久化并查集。值域计数若还需在线带修（插删），单纯主席树不够，要树状数组套主席树或离线整体二分，见 3.4.006 区间第 K 小、排名、前驱后继。下面用含重复值的固定数组核对版本差和排名：

```
signed main()
{
    vector<int> a{5, 1, 4, 1, 3};
    PerSeg ps(6);
    vector<int> root(a.size() + 1);
    for (int i = 0; i < (int)a.size(); i++)
    {
        root[i + 1] = ps.add(root[i], a[i], 1);
    }
    vector<int> got{ps.kth(root[0], root[5], 0), ps.kth(root[0], root[5], 2),
                    ps.kth(root[0], root[5], 4), ps.kth(root[1], root[4], 0),
                    ps.kth(root[1], root[4], 2)};
    assert(got == vector<int>({1, 3, 5, 1, 4}));
}
```

3.3.009 线段树合并

线段树合并把两棵同值域的动态开点权值线段树递归合成一棵：某一侧为空就直接返回另一侧（`p | q`，其中一个必为 0），到叶子把计数相加，内部结点合并左右儿子后重算和。它是「每个树上结点先各建一棵权值树，再自底向上把儿子合并到父亲」这一套路的核心，可替代 DSU on tree 做子树众数 / 子树第 k 小等统计。这里是破坏式合并：`merge(p,q)` 复用 `p` 的结点，调用后 `q`（及被并入的结点）不应再独立使用。总复杂度靠势能分析——每次合并消掉一个重叠结点，全程重叠结点总数是 $O(\text{总插入次数}\cdot\log U)$，故所有合并累计线性对数。

```
struct MergeSeg
{
    struct Node
    {
        // l、r 是左右儿子编号，sum 是当前值域的权值和。
        int l = 0, r = 0;
        long long sum = 0;
    };
```

```
9
10      // lo、hi 是整棵权值树的半开值域，t 是所有树共享的节点池。
11      int lo, hi;
12      vector<Node> t{{}};
13
14      MergeSeg(int lo, int hi) : lo(lo), hi(hi)
15      {
16          // lo、hi 是非空半开值域边界。
17          assert(lo < hi); // 调试检查，可删
18      }
19      int newNode()
20      {
21          // 分配并返回一个全零节点编号。
22          t.push_back({});
23          return (int)t.size() - 1;
24      }
25      int makeRoot() const
26      {
27          // 返回表示空树的根编号 0。
28          return 0;
29      }
30
31      int add(int p, int l, int r, int x, long long v)
32      {
33          // p、[l,r) 是当前树，x 是值域位置，v 是权值增量；返回更新后的根。
34          if (!p)
35          {
36              p = newNode();
37          }
38          t[p].sum += v;
39          if (l + 1 == r)
40          {
41              return p;
42          }
43          int m = midpoint(l, r);
44          if (x < m)
45          {
46              t[p].l = add(t[p].l, l, m, x, v);
47          }
48          else
49          {
50              t[p].r = add(t[p].r, m, r, x, v);
51          }
52          return p;
53      }
54
55      void add(int &p, int x, long long v)
56      {
57          // p 是根，x 是值域位置，v 是权值增量；原地更新 p。
58          assert(lo <= x && x < hi); // 调试检查，可删
59          p = add(p, lo, hi, x, v);
60      }
```

```
61
62      int merge(int p, int q, int l, int r)
63      {
64          // p、q 是当前值域 [l,r) 上的两棵树根；破坏性合并并返回根。
65          if (!p || !q)
66          {
67              return p | q;
68          }
69          if (l + 1 == r)
70          {
71              t[p].sum += t[q].sum;
72              return p;
73          }
74          int m = midpoint(l, r);
75          t[p].l = merge(t[p].l, t[q].l, l, m);
76          t[p].r = merge(t[p].r, t[q].r, m, r);
77          t[p].sum = t[t[p].l].sum + t[t[p].r].sum;
78          return p;
79      }
80
81      int merge(int p, int q)
82      {
83          // p、q 是两棵共享同一值域的树根；破坏性合并并返回根。
84          return merge(p, q, lo, hi);
85      }
86
87      long long sum(int p, int l, int r, int x, int y) const
88      {
89          // p、[l,r) 是当前树；返回值域 [x,y) 的权值和。
90          if (!p || y <= l || r <= x)
91          {
92              return 0;
93          }
94          if (x <= l && r <= y)
95          {
96              return t[p].sum;
97          }
98          int m = midpoint(l, r);
99          return sum(t[p].l, l, m, x, y) + sum(t[p].r, m, r, x, y);
100     }
101
102     long long rangeSum(int p, int l, int r) const
103     {
104         // p 是树根；返回值域 [l,r) 的权值和。
105         assert(lo <= l && l <= r && r <= hi); // 调试检查，可删
106         return sum(p, lo, hi, l, r);
107     }
108 };
```

构造 `MergeSeg ms(lo, hi)` 共用一个结点池；每棵子树各持一个根编号，`ms.add(root, x, v)` 单点加、`ms.rangeSum(root, l, r)` 查区间和、`ms.merge(rootP, rootQ)` 返回合并根。所有树共享 `t` 数组，根用整型编号区分。单点 `add`、区间查询 $O(\log U)$，全部 `merge` 累计 $O(\text{有效更新数} \cdot \log U)$，空间同阶。它

与 3.3.010 线段树分裂 互为逆操作、常配合使用（分裂出一段再并回别处）；只在单棵权值树上做区间第 k 小、无需合并时用 3.3.008 可持久化线段树 / 主席树。

3.3.010 线段树分裂

线段树分裂是合并的逆操作：按值域区间把一棵权值线段树拆成两棵。递归时整段落在保留侧就原地不动、整段落在分出侧就把结点所有权整体转移给新根，只有跨边界的结点才 `newNode` 复制一份并递归拆分左右儿子，最后两侧都 `pull` 重算、和为零的结点回收成 0。这样一次分裂只新建 $O(\log U)$ 个结点。配合按排名分裂即可实现「取出前 k 小 / 某值域段」再并到别处，是可并集合、动态排名截取的主力；`kth` 在单棵树上按计数二分求第 k 小（k 从 1 开始，越界返回 -1）。

```
template <class T> struct SplitSeg
{
    struct Node
    {
        // l、r 是左右儿子编号，sum 是子树内的非负计数和。
        int l = 0, r = 0;
        T sum{};
    };

    // lo、hi 是整棵计数树的半开值域，t 是所有树共享的节点池。
    int lo, hi;
    vector<Node> t{{}};

    SplitSeg(int lo, int hi) : lo(lo), hi(hi)
    {
        // lo、hi 是非空半开值域边界。
        assert(lo < hi); // 调试检查，可删
    }
    int newNode()
    {
        // 分配并返回一个全零节点编号。
        t.push_back({});
        return (int)t.size() - 1;
    }
    int makeRoot() const
    {
        // 返回表示空树的根编号 0。
        return 0;
    }

    void pull(int p)
    {
        // p 是内部节点；由两个儿子的计数和重算当前节点。
        t[p].sum = t[t[p].l].sum + t[t[p].r].sum;
    }

    int add(int p, int l, int r, int x, T v)
    {
        // p、[l,r) 是当前计数树，x 是值域位置，v 是计数增量；返回新根。
```

```
        if (!p)
        {
            p = newNode();
        }
        if (l + 1 == r)
        {
            t[p].sum += v;
            assert(t[p].sum >= T{}); // 调试检查，可删
            return t[p].sum == T{} ? 0 : p;
        }
        int m = midpoint(l, r);
        if (x < m)
        {
            t[p].l = add(t[p].l, l, m, x, v);
        }
        else
        {
            t[p].r = add(t[p].r, m, r, x, v);
        }
        pull(p);
        return t[p].sum == T{} ? 0 : p;
    }

    void add(int &p, int x, T v)
    {
        // p 是根，x 是值域位置，v 是计数增量；更新后所有叶子计数必须非负。
        assert(lo <= x && x < hi); // 调试检查，可删
        p = add(p, lo, hi, x, v);
    }

    pair<int, int> split(int p, int l, int r, int x, int y)
    {
        // p、[l,r) 是当前树，[x,y) 是分出值域；返回 {保留根，分出根}。
        if (!p || y <= l || r <= x)
        {
            return {p, 0};
        }
        if (x <= l && r <= y)
        {
            return {0, p};
        }
        int q = newNode(), m = midpoint(l, r);
        auto [al, bl] = split(t[p].l, l, m, x, y);
        auto [ar, br] = split(t[p].r, m, r, x, y);
        t[p].l = al;
        t[p].r = ar;
        t[q].l = bl;
        t[q].r = br;
        pull(p);
        pull(q);
        if (t[p].sum == T{})
        {
```

```
            p = 0;
        }
        if (t[q].sum == T{})
        {
            q = 0;
        }
        return {p, q};
    }

    int split(int &p, int l, int r)
    {
        // p 是原根；把值域 [l,r) 分出并返回新根，p 保留其余部分。
        assert(lo <= l && l <= r && r <= hi); // 调试检查，可删
        auto [a, b] = split(p, lo, hi, l, r);
        p = a;
        return b;
    }

    int merge(int p, int q, int l, int r)
    {
        // p、q 是当前值域 [l,r) 上的两棵树根；破坏性合并并返回根。
        if (!p || !q)
        {
            return p | q;
        }
        if (l + 1 == r)
        {
            t[p].sum += t[q].sum;
            return p;
        }
        int m = midpoint(l, r);
        t[p].l = merge(t[p].l, t[q].l, l, m);
        t[p].r = merge(t[p].r, t[q].r, m, r);
        pull(p);
        return p;
    }

    int merge(int p, int q)
    {
        // p、q 是两棵所有权独立的计数树根；破坏性合并并返回根。
        return merge(p, q, lo, hi);
    }

    T sum(int p, int l, int r, int x, int y) const
    {
        // p、[l,r) 是当前树；返回值域 [x,y) 的计数和。
        if (!p || y <= l || r <= x)
        {
            return T{};
        }
        if (x <= l && r <= y)
        {
```

```
            return t[p].sum;
        }
        int m = midpoint(l, r);
        return sum(t[p].l, l, m, x, y) + sum(t[p].r, m, r, x, y);
    }

    T rangeSum(int p, int l, int r) const
    {
        // p 是根；返回值域 [l,r) 内的计数和。
        assert(lo <= l && l <= r && r <= hi); // 调试检查，可删
        return sum(p, lo, hi, l, r);
    }

    int kth(int p, T k) const
    {
        // p 是根，k 是从 1 开始的排名；返回第 k 小值，越界返回 -1。
        if (!p || k <= T{} || t[p].sum < k)
        {
            return -1;
        }
        int l = lo, r = hi;
        while (l + 1 < r)
        {
            int m = midpoint(l, r), q = t[p].l;
            if (t[q].sum >= k)
            {
                p = q;
                r = m;
            }
            else
            {
                k -= t[q].sum;
                p = t[p].r;
                l = m;
            }
        }
        return l;
    }
};
```

构造 `SplitSeg<long long> ss(lo, hi)`：`ss.add(root, x, v)` 在值 x 处加计数 v，要求更新后每个叶子计数非负；这一条件保证 `sum==0` 才等价于空树，也保证 `kth` 的方向判断成立。`ss.rangeSum(root, l, r)` 查值域 $[l,r)$ 计数和，`int b = ss.split(root, l, r)` 把 `root` 中值域 $[l,r)$ 的部分拆到新根 `b`（`root` 保留其余），`ss.merge(p, q)` 合回两棵所有权独立的树，`ss.kth(root, k)` 求第 k 小（k 从 1 起）。`add`、`kth`、`split` 均 $O(\log U)$，`merge` 与访问结点数成正比，空间 $O((\text{更新数}+\text{split 次数})\log U)$。它与 3.3.009 线段树合并 互逆、常成对使用；单纯静态区间第 k 小无需拆分时用 3.3.008 可持久化线段树 / 主席树。下面用一次固定拆分、合并闭环核对计数与排名：

```
signed main()
{
    SplitSeg<long long> ss(0, 10);
```

```
    int root = 0;
    for (int x : {1, 1, 4, 7, 9})
    {
        ss.add(root, x, 1);
    }
    int mid = ss.split(root, 4, 8);
    assert(ss.rangeSum(root, 0, 10) == 3 && ss.rangeSum(mid, 0, 10) == 2);
    root = ss.merge(root, mid);
    assert(ss.kth(root, 1) == 1 && ss.kth(root, 4) == 7 && ss.kth(root, 5) == 9);
}
```

3.3.011 Segment Tree Beats

吉司机线段树（Segment Tree Beats）是普通懒标记之外唯一能在线支持区间 chmin/chmax 的做法。关键在于每个节点除最大值外还维护严格次大值 `mx2` 与最大值出现次数 `mxc`（最小侧对称）。执行区间 chmin(x) 时分三种情形：若 $x \geq$ 最大值则整段无变化直接返回；若严格次大值 $< x <$ 最大值，则只有那 `mxc` 个最大值会被压到 x，可以 $O(1)$ 打标记整段完成（`sum` 减去 `(mx-x)*mxc`）；否则 x 会切断次大值层，必须递归下传。势能分析表明这种“标记回收”下连续 q 次操作总代价是 $O((n+q)\log^2 n)$。严格次值不能与普通整数哨兵共用表示，下面用 `hasMx2`、`hasMn2` 单独记录其是否存在，因此所有合法 `long long` 元素值都能参与维护；调用方仍须保证区间和及加法中间量可由 `long long` 表示。

```
struct SegBeats
{
    struct Node
    {
        // sum 是区间和，add 是待下传增量。
        long long sum = 0, add = 0;
        // mx、mn 是最值；mx2、mn2 仅在对应 has 标志为真时保存严格次值。
        long long mx = 0, mx2 = 0, mn = 0, mn2 = 0;
        bool hasMx2 = false, hasMn2 = false;
        // mxc、mnc 是最值出现次数，len 是区间长度。
        int mxc = 0, mnc = 0, len = 0;
    };

    // n 是数组长度，tr 保存各线段树节点的信息。
    int n;
    vector<Node> tr;

    SegBeats(const vector<long long> &a) : n(a.size()), tr(4 * n)
    {
        // a 是非空初始数组。
        assert(n); // 调试检查，可删
        build(1, 0, n, a);
    }

    void add(int l, int r, long long x)
    {
        // 给半开区间 [l,r) 增加 x。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
```

```
        add(1, 0, n, l, r, x);
    }
    void chmin(int l, int r, long long x)
    {
        // 把半开区间 [l,r) 内每个值改成 min(value,x)。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        chmin(1, 0, n, l, r, x);
    }
    void chmax(int l, int r, long long x)
    {
        // 把半开区间 [l,r) 内每个值改成 max(value,x)。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        chmax(1, 0, n, l, r, x);
    }
    long long sum(int l, int r)
    {
        // 返回半开区间 [l,r) 的和，空区间返回 0。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return sum(1, 0, n, l, r);
    }
    long long min(int l, int r)
    {
        // 返回非空半开区间 [l,r) 的最小值。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        return min(1, 0, n, l, r);
    }
    long long max(int l, int r)
    {
        // 返回非空半开区间 [l,r) 的最大值。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        return max(1, 0, n, l, r);
    }

  private:
    static Node merge(const Node &a, const Node &b)
    {
        // a、b 是相邻区间节点；返回合并后的 Beats 信息。
        Node c;
        c.sum = a.sum + b.sum;
        c.len = a.len + b.len;
        c.mx = a.mx < b.mx ? b.mx : a.mx;
        c.mxc = (a.mx == c.mx ? a.mxc : 0) + (b.mx == c.mx ? b.mxc : 0);
        auto takeMax = [&](long long x)
        {
            if (x != c.mx && (!c.hasMx2 || c.mx2 < x))
            {
                c.mx2 = x;
                c.hasMx2 = true;
            }
        };
        takeMax(a.mx);
        takeMax(b.mx);
```

```
        if (a.hasMx2)
        {
            takeMax(a.mx2);
        }
        if (b.hasMx2)
        {
            takeMax(b.mx2);
        }
        c.mn = a.mn < b.mn ? a.mn : b.mn;
        c.mnc = (a.mn == c.mn ? a.mnc : 0) + (b.mn == c.mn ? b.mnc : 0);
        auto takeMin = [&](long long x)
        {
            if (x != c.mn && (!c.hasMn2 || x < c.mn2))
            {
                c.mn2 = x;
                c.hasMn2 = true;
            }
        };
        takeMin(a.mn);
        takeMin(b.mn);
        if (a.hasMn2)
        {
            takeMin(a.mn2);
        }
        if (b.hasMn2)
        {
            takeMin(b.mn2);
        }
        return c;
    }

    void build(int p, int l, int r, const vector<long long> &a)
    {
        // p 是当前节点，[l,r) 是其区间，a 是初始数组。
        if (r - l == 1)
        {
            tr[p].sum = tr[p].mx = tr[p].mn = a[l];
            tr[p].mxc = tr[p].mnc = tr[p].len = 1;
            return;
        }
        int m = (l + r) / 2;
        build(2 * p, l, m, a);
        build(2 * p + 1, m, r, a);
        pull(p);
    }

    void pull(int p)
    {
        // p 是内部节点；合并两个儿子的最值、次值、次数与区间和。
        tr[p] = merge(tr[2 * p], tr[2 * p + 1]);
    }

```

```
    void applyAdd(int p, long long x)
    {
        // p 是整段命中的节点，x 是区间增量。
        auto &v = tr[p];
        v.sum += x * v.len;
        v.mx += x;
        v.mn += x;
        if (v.hasMx2)
        {
            v.mx2 += x;
        }
        if (v.hasMn2)
        {
            v.mn2 += x;
        }
        v.add += x;
    }

    void applyChmin(int p, long long x)
    {
        // p 是可整段处理的节点，x 是上界；仅把当前最大值压到 x。
        auto &v = tr[p];
        if (v.mx <= x)
        {
            return;
        }
        v.sum += (x - v.mx) * v.mxc;
        long long old = v.mx;
        if (v.mn == old)
        {
            v.mn = x;
        }
        else if (v.hasMn2 && v.mn2 == old)
        {
            v.mn2 = x;
        }
        v.mx = x;
    }

    void applyChmax(int p, long long x)
    {
        // p 是可整段处理的节点，x 是下界；仅把当前最小值抬到 x。
        auto &v = tr[p];
        if (v.mn >= x)
        {
            return;
        }
        v.sum += (x - v.mn) * v.mnc;
        long long old = v.mn;
        if (v.mx == old)
        {
            v.mx = x;
```

```
        }
        else if (v.hasMx2 && v.mx2 == old)
        {
            v.mx2 = x;
        }
        v.mn = x;
    }

    void push(int p)
    {
        // p 是当前节点；依次下传区间加和父节点的最值界。
        if (tr[p].add)
        {
            applyAdd(2 * p, tr[p].add);
            applyAdd(2 * p + 1, tr[p].add);
            tr[p].add = 0;
        }
        applyChmin(2 * p, tr[p].mx);
        applyChmin(2 * p + 1, tr[p].mx);
        applyChmax(2 * p, tr[p].mn);
        applyChmax(2 * p + 1, tr[p].mn);
    }

    void add(int p, int l, int r, int ql, int qr, long long x)
    {
        // p、[l,r) 是当前节点， [ql,qr) 是修改区间，x 是增量。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            applyAdd(p, x);
            return;
        }
        push(p);
        int m = (l + r) / 2;
        add(2 * p, l, m, ql, qr, x);
        add(2 * p + 1, m, r, ql, qr, x);
        pull(p);
    }

    void chmin(int p, int l, int r, int ql, int qr, long long x)
    {
        // p、[l,r) 是当前节点；把 [ql,qr) 内大于 x 的值压到 x。
        if (qr <= l || r <= ql || tr[p].mx <= x)
        {
            return;
        }
        if (ql <= l && r <= qr && (!tr[p].hasMx2 || tr[p].mx2 < x))
        {
            applyChmin(p, x);
```

```
            return;
        }
        push(p);
        int m = (l + r) / 2;
        chmin(2 * p, l, m, ql, qr, x);
        chmin(2 * p + 1, m, r, ql, qr, x);
        pull(p);
    }

    void chmax(int p, int l, int r, int ql, int qr, long long x)
    {
        // p、[l,r) 是当前节点；把 [ql,qr) 内小于 x 的值抬到 x。
        if (qr <= l || r <= ql || tr[p].mn >= x)
        {
            return;
        }
        if (ql <= l && r <= qr && (!tr[p].hasMn2 || x < tr[p].mn2))
        {
            applyChmax(p, x);
            return;
        }
        push(p);
        int m = (l + r) / 2;
        chmax(2 * p, l, m, ql, qr, x);
        chmax(2 * p + 1, m, r, ql, qr, x);
        pull(p);
    }

    long long sum(int p, int l, int r, int ql, int qr)
    {
        // p、[l,r) 是当前节点；返回 [ql,qr) 的区间和。
        if (qr <= l || r <= ql)
        {
            return 0;
        }
        if (ql <= l && r <= qr)
        {
            return tr[p].sum;
        }
        push(p);
        int m = (l + r) / 2;
        return sum(2 * p, l, m, ql, qr) + sum(2 * p + 1, m, r, ql, qr);
    }

    long long min(int p, int l, int r, int ql, int qr)
    {
        // p、[l,r) 是当前节点；返回 [ql,qr) 的最小值。
        if (qr <= l || r <= ql)
        {
            return numeric_limits<long long>::max();
        }
        if (ql <= l && r <= qr)
```

```
        {
            return tr[p].mn;
        }
        push(p);
        int m = (l + r) / 2;
        long long a = min(2 * p, l, m, ql, qr), b = min(2 * p + 1, m, r, ql, qr);
        return a < b ? a : b;
    }

    long long max(int p, int l, int r, int ql, int qr)
    {
        // p、[l,r) 是当前节点；返回 [ql,qr) 的最大值。
        if (qr <= l || r <= ql)
        {
            return numeric_limits<long long>::lowest();
        }
        if (ql <= l && r <= qr)
        {
            return tr[p].mx;
        }
        push(p);
        int m = (l + r) / 2;
        long long a = max(2 * p, l, m, ql, qr), b = max(2 * p + 1, m, r, ql, qr);
        return a < b ? b : a;
    }
};
```

构造 `SegBeats st(a)`（a 为 `vector<long long>`，下标 $[0,n)$），此后 `st.add(l,r,x)`、`st.chmin(l,r,x)`、`st.chmax(l,r,x)` 均作用于半开区间 $[l,r)$，`st.sum/min/max(l,r)` 查询同一区间。单次 add、sum、min、max 为 $O(\log n)$，chmin/chmax 均摊 $O(\log^2 n)$、连续 q 次总计 $O((n+q)\log^2 n)$，空间 $O(n)$。若只有区间加和普通区间查询而无 chmin/chmax，用 3.3.002 懒标记线段树 常数更小；只需单点修改则用 3.3.005 ZKW 线段树。下面一组操作覆盖区间取小、取大、加法以及三种查询：

```
signed main()
{
    SegBeats st({5, -2, 7, 1});
    st.chmin(0, 3, 4);
    st.chmax(1, 4, 0);
    st.add(0, 2, 3);
    assert(st.sum(0, 4) == 15);
    assert(st.min(0, 4) == 1);
    assert(st.max(0, 4) == 7);
}
```

3.3.012 扫描线线段树

扫描线线段树把二维矩形问题降到一维：把每个矩形拆成左右两条竖边作为事件按横坐标排序，从左到右扫描，线段树在纵轴上维护「当前有多少矩形覆盖每一段」。这里线段树的下标区间对应离散化后的纵坐标段 $[ys[l],ys[r])$，每个节点存覆盖标记次数 `cov`、被至少一层覆盖的总长度 `len`、覆盖段数 `seg` 以及左右

端点是否被覆盖 `lc/rc`。`pull` 有个关键点：因为区间加减是成对出现且只在完整区间打标记，这里用的是「永久化标记 + 不下传」写法——`cov[p]>0` 时整段被覆盖直接取满长，否则由儿子合并，合并 `seg` 时若左儿子右端与右儿子左端相接要减一去重。相邻两条事件线之间，`len[1]` 乘横向距离累加进面积，`2*seg[1]` 乘横向距离累加进上下横边；同一横坐标先加入再删除，每次 `len[1]` 的变化量绝对值之和恰是该竖线贡献的周长：

```
template <class T> struct Rectangle
{
    // x1、y1、x2、y2 依次是矩形的左、下、右、上边界。
    T x1, y1, x2, y2;
};

template <class T, class Acc = T> class RectUnion
{
    struct Event
    {
        // x 是扫描线横坐标，[y1,y2) 是纵向覆盖段，d 为加入 1 或删除 -1。
        T x, y1, y2;
        int d;
    };

    // ys 保存离散后的纵坐标；下标区间 [l,r) 对应实际区间 [ys[l],ys[r])。
    vector<T> ys;
    // cov、seg、lc、rc 分别保存覆盖次数、覆盖段数和左右端覆盖状态。
    vector<int> cov, seg, lc, rc;
    // len 保存每个节点的实际覆盖长度，类型由 Acc 指定。
    vector<Acc> len;

    void pull(int p, int l, int r)
    {
        // p 是节点编号，[l,r) 是纵坐标下标区间；由覆盖标记和儿子信息更新节点。
        if (cov[p])
        {
            len[p] = (Acc)ys[r] - (Acc)ys[l];
            seg[p] = lc[p] = rc[p] = 1;
        }
        else if (r - l == 1)
        {
            len[p] = 0;
            seg[p] = lc[p] = rc[p] = 0;
        }
        else
        {
            int ls = p * 2, rs = ls + 1;
            len[p] = len[ls] + len[rs];
            seg[p] = seg[ls] + seg[rs] - (rc[ls] && lc[rs]);
            lc[p] = lc[ls];
            rc[p] = rc[rs];
        }
    }

```

```
    void add(int p, int l, int r, int ql, int qr, int d)
    {
        // p、[l,r) 是当前节点，[ql,qr) 是修改区间，d 是覆盖次数增量。
        if (ql <= l && r <= qr)
        {
            cov[p] += d;
            pull(p, l, r);
            return;
        }
        int m = midpoint(l, r);
        if (ql < m)
        {
            add(p * 2, l, m, ql, qr, d);
        }
        if (m < qr)
        {
            add(p * 2 + 1, m, r, ql, qr, d);
        }
        pull(p, l, r);
    }

    pair<Acc, Acc> calc(const vector<Rectangle<T>> &a)
    {
        // a 是输入矩形集合；返回值 first、second 依次为面积并和周长并。
        vector<Event> ev;
        ys.clear();
        for (auto [x1, y1, x2, y2] : a)
        {
            if (x1 >= x2 || y1 >= y2)
            {
                continue;
            }
            ev.push_back({x1, y1, y2, 1});
            ev.push_back({x2, y1, y2, -1});
            ys.push_back(y1);
            ys.push_back(y2);
        }
        if (ev.empty())
        {
            return {};
        }
        ranges::sort(ys);
        ys.erase(unique(ys.begin(), ys.end()), ys.end());
        ranges::sort(ev, {}, &Event::x);
        int n = (int)ys.size() * 4;
        cov.assign(n, 0);
        seg.assign(n, 0);
        lc.assign(n, 0);
        rc.assign(n, 0);
        len.assign(n, 0);

        Acc ar = 0;
```

```
        Acc peri = 0;
        T px = ev[0].x;
        for (int i = 0; i < (int)ev.size();)
        {
            int j = i;
            while (j < (int)ev.size() && ev[j].x == ev[i].x)
            {
                j++;
            }
            Acc dx = (Acc)ev[i].x - (Acc)px;
            ar += len[1] * dx;
            peri += (Acc)2 * seg[1] * dx;

            // 同一横坐标先加入再删除，逐次长度变化之和恰为新旧覆盖集合的对称差长度。
            for (int d : {1, -1})
            {
                for (int k = i; k < j; k++)
                {
                    if (ev[k].d != d)
                    {
                        continue;
                    }
                    Acc old = len[1];
                    int l = lower_bound(ys.begin(), ys.end(), ev[k].y1) - ys.begin();
                    int r = lower_bound(ys.begin(), ys.end(), ev[k].y2) - ys.begin();
                    add(1, 0, (int)ys.size() - 1, l, r, d);
                    Acc delta = len[1] - old;
                    peri += delta < 0 ? -delta : delta;
                }
            }
            px = ev[i].x;
            i = j;
        }
        return {ar, peri};
    }

public:
    Acc area(const vector<Rectangle<T>> &a)
    {
        // a 是输入矩形集合；返回全部轴平行矩形并集的面积。
        return calc(a).first;
    }

    Acc perimeter(const vector<Rectangle<T>> &a)
    {
        // a 是输入矩形集合；返回全部轴平行矩形并集的周长。
        return calc(a).second;
    }

    Acc solve(const vector<Rectangle<T>> &a)
    {
        // a 是输入矩形集合；兼容旧接口并返回矩形并集面积。
```

```
        return area(a);
    }
};
```

构造 `RectUnion<int> q`（或 `RectUnion<long long>`），`q.area(a)` 与 `q.perimeter(a)` 对同一组矩形分别返回面积并与周长并，`q.solve(a)` 是只取面积的兼容接口；默认 `Acc` 与坐标类型 `T` 相同，需要宽累计时显式构造 `RectUnion<T, __int128> q`。调用者须保证坐标差、覆盖长度、面积和周长均能由 `Acc` 表示，代码先转换端点再相减。浮点坐标仍使用 `T`。矩形 `a[i]` 的四个字段依次为左、下、右、上边界，统一左闭右开；退化矩形（`x1>=x2` 或 `y1>=y2`）被跳过，例如矩形 `{0,0,2,2}`、`{1,1,3,2}` 与退化的 `{5,0,5,3}` 面积并为 5、周长并为 10。n 个矩形产生 $2n$ 个事件，时间 $O(n\log n)$，空间 $O(n)$。同一份实现在 3.3.013 矩形面积并、周长并 复用，仅需覆盖长度时删去 `seg`/`lc`/`rc` 即退化为纯面积扫描线。

3.3.013 矩形面积并、周长并

这一条与 3.3.012 扫描线线段树 是同一知识，只是按 “矩形面积并、周长并” 的目录名再列一次，完整实现（`Rectangle<T>` 与 `RectUnion<T, Acc>`）见 3.3.012，不再重复粘贴。构造 `RectUnion<int> q` 后，`q.area(a)` 返回轴平行矩形并集面积、`q.perimeter(a)` 返回周长并；需要宽累计时改用 `RectUnion<int, __int128>`。例如矩形 `{0,0,2,2}`、`{1,1,3,2}` 与退化的 `{5,0,5,3}` 得 `area(a)==5`、`perimeter(a)==10`。n 个矩形共 $2n$ 个事件，时间 $O(n\log n)$、空间 $O(n)$。若还要支持 “矩形加、矩形求和” 这类带权二维差分而非布尔覆盖，改用 3.3.014 静态矩形加、矩形和。

3.3.014 静态矩形加、矩形和

本条就是 3.2.009 矩形修改、矩形求和 的静态离线形式，完整的 `RectAddSum<T>` 只在该处打印。一次矩形加与一次矩形和询问各拆成四个角点，按横坐标扫描；四棵树状数组分别维护 $\sum w$、$\sum wy$、$\sum wx$、$\sum wxy$，从而计算 $w(X-x)(Y-y)$ 的前缀贡献。

先用 `add(x1, y1, x2, y2, w)` 登记合法半开矩形加，再用 `query(x1, y1, x2, y2)` 登记询问并保存返回编号，最后调用一次 `work()`；`ans[id]` 是对应询问矩形内的权值和。所有询问都看到全部已登记修改，因而不能处理依赖先前答案的在线操作。设修改数为 n、询问数为 q，时间 $O((n+q)\log(n+q))$，空间 $O(n+q)$；布尔覆盖面积与周长改用 3.3.013 矩形面积并、周长并。

3.3.015 扫描线求矩形至少二重覆盖面积

求「至少被两个矩形共同覆盖」的面积，普通面积并那套只维护「至少覆盖一次的长度」就不够了，必须同时维护至少一次 `len1` 和至少两次 `len2` 两个长度。这里用永久化的整段覆盖计数 `cnt`（不下传），`pull` 时分三档合成：`cnt[p]>=2` 时整段已被覆盖两次以上，`len1=len2=full`；`cnt[p]==1` 时整段至少一次，故 `len1=full`，而「至少两次」恰是儿子里「至少一次」的部分（本层这一次叠上儿子那一次），即 `len2=` 儿子 `len1` 之和；`cnt[p]==0` 时直接取儿子的 `len1`、`len2` 之和。扫描时每推进一段横距离，累加 `seg.twice()`（即根的 `len2`）乘该横距离：

```
struct Cover2Seg
{
    vector<int> ys;   // ys 保存离散化后的纵坐标，叶子 i 表示 [ys[i], ys[i + 1])。
```

```
    vector<int> cnt;  // cnt[p] 是节点 p 对应区间被整段覆盖的次数。
    vector<int> len1; // len1[p] 是节点 p 内至少覆盖一次的实际长度。
    vector<int> len2; // len2[p] 是节点 p 内至少覆盖两次的实际长度。

    Cover2Seg(vector<int> y = {}) : ys(move(y))
    {
        // y 是严格递增的纵坐标；初始覆盖次数和长度均为零。
        int n = max<int>(1, (int)ys.size() - 1);
        cnt.assign(4 * n, 0);
        len1.assign(4 * n, 0);
        len2.assign(4 * n, 0);
    }

    void pull(int p, int l, int r)
    {
        // p、[l,r) 是当前节点；重算至少一重与至少二重覆盖长度。
        int full = ys[r] - ys[l];
        if (cnt[p] >= 2)
        {
            len1[p] = len2[p] = full;
            return;
        }
        int one = 0;
        int two = 0;
        if (r - l > 1)
        {
            one = len1[2 * p] + len1[2 * p + 1];
            two = len2[2 * p] + len2[2 * p + 1];
        }
        if (cnt[p] == 1)
        {
            len1[p] = full;
            len2[p] = one;
        }
        else
        {
            len1[p] = one;
            len2[p] = two;
        }
    }

    void add(int p, int l, int r, int ql, int qr, int v)
    {
        // p、[l,r) 是当前节点，  [ql,qr) 是修改区间，v 是覆盖次数增量。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            cnt[p] += v;
            pull(p, l, r);
```

```
            return;
        }
        auto m = (l + r) / 2;
        add(2 * p, l, m, ql, qr, v);
        add(2 * p + 1, m, r, ql, qr, v);
        pull(p, l, r);
    }

    void add(int l, int r, int v)
    {
        // [l,r) 是纵坐标段下标，v 为加入 1 或删除 -1。
        if (l < r)
        {
            add(1, 0, (int)ys.size() - 1, l, r, v);
        }
    }

    int twice() const
    {
        // 返回当前纵轴上至少被覆盖两次的总长度。
        return len2[1];
    }
};

// Rect[0..3] 依次是左 x、下 y、右 x、上 y，表示半开矩形 [x1,x2) x [y1,y2)。
using Rect2 = array<int, 4>;

int rectCover2(const vector<Rect2> &a)
{
    // a 是半开矩形集合；返回至少被两个矩形覆盖的区域面积。
    // Event[0..3] 依次是横坐标、下纵坐标、上纵坐标、覆盖增量。
    using Event = array<int, 4>;
    vector<int> ys;
    vector<Event> ev;
    for (auto [x1, y1, x2, y2] : a)
    {
        if (x1 >= x2 || y1 >= y2)
        {
            continue;
        }
        ys.push_back(y1);
        ys.push_back(y2);
        ev.push_back({x1, y1, y2, 1});
        ev.push_back({x2, y1, y2, -1});
    }
    if (ev.empty())
    {
        return 0;
    }
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());
    sort(ev.begin(), ev.end());
```

```
    Cover2Seg seg(ys);
    int ans = 0;
    int pre = ev[0][0];
    for (int i = 0; i < (int)ev.size();)
    {
        auto x = ev[i][0];
        ans += (x - pre) * seg.twice();
        while (i < (int)ev.size() && ev[i][0] == x)
        {
            int l = (int)(lower_bound(ys.begin(), ys.end(), ev[i][1]) - ys.begin());
            int r = (int)(lower_bound(ys.begin(), ys.end(), ev[i][2]) - ys.begin());
            seg.add(l, r, ev[i][3]);
            i++;
        }
        pre = x;
    }
    return ans;
}
```

直接调用 `rectCover2(a)`：`a` 是 `vector<array<int,4>>`，每项 `{x1,y1,x2,y2}` 为左、下、右、上，按左闭右开处理，退化矩形被忽略。调用者须保证坐标差、至少一重和至少二重覆盖长度及最终面积都能由 `int` 表示。例如矩形 `{0,0,4,3}`、`{2,1,5,4}`、`{1,2,3,5}` 的至少二重覆盖面积为 6：三组两两交面积之和为 8，三重交面积为 1，故至少二重部分为 $8-2\times1=6$。时间 $O(n\log n)$、空间 $O(n)$。把 `Cover2Seg` 的 `len2` 层数推广到 k 层即可求“至少 k 重覆盖”；只要普通面积并用 3.3.013 矩形面积并、周长并 即可，无需这里的双长度维护。

3.3.016 线段树分治

把「操作只在某段时间内有效」的离线问题挂到时间线段树上。将时间轴 $[0,q-1]$ 建成线段树，一个存活于时间区间 $[l,r]$ 的操作，被拆挂到覆盖 $[l,r]$ 的 $O(\log q)$ 个结点上。DFS 这棵线段树：进入某结点前把它挂的操作作用到一个可撤销数据结构上，递归到叶子即读出该时刻答案，回溯时按栈撤销本结点的全部操作。这样每个操作只在其有效时间被加入、离开时被撤销，避免了删除难以支持的问题。

下例判定「每个时刻的动态图是否二分图」：边 (u,v) 只在若干时间段存在。用扩展域并查集，把每个点 x 拆成「本色 x」与「异色 $x+n$」，加边即合并 u 与 $v+n$、$u+n$ 与 v（强制 u,v 异色）；若某刻发现 u,v 已在同一集合（同色），则出现奇环，该时间段起图非二分。数据结构必须是可撤销并查集（按秩合并、不路径压缩才能 $O(\log)$ 回滚），见 3.1.004 可撤销并查集，此处直接复用其 `RollbackDSU`（接口 `snap` / `merge` / `rollback` / `same`）。

```
// 依赖 3.1.004 可撤销并查集
// 线段树分治：离线判每个时刻的图是否二分图（扩展域 + 可撤销并查集 3.1.004）。
struct OfflineBip
{
    int n, q; // n 是点数，q 是时间轴长度（操作数）。
    vector<vector<array<int, 2>>> seg; // seg[p] 保存时间节点 p 全程存在的边 {u,v}。
    vector<char> ans;                  // ans[t] 表示时刻 t 的图是否为二分图。

    OfflineBip(int n, int q) : n(n), q(q), seg(4 * max<int>(q, 1)), ans(q)
```

```
    {
        // n 是点数，q 是时刻数；允许空时间轴 q=0。
        assert(n >= 0 && q >= 0); // 调试检查，可删
    }

    void addEdge(int u, int v, int l, int r, int node = 1, int lo = 0, int hi = -1)
    {
        // 边 (u,v) 在时刻区间 [l,r] 存在；挂到覆盖它的 O(log q) 个结点。
        assert(q > 0 && 0 <= u && u < n && 0 <= v && v < n && 0 <= l && l <= r && r < q); // 调
        ↪ 试检查，可删
        if (hi < 0)
        {
            hi = q - 1;
        }
        if (l > hi || r < lo)
        {
            return;
        }
        if (l <= lo && hi <= r)
        {
            seg[node].push_back({u, v});
            return;
        }
        int mid = (lo + hi) / 2;
        addEdge(u, v, l, r, node * 2, lo, mid);
        addEdge(u, v, l, r, node * 2 + 1, mid + 1, hi);
    }

    void solve(RollbackDSU &dsu, int node = 1, int lo = 0, int hi = -1, bool ok = true)
    {
        // dsu 大小为 2n（扩展域），ok 记当前时间段是否仍是二分图。
        if (!q)
        {
            return;
        }
        if (hi < 0)
        {
            hi = q - 1;
        }
        int s = dsu.snap();
        for (auto [u, v] : seg[node])
        {
            if (dsu.same(u, v))
            {
                ok = false;
                break;
            }
            dsu.merge(u, v + n);
            dsu.merge(u + n, v);
        }
        if (lo == hi)
        {
```

```
            ans[lo] = ok;
        }
        else
        {
            int mid = (lo + hi) / 2;
            solve(dsu, node * 2, lo, mid, ok);
            solve(dsu, node * 2 + 1, mid + 1, hi, ok);
        }
        dsu.rollback(s);
    }
};
```

`OfflineBip(n,q)` 后对每条边调用 `addEdge(u,v,l,r)`，把它挂到闭时间段 $[l,r]$；再传入大小为 $2n$ 的 `RollbackDSU` 调 `solve`，`ans[t]` 即时刻 t 的图是否二分。`q=0` 时 `solve` 直接返回空答案。验证：$n=3$、边 $(0,1),(1,2)$ 全程存在、$(0,2)$ 仅在 $t=1$ 存在，答案为 `1 0 1`。同一框架换掉可撤销状态即可处理其他时间区间操作。

设登记了 m 条存活区间，初始化扩展域为 $O(n)$，遍历时间树为 $O(q)$，每条边挂到 $O(\log q)$ 个节点并各做常数次 $O(\log n)$ 的并查集操作，总时间 $O(n+q+m\log q\log n)$；时间树挂边、答案和回滚状态合计空间 $O(n+q+m\log q)$。

3.3.017 动态区间严格众数

严格众数是出现次数大于区间长度一半的值，若存在则唯一。Boyer-Moore 抵消把一段序列压成 `{候选值, 未抵消票数}`：相同候选合并票数，不同候选让较小票数被抵消；无论线段树怎样分段，真正的严格众数都不会被全部抵消。不过抵消结果在没有严格众数时仍可能留下任意候选，甚至票数恰好为零，因此查询后必须先检查剩余票数，再数一次候选在 $[l,r)$ 内的真实频次。本实现预先登记所有可能的单点赋值 `{位置, 新值}`，为每个值单独建立按位置离散的 Fenwick；线段树只找候选，Fenwick 负责验证。

```
class MajRange
{
    // S[0] 是候选值编号，S[1] 是抵消后的剩余票数。
    using S = array<int, 2>;

    // n 是数组长度，z 是线段树叶子基址；a 保存当前值编号，val 保存原值。
    int n, z = 1;
    vector<int> a, val;
    vector<S> tr;
    // pos[k] 是值 k 预登记的位置，bit[k] 维护这些位置当前是否取值 k。
    vector<vector<int>> pos, bit;

    static S merge(S x, S y)
    {
        // x、y 是相邻两段的抵消摘要；返回合并摘要。
        if (!x[1])
        {
            return y;
        }
        if (!y[1])
```

```
        {
            return x;
        }
        if (x[0] == y[0])
        {
            return {x[0], x[1] + y[1]};
        }
        return x[1] >= y[1] ? S{x[0], x[1] - y[1]} : S{y[0], y[1] - x[1]};
    }

    int id(int x) const
    {
        // x 是已预登记的原值；返回其离散编号。
        auto it = lower_bound(val.begin(), val.end(), x);
        assert(it != val.end() && *it == x); // 调试检查，可删
        return it - val.begin();
    }

    void add(int k, int p, int d)
    {
        // k 是值编号，p 是预登记位置，d 是出现次数增量。
        auto it = lower_bound(pos[k].begin(), pos[k].end(), p);
        assert(it != pos[k].end() && *it == p); // 调试检查，可删
        for (int i = it - pos[k].begin() + 1; i < (int)bit[k].size(); i += i & -i)
        {
            bit[k][i] += d;
        }
    }

    int pre(int k, int p) const
    {
        // k 是值编号，p 是位置上界；返回该值在 [0,p) 的出现次数。
        int i = lower_bound(pos[k].begin(), pos[k].end(), p) - pos[k].begin();
        int ans = 0;
        for (; i; i -= i & -i)
        {
            ans += bit[k][i];
        }
        return ans;
    }

    void setLeaf(int p, int x)
    {
        // p 是数组位置，x 是新值编号；修改候选线段树的对应叶子。
        p += z;
        tr[p] = {x, 1};
        while (p >>= 1)
        {
            tr[p] = merge(tr[2 * p], tr[2 * p + 1]);
        }
    }

```

```
    S prod(int l, int r) const
    {
        // [l,r) 是查询区间；返回按原顺序抵消后的候选摘要。
        S x{-1, 0}, y{-1, 0};
        for (l += z, r += z; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1)
            {
                x = merge(x, tr[l++]);
            }
            if (r & 1)
            {
                y = merge(tr[--r], y);
            }
        }
        return merge(x, y);
    }

  public:
    MajRange(const vector<int> &v, const vector<pair<int, int>> &plan = {}) : n(v.size())
    {
        // v 是初始数组，plan 列出以后所有可能的 {赋值位置，新值}。
        val = v;
        for (auto [p, x] : plan)
        {
            assert(0 <= p && p < n); // 调试检查，可删
            val.push_back(x);
        }
        sort(val.begin(), val.end());
        val.erase(unique(val.begin(), val.end()), val.end());
        a.resize(n);
        pos.resize(val.size());
        for (int i = 0; i < n; i++)
        {
            a[i] = id(v[i]);
            pos[a[i]].push_back(i);
        }
        for (auto [p, x] : plan)
        {
            pos[id(x)].push_back(p);
        }
        bit.resize(pos.size());
        for (int k = 0; k < (int)pos.size(); k++)
        {
            auto &p = pos[k];
            sort(p.begin(), p.end());
            p.erase(unique(p.begin(), p.end()), p.end());
            bit[k].assign(p.size() + 1, 0);
        }
        while (z < max<int>(1, n))
        {
            z *= 2;
```

```
        }
        tr.assign(2 * z, S{-1, 0});
        for (int i = 0; i < n; i++)
        {
            tr[z + i] = {a[i], 1};
            add(a[i], i, 1);
        }
        for (int p = z - 1; p; p--)
        {
            tr[p] = merge(tr[2 * p], tr[2 * p + 1]);
        }
    }

    void set(int p, int x)
    {
        // p 是数组位置，x 是已在 plan 登记的新值；完成单点赋值。
        assert(0 <= p && p < n); // 调试检查，可删
        int k = id(x);
        add(a[p], p, -1);
        add(k, p, 1);
        a[p] = k;
        setLeaf(p, k);
    }

    optional<int> query(int l, int r) const
    {
        // [l,r) 是查询区间；返回严格众数，空区间或不存在时返回空。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return nullopt;
        }
        S s = prod(l, r);
        if (!s[1])
        {
            return nullopt;
        }
        int k = s[0];
        int c = pre(k, r) - pre(k, l);
        return 2 * c > r - l ? optional<int>(val[k]) : nullopt;
    }
};
```

构造 `MajRange q(a,plan)` 时，`plan` 必须覆盖以后每次 `set(p,x)` 的位置和值组合；查询端点不必预登记。`set` 与 `query` 均为 $O(\log n)$，预处理 $O((n+q)\log(n+q))$，空间 $O(n+q)$。若数组静态且只需任意众数而非严格过半，用 3.4.008 静态区间众数；严格众数的优势正是候选唯一，不能把本结构改名成一般区间众数。

```
signed main()
{
    vector<int> a{1, 2, 1, 3, 1};
```

```
    vector<pair<int, int>> plan{{1, 1}};
    MajRange q(a, plan);
    assert(q.query(0, 5) == optional<int>(1));
    assert(!q.query(0, 2));
    q.set(1, 1);
    assert(q.query(0, 4) == optional<int>(1));
}
```

3.3.018 可排序线段树

可排序线段树维护一列 {键，幺半群值}，支持单点赋值、区间按键升序或降序排序，以及按当前次序查询区间积。它把序列划成若干已经单调的段：`cut` 记录段首，每段用一棵动态值域线段树保存键，节点同时维护升序积 `f` 和降序积 `b`；外层线段树只在段首保存整段积。查询或排序先用 `splitAt` 在端点按元素个数切段，排序再破坏性合并区间内各值域根并设置方向。所有键必须始终全局互异，否则两棵值域树会在同一叶相撞，排序次序也不再由键唯一确定。

```
template <class M> class SortSeg
{
  public:
    using T = typename M::T;

  private:
    struct Node
    {
        // l、r 是值域左右儿子；sz 是元素数；f、b 分别按键升序、降序合并。
        int l = 0, r = 0, sz = 0;
        T f, b;

        Node(const T &e) : f(e), b(e) {}
    };

    // n 是序列长度，K 是键值域右端点，z 是外层线段树叶子基址。
    int n, K, z = 1;
    // t 是动态值域线段树节点池；rt[p] 是以 p 开头的有序段根；rev[p] 表示该段降序。
    vector<Node> t;
    vector<int> rt;
    vector<char> rev;
    // cut 保存所有有序段左端点及哨兵 n；seg 仅在段首保存整段幺半群积。
    set<int> cut;
    vector<T> seg;

    int newNode()
    {
        // 新建空值域节点并返回编号。
        t.emplace_back(M::id());
        return (int)t.size() - 1;
    }

    int size(int p) const
```

```
    {
        // p 是值域节点编号；返回其元素数，空节点返回 0。
        return p ? t[p].sz : 0;
    }

    void pull(int p)
    {
        // p 是非叶值域节点；由非空儿子重算元素数和双向积。
        int x = t[p].l, y = t[p].r;
        t[p].sz = size(x) + size(y);
        if (!x)
        {
            t[p].f = t[y].f;
            t[p].b = t[y].b;
        }
        else if (!y)
        {
            t[p].f = t[x].f;
            t[p].b = t[x].b;
        }
        else
        {
            t[p].f = M::op(t[x].f, t[y].f);
            t[p].b = M::op(t[y].b, t[x].b);
        }
    }

    int put(int p, int l, int r, int k, const T &x)
    {
        // p 管理键区间 [l,r)，k 是新键，x 是对应值；返回插入后的根。
        if (!p)
        {
            p = newNode();
        }
        if (r - l == 1)
        {
            t[p].sz = 1;
            t[p].f = t[p].b = x;
            return p;
        }
        int m = l + (r - l) / 2;
        if (k < m)
        {
            t[p].l = put(t[p].l, l, m, k, x);
        }
        else
        {
            t[p].r = put(t[p].r, m, r, k, x);
        }
        pull(p);
        return p;
    }
```

```
86
87      pair<int, int> split(int p, int k, int l, int r)
88      {
89          // p 管理键区间 [l,r)，k 是左侧所取元素数；返回按键升序切开的两根。
90          if (!k)
91          {
92              return {0, p};
93          }
94          if (k == size(p))
95          {
96              return {p, 0};
97          }
98          int m = l + (r - l) / 2;
99          int q = newNode();
100         int s = size(t[p].l);
101         if (k <= s)
102         {
103             auto [x, y] = split(t[p].l, k, l, m);
104             int v = t[p].r;
105             t[p].l = x;
106             t[p].r = 0;
107             t[q].l = y;
108             t[q].r = v;
109         }
110         else
111         {
112             auto [x, y] = split(t[p].r, k - s, m, r);
113             int v = t[p].l;
114             t[p].l = v;
115             t[p].r = x;
116             t[q].l = 0;
117             t[q].r = y;
118         }
119         pull(p);
120         pull(q);
121         return {p, q};
122     }
123
124     int join(int p, int q, int l, int r)
125     {
126         // p、q 管理同一键区间 [l,r) 且键集不交；破坏性合并并返回根。
127         if (!p || !q)
128         {
129             return p | q;
130         }
131         assert(r - l > 1); // 调试检查，可删：合法键全局互异，不会在同一叶相遇
132         int m = l + (r - l) / 2;
133         t[p].l = join(t[p].l, t[q].l, l, m);
134         t[p].r = join(t[p].r, t[q].r, m, r);
135         pull(p);
136         return p;
137     }
```

```
138
139     T all(int p, bool d) const
140     {
141         // p 是非空值域根，d 表示是否降序；返回该顺序下的整段积。
142         return d ? t[p].b : t[p].f;
143     }
144
145     void outSet(int p, const T &x)
146     {
147         // p 是序列位置，x 是新的段首积或幺元；修改外层线段树。
148         p += z;
149         seg[p] = x;
150         while (p >>= 1)
151         {
152             seg[p] = M::op(seg[2 * p], seg[2 * p + 1]);
153         }
154     }
155
156     T outProd(int l, int r) const
157     {
158         // [l,r) 是序列区间；返回外层线段树中段首积的顺序合并。
159         T x = M::id(), y = M::id();
160         for (l += z, r += z; l < r; l >>= 1, r >>= 1)
161         {
162             if (l & 1)
163             {
164                 x = M::op(x, seg[l++]);
165             }
166             if (r & 1)
167             {
168                 y = M::op(seg[--r], y);
169             }
170         }
171         return M::op(x, y);
172     }
173
174     void splitAt(int p)
175     {
176         // p 是序列边界；若它落在有序段内部，就把该段切成两段。
177         if (p == n || cut.contains(p))
178         {
179             return;
180         }
181         auto it = cut.upper_bound(p);
182         int r = *it, l = *prev(it);
183         cut.insert(p);
184         if (!rev[l])
185         {
186             auto [x, y] = split(rt[l], p - l, 0, K);
187             rt[l] = x;
188             rt[p] = y;
189             rev[l] = rev[p] = false;
```

```
        }
        else
        {
            auto [x, y] = split(rt[l], r - p, 0, K);
            rt[l] = y;
            rt[p] = x;
            rev[l] = rev[p] = true;
        }
        outSet(l, all(rt[l], rev[l]));
        outSet(p, all(rt[p], rev[p]));
    }

    void sortRange(int l, int r, bool d)
    {
        // [l,r) 是排序区间，d 表示是否降序；合并其中各段并设置方向。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return;
        }
        splitAt(l);
        splitAt(r);
        auto it = cut.upper_bound(l);
        while (*it < r)
        {
            int p = *it;
            rt[l] = join(rt[l], rt[p], 0, K);
            rt[p] = 0;
            rev[p] = false;
            outSet(p, M::id());
            it = cut.erase(it);
        }
        rev[l] = d;
        outSet(l, all(rt[l], d));
    }

  public:
    SortSeg(int K, const vector<int> &key, const vector<T> &a) : n(key.size()), K(K), rt(n),
↪   rev(n)
    {
        // K 是键值域 [0,K) 的右端点；key、a 分别是每个位置的互异键和幺半群值。
        assert(K > 0 && key.size() == a.size()); // 调试检查，可删
        t.emplace_back(M::id());
        while (z < max<int>(1, n))
        {
            z *= 2;
        }
        seg.assign(2 * z, M::id());
        cut.insert(n);
        for (int i = 0; i < n; i++)
        {
            assert(0 <= key[i] && key[i] < K); // 调试检查，可删
```

```
            cut.insert(i);
            rt[i] = put(0, 0, K, key[i], a[i]);
            seg[z + i] = a[i];
        }
        for (int p = z - 1; p; p--)
        {
            seg[p] = M::op(seg[2 * p], seg[2 * p + 1]);
        }
    }

    void assign(int p, int k, const T &x)
    {
        // p 是序列位置，k 是替换后的全局唯一键，x 是新值；完成单点赋值。
        assert(0 <= p && p < n && 0 <= k && k < K); // 调试检查，可删
        splitAt(p);
        splitAt(p + 1);
        rt[p] = put(0, 0, K, k, x);
        rev[p] = false;
        outSet(p, x);
    }

    void sortAsc(int l, int r)
    {
        // [l,r) 是排序区间；按键从小到大重排。
        sortRange(l, r, false);
    }

    void sortDesc(int l, int r)
    {
        // [l,r) 是排序区间；按键从大到小重排。
        sortRange(l, r, true);
    }

    T prod(int l, int r)
    {
        // [l,r) 是查询区间；返回当前顺序下的幺半群积。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        splitAt(l);
        splitAt(r);
        return outProd(l, r);
    }

    T prodAll() const
    {
        // 返回整个序列当前顺序下的幺半群积。
        return seg[1];
    }
};
```

`M::T` 是值类型，`M::id()` 返回幺元，`M::op(x,y)` 按先左后右的次序结合；键位于 $[0,K)$，构造和每次 `assign(p,k,x)` 后都必须两两不同。`sortAsc(l,r)`、`sortDesc(l,r)` 重排 $[l,r)$，`prod(l,r)` 返回当前顺序下的积。设值域树高 $H=O(\log K)$；单点赋值和区间查询为 $O(\log n+H)$。一次排序可能合并很多

现存段，但每个被删段界都来自初始化或一次端点切分，每个被重新访问的值域分支也可计入此前建立或切分出的节点，因此从建树起的 q 次操作总时间为 $O((n+q)(\log n+H))$，空间为 $O(n+(n+q)H)$。下面用非交换的仿射函数复合检查升序、降序、区间积与单点替换：

```cpp
constexpr int mod = 998244353;

struct Aff
{
    int a, b; // 仿射变换 x -> a*x+b 的两个系数（模 mod）。
    bool operator==(const Aff &) const = default;
};

struct AffMonoid
{
    using T = Aff;

    static T id()
    {
        // 无参数；返回恒等变换 x -> x。
        return {1, 0};
    }

    static T op(T x, T y)
    {
        // x、y 是仿射变换；返回先做 x 再做 y 的复合变换。
        return {y.a * x.a % mod, (y.a * x.b + y.b) % mod};
    }
};

signed main()
{
    vector<int> key{3, 1, 2, 0};
    vector<Aff> a{{1, 1}, {2, 0}, {1, 3}, {3, 1}};
    SortSeg<AffMonoid> st(8, key, a);
    assert((st.prodAll() == Aff{6, 16}));
    st.sortAsc(0, 4);
    assert((st.prodAll() == Aff{6, 6}));
    st.sortDesc(1, 4);
    assert((st.prod(1, 3) == Aff{1, 4}));
    st.assign(2, 7, {2, 1});
    st.sortAsc(1, 4);
    assert((st.prodAll() == Aff{12, 7}));
}
```

3.3.019 区间一次函数加、区间最小值

把数组看成点集 (i,a_i)；修改 $[l,r)$ 时给每个纵坐标加 $bi+c$，就是对相应点集施加保持横坐标的仿射剪切，整段修改不会改变该段下凸包的组合结构。`LineMinSeg` 在每个内部节点保存左右儿子下凸包的公切桥，懒标记保存尚未下放的一次函数。部分修改后，`pull` 沿两棵儿子的桥递归寻找新公切点；查询一个完整节点时比较当前桥两端的纵坐标，就能向含最小点的儿子下降。这里的桥判定不是普通线段树的常数时

间合并，不能删成 `min` 信息后仍声称支持区间一次函数加。复用为最大值版时，仅当初值、增量与函数值都能安全取反，才保存 `-a[i]`、把 `add(l,r,b,c)` 改为 `add(l,r,-b,-c)` 并对查询结果取负；只把最后的 `min` 改成 `max` 会破坏下凸包桥判定，遇到最小负数则必须完整反向比较。

```
class LineMinSeg
{
    // 当前维护区间最小值。若 a、b、c 均能安全取反，可保存 -a[i]：每次 add(l,r,b,c) 改为
    // add(l,r,-b,-c)，查询结果再取负；若存在最小负数则完整反向桥判定，不要只改最后一个 min。
    struct P
    {
        // x 是数组下标，y 是该位置的当前值。
        long long x = 0, y = 0;

        bool operator==(const P &) const = default;
    };

    // L[0]、L[1] 分别是一次函数的斜率和截距；B[0]、B[1] 是桥的左右端点。
    using L = array<long long, 2>;
    using B = array<P, 2>;

    // n 是原数组长度，z 是补成二次幂后的叶子数，h 是树高。
    int n, z = 1, h = 0;
    // pt 是叶子点；lz 是整段待加的一次函数；br 是两棵儿子下凸包的公切桥。
    vector<P> pt;
    vector<L> lz;
    vector<B> br;

    static L merge(L a, L b)
    {
        // a、b 是先后累加的一次函数；返回逐项相加后的函数。
        a[0] += b[0];
        a[1] += b[1];
        return a;
    }

    static P apply(P p, L f)
    {
        // p 是平面点，f 是待加的一次函数；返回纵坐标增加 f(p.x) 后的点。
        p.y = (long long)((__int128)p.y + (__int128)p.x * f[0] + f[1]);
        return p;
    }

    static __int128 cross(P a, P b, P c)
    {
        // a、b、c 是三个点；返回向量 ab 与 ac 的叉积。
        return ((__int128)b.x - a.x) * ((__int128)c.y - a.y)
             - ((__int128)b.y - a.y) * ((__int128)c.x - a.x);
    }

    static __int128 cross(P a, P b, P c, P d)
    {
        // a、b、c、d 是四个点；返回向量 ab 与 cd 的叉积。
```

```
        return ((__int128)b.x - a.x) * ((__int128)d.y - c.y)
             - ((__int128)b.y - a.y) * ((__int128)d.x - c.x);
    }

    void pull(int p)
    {
        // p 是内部节点；重新寻找左右儿子下凸包之间的公切桥。
        if (p >= z)
        {
            return;
        }
        L off{0, 0};
        for (int x = p; x; x >>= 1)
        {
            off = merge(off, lz[x]);
        }
        int l = 2 * p, r = 2 * p + 1;
        int mid = r;
        while (mid < z)
        {
            mid *= 2;
        }
        mid -= z;
        L fl = lz[l], fr = lz[r];
        while (l < z || r < z)
        {
            P a = apply(apply(br[l][0], off), fl);
            P b = apply(apply(br[l][1], off), fl);
            P c = apply(apply(br[r][0], off), fr);
            P d = apply(apply(br[r][1], off), fr);
            if (a != b && cross(a, b, c) < 0)
            {
                l *= 2;
                fl = merge(fl, lz[l]);
            }
            else if (c != d && cross(b, c, d) < 0)
            {
                r = 2 * r + 1;
                fr = merge(fr, lz[r]);
            }
            else if (a == b)
            {
                r *= 2;
                fr = merge(fr, lz[r]);
            }
            else if (c == d)
            {
                l = 2 * l + 1;
                fl = merge(fl, lz[l]);
            }
            else
            {
```

```
                __int128 c1 = cross(a, b, c, d);
                __int128 c2 = cross(a, b, c, b);
                bool left;
                if (!c1 && !c2)
                {
                    left = c.x < mid;
                }
                else
                {
                    left = (__int128)c.x * c1 + ((__int128)d.x - c.x) * c2
                         < c1 * mid;
                }
                if (left)
                {
                    l = 2 * l + 1;
                    fl = merge(fl, lz[l]);
                }
                else
                {
                    r *= 2;
                    fr = merge(fr, lz[r]);
                }
            }
        }
        br[p] = {apply(pt[l], fl), apply(pt[r], fr)};
    }

    long long subMin(int p) const
    {
        // p 是完整覆盖的线段树节点；返回其区间最小值。
        L f{0, 0};
        for (int x = p; x; x >>= 1)
        {
            f = merge(f, lz[x]);
        }
        while (p < z)
        {
            P a = apply(br[p][0], f), b = apply(br[p][1], f);
            p = a.y < b.y ? 2 * p : 2 * p + 1;
            f = merge(f, lz[p]);
        }
        return apply(pt[p], f).y;
    }

  public:
    LineMinSeg(const vector<long long> &a) : n(a.size())
    {
        // a 是非空初始数组；建立每个节点的下凸包桥。
        assert(n > 0); // 调试检查，可删
        while (z < n)
        {
            z *= 2;
```

```
            h++;
        }
        pt.resize(2 * z);
        lz.assign(2 * z, L{0, 0});
        br.resize(2 * z);
        for (int i = 0; i < z; i++)
        {
            pt[z + i] = {i, i < n ? a[i] : 4'000'000'000'000'000'000LL};
            br[z + i] = {pt[z + i], pt[z + i]};
        }
        for (int p = z - 1; p; p--)
        {
            pull(p);
        }
    }

    void add(int l, int r, long long b, long long c)
    {
        // [l,r) 是修改区间，b、c 定义一次函数 b*i+c；把它加到每个 a[i]。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return;
        }
        int x = l + z, y = r + z;
        while (x < y)
        {
            if (x & 1)
            {
                lz[x] = merge(lz[x], L{b, c});
                x++;
            }
            if (y & 1)
            {
                --y;
                lz[y] = merge(lz[y], L{b, c});
            }
            x >>= 1;
            y >>= 1;
        }
        x = l + z;
        y = r + z;
        for (int d = 1; d <= h; d++)
        {
            if ((x >> d << d) != x)
            {
                pull(x >> d);
            }
            if ((y >> d << d) != y)
            {
                pull((y - 1) >> d);
            }
```

```
        }
    }

    long long min(int l, int r) const
    {
        // [l,r) 是非空查询区间；返回其中最小值。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        long long ans = 4'000'000'000'000'000'000LL;
        for (l += z, r += z; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1)
            {
                long long v = subMin(l++);
                ans = ans < v ? ans : v;
            }
            if (r & 1)
            {
                long long v = subMin(--r);
                ans = ans < v ? ans : v;
            }
        }
        return ans;
    }
};
```

构造 `LineMinSeg st(a)` 后，`st.add(l,r,b,c)` 执行 $a_i \mathrel{+}= bi + c$ $(l \le i < r)$，`st.min(l,r)` 查询非空区间最小值。建树 $O(n \log n)$，一次修改或查询均为 $O(\log^2 n)$，空间 $O(n)$。调用者必须保证初值、维护中的每个 a_i、累积斜率与截距及 `a_i+bi+c` 都可由 `long long` 表示，且 $|a_i| < 4 \times 10^{18}$；所有叉积和桥侧判定还须落在 `__int128` 内，例如 $n \le 2 \times 10^5$ 且所有维护值绝对值严格小于 4×10^{18} 时满足这一项。只有区间加常数时用 3.3.002 懒标记线段树 更短；只有一次函数修改而查询区间和时，按下标和长度维护两项和即可，不需要凸包桥。下面用四个数检查局部一次函数修改和跨段查询：

```
signed main()
{
    LineMinSeg st(vector<long long>{5, -2, 4, 1});
    assert(st.min(0, 4) == -2);
    st.add(1, 4, 2, -3);
    assert(st.min(0, 4) == -3);
    assert(st.min(2, 4) == 4);
    st.add(0, 2, -1, 4);
    assert(st.min(0, 4) == 0);
}
```

3.3.020 可持久化仿射区间和

给定固定长度的数组，若每次修改都要从任意历史版本分叉，对半开区间 $[l, r)$ 施加 $x \mapsto ax + b$，还要把另一个历史版本的同下标区间覆盖过来，就不能直接使用会原地改写节点的 3.3.003 区间加、乘、赋值（仿射懒标记）。`PerAffSeg<T>` 为每个版本保存一个根编号；`affine(root,l,r,mul,add)` 返回新根，`splice(target,source,l,r)` 返回把 `target` 的 $[l, r)$ 替换为 `source` 同一坐标 $[l, r)$ 后的新根，

`sum(root,l,r)` 则只读查询任意历史版本。它处理的是固定坐标上的版本拼接，不支持插入、删除或变长序列。

每个节点的 `sum` 已经计入自身懒标记 (m, a)，而两个标记按先旧后新的次序复合：

$$(m_2, a_2) \circ (m_1, a_1) = (m_1 m_2, a_1 m_2 + a_2), \qquad S \leftarrow mS + (R - L)a.$$

所有会写节点的路径先复制；`push` 只接收刚复制的父节点，并把标记施加到新复制的两个儿子，所以旧根永不被改写。查询不下传标记，而是携带祖先尚未落实的仿射变换。`splice` 部分相交时同样只复制目标路径；源节点若带标记，仅复制实际进入的源儿子并落实该标记。这样覆盖整段时可以直接复用源根，分叉和交叉复制都保持持久化。

```
// 依赖 1.1.008 静态取模类
template <class T> class PerAffSeg
{
    struct Node
    {
        // l、r 是左右儿子编号；sum 是当前区间已经计入本节点懒标记后的元素和。
        int l = 0, r = 0;
        T sum{};
        // mul、add 是尚未下传的变换 x -> mul*x+add。
        T mul{}, add{};
    };

    // n 是初始数组长度，one 是 T 的乘法单位元，base 是初始版本根。
    int n;
    T one;
    int base = 0;
    // t 是全部版本共用的节点池；0 号节点只作哨兵，不是合法版本根。
    vector<Node> t;

    int newNode()
    {
        // 新建空节点并返回编号；新节点的仿射标记初始化为恒等变换。
        t.push_back(Node{});
        int p = (int)t.size() - 1;
        assert(p <= numeric_limits<int>::max()); // 调试检查，可删
        t[p].mul = one;
        return p;
    }

    int clone(int p)
    {
        // p 是旧版本中的非空节点；复制它并返回新节点编号。
        assert(p > 0); // 调试检查，可删
        t.push_back(t[p]);
        int q = (int)t.size() - 1;
        assert(q <= numeric_limits<int>::max()); // 调试检查，可删
        return q;
    }

```

```
    bool idle(int p) const
    {
        // p 是非空节点；返回其待下传仿射标记是否为恒等变换。
        return t[p].mul == one && t[p].add == T{};
    }

    void pull(int p)
    {
        // p 是已经无本层懒标记的内部节点；由两个儿子重算当前区间和。
        t[p].sum = t[t[p].l].sum + t[t[p].r].sum;
    }

    void applyAt(int p, int l, int r, const T &mul, const T &add)
    {
        // p 是可独占修改的节点，[l,r) 是其区间；就地叠加后执行的仿射变换。
        t[p].sum = t[p].sum * mul + T(r - l) * add;
        t[p].mul = t[p].mul * mul;
        t[p].add = t[p].add * mul + add;
    }

    int copyApply(int p, int l, int r, const T &mul, const T &add)
    {
        // p 是旧节点，[l,r) 是其区间；复制后施加仿射变换并返回新根。
        int q = clone(p);
        applyAt(q, l, r, mul, add);
        return q;
    }

    void push(int p, int l, int r)
    {
        // p 是已复制且可修改的内部节点，[l,r) 是其区间；持久化地下传本层懒标记。
        if (r - l == 1 || idle(p))
        {
            return;
        }
        int m = (l + r) / 2;
        T mul = t[p].mul, add = t[p].add;
        t[p].l = copyApply(t[p].l, l, m, mul, add);
        t[p].r = copyApply(t[p].r, m, r, mul, add);
        t[p].mul = one;
        t[p].add = T{};
    }

    int srcL(int p, int l, int r)
    {
        // p 是源版本节点，[l,r) 是其内部区间；返回已落实 p 标记的左儿子版本根。
        if (idle(p))
        {
            return t[p].l;
        }
        int m = (l + r) / 2;
        T mul = t[p].mul, add = t[p].add;
```

```
92          return copyApply(t[p].l, l, m, mul, add);
93      }
94
95      int srcR(int p, int l, int r)
96      {
97          // p 是源版本节点，[l,r) 是其内部区间；返回已落实 p 标记的右儿子版本根。
98          if (idle(p))
99          {
100             return t[p].r;
101         }
102         int m = (l + r) / 2;
103         T mul = t[p].mul, add = t[p].add;
104         return copyApply(t[p].r, m, r, mul, add);
105     }
106
107     int build(const vector<T> &a, int l, int r)
108     {
109         // a 是初始数组，[l,r) 是当前建树区间；返回该区间的初始版本根。
110         int p = newNode();
111         if (r - l == 1)
112         {
113             t[p].sum = a[l];
114             return p;
115         }
116         int m = (l + r) / 2;
117         t[p].l = build(a, l, m);
118         t[p].r = build(a, m, r);
119         pull(p);
120         return p;
121     }
122
123     int affine(int p, int l, int r, int ql, int qr, const T &mul, const T &add)
124     {
125         // p 是旧版本节点，[l,r) 是其区间；对交集施加仿射变换并返回新节点。
126         if (qr <= l || r <= ql)
127         {
128             return p;
129         }
130         int q = clone(p);
131         if (ql <= l && r <= qr)
132         {
133             applyAt(q, l, r, mul, add);
134             return q;
135         }
136         push(q, l, r);
137         int m = (l + r) / 2;
138         t[q].l = affine(t[q].l, l, m, ql, qr, mul, add);
139         t[q].r = affine(t[q].r, m, r, ql, qr, mul, add);
140         pull(q);
141         return q;
142     }
143
```

```
144     int splice(int target, int source, int l, int r, int ql, int qr)
145     {
146         // target、source 是两个版本节点；把 source 在交集的位置覆盖到 target 并返回新节点。
147         if (qr <= l || r <= ql)
148         {
149             return target;
150         }
151         if (ql <= l && r <= qr)
152         {
153             return source;
154         }
155         int q = clone(target);
156         push(q, l, r);
157         int m = (l + r) / 2;
158         if (ql < m)
159         {
160             int x = srcL(source, l, r);
161             t[q].l = splice(t[q].l, x, l, m, ql, qr);
162         }
163         if (m < qr)
164         {
165             int x = srcR(source, l, r);
166             t[q].r = splice(t[q].r, x, m, r, ql, qr);
167         }
168         pull(q);
169         return q;
170     }
171 
172     T sum(int p, int l, int r, int ql, int qr, const T &outerMul, const T &outerAdd) const
173     {
174         // p 是版本节点，[l,r) 是其区间；outer 表示祖先尚未落实的仿射变换，返回交集和。
175         if (qr <= l || r <= ql)
176         {
177             return T{};
178         }
179         if (ql <= l && r <= qr)
180         {
181             return t[p].sum * outerMul + T(r - l) * outerAdd;
182         }
183         T mul = t[p].mul * outerMul;
184         T add = t[p].add * outerMul + outerAdd;
185         int m = (l + r) / 2;
186         return sum(t[p].l, l, m, ql, qr, mul, add)
187              + sum(t[p].r, m, r, ql, qr, mul, add);
188     }
189 
190   public:
191     explicit PerAffSeg(const vector<T> &a) : n((int)a.size()), one(T(1)), t(1)
192     {
193         // a 是非空初始数组；T 需支持 0、1、加法、乘法、相等比较和由长度构造。
194         assert(n > 0); // 调试检查，可删
195         t.reserve(2 * n + 1);
```

```
        t[0].mul = one;
        base = build(a, 0, n);
    }

    int root() const
    {
        // 返回初始数组对应的版本根；此根可与后续版本根一起长期保存。
        return base;
    }

    int affine(int root, int l, int r, const T &mul, const T &add)
    {
        // root 是旧版本根，[l,r) 是修改区间；返回执行 x->mul*x+add 后的新版本根。
        assert(0 < root && root < (int)t.size()); // 调试检查，可删
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r || (mul == one && add == T{}))
        {
            return root;
        }
        return affine(root, 0, n, l, r, mul, add);
    }

    int splice(int target, int source, int l, int r)
    {
        // target、source 是旧版本根；返回 target 的 [l,r) 被 source 同位置覆盖后的新版本根。
        assert(0 < target && target < (int)t.size()); // 调试检查，可删
        assert(0 < source && source < (int)t.size()); // 调试检查，可删
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (target == source || l == r)
        {
            return target;
        }
        return splice(target, source, 0, n, l, r);
    }

    T sum(int root, int l, int r) const
    {
        // root 是任意历史版本根，[l,r) 是查询区间；返回该区间当前元素和，空区间返回零。
        assert(0 < root && root < (int)t.size()); // 调试检查，可删
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return sum(root, 0, n, l, r, one, T{});
    }
};
```

初始数组必须非空，所有公开区间都采用 $0 \le l \le r \le n$ 的半开约定；根编号只能来自 `root`、`affine` 或 `splice`。`T` 至少要能构造 0、1 和长度，支持加法、乘法和相等比较；模 998244353 时直接取 1.1.008 静态取模类 的 `Z`。例如先给整个初始版本乘 3 加 1，再从另一条分支复制中段：

```
signed main()
{
    vector<Z> a{1, 2, 3, 4};
```

```
    PerAffSeg<Z> seg(a);
    int r0 = seg.root();
    int r1 = seg.affine(r0, 0, 4, Z(3), Z(1));
    int r2 = seg.affine(r0, 1, 3, Z(5), Z(0));
    int r3 = seg.splice(r1, r2, 1, 3);
    assert(seg.sum(r0, 0, 4) == Z(10));
    assert(seg.sum(r3, 0, 4) == Z(42));
}
```

建树时间和初始空间为 $O(n)$；一次 `affine`、`splice` 或 `sum` 的时间为 $O(\log n)$，前两者新增 $O(\log n)$ 个节点，因而保存 q 次修改/覆盖的总空间为 $O(n+q\log n)$。`splice` 的常数会受需要落实的源端懒标记影响，节点池按实际需求增长；$n,q\le 10^5$ 的场景应预留足够内存，不能把普通 3.3.008 可持久化线段树 / 主席树 的单点修改节点数上界照搬过来。只需当前版本的仿射区间和时用更短的 3.3.003 区间加、乘、赋值（仿射懒标记）；只做值域上的历史单点计数或第 k 小时用 3.3.008 可持久化线段树 / 主席树；坐标会插入、删除或翻转时改用 3.5.003 可持久化 Treap。

3.3.021 稀疏仿射区间和

若数组长度可达 10^9、初值全部为零、操作却只有 q 次区间仿射 $x\mapsto ax+b$ 与区间和，就不能建立 3.3.003 区间加、乘、赋值（仿射懒标记） 的 $O(n)$ 棵完整树。`SparseAffSeg<T>` 把未创建子树解释成长度已知、元素全为 0 的区间；只有修改边界经过的节点才开点。每个节点复用 `AffineInfo<T>` 与 `AffineTag<T>`：节点摘要已含本层标记，部分查询不下传、不创建节点，而是把祖先尚未落实且后执行的标记 `outer` 带入子树。因此空子树仍能正确贡献 $len\cdot outer.add$，不会遗漏先前整段修改。

这里的空间性质和 3.3.007 动态开点线段树 不同：一次区间修改最多访问并新建 $O(\log U)$ 个规范分段及其边界孩子，$U=hi-lo$，故总节点数最坏为 $O(q\log U)$，而不是按出现位置计数。`T` 除 $0,1,+,\times$ 外还必须支持相等比较与由区间长度构造；模题取 `Z`，长度先转为 `Z` 再参与计算。它只表达全零初值的稀疏值域，若初始值不是零且没有可压缩表示，应离散化或用普通线段树。

```
// 依赖 3.3.003 区间加、乘、赋值（仿射懒标记）
template <class T> class SparseAffSeg
{
    struct Node
    {
        // l、r 是孩子编号，info 是该完整区间的和与长度，tag 是尚未下传的仿射变换。
        int l = 0, r = 0;
        AffineInfo<T> info;
        AffineTag<T> tag;
    };

    // lo、hi 是总半开值域，rt 是根编号，t 是动态节点池。
    long long lo, hi;
    int rt = 0;
    vector<Node> t{Node{}};

    int newNode(long long l, long long r)
    {
        // l、r 是新节点覆盖的非空半开区间；建立全零区间及恒等仿射标记。
```

```
        Node q;
        q.info = AffineInfo<T>(T{}, r - l);
        t.push_back(move(q));
        assert(t.size() <= (size_t)numeric_limits<int>::max()); // 调试检查，可删
        return (int)t.size() - 1;
    }

    bool idle(int p) const
    {
        // p 是非空节点；返回其延迟标记是否为恒等变换。
        return t[p].tag.mul == T(1) && t[p].tag.add == T{};
    }

    void apply(int p, long long, long long, const AffineTag<T> &tag)
    {
        // p、[l,r) 是完整命中节点，tag 是后执行的仿射变换；原地更新摘要和延迟标记。
        t[p].info.apply(tag);
        t[p].tag.apply(tag);
    }

    void push(int p, long long l, long long r)
    {
        // p、[l,r) 是内部节点；若有标记则按持久化无关的普通方式下传给两个孩子。
        if (r - l == 1 || idle(p))
        {
            return;
        }
        long long m = l + (r - l) / 2;
        AffineTag<T> tag = t[p].tag;
        int a = t[p].l;
        int b = t[p].r;
        if (!a)
        {
            a = newNode(l, m);
        }
        if (!b)
        {
            b = newNode(m, r);
        }
        apply(a, l, m, tag);
        apply(b, m, r, tag);
        t[p].l = a;
        t[p].r = b;
        t[p].tag = AffineTag<T>{};
    }

    void pull(int p, long long l, long long r)
    {
        // p 是已下传标记的内部节点，[l,r) 是其区间；由孩子的和重算摘要。
        T a = t[p].l ? t[t[p].l].info.sum : T{};
        T b = t[p].r ? t[t[p].r].info.sum : T{};
        t[p].info = AffineInfo<T>(a + b, r - l);
```

```
    }

    int affine(int p, long long l, long long r, long long ql, long long qr,
                   const AffineTag<T> &tag)
    {
        // p、[l,r) 是当前节点，[ql,qr) 是修改区间，tag 是后执行变换；返回更新后的根。
        if (qr <= l || r <= ql)
        {
            return p;
        }
        if (!p)
        {
            p = newNode(l, r);
        }
        if (ql <= l && r <= qr)
        {
            apply(p, l, r, tag);
            return p;
        }
        push(p, l, r);
        long long m = l + (r - l) / 2;
        int a = affine(t[p].l, l, m, ql, qr, tag);
        int b = affine(t[p].r, m, r, ql, qr, tag);
        t[p].l = a;
        t[p].r = b;
        pull(p, l, r);
        return p;
    }

    T sum(int p, long long l, long long r, long long ql, long long qr,
          const AffineTag<T> &outer) const
    {
        // p、[l,r) 是当前节点，[ql,qr) 是查询区间，outer 是祖先尚未下传且后执行的变换。
        if (qr <= l || r <= ql)
        {
            return T{};
        }
        long long a = max(l, ql), b = min(r, qr);
        if (!p)
        {
            // 空子树原值全为零，仍要接受祖先已经施加的常数项。
            return T(b - a) * outer.add;
        }
        if (ql <= l && r <= qr)
        {
            return t[p].info.sum * outer.mul + T(r - l) * outer.add;
        }
        AffineTag<T> next = t[p].tag;
        next.apply(outer);
        long long m = l + (r - l) / 2;
        return sum(t[p].l, l, m, ql, qr, next)
             + sum(t[p].r, m, r, ql, qr, next);
```

```
    }

  public:
    SparseAffSeg(long long lo, long long hi) : lo(lo), hi(hi)
    {
        // lo、hi 是非空半开整数值域；初始所有位置为零。
        assert(lo < hi); // 调试检查，可删
    }

    void reserveNodes(int cap)
    {
        // cap 是预计节点数；预留节点池容量，不改变结构语义。
        assert(cap >= 0); // 调试检查，可删
        t.reserve(cap + 1);
    }

    void affine(long long l, long long r, const T &mul, const T &add)
    {
        // l、r 是值域内半开端点，mul、add 表示 x->x*mul+add；原地修改 [l,r)。
        assert(lo <= l && l <= r && r <= hi); // 调试检查，可删
        if (l == r)
        {
            return;
        }
        rt = affine(rt, lo, hi, l, r, AffineTag<T>{mul, add});
    }

    T sum(long long l, long long r) const
    {
        // l、r 是值域内半开端点；返回 [l,r) 的元素和，空区间返回零。
        assert(lo <= l && l <= r && r <= hi); // 调试检查，可删
        return sum(rt, lo, hi, l, r, AffineTag<T>{});
    }
};
```

构造 `SparseAffSeg<T> tr(lo,hi)` 后，`tr.affine(l,r,mul,add)` 对 $[l,r)$ 执行 $x \mapsto x \cdot mul + add$，`tr.sum(l,r)` 返回区间和；半开端点必须位于总值域内，空修改和空查询均允许。单次修改、查询均为 $O(\log U)$，总空间 $O(q \log U)$，其中 q 是修改次数；`reserveNodes(cap)` 只可按可承受内存预留实际节点数。Library Checker `range_affine_range_sum_large_array` 取 `SparseAffSeg<Z> tr(0,N)`，初始零数组与模运算都正好符合接口。固定长度、已有完整初值时仍优先用 3.3.003 区间加、乘、赋值（仿射懒标记），因为它的空间为 $O(n)$、常数更小；历史版本要求改用 3.3.020 可持久化仿射区间和。

3.3.022 区间赋同函数、区间复合

`range_set_range_composite` 的元素本身是一元函数，操作是把一段每个位置改成同一个函数 f，查询却要求按位置顺序算 $f_{r-1} \circ \cdots \circ f_l$。这不是 3.3.003 区间加、乘、赋值（仿射懒标记） 的“把数值作用仿射变换”，所以不能混用 `AffineTag`；节点必须保存函数序列的有序积，整段赋值 f 后的摘要为 f^{r-l}。`RangeSetSeg<T,F>` 把“求幂”写成只依赖结合律和幺元的二进制重复，故同样可用于矩阵、置换等非交换幺半群。

`op(left,right)` 一律表示先执行左段、再执行右段。对仿射函数 $f(x) = ax + b$，用 `array<Z,2>` 存 `{a,b}`，并定义 $op(f, g) = g \circ f$；恒等元是 `{1,0}`。一次全覆盖赋值只覆盖旧标记而不是复合，因为题意是替换每个元素；`push` 才在需要访问儿子时把这个统一值下传。

```
template <class T, class F> class RangeSetSeg
{
    struct Node
    {
        // prod 是该区间按从左到右顺序合并的积，tag 是等待下传的统一赋值幂表编号，-1 表示无标记。
        T prod{};
        int tag = -1;
    };

    // n 是实际序列长度，base 是补齐后的二次幂叶子数，height 是树高。
    int n = 0, base = 1, height = 0;
    T e;
    F op;
    vector<Node> tr;
    // powers 的第 id 块保存同一次赋值 x 的 x^(2^0), ... ,x^(2^height)。
    vector<T> powers;

    const T &power(int id, int level) const
    {
        // id 是赋值幂表编号，level 对应长度 2^level；返回 x^(2^level)。
        return powers[(size_t)id * (height + 1) + level];
    }

    int makeTag(const T &x)
    {
        // x 是一次新的统一赋值；建立按二次幂长度索引的重复积并返回编号。
        assert(powers.size() / (height + 1) < (size_t)numeric_limits<int>::max()); // 调试检查，
        ↪ 可删
        int id = (int)(powers.size() / (height + 1));
        T cur = x;
        for (int level = 0; level <= height; level++)
        {
            powers.push_back(cur);
            if (level != height)
            {
                cur = op(cur, cur);
            }
        }
        return id;
    }

    void pull(int p)
    {
        // p 是内部节点；按原下标顺序合并两个孩子的积。
        tr[p].prod = op(tr[2 * p].prod, tr[2 * p + 1].prod);
    }

    void apply(int p, int level, int tag)
```

```
    {
        // p 是长度 2^level 的完整命中节点，tag 是统一赋值幂表编号；覆盖该段并记录标记。
        tr[p].prod = power(tag, level);
        tr[p].tag = tag;
    }

    void push(int p, int level)
    {
        // p 是长度 2^level 的当前节点；把待执行统一赋值按幂表编号下传给两个孩子。
        if (level == 0 || tr[p].tag < 0)
        {
            return;
        }
        apply(2 * p, level - 1, tr[p].tag);
        apply(2 * p + 1, level - 1, tr[p].tag);
        tr[p].tag = -1;
    }

    void build(const vector<T> &a)
    {
        // a 是完整初始序列；补齐幺元叶子后自底向上建立有序区间积。
        for (int i = 0; i < n; i++)
        {
            tr[base + i].prod = a[i];
        }
        for (int i = n; i < base; i++)
        {
            tr[base + i].prod = e;
        }
        for (int p = base - 1; p; p--)
        {
            pull(p);
        }
    }

    void assign(int p, int l, int r, int ql, int qr, int tag, int level)
    {
        // p、[l,r) 是长度 2^level 的当前节点，[ql,qr) 是赋值区间，tag 是统一新值幂表编号。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
            apply(p, level, tag);
            return;
        }
        push(p, level);
        int m = (l + r) / 2;
        assign(2 * p, l, m, ql, qr, tag, level - 1);
        assign(2 * p + 1, m, r, ql, qr, tag, level - 1);
        pull(p);
```

```
    }

    T prod(int p, int l, int r, int ql, int qr, int level)
    {
        // p、[l,r) 是当前节点；返回 [ql,qr) 与该节点交集按原顺序的积。
        if (qr <= l || r <= ql)
        {
            return e;
        }
        if (ql <= l && r <= qr)
        {
            return tr[p].prod;
        }
        push(p, level);
        int m = (l + r) / 2;
        return op(prod(2 * p, l, m, ql, qr, level - 1),
                  prod(2 * p + 1, m, r, ql, qr, level - 1));
    }

  public:
    RangeSetSeg(const vector<T> &a, T e, F op) : n(a.size()), e(move(e)), op(move(op))
    {
        // a 是初始序列，e 是运算幺元，op 是从左段到右段的结合运算；允许空序列。
        while (base < max<int>(n, 1))
        {
            base *= 2;
            height++;
        }
        tr.assign(2 * base, Node{});
        build(a);
    }

    void reserveTags(int cap)
    {
        // cap 是预计的非空赋值总次数；预留幂缓存容量，不改变结构语义。
        assert(cap >= 0); // 调试检查，可删
        powers.reserve((size_t)cap * (height + 1));
    }

    void assign(int l, int r, const T &x)
    {
        // l、r 是半开端点，x 是新值；把 [l,r) 每个元素统一赋为 x，空区间无操作。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return;
        }
        assign(1, 0, base, l, r, makeTag(x), height);
    }

    T prod(int l, int r)
    {
```

```
        // l、r 是半开端点；返回 [l,r) 的有序积，空区间返回 e。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return e;
        }
        return prod(1, 0, base, l, r, height);
    }
};
```

构造 `RangeSetSeg<T,F> st(a,e,op)`，其中 `a` 是初始序列；可先以预计的非空赋值数调用 `reserveTags(cap)` 预留幂缓存。调用时例如用加法合并：

```
signed main()
{
    auto op = [](int x, int y)
    {
        return x + y;
    };
    RangeSetSeg<int, decltype(op)> st(vector<int>{1, 2, 3, 4}, 0, op);
    st.assign(1, 3, 5);
    assert(st.prod(0, 4) == 15);
}
```

`assign(l,r,x)` 为此次赋值建立一次 $x, x^2, x^4, \ldots$ 的表，节点只保存表编号并按自身二次幂长度 $O(1)$ 取积；`prod(l,r)` 返回有序积并允许空区间。建树 $O(n)$，一次赋值和查询都是 $O(\log n)$；幂缓存额外占 $O(u \log n)$ 个 `T`，其中 u 是发生过的非空赋值次数，故总空间为 $O(n + u \log n)$。对模仿射函数这已覆盖 Library Checker 的 `range_set_range_composite`；其 $N, Q \le 5 \times 10^5$ 时应调用 `reserveTags(Q)`，避免幂表扩容。若只做单点赋函数、区间复合，普通 3.3.001 单点修改线段树与摘要二分 或大值域的 3.3.007 动态开点线段树 更短。若更新语义是把函数施加到数值数组而非替换函数元素，回到 3.3.003 区间加、乘、赋值（仿射懒标记）。

3.4 RMQ 与静态区间结构

3.4.001 ST 表

ST 表用倍增预处理静态数组，回答幂等运算（min、max、gcd、按位与或）的区间查询。`a[k][i]` 存以 i 开头、长度 2^k 的区间结果，递推 `a[k][i]=f(a[k-1][i], a[k-1][i+2^{k-1}])`。查询 $[l, r)$ 时取 $k = \lfloor \log_2(r-l) \rfloor$，用覆盖左右端的两个长度 2^k 的块 `f(a[k][l], a[k][r-2^k])`；因为运算幂等，两块重叠部分被重复计入也不影响结果，这正是它区别于下面 DST 的地方——只有幂等才能这样重叠取块：

```
template <class T, class F> struct ST
{
    // 最小值与最大值只由 f 决定：分别传 min 或 max；ST 的建表和查询代码无需修改。
    // n 是数组长度，f 是幂等合并运算，a[k][i] 保存长度 2^k 的区间信息。
    int n;
    F f;
    vector<vector<T>> a;
```

```
    ST(const vector<T> &v, F f = {}) : n((int)v.size()), f(f)
    {
        // v 是静态数组，f 是满足结合律和幂等性的合并运算。
        int m = n ? bit_width((unsigned)n) : 0;
        a.assign(m, vector<T>(n));
        if (!n)
        {
            return;
        }
        a[0] = v;
        for (int k = 1; k < m; k++)
        {
            for (int i = 0; i + (1 << k) <= n; i++)
            {
                a[k][i] = f(a[k - 1][i], a[k - 1][i + (1 << (k - 1))]);
            }
        }
    }

    // 仅适用于幂等运算（min/max/gcd/and/or），查询 [l,r)。
    T query(int l, int r) const
    {
        // [l,r) 是非空查询区间；返回其合并结果。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        int k = bit_width((unsigned)(r - l)) - 1;
        return f(a[k][l], a[k][r - (1 << k)]);
    }
};
```

若普通 ST 的 $O(n \log n)$ 内存或构建常数过大，可按固定块长 B 保存块内前后缀，并只给整块建 ST：

```
template <class T, class F, int B = 16> struct BlockST
{
    // 与 ST 相同，最小/最大只切换 f；不要改重叠块的查询结构。
    int n = 0, m = 0;
    F f;
    vector<T> val, pre, suf;
    vector<vector<T>> st;

    BlockST(const vector<T> &v, F f = {}) : f(f) { build(v); }

    void build(const vector<T> &v)
    {
        // v 是静态数组；按块长 B 建块内前后缀与块间稀疏表，末块用最后一个元素补齐。
        n = v.size();
        m = (n + B - 1) / B;
        if (!n)
        {
            val.clear(); pre.clear(); suf.clear(); st.clear();
            return;
        }
```

```
        int z = m * B;
        val.assign(z, v.back());
        copy(v.begin(), v.end(), val.begin());
        pre.resize(z);
        suf.resize(z);
        vector<T> top(m);
        for (int i = 0; i < z; i++)
        {
            pre[i] = i % B ? f(pre[i - 1], val[i]) : val[i];
        }
        for (int l = 0; l < z; l += B)
        {
            suf[l + B - 1] = val[l + B - 1];
            for (int i = l + B - 1; i-- > l;)
            {
                suf[i] = f(val[i], suf[i + 1]);
            }
            top[l / B] = suf[l];
        }
        int h = bit_width((unsigned long long)m);
        st.assign(h, vector<T>(m));
        st[0] = move(top);
        for (int k = 1; k < h; k++)
        {
            for (int i = 0; i + (1 << k) <= m; i++)
            {
                st[k][i] = f(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
            }
        }
    }

    T query(int l, int r) const
    {
        // [l,r) 是非空半开区间；返回区间内全部元素按 f 合并的结果。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        r--;
        int x = l / B, y = r / B;
        if (x == y)
        {
            T ans = val[l];
            for (int i = l + 1; i <= r; i++)
            {
                ans = f(ans, val[i]);
            }
            return ans;
        }
        T ans = f(suf[l], pre[r]);
        if (++x < y)
        {
            int k = bit_width((unsigned long long)(y - x)) - 1;
            ans = f(ans, f(st[k][x], st[k][y - (1 << k)]));
        }
```

```
        return ans;
    }
};
```

构造 `ST<T,F> st(v, f)`，`f` 是幂等的二元合并。最小值实例为：

```
signed main()
{
    vector<int> v{5, 2, 7, 3};
    auto f = [](int x, int y)
    {
        return min(x, y);
    };
    ST<int, decltype(f)> st(v, f);
    assert(st.query(1, 4) == 2);
    BlockST<int, decltype(f)> bst(v, f);
    assert(bst.query(1, 4) == 2);
}
```

`st.query(l, r)` 返回半开区间 $[l,r)$ 的合并结果，要求 $0 \le l < r \le n$。普通版预处理 $O(n \log n)$ 时间与空间，单次查询 $O(1)$。`BlockST<T,F,B>` 的预处理和空间为 $O(n + (n/B)\log(n/B))$，跨块查询 $O(1)$，同块查询至多合并 B 项；固定 $B = 16$ 时仍为最坏 $O(1)$。本机 $n = q = 10^6$ 的随机 RMQ 中，分块版构建约 `6ms`、查询约 `10ms`，普通版约为 `31ms+10ms`，同时大幅减少常驻内存。运算只有结合律但不幂等（矩阵乘、字符串拼接）时不能重叠取块，改用 3.4.003 静态区间半群；要第 k 小、排名、前驱后继用 3.4.005 小波矩阵；按小块笛卡尔树形状压缩查询表见 3.4.002 基于状压的 RMQ。

3.4.002 基于状压的 RMQ

把静态数组切成长度约 $b = \lfloor \frac{\log_2 n}{2} \rfloor$ 的小块；两个块若笛卡尔树形状相同，任意块内区间的最优局部下标也相同，因此可按形状共享一张 `l*len+r` 查询表，块间最值另建 ST 表。`shape` 用单调栈建块内笛卡尔树并输出括号序列编码，查询 $[l,r)$ 时合并左块后缀、右块前缀与中间整块。当前实现只为实际出现的形状现场建立 len^2 表，并用 `map` 管理形状；设形状数为 S，表规模是 $O(Sb^2)$，由 $S \le \min(n/b, C_b)$ 可得最坏 $O(n\sqrt{\log n})$，不是 Fischer-Heun 的严格线性构造。`pick` 在值相等时保留较小下标，保证并列最优取最靠前：

```
template <class T, class Cmp = less<T>> struct BlockRMQ
{
    // 默认 less<T> 求最小值；改用 greater<T> 即求最大值，其他建表、并列取较早下标逻辑不变。
    using Key = pair<unsigned long long, int>;

    // a 保存静态序列；cmp 判断前者是否更优；b 是小块长度；type 保存各块的笛卡尔树形状与实际长度。
    vector<T> a;
    Cmp cmp;
    int b = 1;
    vector<Key> type;
    // tab 为每种块形状保存所有块内询问的最优局部下标；mn 保存每块最优全局下标；st 是块间稀疏表。
    map<Key, vector<int>> tab;
    vector<int> mn;
```

```
    vector<vector<int>> st;

    BlockRMQ(const vector<T> &a = {}, Cmp cmp = {}) : a(a), cmp(cmp)
    {
        // a 是静态序列，cmp 是严格弱序比较器；建立块内表与块间 ST 表。
        build();
    }

    int pick(int x, int y) const
    {
        // x、y 表示两个候选下标；返回值更优者，空下标 -1 不参与比较。
        if (x < 0)
        {
            return y;
        }
        if (y < 0)
        {
            return x;
        }
        if (cmp(a[y], a[x]))
        {
            return y;
        }
        if (cmp(a[x], a[y]))
        {
            return x;
        }
        return min(x, y);
    }

    Key shape(int l, int r) const
    {
        // [l,r) 表示一个小块；返回其笛卡尔树括号编码与实际长度。
        int n = r - l;
        vector<int> lc(n, -1), rc(n, -1), stk;
        for (int i = 0; i < n; i++)
        {
            int last = -1;
            while (!stk.empty() && cmp(a[l + i], a[l + stk.back()]))
            {
                last = stk.back();
                stk.pop_back();
            }
            if (!stk.empty())
            {
                rc[stk.back()] = i;
            }
            if (last >= 0)
            {
                lc[i] = last;
            }
            stk.push_back(i);
```

```
        }
        int root = stk.front();
        unsigned long long code = 0;
        auto dfs = [&](auto &&self, int u) -> void
        {
            if (u < 0)
            {
                code <<= 1;
                return;
            }
            code = code << 1 | 1;
            self(self, lc[u]);
            self(self, rc[u]);
        };
        dfs(dfs, root);
        return {code, n};
    }

    void build()
    {
        // 重建块形状表、块最值和块间 ST 表。
        b = 1;
        type.clear();
        tab.clear();
        mn.clear();
        st.clear();
        int n = a.size();
        if (!n)
        {
            return;
        }
        b = max<int>(1, ((int)bit_width((unsigned long long)n) - 1) / 2);
        int m = (n + b - 1) / b;
        type.resize(m);
        mn.resize(m);
        for (int k = 0; k < m; k++)
        {
            int l = k * b, r = min(n, l + b), len = r - l;
            type[k] = shape(l, r);
            if (!tab.contains(type[k]))
            {
                vector<int> f(len * len);
                for (int i = 0; i < len; i++)
                {
                    int best = i;
                    for (int j = i; j < len; j++)
                    {
                        if (cmp(a[l + j], a[l + best]))
                        {
                            best = j;
                        }
                        f[i * len + j] = best;
```

```
                    }
                }
                tab[type[k]] = move(f);
            }
            mn[k] = inBlock(k, 0, len);
        }
        st.push_back(mn);
        for (int d = 1; (1LL << d) <= m; d++)
        {
            int len = m - (1LL << d) + 1;
            st.push_back(vector<int>(len));
            for (int i = 0; i < len; i++)
            {
                st[d][i] = pick(st[d - 1][i], st[d - 1][i + (1LL << (d - 1))]);
            }
        }
    }

    int inBlock(int k, int l, int r) const
    {
        // k 表示块编号，[l,r) 表示块内非空区间；返回对应全局最优下标。
        int len = type[k].second;
        return k * b + tab.at(type[k])[l * len + r - 1];
    }

    int index(int l, int r) const
    {
        // [l,r) 表示非空查询区间；返回其中最优元素的全局下标。
        assert(0 <= l && l < r && r <= (int)a.size()); // 调试检查，可删
        int x = l / b, y = (r - 1) / b;
        if (x == y)
        {
            return inBlock(x, l % b, (r - 1) % b + 1);
        }
        int ans = pick(inBlock(x, l % b, type[x].second), inBlock(y, 0, (r - 1) % b + 1));
        int ql = x + 1, qr = y;
        if (ql < qr)
        {
            int d = (int)bit_width((unsigned)(qr - ql)) - 1;
            ans = pick(ans, pick(st[d][ql], st[d][qr - (1LL << d)]));
        }
        return ans;
    }

    const T &query(int l, int r) const
    {
        // [l,r) 表示非空查询区间；返回其中最优元素的常引用。
        return a[index(l, r)];
    }
};
```

构造 `BlockRMQ<T,Cmp> q(a, cmp)`，`q.index(l, r)` 返回半开区间 $[l, r)$ 内最优元素的全局下标、

`q.query(l, r)` 返回该元素常引用，区间必须非空，`cmp` 须为严格弱序；修改公开成员 `a` 后可再次调用 `build()` 完整重建。例如 `a={5,2,2,7,-1,3,0}` 时 `index(0,7)==4`、`query(1,4)==2`、`index(1,3)==1`（并列最小取较早下标）。当前实现最坏预处理时间与空间均为 $O(n\sqrt{\log n})$，单次查询 $O(1)$；一般幂等 RMQ 用 3.4.001 ST 表 更简单且界更直接，确需严格线性预处理时应换完整 Fischer-Heun 构造。非幂等结合运算用 3.4.003 静态区间半群。下面用固定数组检查并列最小值和重复建表：

```
signed main()
{
    BlockRMQ<int> q({5, 2, 2, 7, -1, 3, 0});
    assert(q.index(0, 7) == 4);
    assert(q.query(1, 4) == 2);
    assert(q.index(1, 3) == 1);
    q.a = {9, 8, 7, 6, 5};
    q.build();
    assert(q.index(0, 5) == 4);
    cout << "OK\n";
}
```

3.4.003 静态区间半群

Disjoint Sparse Table（DST）回答静态数组上任意满足结合律的区间运算，不要求幂等或可逆，典型如矩阵乘、字符串拼接。它的第 k 层把数组按 2^{k+1} 分段，每段以中点为界向左记后缀积、向右记前缀积存进 `t[k]`。查询 $[l,r)$ 时（$r-1$ 记末位），取 l 与 $r-1$ 最高不同二进制位确定唯一的层 $k=\lfloor\log_2(l\oplus(r-1))\rfloor$，在这一层 l 与 $r-1$ 恰好分居某段中点两侧，于是答案就是“从 l 到中点的后缀积”合并“从中点到 $r-1$ 的前缀积”，即 `f(t[k][l], t[k][r-1])`——两段不重叠、恰好拼满，所以只需结合律：

```
// 静态半群：运算只需满足结合律，不要求可逆或幂等。
template <class T, class F> struct DST
{
    // n 是数组长度，f 是结合律运算，a 是原数组，t 保存各层左右半段积。
    int n;
    F f;
    vector<T> a;
    vector<vector<T>> t;

    DST(const vector<T> &a, F f = {}) : n((int)a.size()), f(f), a(a)
    {
        // a 是静态数组，f 是满足结合律的合并运算。
        int lg = n <= 1 ? 0 : bit_width((unsigned)(n - 1));
        t.assign(lg, vector<T>(n));
        for (int k = 0; k < lg; k++)
        {
            int len = 1 << k;
            for (int l = 0; l < n; l += 2 * len)
            {
                int m = min(l + len, n), r = min(l + 2 * len, n);
                if (m == l || m == r)
                {
                    continue;
```

```
                }
                t[k][m - 1] = a[m - 1];
                for (int i = m - 2; i >= l; i--)
                {
                    t[k][i] = f(a[i], t[k][i + 1]);
                }
                t[k][m] = a[m];
                for (int i = m + 1; i < r; i++)
                {
                    t[k][i] = f(t[k][i - 1], a[i]);
                }
            }
        }
    }

    T query(int l, int r) const
    {
        // [l,r) 是非空查询区间；返回按原顺序合并的结果。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        if (r - l == 1)
        {
            return a[l];
        }
        int k = bit_width((unsigned)(l ^ (r - 1))) - 1;
        return f(t[k][l], t[k][r - 1]);
    }
};
```

构造 `DST<T,F> d(a, f)`，例如维护字符串拼接：

```
signed main()
{
    vector<string> a{"a", "b", "c", "d"};
    auto f = [](const string &x, const string &y)
    {
        return x + y;
    };
    DST<string, decltype(f)> d(a, f);
    assert(d.query(1, 4) == "bcd");
}
```

`d.query(l, r)` 返回半开区间 $[l, r)$ 上按 `f` 折叠的结果（长度 1 时直接返回 `a[l]`），要求 $0 \le l < r \le n$。预处理 $O(n \log n)$ 时间与空间，单次查询 $O(1)$。它比 ST 表适用面更广（不要求幂等），代价是常数略大、无法像 ST 表那样省一层；若运算幂等就用更简单的 3.4.001 ST 表，状压块内查询见 3.4.002 基于状压的 RMQ。

3.4.004 笛卡尔树

笛卡尔树对下标 $0..n-1$ 建一棵二叉树，使中序遍历仍是原下标顺序、而结点值满足堆序（这里建最小堆，即父值不大于子值）。它把” 区间最小值位置” 转化为” 两下标的 LCA”，也是很多分治计数、单

调栈问题的结构骨架。构造用单调栈维护当前根到最右结点的这条” 右链”：新元素 i 沿右链弹出所有比它大的结点（弹出的最后一个成为 i 的左儿子），再把 i 接到栈顶结点的右儿子，$O(n)$ 完成。相等元素靠前者在上，保证形状唯一：

```
struct Cart
{
    // root 是树根；parent、left、right 分别保存父亲与左右儿子编号。
    int root = -1;
    vector<int> parent, left, right;
};

// 最小笛卡尔树；中序遍历为原下标，相等时靠前元素在上方。
template <class T> Cart cartesian(const vector<T> &a)
{
    // a 是静态数组；返回其最小笛卡尔树的根、父亲和左右儿子。
    int n = a.size();
    Cart c{-1, vector<int>(n, -1), vector<int>(n, -1), vector<int>(n, -1)};
    vector<int> s;
    for (int i = 0; i < n; i++)
    {
        int last = -1;
        while (!s.empty() && a[i] < a[s.back()])
        {
            last = s.back();
            s.pop_back();
        }
        if (!s.empty())
        {
            c.parent[i] = s.back();
            c.right[s.back()] = i;
        }
        if (last != -1)
        {
            c.parent[last] = i;
            c.left[i] = last;
        }
        s.push_back(i);
    }
    if (n)
    {
        c.root = s[0];
    }
    return c;
}
```

调用 `Cart c = cartesian(a)`,返回的 `Cart` 结构里 `c.root` 是根下标,`c.parent[i]`、`c.left[i]`、`c.right[i]` 分别是结点 i 的父、左儿子、右儿子（无则为 -1）。$O(n)$ 时间与空间。它给出的是 “结构” 而非直接的区间查询接口：配一棵 LCA 即得 $O(1)$ RMQ（下标形式），本身也常用于 3.4.002 基于状压的 RMQ 里对小块求形状；若只想直接查区间最值用 3.4.001 ST 表 更省事。

3.4.005 小波矩阵

小波矩阵在静态序列上按二进制位从高到低逐层「稳定划分」：每层把当前序列里该位为 0 的元素依原相对顺序放到前半、为 1 的放到后半，并记录每个前缀里 1 的个数 `prefixOne` 以及前缀里 0 元素的值和 `pre0`。有了这些，任一区间 $[l, r)$ 在下一层都能 $O(1)$ 映射到对应子区间：为 0 的元素落在 `[l-onesLeft, r-onesR)`，为 1 的落在 `[middle+onesLeft, middle+onesR)`。`kth` 从高位到低位，看当前区间里 0 的个数决定第 k 小走哪半；`countSumLT` 沿上界 `hi` 的各位走，遇到该位为 1 就把左半（更小）整段计入个数与值和。频次由两次 `countLess` 相减得到。注意 `rank` 是 0 基的，返回的是原值：

```cpp
template <int LOG = 30> struct Wavelet
{
    static_assert(1 <= LOG && LOG <= 62);

    // size 是序列长度，middle[bit] 是该层零段长度。
    int size;
    array<int, LOG> middle;
    // prefixOne[bit][i] 是该层前 i 位的一数；pre0 是零分支原值和。
    array<vector<int>, LOG> prefixOne;
    array<vector<int>, LOG> pre0;
    // prefixSum[i] 是原序列前 i 项的值和。
    vector<int> prefixSum;
    explicit Wavelet(vector<int> values = {})
    {
        // values 是位于 [0,2^LOG) 的初始序列。
        init(move(values));
    }

    void init(vector<int> values)
    {
        // values 是新的静态序列；清空旧层并重新稳定划分。
        size = values.size();
        prefixSum.assign(size + 1, 0);
        constexpr unsigned long long limit = 1ULL << LOG;
        for (int i = 0; i < size; i++)
        {
            assert(0 <= values[i] && (unsigned long long)values[i] < limit); // 调试检查，可删
            prefixSum[i + 1] = prefixSum[i] + values[i];
        }
        vector<int> next(size);
        for (int bit = LOG - 1; bit >= 0; --bit)
        {
            prefixOne[bit].assign(size + 1, 0);
            pre0[bit].assign(size + 1, 0);
            for (int i = 0; i < size; ++i)
            {
                bool one = values[i] >> bit & 1;
                prefixOne[bit][i + 1] = prefixOne[bit][i] + one;
                pre0[bit][i + 1] = pre0[bit][i] + (one ? 0 : values[i]);
            }
            middle[bit] = size - prefixOne[bit][size];
            int zero = 0, one = middle[bit];
```

```
            for (int value : values)
            {
                if (value >> bit & 1)
                {
                    next[one++] = value;
                }
                else
                {
                    next[zero++] = value;
                }
            }
            values.swap(next);
        }
    }

    int kth(int left, int right, int rank) const
    {
        // left、right 是半开区间端点，rank 从 0 开始；返回第 rank 小的原值。
        assert(0 <= left && left < right && right <= size); // 调试检查，可删
        assert(0 <= rank && rank < right - left); // 调试检查，可删
        int answer = 0;
        for (int bit = LOG - 1; bit >= 0; --bit)
        {
            int onesLeft = prefixOne[bit][left];
            int onesR = prefixOne[bit][right];
            int zeros = right - left - onesR + onesLeft;
            if (rank < zeros)
            {
                left -= onesLeft;
                right -= onesR;
            }
            else
            {
                answer |= 1LL << bit;
                rank -= zeros;
                left = middle[bit] + onesLeft;
                right = middle[bit] + onesR;
            }
        }
        return answer;
    }

    pair<int, int> countSumLT(int left, int right, int hi) const
    {
        // 返回 [left,right) 内严格小于 hi 的元素个数与值和。
        assert(0 <= left && left <= right && right <= size); // 调试检查，可删
        if (hi <= 0)
        {
            return {0, 0};
        }
        constexpr unsigned long long limit = 1ULL << LOG;
        if ((unsigned long long)hi >= limit)
```

```
        {
            return {right - left, prefixSum[right] - prefixSum[left]};
        }
        int count = 0;
        int sum = 0;
        for (int bit = LOG - 1; bit >= 0; --bit)
        {
            int onesLeft = prefixOne[bit][left];
            int onesR = prefixOne[bit][right];
            int zeros = right - left - onesR + onesLeft;
            if (hi >> bit & 1)
            {
                count += zeros;
                sum += pre0[bit][right] - pre0[bit][left];
                left = middle[bit] + onesLeft;
                right = middle[bit] + onesR;
            }
            else
            {
                left -= onesLeft;
                right -= onesR;
            }
        }
        return {count, sum};
    }

    int countLess(int left, int right, int hi) const
    {
        // 返回 [left,right) 内严格小于 hi 的元素个数。
        return countSumLT(left, right, hi).first;
    }

    int frequency(int left, int right, int value) const
    {
        // 返回 [left,right) 内 value 的出现次数，值域外返回 0。
        assert(0 <= left && left <= right && right <= size); // 调试检查，可删
        constexpr unsigned long long limit = 1ULL << LOG;
        if (value < 0 || (unsigned long long)value >= limit)
        {
            return 0;
        }
        return countLess(left, right, value + 1) - countLess(left, right, value);
    }

    pair<int, int> countSumLE(int left, int right, int hi) const
    {
        // 返回 [left,right) 内不大于 hi 的元素个数与值和。
        assert(0 <= left && left <= right && right <= size); // 调试检查，可删
        if (hi < 0)
        {
            return {0, 0};
        }
```

```
        constexpr unsigned long long limit = 1ULL << LOG;
        if ((unsigned long long)hi >= limit - 1)
        {
            return {right - left, prefixSum[right] - prefixSum[left]};
        }
        return countSumLT(left, right, hi + 1);
    }
};
```

构造 `Wavelet<LOG> w(values)`（$1 \le LOG \le 62$，`values` 位于 $[0, 2^{LOG})$），`w.kth(l,r,rank)` 返回 $[l, r)$ 内第 `rank` 小的值（`rank` 从 0 起）、`w.frequency(l,r,v)` 返回 v 的出现次数、`w.countLess(l,r,x)` 返回严格小于 x 的个数、`w.countSumLT/LE(l,r,x)` 返回小于/不大于 x 的个数与值和。调用者必须保证任意前缀值和可由 64 位有符号整数表示。计数接口允许空区间，`kth` 要求非空且排名有效。构造 $O(n \cdot LOG)$，各查询 $O(LOG)$。它是静态区间顺序统计的首选，把这些接口打包成的对外用法见 3.4.006 区间第 K 小、排名、前驱后继；带修改则要树套树或离线算法。

若题目只求第 k 小，不需要频次、排名与值和，可使用下面的 `KthWavelet`。它只保存每层的 `prefixOne` 和零段长度，省去 `Wavelet` 的零分支和值前缀和，常驻空间约减半；索引和计数遵循打印版的普通 `int` 规则。其取值范围、半开区间和排名约定与 `Wavelet::kth` 相同：

```
template <int LOG = 30> struct KthWavelet
{
    static_assert(1 <= LOG && LOG <= 62);

    // size 是序列长度，middle[bit] 是该层零段长度；只保存第 k 小查询所需的紧凑状态。
    int size = 0;
    array<int, LOG> middle;
    // prefixOne[bit][i] 是该层前 i 位的一数。
    array<vector<int>, LOG> prefixOne;

    explicit KthWavelet(vector<int> values = {})
    {
        // values 是位于 [0,2^LOG) 的初始序列。
        init(move(values));
    }

    void init(vector<int> values)
    {
        // values 是新的静态序列；清空旧层并重新稳定划分。
        assert(values.size() <= numeric_limits<int>::max()); // 调试检查，可删
        size = (int)values.size();
        constexpr unsigned long long limit = 1ULL << LOG;
        for (int value : values)
        {
            assert(0 <= value && (unsigned long long)value < limit); // 调试检查，可删
        }
        vector<int> next(size);
        for (int bit = LOG - 1; bit >= 0; --bit)
        {
            prefixOne[bit].assign(size + 1, 0);
            for (int i = 0; i < size; ++i)
```

```
            {
                prefixOne[bit][i + 1] = prefixOne[bit][i] + (values[i] >> bit & 1);
            }
            middle[bit] = size - prefixOne[bit][size];
            int zero = 0, one = middle[bit];
            for (int value : values)
            {
                if (value >> bit & 1)
                {
                    next[one++] = value;
                }
                else
                {
                    next[zero++] = value;
                }
            }
            values.swap(next);
        }
    }

    int kth(int left, int right, int rank) const
    {
        // left、right 是半开区间端点，rank 从 0 开始；返回第 rank 小的原值。
        assert(0 <= left && left < right && right <= size); // 调试检查，可删
        assert(0 <= rank && rank < right - left); // 调试检查，可删
        int answer = 0;
        for (int bit = LOG - 1; bit >= 0; --bit)
        {
            int onesLeft = prefixOne[bit][left];
            int onesR = prefixOne[bit][right];
            int zeros = right - left - onesR + onesLeft;
            if (rank < zeros)
            {
                left -= onesLeft;
                right -= onesR;
            }
            else
            {
                answer |= 1LL << bit;
                rank -= zeros;
                left = middle[bit] + onesLeft;
                right = middle[bit] + onesR;
            }
        }
        return answer;
    }
};
```

`KthWavelet<LOG> w(values)` 构造为 $O(n \cdot LOG)$，`w.kth(l,r,rank)` 为 $O(LOG)$，额外空间为 $O(n \cdot LOG)$，但没有查询频次、排名和值和的接口；需要它们时仍用完整的 `Wavelet`。下面用值域两端的固定数据核对全部边界：

```
signed main()
{
    vector<int> a{0, 1, 255, 511, 511};
    Wavelet<9> w(a);
    assert(w.kth(0, 5, 0) == 0);
    assert(w.kth(0, 5, 4) == 511);
    assert(w.countLess(0, 5, 0) == 0);
    assert(w.countLess(0, 5, 512) == 5);
    assert(w.frequency(0, 5, 511) == 2);
    auto [cnt, sum] = w.countSumLE(0, 5, 511);
    assert(cnt == 5 && sum == 1278);
    assert(w.countLess(2, 2, 512) == 0);
    cout << "0 511 5 2 1278\n";
    return 0;
}
```

3.4.006 区间第 K 小、排名、前驱后继

这是「区间第 K 小、排名、前驱后继」这一类需求的对外入口，底层就是 3.4.005 小波矩阵 的 `Wavelet`，不再重复粘贴代码。第 k 小直接 `kth(l, r, k)`（0 基）；某值 v 在 $[l,r)$ 内的排名即 `countLess(l, r, v)`；前驱（严格小于 v 的最大值）在个数 `c=countLess(l,r,v)` 大于 0 时为 `kth(l, r, c-1)`，后继（严格大于 v 的最小值）在 `c2=countSumLE(l,r,v).first < r-l` 时为 `kth(l, r, c2)`。全部为 $O(LOG)$。静态区间顺序统计就用它；若元素需要插入删除，用 3.5.002 FHQ Treap 维护，或对下标序列套线段树；单点带修区间第 k 小则要树状数组套主席树等结构。

3.4.007 离线静态区间频次

若全部询问 (l,r,x) 会预先给出，只求 x 在 $[l,r)$ 的出现次数，无需建立小波矩阵。把询问拆成 “前 r 项计数” 减 “前 l 项计数” 两个事件，按前缀长度从 0 扫到 n，哈希表维护各值已经出现的次数；每个数组元素与事件都只处理一次。下面依赖 1.4.004 防 hack 哈希 的 `CustomHash`：

```
vector<int> offlineFreq(const vector<int> &a, const vector<array<int, 3>> &qs)
{
    // a 是静态数组；qs 每项为 {l,r,x}；返回各询问中 x 在 [l,r) 的出现次数。
    int n = a.size();
    using E = array<int, 3>; // 三维依次是询问编号、查询值、容斥符号。
    vector<vector<E>> ev(n + 1);
    for (int i = 0; i < (int)qs.size(); i++)
    {
        auto [l, r, x] = qs[i];
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        ev[l].push_back({i, x, -1});
        ev[r].push_back({i, x, 1});
    }

    unordered_map<int, int, CustomHash> cnt;
    cnt.reserve(2 * (a.size() + qs.size()) + 1);
```

```
    vector<int> ans(qs.size());
    for (int p = 0; p <= n; p++)
    {
        if (p)
        {
            cnt[a[p - 1]]++;
        }
        for (auto [i, x, s] : ev[p])
        {
            auto it = cnt.find(x);
            int c = it == cnt.end() ? 0 : it->second;
            ans[i] += s * c;
        }
    }
    return ans;
}
```

`offlineFreq(a,qs)` 保持询问顺序返回答案，允许空区间和从未在数组出现的查询值。例如取 `a={1,2,1,3,1}`，查询分别为 `{0,5,1}`、`{1,4,1}`、`{2,2,1}`、`{0,5,4}`，答案是 $3,1,0,0$。在 `CustomHash` 的通常假设下，时间期望 $O(n+q)$、空间 $O(n+q)$；若要求确定性或还要第 k 小、值域计数与值和，改用 3.4.005 小波矩阵。

3.4.008 静态区间众数（秩优化）

把数组按块长 $B \approx \sqrt{n}$ 分块，预处理任意连续整块区间的众数及其次数。查询 $[l,r)$ 时先取中间整块答案，再只枚举两端至多 $2B$ 个位置。为去掉 “每个边界候选在出现位置表上二分” 的额外对数，预先保存 `rk[i]`，即下标 i 在同值出现序列 `pos[a[i]]` 中的秩；已知候选在询问内最左或最右的一次出现后，可直接检查相距当前最优次数的那一次出现是否仍在区间内。只有真正超过当前答案时才继续沿出现序列扩展，而答案总共至多增加 $O(B)$ 次。

```
class RangeMode
{
    int n, B, m; // n 是序列长度，B 是块长，m 是块数。
    // a 是离散值，val 是原值，rk[i] 是 i 在 pos[a[i]] 中的秩。
    vector<int> a, val, rk;
    vector<vector<int>> pos;
    // mode[l][r]、mc[l][r] 是第 l 到第 r 块合并后的众数及次数。
    vector<vector<int>> mode, mc;
    vector<unsigned long long> seen;
    unsigned long long stamp = 0;

    void nextStamp()
    {
        // 开始一次新查询；极端回绕时清空访问标记。
        stamp++;
        if (!stamp)
        {
            fill(seen.begin(), seen.end(), 0);
            stamp = 1;
```

```
        }
    }

public:
    explicit RangeMode(vector<int> x) : n(x.size())
    {
        // x 是静态数组；完成离散化、出现秩与整块众数预处理。
        val = x;
        sort(val.begin(), val.end());
        val.erase(unique(val.begin(), val.end()), val.end());
        a.resize(n);
        rk.resize(n);
        pos.resize(val.size());
        for (int i = 0; i < n; i++)
        {
            a[i] = lower_bound(val.begin(), val.end(), x[i]) - val.begin();
            rk[i] = pos[a[i]].size();
            pos[a[i]].push_back(i);
        }

        B = max<int>(1, sqrt((long double)max<int>(1, n)));
        m = (n + B - 1) / B;
        mode.assign(m, vector<int>(m, -1));
        mc.assign(m, vector<int>(m));
        for (int l = 0; l < m; l++)
        {
            vector<int> f(val.size());
            int best = -1, c = 0;
            for (int i = l * B; i < n; i++)
            {
                int v = a[i];
                f[v]++;
                if (f[v] > c)
                {
                    best = v;
                    c = f[v];
                }
                if ((i + 1) % B == 0 || i + 1 == n)
                {
                    mode[l][i / B] = best;
                    mc[l][i / B] = c;
                }
            }
        }
        seen.assign(val.size(), 0);
    }

    optional<pair<int, int>> query(int l, int r)
    {
        // l、r 是半开区间端点；返回 {一个众数，出现次数}，空区间返回空。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
```

```
        {
            return nullopt;
        }
        nextStamp();
        auto use = [&](int v)
        {
            if (seen[v] == stamp)
            {
                return false;
            }
            seen[v] = stamp;
            return true;
        };

        int L = (l + B - 1) / B;
        int R = r / B;
        int best = -1, c = 0;
        if (L < R)
        {
            best = mode[L][R - 1];
            c = mc[L][R - 1];
            for (int i = l; i < L * B; i++)
            {
                c += a[i] == best;
            }
            for (int i = R * B; i < r; i++)
            {
                c += a[i] == best;
            }
            use(best);
        }

        int le = min(r, L * B);
        for (int i = l; i < le; i++)
        {
            int v = a[i];
            if (!use(v))
            {
                continue;
            }
            int first = rk[i], e = first + c;
            if (e < (int)pos[v].size() && pos[v][e] < r)
            {
                e++;
                while (e < (int)pos[v].size() && pos[v][e] < r)
                {
                    e++;
                }
                best = v;
                c = e - first;
            }
        }
```

```
124
125         int rs = max(l, R * B);
126         for (int i = r - 1; i >= rs; i--)
127         {
128             int v = a[i];
129             if (!use(v))
130             {
131                 continue;
132             }
133             int last = rk[i], e = last - c;
134             if (e >= 0 && pos[v][e] >= l)
135             {
136                 e--;
137                 while (e >= 0 && pos[v][e] >= l)
138                 {
139                     e--;
140                 }
141                 best = v;
142                 c = last - e;
143             }
144         }
145         return pair{val[best], c};
146     }
147 };
```

`RangeMode rm(a)` 后，`query(l,r)` 返回任意一个众数及其出现次数；并列众数不保证取最小值，空区间返回 `nullopt`。例如 `a={1,2,1,3,2,1,2,2}` 时，`query(0,8)` 与 `query(1,8)` 均返回 $2,4$，`query(0,5)` 可返回 $1,2$。预处理 $O(n\sqrt{n})$ 时间、$O(n+(n/B)^2)$ 空间，单次查询 $O(B)$；只查某个指定值的频次且询问离线时用 3.4.007 离线静态区间频次，要顺序统计则用 3.4.005 小波矩阵。

下面用同一组数核对跨块区间、并列众数和空区间：

```
1  signed main()
2  {
3      vector<int> a{1, 2, 1, 3, 2, 1, 2, 2};
4      RangeMode rm(a);
5      assert(rm.query(0, 8) == optional(pair{2LL, 4LL}));
6      assert(rm.query(1, 8) == optional(pair{2LL, 4LL}));
7      auto q = rm.query(0, 5);
8      assert(q && q->second == 2 && (q->first == 1 || q->first == 2));
9      assert(!rm.query(3, 3));
10 }
```

3.4.009 静态区间 LIS 查询

当静态数组本身是值域 $[0,n)$ 的排列、而且要回答很多个区间的严格 LIS 时，逐次调用普通 patience sorting 的 $O((r-l)\log(r-l))$ 不能承受。把排列按值域递归二分，可以把每半边的半局部 LCS/LIS 临界路径压成一个允许空位的部分排列（seaweed 编码）。两个相邻值域半边的编码对应 subunit-Monge 矩阵乘法：补齐空位后得到两个单位 Monge 排列，分治地相乘，再丢回补齐路径。递归结束后得到 `edge`，把空位替换为 n。

其关键不变量是对任意 $0 \le l \le r \le n$:

$$\mathrm{LIS}(p[l,r)) = (r-l) - \#\{i \in [l,n) \mid edge_i < r\}.$$

右边仅是一段静态数组的“小于阈值计数”，故复用 3.4.005 小波矩阵 的 `Wavelet<LOG>::countLess`。这里 `LOG` 要使 $n < 2^{LOG}$；默认 `17` 正好覆盖 $n \le 10^5$，内部临界路径和工作区索引使用普通 `int`。它只接受排列，重复值的非严格/严格 LIS 都不能直接套用。

```cpp
// 依赖 3.4.005 小波矩阵
template <int LOG = 17> class StaticRngLIS
{
    static constexpr int none = -1; // none 表示部分排列中未定义的位置。

    // n 是原排列长度；wt 保存海草边界编码后的静态顺序统计结构。
    int n = 0;
    Wavelet<LOG> wt;

    static vector<int> inverse(const vector<int> &p)
    {
        // p 是允许 none 的部分排列；返回其反向部分排列，none 保持为 none。
        int n = p.size();
        vector<int> q(n, none);
        for (int i = 0; i < n; i++)
        {
            if (p[i] == none)
            {
                continue;
            }
            assert(0 <= p[i] && p[i] < n && q[p[i]] == none); // 调试检查，可删
            q[p[i]] = (int)i;
        }
        return q;
    }

    static int workSize(int n)
    {
        // n 是单位 Monge 排列阶数；返回一次递归乘法所需的 int 工作区长度。
        int ans = 1;
        while (n > 1)
        {
            ans += 2 * n;
            n = (n + 1) / 2;
            ans += 4 * n;
        }
        return ans;
    }

    static void unitMongeMul(int n, int *work, const int *a, const int *b)
    {
        // n 是排列阶数，work 是足够大的工作区，a、b 是两个单位 Monge 排列；把乘积行映射写入 work[0,n)。
        if (n == 1)
```

```
        {
            work[0] = 0;
            return;
        }
        int *rowToCol = work;
        work += n;
        int *colToRow = work;
        work += n;
        int mid = n / 2;

        auto solveHalf = [&](int len, auto inHalf, auto shift)
        {
            // len 是当前值域半边长度；inHalf 判定元素是否属于该半边；shift 把值压到半边内。
            int *aHead = work;
            int *aWhere = work + len;
            int *bHead = work + 2 * len;
            int *bWhere = work + 3 * len;
            auto split = [&](const int *src, int *head, int *where)
            {
                // src 是完整排列；head 写压缩后的子排列，where 写对应原下标。
                for (int i = 0; i < n; i++)
                {
                    if (inHalf(src[i]))
                    {
                        *head++ = (int)shift(src[i]);
                        *where++ = (int)i;
                    }
                }
            };
            split(a, aHead, aWhere);
            split(b, bHead, bWhere);
            unitMongeMul(len, work + 4 * len, aHead, bHead);
            int *part = work + 4 * len;
            for (int i = 0; i < len; i++)
            {
                int row = aWhere[i];
                int col = bWhere[part[i]];
                rowToCol[row] = (int)col;
                colToRow[col] = (int)row;
            }
        };
        solveHalf(mid, [=](int x)
        {
            return x < mid;
        }, [](int x)
        {
            return x;
        });
        solveHalf(n - mid, [=](int x)
        {
            return x >= mid;
        }, [=](int x)
```

```
        {
            return x - mid;
        });

        struct Cursor
        {
            // delta 是当前 Monge 边界相对平衡，col 是当前列位置。
            int delta = 0, col = 0;
        };
        int row = n;
        auto goRight = [&](Cursor &it)
        {
            // it 是当前边界游标；向右越过一列并维护其平衡量。
            if (b[it.col] < mid)
            {
                if (colToRow[it.col] >= row)
                {
                    it.delta++;
                }
            }
            else if (colToRow[it.col] < row)
            {
                it.delta++;
            }
            it.col++;
        };
        auto goUp = [&](Cursor &it)
        {
            // it 是当前边界游标；把行边界上移一格并维护其平衡量。
            if (a[row] < mid)
            {
                if (rowToCol[row] >= it.col)
                {
                    it.delta--;
                }
            }
            else if (rowToCol[row] < it.col)
            {
                it.delta--;
            }
        };
        Cursor neg, pos;
        while (row)
        {
            while (pos.col != n)
            {
                Cursor next = pos;
                goRight(next);
                if (next.delta != 0)
                {
                    break;
                }
```

```
                pos = next;
            }
            row--;
            goUp(neg);
            goUp(pos);
            while (neg.delta != 0)
            {
                goRight(neg);
            }
            if (neg.col > pos.col)
            {
                rowToCol[row] = (int)pos.col;
            }
        }
    }

    static vector<int> partMongeMul(vector<int> a, vector<int> b)
    {
        // a、b 是同阶部分 Monge 排列；返回它们复合后的部分排列。
        int n = a.size();
        vector<int> aInv = inverse(a), bInv = inverse(b);
        swap(b, bInv);
        vector<int> aMap, bMap;
        for (int i = n - 1; i >= 0; i--)
        {
            if (a[i] != none)
            {
                aMap.push_back((int)i);
                a[n - (int)aMap.size()] = a[i];
            }
        }
        reverse(aMap.begin(), aMap.end());
        int ptr = 0;
        for (int i = 0; i < n; i++)
        {
            if (aInv[i] == none)
            {
                a[ptr++] = (int)i;
            }
        }
        for (int i = 0; i < n; i++)
        {
            if (b[i] != none)
            {
                b[(int)bMap.size()] = b[i];
                bMap.push_back((int)i);
            }
        }
        ptr = bMap.size();
        for (int i = 0; i < n; i++)
        {
            if (bInv[i] == none)
```

```
            {
                b[ptr++] = (int)i;
            }
        }

        vector<int> work(workSize(n));
        unitMongeMul(n, work.data(), a.data(), b.data());
        vector<int> c(n, none);
        int start = n - (int)aMap.size();
        for (int i = 0; i < (int)aMap.size(); i++)
        {
            int to = work[start + i];
            if (to < (int)bMap.size())
            {
                c[aMap[i]] = bMap[to];
            }
        }
        return c;
    }

    static vector<int> seaweed(const vector<int> &p)
    {
        // p 是值域 [0,n) 的排列；返回其半局部 LIS 的 seaweed 部分排列编码。
        int n = p.size();
        if (n <= 1)
        {
            return vector<int>(n, none);
        }
        int mid = n / 2;
        vector<int> low, high, lowMap, highMap;
        low.reserve(mid);
        high.reserve(n - mid);
        lowMap.reserve(mid);
        highMap.reserve(n - mid);
        for (int i = 0; i < n; i++)
        {
            assert(0 <= p[i] && p[i] < n); // 调试检查，可删
            if (p[i] < mid)
            {
                low.push_back(p[i]);
                lowMap.push_back((int)i);
            }
            else
            {
                high.push_back((int)(p[i] - mid));
                highMap.push_back((int)i);
            }
        }
        low = seaweed(low);
        high = seaweed(high);
        vector<int> lowPad(n), highPad(n);
        iota(lowPad.begin(), lowPad.end(), (int)0);
```

```
        iota(highPad.begin(), highPad.end(), (int)0);
        for (int i = 0; i < mid; i++)
        {
            if (low[i] == none)
            {
                lowPad[lowMap[i]] = none;
            }
            else
            {
                lowPad[lowMap[i]] = lowMap[low[i]];
            }
        }
        for (int i = 0; mid + i < n; i++)
        {
            if (high[i] == none)
            {
                highPad[highMap[i]] = none;
            }
            else
            {
                highPad[highMap[i]] = highMap[high[i]];
            }
        }
        return partMongeMul(move(lowPad), move(highPad));
    }

  public:
    explicit StaticRngLIS(vector<int> p = {})
    {
        // p 是值域 [0,n) 的静态排列；构造其区间严格 LIS 查询结构。
        init(move(p));
    }

    void init(vector<int> p)
    {
        // p 是值域 [0,n) 的新静态排列；重建结构，不保留旧查询状态。
        n = p.size();
        assert(n <= numeric_limits<int>::max()); // 调试检查，可删
        assert((unsigned long long)n < (1ULL << LOG)); // 调试检查，可删
        vector<char> used(n);
        vector<int> q(n);
        for (int i = 0; i < n; i++)
        {
            assert(0 <= p[i] && p[i] < n && !used[p[i]]); // 调试检查，可删
            used[p[i]] = true;
            q[i] = (int)p[i];
        }
        vector<int> edge = seaweed(q);
        vector<int> code(n);
        for (int i = 0; i < n; i++)
        {
            code[i] = edge[i] == none ? n : edge[i];
```

```
        }
        wt.init(move(code));
    }

    int query(int l, int r) const
    {
        // l、r 是半开区间端点；返回排列 p[l,r) 的严格最长递增子序列长度。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return r - l - wt.countLess(l, n, r);
    }
};
```

构造 `StaticRngLIS<LOG> st(p)` 或重建 `st.init(p)`；`p` 必须是 $[0,n)$ 的排列且 $n < 2^{LOG}$。`st.query(l,r)` 允许空半开区间并返回其严格 LIS 长度。预处理为 $O(n\log^2 n)$ 时间、$O(n\log n)$ 额外空间，单次查询 $O(\log n)$；这是为大量查询准备的专用重型结构。只做一个区间或数组允许重复时用 4.5.003 区间 LCS / 区间 LIS，普通静态排名/频次则直接用 3.4.005 小波矩阵。下面固定样例同时核对空区间：

```
signed main()
{
    StaticRngLIS<5> st({0, 2, 1, 3});
    assert(st.query(0, 2) == 2);
    assert(st.query(0, 4) == 3);
    assert(st.query(1, 3) == 1);
    assert(st.query(2, 2) == 0);
}
```

3.4.010 静态区间不同值个数

若数组和全部区间都已给出，只要每个半开区间 $[l,r)$ 内不同值的数量，直接复用 2.3.005 区间不同数统计 的 `distinct(a,qs)`。这是一条面向静态区间结构的配置入口，不再复制其中的「按右端点扫、Fenwick 仅保留每个值最后位置」实现：`a` 是静态数组，`qs` 按原顺序保存半开区间，返回的 `vector<int>` 与 `qs` 一一对应，空数组、空询问集和空区间均可用。原接口已经逐项满足这里的语义，不能把只统计指定值频次的 3.4.007 离线静态区间频次 或普通莫队替代为同一个问题。

调用仍是 `auto ans = distinct(a, qs);`。`distinct` 内部用防卡哈希记录每个值的最后位置，故时间在 `CustomHash` 的通常假设下为 $O((n+q)\log n)$、空间为 $O(n+q)$，其中 $n=|a|$、$q=|qs|$；需确定性值域索引或还要顺序统计时，先离散化，或改用 3.4.005 小波矩阵。查询间若夹杂修改，最后出现位置会整体变化，本接口不适用，应另行选择带修改的离线算法。

3.4.011 静态区间逆序对查询

2.2.003 逆序对及其扩展 的 `invMerge` 和 `invBit` 只能统计整个数组；对很多预先给出的区间，要离线重排询问。下面复用 2.2.002 离散化与坐标压缩 的 `Discrete`，并在函数内用普通 `int` 频次数组实现 Fenwick：本题热点只存排名、端点和频次，公共 `Fenwick` 的一般 64 位和类型不适合这条固定容量路径。查询按普通莫队的块长和蛇形右端排序。`len` 是当前长度，`ans` 是当前严格逆序对数。向左加入排名 `x` 时，新元素在所有旧元素之前，新增的是旧元素中严格小于 `x` 的数量 `sum(x)`；向右加入时，新元素在所有旧元素之

后，新增的是严格大于 x 的数量 `len-sum(x+1)`。删除时先从 Fenwick 去掉本元素，再减去对应贡献，因此相等元素在四种移动中都不计入答案。

```cpp
// 依赖 2.2.002 离散化与坐标压缩
vector<int> rangeInv(const vector<int> &a, const vector<pair<int, int>> &qs)
{
    // a 是静态数组，qs 是半开区间；返回每个区间内满足 i<j 且 a[i]>a[j] 的数量。
    int n = (int)a.size();
    if (qs.empty())
    {
        return {};
    }

    // Query[0..2] 依次表示左端点、右端点、原询问编号；普通 int 足以覆盖题目下标范围。
    using Query = array<int, 3>;
    vector<Query> ord(qs.size());
    for (int id = 0; id < (int)qs.size(); id++)
    {
        auto [l, r] = qs[id];
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        ord[id] = {(int)l, (int)r, (int)id};
    }
    Discrete<int> dis(a);
    vector<int> rank(n);
    for (int i = 0; i < n; i++)
    {
        rank[i] = dis.id(a[i]);
    }
    vector<int> bit(dis.a.size() + 1);
    auto add = [&](int x, int delta)
    {
        // x 是离散排名，delta 是频次变化；修改紧凑的 Fenwick 计数树。
        for (x++; x < (int)bit.size(); x += x & -x)
        {
            bit[x] += delta;
        }
    };
    auto sum = [&](int x)
    {
        // x 是排名上界；返回所有排名小于 x 的当前元素个数。
        int ans = 0;
        for (; x; x -= x & -x)
        {
            ans += bit[x];
        }
        return ans;
    };

    int block = max<int>(1, (int)(n / sqrtl(max<int>(1, (int)ord.size()))));
    sort(ord.begin(), ord.end(), [&](const Query &x, const Query &y)
         {
             int bx = x[0] / block, by = y[0] / block;
```

```
            if (bx != by)
            {
                return bx < by;
            }
            return bx & 1 ? x[1] > y[1] : x[1] < y[1];
        });

    vector<int> out(qs.size());
    int l = 0, r = 0, len = 0, ans = 0;
    for (auto q : ord)
    {
        while (r < q[1])
        {
            int x = rank[r++];
            ans += len - sum(x + 1);
            add(x, 1);
            len++;
        }
        while (r > q[1])
        {
            int x = rank[--r];
            add(x, -1);
            len--;
            ans -= len - sum(x + 1);
        }
        while (l > q[0])
        {
            int x = rank[--l];
            ans += sum(x);
            add(x, 1);
            len++;
        }
        while (l < q[0])
        {
            int x = rank[l++];
            add(x, -1);
            len--;
            ans -= sum(x);
        }
        out[q[2]] = ans;
    }
    return out;
}
```

`rangeInv(a,qs)` 返回顺序与 `qs` 相同，区间均为 $[l,r)$，空区间答案为 0；本模板的 `int` 为 64 位，能保存题目规模内至多 $n(n-1)/2$ 个逆序对。令 V 为不同值数，离散化花 $O(n\log V)$，莫队端点移动次数为 $O(n\sqrt{q})$，每次移动做 $O(\log V)$ 的 Fenwick 操作，连同排序总时间为 $O(n\log V+(n\sqrt{q}+q\log q)\log V)$，额外空间 $O(n+q+V)$。紧凑节点要求 $n,q,V\le 2^{31}-1$，这覆盖常见题目限制。只问整个数组时 2.2.003 的归并版更短且为 $O(n\log n)$；区间中有修改时必须改用带时间轴的 2.3.002 带修改莫队，不能继续使用本静态接口。

```cpp
signed main()
{
    vector<int> a{3, 1, 2, 1};
    vector<pair<int, int>> qs{{0, 4}, {1, 4}, {2, 2}};
    assert((rangeInv(a, qs) == vector<int>{4, 1, 0}));
    cout << "4 1 0\n";
}
```

3.4.012 静态区间和

静态数组只做区间加法和时，线段树或 Fenwick 都多了一层不需要的修改能力，直接复用 2.4.001 一维/二维前缀和 的一维 `prefix` 即可。令 `pre=prefix(a)`，其中 `pre[0]=0`、`pre[i+1]=pre[i]+a[i]`，则半开区间的答案是 `pre[r]-pre[l]`；这与题目输入的左右端点完全一致。

```cpp
auto pre = prefix(a);
auto ans = pre[r] - pre[l]; // [l,r) 的元素和。
```

`a` 可以含负数，`0\le l\le r\le |a|`，空区间自然返回零。模板参数 `T` 必须容纳全部前缀和及其差；例如元素与总和都在 64 位范围内时使用 `vector<int>`，需更宽累计时显式用 `vector<__int128>`。预处理 $O(n)$、每次查询 $O(1)$、额外空间 $O(n)$。有单点修改时改用 3.2.001 单点修改、前缀和，有区间修改时改用 3.2.004 区间修改、区间求和。

3.4.013 Fischer-Heun 线性预处理 RMQ

静态区间最小值，$O(n)$ 预处理、$O(1)$ 查询。数组的笛卡尔树（小根堆序，中序是原下标）上，区间 $[l, r)$ 的最小值就是结点 l 与 $r-1$ 的 LCA；对笛卡尔树做欧拉序，LCA 又变成欧拉序上两次首次出现位置之间深度最小的位置。欧拉序长 $m = 2n-1$，相邻深度恰差 ± 1。

把欧拉序切成长 $b = \max(1, \lfloor \frac{\lfloor \log_2 m \rfloor + 1}{2} \rfloor)$ 的块：块内相对深度由 $b-1$ 个 ± 1 决定，至多 $2^{b-1} \le \sqrt{2m}$ 种模式，每种模式暴力建 b^2 的块内答案表，总计 $O(\sqrt{m}\log^2 m)$；块最小值另建稀疏表，只有 $O(\frac{m}{b}\log m) = O(m)$ 项。查询拆成左块后缀、右块前缀和中间整块三段。

```cpp
struct FischerHeun
{
    // a 是原序列；first[i] 是 i 在欧拉序中第一次出现的位置。
    vector<int> a, first;
    // euler、dep 是笛卡尔树欧拉序上的结点编号与深度，相邻深度恰差 1。
    vector<int> euler, dep;
    // b 是块长；pat[k] 是第 k 块深度差模式对应的块内表编号。
    int b = 1;
    vector<int> pat;
    // tab[id][l*b+r] 是该模式块内 [l,r] 的最小深度位置；st 是块最小值的稀疏表。
    vector<vector<int>> tab, st;

    FischerHeun(const vector<int> &v) : a(v), first(v.size(), -1)
    {
        // v 是静态序列；建立 O(n) 预处理、O(1) 查询的区间最小值结构。
```

```cpp
        int n = a.size();
        if (!n)
        {
            return;
        }
        // 单调栈建笛卡尔树：lc、rc 是左右儿子，栈底是根。
        // 改区间最大值：只把下面的 a[i] < a[stk.back()] 改成 a[i] > a[stk.back()]，其余深度比较不变。
        vector<int> lc(n, -1), rc(n, -1), stk;
        for (int i = 0; i < n; i++)
        {
            int last = -1;
            while (!stk.empty() && a[i] < a[stk.back()])
            {
                last = stk.back();
                stk.pop_back();
            }
            if (!stk.empty())
            {
                rc[stk.back()] = i;
            }
            lc[i] = last;
            stk.push_back(i);
        }
        auto emit = [&](int v, int d)
        {
            // v 是结点，d 是深度；把 v 追加进欧拉序。
            if (first[v] < 0)
            {
                first[v] = euler.size();
            }
            euler.push_back(v);
            dep.push_back(d);
        };
        auto dfs = [&](auto &&self, int v, int d) -> void
        {
            // v 是当前结点，d 是其深度；进入以及每个儿子返回后各记录一次 v。
            emit(v, d);
            if (lc[v] >= 0)
            {
                self(self, lc[v], d + 1);
                emit(v, d);
            }
            if (rc[v] >= 0)
            {
                self(self, rc[v], d + 1);
                emit(v, d);
            }
        };
        dfs(dfs, stk[0], 0);
        int m = dep.size();
        b = max(1LL, (int)bit_width((unsigned long long)m) / 2);
        int cnt = (m + b - 1) / b;
```

```
        pat.resize(cnt);
        vector<int> mins(cnt);
        map<int, int> id; // id[code] 是该深度差模式在 tab 中的编号。
        for (int k = 0; k < cnt; k++)
        {
            int l = k * b, r = min(m, l + b), code = 0;
            mins[k] = l;
            for (int i = l + 1; i < r; i++)
            {
                if (dep[i] > dep[i - 1])
                {
                    code |= 1LL << (i - l - 1);
                }
                if (dep[i] < dep[mins[k]])
                {
                    mins[k] = i;
                }
            }
            if (!id.count(code))
            {
                // 按模式还原相对深度 d，再暴力填满块内所有 [i,j] 的最小位置。
                vector<int> d(b), t(b * b);
                for (int i = 1; i < b; i++)
                {
                    d[i] = d[i - 1] + ((code >> (i - 1) & 1) ? 1 : -1);
                }
                for (int i = 0; i < b; i++)
                {
                    int best = i;
                    for (int j = i; j < b; j++)
                    {
                        if (d[j] < d[best])
                        {
                            best = j;
                        }
                        t[i * b + j] = best;
                    }
                }
                id[code] = tab.size();
                tab.push_back(move(t));
            }
            pat[k] = id[code];
        }
        st.push_back(move(mins));
        for (int k = 1; (1LL << k) <= cnt; k++)
        {
            st.emplace_back(cnt - (1LL << k) + 1);
            for (int i = 0; i < (int)st[k].size(); i++)
            {
                st[k][i] = pick(st[k - 1][i], st[k - 1][i + (1LL << (k - 1))]);
            }
        }
```

```
    }

    int pick(int x, int y) const
    {
        // x、y 是欧拉序位置；返回深度较小者，相等时取 x。
        return dep[x] <= dep[y] ? x : y;
    }

    int inside(int k, int l, int r) const
    {
        // k 是块号，l、r 是块内闭区间；返回其中深度最小的欧拉序位置。
        return k * b + tab[pat[k]][l * b + r];
    }

    int argmin(int l, int r) const
    {
        // [l,r) 是非空半开区间；返回其中一个最小值的下标。
        assert(0 <= l && l < r && r <= (int)a.size()); // 调试检查，可删。
        int x = first[l], y = first[r - 1];
        if (x > y)
        {
            swap(x, y);
        }
        int u = x / b, v = y / b;
        if (u == v)
        {
            return euler[inside(u, x % b, y % b)];
        }
        int ans = pick(inside(u, x % b, b - 1), inside(v, 0, y % b));
        if (v > u + 1)
        {
            int k = bit_width((unsigned long long)(v - u - 1)) - 1;
            ans = pick(ans, pick(st[k][u + 1], st[k][v - (1LL << k)]));
        }
        return euler[ans];
    }

    int query(int l, int r) const
    {
        // [l,r) 是非空半开区间；返回区间最小值。
        return a[argmin(l, r)];
    }
};
```

构造 `FischerHeun rmq(a)`；`rmq.argmin(l,r)` 返回半开区间 $[l,r)$ 中一个最小值的下标，`rmq.query(l,r)` 返回最小值，区间必须非空。模式编号用 `map` 去重，查找次数是块数 $\frac{m}{b}$、每次 $O(\log m)$，整体仍是 $O(n)$ 预处理时间与空间，查询 $O(1)$。改求区间最大值只需改代码中注明的单调栈比较。

$O(n\log n)$ 内存可以接受时，3.4.001 ST 表 代码短得多；块内用位运算的近线性写法见 3.4.002 基于状压的 RMQ，笛卡尔树本身见 3.4.004 笛卡尔树。下面用固定种子生成 2×10^5 个数和同样多次查询，核对滚动哈希：

```cpp
signed main()
{
    mt19937 rng(20261015);
    vector<int> a(200000);
    for (auto &x : a)
    {
        x = rng() % 1000000000;
    }
    FischerHeun rmq(a);
    unsigned long long h = 0;
    for (int i = 0; i < 200000; i++)
    {
        int l = rng() % a.size(), r = rng() % a.size();
        if (l > r)
        {
            swap(l, r);
        }
        h = h * 1000003 + rmq.query(l, r + 1);
    }
    cout << rmq.query(3, 4) << ' ' << h << '\n';
    // 期望输出：98350465 3687840806893658951
}
```

3.5 平衡树与堆

3.5.001 Treap

这是旋转式 Treap：每个结点同时带一个键值（满足二叉搜索树的中序有序）和一个随机优先级（满足小根堆序）。插入后若儿子优先级更小就旋转上来修复堆序，删除则把待删结点转成叶子再摘掉。由于优先级随机，树高期望 $O(\log n)$。相同键用 `cnt` 计数而不新建结点，`sz` 维护含重复的子树大小，从而支持 `rank`（严格小于某键的个数）、`kth`（第 k 小，0 基）、`prev`/`next`（严格前驱后继）。这里用 xorshift 自带随机、结点池从下标 1 起（0 是空哨兵）：

```cpp
template <class T> struct Treap
{
    struct Node
    {
        T x{}; // x 是节点保存的键。
        // pri 是随机优先级；l、r 是儿子；cnt 是键重复数；sz 是含重复的子树大小。
        unsigned pri = 0;
        int l = 0, r = 0, cnt = 0, sz = 0;
    };

    vector<Node> t{{}};
    // root 是树根，seed 是优先级生成器状态。
    int root = 0;
    unsigned seed = 712367821;

    unsigned rnd()
```

```
    {
        // 推进内部伪随机状态并返回新值。
        seed ^= seed << 13;
        seed ^= seed >> 17;
        seed ^= seed << 5;
        return seed;
    }

    int size(int u) const
    {
        // u 是节点编号；返回含重复键的子树大小，空节点返回 0。
        return u ? t[u].sz : 0;
    }

    void pull(int u)
    {
        // u 是非空节点；由两个儿子和本键重数重算子树大小。
        t[u].sz = size(t[u].l) + size(t[u].r) + t[u].cnt;
    }

    int rot(int u, int d)
    {
        // u 是子树根；d=1 右旋、d=0 左旋，返回旋转后的根。
        int v = d ? t[u].l : t[u].r;
        if (d)
        {
            t[u].l = t[v].r;
            t[v].r = u;
        }
        else
        {
            t[u].r = t[v].l;
            t[v].l = u;
        }
        pull(u);
        pull(v);
        return v;
    }

    int add(int u, const T &x)
    {
        // u 是子树根，x 是待插入键；返回插入后的根。
        if (!u)
        {
            u = t.size();
            t.push_back({x, rnd(), 0, 0, 1, 1});
            return u;
        }
        if (t[u].x == x)
        {
            t[u].cnt++;
        }
```

```
        else if (x < t[u].x)
        {
            t[u].l = add(t[u].l, x);
            if (t[t[u].l].pri < t[u].pri)
            {
                u = rot(u, 1);
            }
        }
        else
        {
            t[u].r = add(t[u].r, x);
            if (t[t[u].r].pri < t[u].pri)
            {
                u = rot(u, 0);
            }
        }
        pull(u);
        return u;
    }

    void add(const T &x)
    {
        // x 是待插入键；插入一份并更新树根。
        root = add(root, x);
    }

    pair<int, bool> erase(int u, const T &x)
    {
        // u 是子树根，x 是待删键；返回删除后的根与删除前是否存在。
        if (!u)
        {
            return {0, false};
        }
        bool ok;
        if (x < t[u].x)
        {
            auto [v, z] = erase(t[u].l, x);
            t[u].l = v;
            ok = z;
        }
        else if (t[u].x < x)
        {
            auto [v, z] = erase(t[u].r, x);
            t[u].r = v;
            ok = z;
        }
        else
        {
            ok = true;
            if (t[u].cnt > 1)
            {
                t[u].cnt--;
```

```
            }
            else if (!t[u].l || !t[u].r)
            {
                u = t[u].l | t[u].r;
            }
            else
            {
                int d = t[t[u].l].pri < t[t[u].r].pri;
                u = rot(u, d);
                if (d)
                {
                    auto [v, z] = erase(t[u].r, x);
                    t[u].r = v;
                }
                else
                {
                    auto [v, z] = erase(t[u].l, x);
                    t[u].l = v;
                }
            }
        }
        if (u)
        {
            pull(u);
        }
        return {u, ok};
    }

    bool erase(const T &x)
    {
        // x 是待删键；删除一份并返回删除前是否存在。
        auto [u, ok] = erase(root, x);
        root = u;
        return ok;
    }

    int rank(const T &x) const
    {
        // x 是查询键；返回严格小于 x 的元素数。
        int u = root, ans = 0;
        while (u)
        {
            if (x <= t[u].x)
            {
                u = t[u].l;
            }
            else
            {
                ans += size(t[u].l) + t[u].cnt;
                u = t[u].r;
            }
        }
```

```
173         return ans;
174     }
175
176     T kth(int k) const
177     {
178         // k 是 0 基排名；返回第 k 小键。
179         assert(0 <= k && k < size(root)); // 调试检查，可删
180         int u = root;
181         while (true)
182         {
183             int z = size(t[u].l);
184             if (k < z)
185             {
186                 u = t[u].l;
187             }
188             else if (k < z + t[u].cnt)
189             {
190                 return t[u].x;
191             }
192             else
193             {
194                 k -= z + t[u].cnt;
195                 u = t[u].r;
196             }
197         }
198     }
199
200     optional<T> prev(const T &x) const
201     {
202         // x 是查询键；返回严格小于 x 的最大键，不存在时返回空。
203         int k = rank(x);
204         return k ? optional<T>(kth(k - 1)) : nullopt;
205     }
206
207     optional<T> next(const T &x) const
208     {
209         // x 是查询键；返回严格大于 x 的最小键，不存在时返回空。
210         int u = root, k = 0;
211         while (u)
212         {
213             if (x < t[u].x)
214             {
215                 u = t[u].l;
216             }
217             else
218             {
219                 k += size(t[u].l) + t[u].cnt;
220                 u = t[u].r;
221             }
222         }
223         return k < size(root) ? optional<T>(kth(k)) : nullopt;
224     }
```

```
};
```

用法：`Treap<int> tr;` 之后 `tr.add(x)` 插入、`tr.erase(x)` 删除一份、`tr.rank(x)` 返回严格小于 x 的个数、`tr.kth(k)` 返回第 k 小（0 基，需 $0 \le k <$ 元素数）、`tr.prev(x)`/`tr.next(x)` 返回严格前驱/后继的 `optional`（不存在为 `nullopt`）。单次操作期望 $O(\log n)$、确定性最坏 $O(n)$，空间与插入次数同阶。只需标准有序集合用 `set`；要序列翻转、区间懒标记用 3.5.002 FHQ Treap（分裂合并更自然）；只求第 k 小/排名且允许 GNU 扩展用 3.5.007 PBDS 平衡树。下面用固定序列覆盖重复值、删除、排名以及不存在的前驱后继：

```
signed main()
{
    Treap<int> tr;
    for (int x : {4, 1, 7, 1, 3})
    {
        tr.add(x);
    }
    assert(tr.rank(3) == 2);
    assert(tr.kth(0) == 1 && tr.kth(2) == 3 && tr.kth(4) == 7);
    assert(tr.erase(1) && !tr.erase(9));
    assert(tr.kth(0) == 1 && tr.kth(1) == 3);
    assert(!tr.prev(1) && tr.prev(4) == optional<int>(3));
    assert(tr.next(4) == optional<int>(7) && !tr.next(7));
    cout << "2 1 3 7\n";
    return 0;
}
```

3.5.002 FHQ Treap

FHQ（范浩强）Treap 完全不写旋转，只靠 `split` 与 `merge` 两个操作组合出一切。`split(p, x)` 把树按键拆成「小于 x」与「不小于 x」两棵，`splitLE(p, x)` 拆成「不大于 x」与「大于 x」；`merge(p, q)` 要求 `p` 的所有键不大于 `q` 的所有键，按随机优先级合并成一棵。有了这两把刀，插入就是「按键切两半、中间塞新结点、合三段」，删除一份是「切出等于 x 的那一个结点、丢掉它自身、把它的左右儿子并回去」，`rank` 是「切出小于 x 的子树看大小再合回」。这种无旋写法天然适合可持久化和序列区间操作（把键换成隐式下标即可维护可翻转序列）：

```
template <class T> struct FHQ
{
    struct Node
    {
        // x 保存键值；l、r 保存左右儿子；sz 保存子树大小；pri 保存随机优先级。
        T x{};
        int l = 0, r = 0, sz = 1;
        unsigned pri = 0;
    };

    // tr 保存节点池；root 保存根；seed 保存随机数状态。
    vector<Node> tr{{}};
    int root = 0; // root 是当前根，0 表示空树。
```

```
    unsigned seed = 712367821; // seed 是 xorshift 随机优先级状态。

    unsigned rnd()
    {
        // 推进内部伪随机状态并返回新的优先级。
        seed ^= seed << 13;
        seed ^= seed >> 17;
        seed ^= seed << 5;
        return seed;
    }

    int size(int p) const
    {
        // p 是节点编号；返回其子树大小，空节点返回 0。
        return p ? tr[p].sz : 0;
    }

    void pull(int p)
    {
        // p 是非空节点；根据两个儿子重算子树大小。
        tr[p].sz = size(tr[p].l) + size(tr[p].r) + 1;
    }

    pair<int, int> split(int p, const T &x)
    {
        // p 是树根，x 是分界键；返回键值小于 x 与不小于 x 的两棵树根。
        if (!p)
        {
            return {0, 0};
        }
        if (tr[p].x < x)
        {
            auto [a, b] = split(tr[p].r, x);
            tr[p].r = a;
            pull(p);
            return {p, b};
        }
        auto [a, b] = split(tr[p].l, x);
        tr[p].l = b;
        pull(p);
        return {a, p};
    }

    pair<int, int> splitLE(int p, const T &x)
    {
        // p 是树根，x 是分界键；返回键值不大于 x 与大于 x 的两棵树根。
        if (!p)
        {
            return {0, 0};
        }
        if (!(x < tr[p].x))
        {
```

```
            auto [a, b] = splitLE(tr[p].r, x);
            tr[p].r = a;
            pull(p);
            return {p, b};
        }
        auto [a, b] = splitLE(tr[p].l, x);
        tr[p].l = b;
        pull(p);
        return {a, p};
    }

    int merge(int p, int q)
    {
        // p、q 是两棵树根，且 p 中所有键不大于 q 中所有键；返回合并后的根。
        if (!p || !q)
        {
            return p | q;
        }
        if (tr[p].pri < tr[q].pri)
        {
            tr[p].r = merge(tr[p].r, q);
            pull(p);
            return p;
        }
        tr[q].l = merge(p, tr[q].l);
        pull(q);
        return q;
    }

    void insert(const T &x)
    {
        // x 是待插入键；允许重复值。
        auto [p, q] = split(root, x);
        int u = tr.size();
        tr.push_back({x, 0, 0, 1, rnd()});
        root = merge(merge(p, u), q);
    }

    bool erase(const T &x)
    {
        // x 是待删除键；只删除一个副本，成功返回 true。
        auto [p, q] = split(root, x);
        auto [u, v] = splitLE(q, x);
        bool ok = u;
        if (u)
        {
            u = merge(tr[u].l, tr[u].r);
        }
        root = merge(merge(p, u), v);
        return ok;
    }

```

```
    int rank(const T &x)
    {
        // x 是查询键；返回严格小于 x 的元素个数。
        auto [p, q] = split(root, x);
        int ans = size(p);
        root = merge(p, q);
        return ans;
    }

    T kth(int k) const
    {
        // k 是 0 基排名；返回第 k 小键。
        assert(0 <= k && k < size(root)); // 调试检查，可删
        int p = root;
        while (p)
        {
            int z = size(tr[p].l);
            if (k < z)
            {
                p = tr[p].l;
            }
            else if (k == z)
            {
                return tr[p].x;
            }
            else
            {
                k -= z + 1;
                p = tr[p].r;
            }
        }
        return T{};
    }
};
```

构造 `FHQ<int> tr` 后 `tr.insert(x)`（允许重复）、`tr.erase(x)`（删一份，成功返回 `true`）、`tr.rank(x)`（严格小于 x 的个数）、`tr.kth(k)`（第 k 小，0 基）。`split`/`splitLE`/`merge` 也公开可用，`merge(p,q)` 必须满足左树键全部不大于右树键的前提。单次操作期望 $O(\log n)$、最坏 $O(n)$，空间与插入次数同阶。相比 3.5.001 Treap 的旋转写法，它代码更短且是 3.5.003 可持久化 Treap、序列区间翻转的基础；静态有序数据无需修改时直接排序即可。下面用固定序列核对重复值、删除与排名：

```
signed main()
{
    FHQ<int> tr;
    for (int x : {4, 1, 7, 1, 3})
    {
        tr.insert(x);
    }
    assert(tr.rank(3) == 2);
    assert(tr.kth(0) == 1 && tr.kth(2) == 3 && tr.kth(4) == 7);
    assert(tr.erase(1) && !tr.erase(9));
```

```
    assert(tr.kth(0) == 1 && tr.kth(1) == 3 && tr.kth(3) == 7);
    cout << "2 1 3 7\n";
    return 0;
}
```

3.5.003 可持久化 Treap

把无旋 FHQ Treap 可持久化：`split` 与 `merge` 每经过一个结点就先 `copy` 出一份副本再改，未被触碰的子树在新旧版本间共享，于是每个历史根都对应一个不可变的完整版本。这里用隐式下标（按 `sz` 定位而非按键），因此支持序列的按位置插入、删除、区间翻转和区间求和。翻转懒标记 `rev` 是难点：`push` 下传前必须先 `flip`（复制并交换左右儿子）两个儿子，否则会破坏其他版本共享的子树——这也是可持久化带懒标记时最容易写错的地方。`values` 遍历时把一路累积的 `rev` 折进方向，故不落实标记也能按正确中序取值：

```
template <class T> struct PerTreap
{
    struct N
    {
        // l、r 是左右儿子编号，sz 是子树大小。
        int l = 0, r = 0, sz = 0;
        // p 是随机优先级；v 是本点值，s 是子树和。
        unsigned p = 0;
        T v{}, s{};
        // rev 表示当前子树的中序顺序需要翻转。
        bool rev = false;
    };

    // t 是持久化节点池，seed 是优先级生成器状态。
    vector<N> t{N{}};
    unsigned seed = 712367821; // seed 是 xorshift 随机优先级状态。

    unsigned rnd()
    {
        // 推进内部伪随机状态并返回新值。
        seed ^= seed << 13;
        seed ^= seed >> 17;
        seed ^= seed << 5;
        return seed;
    }

    int size(int x) const
    {
        // x 是节点编号；返回其子树大小，空节点返回 0。
        return t[x].sz;
    }
    T sum(int x) const
    {
        // x 是节点编号；返回其子树和，空节点返回零。
        return t[x].s;
    }
```

```
37
38      int node(T v)
39      {
40          // v 是新元素的值；新建单点并返回节点编号。
41          t.push_back({0, 0, 1, rnd(), v, v, false});
42          return t.size() - 1;
43      }
44      int copy(int x)
45      {
46          // x 是旧节点编号；复制非空节点并返回副本编号，空节点仍返回 0。
47          if (x)
48          {
49              t.push_back(t[x]);
50          }
51          return x ? (int)t.size() - 1 : 0;
52      }
53      void pull(int x)
54      {
55          // x 是非空节点编号；由两个儿子重算大小与和。
56          t[x].sz = size(t[x].l) + 1 + size(t[x].r);
57          t[x].s = sum(t[x].l) + t[x].v + sum(t[x].r);
58      }
59      int flip(int x)
60      {
61          // x 是旧子树根；复制根、交换儿子并翻转标记，返回新根。
62          x = copy(x);
63          if (x)
64          {
65              swap(t[x].l, t[x].r);
66              t[x].rev ^= 1;
67          }
68          return x;
69      }
70      void push(int x)
71      {
72          // x 是已复制的非空节点；持久化下传翻转标记。
73          if (!t[x].rev)
74          {
75              return;
76          }
77          t[x].l = flip(t[x].l);
78          t[x].r = flip(t[x].r);
79          t[x].rev = false;
80      }
81
82      pair<int, int> split(int x, int k)
83      {
84          // x 是旧版本根，k 是左段长度；返回前 k 项与其余项的新版本根。
85          if (!x)
86          {
87              return {0, 0};
88          }
```

```
        x = copy(x);
        push(x);
        if (size(t[x].l) >= k)
        {
            auto [a, b] = split(t[x].l, k);
            t[x].l = b;
            pull(x);
            return {a, x};
        }
        auto [a, b] = split(t[x].r, k - size(t[x].l) - 1);
        t[x].r = a;
        pull(x);
        return {x, b};
    }

    int merge(int x, int y)
    {
        // x、y 是相邻序列的根；返回保持中序顺序的合并版本根。
        if (!x || !y)
        {
            return x | y;
        }
        if (t[x].p < t[y].p)
        {
            x = copy(x);
            push(x);
            t[x].r = merge(t[x].r, y);
            pull(x);
            return x;
        }
        y = copy(y);
        push(y);
        t[y].l = merge(x, t[y].l);
        pull(y);
        return y;
    }

    int insert(int rt, int k, T v)
    {
        // rt 是旧版本根，k 是插入位置，v 是新值；返回插入后的版本根。
        assert(0 <= k && k <= size(rt)); // 调试检查，可删
        auto [a, b] = split(rt, k);
        return merge(merge(a, node(v)), b);
    }
    int erase(int rt, int k)
    {
        // rt 是旧版本根，k 是删除位置；返回删除后的版本根。
        assert(0 <= k && k < size(rt)); // 调试检查，可删
        auto [a, b] = split(rt, k);
        auto [c, d] = split(b, 1);
        return merge(a, d);
    }
```

```
    int reverse(int rt, int l, int r)
    {
        // rt 是旧版本根；翻转半开区间 [l,r) 并返回新版本根。
        assert(0 <= l && l <= r && r <= size(rt)); // 调试检查，可删
        auto [a, b] = split(rt, l);
        auto [c, d] = split(b, r - l);
        return merge(merge(a, flip(c)), d);
    }
    T query(int x, int l, int r, bool f) const
    {
        // x 是当前子树根，[l,r) 是相对子树的询问，f 是祖先累计的翻转状态。
        if (!x || l == r)
        {
            return T{};
        }
        if (l == 0 && r == size(x))
        {
            return sum(x);
        }
        int a = f ? t[x].r : t[x].l;
        int b = f ? t[x].l : t[x].r;
        bool g = f ^ t[x].rev;
        int z = size(a);
        T ans{};
        if (l < z)
        {
            ans += query(a, l, min(r, z), g);
        }
        if (l <= z && z < r)
        {
            ans += t[x].v;
        }
        if (z + 1 < r)
        {
            ans += query(b, max<int>(0, l - z - 1), r - z - 1, g);
        }
        return ans;
    }

    T query(int rt, int l, int r) const
    {
        // rt 是版本根；只读返回半开区间 [l,r) 的和，不新建节点。
        assert(0 <= l && l <= r && r <= size(rt)); // 调试检查，可删
        return query(rt, l, r, false);
    }

    vector<T> values(int rt) const
    {
        // rt 是版本根；返回该版本的序列值，顺序与隐式下标一致。
        vector<T> a;
        auto dfs = [&](auto &&self, int x, bool f) -> void
        {
```

```
            if (!x)
            {
                return;
            }
            int l = f ? t[x].r : t[x].l;
            int r = f ? t[x].l : t[x].r;
            bool g = f ^ t[x].rev;
            self(self, l, g);
            a.push_back(t[x].v);
            self(self, r, g);
        };
        dfs(dfs, rt, false);
        return a;
    }
};
```

用法：`PerTreap<long long> tr`，根用 `int` 版本号表示，0 是空树。`insert(rt, k, v)`、`erase(rt, k)`、`reverse(rt, l, r)` 分别在旧版本上插入、删除和翻转，并返回新版本根；三者不改旧根。`query(rt, l, r)` 只读返回 $[l,r)$ 的和，不分裂也不分配节点；`values(rt)` 按隐式下标展开该版本。更新与查询期望 $O(\log n)$、最坏 $O(n)$，每次更新新增期望 $O(\log n)$ 个结点，查询额外空间为递归栈高度。不需要历史版本时用更省内存的 3.5.002 FHQ Treap 或 3.5.004 Splay。下面检查插入、翻转、删除、区间和及旧版本持久性：

```
signed main()
{
    PerTreap<long long> tr;
    int a = tr.insert(0, 0, 1);
    int b = tr.insert(a, 1, 2);
    int c = tr.insert(b, 1, 3);       // 序列变为 [1,3,2]。
    int d = tr.reverse(c, 0, 3);      // 序列变为 [2,3,1]。
    int e = tr.erase(d, 1);           // 序列变为 [2,1]。
    assert(tr.query(c, 0, 3) == 6);
    assert(tr.query(d, 0, 2) == 5);
    assert(tr.values(e) == vector<long long>({2, 1}));
    assert(tr.values(c) == vector<long long>({1, 3, 2}));
}
```

3.5.004 Splay

Splay 靠「每次访问就把目标结点旋转到根」维持均摊 $O(\log n)$，不依赖随机也不存额外平衡信息。`splay(u, g)` 把 u 旋到 g 的儿子处（$g=0$ 即旋到根），内部按 zig-zig / zig-zag 分类：祖父、父、自身三点共线时先转父再转自身，否则连转两次自身。查询、插入、删除、`rank`、`kth` 完成后都要 splay 一下相关结点保证均摊。删除时把待删点旋到根，若计数大于 1 只减计数，否则把它的前驱旋到根再挂上右子树。它相比 Treap 的价值在于「把区间 $[l,r)$ 夹成一棵子树」（左端前驱旋到根、右端后继旋到根的左儿子），从而支持区间翻转、区间懒标记：

```
template <class T> struct Splay
{
```

```
    struct Node
    {
        // x 是键值，ch[0]、ch[1] 是左右儿子。
        T x{};
        array<int, 2> ch{};
        // fa 是父亲，cnt 是本键重数，sz 是子树元素总数。
        int fa = 0, cnt = 0, sz = 0;
    };

    // t 是节点池，root 是当前根编号。
    vector<Node> t{{}};
    int root = 0; // root 是当前根，0 表示空树。

    int size(int u) const
    {
        // u 是节点编号；返回其子树元素数，空节点返回 0。
        return u ? t[u].sz : 0;
    }

    void pull(int u)
    {
        // u 是非空节点编号；由儿子与本键重数重算子树大小。
        t[u].sz = size(t[u].ch[0]) + size(t[u].ch[1]) + t[u].cnt;
    }

    void rotate(int u)
    {
        // u 是非根节点；把 u 向父亲方向旋转一层。
        int p = t[u].fa, g = t[p].fa;
        int d = t[p].ch[1] == u, v = t[u].ch[d ^ 1];
        if (g)
        {
            t[g].ch[t[g].ch[1] == p] = u;
        }
        t[u].fa = g;
        t[u].ch[d ^ 1] = p;
        t[p].fa = u;
        t[p].ch[d] = v;
        if (v)
        {
            t[v].fa = p;
        }
        pull(p);
        pull(u);
    }

    void splay(int u, int g = 0)
    {
        // u 是待旋节点，g 是停止位置；把 u 旋成 g 的儿子，g=0 时旋到根。
        while (t[u].fa != g)
        {
            int p = t[u].fa, q = t[p].fa;
```

```
            if (q != g)
            {
                if ((t[p].ch[1] == u) == (t[q].ch[1] == p))
                {
                    rotate(p);
                }
                else
                {
                    rotate(u);
                }
            }
            rotate(u);
        }
        if (!g)
        {
            root = u;
        }
    }

    int find(const T &x)
    {
        // x 是查询键；返回对应节点，不存在时返回 0，并伸展最后访问点。
        int u = root, p = 0;
        while (u && t[u].x != x)
        {
            p = u;
            u = t[u].ch[t[u].x < x];
        }
        if (u)
        {
            splay(u);
        }
        else if (p)
        {
            splay(p);
        }
        return u;
    }

    void add(const T &x)
    {
        // x 是待插入键；重复键只增加重数。
        if (!root)
        {
            root = t.size();
            t.push_back({x, {}, 0, 1, 1});
            return;
        }
        int u = root, p = 0, d = 0;
        while (u)
        {
            if (t[u].x == x)
```

```
            {
                t[u].cnt++;
                pull(u);
                splay(u);
                return;
            }
            p = u;
            d = t[u].x < x;
            u = t[u].ch[d];
        }
        u = t.size();
        t.push_back({x, {}, p, 1, 1});
        t[p].ch[d] = u;
        splay(u);
    }

    bool erase(const T &x)
    {
        // x 是待删除键；删除一份并返回删除前是否存在。
        int u = find(x);
        if (!u)
        {
            return false;
        }
        if (t[u].cnt > 1)
        {
            t[u].cnt--;
            pull(u);
            return true;
        }
        int l = t[u].ch[0], r = t[u].ch[1];
        if (!l)
        {
            root = r;
            if (r)
            {
                t[r].fa = 0;
            }
            return true;
        }
        t[l].fa = 0;
        root = l;
        int p = l;
        while (t[p].ch[1])
        {
            p = t[p].ch[1];
        }
        splay(p);
        t[p].ch[1] = r;
        if (r)
        {
            t[r].fa = p;
```

```
        }
        pull(p);
        return true;
    }

    int rank(const T &x)
    {
        // x 是查询键；返回严格小于 x 的元素数。
        int u = root, p = 0, ans = 0;
        while (u)
        {
            p = u;
            if (x <= t[u].x)
            {
                u = t[u].ch[0];
            }
            else
            {
                ans += size(t[u].ch[0]) + t[u].cnt;
                u = t[u].ch[1];
            }
        }
        if (p)
        {
            splay(p);
        }
        return ans;
    }

    T kth(int k)
    {
        // k 是 0 基排名；返回第 k 小并把对应节点伸展到根。
        assert(0 <= k && k < size(root)); // 调试检查，可删
        int u = root;
        while (true)
        {
            int z = size(t[u].ch[0]);
            if (k < z)
            {
                u = t[u].ch[0];
            }
            else if (k < z + t[u].cnt)
            {
                splay(u);
                return t[u].x;
            }
            else
            {
                k -= z + t[u].cnt;
                u = t[u].ch[1];
            }
        }
```

```
    }

    optional<T> prev(const T &x)
    {
        // x 是查询键；返回严格前驱，不存在时返回空。
        int k = rank(x);
        return k ? optional<T>(kth(k - 1)) : nullopt;
    }

    optional<T> next(const T &x)
    {
        // x 是查询键；返回严格后继，不存在时返回空。
        int u = root, p = 0, best = 0;
        while (u)
        {
            p = u;
            if (x < t[u].x)
            {
                best = u;
                u = t[u].ch[0];
            }
            else
            {
                u = t[u].ch[1];
            }
        }
        if (best)
        {
            splay(best);
            return t[best].x;
        }
        if (p)
        {
            splay(p);
        }
        return nullopt;
    }
};
```

用法与前两种平衡树一致：`Splay<int> tr;` 后 `tr.add(x)`、`tr.erase(x)`（成功返回 `true`）、`tr.rank(x)`（严格小于 x 的个数）、`tr.kth(k)`（第 k 小，0 基）、`tr.prev(x)`/`tr.next(x)`（严格前驱后继的 `optional`）。单次操作最坏 $O(n)$、连续 m 次摊还 $O(m \log n)$，空间与插入次数同阶。纯有序集合它并不比 3.5.002 FHQ Treap 有优势且常数更大；它的不可替代之处是 LCT 的底层和需要频繁 “区间提取” 的场景。长期只增不查退化时可用 3.5.005 Splay 重构 拍平。下面用一组固定操作覆盖重复键、删除、排名和前驱后继：

```
signed main()
{
    Splay<int> tr;
    for (int x : {4, 1, 7, 1, 3})
    {
        tr.add(x);
```

```
    }
    assert(tr.rank(3) == 2);
    assert(tr.kth(0) == 1 && tr.kth(2) == 3 && tr.kth(4) == 7);
    assert(tr.erase(1) && !tr.erase(9));
    assert(tr.kth(0) == 1 && tr.kth(1) == 3);
    assert(!tr.prev(1) && tr.prev(4) == optional<int>(3));
    assert(tr.next(4) == optional<int>(7) && !tr.next(7));
    cout << "2 1 3 7\n";
    return 0;
}
```

3.5.005 Splay 重构

当 Splay 长期只按同向插入而退化成一条链，或想一次性批量建树时，可按中序收集结点后重建。收集和按中点建树都直接使用递归 lambda；建树阶段深度只有 $O(\log n)$。重建后树高回到 $O(\log n)$，之后仍靠正常 Splay 操作维持均摊。它依赖 3.5.004 Splay 的结点结构，作为其外挂函数直接访问 `s.t`、`s.root`、`s.pull`：

```
template <class T> void rebuild(Splay<T> &s)
{
    // s 表示待重构的 Splay；保留键、重复计数和节点编号，重连成高度为 O(log n) 的平衡树。
    vector<int> id;
    auto collect = [&](auto &&self, int p) -> void
    {
        if (!p)
        {
            return;
        }
        self(self, s.t[p].ch[0]);
        id.push_back(p);
        self(self, s.t[p].ch[1]);
    };
    collect(collect, s.root);

    auto build = [&](auto &&self, int l, int r, int fa) -> int
    {
        if (l == r)
        {
            return 0;
        }
        int m = midpoint(l, r), p = id[m];
        s.t[p].fa = fa;
        s.t[p].ch[0] = self(self, l, m, p);
        s.t[p].ch[1] = self(self, m + 1, r, p);
        s.pull(p);
        return p;
    };
    s.root = build(build, 0, id.size(), 0);
}
```

对已有 `Splay<T> tr` 调用 `rebuild(tr)`：保留所有键与重复计数、结点编号不变，只重连成 $O(\log n)$ 高的平衡形态，重建本身 $O(n)$。例如依次插入 $4,1,7,1,3$ 再 `rebuild` 后 `size(root)==5`、`kth(0)==1`、`kth(2)==3`、`rank(7)==4`。它只是 3.5.004 Splay 的辅助手段；若本就用无旋 3.5.002 FHQ Treap 则无需重建，随机优先级已保证期望树高。

3.5.006 珂朵莉树（ODT）

珂朵莉树（ODT / Chtholly Tree）用一个 `set` 维护若干两两不交的常值区间，每个结点是 $[l,r)$ 加统一值 `v`。核心只有一个 `split(x)`：把包含 x 的区间在 x 处切成两段并返回后一段的迭代器，之后任何区间操作都先 `split(r)` 再 `split(l)`（顺序不能反，否则 `split(l)` 可能使 `split(r)` 得到的迭代器失效）拿到恰好覆盖 $[l,r)$ 的一批结点。真正让它在部分数据上有效的是 `assign`——区间赋值把触及的结点全删掉换成一个，实际段数可能长期较小；但没有明确随机模型时不存在可复用的期望对数保证，构造数据可令一系列操作达到二次时间。`mutable T v` 是为了能在 `set` 里原地改值：

```cpp
template <class T> struct ODT
{
    struct Node
    {
        // [l,r) 是常值段，v 是该段的值。
        int l, r;
        mutable T v;

        bool operator<(const Node &o) const
        {
            // o 是另一常值段；只按左端点排序。
            return l < o.l;
        }
    };

    // n 是数组长度，s 按左端点保存两两不交的常值段。
    int n;
    set<Node> s;

    ODT(const vector<T> &a) : n(a.size())
    {
        // a 是初始数组；空数组得到空段集。
        for (int l = 0, r; l < n; l = r)
        {
            for (r = l + 1; r < n && a[r] == a[l]; r++)
            {
                ;
            }
            s.insert({l, r, a[l]});
        }
    }

    auto split(int x)
    {
        // x 是分割位置；返回左端点为 x 的段，x=n 时返回 end()。
```

```
        assert(0 <= x && x <= n); // 调试检查，可删
        if (x == n)
        {
            return s.end();
        }
        auto it = prev(s.upper_bound({x, 0, {}}));
        if (it->l == x)
        {
            return it;
        }
        auto [l, r, v] = *it;
        s.erase(it);
        s.insert({l, x, v});
        return s.insert({x, r, v}).first;
    }

    void assign(int l, int r, const T &v)
    {
        // l、r 是半开区间端点，v 是新值；空区间无操作。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return;
        }
        auto y = split(r), x = split(l);
        s.erase(x, y);
        s.insert({l, r, v});
    }

    template <class F> void apply(int l, int r, F f)
    {
        // l、r 是半开区间端点；对相交常值段的值逐段调用 f(T&)，空区间不调用。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        if (l == r)
        {
            return;
        }
        auto y = split(r), x = split(l);
        for (auto it = x; it != y; it++)
        {
            f(it->v);
        }
    }

    T kth(int l, int r, int k)
    {
        // l、r 是非空半开区间端点，k 是 0 基排名；返回按位置计重后的第 k 小值。
        assert(0 <= l && l < r && r <= n && 0 <= k && k < r - l); // 调试检查，可删
        auto y = split(r), x = split(l);
        vector<pair<T, int>> a;
        for (auto it = x; it != y; it++)
        {
```

```
            a.push_back({it->v, it->r - it->l});
        }
        sort(a.begin(), a.end());
        for (auto [v, c] : a)
        {
            if (k < c)
            {
                return v;
            }
            k -= c;
        }
        assert(false);
        return {};
    }

    vector<T> values() const
    {
        // 返回当前数组，元素顺序按下标排列。
        vector<T> a(n);
        for (auto [l, r, v] : s)
        {
            fill(a.begin() + l, a.begin() + r, v);
        }
        return a;
    }
};
```

构造 `ODT<long long> odt(a)`，此后所有区间都用左闭右开 $[l, r)$：`assign(l, r, v)` 区间赋值，`apply(l, r, f)` 对区间内每个段的值调用 `f(T&)`，`kth(l, r, k)` 返回区间内按位置计重的第 k 小（0 基，要求 $l < r$），`values()` 展开成完整数组。空初始数组合法；`assign` 和 `apply` 接受空区间并直接返回。`split` 单次 $O(\log S)$（S 为段数），一次区间操作触及 k 段则 $O((k+1)\log S)$，最坏 $O(n \log n)$，空间最坏 $O(n)$。没有大量区间赋值时它会退化，此时该用 3.3.002 懒标记线段树；确定性复杂度需求下也应优先线段树。下面用固定数组覆盖空结构、空修改、赋值、逐段加和合法排名：

```
signed main()
{
    ODT<long long> empty(vector<long long>{});
    empty.assign(0, 0, 7);
    empty.apply(0, 0, [](long long &x)
    {
        x++;
    });
    assert(empty.values().empty());

    ODT<long long> odt({1, 1, 2, 2, 3});
    odt.assign(1, 4, 5);
    odt.assign(2, 2, 9);
    odt.apply(2, 5, [](long long &x)
    {
        x += 2;
    });
```

```
    assert((odt.values() == vector<long long>{1, 5, 7, 7, 5}));
    assert(odt.kth(0, 5, 0) == 1);
    assert(odt.kth(0, 5, 2) == 5);
    assert(odt.kth(0, 5, 4) == 7);
    cout << "1 5 7\n";
    return 0;
}
```

3.5.007 PBDS 平衡树

这是一个选型入口而非新实现：当你需要「有序集合 + 第 k 小 / 排名」且允许 GNU 扩展时，用 PBDS 的 `tree` 最省事，它在红黑树结点上多维护子树大小，`order_of_key(x)` 给出严格小于 x 的元素数、`find_by_order(k)` 给出第 k 小（0 基）。完整类型定义见 3.9.007 tree_order_statistics_node_update，调用细节见 3.9.008 find_by_order 与 order_of_key，此处不提前重复。只需按键插删、前驱后继而不要排名时，标准 `set`/`multiset` 就够：

```
using OrderedSet = set<int>;
using OrderedMSet = multiset<int>;

// 只需要按键插删、前驱和后继时直接使用上述标准容器；第 k 小与排名接口见 3.9.007、3.9.008。
```

PBDS `tree` 的 insert、erase、order_of_key、find_by_order 均 $O(\log n)$，空间 $O(n)$。不想引入 GNU 依赖、又要第 k 小和排名时，改用手写的 3.5.001 Treap 或 3.5.002 FHQ Treap。PBDS 接口省事，但节点布局、更新策略、编译器和数据分布都会影响常数；此前某个平台上的快慢只能保留为该环境的参考，接近时限时应与手写实现用同一数据实测，不能写成普遍优劣。

3.5.008 左偏树 / 可并堆

左偏树是可并堆：$O(\log n)$ 合并两个堆，同时支持取最值、删堆顶。每个结点维护「零路径长」`distance`（到最近空儿子的距离），并保持右儿子的 `distance` 不大于左儿子（这就是「左偏」），从而右链长度是 $O(\log n)$。合并 `merge(x,y)` 只沿右链递归：取根较优者作根，把它的右子树与另一棵合并回去，若合并后右儿子比左儿子深就交换，再更新 `distance`。插入是与单点堆合并，删堆顶是把根的左右子树合并。它比 STL 堆多出的能力就是合并，典型用于图上连通块各持一堆、频繁 union 的题：

```
template <class T, class Compare = less<T>> class LeftistHeap
{
    struct Node
    {
        // x 是键值；l、r 是儿子；d 是到空儿子的最短距离。
        T x{};
        int l = 0, r = 0, d = 1;
    };

    // cmp 决定堆序，t 是节点池，0 表示空节点且 d=0。
    Compare cmp;
    vector<Node> t{{T{}, 0, 0, 0}};

```

```
  public:
    int makeHeap(const T &x)
    {
        // x 是键值；新建零路径长为 1 的单点堆并返回根编号。
        t.push_back({x});
        return t.size() - 1;
    }

    int merge(int x, int y)
    {
        // x、y 是节点不共享的两个堆根；破坏性合并并返回新根，两个旧堆句柄随即失效。
        if (!x || !y)
        {
            return x | y;
        }
        if (cmp(t[y].x, t[x].x))
        {
            swap(x, y);
        }
        t[x].r = merge(t[x].r, y);
        if (t[t[x].l].d < t[t[x].r].d)
        {
            swap(t[x].l, t[x].r);
        }
        t[x].d = t[t[x].r].d + 1;
        return x;
    }

    const T &top(int root) const
    {
        // root 是非空堆根；返回堆顶键值。
        assert(root); // 调试检查，可删
        return t[root].x;
    }

    int pop(int root)
    {
        // root 是非空堆根；删除堆顶并返回新根。
        assert(root); // 调试检查，可删
        return merge(t[root].l, t[root].r);
    }
};
```

用法：`LeftistHeap<int> h;`，堆用根编号表示，0 是空堆。`int r = h.makeHeap(v)` 建单点堆，`r = h.merge(x, y)` 破坏性合并两棵节点不共享的堆并返回新根，`h.top(r)` 取最优（默认 `less` 即小根堆返回最小），`r = h.pop(r)` 删堆顶返回新根。`merge` 后 `x`、`y` 都不能再代表原来的独立堆；`pop` 后旧根及由它表示的旧堆句柄同样失效。`makeHeap`、`top` 为 $O(1)$，`merge`、`pop` 最坏 $O(\log n)$，空间 $O(n)$。只取最值不合并时 STL `priority_queue` 更简单更快；这里唯一价值是可并性。下面分别建立两堆，合并后依次弹出：

```
signed main()
{
```

```
    LeftistHeap<int> h;
    int a = h.merge(h.makeHeap(4), h.makeHeap(1));
    a = h.merge(a, h.makeHeap(7));
    int b = h.merge(h.makeHeap(3), h.makeHeap(2));
    assert(h.top(a) == 1 && h.top(b) == 2);
    int root = h.merge(a, b);
    for (int x : {1, 2, 3, 4, 7})
    {
        assert(h.top(root) == x);
        root = h.pop(root);
    }
    assert(root == 0);
    cout << "1 2 3 4 7\n";
}
```

3.5.009 带删堆

带删堆（懒删除堆）用两个普通 `priority_queue` 模拟支持任意值删除的堆：`a` 存所有插入、`b` 存所有待删除。删除某值只是把它压进 `b`，并不立即从 `a` 移除；每次要读堆顶时先 `clean()`——当 `a` 与 `b` 的堆顶相等时成对弹出，这样暴露出来的 `a.top()` 一定是真正有效的最值。前提是删除的值此前确实存在（否则会错误抵消掉未来出现的相等值），调用方需自行保证。用 `n` 单独记录有效元素个数，避免遍历统计：

```
template <class T, class Cmp = less<T>> struct ErasableHeap
{
    // a 保存全部插入，b 保存待抵消删除，n 是当前有效元素数。
    priority_queue<T, vector<T>, Cmp> a, b;
    int n = 0;

    void clean()
    {
        // 抵消已经同时暴露在两个堆顶的失效元素。
        while (!a.empty() && !b.empty() && a.top() == b.top())
        {
            a.pop();
            b.pop();
        }
    }
    void push(const T &x)
    {
        // x 是新键；插入一份。
        a.push(x);
        n++;
    }
    void erase(const T &x)
    {
        // x 是当前确实存在的键；登记删除一份。
        b.push(x);
        n--;
        clean();
    }
```

```
    const T &top()
    {
        // 返回当前非空堆的最优键。
        assert(n > 0); // 调试检查，可删
        clean();
        return a.top();
    }
    void pop()
    {
        // 删除当前非空堆的最优键。
        assert(n > 0); // 调试检查，可删
        clean();
        a.pop();
        n--;
        clean();
    }
    int size() const
    {
        // 返回有效元素数。
        return n;
    }
    bool empty() const
    {
        // 返回是否没有有效元素。
        return n == 0;
    }
};
```

用法：`ErasableHeap<int> h;`（默认 `less` 为大根堆，取最大值），`h.push(x)` 插入，`h.erase(x)` 删除一份已存在的 x，`h.top()` 取当前最值，`h.pop()` 删最值，`h.size()`/`h.empty()` 查询规模。各操作摊还 $O(\log n)$，某次 `clean` 若清掉 k 个失效元素则耗时 $O(k\log n)$，但每个元素至多被清一次，空间 $O(q)$。要按任意值删且还要第 k 小或前驱后继时用 3.5.002 FHQ Treap；只删堆顶用 STL `priority_queue` 即可。下面固定检查重复最大值、删除非堆顶与连续弹出：

```
signed main()
{
    ErasableHeap<int> h;
    for (int x : {4, 1, 7, 7, 3})
    {
        h.push(x);
    }
    assert(h.size() == 5 && h.top() == 7);
    h.erase(7);
    assert(h.size() == 4 && h.top() == 7);
    h.erase(1);
    h.pop();
    assert(h.size() == 2 && h.top() == 4);
    h.erase(3);
    h.pop();
    assert(h.empty());
    cout << "7 7 4 0\n";
```

```
    return 0;
}
```

3.5.010 Top-K 和维护

动态维护一个可增删的多重集合里「最大的 k 个元素之和」。用两个 `multiset`：`hi` 存当前入选的最大 k 个、`lo` 存其余，`sum` 是 `hi` 的总和。插入时按它和 `hi` 最小值的大小决定进哪边，删除时在两边查找并相应更新 `sum`，之后统一 `fix()`：若 `hi` 多了就把它的最小值移到 `lo`，若 `hi` 少了且 `lo` 非空就把 `lo` 的最大值移进来，始终保持 `hi` 恰含最大的 $\min(k, \text{总数})$ 个。这样 `query` 只需 $O(1)$ 读 `sum`，避免每次重新求和：

```
template <class T> struct TopK
{
    // k 是目标个数，sum 是 hi 中元素和。
    int k;
    T sum{};
    // hi 保存当前最大的 min(k,n) 个元素，lo 保存其余元素。
    multiset<T> hi, lo;

    TopK(int k) : k(k)
    {
        // k 是要取的最大元素个数，允许 k=0。
        assert(k >= 0); // 调试检查，可删
    }

    void fix()
    {
        // 调整两组大小，使 hi 恰含最大的 min(k,n) 个元素。
        while ((int)hi.size() > k)
        {
            auto it = hi.begin();
            sum -= *it;
            lo.insert(*it);
            hi.erase(it);
        }
        while ((int)hi.size() < k && !lo.empty())
        {
            auto it = prev(lo.end());
            sum += *it;
            hi.insert(*it);
            lo.erase(it);
        }
    }

    void insert(T x)
    {
        // x 是新元素；插入一份并恢复分组。
        if (!hi.empty() && x >= *hi.begin())
        {
            hi.insert(x);
            sum += x;
```

```
        }
        else
        {
            lo.insert(x);
        }
        fix();
    }

    bool erase(T x)
    {
        // x 是待删元素；删除一份并返回删除前是否存在。
        auto it = hi.find(x);
        if (it != hi.end())
        {
            sum -= x;
            hi.erase(it);
            fix();
            return true;
        }
        it = lo.find(x);
        if (it == lo.end())
        {
            return false;
        }
        lo.erase(it);
        return true;
    }

    optional<T> query() const
    {
        // 返回最大 k 个元素之和；总数不足 k 时返回空，k=0 时返回零。
        if ((int)hi.size() < k)
        {
            return nullopt;
        }
        return sum;
    }
};
```

构造 `TopK<long long> t(k)`，`t.insert(x)` 插入、`t.erase(x)` 删一份（成功返回 `true`）、`t.query()` 返回当前最大 k 个之和；元素不足 k 个时为 `nullopt`，$k=0$ 时恒为零。`insert`、`erase` 均 $O(\log n)$，`query` 为 $O(1)$，空间 $O(n)$。若要的是第 k 大的值而非前 k 大之和，用 3.5.002 FHQ Treap 的 `kth`；只求静态一次 Top-K 用 `nth_element` 即可。下面固定检查元素不足、重复值、删除和 $k=0$：

```
signed main()
{
    TopK<long long> t(3);
    assert(!t.query());
    t.insert(5);
    t.insert(1);
    assert(!t.query());
```

```
    t.insert(4);
    assert(t.query() == optional<long long>(10));
    t.insert(9);
    assert(t.query() == optional<long long>(18));
    t.insert(4);
    assert(t.query() == optional<long long>(18));
    assert(t.erase(5) && !t.erase(8));
    assert(t.query() == optional<long long>(17));
    assert(t.erase(9));
    assert(t.query() == optional<long long>(9));

    TopK<long long> zero(0);
    zero.insert(7);
    assert(zero.query() == optional<long long>(0));
    cout << "10 18 17 9 0\n";
    return 0;
}
```

3.5.011 隐式 Treap（动态序列、翻转、仿射与区间和）

当元素按当前位置插入、删除，且查询或修改区间会随之移动时，固定下标的线段树不能直接复用。隐式 Treap 把中序排名当作键：结点的随机优先级满足小根堆序，`sz` 记录左子树大小，于是 `split(root,k)` 可以切成前 k 项和其余项，`merge` 再把相邻两段接回。当前结构只维护一个可变版本；它与 3.5.003 可持久化 Treap 的历史根语义不同，也比 3.5.002 FHQ Treap 的有序键集合多出位置、区间和与懒标记。

每个结点还维护子树和 `sum`、翻转标记 `rev` 与仿射标记 (mul, add)。后者表示尚未下传的 $g(x) = x \cdot mul + add$；若随后对整段施加 $f(x) = x \cdot a + b$，组合是 $f(g(x)) = x \cdot (mul \cdot a) + (add \cdot a + b)$，故 `mul=mul*a`、`add=add*a+b`。区间和同步变成 $sum \cdot a + len \cdot b$；翻转只交换儿子，且与逐点仿射可交换。`T` 需要零元 `T{}`、单位元 `T(1)`、`+`、`*` 和相等比较，乘法须可交换；普通精确和可取 `long long`（调用者保证所有值和区间和不溢出），模 998244353 则直接复用 1.1.008 模整数 的 `Z`。节点池不回收删除结点，预知最多会插入 q 个元素时以 `reserveNodes(n+q)` 预留容量：

```
template <class T> struct ImpTreap
{
  private:
    struct Node
    {
        // l、r 是左右儿子，sz 是子树元素数，pri 是随机小根堆优先级。
        int l = 0, r = 0, sz = 0;
        unsigned pri = 0;
        // val 是本点值，sum 是子树和，mul、add 表示尚未下传的 x -> x*mul+add。
        T val{}, sum{}, mul{1}, add{};
        // rev 表示当前子树的中序顺序需要翻转。
        bool rev = false;
    };

    // tr 是节点池，root 是当前序列根，seed 是 xorshift 优先级状态。
    vector<Node> tr{Node{}};
    int root = 0; // root 是当前序列根，0 表示空序列。
```

```
    unsigned seed = 712367821; // seed 是 xorshift 随机优先级状态。

    unsigned rnd()
    {
        // 推进内部伪随机状态并返回新的优先级。
        seed ^= seed << 13;
        seed ^= seed >> 17;
        seed ^= seed << 5;
        return seed;
    }

    int sizeNode(int x) const
    {
        // x 是节点编号；返回其子树元素数，0 号空节点返回 0。
        return tr[x].sz;
    }

    T sumNode(int x) const
    {
        // x 是节点编号；返回其子树和，0 号空节点返回零元。
        return tr[x].sum;
    }

    int newNode(const T &x)
    {
        // x 是新元素值；新建一个无儿子的节点并返回编号。
        tr.push_back({0, 0, 1, rnd(), x, x, T(1), T{}, false});
        return (int)tr.size() - 1;
    }

    void pull(int x)
    {
        // x 是非空节点；由儿子与本点值重算大小和子树和。
        tr[x].sz = sizeNode(tr[x].l) + 1 + sizeNode(tr[x].r);
        tr[x].sum = sumNode(tr[x].l) + tr[x].val + sumNode(tr[x].r);
    }

    void applyAffine(int x, const T &mul, const T &add)
    {
        // x 是子树根，mul、add 表示后执行的 x -> x*mul+add；原地更新节点摘要和懒标记。
        if (!x)
        {
            return;
        }
        tr[x].val = tr[x].val * mul + add;
        tr[x].sum = tr[x].sum * mul + add * T(sizeNode(x));
        tr[x].mul = tr[x].mul * mul;
        tr[x].add = tr[x].add * mul + add;
    }

    void applyReverse(int x)
    {
```

```
        // x 是子树根；交换左右儿子并记录一次延迟翻转。
        if (!x)
        {
            return;
        }
        swap(tr[x].l, tr[x].r);
        tr[x].rev = !tr[x].rev;
    }

    void push(int x)
    {
        // x 是非空节点；把翻转与仿射懒标记按语义下传给两个儿子。
        if (tr[x].rev)
        {
            applyReverse(tr[x].l);
            applyReverse(tr[x].r);
            tr[x].rev = false;
        }
        if (tr[x].mul != T(1) || tr[x].add != T{})
        {
            applyAffine(tr[x].l, tr[x].mul, tr[x].add);
            applyAffine(tr[x].r, tr[x].mul, tr[x].add);
            tr[x].mul = T(1);
            tr[x].add = T{};
        }
    }

    void pullAll(int x)
    {
        // x 是新建 Cartesian 树的根；递归重算该子树全部摘要。
        if (!x)
        {
            return;
        }
        pullAll(tr[x].l);
        pullAll(tr[x].r);
        pull(x);
    }

    pair<int, int> split(int x, int k)
    {
        // x 是子树根，k 是左段长度；返回前 k 项与其余项的两棵树根。
        if (!x)
        {
            return {0, 0};
        }
        push(x);
        int z = sizeNode(tr[x].l);
        if (k <= z)
        {
            auto [a, b] = split(tr[x].l, k);
            tr[x].l = b;
```

```
            pull(x);
            return {a, x};
        }
        auto [a, b] = split(tr[x].r, k - z - 1);
        tr[x].r = a;
        pull(x);
        return {x, b};
    }

    int merge(int x, int y)
    {
        // x、y 是相邻两段的根；按中序保持 x 在前，返回合并后的根。
        if (!x || !y)
        {
            return x ? x : y;
        }
        if (tr[x].pri < tr[y].pri)
        {
            push(x);
            tr[x].r = merge(tr[x].r, y);
            pull(x);
            return x;
        }
        push(y);
        tr[y].l = merge(x, tr[y].l);
        pull(y);
        return y;
    }

  public:
    explicit ImpTreap(int cap = 0)
    {
        // cap 是预计节点总数，只用于预留池容量；构造空序列。
        assert(cap >= 0); // 调试检查，可删
        reserveNodes(cap);
    }

    explicit ImpTreap(const vector<T> &a, int cap = 0)
    {
        // a 是初始序列，cap 是预计节点总数；在线性时间建出当前序列。
        assert(cap >= 0); // 调试检查，可删
        reserveNodes(max<int>(cap, (int)a.size()));
        build(a);
    }

    void reserveNodes(int cap)
    {
        // cap 是预计节点总数；预留 0 号哨兵外的池容量，不改变现有序列。
        assert(cap >= 0); // 调试检查，可删
        tr.reserve(cap + 1);
    }

```

```
    void clear()
    {
        // 清空当前序列和节点池内容，保留已申请容量并重置随机状态。
        tr.clear();
        tr.push_back({});
        root = 0;
        seed = 712367821;
    }

    void build(const vector<T> &a)
    {
        // a 是新的完整序列；清空旧序列，用优先级 Cartesian 树在线性时间重建。
        clear();
        reserveNodes((int)a.size());
        vector<int> st;
        st.reserve(a.size());
        for (const T &x : a)
        {
            int u = newNode(x), last = 0;
            while (!st.empty() && tr[st.back()].pri > tr[u].pri)
            {
                last = st.back();
                st.pop_back();
            }
            tr[u].l = last;
            if (!st.empty())
            {
                tr[st.back()].r = u;
            }
            st.push_back(u);
        }
        root = st.empty() ? 0 : st[0];
        pullAll(root);
    }

    int size() const
    {
        // 返回当前序列元素数。
        return sizeNode(root);
    }

    bool empty() const
    {
        // 返回当前序列是否为空。
        return root == 0;
    }

    void insert(int k, const T &x)
    {
        // k 是插入位置，x 是新值；把 x 插到原第 k 项前，允许 k=size()。
        assert(0 <= k && k <= size()); // 调试检查，可删
        auto [a, b] = split(root, k);
```

```
226         root = merge(merge(a, newNode(x)), b);
227     }
228 
229     void erase(int k)
230     {
231         // k 是删除位置；删除原第 k 项，已分配节点不回收。
232         assert(0 <= k && k < size()); // 调试检查，可删
233         auto [a, b] = split(root, k);
234         root = merge(a, split(b, 1).second);
235     }
236 
237     void reverse(int l, int r)
238     {
239         // l、r 是半开区间端点；原地翻转 [l,r)，空区间允许。
240         assert(0 <= l && l <= r && r <= size()); // 调试检查，可删
241         auto [a, b] = split(root, l);
242         auto [c, d] = split(b, r - l);
243         applyReverse(c);
244         root = merge(merge(a, c), d);
245     }
246 
247     void affine(int l, int r, const T &mul, const T &add)
248     {
249         // l、r 是半开区间端点，mul、add 是变换参数；对 [l,r) 每项执行 x -> x*mul+add。
250         assert(0 <= l && l <= r && r <= size()); // 调试检查，可删
251         auto [a, b] = split(root, l);
252         auto [c, d] = split(b, r - l);
253         applyAffine(c, mul, add);
254         root = merge(merge(a, c), d);
255     }
256 
257     T sum(int l, int r)
258     {
259         // l、r 是半开区间端点；返回 [l,r) 的元素和，空区间返回零元。
260         assert(0 <= l && l <= r && r <= size()); // 调试检查，可删
261         auto [a, b] = split(root, l);
262         auto [c, d] = split(b, r - l);
263         T ans = sumNode(c);
264         root = merge(merge(a, c), d);
265         return ans;
266     }
267 
268     vector<T> values()
269     {
270         // 把当前序列按下标顺序复制到 vector；会下传懒标记但不改变序列语义。
271         vector<T> a;
272         a.reserve(size());
273         auto dfs = [&](auto &&self, int x) -> void
274         {
275             if (!x)
276             {
277                 return;
```

```
            }
            push(x);
            self(self, tr[x].l);
            a.push_back(tr[x].val);
            self(self, tr[x].r);
        };
        dfs(dfs, root);
        return a;
    }
};
```

构造 `ImpTreap<T> tr(cap)` 得空序列，或 `ImpTreap<T> tr(a,cap)` 以 $O(n)$ 建好 `a`；`cap` 仅预留节点数。`insert(k,x)` 在原第 k 项前插入，`erase(k)` 删除第 k 项，`reverse(l,r)` 翻转、`affine(l,r,mul,add)` 施加仿射、`sum(l,r)` 求和，所有位置都是 0 基半开区间且允许空区间（删除除外）；`values()` 仅用于调试，线性返回当前全序列。随机优先级模型下，建树 $O(n)$，每个区间操作期望 $O(\log n)$、最坏 $O(n)$，`values()` 为 $O(n)$；空间 $O(n+i)$，其中 i 是累计插入数。固定长度的区间仿射和优先用 3.3.003 区间加、乘、赋值（仿射懒标记），要保存历史版本用 3.5.003 可持久化 Treap，只维护按值排序的集合用 3.5.002 FHQ Treap。

下面固定核对位置插删、翻转、仿射、非空与空区间和：

```
signed main()
{
    ImpTreap<long long> tr(vector<long long>{1, 2, 3, 4}, 16);
    tr.reverse(1, 4);
    tr.affine(1, 3, 2, 1);
    tr.insert(2, 5);
    tr.erase(1);
    assert(tr.values() == vector<long long>({1, 5, 7, 2}));
    assert(tr.sum(1, 4) == 14 && tr.sum(2, 2) == 0);
    cout << "1 5 7 2 14 0\n";
    return 0;
}
```

3.5.012 可持久化队列

可持久化队列从任意历史版本分叉，只支持尾部追加和删除队首。不能把普通两栈队列直接持久化：一次把后栈倒到前栈会复制整段，`two_stacks_killer` 这类分叉输入可把均摊重建反复触发，达到二次时间。这里把每次 `push` 的元素看成一条从新队尾指向旧队尾的不可变父链；一个版本只保存 `tail` 和逻辑长度 `len`。当 `len>0` 时，从 `tail` 往前数 `len-1` 步恰是队首；`pop` 不修改节点，只把 `len` 减一并新建一个版本。

为了快速跳到队首，每个元素节点保存二进制祖先 `up[k]`：

$$up(v,0)=parent(v),\qquad up(v,k)=up(up(v,k-1),k-1).$$

于是把距离按二进制拆开即可 $O(\log Q)$ 取祖先。旧节点和旧版本从不改写，所有版本编号都可以长期保存。构造时应传入本题总操作数 Q 作为 `cap` 以预留节点与版本数组；默认 `LOG=20` 覆盖 $Q\le 5\times 10^5$。

若操作数上界更大，实例化 PersistQueue<T, LOG> 时要选满足 $2^{LOG} > Q$ 的 LOG。这不是支持任意区间修改的通用持久化序列；需要插入、删除、翻转和区间和时用 3.5.003 可持久化 Treap。

```cpp
template <class T, int LOG = 20> class PersistQueue
{
    static_assert(1 <= LOG && LOG < 63);

    struct Node
    {
        // value 是本次入队的值；up[k] 是沿历史尾链向前 2^k 步的节点编号。
        T value{};
        array<int, LOG> up{};
    };

    struct Version
    {
        // tail 是逻辑队尾节点，len 是该版本未被弹出的逻辑元素数。
        int tail = 0, len = 0;
    };

    // nodes[0] 是空哨兵；versions[0] 是初始空队列。
    vector<Node> nodes{Node{}};
    vector<Version> versions{Version{}};

    int anc(int p, int distance) const
    {
        // p 是非空尾节点，distance 是向前跳的步数；返回对应祖先节点编号。
        assert(0 < p && p < (int)nodes.size()); // 调试检查，可删
        assert(0 <= distance && (unsigned long long)distance < (1ULL << LOG)); // 调试检查，可删
        for (int k = 0; distance; k++, distance >>= 1)
        {
            if (distance & 1)
            {
                p = nodes[p].up[k];
            }
        }
        return p;
    }

  public:
    explicit PersistQueue(int cap = 0)
    {
        // cap 是预计总操作数，仅用于预留节点和版本容量；默认 LOG=20 覆盖 Q<=5e5。
        assert(0 <= cap && (unsigned long long)cap < (1ULL << LOG)
               && cap <= numeric_limits<int>::max()); // 调试检查，可删
        nodes.reserve(cap + 1);
        versions.reserve(cap + 1);
    }

    int emptyVer() const
    {
        // 返回初始空队列的版本编号。
```

```
        return 0;
    }

    int verCnt() const
    {
        // 返回当前已经创建的版本数，包含初始空版本。
        return (int)versions.size();
    }

    int size(int version) const
    {
        // version 是版本编号；返回该版本的逻辑队列长度。
        assert(0 <= version && version < (int)versions.size()); // 调试检查，可删
        return versions[version].len;
    }

    bool empty(int version) const
    {
        // version 是版本编号；返回该版本是否为空。
        return size(version) == 0;
    }

    int push(int version, const T &value)
    {
        // version 是旧版本，value 是新队尾元素；返回追加后的新版本编号。
        assert(0 <= version && version < (int)versions.size()); // 调试检查，可删
        int p = (int)nodes.size();
        assert((unsigned long long)p < (1ULL << LOG)
               && p <= numeric_limits<int>::max()); // 调试检查，可删
        Node cur;
        cur.value = value;
        cur.up[0] = versions[version].len ? versions[version].tail : 0;
        for (int k = 1; k < LOG; k++)
        {
            cur.up[k] = nodes[cur.up[k - 1]].up[k - 1];
        }
        nodes.push_back(move(cur));
        int len = versions[version].len + 1;
        assert(len <= numeric_limits<int>::max()); // 调试检查，可删
        versions.push_back({(int)p, (int)len});
        return (int)versions.size() - 1;
    }

    T front(int version) const
    {
        // version 是非空版本编号；返回该版本的队首值，不生成新版本。
        assert(0 <= version && version < (int)versions.size()); // 调试检查，可删
        assert(versions[version].len > 0); // 调试检查，可删
        int p = anc(versions[version].tail, versions[version].len - 1);
        return nodes[p].value;
    }

```

```
    int pop(int version)
    {
        // version 是非空版本编号；删除其队首并返回新版本编号。
        assert(0 <= version && version < (int)versions.size()); // 调试检查，可删
        assert(versions[version].len > 0); // 调试检查，可删
        versions.push_back({versions[version].tail, (int)(versions[version].len - 1)});
        return (int)versions.size() - 1;
    }
};
```

`emptyVer()` 返回初始空版本；`push(version,value)` 和 `pop(version)` 都返回一个新的版本编号，`front(version)` 只读队首，`size` / `empty` 查询版本状态。所有版本号必须来自这些接口，`front` / `pop` 要求非空。本题可写成 `PersistQueue<int> q(Q)`，把第 i 次操作得到的版本编号保存到 `ver[i+1]`，并以 `ver[t+1]` 访问题面里的 S_t：

```
signed main()
{
    PersistQueue<int> q(10);
    int v0 = q.emptyVer();
    int v1 = q.push(v0, 4);
    int v2 = q.push(v1, 7);
    int v3 = q.pop(v2);
    int v4 = q.push(v1, 9);
    assert(q.front(v2) == 4 && q.front(v3) == 7);
    assert(q.front(v4) == 4 && q.size(v4) == 2);
}
```

在预留 `cap` 后，`push` 和 `front` 最坏 $O(LOG) = O(\log Q)$，`pop` 最坏 $O(1)$；每个 `push` 新增一个含 LOG 个祖先的节点，每次 `push` / `pop` 新增一个版本记录，所以 q 次操作总空间 $O(q \log Q)$。普通两栈队列只适用于不保留分叉历史、接受均摊界的场景；若版本只是单点修改的定长数组，使用 3.6.007 可持久化数组 更直接；若要一般持久化序列操作改用 3.5.003 可持久化 Treap。

3.5.013 Radix Heap / 基数堆

Radix Heap 是键单调的整数最小优先队列：每次新插入的键都必须不小于最近一次弹出的键。它把键按“与最近弹出键 `last` 的最高不同位” 分到 65 个桶；当第 0 桶为空时，找到首个非空桶，以桶内最小键更新 `last`，再把整桶重新分配。更新后该最小键必进入第 0 桶，而其余元素进入编号更小的桶，所以每个元素至多被重新分配 64 次。这个前提正好适用于非负边权 Dijkstra 等松弛只会产生不小于当前距离的新键的算法；普通可减小键、负权或任意顺序优先队列不能使用本结构。

```
template <class T> class RadixHeap
{
    using Key = unsigned long long; // 键依赖 64 位无符号异或与位宽。
    // b[i] 保存与 last 的最高不同位落在第 i 桶的键值对。
    array<vector<pair<Key, T>>, 65> b;
    // last 是最近弹出的键，n 是当前元素数。
    Key last = 0;
    int n = 0;
```

```

    int id(Key x) const
    {
        // x 是不小于 last 的键；返回它当前所属的桶编号。
        return x == last ? 0 : bit_width(x ^ last);
    }

    void pull()
    {
        // 无参数；把首个非空高桶按其中最小键重新分桶，使 0 号桶非空。
        if (!b[0].empty())
        {
            return;
        }
        int p = 1;
        while (b[p].empty())
        {
            p++;
        }
        last = min_element(b[p].begin(), b[p].end(),
                           [](const auto &x, const auto &y)
                           {
                               return x.first < y.first;
                           })->first;
        for (auto x : b[p])
        {
            b[id(x.first)].push_back(move(x));
        }
        b[p].clear();
    }

  public:
    bool empty() const
    {
        // 无参数；返回堆是否为空。
        return n == 0;
    }

    int size() const
    {
        // 无参数；返回当前元素数。
        return n;
    }

    void push(Key k, const T &v)
    {
        // k 是单调键，v 是关联值；插入一项，要求 k 不小于最近弹出的键。
        assert(k >= last); // 调试检查，可删
        b[id(k)].push_back({k, v});
        n++;
    }

```

```
    const pair<Key, T> &top()
    {
        // 无参数；返回键最小的一项，要求堆非空。
        assert(n > 0); // 调试检查，可删
        pull();
        return b[0].back();
    }

    pair<Key, T> pop()
    {
        // 无参数；删除并返回键最小的一项，要求堆非空。
        auto ans = top();
        b[0].pop_back();
        n--;
        return ans;
    }
};
```

构造 `RadixHeap<T> q` 后，`push(k,v)` 插入键值对，`top()` 只读一个最小键元素，`pop()` 删除并返回一个最小键元素；同键元素之间没有稳定顺序。`top` 返回的引用会在下一次修改操作后失效。键必须是 $[0, 2^{64})$ 内的整数，并始终满足 `k>=last`；这里保留 `unsigned long long` 是因为分桶确实依赖精确的 64 位无符号异或和位宽。`push` 为 $O(1)$，每个元素一生最多移动 64 次，故 n 次插入、弹出共 $O(64n)$ 时间、$O(n+64)$ 空间；若把键宽记为 W，则为 $O(Wn)$。不满足单调键时使用标准 `priority_queue`；需要可并堆时使用 3.5.008 左偏树 / 可并堆。

下面检查重复键、跨高位键和单调弹出：

```
signed main()
{
    RadixHeap<int> q;
    q.push(7, 1);
    q.push(3, 2);
    q.push(3, 3);
    q.push(1ULL << 40, 4);
    assert(q.pop().first == 3);
    assert(q.pop().first == 3);
    q.push(8, 5);
    assert(q.pop().first == 7);
    assert(q.pop().first == 8);
    assert(q.pop().first == (1ULL << 40) && q.empty());
}
```

3.6 Trie 与集合结构

3.6.001 字符 Trie

字符 Trie 按前缀建树，公共前缀只存一次。根是空前缀，每条边代表一个字符，每个结点唯一对应一段前缀。两个计数是核心：`cnt[u]` 记经过 u 的插入次数（即以 u 对应前缀为前缀的串数），`end[u]` 记恰好在 u 结束的完整串数。于是 `prefixCnt` 读 `cnt`、`stringCnt` 读 `end`，`lcp` 沿边走到走不动为止。字符集大小

`A` 与最小字符 `S` 作模板参数，字符必须落在 $[S, S+A)$。它与 4.2.001 是同一知识分属数据结构与字符串两章的入口，两处都完整打印：

```cpp
template <int A = 26, char S = 'a'> struct Trie
{
    // ch[u][c] 是节点 u 沿字符 S+c 的儿子。
    vector<array<int, A>> ch;
    // cnt[u] 是经过 u 的插入次数，end[u] 是恰在 u 结束的字符串数。
    vector<int> cnt, end;

    Trie()
    {
        // 建立表示空前缀的根节点 0。
        newNode();
    }

    int newNode()
    {
        // 新建全零节点并返回编号。
        ch.push_back({});
        cnt.push_back(0);
        end.push_back(0);
        return (int)ch.size() - 1;
    }

    int insert(const string &s)
    {
        // s 是字符均位于 [S,S+A) 的字符串；插入一份并返回终点节点。
        int p = 0;
        cnt[p]++;
        for (char c : s)
        {
            int x = c - S;
            assert(0 <= x && x < A); // 调试检查，可删
            if (!ch[p][x])
            {
                ch[p][x] = newNode();
            }
            p = ch[p][x];
            cnt[p]++;
        }
        end[p]++;
        return p;
    }

    int find(const string &s) const
    {
        // s 是查询串；返回其前缀节点，不存在或字符越界时返回 -1。
        int p = 0;
        for (char c : s)
        {
            int x = c - S;
```

```
            if (x < 0 || x >= A || !ch[p][x])
            {
                return -1;
            }
            p = ch[p][x];
        }
        return p;
    }

    int lcp(const string &s) const
    {
        // s 是查询串；返回它与已插入字符串的最长公共前缀长度。
        int p = 0, n = 0;
        for (char c : s)
        {
            int x = c - S;
            if (x < 0 || x >= A || !ch[p][x])
            {
                break;
            }
            p = ch[p][x];
            n++;
        }
        return n;
    }

    int prefixCnt(const string &s) const
    {
        // s 是查询前缀；返回以 s 为前缀的已插入字符串数。
        int p = find(s);
        return p < 0 ? 0 : cnt[p];
    }
    int stringCnt(const string &s) const
    {
        // s 是完整查询串；返回它被插入的次数。
        int p = find(s);
        return p < 0 ? 0 : end[p];
    }
};
```

构造 `Trie<26,'a'> tr` 后 `tr.insert(s)` 插入并返回终点结点、`tr.find(s)` 返回终点结点（不存在返回 -1）、`tr.prefixCnt(s)` 返回以 s 为前缀的串数、`tr.stringCnt(s)` 返回 s 被完整插入的次数、`tr.lcp(s)` 返回 s 与库中任意串的最长公共前缀长度。例如依次插入 `ab`、`abc`、`ab` 后 `stringCnt("ab")==2`、`prefixCnt("ab")==3`、`lcp("abd")==2`、`find("ac")==-1`。各操作 $O(|s|)$，空间 $O(\text{总串长} \times A)$。字符集为 01 且要求异或最值时用 3.6.002 01 Trie / 压位 Trie；需要多模式串匹配则在此基础上加 fail 指针成为 3.6.004 Trie Graph；整数异或查询用 3.6.002 01 Trie，需要删除时启用 3.6.003 可删二进制 Trie，查询历史区间异或时改用 3.6.010 可持久化 01 Trie。本条与 4.2.001 Trie 都完整打印字符 Trie（同一知识分属两章、各有独立阅读价值）；只做键值映射时哈希表比 Trie 更直接，但最坏复杂度不稳定。

3.6.002 01 Trie / 压位 Trie

把整数按二进制位从高到低插入 Trie，两个儿子对应下一位取 0 或 1，每个节点额外记一个经过计数以支持重复值、第 k 小与阈值计数。求最大异或时贪心走与当前位相反的分支（若非空），求最小异或时贪心走相同分支；第 k 小异或沿当前位为 0 的分支计数决定向左还是向右；`countXorLess` 则在 k 的当前位为 1 时把与之同侧的整棵子树计数累加、再转向另一侧。位宽 W 由模板参数固定，本节不做删除、故计数只增：

```cpp
template <int W = 30> struct BinaryTrie
{
    // Node[0..2] 依次是 0 儿子、1 儿子、经过节点的元素个数。
    using Node = array<int, 3>;

    // tr[0] 是空节点，tr[1] 是根；节点只增不删。
    vector<Node> tr;

    BinaryTrie(int n = 0)
    {
        // n 是预计插入次数，只用于预留空间；W 是参与比较的二进制位数。
        static_assert(1 <= W && W <= 64);
        if (n)
        {
            tr.reserve(n * (W + 1) + 2);
        }
        clear();
    }

    int newNode()
    {
        // 新建全零节点并返回编号。
        tr.push_back({});
        return (int)tr.size() - 1;
    }

    bool valid(unsigned long long x) const
    {
        // x 是待操作整数；返回 x 是否能由 W 位无符号整数表示。
        if constexpr (W == 64)
        {
            return true;
        }
        else
        {
            return x < ((unsigned long long)1 << W);
        }
    }

    int size() const
    {
        // 返回 Trie 中元素总数，重复值重复计数。
        return tr[1][2];
```

```
    }

    bool empty() const
    {
        // 返回 Trie 是否为空。
        return size() == 0;
    }

    void clear()
    {
        // 清空全部元素并重建空节点和根节点。
        tr.clear();
        newNode();
        newNode();
    }

    int count(unsigned long long x) const
    {
        // x 是待查询整数；返回 x 的出现次数。
        assert(valid(x)); // 调试检查，可删
        int p = 1;
        for (int i = W - 1; i >= 0; i--)
        {
            p = tr[p][x >> i & 1];
            if (!p)
            {
                return 0;
            }
        }
        return tr[p][2];
    }

    void insert(unsigned long long x, int v = 1)
    {
        // x 是待插入整数，v 是一次插入的份数，必须为正。
        assert(valid(x) && v > 0); // 调试检查，可删
        int p = 1;
        tr[p][2] += v;
        for (int i = W - 1; i >= 0; i--)
        {
            int b = x >> i & 1;
            if (!tr[p][b])
            {
                tr[p][b] = newNode();
            }
            p = tr[p][b];
            tr[p][2] += v;
        }
    }

    unsigned long long maxXor(unsigned long long x) const
    {
```

```
        // x 是询问整数；返回集合中最大的 x xor y。
        assert(valid(x) && !empty()); // 调试检查，可删
        int p = 1;
        unsigned long long ans = 0;
        for (int i = W - 1; i >= 0; i--)
        {
            int b = x >> i & 1;
            int q = tr[p][b ^ 1];
            int c = q && tr[q][2] ? b ^ 1 : b;
            if (c != b)
            {
                ans |= (unsigned long long)1 << i;
            }
            p = tr[p][c];
        }
        return ans;
    }

    unsigned long long maxElement(unsigned long long x) const
    {
        // x 是询问整数；返回一个使 x xor y 最大的集合元素 y。
        return x ^ maxXor(x);
    }

    unsigned long long minXor(unsigned long long x) const
    {
        // x 是询问整数；返回集合中最小的 x xor y。
        assert(valid(x) && !empty()); // 调试检查，可删
        int p = 1;
        unsigned long long ans = 0;
        for (int i = W - 1; i >= 0; i--)
        {
            int b = x >> i & 1;
            int q = tr[p][b];
            int c = q && tr[q][2] ? b : b ^ 1;
            if (c != b)
            {
                ans |= (unsigned long long)1 << i;
            }
            p = tr[p][c];
        }
        return ans;
    }

    unsigned long long minElement(unsigned long long x) const
    {
        // x 是询问整数；返回一个使 x xor y 最小的集合元素 y。
        return x ^ minXor(x);
    }

    unsigned long long kthXor(unsigned long long x, int k) const
    {
```

```
        // x 是询问整数，k 从 0 开始；返回第 k 小的 x xor y。
        assert(valid(x) && 0 <= k && k < size()); // 调试检查，可删
        int p = 1;
        unsigned long long ans = 0;
        for (int i = W - 1; i >= 0; i--)
        {
            int b = x >> i & 1;
            int q = tr[p][b];
            int c = q ? tr[q][2] : 0;
            if (k < c)
            {
                p = q;
            }
            else
            {
                k -= c;
                p = tr[p][b ^ 1];
                ans |= (unsigned long long)1 << i;
            }
        }
        return ans;
    }

    unsigned long long kthElement(unsigned long long x, int k) const
    {
        // x 是询问整数，k 从 0 开始；返回使 x xor y 第 k 小的元素 y。
        return x ^ kthXor(x, k);
    }

    int countXorLess(unsigned long long x, unsigned long long k) const
    {
        // x、k 是询问参数；返回满足 (x xor y) < k 的元素个数。
        assert(valid(x)); // 调试检查，可删
        if constexpr (W < 64)
        {
            if (k >= ((unsigned long long)1 << W))
            {
                return size();
            }
        }
        int p = 1, ans = 0;
        for (int i = W - 1; i >= 0 && p; i--)
        {
            int a = x >> i & 1;
            int b = k >> i & 1;
            if (b)
            {
                int q = tr[p][a];
                if (q)
                {
                    ans += tr[q][2];
                }
```

```
                p = tr[p][a ^ 1];
            }
            else
            {
                p = tr[p][a];
            }
        }
        return ans;
    }
};
```

构造 `BinaryTrie<W> tr(n)`，W 是参与比较的二进制位数、n 仅用于 `reserve`；`insert(x,v)` 插入 v 份，`count`、`maxXor`、`maxElement`、`minXor`、`minElement`、`kthXor`（k 从 0 起）、`kthElement`、`countXorLess(x,k)` 查询。所有值须落在 $[0,2^W)$，$W=64$ 时允许全部 `unsigned long long`。例如 `BinaryTrie<4> tr` 依次插入 1、6、6 后 `count(6)==2`、`maxXor(8)==14`、`maxElement(8)==6`、`minXor(7)==1`、`kthXor(2,0)==3`、`kthXor(2,2)==4`、`countXorLess(2,4)==1`。每次操作 $O(W)$，空间 $O(nW)$（n 为插入总次数）。需要删除时启用 3.6.003 可删二进制 Trie（在此基础上加 `erase` 并令查询检查正计数）；查询历史版本或区间异或最值时改用 3.6.010 可持久化 01 Trie；仅做键值映射而非异或查询时 3.6.006 哈希表 更直接，但最坏复杂度不稳定。

下面用 64 位边界值固定检查重复计数、异或最值、第 k 小、对应原元素和阈值计数；另用 $W=4$ 检查阈值达到 2^W 时返回全部元素：

```
signed main()
{
    constexpr unsigned long long HI = 1ULL << 63;
    constexpr unsigned long long ALL = numeric_limits<unsigned long long>::max();
    BinaryTrie<64> tr;
    tr.insert(0);
    tr.insert(HI);
    tr.insert(ALL, 2);
    assert(tr.size() == 4 && tr.count(ALL) == 2);
    assert(tr.maxXor(0) == ALL && tr.maxElement(0) == ALL);
    assert(tr.minXor(0) == 0 && tr.minElement(0) == 0);
    assert(tr.kthXor(ALL, 0) == 0);
    assert(tr.kthXor(ALL, 2) == HI - 1);
    assert(tr.kthElement(ALL, 2) == HI);
    assert(tr.countXorLess(ALL, 1) == 2);
    assert(tr.countXorLess(ALL, HI) == 3);

    BinaryTrie<4> small;
    small.insert(15);
    assert(small.countXorLess(0, 16) == 1);
    cout << "4 2 0 9223372036854775807 3\n";
    return 0;
}
```

3.6.003 可删二进制 Trie

在 3.6.002 01 Trie 的每个节点上维护经过次数，`erase(x,v)` 沿根到叶把路径计数各减 v、但不回收节点，从而支持重复值与单值删除。关键坑点在于删除后计数归零的分支仍然物理存在，`maxXor`/`minXor` 因此都必须同时检查儿子编号非零和其经过计数为正（代码中 `q && tr[q][2]` 一并判断），否则会误走进已删空的历史分支。本节复用 3.6.002，只打印新增的派生层：

```cpp
// 依赖 3.6.002 01 Trie / 压位 Trie
template <int W = 30> struct EraseBinTrie : BinaryTrie<W>
{
    using BinaryTrie<W>::BinaryTrie;
    using BinaryTrie<W>::count;
    using BinaryTrie<W>::tr;
    using BinaryTrie<W>::valid;

    bool erase(unsigned long long x, int v = 1)
    {
        // x 是待删除整数，v 是删除份数；数量不足时不修改并返回 false。
        assert(valid(x) && v > 0); // 调试检查，可删
        if (count(x) < v)
        {
            return false;
        }
        int p = 1;
        tr[p][2] -= v;
        for (int i = W - 1; i >= 0; i--)
        {
            p = tr[p][x >> i & 1];
            tr[p][2] -= v;
        }
        return true;
    }
};
```

先放置 3.6.002 的 `BinaryTrie`，再构造 `EraseBinTrie<W> tr(n)`；`insert(x,v)` 插入 v 份，`erase(x,v)` 删除 v 份并在数量不足时不改动而返回 `false`。其余查询接口继承自普通版：`maxXor`、`minXor`、`kthXor` 返回异或值，`maxElement`、`minElement`、`kthElement` 返回取得该异或值的现存元素，`countXorLess` 做阈值计数。删除不回收节点，但计数归零的分支不会再参与查询。各操作 $O(W)$，空间 $O(\text{插入次数} \times W)$。不需要删除时用更精简的 3.6.002 01 Trie；需要查询历史版本则改用 3.6.010 可持久化 01 Trie。

下面沿用 64 位边界值，删除重复最大值后检查剩余分支、对应原元素和删除失败不改状态：

```cpp
signed main()
{
    constexpr unsigned long long HI = 1ULL << 63;
    constexpr unsigned long long ALL = numeric_limits<unsigned long long>::max();
    EraseBinTrie<64> tr;
    tr.insert(0);
    tr.insert(HI);
    tr.insert(ALL, 2);
    assert(tr.erase(ALL) && tr.count(ALL) == 1);
```

```
    assert(tr.maxXor(0) == ALL && tr.maxElement(0) == ALL);
    assert(tr.minXor(0) == 0 && tr.minElement(0) == 0);
    assert(tr.kthXor(ALL, 1) == HI - 1);
    assert(tr.kthElement(ALL, 1) == HI);
    assert(tr.erase(ALL) && !tr.erase(ALL));
    assert(tr.size() == 2 && tr.maxXor(0) == HI);
    assert(tr.countXorLess(ALL, HI) == 1);
    cout << "1 9223372036854775808 1\n";
    return 0;
}
```

3.6.004 Trie Graph

Trie 只负责把一组字符串的公共前缀压成状态节点，题目给出的跳转、代价或约束再作为普通有向边挂到这些状态上，随后按边权性质跑 BFS、Dijkstra 或 DP。要点是把「压前缀」与「图上转移」分开看待，而不是把 AC 自动机的 fail 转移当成通用 Trie Graph——这里的 `shortest` 用 Dijkstra，故要求边权非负：

```
template <int A = 26, char B = 'a', class Dist = long long> struct TrieGraph
{
    struct Node
    {
        // ch 保存 Trie 字符转移；e 保存从当前状态出发的 {目标状态, 非负边权}。
        array<int, A> ch{};
        vector<pair<int, Dist>> e;
    };

    // tr 保存所有 Trie 状态及额外图边。
    vector<Node> tr{{}};

    int insert(const string &s)
    {
        // s 表示待插入字符串；返回其终止状态编号。
        int p = 0;
        for (char c : s)
        {
            int x = c - B;
            assert(0 <= x && x < A); // 调试检查，可删
            if (!tr[p].ch[x])
            {
                tr[p].ch[x] = tr.size();
                tr.push_back({});
            }
            p = tr[p].ch[x];
        }
        return p;
    }

    void addEdge(int u, int v, Dist w, bool directed = true)
    {
        // u、v 表示状态编号，w 表示非负边权；directed 为 false 时同时加入反向边。
```

```
        assert(0 <= u && u < (int)tr.size() && 0 <= v && v < (int)tr.size() && w >= 0); // 调试检查，可删
        tr[u].e.push_back({v, w});
        if (!directed)
        {
            tr[v].e.push_back({u, w});
        }
    }

    vector<optional<Dist>> shortest(int s) const
    {
        // s 是源状态；返回到各状态的 Dist 距离，不可达位置为空。
        assert(0 <= s && s < (int)tr.size()); // 调试检查，可删
        vector<optional<Dist>> d(tr.size());
        priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> q;
        d[s] = 0;
        q.push({0, s});
        while (!q.empty())
        {
            auto [du, u] = q.top();
            q.pop();
            if (!d[u] || du != *d[u])
            {
                continue;
            }
            for (auto [v, w] : tr[u].e)
            {
                Dist nd = du + w;
                if (!d[v] || nd < *d[v])
                {
                    d[v] = nd;
                    q.push({nd, v});
                }
            }
        }
        return d;
    }
};
```

构造 `TrieGraph<> g`，用 `insert(s)` 得到字符串终止状态编号，再以 `addEdge(u,v,w,directed)` 加题目额外转移（`directed=false` 同时加反向边）。边权非负时，`shortest(s)` 默认返回 `vector<optional<long long>>`；需要宽距离时构造 `TrieGraph<A, B, __int128>`。有值表示可达并保存最短距离，`nullopt` 表示不可达；调用者须保证最短路长度能由 `Dist` 表示。Trie 的父子字符边是否加入、方向与权值均由题意决定，本结构不自动连它们。例如插入 `ab`、`ac` 得终点 u、v，加根到 u 权 5、u 到 v 权 2、根到 v 权 10，则 `*d[u]==5`、`*d[v]==7`。插入长度 L 的串为 $O(L)$，`shortest` 为 $O((S+E)\log S)$，空间 $O(AS+E)$，其中 S、E 是状态数与额外边数。需要多模式串在文本上匹配的 fail 自动机见 4.2.002 AC 自动机；纯字符字典查询见 3.6.001 字符 Trie。

3.6.005 线段树优化 Trie Graph

当一个状态需要向整段连续编号的目标批量连边时，直接连会退化成稠密图；线段树优化建图把编号区间建成两棵线段树 DAG（一棵「向下」用于点到区间、一棵「向上」用于区间到点），每条覆盖 $[l, r)$ 的批量转移只落在 $O(\log n)$ 个区间节点上，再统一跑最短路。`pointRange`/`rangePoint`/`rangeRange` 分别对应三种批量连边，附带的 `dualShortest` 是平面图对偶最短路的独立小工具。这份底层与图论章的线段树优化建图 5.2.006 同源：

```
template <class Dist = long long> struct RangeGraph
{
    // n 是原图点数，sz 是补成二次幂后的线段树叶子数。
    int n, sz;
    // g 保存两棵线段树节点、中转点及其非负权边。
    vector<vector<pair<int, Dist>>> g;

    RangeGraph(int n) : n(n), sz(bit_ceil((unsigned)(max<int>(1, n)))), g(4 * sz)
    {
        // n 是原图点数；建立向下树、向上树及对应叶子的零权连接。
        assert(n >= 0); // 调试检查，可删
        for (int i = 1; i < sz; i++)
        {
            g[i].push_back({i * 2, 0});
            g[i].push_back({i * 2 + 1, 0});
            g[2 * sz + i * 2].push_back({2 * sz + i, 0});
            g[2 * sz + i * 2 + 1].push_back({2 * sz + i, 0});
        }
        for (int i = 0; i < sz; i++)
        {
            g[sz + i].push_back({3 * sz + i, 0});
            g[3 * sz + i].push_back({sz + i, 0});
        }
    }

    vector<int> cover(int l, int r, int off) const
    {
        // l、r 是原点半开区间，off 是 0 或 2*sz；返回其线段树分解节点。
        assert(0 <= l && l <= r && r <= n && (off == 0 || off == 2 * sz)); // 调试检查，可删
        vector<int> q;
        for (l += sz, r += sz; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1)
            {
                q.push_back(off + l++);
            }
            if (r & 1)
            {
                q.push_back(off + --r);
            }
        }
        return q;
    }
```

```cpp

    void pointRange(int u, int l, int r, Dist w)
    {
        // u 是起点，[l,r) 是终点区间，w 是非负边权。
        assert(0 <= u && u < n && w >= 0); // 调试检查，可删
        for (int v : cover(l, r, 0))
        {
            g[sz + u].push_back({v, w});
        }
    }

    void rangePoint(int l, int r, int v, Dist w)
    {
        // [l,r) 是起点区间，v 是终点，w 是非负边权。
        assert(0 <= v && v < n && w >= 0); // 调试检查，可删
        for (int u : cover(l, r, 2 * sz))
        {
            g[u].push_back({sz + v, w});
        }
    }

    void rangeRange(int l, int r, int x, int y, Dist w)
    {
        // [l,r) 是起点区间，[x,y) 是终点区间，w 是非负边权。
        assert(0 <= l && l <= r && r <= n && 0 <= x && x <= y && y <= n && w >= 0); // 调试检查，
        ↪ 可删
        if (l == r || x == y)
        {
            return;
        }
        int p = g.size();
        g.push_back({});
        for (int u : cover(l, r, 2 * sz))
        {
            g[u].push_back({p, 0});
        }
        for (int v : cover(x, y, 0))
        {
            g[p].push_back({v, w});
        }
    }

    vector<optional<Dist>> dijkstra(int s) const
    {
        // s 是原图源点；返回到各原点的 Dist 距离，不可达位置为空。
        assert(0 <= s && s < n); // 调试检查，可删
        vector<optional<Dist>> d(g.size());
        priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> q;
        d[sz + s] = 0;
        q.push({0, sz + s});
        while (!q.empty())
        {
```

```
            auto [du, u] = q.top();
            q.pop();
            if (!d[u] || du ≠ *d[u])
            {
                continue;
            }
            for (auto [v, w] : g[u])
            {
                Dist nd = du + w;
                if (!d[v] || nd < *d[v])
                {
                    d[v] = nd;
                    q.push({nd, v});
                }
            }
        }
        vector<optional<Dist>> ans(n);
        for (int i = 0; i < n; i++)
        {
            ans[i] = d[sz + i];
        }
        return ans;
    }
};

template <class Dist = long long>
vector<optional<Dist>> dualShortest(int faces, int s, const vector<array<long long, 3>> &edge)
{
    // faces 是面数，s 是源面；edge 每项为 {左面，右面，非负边权}，负面编号表示外部而忽略。
    assert(0 < faces && 0 <= s && s < faces); // 调试检查，可删
    vector<vector<pair<int, Dist>>> g(faces);
    for (auto [l, r, w] : edge)
    {
        if (l >= 0 && r >= 0)
        {
            assert(l < faces && r < faces && w >= 0); // 调试检查，可删
            g[l].push_back({(int)r, (Dist)w});
            g[r].push_back({(int)l, (Dist)w});
        }
    }
    vector<optional<Dist>> d(faces);
    priority_queue<pair<Dist, int>, vector<pair<Dist, int>>, greater<>> q;
    d[s] = 0;
    q.push({0, s});
    while (!q.empty())
    {
        auto [du, u] = q.top();
        q.pop();
        if (!d[u] || du ≠ *d[u])
        {
            continue;
        }
```

```
        for (auto [v, w] : g[u])
        {
            Dist nd = du + w;
            if (!d[v] || nd < *d[v])
            {
                d[v] = nd;
                q.push({nd, v});
            }
        }
    }
    return d;
}
```

构造 `RangeGraph<> g(n)`（原点编号 $0,n$）， `pointRange(u,l,r,w)` 从点 u 向区间 $[l,r)$ 连权 w 边，`rangePoint(l,r,v,w)` 反向，`rangeRange(l,r,x,y,w)` 从区间连向区间；空起点或终点区间不加边。所有边权须非负。默认 `Dist=long long`，需要宽距离时构造 `RangeGraph<__int128> g(n)` 或调用 `dualShortest<__int128>`；两者都用 `optional` 表示不可达，调用者须保证最短路长度能由 `Dist` 表示。n 个原点、q 次区间加边后图规模 $O(n+q\log n)$，每次加边 $O(\log n)$，最短路为 $O((n+q\log n)\log(n+q))$；对偶建图 $O(m)$、最短路 $O((f+m)\log f)$。批量连边模型与图论章的 [5.2.006 线段树优化建图同源，只是这里放在 Trie 状态图语境下使用。

3.6.006 哈希表

键到值的映射用 `unordered_map`，但整数键不能保留默认哈希：它近似恒等，存在针对桶号的构造性攻击。这里直接复用 1.4.004 `unordered_map` / `unordered_set` 防 hack 哈希 已定义的 `CustomHash`，不再复制 `splitmix64`；插入前先设装载因子并预留容量，避免 $Q=10^6$ 时反复重哈希。若题目能事先确定不同键数上界且不删除键，可用定长开放寻址版：

```
// 依赖 1.4.004 unordered_map / unordered_set 防 hack 哈希
template <class K, class V> using HashMap = unordered_map<K, V, CustomHash>;
template <class K> using HashSet = unordered_set<K, CustomHash>;

template <class K, class V, int N, class H = CustomHash> class FixedHashMap
{
    static_assert(1 <= N && N <= 30);
    static constexpr int CAP = 1LL << N; // CAP 是槽数 2^N，须严格大于不同键数。
    static constexpr int MASK = CAP - 1; // MASK 用于线性探测时回绕下标。

    int cnt = 0; // cnt 是当前不同键的个数。
    unique_ptr<K[]> key{new K[CAP]}; // key[p] 是槽 p 中的键。
    unique_ptr<V[]> val{new V[CAP]}; // val[p] 是槽 p 中键对应的值。
    vector<unsigned long long> used = vector<unsigned long long>((CAP + 63) / 64); // used 的第 p 位表示槽 p 已占用。
    H hs; // hs 是哈希函数对象。

    bool test(int p) const { return used[p >> 6] >> (p & 63) & 1ULL; }
    void set(int p) { used[p >> 6] |= 1ULL << (p & 63); }

    int slot(const K &x) const
```

```
    {
        // x 是键；返回 Fibonacci 散列后的初始槽位，取乘积的高 N 位。
        unsigned long long h = (unsigned long long)hs(x);
        h *= 11400714819323198485ULL;
        return h >> (64 - N);
    }

    int findPos(const K &x) const
    {
        // x 是键；线性探测返回 x 所在槽，不存在时返回第一个空槽。
        int p = slot(x);
        while (test(p) && key[p] != x)
        {
            p = (p + 1) & MASK;
        }
        return p;
    }

  public:
    V &operator[](const K &x)
    {
        // x 是键；不存在时插入值为 V{} 的新键，返回值的引用。
        assert(cnt < CAP); // 调试检查，可删；容量必须严格大于不同键数。
        int p = findPos(x);
        if (!test(p))
        {
            set(p);
            key[p] = x;
            val[p] = V{};
            cnt++;
        }
        return val[p];
    }

    V *find(const K &x)
    {
        // x 是键；返回值的指针，不存在时返回 nullptr。
        int p = findPos(x);
        return test(p) ? &val[p] : nullptr;
    }

    const V *find(const K &x) const
    {
        // x 是键；返回值的只读指针，不存在时返回 nullptr。
        int p = findPos(x);
        return test(p) ? &val[p] : nullptr;
    }

    bool contains(const K &x) const { return find(x) != nullptr; }

    V get(const K &x) const
    {
```

```
        // x 是键；返回对应值，不存在时返回 V{} 且不插入。
        auto p = find(x);
        return p ? *p : V{};
    }

    int size() const { return cnt; }

    void clear()
    {
        // 无参数；清空全部键，键值数组不重置，只清占用标记。
        fill(used.begin(), used.end(), 0);
        cnt = 0;
    }
};

template <class K, int N, class H = CustomHash> class FixedHashSet
{
    static_assert(1 <= N && N <= 30);
    static constexpr int CAP = 1LL << N; // CAP 是槽数 2^N，须严格大于元素数。
    static constexpr int MASK = CAP - 1; // MASK 用于线性探测时回绕下标。

    // cnt 是当前元素数，key 保存槽内键，used 标记槽是否占用。
    int cnt = 0;
    unique_ptr<K[]> key{new K[CAP]};
    vector<unsigned long long> used = vector<unsigned long long>((CAP + 63) / 64);
    H hs;

    bool test(int p) const
    {
        // p 是槽编号；返回该槽是否已被占用。
        return used[p >> 6] >> (p & 63) & 1ULL;
    }

    void set(int p)
    {
        // p 是空槽编号；把该槽标记为占用。
        used[p >> 6] |= 1ULL << (p & 63);
    }

    int slot(const K &x) const
    {
        // x 是待查键；返回经过随机哈希与乘法混合后的初始槽。
        unsigned long long h = (unsigned long long)hs(x);
        h *= 11400714819323198485ULL;
        return h >> (64 - N);
    }

    int findPos(const K &x) const
    {
        // x 是待查键；返回保存 x 的槽，或其探测链上的第一个空槽。
        int p = slot(x);
        while (test(p) && key[p] != x)
```

```
        {
            p = (p + 1) & MASK;
        }
        return p;
    }

  public:
    bool insert(const K &x)
    {
        // x 是待插入键；首次插入返回 true，已经存在返回 false。
        assert(cnt < CAP); // 调试检查，可删；容量必须严格大于不同键数。
        int p = findPos(x);
        if (test(p))
        {
            return false;
        }
        set(p);
        key[p] = x;
        cnt++;
        return true;
    }

    bool contains(const K &x) const
    {
        // x 是待查询键；返回集合是否包含 x。
        return test(findPos(x));
    }

    int size() const
    {
        // 返回当前不同键的数量。
        return cnt;
    }

    void clear()
    {
        // 清空全部键，保留已经分配的固定容量。
        fill(used.begin(), used.end(), 0);
        cnt = 0;
    }
};
```

以整数键为例，构造 `HashMap<int,int> mp;` 后先执行 `mp.max_load_factor(.7); mp.reserve(2 * q);`，再用 `mp[x]++` 计数或用标准迭代器 `find`；只保存键时使用 `HashSet<int>`。已知不同键数小于 2^N 时可构造 `FixedHashMap<int,int,N> mp`，用 `operator[]`、返回指针的 `find`、`contains`、缺失时返回默认值的 `get`、`size` 和 `clear`；只需判重或成员查询时改用 `FixedHashSet<int,N> st`，其 `insert(x)` 返回是否首次插入，并提供 `contains`、`size` 和 `clear`。Set 不分配值数组：例如 $N = 21$ 且键为 64 位整数时，键和占用位图约需 `16.25MiB`，同规格 64 位键值 Map 约需 `32.25MiB`。

两种定长结构都建议装载率不超过 0.7，实际不同键数必须严格小于容量，否则探测无法终止；均不支持删除、扩容和迭代。不能通过直接清除占用位实现删除，因为那会截断后续键的线性探测链；需要删除时

改用通用容器或另写墓碑/后移删除版本。各版本默认都用随机种子的 `CustomHash`，不接受确定性乘法哈希的构造性碰撞；字符串或其他复合键须提供对应哈希器。操作期望 $O(1)$、最坏仍为 $O(n)$，通用容器空间期望 $O(n)$、定长版空间 $O(2^N)$。

在 Ryzen 9 7945HX、MinGW GCC 14.2、`-O2 -DNDEBUG` 下，以 2^{20} 个槽执行一百万次固定随机混合操作（约三分之二插入，最终装载率约 0.64），预热两轮后的 7 轮中位数约为 `12.2ms`；同样使用 `CustomHash`、装载因子 0.7 且预留一百万项的 `unordered_set` 约为 `122.0ms`。计时不包含容器构造和 `reserve`，每轮都核对查询校验和与最终大小；这是特定机器与分布下的热点结果。Map 在此前同机一百万次混合读写中约为 `15ms`，同样防卡的 `unordered_map` 约为 `70ms`。要求稳定最坏复杂度时改用 `map` / `set`；需要删除或未知容量时仍用 `HashMap` / `HashSet` 或 3.9.011 cc_hash_table 与 gp_hash_table。若键是字符串前缀且需前缀/整串计数，用 3.6.001 字符 Trie 更稳定。

3.6.007 可持久化数组

把数组用一棵线段树表示、叶子存单个下标的值，则单点赋值只需复制从根到该叶的一条链（$O(\log n)$ 个新节点），其余节点与旧版本共享，于是既能查询任意历史版本的某位置、又能从任意旧版本继续修改。这是可持久化数据结构的最基本形态，也是可持久化并查集等结构的底座：

```
template <class T> class PerArray
{
    struct Node
    {
        // left、right 是儿子编号，value 仅在叶子保存数组值。
        int left = 0, right = 0;
        T value{};
    };

    // n 是数组长度，nodes 是节点池，roots[v] 是版本 v 的根。
    int n;
    vector<Node> nodes{{}};
    vector<int> roots;

    int build(const vector<T> &a, int l, int r)
    {
        // a 是初始数组，[l,r) 是当前区间；返回建成的节点编号。
        int p = nodes.size();
        nodes.push_back({});
        if (r - l == 1)
        {
            nodes[p].value = a[l];
            return p;
        }
        int m = midpoint(l, r);
        nodes[p].left = build(a, l, m);
        nodes[p].right = build(a, m, r);
        return p;
    }

    int set(int p, int l, int r, int x, const T &value)
```

```
32      {
33          // p 是旧节点，[l,r) 是其区间；把位置 x 改为 value 并返回新节点。
34          int q = nodes.size();
35          nodes.push_back(nodes[p]);
36          if (r - l == 1)
37          {
38              nodes[q].value = value;
39              return q;
40          }
41          int m = midpoint(l, r);
42          if (x < m)
43          {
44              nodes[q].left = set(nodes[p].left, l, m, x, value);
45          }
46          else
47          {
48              nodes[q].right = set(nodes[p].right, m, r, x, value);
49          }
50          return q;
51      }
52
53    public:
54      explicit PerArray(const vector<T> &a) : n(a.size())
55      {
56          // a 是初始数组；建立版本 0，空数组的根为 0。
57          roots.push_back(n ? build(a, 0, n) : 0);
58      }
59
60      int set(int version, int position, const T &value)
61      {
62          // version 是旧版本，position 是位置，value 是新值；返回新增版本编号。
63          assert(0 <= version && version < (int)roots.size()); // 调试检查，可删
64          assert(0 <= position && position < n); // 调试检查，可删
65          roots.push_back(set(roots[version], 0, n, position, value));
66          return roots.size() - 1;
67      }
68
69      T get(int version, int position) const
70      {
71          // version 是历史版本，position 是位置；按值返回该位置的内容。
72          assert(0 <= version && version < (int)roots.size()); // 调试检查，可删
73          assert(0 <= position && position < n); // 调试检查，可删
74          int p = roots[version], l = 0, r = n;
75          while (r - l > 1)
76          {
77              int m = midpoint(l, r);
78              if (position < m)
79              {
80                  p = nodes[p].left;
81                  r = m;
82              }
83              else
```

```
            {
                p = nodes[p].right;
                l = m;
            }
        }
        return nodes[p].value;
    }
};
```

用初始序列构造 `PerArray<T> pa(a)` 并生成版本 0；空数组也有版本 0，但没有合法位置可改查。`set(version,position,value)` 基于旧版本生成新版本并返回编号，`get(version,position)` 按值返回某版本的位置内容，因此随后扩容节点池也不会使查询结果失效。初始构造 $O(n)$，每次 `set` 为 $O(\log n)$ 时间与新增空间，`get` 为 $O(\log n)$。需要按版本回答区间第 k 小用 3.3.008 可持久化线段树 / 主席树；需要可持久化的整数异或查询用 3.6.010 可持久化 01 Trie；把它套上按大小合并即得 3.1.005 可持久化并查集。

3.6.008 手写 bitset

标准 `bitset` 的长度必须编译期确定，运行期才知长度时改用这份把位装进 64 位字数组的手写版：按位与、或、异或逐字执行；左右移要同时处理整字偏移 `shift/64` 与字内偏移 `shift%64`，并把跨字进位用相邻字的补偿位拼上。唯一容易写错的地方是每次可能改动最高字的操作后都要 `trim()`，把最后一个字里超出逻辑长度 `n` 的高位清零，否则 `count`、移位链会把垃圾位算进去：

```
class BitVec
{
    // n 是逻辑位数，words 按低位到高位保存 64 位块。
    int n = 0;
    vector<unsigned long long> words;

    void trim()
    {
        // 清除最高字中超出逻辑长度的位。
        if (n % 64)
        {
            words.back() &= (1ULL << (n % 64)) - 1;
        }
    }

  public:
    explicit BitVec(int n = 0) : n(n)
    {
        // n 是逻辑位数；初值全为零。
        assert(n >= 0); // 调试检查，可删
        words.resize(n / 64 + (n % 64 != 0));
    }

    bool test(int x) const
    {
        // x 是位下标；返回该位是否为一。
```

```
        assert(0 <= x && x < n); // 调试检查，可删
        return words[x / 64] >> (x % 64) & 1;
    }

    void set(int x, bool value = true)
    {
        // x 是位下标，value 是新位值。
        assert(0 <= x && x < n); // 调试检查，可删
        if (value)
        {
            words[x / 64] |= 1ULL << (x % 64);
        }
        else
        {
            reset(x);
        }
    }

    void reset(int x)
    {
        // x 是位下标；把该位清零。
        assert(0 <= x && x < n); // 调试检查，可删
        words[x / 64] &= ~(1ULL << (x % 64));
    }

    void setAll()
    {
        // 把全部有效位设为一。
        ranges::fill(words, ~0ULL);
        if (!words.empty())
        {
            trim();
        }
    }

    void resetAll()
    {
        // 把全部位清零。
        ranges::fill(words, 0);
    }

    int count() const
    {
        // 返回一的个数。
        int result = 0;
        for (auto word : words)
        {
            result += popcount(word);
        }
        return result;
    }

```

```
    BitVec &operator|=(const BitVec &other)
    {
        // other 是等长位向量；逐位或后返回自身。
        assert(n == other.n); // 调试检查，可删
    for (int i = 0; i < (int)words.size(); ++i)
        {
            words[i] |= other.words[i];
        }
        return *this;
    }

    BitVec &operator&=(const BitVec &other)
    {
        // other 是等长位向量；逐位与后返回自身。
        assert(n == other.n); // 调试检查，可删
    for (int i = 0; i < (int)words.size(); ++i)
        {
            words[i] &= other.words[i];
        }
        return *this;
    }

    BitVec &operator^=(const BitVec &other)
    {
        // other 是等长位向量；逐位异或后返回自身。
        assert(n == other.n); // 调试检查，可删
    for (int i = 0; i < (int)words.size(); ++i)
        {
            words[i] ^= other.words[i];
        }
        return *this;
    }

    BitVec &operator<<=(int shift)
    {
        // shift 是非负左移量；越界位丢弃，返回自身。
        assert(shift >= 0); // 调试检查，可删
        if (shift >= n)
        {
            resetAll();
            return *this;
        }
        int block = shift / 64, offset = shift % 64;
    for (int i = (int)words.size() - 1; i >= 0; --i)
        {
            unsigned long long value = i >= block ? words[i - block] << offset : 0;
            if (offset && i > block)
            {
                value |= words[i - block - 1] >> (64 - offset);
            }
            words[i] = value;
        }
```

```
        trim();
        return *this;
    }

    BitVec &operator>>=(int shift)
    {
        // shift 是非负右移量；越界位丢弃，返回自身。
        assert(shift >= 0); // 调试检查，可删
        if (shift >= n)
        {
            resetAll();
            return *this;
        }
        int block = shift / 64, offset = shift % 64;
    for (int i = 0; i < (int)words.size(); ++i)
        {
        unsigned long long value = i + block < (int)words.size() ? words[i + block] >> offset :
        ↪ 0;
        if (offset && i + block + 1 < (int)words.size())
            {
                value |= words[i + block + 1] << (64 - offset);
            }
            words[i] = value;
        }
        return *this;
    }
};
```

构造 `BitVec v(n)` 得长度 n 的全零位向量，`test`/`set`/`reset` 单点读写，`setAll`/`resetAll` 整表置位清零，`count` 返回一的个数，`|= & ^=` 要求两侧等长，`<<= >>=` 按位移量整体移动。单点操作 $O(1)$，`count`、整表位运算与移位 $O(n/64)$，空间 $O(n/64)$。长度编译期已知时直接用标准 `bitset` 常数更小；只需按块判 01 异或最值用 3.6.002 01 Trie。

下面固定检查第 $63/64$ 位交界、整字移动、最高字裁剪、按位运算、全置位与空位向量：

```
signed main()
{
    BitVec b(70);
    for (int x : {0, 63, 64, 69})
    {
        b.set(x);
    }
    assert(b.count() == 4);

    BitVec l = b;
    l <<= 1;
    assert(l.count() == 3 && l.test(1) && l.test(64) && l.test(65));
    BitVec r = b;
    r >>= 1;
    assert(r.count() == 3 && r.test(62) && r.test(63) && r.test(68));
    BitVec x = b;
```

```
    x <<= 64;
    assert(x.count() == 1 && x.test(64));
    BitVec y = b;
    y >>= 64;
    assert(y.count() == 2 && y.test(0) && y.test(5));
    x = b;
    x <<= 70;
    assert(x.count() == 0);

    BitVec p(70), q(70);
    p.set(63);
    p.set(64);
    q.set(64);
    q.set(69);
    BitVec z = p;
    z |= q;
    assert(z.count() == 3);
    z = p;
    z &= q;
    assert(z.count() == 1 && z.test(64));
    z = p;
    z ^= q;
    assert(z.count() == 2 && z.test(63) && z.test(69));

    BitVec all(70);
    all.setAll();
    assert(all.count() == 70 && all.test(69));
    all <<= 70;
    assert(all.count() == 0);
    BitVec empty(0);
    empty >>= 0;
    assert(empty.count() == 0);
    cout << "OK\n";
}
```

3.6.009 动态区间频率索引

要在带单点修改的数组上查「值 v 在下标区间 $[l, r)$ 出现几次」，做法是离线先扫出每个位置将来可能取到的所有值，为每个值单独建一棵「以该值出现过的位置为下标」的树状数组；区间频次就是该值树状数组上两次前缀和之差。因为位置被离散化进各值自己的坐标系，`add` 里先 `lower_bound` 找到该位置在此值坐标下的秩再更新。修改一个位置就是在旧值树状数组减一、新值树状数组加一：

```
class Freq
{
    // a 是当前数组，id 把值映射到内部编号。
    vector<int> a;
    unordered_map<int, int, CustomHash> id;
    // pos[k] 是值 k 预登记过的位置，bit[k] 是对应树状数组。
    vector<vector<int>> pos, bit;

```

```
    void add(int v, int p, int d)
    {
        // v 是值，p 是预登记位置，d 是出现次数增量。
        auto it = id.find(v);
        assert(it != id.end()); // 调试检查，可删
        int k = it->second;
        auto jt = lower_bound(pos[k].begin(), pos[k].end(), p);
        assert(jt != pos[k].end() && *jt == p); // 调试检查，可删
        int x = jt - pos[k].begin() + 1;
        for (int i = x; i < (int)bit[k].size(); i += i & -i)
        {
            bit[k][i] += d;
        }
    }

    int pre(int k, int p) const
    {
        // k 是内部值编号，p 是位置上界；返回该值在 [0,p) 的出现次数。
        int x = lower_bound(pos[k].begin(), pos[k].end(), p) - pos[k].begin();
        int ans = 0;
        for (int i = x; i > 0; i -= i & -i)
        {
            ans += bit[k][i];
        }
        return ans;
    }

public:
    Freq(vector<int> init, const vector<pair<int, int>> &qs) : a(move(init))
    {
        // init 是初始数组，qs 是以后可能写入的全部 {位置, 值}。
        vector<pair<int, int>> all = qs;
        all.reserve(all.size() + a.size());
        for (int p = 0; p < (int)a.size(); p++)
        {
            all.push_back({p, a[p]});
        }
        id.max_load_factor(.7);
        id.reserve(2 * all.size() + 1);
        for (auto [p, v] : all)
        {
            assert(0 <= p && p < (int)a.size()); // 调试检查，可删
            auto [it, ok] = id.emplace(v, id.size());
            if (ok)
            {
                pos.emplace_back();
            }
            pos[it->second].push_back(p);
        }
        bit.resize(pos.size());
        for (int k = 0; k < (int)pos.size(); k++)
        {
```

```
            auto &p = pos[k];
            sort(p.begin(), p.end());
            p.erase(unique(p.begin(), p.end()), p.end());
            bit[k].assign(p.size() + 1, 0);
        }
        for (int p = 0; p < (int)a.size(); p++)
        {
            add(a[p], p, 1);
        }
    }

    void setValue(int p, int v)
    {
        // p 是位置，v 是新值；要求 {p,v} 已在构造时预登记。
        assert(0 <= p && p < (int)a.size()); // 调试检查，可删
        auto it = id.find(v);
        assert(it != id.end() && binary_search(pos[it->second].begin(), pos[it->second].end(),
        ↪  p)); // 调试检查，可删
        add(a[p], p, -1);
        a[p] = v;
        add(v, p, 1);
    }

    int frequency(int l, int r, int v) const
    {
        // l、r 是半开区间端点，v 是查询值；返回 v 在 [l,r) 的出现次数。
        assert(0 <= l && l <= r && r <= (int)a.size()); // 调试检查，可删
        auto it = id.find(v);
        if (it == id.end())
        {
            return 0;
        }
        int k = it->second;
        return pre(k, r) - pre(k, l);
    }
};
```

构造 `Freq f(initial, qs)`：`initial` 是初始数组、`qs` 列出所有将来会被 `setValue` 写入的 `{位置, 值}`（离线预登记，保证每个值都有坐标）；`setValue(position,value)` 单点改值，`frequency(left,right,value)` 查半开区间 $[left, right)$ 内 `value` 的出现次数（未登记的值返回 0）。这里显式依赖 1.4.004 `unordered_map` / `unordered_set` 防 hack 哈希 的 `CustomHash`，并在建表前设置装载因子与桶容量；在该哈希的通常假设下，预处理为 $O((n+q)\log(n+q))$，`setValue` 与 `frequency` 各期望 $O(\log(n+q))$，最坏哈希冲突时可退化，空间 $O(n+q)$。若查询的是区间第 k 小/排名而非指定值频次，用带修主席树或树状数组套线段树；离线且无修改时莫队更简单。

3.6.010 可持久化 01 Trie

给数组每个前缀存一棵 01 Trie 版本，`rt[i]` 是前 i 个数构成的 Trie 根；每次 `append` 只复制从最高位到最低位的一条链，其余节点与上一版共享。查询下标区间 $[l, r)$ 时用版本 r 与版本 l 两棵 Trie 对应位置的

计数差（count）判断某个方向的子树在该区间内是否有元素，从高位起贪心走能令答案该位为 1 的儿子，即得区间与 x 的最大异或：

```
struct PerBitTrie
{
    // Node[0..2] 依次是 0 儿子编号、1 儿子编号、经过该节点的版本区间元素数。
    using Node = array<int, 3>;
    int bit;          // bit 是处理的最高二进制位，默认 62 覆盖非负 long long。
    vector<Node> tr; // tr 保存所有版本共享的节点，0 号节点为空节点。
    vector<int> rt;  // rt[i] 是前 i 个数构成的 Trie 根，rt[0] 为空版本。

    PerBitTrie(int bit = 62) : bit(bit), tr(1, Node{}), rt(1, 0)
    {
        // bit 是最高处理位；构造只含空版本的持久化 Trie。
        assert(0 <= bit && bit <= 62); // 调试检查，可删
    }

    int clone(int p)
    {
        // p 是旧节点编号；复制该节点并返回新节点编号。
        tr.push_back(tr[p]);
        return tr.size() - 1;
    }

    void append(int x)
    {
        // x 是能由 bit+1 位表示的非负整数；追加后生成新前缀版本。
        assert(0 <= x && (bit == 62 || (unsigned long long)x < (1ULL << (bit + 1)))); // 调试检
        ↪ 查，可删
        int old = rt.back();
        int cur = clone(old);
        rt.push_back(cur);
        tr[cur][2]++;
        for (int b = bit; b >= 0; b--)
        {
            int c = x >> b & 1;
            int nxt = clone(tr[old][c]);
            tr[cur][c] = nxt;
            old = tr[old][c];
            cur = nxt;
            tr[cur][2]++;
        }
    }

    int count(int l, int r, int c) const
    {
        // l、r 是两个版本节点，c 是儿子方向；返回版本差中该儿子的元素数。
        return tr[tr[r][c]][2] - tr[tr[l][c]][2];
    }

    int maxXor(int l, int r, int x) const
    {
```

```
        // [l,r) 是下标区间，x 是询问值；返回最大异或值，空区间返回 -1。
        assert(0 <= l && l <= r && r < (int)rt.size()); // 调试检查，可删
        assert(0 <= x && (bit == 62 || (unsigned long long)x < (1ULL << (bit + 1)))); // 调试检查，可删
        if (l == r)
        {
            return -1;
        }
        int u = rt[l];
        int v = rt[r];
        int ans = 0;
        for (int b = bit; b >= 0; b--)
        {
            int c = x >> b & 1;
            int go = c ^ 1;
            if (count(u, v, go) == 0)
            {
                go ^= 1;
            }
            if (go != c)
            {
                ans |= 1LL << b;
            }
            u = tr[u][go];
            v = tr[v][go];
        }
        return ans;
    }
};
```

构造 `PerBitTrie t(bit)`，`bit` 是最高处理位，默认 62，恰好覆盖全部非负 `long long`。逐个 `append(x)` 建立前缀版本，`maxXor(l,r,x)` 返回下标半开区间 $[l,r)$ 内元素与 x 的最大异或，空区间返回 -1。建树 $O(n\cdot bit)$，单次查询 $O(bit)$，空间 $O(n\cdot bit)$。不需要区间限制、只对全集查异或最值用 3.6.002 01 Trie；查区间第 k 小而非异或时用 3.3.008 可持久化线段树 / 主席树。

下面固定检查默认最高位、单元素区间、普通小值和空区间：

```
signed main()
{
    PerBitTrie t;
    t.append(0);
    t.append(numeric_limits<int>::max());
    t.append(1LL << 62);
    t.append(5);
    assert(t.maxXor(0, 4, 0) == numeric_limits<int>::max());
    assert(t.maxXor(2, 3, numeric_limits<int>::max()) == (1LL << 62) - 1);
    assert(t.maxXor(3, 4, 2) == 7);
    assert(t.maxXor(2, 2, 0) == -1);
    cout << "9223372036854775807 4611686018427387903 7 -1\n";
    return 0;
}
```

3.6.011 静态区间 MEX

区间 MEX 是「区间内没出现的最小非负整数」。做法是在值域上建可持久化线段树：扫描到位置 i 时把值 `a[i]` 的「最后出现位置」更新为 i 生成版本 `rt[i+1]`，每个值域节点维护子树内最小的最后出现位置。查询 $[l,r)$ 就在版本 r 的值域树上找第一个「最后出现位置 $< l$」的值——它在 $[l,r)$ 里没出现过且是最小的，即为 MEX。空节点视作最后出现位置 -1，天然小于任何 l：

```
struct StaticMex
{
    // Node[0..2] 依次是左儿子、右儿子、该值域区间内最小的最后出现位置。
    using Node = array<int, 3>;
    int n;           // n 是原序列长度，值域只需保留 [0,n]。
    vector<Node> tr; // tr 保存所有前缀版本的动态线段树节点，0 号节点表示全为 -1。
    vector<int> rt;  // rt[i] 表示扫描完前 i 个元素后的版本根。

    StaticMex(const vector<int> &a) : n(a.size()), tr(1, Node{0, 0, -1}), rt(n + 1)
    {
        // a 是原序列；构造所有前缀版本供区间 MEX 查询。
        for (int i = 0; i < n; i++)
        {
            rt[i + 1] = rt[i];
            if (0 <= a[i] && a[i] <= n)
            {
                rt[i + 1] = set(rt[i], 0, n + 1, a[i], i);
            }
        }
    }

    int copy(int p)
    {
        // p 是旧节点编号；复制节点并返回新编号。
        tr.push_back(tr[p]);
        return tr.size() - 1;
    }

    int val(int p) const
    {
        // p 是节点编号；返回子树内最小最后出现位置，空节点返回 -1。
        return p ? tr[p][2] : -1;
    }

    int set(int p, int l, int r, int x, int v)
    {
        // p、[l,r) 是旧节点和值域，x 是值，v 是最后出现下标；返回新根。
        auto q = copy(p);
        if (r - l == 1)
        {
            tr[q][2] = v;
            return q;
        }
        auto m = (l + r) / 2;
```

```
        if (x < m)
        {
            tr[q][0] = set(tr[q][0], l, m, x, v);
        }
        else
        {
            tr[q][1] = set(tr[q][1], m, r, x, v);
        }
        tr[q][2] = min(val(tr[q][0]), val(tr[q][1]));
        return q;
    }

    int find(int p, int l, int r, int lim) const
    {
        // p、[l,r) 是当前节点和值域，lim 是询问左端点；返回首个最后出现位置小于 lim 的值。
        if (r - l == 1)
        {
            return l;
        }
        auto m = (l + r) / 2;
        if (val(tr[p][0]) < lim)
        {
            return find(tr[p][0], l, m, lim);
        }
        return find(tr[p][1], m, r, lim);
    }

    int query(int l, int r) const
    {
        // [l,r) 是原序列下标区间；返回其中没有出现的最小非负整数。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删
        return find(rt[r], 0, n + 1, l);
    }
};
```

构造 `StaticMex sm(a)` 传入完整数组，`query(l,r)` 查半开区间 $[l,r)$ 的 MEX；负数与大于 n 的元素不影响 MEX，直接跳过不更新。例如 `a={0,1,3,1,0}` 时 `query(0,5)==2`、`query(1,4)==0`、`query(2,5)==2`。预处理 $O(n\log n)$，单次查询 $O(\log n)$，空间 $O(n\log n)$。带修改的区间 MEX 需回滚莫队或树套树；求整个数组的 MEX 只需一次线性扫描。

下面固定检查整段、子段、空区间，以及会被忽略的负数和大于 n 的值：

```
signed main()
{
    StaticMex sm({0, 1, 3, 1, 0, -2, 100});
    assert(sm.query(0, 7) == 2);
    assert(sm.query(1, 4) == 0);
    assert(sm.query(2, 5) == 2);
    assert(sm.query(3, 3) == 0);
    assert(sm.query(0, 2) == 2);
    StaticMex empty(vector<int>{});
```

```
    assert(empty.query(0, 0) == 0);
    cout << "2 0 2 0 2 0\n";
    return 0;
}
```

3.6.012 按标签维护的 01 Trie

普通 01 Trie 的一个节点只记录总经过次数，无法限定查询属于哪个集合。本版让每个节点保存 `cnt[u][0]`、`cnt[u][1]` 两套计数，相同整数可以分别带两个标签插入；查询标签 t 时仍从高位到低位贪心，但只有 `cnt[son][t]>0` 的儿子可走。`add(x,t,v)` 允许用正 `v` 插入、负 `v` 删除，删除属于合法调用的前提是该标签下已有足够多份 x；标签集合可能为空，所以异或值与对应元素都用 `optional` 返回。

```
template <int W = 30> struct TaggedTrie
{
    using U = unsigned long long;
    static_assert(1 <= W && W <= 64);

    struct Node
    {
        array<int, 2> ch{}; // ch[0..1] 是比特 0、1 方向的儿子，0 表示空。
        array<int, 2> cnt{}; // cnt[t] 是标签 t 的元素经过本节点的次数。
    };

    // tr[0] 是空节点，tr[1] 是根；cnt[u][t] 是标签 t 经过 u 的次数。
    vector<Node> tr{Node{}, Node{}};

    bool valid(U x) const
    {
        // x 是待插入或查询的值；返回 x 是否落在 [0,2^W)。
        if constexpr (W == 64)
        {
            return true;
        }
        return x < (U(1) << W);
    }

    int newNode()
    {
        // 无参数；新建空节点并返回编号。
        tr.push_back(Node{});
        return (int)tr.size() - 1;
    }

    void add(U x, int t, int v = 1)
    {
        // t 是 0/1 标签，v 为正时插入、为负时删除。
        assert(valid(x) && 0 <= t && t < 2 && v != 0); // 调试检查，可删
        assert(v > 0 || tr[1].cnt[t] >= -v);           // 调试检查，可删
        int p = 1;
        tr[p].cnt[t] += v;
```

```
        for (int i = W - 1; i >= 0; i--)
        {
            int b = x >> i & 1;
            if (!tr[p].ch[b])
            {
                assert(v > 0); // 删除时路径必须已经存在
                tr[p].ch[b] = newNode();
            }
            int q = tr[p].ch[b];
            assert(v > 0 || tr[q].cnt[t] >= -v); // 调试检查，可删
            p = q;
            tr[p].cnt[t] += v;
        }
    }

    optional<U> maxXor(U x, int t) const
    {
        // 只在标签 t 的元素中查询最大的 x xor y。
        assert(valid(x) && 0 <= t && t < 2); // 调试检查，可删
        if (!tr[1].cnt[t])
        {
            return nullopt;
        }
        U ans = 0;
        int p = 1;
        for (int i = W - 1; i >= 0; i--)
        {
            int b = x >> i & 1;
            int c = b ^ 1;
            int q = tr[p].ch[c];
            if (!q || !tr[q].cnt[t])
            {
                c ^= 1;
            }
            if (c != b)
            {
                ans |= U(1) << i;
            }
            p = tr[p].ch[c];
        }
        return ans;
    }

    optional<U> maxElement(U x, int t) const
    {
        // 在标签 t 的元素中取使异或值最大的元素；集合为空时返回空。
        auto ans = maxXor(x, t);
        return ans.has_value() ? optional<U>(*ans ^ x) : nullopt;
    }
};
```

构造 `TaggedTrie<W> tr` 后用 `add(x,t)` 插入、`add(x,t,-1)` 删除一份，`maxXor(x,t)` 与 `maxElement(x,t)`

只查看标签 t 的集合；数值须落在 $[0,2^W)$，$W=64$ 时覆盖全部 `unsigned long long`。例如标签 0 插入 $1,6$、标签 1 插入 $6,10$ 后，`maxXor(8,0)==14`、`maxElement(1,1)==10`；删去标签 0 的 6 不影响标签 1。每次操作 $O(W)$，空间 $O(nW)$。不需要标签维度时使用接口更全的 3.6.002 01 Trie / 压位 Trie，需要历史区间时使用 3.6.010 可持久化 01 Trie。

```
signed main()
{
    TaggedTrie<4> tr;
    tr.add(1, 0);
    tr.add(6, 0);
    tr.add(6, 1);
    tr.add(10, 1);
    assert(tr.maxXor(8, 0) == optional<unsigned long long>(14));
    assert(tr.maxElement(1, 1) == optional<unsigned long long>(10));
    tr.add(6, 0, -1);
    assert(tr.maxXor(8, 0) == optional<unsigned long long>(9));
    tr.add(1, 0, -1);
    assert(!tr.maxXor(8, 0));

    TaggedTrie<4> one;
    one.add(5, 0);
    assert(one.maxXor(5, 0) == optional<unsigned long long>(0));
    assert(one.maxElement(5, 0) == optional<unsigned long long>(5));
    cout << "OK\n";
    return 0;
}
```

3.6.013 左儿子右兄弟 Trie

固定数组 Trie 给每个节点预留整个字符集的儿子表，字符集大而实际分支稀疏时浪费明显。左儿子右兄弟表示把 `child` 指向首个儿子、`next` 串起同一父节点的其余儿子，每个非根节点的 `c` 同时表示父边字符；插入新边时挂到兄弟链表头。它把每节点儿子表降成两个下标，但 `childOf` 必须在线性兄弟链上找字符，所以不能沿用固定数组版每字符 $O(1)$ 的说法。

```
template <class Val = int> struct SiblingTrie
{
    struct Node
    {
        char c = 0; // c 是从父亲走到本节点的字符。
        int child = 0; // child 是第一个儿子，0 表示没有。
        int next = 0; // next 是下一个兄弟，0 表示没有。
        int pass = 0; // pass 是经过本节点（以其为前缀）的串数。
        int end = 0; // end 是恰好在本节点结束的串数。
        Val val{}; // val 是最后一次插入在此结束的串时附带的值。
    };

    // 根为 0；非根节点同时充当字符边，child/next 构成儿子兄弟链。
    vector<Node> tr{Node{}};

```

```
    int newNode(char c)
    {
        // c 是边上的字符；新建节点并返回编号。
        tr.push_back(Node{});
        tr.back().c = c;
        return (int)tr.size() - 1;
    }

    int childOf(int p, char c) const
    {
        // p 是节点，c 是字符；顺兄弟链查找字符为 c 的儿子，不存在返回 0。
        for (int q = tr[p].child; q; q = tr[q].next)
        {
            if (tr[q].c == c)
            {
                return q;
            }
        }
        return 0;
    }

    int insert(string_view s, Val v = Val(1))
    {
        // s 是待插入串，v 是附带值；沿途 pass 加一、终点 end 加一并记录 v，返回终点编号。
        int p = 0;
        tr[p].pass++;
        for (char c : s)
        {
            int q = childOf(p, c);
            if (!q)
            {
                q = newNode(c);
                tr[q].next = tr[p].child;
                tr[p].child = q;
            }
            p = q;
            tr[p].pass++;
        }
        tr[p].end++;
        tr[p].val = v;
        return p;
    }

    int find(string_view s) const
    {
        // s 是查询串；返回 s 对应的节点编号，s 不是任何插入串的前缀时返回 -1。
        int p = 0;
        for (char c : s)
        {
            p = childOf(p, c);
            if (!p)
            {
```

```
                return -1;
            }
        }
        return p;
    }

    int lcp(string_view s) const
    {
        // s 是查询串；返回 s 与全部插入串的最长公共前缀长度。
        int p = 0, len = 0;
        for (char c : s)
        {
            p = childOf(p, c);
            if (!p)
            {
                break;
            }
            len++;
        }
        return len;
    }
};
```

`insert(s,v)` 插入一份字符串、把终点值更新为 `v` 并返回节点编号；`find(s)` 返回前缀节点，缺边时返回 -1；`lcp(s)` 返回与已建 Trie 的最长可走前缀长度。节点的 `pass` 是经过次数，`end` 是完整串次数，重复插入会累加两者而覆盖终点 `val`。若沿字符串各节点的度数为 $d_1, d_2, \ldots$，一次操作为 $O(\sum d_i)$，最坏 $O(|s|D)$，空间 $O(\text{总新边数})$；小固定字符集仍用 $O(|s|)$ 的 3.6.001 字符 Trie。

```
signed main()
{
    SiblingTrie<int> tr;
    int ab = tr.insert("ab", 7);
    int ac = tr.insert("ac", 8);
    int ab2 = tr.insert("ab", 9);
    assert(ab == ab2 && tr.tr[ab].end == 2 && tr.tr[ab].val == 9);
    assert(tr.tr[ac].end == 1 && tr.find("ad") == -1);
    assert(tr.lcp("abd") == 2 && tr.lcp("ax") == 1);
    cout << "OK\n";
    return 0;
}
```

3.6.014 分层位集前驱/后继集合

在宇宙 $[0, n)$ 上只做插入、删除、成员判定和前驱/后继时，`set<int>` 为每个已插入元素分配红黑树节点；当 $n \le 10^7$、查询达到 10^6 时，这个节点开销和 $O(\log |S|)$ 查询都不如按机器字组织的集合。3.6.008 手写 bitset 的 `BitVec` 虽也按字存位，但没有非空字摘要和前驱接口，逐字扫描最坏为 $O(n/64)$，不能直接替代。`FastSet` 的第 0 层按元素存位，之后每层把下一层中非零 64 位字再压成一位。插入/删除只在某一字从零变非零或从非零变零时继续修改摘要层；`next` 在当前层右移并找最低位，若该字无元素就上升到摘要层找下一个非空字，找到后沿各层最低位下降；`prev` 对称地用最高位。

这不是动态键域结构：宇宙大小必须先给定，空间随 n 而不是当前 $|S|$ 增长。第 0 层有 $\lceil n/64 \rceil$ 个 64 位字，后续层构成等比级数；$n = 10^7$ 时有效字载荷为 $156250 + 2442 + 39 + 1 = 158732$ 个字，即 1269856 字节，且没有每个元素一个平衡树节点的分配。

```
class FastSet
{
    // n 是元素宇宙 [0,n) 的大小。
    int n = 0;
    // a[0] 按元素位存集合；a[k+1] 按 a[k] 中非空 64 位字存摘要位。
    vector<vector<unsigned long long>> a;

  public:
    FastSet(int n = 0)
    {
        // n 是元素宇宙大小；构造一个初始为空的集合。
        init(n);
    }

    void init(int m)
    {
        // m 是新的元素宇宙大小；清空集合并建立所有 64 叉摘要层。
        assert(m >= 0); // 调试检查，可删
        n = m;
        a.clear();
        do
        {
            a.emplace_back((m + 63) / 64, 0);
            m = (m + 63) / 64;
        } while (m > 1);
    }

    void set(int x)
    {
        // x 是 [0,n) 内元素；把 x 插入集合，已经存在时不改变状态。
        assert(0 <= x && x < n); // 调试检查，可删
        for (auto &level : a)
        {
            int p = x / 64;
            unsigned long long b = 1ULL << (x % 64);
            if (level[p] & b)
            {
                break;
            }
            level[p] |= b;
            x /= 64;
        }
    }

    void reset(int x)
    {
        // x 是 [0,n) 内元素；把 x 从集合删除，原本不存在时不改变状态。
        assert(0 <= x && x < n); // 调试检查，可删
```

```
        for (auto &level : a)
        {
            int p = x / 64;
            unsigned long long b = 1ULL << (x % 64);
            if (!(level[p] & b))
            {
                return;
            }
            level[p] &= ~b;
            if (level[p])
            {
                break;
            }
            x /= 64;
        }
    }

    bool test(int x) const
    {
        // x 是 [0,n) 内元素；返回 x 当前是否属于集合。
        assert(0 <= x && x < n); // 调试检查，可删
        return a[0][x / 64] >> (x % 64) & 1ULL;
    }

    int next(int x) const
    {
        // x 是 [0,n] 内起点；返回不小于 x 的最小集合元素，不存在时返回 -1。
        assert(0 <= x && x <= n); // 调试检查，可删
        if (x == n)
        {
            return -1;
        }
        for (int h = 0; h < (int)a.size(); h++)
        {
            int p = x / 64;
            if (p >= (int)a[h].size())
            {
                return -1;
            }
            unsigned long long w = a[h][p] >> (x % 64);
            if (!w)
            {
                x = p + 1;
                continue;
            }
            x += countr_zero(w);
            while (h)
            {
                h--;
                x *= 64;
                x += countr_zero(a[h][x / 64]);
            }
```

```
            return x < n ? x : -1;
        }
        return -1;
    }

    int prev(int x) const
    {
        // x 是 [-1,n) 内起点；返回不大于 x 的最大集合元素，不存在时返回 -1。
        assert(-1 <= x && x < n); // 调试检查，可删
        if (x < 0)
        {
            return -1;
        }
        for (int h = 0; h < (int)a.size() && x >= 0; h++)
        {
            int p = x / 64;
            unsigned long long w = a[h][p] << (63 - x % 64);
            if (!w)
            {
                x = p - 1;
                continue;
            }
            x -= countl_zero(w);
            while (h)
            {
                h--;
                x = x * 64 + 63;
                x -= countl_zero(a[h][x / 64]);
            }
            return x;
        }
        return -1;
    }
};
```

`FastSet st(n)` 与 `init(n)` 建立宇宙 $[0,n)$ 的空集合，`set(x)`/`reset(x)` 是幂等插入/删除，`test(x)` 判成员；`next(x)` 返回最小的 $y\in S, y\ge x$，允许 $x=n$ 并在无解时返回 `-1`；`prev(x)` 返回最大的 $y\in S, y\le x$，允许 $x=-1$ 并在无解时返回 `-1`。其余下标均须落在 $[0,n)$。若 $H=1+\lfloor \log_{64}\max(1,n-1)\rfloor$ 是层数，`set`、`reset`、`next`、`prev` 均为 $O(H)$ 最坏时间，`test` 为 $O(1)$，初始化和空间均为 $O(\lceil n/64\rceil)$ 个机器字。需要秩、第 k 个或区间计数时用 3.2.003 前缀和上的二分 的树状数组；宇宙很小且编译期已知时标准 `bitset` 更直接；只需少量元素迭代或键域无法预先确定时再用 3.5.007 PBDS 平衡树 或 `set`。

下面固定检查空集、$0/63/64$ 字边界、四层摘要、最大宇宙端点和幂等删除：

```
signed main()
{
    FastSet s(130);
    for (int x : {0, 63, 64, 129})
    {
        s.set(x);
    }
```

```
    assert(s.next(1) == 63 && s.next(65) == 129 && s.next(130) == -1);
    assert(s.prev(-1) == -1 && s.prev(62) == 0 && s.prev(128) == 64);
    s.reset(64);
    s.reset(64);
    assert(s.next(64) == 129 && s.prev(64) == 63);

    FastSet deep(64 * 64 + 1);
    deep.set(0);
    deep.set(4096);
    assert(deep.next(1) == 4096 && deep.prev(4095) == 0);
    cout << "OK\n";
}
```

3.7 函数与空间数据结构

3.7.001 李超线段树

李超线段树维护一族一次函数 $y = kx + b$，在预先给定的横坐标集合上查某点的最小（或最大）值。每个线段树节点保存「在本区间中点处最优」的那条直线：插入新直线时先在中点比较，把中点更优者留在节点上，另一条至多只能在左半或右半区间的一侧翻盘，于是递归进入那一侧。这样每条直线只沿 $O(\log)$ 条边下沉。本版把可能查询的横坐标离散化后建静态树，比较时用真实横坐标 `xs[·]` 取值以避免斜率与整型除法的精度问题：

```
template <class T, class C = T> class LiChao
{
    // 当前实现维护最小值。改成最大值有两种等价方式：反向所有优劣比较并把空值改为负无穷，
    // 或在 k、b 与函数值都能安全取反时插入 {-k,-b}，再把非空结果取反。
  public:
    static_assert(is_floating_point_v<T> || (is_integral_v<T> && is_signed_v<T>));

    struct Line
    {
        // k、b 是一次函数 y=kx+b 的斜率与截距。
        T k{}, b{};

        C get(T x) const
        {
            // x 是查询横坐标；用比较类型 C 计算函数值。
            return (C)k * x + b;
        }
    };

  private:
    // xs 是全部允许查询的横坐标，tr[p] 是线段树节点 p 保存的候选直线。
    vector<T> xs;
    vector<optional<Line>> tr;

    void add(int p, int l, int r, Line f)
    {
```

```
        // p 是节点编号，[l,r) 是横坐标下标段，f 是待加入直线。
        if (!tr[p])
        {
            tr[p] = f;
            return;
        }
        int m = midpoint(l, r);
        bool lef = f.get(xs[l]) < tr[p]->get(xs[l]);
        bool mid = f.get(xs[m]) < tr[p]->get(xs[m]);
        if (mid)
        {
            swap(f, *tr[p]);
        }
        if (r - l == 1)
        {
            return;
        }
        if (lef != mid)
        {
            add(p << 1, l, m, f);
        }
        else
        {
            add(p << 1 | 1, m, r, f);
        }
    }

public:
    LiChao(vector<T> xs) : xs(move(xs))
    {
        // xs 是所有可能查询的横坐标；排序去重并建立最小值李超树。
        // 改最大值时可在系数可安全取反时优先对直线取反再对答案取反，可保留李超树的比较不变量。
        sort(this->xs.begin(), this->xs.end());
        this->xs.erase(unique(this->xs.begin(), this->xs.end()), this->xs.end());
        assert(!this->xs.empty()); // 调试检查，可删
        tr.resize(4 * this->xs.size());
    }

    void add(Line f)
    {
        // f 是待加入直线；使其参与全部预存横坐标的最小值查询。
        // 最大值版可在 -f.k、-f.b 不溢出时调用 add({-f.k, -f.b})，并在 query 的返回值上取负。
        add(1, 0, xs.size(), f);
    }

    optional<C> query(T x) const
    {
        // x 必须在构造坐标中；返回最小函数值，尚未加入直线时返回空。
        // 最大值版若按直线取反，调用者取负即可；空结果仍保持 nullopt。
        int q = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        assert(q < (int)xs.size() && xs[q] == x); // 调试检查，可删
        optional<C> ans;
```

```
        int p = 1, l = 0, r = xs.size();
        while (true)
        {
            if (tr[p])
            {
                C y = tr[p]->get(x);
                if (!ans || y < *ans)
                {
                    ans = y;
                }
            }
            if (r - l == 1)
            {
                break;
            }
            int m = midpoint(l, r);
            if (q < m)
            {
                p <<= 1;
                r = m;
            }
            else
            {
                p = p << 1 | 1;
                l = m;
            }
        }
        return ans;
    }
};
```

先用全部可能查询的横坐标构造 `LiChao<T> lc(xs)`（自动排序去重），`add({k,b})` 插入直线 $y = kx + b$，`query(x)` 返回 `optional<C>`：有直线时保存最小值，空函数集返回 `nullopt`；x 必须在构造坐标中。`T` 是系数与横坐标类型，`C` 是 $kx + b$ 的计算、比较和返回类型，默认与 `T` 相同；若 `T=long long` 且乘积可能溢出，使用 `LiChao<long long, __int128>`。求最大值可把 k, b 取反并对答案取反。插入与查询均 $O(\log |xs|)$，空间 $O(|xs|)$。横坐标未知时用 3.7.002 动态开点李超树，需要历史版本用 3.7.003 可持久化李超树；斜率与查询都单调时可换 12.3.004 斜率优化 / 李超树优化的单调凸壳。

3.7.002 动态开点李超树

当横坐标定义域 $[lo, hi)$ 很大而无法离散化时，把 3.7.001 李超线段树 改成动态开点：节点在插线或查询实际经过时才创建，比较规则不变（比较的是整数横坐标本身），未分配的子树表示该段没有候选直线。这样空间只随实际访问路径增长，适合横坐标范围大、操作数相对少的场景：

```
template <class T, class C = T> class DynLiChao
{
    // 当前实现维护最小值。改求最大值时，插入 {-k,-b} 并对非空 query 结果取负；
    // 若直接改比较，插线中 lef/mid 与查询累计的比较必须全部同步反向。
  public:
```

```
    struct Line
    {
        // k、b 是一次函数 y=kx+b 的斜率与截距。
        T k{}, b{};

        C get(T x) const
        {
            // x 是查询横坐标；用比较类型 C 计算函数值。
            return (C)k * x + b;
        }
    };

  private:
    struct Node
    {
        // f 是当前区间中点占优的直线，ch[0..1] 是左右儿子编号，-1 表示尚未分配。
        optional<Line> f;
        array<int, 2> ch{-1, -1};
    };

    // lo、hi 定义整数查询域 [lo,hi)，tr 保存实际访问到的节点。
    T lo, hi;
    vector<Node> tr{{}};

    int newNode()
    {
        // 分配一个空节点并返回编号。
        tr.push_back({});
        return tr.size() - 1;
    }

    void add(int p, T l, T r, Line f)
    {
        // p 是节点编号，[l,r) 是整数域，f 是待加入直线。
        if (!tr[p].f)
        {
            tr[p].f = f;
            return;
        }
        T m = midpoint(l, r);
        bool lef = f.get(l) < tr[p].f->get(l);
        bool mid = f.get(m) < tr[p].f->get(m);
        if (mid)
        {
            swap(f, *tr[p].f);
        }
        if (l + 1 == r)
        {
            return;
        }
        int d = lef != mid ? 0 : 1;
        if (tr[p].ch[d] == -1)
```

```
        {
            tr[p].ch[d] = newNode();
        }
        if (d == 0)
        {
            add(tr[p].ch[d], l, m, f);
        }
        else
        {
            add(tr[p].ch[d], m, r, f);
        }
    }

public:
    DynLiChao(T lo, T hi) : lo(lo), hi(hi)
    {
        // lo、hi 定义非空整数半开区间；构造最小值动态李超树。
        // 最大值版优先采用直线和答案整体取反，动态开点方向无需变化。
        static_assert(is_integral_v<T> && is_signed_v<T>);
        assert(lo < hi); // 调试检查，可删
    }

    void add(Line f)
    {
        // f 是待加入直线；使其参与整个整数域的最小值查询。
        // 最大值版可插入 {-f.k,-f.b}，查询非空结果再取负。
        add(0, lo, hi, f);
    }

    optional<C> query(T x) const
    {
        // x 是 [lo,hi) 内整数；返回最小函数值，空函数集返回空。
        // 若直接维护最大值，插线与这里累计答案的全部优劣比较必须同步反向。
        assert(lo <= x && x < hi); // 调试检查，可删
        optional<C> ans;
        int p = 0;
        T l = lo, r = hi;
        while (p != -1)
        {
            if (tr[p].f)
            {
                C y = tr[p].f->get(x);
                if (!ans || y < *ans)
                {
                    ans = y;
                }
            }
            if (l + 1 == r)
            {
                break;
            }
            T m = midpoint(l, r);
```

```
            if (x < m)
            {
                p = tr[p].ch[0];
                r = m;
            }
            else
            {
                p = tr[p].ch[1];
                l = m;
            }
        }
        return ans;
    }
};
```

用 `DynLiChao<T> lc(lo, hi)` 维护有符号整数域 $[lo, hi)$；`add({k,b})` 插入直线，`query(x)` 返回 `optional<C>`，空函数集为 `nullopt`。求最大值时插入 `{-k,-b}` 并对非空结果取反，空集仍为 `nullopt`。`T` 是系数与横坐标类型，`C` 是 $kx + b$ 的计算、比较和返回类型，默认与 `T` 相同；乘积有溢出风险时使用 `DynLiChao<long long, __int128>`。`midpoint` 使定义域横跨有符号数两端时也不会因求中点溢出。插入与查询均 $O(\log U)$，U 是定义域内整数个数；空间 $O(q \log U)$。查询坐标预先已知时用 3.7.001 李超线段树 常数更小，需要历史版本时用 3.7.003 可持久化李超树。

3.7.003 可持久化李超树

给动态开点李超树加上可持久化：每次插线只复制被访问路径上的节点、返回新根，未访问的子树与旧版本共享，于是可以在任意历史版本上查询、也可从任意旧版本分叉出新版本。常用于按时间轴维护函数集合、或树上把父版本传给子节点：

```
template <class T, class C = T> struct PerLiChao
{
    // 当前实现维护最小值。改最大值可反向所有优劣比较并把空值改为负无穷，
    // 或在 k、b 与函数值都能安全取反时对插入直线同时取反、再对 query 结果取反；后者不改变持久化不变量。
    struct Line
    {
        // k、b 是一次函数 y=kx+b 的斜率与截距。
        T k{}, b{};
        C get(T x) const
        {
            // x 是查询横坐标；用比较类型 C 计算函数值。
            return (C)k * x + b;
        }
    };
    struct Node
    {
        // f 是当前区间中点占优的直线，l、r 是左右儿子编号。
        optional<Line> f;
        int l = -1, r = -1;
    };

```

```
    // lo、hi 是整数定义域，t 是所有版本共享的节点池。
    T lo, hi;
    vector<Node> t;

    PerLiChao(T lo, T hi) : lo(lo), hi(hi)
    {
        // lo、hi 定义非空整数半开区间；初始空版本根为 -1。
        static_assert(is_integral_v<T> && is_signed_v<T>);
        assert(lo < hi); // 调试检查，可删
    }

    int clone(int p)
    {
        // p 是旧节点编号；复制它，-1 则新建空节点，返回新编号。
        t.push_back(p < 0 ? Node{} : t[p]);
        return t.size() - 1;
    }

    int add(int root, Line f)
    {
        // root 是旧版本根，f 是新直线；返回插入后的版本根。
        return add(root, lo, hi, f);
    }

    int add(int p, T l, T r, Line f)
    {
        // p 是旧节点，[l,r) 是当前整数域，f 是新直线；返回新节点。
        p = clone(p);
        if (!t[p].f)
        {
            t[p].f = f;
            return p;
        }
        T m = midpoint(l, r);
        bool x = f.get(l) < t[p].f->get(l);
        bool y = f.get(m) < t[p].f->get(m);
        if (y)
        {
            swap(f, *t[p].f);
        }
        if (l + 1 == r)
        {
            return p;
        }
        if (x != y)
        {
            t[p].l = add(t[p].l, l, m, f);
        }
        else
        {
            t[p].r = add(t[p].r, m, r, f);
        }
```

```
        return p;
    }

    optional<C> query(int p, T x) const
    {
        // p 是版本根，x 是查询横坐标；返回最小值，空版本返回空。
        // 最大值版按直线取反时，对非空返回值取负，空版本仍返回 nullopt。
        assert(lo <= x && x < hi); // 调试检查，可删
        return qval(p, lo, hi, x);
    }

    optional<C> qval(int p, T l, T r, T x) const
    {
        // p 是当前节点，[l,r) 是其整数域，x 是横坐标；返回路径最小值。
        // 若改为直接维护最大值，必须同步反向这里的比较与空值语义。
        if (p < 0)
        {
            return nullopt;
        }
        optional<C> ans;
        if (t[p].f)
        {
            ans = t[p].f->get(x);
        }
        if (l + 1 == r)
        {
            return ans;
        }
        T m = midpoint(l, r);
        auto z = x < m ? qval(t[p].l, l, m, x) : qval(t[p].r, m, r, x);
        if (!ans || (z && *z < *ans))
        {
            ans = z;
        }
        return ans;
    }
};
```

用 `PerLiChao<T> lc(lo, hi)` 建对象，`T` 是有符号整数，版本根 `-1` 表示空函数集。`add(root,{k,b})` 基于旧版本插线并返回新根，`query(root,x)` 返回 `optional<C>`，不把空版本与某条极大值直线混用。最大值版仍可对每个版本插入的 `k,b` 取反并对非空查询结果取反，不改变根和节点共享关系。`C` 是 $kx+b$ 的计算、比较和返回类型，默认与 `T` 相同；乘积有溢出风险时使用 `PerLiChao<long long, __int128>`。中点使用 `midpoint`。例如值域 $[-5,6)$，令 `r1=add(-1,{2,3})`、`r2=add(r1,{-1,1})`，再从 `r1` 分叉 `r3=add(r1,{0,-3})`，则 `*query(r1,0)==3`、`*query(r2,0)==1`、`*query(r3,0)==-3`，而 `query(-1,0)==nullopt`。每次插线与查询均为 $O(\log U)$，每次插线新增 $O(\log U)$ 个节点；不需要历史版本时用 3.7.002 动态开点李超树。

3.7.004 K-D Tree

K-D Tree 递归地按当前跨度较大的坐标轴用 `nth_element` 把点集对半劈开，每个节点存自己子树的包围盒（x, y 的上下界）与权值和。矩形求和时用包围盒做整块命中/整块剪除、只在跨界时下递归；最近点查询先用「查询点到子树包围盒的最小距离平方」`boxDist` 估下界，先递归更近的一侧、当另一侧下界已不小于当前最优就剪掉。距离平方类型由 `Dist` 指定：

```
template <class Dist = long long> class KDTree
{
    // a[i][0..2] 依次是第 i 个点的横坐标、纵坐标和可加权值。
    vector<array<int, 3>> a;
    // t[u][0..9] 依次是横坐标、纵坐标、点权、左儿子、右儿子、横坐标下界、横坐标上界、纵坐标下界、纵坐标
    ↪ 上界、子树权值和。
    vector<array<int, 10>> t;
    // rt 是 K-D Tree 根编号，-1 表示空树。
    int rt = -1;

    void pull(int u, int v)
    {
        // u 是父节点，v 是儿子编号；把非空儿子的包围盒与权值和合入 u。
        if (v < 0)
        {
            return;
        }
        t[u][5] = min(t[u][5], t[v][5]);
        t[u][6] = max(t[u][6], t[v][6]);
        t[u][7] = min(t[u][7], t[v][7]);
        t[u][8] = max(t[u][8], t[v][8]);
        t[u][9] += t[v][9];
    }

    int build(int l, int r)
    {
        // l、r 是 a 中待建树的半开区间；返回该批点的 K-D Tree 根编号。
        if (l == r)
        {
            return -1;
        }
        int xl = a[l][0], xr = xl, yl = a[l][1], yr = yl;
        for (int i = l + 1; i < r; i++)
        {
            xl = min(xl, a[i][0]);
            xr = max(xr, a[i][0]);
            yl = min(yl, a[i][1]);
            yr = max(yr, a[i][1]);
        }
        int d = (Dist)xr - (Dist)xl < (Dist)yr - (Dist)yl;
        int m = midpoint(l, r);
        nth_element(a.begin() + l,
                    a.begin() + m,
                    a.begin() + r,
```

```
                    [&](const auto &x, const auto &y)
                    {
                        // x、y 是两个三维点记录；按当前跨度更大的坐标排序。
                        return x[d] < y[d];
                    });
        int u = (int)t.size();
        t.push_back({a[m][0], a[m][1], a[m][2], -1, -1, a[m][0], a[m][0], a[m][1], a[m][1],
↪   a[m][2]});
        t[u][3] = build(l, m);
        t[u][4] = build(m + 1, r);
        pull(u, t[u][3]);
        pull(u, t[u][4]);
        return u;
    }

    int rectSum(int u, int xl, int xr, int yl, int yr) const
    {
        // u 是当前节点，四个边界给出闭矩形；返回矩形内点权和。
        if (u < 0 || t[u][6] < xl || xr < t[u][5] || t[u][8] < yl || yr < t[u][7])
        {
            return 0;
        }
        if (xl <= t[u][5] && t[u][6] <= xr && yl <= t[u][7] && t[u][8] <= yr)
        {
            return t[u][9];
        }
        int ans = 0;
        if (xl <= t[u][0] && t[u][0] <= xr && yl <= t[u][1] && t[u][1] <= yr)
        {
            ans += t[u][2];
        }
        return ans + rectSum(t[u][3], xl, xr, yl, yr) + rectSum(t[u][4], xl, xr, yl, yr);
    }

    Dist boxDist(int u, int x, int y) const
    {
        // u 是非空子树根，x、y 是查询点；返回查询点到该子树包围盒的最小距离平方。
        Dist dx = 0, dy = 0;
        if (x < t[u][5])
        {
            dx = (Dist)t[u][5] - (Dist)x;
        }
        else if (t[u][6] < x)
        {
            dx = (Dist)x - (Dist)t[u][6];
        }
        if (y < t[u][7])
        {
            dy = (Dist)t[u][7] - (Dist)y;
        }
        else if (t[u][8] < y)
        {
```

```
            dy = (Dist)y - (Dist)t[u][8];
        }
        return dx * dx + dy * dy;
    }

    void nearest(int u, int x, int y, Dist &ans) const
    {
        // u 是当前子树根，x、y 是查询点，ans 是当前最优距离平方。
        if (u < 0 || boxDist(u, x, y) >= ans)
        {
            return;
        }
        Dist dx = (Dist)t[u][0] - (Dist)x;
        Dist dy = (Dist)t[u][1] - (Dist)y;
        ans = min(ans, dx * dx + dy * dy);
        int p = t[u][3], q = t[u][4];
        if (p >= 0 && q >= 0 && boxDist(p, x, y) > boxDist(q, x, y))
        {
            swap(p, q);
        }
        if (p >= 0)
        {
            nearest(p, x, y, ans);
        }
        if (q >= 0)
        {
            nearest(q, x, y, ans);
        }
    }

  public:
    KDTree(vector<array<int, 3>> a) : a(move(a))
    {
        // a 的三维依次是横坐标、纵坐标和点权；一次性建立静态 K-D Tree。
        t.reserve(this->a.size());
        rt = build(0, (int)this->a.size());
    }

    int rectSum(int xl, int xr, int yl, int yr) const
    {
        // xl、xr、yl、yr 给出闭矩形且左右、上下边界有序；返回矩形内全部点权和。
        assert(xl <= xr && yl <= yr); // 调试检查，可删
        return rectSum(rt, xl, xr, yl, yr);
    }

    optional<Dist> nearest(int x, int y) const
    {
        // x、y 是查询点坐标；返回到输入点集的最小欧氏距离平方，空点集返回空。
        if (rt < 0)
        {
            return nullopt;
        }
```

```
        Dist ans = numeric_limits<Dist>::max();
        nearest(rt, x, y, ans);
        return ans;
    }
};
```

把每个点写成 `{x,y,w}` 装进 `vector<array<int,3>>` 后构造 `KDTree tr(a)`；`rectSum(xl,xr,yl,yr)` 返回闭矩形内点权和，`nearest(x,y)` 返回到点集的最小欧氏距离平方，空点集返回 `nullopt`。默认 `Dist=long long`，调用者须保证坐标差、坐标平方和能由 `Dist` 表示；需要宽距离时构造 `KDTree<__int128> tr(a)`。`Dist` 只控制距离计算，点权与子树权值和仍由 `int` 表示，调用者必须另行保证它们不溢出；扩大 `Dist` 不会扩大点权累计范围，反之亦然。点集构造后固定。例如点 `{(0,0,2),(3,0,5),(1,2,7)}` 在 $[0,1]\times[0,2]$ 内权和为 9，查询点 $(2,0)$ 的最近距离平方为 1。建树 $O(n\log n)$，两类查询确定性最坏 $O(n)$；随机均匀点集上矩形聚合通常接近 $O(\sqrt{n})$，但不是确定性保证。在线加点用 3.7.005 K-D Tree 二进制分组。

下面检查空集、闭矩形边界、最近点和宽距离类型：

```
signed main()
{
    KDTree<> empty(vector<array<int, 3>>{});
    assert(!empty.nearest(0, 0));
    assert(empty.rectSum(-1, 1, -1, 1) == 0);

    KDTree<> tr(vector<array<int, 3>>{{0, 0, 2}, {3, 0, 5}, {1, 2, 7}});
    assert(tr.rectSum(0, 1, 0, 2) == 9);
    assert(tr.rectSum(3, 3, 0, 0) == 5);
    assert(tr.nearest(2, 0) == optional<long long>(1));

    KDTree<__int128> wide(vector<array<int, 3>>{{3000000000LL, 3000000000LL, 1}});
    __int128 d = 6000000000LL;
    assert(wide.nearest(-3000000000LL, -3000000000LL) == optional<__int128>(2 * d * d));
}
```

3.7.005 K-D Tree 二进制分组

K-D Tree 难以在线增量维护平衡,二进制分组用「重建代替旋转」绕开:把已插入的点按块大小 $2^0, 2^1, 2^2, \dots$ 组织成若干静态 K-D Tree，插入一个点时若第 k 层已占用就把它与新点合并、进位到第 $k+1$ 层重建（类似二进制加一）。查询时遍历所有非空块累加。因为每个点被重建的次数是它经过的层数（对数级），整体插入摊还可控：

```
template <class T = long long> struct KDBlocks
{
    struct KDPoint
    {
        // x、y 是点坐标，w 是可加权值。
        int x, y;
        T w;
    };

    struct Block
```

```
    {
        struct Node
        {
            // p 是本点，l、r 是儿子，四个边界与 sum 描述整棵子树。
            KDPoint p;
            int l = -1, r = -1;
            int xl, xr, yl, yr;
            T sum{};
        };

        // a 是本块的全部点，t 是静态 K-D Tree 节点池，root 是根。
        vector<KDPoint> a;
        vector<Node> t;
        int root = -1;

        Block(vector<KDPoint> a = {}) : a(move(a))
        {
            // a 是本块点集；一次性建立静态 K-D Tree。
            t.reserve(this->a.size());
            root = build(0, this->a.size(), 0);
        }

        int build(int l, int r, int d)
        {
            // [l,r) 是待建点段，d=0/1 表示按横/纵坐标切分；返回根编号。
            if (l == r)
            {
                return -1;
            }
            int m = midpoint(l, r);
            nth_element(a.begin() + l,
                        a.begin() + m,
                        a.begin() + r,
                        [&](const KDPoint &p, const KDPoint &q)
                        {
                            return d ? p.y < q.y : p.x < q.x;
                        });
            int u = t.size();
            t.push_back({a[m], -1, -1, a[m].x, a[m].x, a[m].y, a[m].y, a[m].w});
            t[u].l = build(l, m, d ^ 1);
            t[u].r = build(m + 1, r, d ^ 1);
            pull(u, t[u].l);
            pull(u, t[u].r);
            return u;
        }

        void pull(int u, int v)
        {
            // u 是父节点，v 是儿子；把非空儿子的包围盒与权值和合入 u。
            if (v < 0)
            {
                return;
```

```
            }
            t[u].xl = min(t[u].xl, t[v].xl);
            t[u].xr = max(t[u].xr, t[v].xr);
            t[u].yl = min(t[u].yl, t[v].yl);
            t[u].yr = max(t[u].yr, t[v].yr);
            t[u].sum += t[v].sum;
        }

        T query(int u, int xl, int xr, int yl, int yr) const
        {
            // u 是当前节点；返回闭矩形 [xl,xr] x [yl,yr] 内的点权和。
            if (u < 0 || t[u].xr < xl || xr < t[u].xl || t[u].yr < yl || yr < t[u].yl)
            {
                return {};
            }
            if (xl <= t[u].xl && t[u].xr <= xr && yl <= t[u].yl && t[u].yr <= yr)
            {
                return t[u].sum;
            }
            T ans{};
            if (xl <= t[u].p.x && t[u].p.x <= xr && yl <= t[u].p.y && t[u].p.y <= yr)
            {
                ans += t[u].p.w;
            }
            return ans + query(t[u].l, xl, xr, yl, yr) + query(t[u].r, xl, xr, yl, yr);
        }

        T query(int xl, int xr, int yl, int yr) const
        {
            // 四个参数是有序的闭矩形边界；返回本块内点权和。
            assert(xl <= xr && yl <= yr); // 调试检查，可删
            return query(root, xl, xr, yl, yr);
        }
    };

    // b[k] 保存大小为 2^k 的静态块，n 是当前点数。
    vector<optional<Block>> b;
    int n = 0;

    void add(int x, int y, T w)
    {
        // x、y 是新点坐标，w 是权值；插入后按二进制进位重建块。
        vector<KDPoint> a{{x, y, w}};
        int k = 0;
        while (k < (int)b.size() && b[k])
        {
            a.insert(a.end(), b[k]->a.begin(), b[k]->a.end());
            b[k].reset();
            k++;
        }
        if (k == (int)b.size())
        {
```

```
            b.emplace_back();
        }
        b[k].emplace(move(a));
        n++;
    }

    T query(int xl, int xr, int yl, int yr) const
    {
        // 四个参数是有序的闭矩形边界；返回所有块内的点权和。
        assert(xl <= xr && yl <= yr); // 调试检查，可删
        T ans{};
        for (auto &x : b)
        {
            if (x)
            {
                ans += x->query(xl, xr, yl, yr);
            }
        }
        return ans;
    }
};
```

默认 `KDBlocks<> kd`（权值类型 `long long`），`add(x,y,w)` 在线插入带权点、`query(xl,xr,yl,yr)` 求闭矩形 $[xl,xr]\times[yl,yr]$ 内权值和。只支持插入与矩形和，不支持删除或最近点（分块后最近点无法跨块统一剪枝）。插入 n 点总建树 $O(n\log^2 n)$、空间 $O(n)$，单次矩形查询遍历 $O(\log n)$ 个块、确定性最坏 $O(n)$；随机均匀点集通常快得多，但没有统一的次线性最坏界。点集静态时直接用常数更小的 3.7.004 K-D Tree；只需一维带修区间和用 3.2.001 树状数组；带删除的二维数点常改用 CDQ 分治或树套树离线处理。

3.7.006 分散层叠

分散层叠（分数级联）用于在 k 张静态有序表上反复把同一值 x 送进每张表求 `lower_bound`：预处理后只在第一张增强表二分，随后沿预存的相邻层「桥」指针每层常数转移，把单次多表查询从 $O(\sum_i \log n_i)$ 降到 $O(\log N + k)$（$N=\sum_i n_i$）。这份底层与其接口 `lowerBound` 已在基础算法章完整给出且带自测，本节不重复粘贴，直接见 2.2.006 分数级联（Fractional Cascading）——构造 `FracCascade<T> fc(tabs)`（各表升序）、`fc.lowerBound(x)` 返回各表下界位置数组，构造 $O(N\log N)$、单次查询 $O(\log N + k)$。它服务于线段树/树链分解后同时查多份有序目录，或几何中沿相邻层定位同一坐标；只有一两次查询时预处理不划算，有修改时改用平衡树或树套有序结构。

3.7.007 双端队列全局查询

用两个栈模拟双端队列，即可在两端插入删除的同时维护整段的结合律聚合（甚至非交换的，如仿射函数复合）。左栈 `l` 从队首往里、右栈 `r` 从队尾往里，每个栈元素额外存「从该栈栈底到本元素」的前缀聚合，因此两栈栈顶的聚合按队列方向 `op(l 顶, r 顶)` 合并就是全局值。当某一侧栈空又要从该侧弹出或取端点时，把当前所有元素按队列顺序取出、对半 `rebuild` 回两栈，保证均摊 $O(1)$：

```cpp
template <class T, class F> struct AggDeque
{
    // op 是结合律运算，e 是幺元。
    F op;
    T e;
    // l、r 是队首侧与队尾侧栈；每项为 {原值，按队列方向的本栈聚合}。
    vector<pair<T, T>> l, r;

    AggDeque(F combine, T identity) : op(combine), e(identity)
    {
        // combine 是结合律二元运算，identity 是其幺元；初始队列为空。
    }

    int size() const
    {
        // 返回队列元素数。
        return l.size() + r.size();
    }
    bool empty() const
    {
        // 返回队列是否为空。
        return l.empty() && r.empty();
    }
    void pushFront(T x)
    {
        // x 是新元素；插入队首并更新左栈聚合。
        l.push_back({x, l.empty() ? x : op(x, l.back().second)});
    }
    void pushBack(T x)
    {
        // x 是新元素；插入队尾并更新右栈聚合。
        r.push_back({x, r.empty() ? x : op(r.back().second, x)});
    }
    void rebuild(int k)
    {
        // k 是重建后放入队首侧的元素数；保持整个队列顺序不变。
        assert(0 <= k && k <= size()); // 调试检查，可删
        vector<T> a;
        for (auto i = l.rbegin(); i != l.rend(); i++)
        {
            a.push_back(i->first);
        }
        for (auto x : r)
        {
            a.push_back(x.first);
        }
        l.clear();
        r.clear();
        for (int i = k - 1; i >= 0; i--)
        {
            pushFront(a[i]);
        }
```

```
        for (int i = k; i < (int)a.size(); i++)
        {
            pushBack(a[i]);
        }
    }
    void popFront()
    {
        // 删除队首元素。
        assert(!empty()); // 调试检查，可删
        if (l.empty())
        {
            rebuild((size() + 1) / 2);
        }
        l.pop_back();
    }
    void popBack()
    {
        // 删除队尾元素。
        assert(!empty()); // 调试检查，可删
        if (r.empty())
        {
            rebuild(size() / 2);
        }
        r.pop_back();
    }
    T front() const
    {
        // 返回队首元素；只读端点，不为读取重建两侧栈。
        assert(!empty()); // 调试检查，可删
        return l.empty() ? r.front().first : l.back().first;
    }
    T back() const
    {
        // 返回队尾元素；只读端点，不为读取重建两侧栈。
        assert(!empty()); // 调试检查，可删
        return r.empty() ? l.front().first : r.back().first;
    }
    T query() const
    {
        // 按队首到队尾顺序聚合全部元素；空队列返回 e。
        if (l.empty())
        {
            return r.empty() ? e : r.back().second;
        }
        if (r.empty())
        {
            return l.back().second;
        }
        return op(l.back().second, r.back().second);
    }
};
```

构造 AggDeque<T,F> dq(op, e)，例如维护队列和：

```
signed main()
{
    auto op = [](int x, int y)
    {
        return x + y;
    };
    AggDeque<int, decltype(op)> dq(op, 0);
    dq.pushBack(2);
    dq.pushFront(3);
    assert(dq.query() == 5);
    dq.popFront();
    assert(dq.front() == 2);
}
```

op 是结合律二元运算、e 是其幺元（可非交换，注意 op(l,r) 的左右顺序对应队列方向）；pushFront/pushBack/popFront/popBack 两端增删，front/back 只读对应一侧栈顶或另一侧栈底，query() 返回按队首到队尾顺序 op 折叠的全局聚合（空队列返回 e）。只有删除时所在侧为空才会 rebuild；读取端点绝不重建，因而所有公开操作均摊 $O(1)$，单次 rebuild 最坏 $O(n)$，空间 $O(n)$。若只在一端进出用普通栈即可；若聚合可逆（如加法、异或）用双指针前缀更简单；需要区间（而非全局）查询用线段树。它区别于滑动窗口最值 12.3.001 单调队列优化之处在于支持任意结合律与非交换运算。

3.7.008 析合树（$O(n \log n)$ 构造）

排列区间 $[l, r)$ 是连续段当且仅当其中的值构成连续整数，即 $\max - \min = r - l - 1$；不与其他连续段交叉的连续段称为强区间，全部强区间按包含关系构成析合树。增量扫描右端点 i 时，线段树在位置 j 维护 $q_j = \max_{j \le k \le i} a_k - \min_{j \le k \le i} a_k - (i - j)$，两个单调栈把新元素造成的最大值、最小值变化拆成区间加；最左的 $q_j = 0$ 给出本轮可能合并到的最远左端点，再在线性维护的析合森林中建立叶、析点或合点。线段树的差分与懒标记使用 __int128，避免输入从 LLONG_MIN 开始时对端点取负溢出。

```
struct PermTree
{
    struct Node
    {
        int l = 0;        // l 保存强区间左端点。
        int r = 0;        // r 保存强区间右端点，区间为 [l,r)。
        int type = 0;     // type 为 0、1、2 时分别表示叶、析点、合点。
        int par = -1;     // par 保存父结点，根的父亲为 -1。
        int mid = -1;     // 合点中 mid 保存最右孩子的左端点。
        vector<int> ch;   // ch 按原排列顺序保存直接孩子。
    };

    struct RMQ
    {
        vector<int> lg;         // lg[len] 保存 floor(log2(len))。
        vector<vector<int>> mn; // mn[k][i] 保存长 2^k 区间最小值。
        vector<vector<int>> mx; // mx[k][i] 保存长 2^k 区间最大值。

```

```
        RMQ(const vector<int> &a)
        {
            // a 是排列；建立区间最大值和最小值稀疏表。
            int n = a.size();
            lg.assign(n + 1, 0);
            for (int i = 2; i <= n; i++)
            {
                lg[i] = lg[i / 2] + 1;
            }
            int z = n ? lg[n] + 1 : 0;
            mn.assign(z, vector<int>(n));
            mx.assign(z, vector<int>(n));
            if (n)
            {
                mn[0] = mx[0] = a;
            }
            for (int k = 1; k < z; k++)
            {
                for (int i = 0; i + (1LL << k) <= n; i++)
                {
                    mn[k][i] = min(mn[k - 1][i], mn[k - 1][i + (1LL << (k - 1))]);
                    mx[k][i] = max(mx[k - 1][i], mx[k - 1][i + (1LL << (k - 1))]);
                }
            }
        }

        pair<int, int> query(int l, int r) const
        {
            // l、r 是非空半开区间端点；返回该区间的 {最小值, 最大值}。
            int k = lg[r - l];
            int d = r - (1LL << k);
            return {min(mn[k][l], mn[k][d]), max(mx[k][l], mx[k][d])};
        }
    };

    struct Seg
    {
        int n;                // n 保存叶子个数。
        vector<__int128> mn, tag; // mn、tag 保存区间最小值与区间加懒标记。

        Seg(int n) : n(n), mn(4 * n + 4), tag(4 * n + 4)
        {
            // n 是排列长度；初始 q 全为 0。
        }

        void apply(int u, __int128 v)
        {
            // u 是线段树结点，v 是增量；给整段加 v。
            mn[u] += v;
            tag[u] += v;
        }

```

```
        void push(int u)
        {
            // u 是线段树结点；把区间加标记下传给两个儿子。
            if (tag[u])
            {
                apply(u * 2, tag[u]);
                apply(u * 2 + 1, tag[u]);
                tag[u] = 0;
            }
        }

        void add(int u, int l, int r, int ql, int qr, __int128 v)
        {
            // u 管理 [l,r)，给 [ql,qr) 加 v。
            if (ql <= l && r <= qr)
            {
                apply(u, v);
                return;
            }
            push(u);
            int m = (l + r) / 2;
            if (ql < m)
            {
                add(u * 2, l, m, ql, qr, v);
            }
            if (m < qr)
            {
                add(u * 2 + 1, m, r, ql, qr, v);
            }
            mn[u] = min(mn[u * 2], mn[u * 2 + 1]);
        }

        void add(int l, int r, __int128 v)
        {
            // l、r 是半开区间端点；给 q[l,r) 加 v。
            if (l < r)
            {
                add(1, 0, n, l, r, v);
            }
        }

        int firstZero(int u, int l, int r)
        {
            // u 管理 [l,r)，且段内最小值为 0；返回最左的零位置。
            if (r - l == 1)
            {
                return l;
            }
            push(u);
            int m = (l + r) / 2;
            if (mn[u * 2] == 0)
            {
```

```
                return firstZero(u * 2, l, m);
            }
            return firstZero(u * 2 + 1, m, r);
        }

        int firstZero()
        {
            // 无参数；返回 q 中最左的零位置。
            assert(mn[1] == 0); // 调试检查，可删
            return firstZero(1, 0, n);
        }
    };

    vector<Node> t; // t 保存全部析合树结点。
    int root = -1;  // root 保存整段排列对应的根结点。

    PermTree(const vector<int> &a)
    {
        // a 是一段连续整数的排列；用增量析合森林构造析合树。
        int n = a.size();
        if (!n)
        {
            return;
        }
        vector<int> chk = a;
        sort(chk.begin(), chk.end());
        assert(unique(chk.begin(), chk.end()) == chk.end()); // 调试检查，可删
        assert((__int128)chk.back() - (__int128)chk[0] == n - 1); // 调试检查，可删
        RMQ rmq(a);
        Seg seg(n);
        vector<int> low, high, forest;

        auto common = [&](int l, int r)
        {
            auto [mn, mx] = rmq.query(l, r);
            return (__int128)mx - (__int128)mn == r - l - 1;
        };
        auto node = [&](int l, int r, int type)
        {
            t.push_back({l, r, type, -1, -1, {}});
            return (int)t.size() - 1;
        };
        auto link = [&](int u, int v)
        {
            t[u].ch.push_back(v);
            t[v].par = u;
        };

        for (int i = 0; i < n; i++)
        {
            int lp = low.empty() ? 0 : low.back() + 1;
            while (!low.empty() && a[i] <= a[low.back()])
```

```
            {
                int p = low.back();
                low.pop_back();
                int l = low.empty() ? 0 : low.back() + 1;
                seg.add(l, p + 1, (__int128)a[p]);
                lp = l;
            }
            seg.add(lp, i + 1, -(__int128)a[i]);
            low.push_back(i);

            lp = high.empty() ? 0 : high.back() + 1;
            while (!high.empty() && a[i] >= a[high.back()])
            {
                int p = high.back();
                high.pop_back();
                int l = high.empty() ? 0 : high.back() + 1;
                seg.add(l, p + 1, -(__int128)a[p]);
                lp = l;
            }
            seg.add(lp, i + 1, (__int128)a[i]);
            high.push_back(i);

            int now = node(i, i + 1, 0);
            int le = seg.firstZero();
            while (!forest.empty() && t[forest.back()].l >= le)
            {
                int x = forest.back();
                if (t[x].type == 2 && common(t[x].mid, i + 1))
                {
                    forest.pop_back();
                    t[x].r = i + 1;
                    t[x].mid = t[now].l;
                    link(x, now);
                    now = x;
                }
                else if (common(t[x].l, i + 1))
                {
                    forest.pop_back();
                    int u = node(t[x].l, i + 1, 2);
                    t[u].mid = t[now].l;
                    link(u, x);
                    link(u, now);
                    now = u;
                }
                else
                {
                    int u = node(0, i + 1, 1);
                    link(u, now);
                    while (true)
                    {
                        x = forest.back();
                        forest.pop_back();
```

```
                        link(u, x);
                        if (common(t[x].l, i + 1))
                        {
                            break;
                        }
                    }
                    t[u].l = t[x].l;
                    now = u;
                }
            }
            forest.push_back(now);
            seg.add(0, i + 1, -1);
        }
        assert(forest.size() == 1); // 调试检查，可删
        root = forest[0];
        for (auto &x : t)
        {
            sort(x.ch.begin(), x.ch.end(), [&](int u, int v)
            {
                return t[u].l < t[v].l;
            });
        }
    }
};
```

构造 `PermTree tr(a)` 后，`tr.t[u]` 给出强区间 $[l,r)$、类型、父亲和按位置排列的孩子，`tr.root` 是整段排列；空排列得到 `root=-1` 和空结点表，非空输入的值不必从 0 开始，但必须互异并恰好构成连续整数（包括从 `LLONG_MIN` 开始的连续排列）。稀疏表预处理为 $O(n\log n)$ 时间和空间，每次查询 $O(1)$；线段树操作合计 $O(n\log n)$ 时间、占 $O(n)$ 空间，析合森林总共只压入、弹出线性次。因此整个构造为 $O(n\log n)$ 时间、$O(n\log n)$ 空间。合点的连续若干孩子仍可组成连续段，析点则只能取整个结点；需要统计全部连续段时按这个区别做树上 DP。

3.7.009 动态点集矩形仿射修改与矩形和

把初始点和以后可能插入的点坐标全部离线收集，用 `optional` 空值表示尚未激活的点。平衡 K-D Tree 每个子树维护包围盒、激活点数、权值和与仿射懒标记；矩形整体覆盖子树时，$w\leftarrow aw+b$ 使子树和变成 $a\cdot sum+b\cdot cnt$。未来点激活前会先下传祖先标记再覆盖叶子，因此不会错误继承它插入以前的修改。

```
template <class Coord = long long, class T = long long>
struct KDRectAffine
{
    struct Node
    {
        Coord xl{}, xr{}, yl{}, yr{}; // 子树内全部点的坐标包围盒（闭区间）。
        int cnt = 0; // cnt 是子树内已激活点数。
        T sum = 0, mul = 1, add = 0; // sum 是激活点权值和，mul、add 是未下传的 w->mul*w+add 标记。
    };

    int n; // n 是预收集的点数。
```

```
    vector<Coord> x, y; // x[id]、y[id] 是第 id 个点的坐标。
    vector<int> ord, pos; // ord 是建树时的点序，pos[id] 是点 id 所在叶子的节点编号。
    vector<Node> tr; // tr 是按堆式编号存放的 KD 树节点。

    KDRectAffine(vector<Coord> x, vector<Coord> y, const vector<optional<T>> &w)
        : n(x.size()), x(move(x)), y(move(y)), ord(n), pos(n), tr(max<int>(1, 4 * n))
    {
        // x、y 是全部已知点坐标，w[id] 为空表示该点尚未激活。
        assert((int)this->y.size() == n && (int)w.size() == n); // 调试检查，可删。
        iota(ord.begin(), ord.end(), 0);
        if (n)
        {
            build(1, 0, n, 0, w);
        }
    }

    void pull(int k)
    {
        // k 是内部节点；由两个儿子重算包围盒、激活点数与权值和。
        auto &a = tr[2 * k];
        auto &b = tr[2 * k + 1];
        tr[k].xl = min(a.xl, b.xl);
        tr[k].xr = max(a.xr, b.xr);
        tr[k].yl = min(a.yl, b.yl);
        tr[k].yr = max(a.yr, b.yr);
        tr[k].cnt = a.cnt + b.cnt;
        tr[k].sum = a.sum + b.sum;
    }

    void build(int k, int l, int r, int d, const vector<optional<T>> &w)
    {
        // k 管辖 ord[l..r)，d 是切分维度（0 按 x、1 按 y），w 是初始权值；递归建 KD 树。
        if (r - l == 1)
        {
            int id = ord[l];
            pos[id] = k;
            tr[k].xl = tr[k].xr = x[id];
            tr[k].yl = tr[k].yr = y[id];
            if (w[id])
            {
                tr[k].cnt = 1;
                tr[k].sum = *w[id];
            }
            return;
        }
        int m = (l + r) / 2;
        nth_element(ord.begin() + l, ord.begin() + m, ord.begin() + r, [&](int a, int b)
        {
            return d == 0 ? x[a] < x[b] : y[a] < y[b];
        });
        build(2 * k, l, m, d ^ 1, w);
        build(2 * k + 1, m, r, d ^ 1, w);
```

```
64          pull(k);
65      }
66
67      void applyAt(int k, T a, T b)
68      {
69          // k 是节点；对其子树内激活点执行 w=a*w+b，并复合到延迟标记。
70          tr[k].sum = a * tr[k].sum + b * T(tr[k].cnt);
71          tr[k].mul = a * tr[k].mul;
72          tr[k].add = a * tr[k].add + b;
73      }
74
75      void push(int k)
76      {
77          // k 是内部节点；把延迟标记下传给两个儿子。
78          if (tr[k].mul == T(1) && tr[k].add == T(0))
79          {
80              return;
81          }
82          applyAt(2 * k, tr[k].mul, tr[k].add);
83          applyAt(2 * k + 1, tr[k].mul, tr[k].add);
84          tr[k].mul = T(1);
85          tr[k].add = T(0);
86      }
87
88      bool disjoint(int k, Coord xl, Coord xr, Coord yl, Coord yr) const
89      {
90          // 返回节点 k 的包围盒是否与半开矩形 [xl,xr)x[yl,yr) 不相交。
91          return xr <= tr[k].xl || tr[k].xr < xl || yr <= tr[k].yl || tr[k].yr < yl;
92      }
93
94      bool covered(int k, Coord xl, Coord xr, Coord yl, Coord yr) const
95      {
96          // 返回节点 k 的包围盒是否完全落在半开矩形 [xl,xr)x[yl,yr) 内。
97          return xl <= tr[k].xl && tr[k].xr < xr && yl <= tr[k].yl && tr[k].yr < yr;
98      }
99
100     T query(int k, Coord xl, Coord xr, Coord yl, Coord yr)
101     {
102         // k 是当前节点；返回其子树内落在半开矩形中的激活点权值和。
103         if (disjoint(k, xl, xr, yl, yr))
104         {
105             return T(0);
106         }
107         if (covered(k, xl, xr, yl, yr))
108         {
109             return tr[k].sum;
110         }
111         push(k);
112         return query(2 * k, xl, xr, yl, yr) + query(2 * k + 1, xl, xr, yl, yr);
113     }
114
115     void apply(int k, Coord xl, Coord xr, Coord yl, Coord yr, T a, T b)
```

```
    {
        // k 是当前节点；对其子树内落在半开矩形中的激活点执行 w=a*w+b。
        if (disjoint(k, xl, xr, yl, yr))
        {
            return;
        }
        if (covered(k, xl, xr, yl, yr))
        {
            applyAt(k, a, b);
            return;
        }
        push(k);
        apply(2 * k, xl, xr, yl, yr, a, b);
        apply(2 * k + 1, xl, xr, yl, yr, a, b);
        pull(k);
    }

    void set(int id, T w)
    {
        // id 是预收集点编号，w 是当前新权值；激活或覆盖该点。
        assert(0 <= id && id < n); // 调试检查，可删。
        int k = pos[id];
        vector<int> path;
        for (int z = k / 2; z; z /= 2)
        {
            path.push_back(z);
        }
        reverse(path.begin(), path.end());
        for (int z : path)
        {
            push(z);
        }
        tr[k].cnt = 1;
        tr[k].sum = w;
        tr[k].mul = T(1);
        tr[k].add = T(0);
        for (k /= 2; k; k /= 2)
        {
            pull(k);
        }
    }

    T query(Coord xl, Coord xr, Coord yl, Coord yr)
    {
        // 四个边界给出半开矩形；返回全部激活点权值和。
        assert(xl <= xr && yl <= yr); // 调试检查，可删。
        return n ? query(1, xl, xr, yl, yr) : T(0);
    }

    void apply(Coord xl, Coord xr, Coord yl, Coord yr, T a, T b)
    {
        // 四个边界给出半开矩形；对其中激活点执行 w=a*w+b。
```

```
        assert(xl <= xr && yl <= yr); // 调试检查，可删。
        if (n)
        {
            apply(1, xl, xr, yl, yr, a, b);
        }
    }
};
```

构造时必须预收集全部点编号及坐标，初始未出现的点令 `w[id]=nullopt`；随后 `set(id,w)` 激活或重设单点，`apply(xl,xr,yl,yr,a,b)` 修改半开矩形，`query` 求和。重复坐标按不同编号独立保存并一起受矩形操作影响。`T` 必须容纳仿射乘加与子树和，模意义题直接使用模整数。建树期望 $O(n\log n)$、单点设置 $O(\log n)$、空间 $O(n)$；矩形操作依赖包围盒剪枝，确定性最坏 $O(n)$，普通分布下通常接近 $O(\sqrt{n})$，具体常数会随数据和平台变化。

下面用四个点覆盖重复坐标、延迟激活与半开边界：

```
signed main()
{
    vector<long long> x{0, 1, 1, 3}, y{0, 1, 1, 0};
    vector<optional<long long>> w{2, nullopt, 5, 4};
    KDRectAffine tr(x, y, w);
    assert(tr.query(0, 2, 0, 2) == 7);
    tr.apply(0, 2, 0, 2, 2, 1);
    assert(tr.query(0, 2, 0, 2) == 16);
    tr.set(1, 3);
    assert(tr.query(0, 2, 0, 2) == 19);
    tr.apply(1, 2, 1, 2, -1, 4);
    assert(tr.query(0, 2, 0, 2) == -1);
    assert(tr.query(3, 4, 0, 1) == 4);
}
```

3.7.010 离散坐标线段李超树（线段插入、点查询）

若直线只在自己的横坐标半开区间 $[l,r)$ 内有效，先离线收集所有点查询的横坐标并排序去重。把每条线段映射为这些坐标中的连续下标段 $[\text{lower_bound}(l),\text{lower_bound}(r))$；在线段树上递归时，完整覆盖的节点按普通李超规则放一条“在该节点中点最优”的直线。被换下的直线只会在一个儿子区间翻盘，故继续向那个儿子下传即可。所有节点共用同一棵离散线段树，每个节点至多存一条线，不为每个区间额外分配一棵李超树：

```
template <class T, class C = T> class SegLiChao
{
    // 当前实现维护最小值。改最大值可直接反向所有优劣比较并把空值改为负无穷，
    // 或在系数与函数值能安全取反时对每条线取反后复用本实现，查询非空结果再取反。
  public:
    static_assert(is_floating_point_v<T> || (is_integral_v<T> && is_signed_v<T>));

    struct Line
    {
        // k、b 是一次函数 y=kx+b 的斜率与截距。
```

```
        T k{}, b{};

        C get(T x) const
        {
            // x 是查询横坐标；用比较类型 C 计算函数值。
            return (C)k * x + b;
        }
    };

  private:
    // xs 是全部允许查询的横坐标，tr[p] 是节点 p 上在中点占优且整段有效的直线。
    vector<T> xs;
    vector<optional<Line>> tr;

    void addLine(int p, int l, int r, Line f)
    {
        // p 是节点编号，[l,r) 是横坐标下标段，f 已在整段内有效；把 f 加入该段。
        if (!tr[p])
        {
            tr[p] = f;
            return;
        }
        int m = midpoint(l, r);
        bool lef = f.get(xs[l]) < tr[p]->get(xs[l]);
        bool mid = f.get(xs[m]) < tr[p]->get(xs[m]);
        if (mid)
        {
            swap(f, *tr[p]);
        }
        if (r - l == 1)
        {
            return;
        }
        if (lef != mid)
        {
            addLine(p << 1, l, m, f);
        }
        else
        {
            addLine(p << 1 | 1, m, r, f);
        }
    }

    void addSegment(int p, int l, int r, int ql, int qr, Line f)
    {
        // p、[l,r) 是当前节点，[ql,qr) 是 f 有效的离散下标段；插入二者交集。
        if (qr <= l || r <= ql)
        {
            return;
        }
        if (ql <= l && r <= qr)
        {
```

```
            addLine(p, l, r, f);
            return;
        }
        int m = midpoint(l, r);
        addSegment(p << 1, l, m, ql, qr, f);
        addSegment(p << 1 | 1, m, r, ql, qr, f);
    }

public:
    SegLiChao(vector<T> xs) : xs(move(xs))
    {
        // xs 是全部可能查询的横坐标；排序去重并建立最小值线段李超树。
        // 最大值版可对插入线的 k、b 取反，并对 query 返回值取反。
        sort(this->xs.begin(), this->xs.end());
        this->xs.erase(unique(this->xs.begin(), this->xs.end()), this->xs.end());
        assert(!this->xs.empty()); // 调试检查，可删
        tr.resize(4 * this->xs.size());
    }

    void add(Line f, T l, T r)
    {
        // f 是待插入直线，[l,r) 是其按真实横坐标定义的有效半开区间。
        assert(l < r); // 调试检查，可删
        int ql = lower_bound(xs.begin(), xs.end(), l) - xs.begin();
        int qr = lower_bound(xs.begin(), xs.end(), r) - xs.begin();
        if (ql < qr)
        {
            addSegment(1, 0, xs.size(), ql, qr, f);
        }
    }

    optional<C> query(T x) const
    {
        // x 必须在构造坐标中；返回所有在 x 有效直线的最小值，无线时返回空。
        // 采用取反线的最大值版仍保留无线时的 nullopt 语义。
        int q = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        assert(q < (int)xs.size() && xs[q] == x); // 调试检查，可删
        optional<C> ans;
        int p = 1, l = 0, r = xs.size();
        while (true)
        {
            if (tr[p])
            {
                C y = tr[p]->get(x);
                if (!ans || y < *ans)
                {
                    ans = y;
                }
            }
            if (r - l == 1)
            {
                break;
```

```
            }
            int m = midpoint(l, r);
            if (q < m)
            {
                p <<= 1;
                r = m;
            }
            else
            {
                p = p << 1 | 1;
                l = m;
            }
        }
        return ans;
    }
};
```

先收集至少一个可能被 query 的横坐标，以 `SegLiChao<T,C> lc(xs)` 构造；`lc.add({k,b},l,r)` 插入仅在真实区间 $[l,r)$ 有效的 $y=kx+b$，`lc.query(x)` 返回 `optional<C>`，没有有效线段时为 `nullopt`。最大值版对每条线段的 `k,b` 取反后按相同有效区间插入，并对非空查询结果取反；有效域和空值语义不变。端点不需要出现在 `xs` 中；若 $[l,r)$ 内没有任何查询坐标，本次插入自动忽略。T 是横坐标与系数类型，C 是函数值类型；积可能超过 T 时用 `SegLiChao<long long,__int128>`。对 $m=|xs|$，一次插入为 $O(\log^2 m)$，点查询为 $O(\log m)$，空间为 $O(m)$；前者来自区间分解的 $O(\log m)$ 个完整节点及每次李超下沉。若所有直线都在完整定义域有效，使用更短的 3.7.001 李超线段树；横坐标不能离线收集时改用动态开点结构并为线段有效域补充动态区间插入；需要历史版本时再扩展 3.7.003 可持久化李超树。

3.8 线性基

3.8.001 异或线性基

线性基把一组整数在异或（$\mathbb{F}_2$ 上的向量加法）意义下的张成空间压成至多 W 个主元向量：插入 x 时从最高位往低扫，遇到某位已有主元就异或消去，否则让 x 占据这个主元位并使秩加一；最终非零主元张成的空间与原集合相同。于是能判定某值可否表示（`has`）、求与初值异或的最大值（`maxXor`，逐位贪心取更大者）、求秩，以及在约化后按排名取第 k 小可表示值（`kth`）：

```
template <int W = 64> struct XorBasis
{
    static_assert(1 <= W && W <= 64);

    static bool valid(unsigned long long x)
    {
        // x 是待处理的整数；返回它是否只使用低 W 位。
        if constexpr (W == 64)
        {
            return true;
        }
        else
        {
```

```
            return x < (1ULL << W);
        }
    }

    // a[i] 是最高位为 i 的主元，rk 是当前秩。
    array<unsigned long long, W> a{};
    int rk = 0;

    bool add(unsigned long long x)
    {
        // x 是待插入向量；返回它是否使秩增加。
        assert(valid(x)); // 调试检查，可删。
        for (int i = W - 1; i >= 0; i--)
        {
            if (x >> i & 1)
            {
                if (!a[i])
                {
                    a[i] = x;
                    rk++;
                    return true;
                }
                x ^= a[i];
            }
        }
        return false;
    }

    bool has(unsigned long long x) const
    {
        // x 是查询向量；返回它能否由当前基表示。
        assert(valid(x)); // 调试检查，可删。
        for (int i = W - 1; i >= 0; i--)
        {
            if ((x >> i & 1) && a[i])
            {
                x ^= a[i];
            }
        }
        return x == 0;
    }

    unsigned long long maxXor(unsigned long long x = 0) const
    {
        // x 是初值；返回 x 与张成空间中某值异或后的最大结果。
        assert(valid(x)); // 调试检查，可删。
        for (int i = W - 1; i >= 0; i--)
        {
            chmax(x, x ^ a[i]);
        }
        return x;
    }
```

```
66
67      void merge(const XorBasis &o)
68      {
69          // o 是另一组基；把它的张成空间并入当前基。
70          for (auto x : o.a)
71          {
72              if (x)
73              {
74                  add(x);
75              }
76          }
77      }
78
79      optional<unsigned long long> kth(unsigned long long k) const
80      {
81          // k 是 0 基排名；返回第 k 小不同可表示值，越界返回空。
82          if (rk < 64 && k >= (1ULL << rk))
83          {
84              return nullopt;
85          }
86          auto b = a;
87          for (int i = 0; i < W; i++)
88          {
89              if (!b[i])
90              {
91                  continue;
92              }
93              for (int j = i + 1; j < W; j++)
94              {
95                  if (b[j] >> i & 1)
96                  {
97                      b[j] ^= b[i];
98                  }
99              }
100         }
101         vector<unsigned long long> v;
102         for (auto x : b)
103         {
104             if (x)
105             {
106                 v.push_back(x);
107             }
108         }
109         unsigned long long ans = 0;
110         for (int i = 0; i < (int)v.size(); i++)
111         {
112             if (k >> i & 1)
113             {
114                 ans ^= v[i];
115             }
116         }
117         return ans;
```

```
    }
};
```

默认 `XorBasis<> xb`（$W = 64$ 位）：`add(x)` 插入并返回是否使秩增加，`has(x)` 判可表示性，`maxXor(x)` 求与初值 x 异或的最大值，`rk` 是当前秩，`merge(o)` 并入另一组基，`kth(k)`（k 从 0 起，含 0）返回第 k 小不同可表示值、越界返回 `nullopt`。当 $W < 64$ 时，`add`、`has`、`maxXor` 以及并入的主元都必须在 $[0, 2^W)$ 内，代码以 `assert` 检查；$W = 64$ 时允许任意 `unsigned long long`。`add`、`has`、`maxXor` 均 $O(W)$，`kth`/`merge` 为 $O(W^2)$，空间 $O(W)$。只需第 k 大且从裸序列一次算出用 3.8.002 第 K 大线性基；带下标区间限制用 3.8.003 区间线性基；树上/分治合并两组基用 3.8.004 线性基合并。

下面固定检查线性相关向量、可表示性、排名、合并，以及秩为 64 时的边界：

```
signed main()
{
    XorBasis<4> a;
    assert(a.add(3));
    assert(a.add(5));
    assert(!a.add(6));
    assert(a.rk == 2 && a.has(6) && !a.has(1));
    assert(a.maxXor() == 6);
    array<unsigned long long, 4> want{0, 3, 5, 6};
    for (int k = 0; k < 4; k++)
    {
        assert(a.kth(k) == optional<unsigned long long>(want[k]));
    }
    assert(!a.kth(4));

    XorBasis<4> b;
    b.add(8);
    a.merge(b);
    assert(a.rk == 3 && a.maxXor() == 14);
    assert(a.kth(5) == optional<unsigned long long>(11));

    XorBasis<> full;
    for (int i = 0; i < 64; i++)
    {
        full.add(1ULL << i);
    }
    assert(full.rk == 64);
    assert(full.kth(ULLONG_MAX) == optional<unsigned long long>(ULLONG_MAX));
    cout << "2 6 0 3 5 6 3 14 11 18446744073709551615\n";
    return 0;
}
```

3.8.002 第 K 大线性基

要按大小有序枚举可表示的异或值，先把线性基化成约化阶梯形——让每个主元位只在一个主元中出现为 1。这样把非零主元从低位到高位排好后，它们两两「互不影响高位」，于是排名 k 的二进制各位恰好对应「选或不选」每个主元，异或起来就是第 k 小可表示值；求第 k 大只需把 k 换成 $2^{rk} - 1 - k$。本条把「插

入建基 + 约化 + 按排名取值」打包成一个独立函数：

```cpp
template <int W = 64>
optional<unsigned long long> kthXor(const vector<unsigned long long> &a, unsigned long long k,
    ↪ bool largest = false)
{
    // a 是生成空间的整数集合，k 从 0 开始，largest 表示按从大到小排名；返回第 k 个不同异或值，越界返回
    ↪ 空。
    static_assert(1 <= W && W <= 64);
    array<unsigned long long, W> b{};
    int rk = 0;
    for (auto x : a)
    {
        if constexpr (W < 64)
        {
            assert(x < (1ULL << W)); // 调试检查，可删。
        }
        for (int i = W - 1; i >= 0; i--)
        {
            if (!(x >> i & 1))
            {
                continue;
            }
            if (b[i])
            {
                x ^= b[i];
            }
            else
            {
                b[i] = x;
                rk++;
                break;
            }
        }
    }
    if (rk < 64 && k >= (1ULL << rk))
    {
        return nullopt;
    }
    for (int i = 0; i < W; i++)
    {
        if (!b[i])
        {
            continue;
        }
        for (int j = i + 1; j < W; j++)
        {
            if (b[j] >> i & 1)
            {
                b[j] ^= b[i];
            }
        }
```

```
    }
    if (largest)
    {
        k = (rk == 64 ? ULLONG_MAX : (1ULL << rk) - 1) - k;
    }
    unsigned long long ans = 0;
    int p = 0;
    for (int i = 0; i < W; i++)
    {
        if (!b[i])
        {
            continue;
        }
        if (k >> p & 1)
        {
            ans ^= b[i];
        }
        p++;
    }
    return ans;
}
```

`kthXor(a,k)` 从整数集合 `a` 求第 k 小不同异或值，`kthXor(a,k,true)` 求第 k 大；k 从 0 起，越界返回 `nullopt`，线性相关的输入不增加可表示值个数。注意 $k=0$ 对应最小值 0。当 $W<64$ 时，`a` 的每项必须在 $[0,2^W)$ 内，代码以 `assert` 检查；$W=64$ 时允许任意 `unsigned long long`。例如 `a={1,2,3}` 可表示 $\{0,1,2,3\}$：第 2 小（$k=2$）为 2，第 1 大（`largest`, $k=1$）为 2，$k=4$ 返回空。总时间 $O(nW+W^2)$、额外空间 $O(W)$。若还要反复插入/查询同一组基，改用维护状态的 3.8.001 异或线性基 的 `kth` 成员，避免每次重建。

3.8.003 区间线性基

要回答「下标区间 $[l,r]$ 内元素能异或出的最大值」，给每个主元额外记它当前来自哪个下标位置：对每个右端点保存一份前缀线性基，插入新元素时贪心让「出现位置更靠右」的向量占据主元（把旧主元换下继续消元）。查询时在版本 r 的基上只采纳「位置 $\geq l$」的主元做最大异或贪心，就等价于只用区间内元素建的基：

```
template <class T, int Bits = numeric_limits<T>::digits> class RangeBasis
{
    static_assert(is_integral_v<T> && is_unsigned_v<T>);
    static_assert(1 <= Bits && Bits <= numeric_limits<T>::digits);

    struct Entry
    {
        // v 是主元，p 是它在前缀基中保留的最靠右来源位置。
        T v{};
        int p = -1;
    };

    // pre[r] 是扫描到下标 r 后的带位置线性基。
```

```
    vector<array<Entry, Bits>> pre;

    static bool valid(T x)
    {
        // x 是待处理的无符号整数；返回它是否只使用低 Bits 位。
        if constexpr (Bits == numeric_limits<T>::digits)
        {
            return true;
        }
        else
        {
            return !(x >> Bits);
        }
    }

  public:
    explicit RangeBasis(const vector<T> &a) : pre(a.size())
    {
        // a 是静态无符号整数序列；建立每个右端点的前缀基。
        for (int i = 0; i < (int)a.size(); i++)
        {
            assert(valid(a[i])); // 调试检查，可删。
            if (i)
            {
                pre[i] = pre[i - 1];
            }
            T v = a[i];
            int p = i;
            for (int b = Bits - 1; b >= 0 && v; b--)
            {
                if (!(v >> b & 1))
                {
                    continue;
                }
                auto &e = pre[i][b];
                if (e.p < p)
                {
                    swap(e.v, v);
                    swap(e.p, p);
                }
                v ^= e.v;
            }
        }
    }

    T maxXor(int l, int r, T x = 0) const
    {
        // l、r 是闭区间端点，x 是初值；返回可取得的最大异或值。
        assert(0 <= l && l <= r && r < (int)pre.size()); // 调试检查，可删
        assert(valid(x)); // 调试检查，可删。
        for (int b = Bits - 1; b >= 0; b--)
        {
```

```
            auto [v, p] = pre[r][b];
            if (p >= l && (x ^ v) > x)
            {
                x ^= v;
            }
        }
        return x;
    }
};
```

用无符号整数序列构造 `RangeBasis<T> rb(a)`，`rb.maxXor(l,r,x)` 返回闭区间 $[l,r]$ 内元素与初值 x（默认 0）异或的最大值，要求 $0 \le l \le r < n$。有符号类型不满足按位右移与无符号大小比较的语义，已由静态断言禁止。当 `Bits` 小于 `T` 的有效位数时，`a` 的每项与 `x` 都必须在 $[0, 2^{Bits})$ 内，构造和查询会以 `assert` 检查；取满位宽时允许任意 `T` 值。构造 $O(nW)$ 时间与空间，单次查询 $O(W)$。无下标限制时用 3.8.001 异或线性基，离线带修改时用线段树分治套线性基。

3.8.004 线性基合并

合并两个子空间的线性基，只需把第二组的每个主元依次插入第一组：插入时的消元会自动丢弃线性相关的向量，最终张成空间恰为两者子空间之和。常用于树上启发式合并、线段树/分治节点自底向上归并各区间的线性基：

```
template <int W = 64>
array<unsigned long long, W> mergeBasis(array<unsigned long long, W> a, const array<unsigned
↪ long long, W> &b)
{
    // a、b 都按最高位保存一组异或线性基主元；返回两者张成空间之和的线性基。
    static_assert(1 <= W && W <= 64);
    if constexpr (W < 64)
    {
        for (auto x : a)
        {
            assert(x < (1ULL << W)); // 调试检查，可删。
        }
    }
    for (auto x : b)
    {
        if constexpr (W < 64)
        {
            assert(x < (1ULL << W)); // 调试检查，可删。
        }
        for (int i = W - 1; i >= 0; i--)
        {
            if (!(x >> i & 1))
            {
                continue;
            }
            if (a[i])
            {
```

```
                x ^= a[i];
            }
            else
            {
                a[i] = x;
                break;
            }
        }
    }
    return a;
}
```

两组基都用 `array<unsigned long long,W>`（下标 i 保存最高位为 i 的主元，与 3.8.001 异或线性基 的 `a` 布局一致），调用 `mergeBasis(a,b)` 返回并空间的基，两个输入数组本身不被修改（`a` 按值传入）。当 $W < 64$ 时，两个数组的每个值都必须在 $[0, 2^W)$ 内，代码以 `assert` 检查；$W = 64$ 时允许任意 `unsigned long long`。$O(W^2)$ 时间、$O(W)$ 返回空间。这与 3.8.001 异或线性基 的成员 `merge` 等价，区别只是这里操作裸数组、便于在没有对象包装的分治节点上直接合并。

3.9 GNU PBDS

3.9.001 PBDS 头文件、命名空间、模板策略与迭代器体系

GNU PBDS 的容器类型由底层标签（`rb_tree_tag`、各种堆 tag 等）、比较器和节点更新策略三者共同拼出，使用前必须显式包含 `ext/pb_ds` 下对应的头文件，且要区分两类迭代器：`tree`/哈希表的普通迭代器与堆的“点迭代器”能力不同，后者代表一个具体节点、可用于 `modify`/`erase`，不能与普通容器迭代器混用。下面的示例把有序树封在语义命名空间里，用它同时求排名与后继。

```
// 知识点 3.9.001：PBDS 头文件、命名空间、模板策略与迭代器体系。
namespace pbds_ordered
{
using namespace __gnu_pbds;

template <class T> using OrdSet = tree<T, null_type, less<T>, rb_tree_tag,
 ↪  tree_order_statistics_node_update>;

array<int, 2> rankInfo(const vector<int> &a, int x)
{
    // a 是待插入值，x 是查询值；返回严格小于 x 的数量与第一个不小于 x 的值，无后继时第二维为 -1。
    OrdSet<pair<int, int>> tr;
    for (int i = 0; i < (int)a.size(); i++)
    {
        tr.insert({a[i], i});
    }
    int rk = tr.order_of_key({x, -1});
    auto it = tr.lower_bound({x, -1});
    return {rk, it == tr.end() ? -1 : it->first};
}
}
```

PBDS 是 GNU libstdc++ 扩展。第 3 章公共前导已在 `#define int long long` 之前包含以下三个头文件：

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/priority_queue.hpp>
#include <ext/pb_ds/tree_policy.hpp>
```

它们分别提供 `tree`/哈希表、PBDS 堆、`tree_order_statistics_node_update`。切忌全局 `using namespace __gnu_pbds`，否则其 `priority_queue` 会与 STL 同名冲突，故示例都封在命名空间里、按 `pbds_ordered::rankInfo(a,x)` 这样限定调用；Clang 配 libc++ 或 MSVC 不能直接用这些头。插入与排名查询最坏 $O(\log n)$，空间 $O(n)$。后续 3.9.002–3.9.006 分述各种 tag，3.9.007 tree_order_statistics_node_update 与 3.9.008 find_by_order 与 order_of_key 给出排名/第 k 小的最小接口，3.9.011 gp_hash_table 讨论哈希容器。

3.9.002 PBDS `priority_queue` 策略

PBDS 的 `priority_queue` 可换五种底层堆标签，它们在合并、改键和常数上各有取舍：`binary_heap_tag` 常数最小但不支持高效合并；`binomial_heap_tag`、`rc_binomial_heap_tag` 支持对数级合并；`pairing_heap_tag` 的 `push`、`join` 均摊 $O(1)$，`modify` 均摊 $O(\log n)$，是需要句柄修改与合并时的默认选择；`thin_heap_tag` 在只减键的场景（如 Dijkstra）表现好。纯取堆顶而无需合并或改键时，STL 二叉堆通常更快。

```
// 知识点 3.9.002：PBDS priority_queue 的五种策略标签。
namespace pbdsHeap
{
using namespace __gnu_pbds;

template <class T> using BinaryHeap = __gnu_pbds::priority_queue<T, less<T>, binary_heap_tag>;
template <class T> using BinomialHeap = __gnu_pbds::priority_queue<T, less<T>,
↪ binomial_heap_tag>;
template <class T> using RcBinHeap = __gnu_pbds::priority_queue<T, less<T>,
↪ rc_binomial_heap_tag>;
template <class T> using PairingHeap = __gnu_pbds::priority_queue<T, less<T>,
↪ pairing_heap_tag>;
template <class T> using ThinHeap = __gnu_pbds::priority_queue<T, less<T>, thin_heap_tag>;

int heapTop(const vector<int> &a)
{
    // a 是初始键值；用默认推荐的 pairing heap 返回最大值，空表返回 -1。
    PairingHeap<int> hp;
    for (int x : a)
    {
        hp.push(x);
    }
    return hp.empty() ? -1 : hp.top();
}
}
```

`pbdsHeap::heapTop(a)` 用推荐的 pairing heap 建堆并取最大值，空表返回 `-1`；换 tag 只需改 `using` 别名。示例建堆 $O(n \log n)$、取堆顶 $O(1)$，各 tag 的 `modify`/`join` 界以 GNU PBDS 实现为准。要真正用

上点迭代器改键、删点、合并见 3.9.003 PBDS 堆点迭代器；只取堆顶时直接用 STL `priority_queue`。

3.9.003 PBDS 堆的 `modify`、`erase`、`join` 与点迭代器

PBDS 堆相对 STL 堆的核心增值在于点迭代器：`push` 返回的句柄代表一个具体节点，据此可 `modify(it, 新值)` 改键或 `erase(it)` 删除任意元素，而 `join(b)` 把另一堆的全部节点并入当前堆、并使 `b` 变空。句柄一旦对应节点被删就失效，改键/删点务必用仍有效的句柄。

```cpp
// 知识点 3.9.003：PBDS 堆的点迭代器、modify、erase 与 join。
namespace pbdsMutHeap
{
using namespace __gnu_pbds;

using Heap = __gnu_pbds::priority_queue<int, less<int>, pairing_heap_tag>;

int editHeap()
{
    // 演示点迭代器改键、删除与合并，并返回最终堆顶。
    Heap a, b;
    auto p = a.push(10);
    auto q = a.push(30);
    a.modify(p, 40);
    a.erase(q);
    b.push(25);
    a.join(b);
    return a.top();
}
}
```

`pbdsMutHeap::editHeap()` 走一遍改键、删点、合并后返回堆顶。pairing heap 的 `push`、`join` 均摊 $O(1)$，`modify`、`pop`、`erase` 均摊 $O(\log n)$，空间 $O(n)$。这套接口正是 Dijkstra 减键、堆式贪心合并等场景相对 STL 堆的优势所在；tag 的选型见 3.9.002 五种堆策略标签，与其他容器的取舍见 3.9.004 PBDS 堆与 STL 堆、线段树的选型。

3.9.004 PBDS 堆和 STL 堆、二叉堆、线段树的性能与适用范围

同是“取极值”的需求，STL `priority_queue` 接口稳定、常数最小，适合只做 push/pop/top；PBDS 堆的价值在可合并与句柄改键；手写二叉堆便于内联定制；线段树则适合按下标随机聚合与区间极值。选型看真正需要哪些操作，而不是只比理论复杂度——为用不到的 `modify`/`join` 付常数是常见浪费。

只做 push/pop/top 时直接使用 STL `priority_queue`；需要 `modify`、`erase` 或 `join` 时使用 3.9.003 PBDS 堆点迭代器；键具有固定下标并需要区间查询或修改时使用本章线段树。这里是接口选型，不另造比较函数；具体性能取决于操作比例、数据分布、编译器和平台，应在目标环境用实际数据单独基准。

3.9.005 `tree`：`rb_tree_tag`、`splay_tree_tag`、`ov_tree_tag`

PBDS `tree` 的底层平衡策略同样可换 tag：`rb_tree_tag` 红黑树给稳定的最坏对数界，是默认选择；`splay_tree_tag` 偏向访问局部性，适合热点集中的序列；`ov_tree_tag` 用连续向量存储换取查询紧凑，但插删最坏 $O(n)$，只宜读多写少。三种策略与节点更新策略（下面几条）互相独立，可任意组合。

```cpp
// 知识点 3.9.005：PBDS tree 的三种底层平衡策略。
namespace pbdsTree
{
using namespace __gnu_pbds;

template <class T> using RBTree = tree<T, null_type, less<T>, rb_tree_tag>;
template <class T> using SplayTree = tree<T, null_type, less<T>, splay_tree_tag>;
template <class T> using OVTree = tree<T, null_type, less<T>, ov_tree_tag>;
}
```

按操作特征选一个别名实例化即可，例如 `pbdsTree::RBTree<int> s`。红黑树操作最坏 $O(\log n)$，Splay 均摊 $O(\log n)$，ov_tree 查询 $O(\log n)$ 而插删最坏 $O(n)$。挂上 order-statistics 或自定义更新策略见 3.9.006 set/map/split/join、3.9.007 tree_order_statistics_node_update 与 3.9.009 自定义 Node_Update。

3.9.006 PBDS `tree` 的 set/map 用法、`split` 与 `join`

PBDS `tree` 把值类型设为 `null_type` 即是 set，设为具体类型即是 map，键须满足严格弱序。它比 STL `set`/`map` 多出 $O(\log n)$ 的整体 `split`/`join`：`split(x, b)` 把所有严格大于 `x` 的键搬到（必须为空的）`b`，`join(b)` 要求 `a` 的全部键严格小于 `b` 的全部键（键域完全分离）才合法，否则不满足接口前提。

```cpp
// 知识点 3.9.006：PBDS tree 的 set/map、split 与 join。
namespace pbdsSplit
{
using namespace __gnu_pbds;

using Map = tree<int, int, less<int>, rb_tree_tag>;

void splitAt(Map &a, Map &b, int x)
{
    // a 是原映射，b 必须为空，x 是分界键；把所有严格大于 x 的键移入 b。
    a.split(x, b);
}

void joinOrdered(Map &a, Map &b)
{
    // a、b 的键域必须严格分离且 a 的键全小于 b；把 b 合入 a 并清空 b。
    a.join(b);
}
}
```

`splitAt(a, b, x)` 要求 `b` 为空，`joinOrdered(a, b)` 要求键域分离且 `a` 全小于 `b`，两者都原地修改传入的树。红黑树插入、查找最坏 $O(\log n)$，`split`/`join` 的界依赖策略与键域前提，空间 $O(n)$。要顺带

维护子树大小以支持排名见 3.9.007 tree_order_statistics_node_update，要维护子树和等可合并信息见 3.9.009 自定义 Node_Update。

3.9.007 `tree_order_statistics_node_update`

把节点更新策略设为 `tree_order_statistics_node_update`，红黑树的每个节点就自动维护子树大小并在旋转后同步，从而开放 “按排名取值” 和 “统计严格小于某键的个数” 两个接口——这正是竞赛里最常用的 “权值平衡树” 配置，本书 3.5.007 等条目就直接借它当第 k 小结构。

```
template <class T>
using OSet = __gnu_pbds::
    tree<T, __gnu_pbds::null_type, less<T>, __gnu_pbds::rb_tree_tag,
    __gnu_pbds::tree_order_statistics_node_update>;
```

`OSet<T>` 的模板参数 `T` 是键类型，默认 `less<T>` 升序；`tree` 的键唯一，要存重复值须把键换成 `pair< 值, 唯一编号 >` 以人为区分。该类型依赖 GNU PBDS 与 `tree_policy.hpp`，insert、erase、order_of_key、find_by_order 均最坏 $O(\log n)$，空间 $O(n)$。定义好后按 3.9.008 find_by_order 与 order_of_key 做第 k 小与排名查询；要维护除大小以外的可合并信息见 3.9.009 自定义 Node_Update。

3.9.008 `find_by_order` 与 `order_of_key`

有了 3.9.007 的 `OSet` 后，两个成员即可完成排名类查询：`find_by_order(k)` 返回 0 下标第 k 小元素的迭代器、越界时等于 `end()`；`order_of_key(x)` 返回严格小于 `x` 的键数（即 `x` 的 0 基排名）。两者互为逆向，组合即 “查第 k 小” 与 “查某值排名”。

```
// 依赖 3.9.007 tree_order_statistics_node_update
OSet<int> tr;

// 直接在题目逻辑中执行 tr.insert(x)。
// tr.find_by_order(k) 返回 0 下标第 k 小的迭代器，越界时等于 tr.end()。
// tr.order_of_key(x) 返回严格小于 x 的键数。
```

类型定义取自 3.9.007 tree_order_statistics_node_update，本条只演示这两个查询接口，均最坏 $O(\log n)$，空间 $O(n)$。若查询需求超出 “大小/排名”（如子树和、区间和），改用 3.9.009 自定义 Node_Update 或 3.9.010 自定义子树信息 定制 metadata。

3.9.009 自定义 `Node_Update`、`metadata_type` 与节点迭代器

要维护子树大小以外的信息，就自定义一个 `Node_Update` 仿函数：声明它的 `metadata_type`，并实现 `operator()(it, end)`——PBDS 会在插入、删除、旋转后对受影响节点回调它，函数内通过节点迭代器 `get_l_child`/`get_r_child`/`get_metadata` 读左右儿子的 metadata，按 “当前键 + 左子树 + 右子树” 的公式算出本节点 metadata 写回。合并公式只能依赖这三者，才能随旋转局部正确重算。下例维护子树节点数（等价于内置的 order-statistics）。

```cpp
// 知识点 3.9.009：自定义 Node_Update、metadata_type 与节点迭代器。
namespace pbdsMeta
{
using namespace __gnu_pbds;

template <class NodeIt, class NodeCIt, class Cmp, class Alloc> struct SizeUpdate
{
    using metadata_type = int;

    void operator()(NodeIt it, NodeCIt end) const
    {
        // it 是待更新节点，end 是空节点；把元信息更新为子树节点数。
        int s = 1;
        auto l = it.get_l_child();
        auto r = it.get_r_child();
        if (l != end)
        {
            s += l.get_metadata();
        }
        if (r != end)
        {
            s += r.get_metadata();
        }
        const_cast<int &>(it.get_metadata()) = s;
    }
};

using Tree = tree<int, null_type, less<int>, rb_tree_tag, SizeUpdate>;

int rootSize(const vector<int> &a)
{
    // a 是待插入值；返回去重后的树根元信息，空树返回 0。
    Tree tr;
    for (int x : a)
    {
        tr.insert(x);
    }
    return tr.empty() ? 0 : tr.node_begin().get_metadata();
}
}
```

`pbdsMeta::rootSize(a)` 插入去重后返回树根 metadata（即元素总数），空树返回 0。每次 Node_Update 合并 $O(1)$，红黑树单次插删最坏 $O(\log n)$，空间 $O(n)$。只求大小/排名时直接用现成的 3.9.007 tree_order_statistics_node_update 即可；把 metadata 换成子树和就得到 3.9.010 自定义子树信息 的前缀和查询。

3.9.010 自定义子树信息：区间和、前缀和及其他可合并信息

把 3.9.009 的 metadata 换成 “子树内所有键之和”，就能在动态插删的平衡树上回答键值前缀和：查 “严格小于 x 的键之和” 时沿搜索路径下行，每当当前键小于 x 就把它连同其左子树的 metadata 整块累

加、再转向右子树，否则只转向左子树。整块跳过左子树是 $O(\log n)$ 的关键，前提是键值更新后 metadata 已由树维护到位。

```cpp
// 知识点 3.9.010：用自定义子树和回答键值前缀和。
namespace pbdsSubSum
{
using namespace __gnu_pbds;

template <class NodeIt, class NodeCIt, class Cmp, class Alloc> struct SumUpdate
{
    using metadata_type = int;

    void operator()(NodeIt it, NodeCIt end) const
    {
        // it 是待更新节点，end 是空节点；元信息保存本节点键与左右子树键之和。
        int s = **it;
        auto l = it.get_l_child();
        auto r = it.get_r_child();
        if (l != end)
        {
            s += l.get_metadata();
        }
        if (r != end)
        {
            s += r.get_metadata();
        }
        const_cast<int &>(it.get_metadata()) = s;
    }
};

using Tree = tree<int, null_type, less<int>, rb_tree_tag, SumUpdate>;

int prefixSum(const Tree &tr, int x)
{
    // tr 是键集合，x 是右边界；返回所有严格小于 x 的不同键之和。
    int ans = 0;
    auto it = tr.node_begin();
    auto end = tr.node_end();
    while (it != end)
    {
        auto l = it.get_l_child();
        int v = **it;
        if (v < x)
        {
            if (l != end)
            {
                ans += l.get_metadata();
            }
            ans += v;
            it = it.get_r_child();
        }
        else
```

```
        {
            it = l;
        }
    }
    return ans;
}
}
```

`pbdsSubSum::prefixSum(tr, x)` 返回所有严格小于 `x` 的不同键之和，更新与查询均最坏 $O(\log n)$，空间 $O(n)$。这是“可修改的权值前缀和”，键固定时用普通 3.2.001 树状数组 更省常数；只要计数不要和则回到 3.9.007 tree_order_statistics_node_update。metadata 的重算框架同 3.9.009 自定义 Node_Update。

3.9.011 `cc_hash_table` 与 `gp_hash_table`

PBDS 提供两种哈希表替代 `unordered_map`：`cc_hash_table` 用拉链法处理冲突，`gp_hash_table` 用开放寻址，后者局部性好、通常更快，但对负载因子和哈希质量更敏感，二者接口也与 `unordered_map` 不完全一致。防 hack 的混合函数直接复用 1.4.004 `unordered_map` / `unordered_set` 防 hack 哈希 的 `CustomHash`，这里仅定义整数键 PBDS 别名，不重复打印实现。

```
// 依赖 1.4.004 unordered_map / unordered_set 防 hack 哈希
namespace pbds_hash
{
using namespace __gnu_pbds;

template <class K, class V> using HashMap = gp_hash_table<K, V, CustomHash>;
}
```

`pbds_hash::HashMap<K,V>` 是挂有 `CustomHash` 的 `gp_hash_table`，按 `[]`、`find` 使用；`K` 使用 64 位整数。若要使用 `pair<int,int>` 或其它键，按实际键型直接定义 `gp_hash_table`，不要再给这一行别名堆约束。操作期望 $O(1)$、确定性最坏 $O(n)$，空间 $O(n)$。追求稳定最坏界或有序排名时改用 3.9.007 tree_order_statistics_node_update，策略与常数见 3.9.012 哈希策略与性能测试。

3.9.012 哈希策略、探测策略、扩容策略与性能测试

这一条是选型总结而非新代码，底层实现即 3.9.011 cc_hash_table 与 gp_hash_table 的 `HashMap` 与 1.4.004 防 hack 哈希 的 `CustomHash`。三类策略各管一段：哈希函数决定键映射是否均匀，探测或拉链策略决定冲突后的查找路径长度，扩容策略决定重散列频率与内存峰值。`gp_hash_table` 的开放寻址在高负载下探测链会急剧变长，故它比 `cc_hash_table` 更依赖负载因子和哈希质量。性能比较必须注明数据规模、键分布、读写比例与编译参数，并覆盖接近题面的连续、等差和聚簇键；不能把单组随机计时当作普遍结论。操作期望 $O(1)$、确定性最坏 $O(n)$，空间 $O(n)$。需要有序或排名时用 3.9.007 tree_order_statistics_node_update。

3.9.013 编译器版本兼容性、常数、失效规则与不适用场景

PBDS 是 GCC/libstdc++ 的私有扩展，不属于 C++ 标准，Clang 配 libc++、MSVC 都不提供，因此它只在确认评测机用 GNU 工具链时才可依赖；受限环境要提前准备 STL 或自写替代。此外堆的点迭代器在对

应节点被删后即失效，跨 GCC 大版本升级时接口细节、句柄失效规则与常数都可能变，移植前应重新核对。下例用宏 `__GLIBCXX__` 做一次编译期可用性判断。

```cpp
// 知识点 3.9.013：PBDS 的编译器边界与最小可用性检查。
namespace pbds_compat
{
using namespace __gnu_pbds;

bool available()
{
    // 当前源文件已在 GNU libstdc++ 环境中通过预处理时返回 true。
#ifdef __GLIBCXX__
    return true;
#else
    return false;
#endif
}
}
```

`pbds_compat::available()` 只能说明这份源文件已经在 libstdc++ 环境中成功通过预处理；若环境根本没有 PBDS 头，文件会在到达该函数之前编译失败，因此它不能在同一源文件中运行时探测或补救“缺少 PBDS”。真要维护跨工具链版本，应在构建配置中选择不同源文件，或在包含扩展头之前用 `__has_include` 条件编译整段 PBDS 代码。有序容器可回退 STL `set`/`map`，排名用手写平衡树或 3.2.001 树状数组，哈希表用 1.4.004 防 hack 哈希 的 `CustomHash` 配标准 `unordered_map`。

3.10 根号分块

3.10.001 基础数列分块

把长度 n 的数列按块长 $B \approx \sqrt{n}$ 划分，整块保存加法标记、区间和与最值，零散部分直接枚举。部分修改前把整块标记下放，修改后重算该块；完整块只改摘要，因此代码短且适合同时维护多种容易合并的块信息。

```cpp
struct SqrtBlock
{
    int n, B, m; // n 是长度，B 是块长，m 是块数。
    vector<int> a, tag, sum, mn, mx; // a 是元素（不含所在块标记），tag 是块加标记，sum、mn、mx 是含标记的块和、块最小、块最大。

    int L(int k) const
    {
        // k 是块编号；返回第 k 块的起始下标（含）。
        return k * B;
    }

    int R(int k) const
    {
        // k 是块编号；返回第 k 块的结束下标（不含），末块截到 n。
        return min(n, (k + 1) * B);
```

```
    }

    void pull(int k)
    {
        // k 是块编号；按块内 a 值重算 sum、mn、mx，要求该块标记已下放（tag[k]=0）。
        sum[k] = 0;
        mn[k] = numeric_limits<int>::max();
        mx[k] = numeric_limits<int>::lowest();
        for (int i = L(k); i < R(k); i++)
        {
            sum[k] += a[i];
            mn[k] = min(mn[k], a[i]);
            mx[k] = max(mx[k], a[i]);
        }
    }

    void push(int k)
    {
        // k 是块编号；把 tag[k] 加到块内每个 a[i] 后清零，块摘要已含标记故不变。
        if (!tag[k])
        {
            return;
        }
        for (int i = L(k); i < R(k); i++)
        {
            a[i] += tag[k];
        }
        tag[k] = 0;
    }

    SqrtBlock(vector<int> v) : n(v.size()), a(move(v))
    {
        // v 是初始数列（0 起下标）；取块长 B≈√n 分块，清零标记并算出每块的和、最小值、最大值。
        B = max<int>(1, (int)sqrtl(max<int>(1, n)));
        m = (n + B - 1) / B;
        tag.assign(m, 0);
        sum.resize(m);
        mn.resize(m);
        mx.resize(m);
        for (int k = 0; k < m; k++)
        {
            pull(k);
        }
    }

    void add(int l, int r, int v)
    {
        // [l,r) 是 0 起半开区间，v 是增量；给区间内每个元素加 v，整块只改标记与摘要，零散块下放后重算。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删。
        if (l == r)
        {
            return;
```

```
        }
        for (int k = l / B; L(k) < r; k++)
        {
            int x = max(l, L(k)), y = min(r, R(k));
            if (x == L(k) && y == R(k))
            {
                tag[k] += v;
                sum[k] += v * (y - x);
                mn[k] += v;
                mx[k] += v;
            }
            else
            {
                push(k);
                for (int i = x; i < y; i++)
                {
                    a[i] += v;
                }
                pull(k);
            }
        }
    }

    array<int, 3> query(int l, int r) const
    {
        // 返回区间的 {和, 最小值, 最大值}。
        assert(0 <= l && l < r && r <= n); // 调试检查，可删。
        array<int, 3> ans{0, numeric_limits<int>::max(), numeric_limits<int>::lowest()};
        for (int k = l / B; L(k) < r; k++)
        {
            int x = max(l, L(k)), y = min(r, R(k));
            if (x == L(k) && y == R(k))
            {
                ans[0] += sum[k];
                ans[1] = min(ans[1], mn[k]);
                ans[2] = max(ans[2], mx[k]);
            }
            else
            {
                for (int i = x; i < y; i++)
                {
                    int v = a[i] + tag[k];
                    ans[0] += v;
                    ans[1] = min(ans[1], v);
                    ans[2] = max(ans[2], v);
                }
            }
        }
        return ans;
    }
};
```

构造 `SqrtBlock q(a)` 后，`add(l,r,v)` 修改 $[l,r)$，`query(l,r)` 对非空区间返回 `{和，最小值，最大值}`。所有单点值、块标记、块和与 `v*(r-l)` 必须能由当前 64 位 `int` 表示。构造 $O(n)$，单次修改或查询 $O(\sqrt{n})$，空间 $O(n)$；只维护区间和时用 3.2.004 区间修改、区间求和 更快。

3.10.002 有序数列分块

每块再保存一份排序后的元素，完整块可二分统计小于或小于等于给定值的个数；区间前驱、下界和第 K 小都建立在同一计数上。整块加只改标记，零散修改下放标记后重排该块。

```
struct OrderBlock
{
    int n, B, m; // n 是长度，B 是块长，m 是块数。
    vector<int> a, tag; // a 是元素（不含所在块标记），tag 是块加标记。
    vector<vector<int>> s; // s[k] 是第 k 块元素（不含标记）的升序副本。

    int L(int k) const
    {
        // k 是块编号；返回第 k 块的起始下标（含）。
        return k * B;
    }

    int R(int k) const
    {
        // k 是块编号；返回第 k 块的结束下标（不含），末块截到 n。
        return min(n, (k + 1) * B);
    }

    void pull(int k)
    {
        // k 是块编号；用块内 a 值重建升序副本 s[k]，要求该块标记已下放（tag[k]=0）。
        s[k].assign(a.begin() + L(k), a.begin() + R(k));
        sort(s[k].begin(), s[k].end());
    }

    void push(int k)
    {
        // k 是块编号；把 tag[k] 加到块内每个 a[i] 后清零。
        if (!tag[k])
        {
            return;
        }
        for (int i = L(k); i < R(k); i++)
        {
            a[i] += tag[k];
        }
        tag[k] = 0;
    }

    OrderBlock(vector<int> v) : n(v.size()), a(move(v))
    {
```

```
        // v 是初始数列（0 起下标）；取块长 B≈√n 分块，清零标记并为每块建立升序副本。
        B = max<int>(1, (int)sqrtl(max<int>(1, n)));
        m = (n + B - 1) / B;
        tag.assign(m, 0);
        s.resize(m);
        for (int k = 0; k < m; k++)
        {
            pull(k);
        }
    }

    void add(int l, int r, int v)
    {
        // [l,r) 是 0 起半开区间，v 是增量；给区间内每个元素加 v，整块只改标记，零散块下放后重排。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删。
        if (l == r)
        {
            return;
        }
        for (int k = l / B; L(k) < r; k++)
        {
            int x = max(l, L(k)), y = min(r, R(k));
            if (x == L(k) && y == R(k))
            {
                tag[k] += v;
            }
            else
            {
                push(k);
                for (int i = x; i < y; i++)
                {
                    a[i] += v;
                }
                pull(k);
            }
        }
    }

    int count(int l, int r, int x, bool le = false) const
    {
        // [l,r) 是查询区间，x 是阈值；le 为假时返回区间内 <x 的元素个数，为真时返回 ≤x 的个数。
        assert(0 <= l && l <= r && r <= n); // 调试检查，可删。
        int ans = 0;
        for (int k = l / B; L(k) < r; k++)
        {
            int p = max(l, L(k)), q = min(r, R(k));
            if (p == L(k) && q == R(k))
            {
                auto it = partition_point(s[k].begin(), s[k].end(),
                                          [&](int v)
                                          {
                                              return le ? v + tag[k] <= x : v + tag[k] < x;
```

```
                                        });
                ans += it - s[k].begin();
            }
            else
            {
                for (int i = p; i < q; i++)
                {
                    int v = a[i] + tag[k];
                    ans += le ? v <= x : v < x;
                }
            }
        }
        return ans;
    }

    optional<int> prev(int l, int r, int x) const
    {
        // [l,r) 是查询区间，x 是阈值；返回区间内严格小于 x 的最大元素，不存在时为空。
        optional<int> ans;
        for (int k = l / B; L(k) < r; k++)
        {
            int p = max(l, L(k)), q = min(r, R(k));
            if (p == L(k) && q == R(k))
            {
                auto it = partition_point(s[k].begin(), s[k].end(),
                                          [&](int v)
                                          {
                                              return v + tag[k] < x;
                                          });
                if (it != s[k].begin())
                {
                    int v = *(it - 1) + tag[k];
                    if (!ans || *ans < v)
                    {
                        ans = v;
                    }
                }
            }
            else
            {
                for (int i = p; i < q; i++)
                {
                    int v = a[i] + tag[k];
                    if (v < x && (!ans || *ans < v))
                    {
                        ans = v;
                    }
                }
            }
        }
        return ans;
    }
```

```
146
147     optional<int> next(int l, int r, int x) const
148     {
149         // [l,r) 是查询区间，x 是阈值；返回区间内大于等于 x 的最小元素，不存在时为空。
150         optional<int> ans;
151         for (int k = l / B; L(k) < r; k++)
152         {
153             int p = max(l, L(k)), q = min(r, R(k));
154             if (p == L(k) && q == R(k))
155             {
156                 auto it = partition_point(s[k].begin(), s[k].end(),
157                                           [&](int v)
158                                           {
159                                               return v + tag[k] < x;
160                                           });
161                 if (it != s[k].end())
162                 {
163                     int v = *it + tag[k];
164                     if (!ans || v < *ans)
165                     {
166                         ans = v;
167                     }
168                 }
169             }
170             else
171             {
172                 for (int i = p; i < q; i++)
173                 {
174                     int v = a[i] + tag[k];
175                     if (x <= v && (!ans || v < *ans))
176                     {
177                         ans = v;
178                     }
179                 }
180             }
181         }
182         return ans;
183     }
184
185     int kth(int l, int r, int k, int lo, int hi) const
186     {
187         // [l,r) 是查询区间，k 从 0 计数，[lo,hi] 是须覆盖答案的整数值域；二分返回区间第 k 小的元素值。
188         assert(0 <= k && k < r - l && lo <= hi); // 调试检查，可删。
189         while (lo < hi)
190         {
191             int mid = midpoint(lo, hi);
192             if (count(l, r, mid, true) > k)
193             {
194                 hi = mid;
195             }
196             else
197             {
```

```
                lo = mid + 1;
            }
        }
        return lo;
    }
};
```

`count(l,r,x)` 默认统计 $< x$，末参传 `true` 时统计 $\le x$；`prev` 返回严格前驱，`next` 返回大于等于 `x` 的下界，均以空 `optional` 表示不存在。`kth(l,r,k,lo,hi)` 的 `k` 从零计数，且调用者给出的整数值域必须覆盖真实答案。所有元素加块标记后须能由 64 位 `int` 表示。修改、计数、前驱和下界均为 $O(\sqrt{n}\log n)$，第 K 小再乘值域二分次数，空间 $O(n)$；静态区间查询优先用 3.4.005 小波矩阵。

3.10.003 王室联邦树分块

树分块把 DFS 完成的子树点暂存在栈中；某个儿子返回后，若当前节点以下尚未分块的点达到 B 个，就弹出为一块并把当前节点记作代表点。最后不足 B 个点并入最后一块。代表点只负责连接这一批点，不保证自身属于该块。

```
struct TreeBlock
{
    vector<int> bel, cap; // bel[u] 是点 u 所属块编号，cap[k] 是第 k 块的块根（省会）。

    TreeBlock(const vector<vector<int>> &g, int root, int B) : bel(g.size(), -1)
    {
        // g 是树的邻接表，root 是根，B 是块大小下界；王室联邦分块，每块至少 B 个点（剩余点并入最后一
        ↪ 块），块根可能不在块内。
        assert(!g.empty() && 0 <= root && root < (int)g.size() && B > 0); // 调试检查，可删。
        vector<int> st;
        auto dfs = [&](auto &&self, int u, int f) -> void
        {
            int base = st.size();
            for (int v : g[u])
            {
                if (v == f)
                {
                    continue;
                }
                self(self, v, u);
                if ((int)st.size() - base >= B)
                {
                    cap.push_back(u);
                    while ((int)st.size() > base)
                    {
                        bel[st.back()] = (int)cap.size() - 1;
                        st.pop_back();
                    }
                }
            }
            st.push_back(u);
```

```
        };
        dfs(dfs, root, -1);
        if (cap.empty())
        {
            cap.push_back(root);
        }
        while (!st.empty())
        {
            bel[st.back()] = (int)cap.size() - 1;
            st.pop_back();
        }
    }
};
```

输入必须是以 `root` 为根的连通无向树，`B>0`；`bel[u]` 是点 `u` 所属块，`cap[k]` 是第 `k` 块代表点。除最后一块外每块至少有 B 个点，块数 $O(n/B)$；构造 $O(n)$ 时间、$O(n)$ 空间，递归深度最坏 $O(n)$。它只给出分块，不自动维护块间路径；需要完整路径查询时使用第 6 章的 LCA、树链剖分或点分治。

3.10.004 图上按度数分块

把度数至少为 $B \approx \sqrt{2m}$ 的点设为重点。重点预存邻点权值和；修改点权时只更新与该点相邻的重点摘要，查询轻点则直接枚举邻边。重点数至多 $2m/B$，于是修改和查询都被根号界控制。

```
struct DegreeBlock
{
    int B; // B 是重点度数阈值，约为 sqrt(2m)。
    vector<vector<int>> g, big; // g 是邻接表，big[u] 是 u 的重点邻居编号（id 值）。
    vector<int> id, w, sum; // id[u] 是重点编号（轻点为 -1），w 是点权，sum[k] 是重点 k 的邻点权值和。

    DegreeBlock(int n, const vector<array<int, 2>> &e, vector<int> init = {})
        : g(n), big(n), id(n, -1), w(move(init))
    {
        // n 是点数，e 是无向边 {u,v}（0 起、无自环），init 是初始点权（空表示全 0）。
        // 建邻接表，把度数不小于 B≈⌈√(2m)⌉ 的点设为重点并预存其邻点权值和。
        assert(n >= 0 && (w.empty() || (int)w.size() == n)); // 调试检查，可删。
        w.resize(n);
        for (auto [u, v] : e)
        {
            assert(0 <= u && u < n && 0 <= v && v < n && u != v); // 调试检查，可删。
            g[u].push_back(v);
            g[v].push_back(u);
        }
        B = max<int>(1, (int)sqrtl(2 * (int)e.size() + 1));
        for (int u = 0; u < n; u++)
        {
            if ((int)g[u].size() >= B)
            {
                id[u] = sum.size();
                sum.push_back(0);
            }
```

```
        }
        for (int u = 0; u < n; u++)
        {
            for (int v : g[u])
            {
                if (id[v] >= 0)
                {
                    big[u].push_back(id[v]);
                }
                if (id[u] >= 0)
                {
                    sum[id[u]] += w[v];
                }
            }
        }
    }

    void add(int u, int v)
    {
        // u 是点编号，v 是增量；令 w[u]+=v，并同步更新与 u 相邻的所有重点的邻点和。
        w[u] += v;
        for (int k : big[u])
        {
            sum[k] += v;
        }
    }

    int query(int u) const
    {
        // u 是点编号；返回 u 所有邻点的权值和（重边按重数计），重点直接取预存值。
        if (id[u] >= 0)
        {
            return sum[id[u]];
        }
        int ans = 0;
        for (int v : g[u])
        {
            ans += w[v];
        }
        return ans;
    }
};
```

构造 `DegreeBlock q(n,e,w)` 后，`add(u,v)` 给点 `u` 的权值增加 `v`，`query(u)` 返回所有邻点权值和。输入必须是简单无向图；重边会被当成多条边重复累计。点权和任意邻点权值和须能由 64 位 `int` 表示。构造 $O(n+m)$，单次修改、查询 $O(\sqrt{m})$，空间 $O(n+m)$；若操作是边增删而不是点权修改，需要另外维护动态重点结构。

4. 字符串

4.1 基础匹配与哈希

4.1.001 字符串哈希（Mersenne 单模、随机底数）

把长度 n 的串预处理成前缀哈希 $h[i]=\sum_{j<i}(s_j+1)base^{i-1-j} \bmod P$ 后，子串 $s[l,r)$ 的哈希就是 $h[r]-h[l]base^{r-l}$。这里取 Mersenne 素数 $P=2^{61}-1$：两个代表元相乘先放入 `__uint128_t`，再利用 $2^{61}\equiv 1 \pmod P$ 把高 61 位与低 61 位相加约简，避免普通 `%`。`HASH_BASE` 在进程启动时从 $[P/2,P-2]$ 随机一次，所有对象共享它；因此固定测试数据不能预先针对一个公开底数构造碰撞。对长度不超过 n 的两个不同等长串，其差是次数至多 $n-1$ 的非零多项式，在当前候选集合上碰撞概率至多约 $2(n-1)/P$，但哈希判等仍不是确定性证明。

```
struct HashValue
{
    // x 是模 2^61-1 的代表元；无符号 64 位与 128 位乘积是 Mersenne 快速约简所必需的。
    static constexpr unsigned long long mod = (1ULL << 61) - 1;
    unsigned long long x = 0;

    HashValue(unsigned long long x = 0) : x((x & mod) + (x >> 61))
    {
        // x 是待规范化整数；构造模 mod 的哈希值。
        if (this->x >= mod)
        {
            this->x -= mod;
        }
    }

    friend HashValue operator+(HashValue a, HashValue b)
    {
        // a、b 是同模哈希值；返回模 mod 的和。
        unsigned long long x = a.x + b.x;
        if (x >= mod)
        {
            x -= mod;
        }
        return x;
    }

    friend HashValue operator-(HashValue a, HashValue b)
    {
        // a、b 是同模哈希值；返回模 mod 的差。
        unsigned long long x = a.x + mod - b.x;
        if (x >= mod)
        {
            x -= mod;
        }
        return x;
    }
```

```cpp

    friend HashValue operator*(HashValue a, HashValue b)
    {
        // a、b 是同模哈希值；用 128 位乘积返回模 mod 的积。
        __uint128_t x = (__uint128_t)a.x * b.x;
        return (unsigned long long)(x >> 61) + (unsigned long long)(x & mod);
    }

    friend bool operator==(HashValue, HashValue) = default;

    friend ostream &operator<<(ostream &out, HashValue a)
    {
        // out 是输出流，a 是哈希值；输出 a 的规范代表元并返回 out。
        return out << a.x;
    }
};

inline HashValue hashBase()
{
    // 无参数；返回本进程随机选定的大底数。
    unsigned long long seed = (unsigned long
    ↪  long)chrono::steady_clock::now().time_since_epoch().count();
    seed ^= (unsigned long long)random_device{}() << 32;
    seed ^= random_device{}();
    mt19937_64 mt(seed);
    uniform_int_distribution<unsigned long long> dist(HashValue::mod / 2, HashValue::mod - 2);
    return dist(mt);
}

inline const HashValue HASH_BASE = hashBase();

struct HashBase
{
    // pw[i] 是 HASH_BASE 的 i 次幂，按实际需要延长。
    vector<HashValue> pw{HashValue(1)};

    void ensure(int n)
    {
        // n 是需要支持的最大字符串长度；保证 pw[0..n] 已计算。
        while ((int)pw.size() <= n)
        {
            pw.push_back(pw.back() * HASH_BASE);
        }
    }
};

inline HashBase HASH_POWER;

struct DynStrHash
{
    // pre[i] 是当前串前 i 个字符的哈希。
    vector<HashValue> pre{HashValue(0)};
```

```
88
89      int size() const
90      {
91          // 无参数；返回当前字符串长度。
92          return pre.size() - 1;
93      }
94
95      void push_back(char c)
96      {
97          // c 是追加字符；在串尾加入 c。
98          HASH_POWER.ensure(size() + 1);
99          pre.push_back(pre.back() * HASH_BASE + HashValue((unsigned char)c + 1));
100     }
101
102     HashValue get(int l, int r) const
103     {
104         // l、r 是当前串下标；返回半开子串 [l,r) 的哈希值。
105         assert(0 <= l && l <= r && r <= size()); // 调试检查，可删。
106         return pre[r] - pre[l] * HASH_POWER.pw[r - l];
107     }
108
109     HashValue concat(HashValue value, int l, int r) const
110     {
111         // value 是已有串的哈希，l、r 指定当前串的半开子串；返回两段拼接后的哈希。
112         return value * HASH_POWER.pw[r - l] + get(l, r);
113     }
114 };
115
116 struct StrHash
117 {
118     // pre[i] 是原串前 i 个字符的哈希，rev[i] 是反转串前 i 个字符的哈希。
119     vector<HashValue> pre, rev;
120
121     StrHash(const string &s) : pre(s.size() + 1), rev(s.size() + 1)
122     {
123         // s 是原字符串；预处理正向、反向前缀哈希与所需底数幂。
124         int n = s.size();
125         HASH_POWER.ensure(n);
126         for (int i = 0; i < n; i++)
127         {
128             pre[i + 1] = pre[i] * HASH_BASE + HashValue((unsigned char)s[i] + 1);
129             rev[i + 1] = rev[i] * HASH_BASE + HashValue((unsigned char)s[n - 1 - i] + 1);
130         }
131     }
132
133     int size() const
134     {
135         // 无参数；返回原字符串长度。
136         return pre.size() - 1;
137     }
138
139     HashValue get(int l, int r) const
```

```
    {
        // l、r 是原串下标；返回半开子串 [l,r) 的正向哈希。
        assert(0 <= l && l <= r && r <= size()); // 调试检查，可删。
        return pre[r] - pre[l] * HASH_POWER.pw[r - l];
    }

    HashValue getrev(int l, int r) const
    {
        // l、r 是原串下标；返回反转 s[l,r) 后的哈希。
        assert(0 <= l && l <= r && r <= size()); // 调试检查，可删。
        int n = size();
        return rev[n - l] - rev[n - r] * HASH_POWER.pw[r - l];
    }

    HashValue concat(HashValue value, int l, int r) const
    {
        // value 是已有串的哈希，l、r 指定原串半开子串；返回两段拼接后的哈希。
        return value * HASH_POWER.pw[r - l] + get(l, r);
    }

    HashValue concatrev(HashValue value, int l, int r) const
    {
        // value 是已有串的哈希，l、r 指定原串半开子串；返回拼接反转子串后的哈希。
        return value * HASH_POWER.pw[r - l] + getrev(l, r);
    }

    static vector<int> match(const string &s, const string &t)
    {
        // s 是文本串，t 是模式串；返回 t 的全部匹配起点，按下标递增排列。
        if (t.size() > s.size())
        {
            return {};
        }
        StrHash a(s), b(t);
        HashValue target = b.get(0, t.size());
        vector<int> ans;
        for (int i = 0; i + (int)t.size() <= (int)s.size(); i++)
        {
            if (a.get(i, i + t.size()) == target)
            {
                ans.push_back(i);
            }
        }
        return ans;
    }
};
```

构造 `StrHash a(s)` 后，`get(l,r)`、`getrev(l,r)` 分别取半开区间 $[l,r)$ 的正向哈希与反转哈希，`concat`、`concatrev` 把对应子串接到已有哈希后；`DynStrHash` 支持逐字符 `push_back` 和同样的正向查询。幂表由全部对象共享并按最大实际长度增长，不要求预设 `N`。构造与追加总计 $O(n)$，子串哈希和拼接 $O(1)$，`match(s,t)` 为 $O(n+m)$，空间 $O(n+m)$；单模静态版只保存正反两组前缀与一份共享幂表，百万字

符约占 24 MB，旧双模正向实现的四个整数数组约占 32 MB。Windows x86-64、GCC 14.2、`-O2` 下，对随机百万字符冷构造并执行 200 万组最长 1000 的等长子串比较，7 轮中位数从旧版 `8.029 ms + 100.425 ms` 降至 `7.437 ms + 55.973 ms`；新版构造还同时得到旧版没有的反向前缀，该结果只代表当前机器。要确定性线性匹配改用 4.1.002 KMP / 前缀函数 或 4.1.003 扩展 KMP / Z 函数；滑窗用法单列为 4.1.008 Karp-Rabin，只找首次出现见 4.1.010 `strstr` 类匹配。

比赛现场只需要静态子串判等时，可改抄下面的 `ShortHash`。它保留固定底数、$2^{61}-1$ 单模、`__uint128_t` 乘积和 Mersenne 快速约简，只去掉值类型运算符、反向前缀、动态追加、拼接和模式匹配；原版接口仍用于需要这些能力的题目。固定底数不具备运行时随机化，不能抵御针对该常数构造的碰撞数据。

```cpp
struct ShortHash
{
    static constexpr unsigned long long M = (1ULL << 61) - 1; // M 是梅森素数模数 2^61-1。
    // B 从 [M / 2, M - 2] 中选取并固定，避免现场引入随机设备。
    static constexpr unsigned long long B = 123456789012345678 9ULL;
    inline static vector<unsigned long long> p{1}; // p[i] 是 B 的 i 次幂。
    vector<unsigned long long> h;                  // h[i] 是原串前 i 个字符的哈希。

    static unsigned long long red(unsigned long long x)
    {
        // x 小于 2M；返回 x 模 M 的规范代表元。
        x = (x & M) + (x >> 61);
        return x >= M ? x - M : x;
    }

    static unsigned long long mul(unsigned long long a, unsigned long long b)
    {
        // a、b 是模 M 代表元；返回二者乘积模 M。
        __uint128_t x = (__uint128_t)a * b;
        return red((unsigned long long)(x >> 61) + (unsigned long long)(x & M));
    }

    ShortHash(string_view s) : h(s.size() + 1)
    {
        // s 是原字符串；预处理前缀哈希和所需底数幂。
        while (p.size() <= s.size())
        {
            p.push_back(mul(p.back(), B));
        }
        for (int i = 0; i < (int)s.size(); i++)
        {
            h[i + 1] = red(mul(h[i], B) + (unsigned char)s[i] + 1);
        }
    }

    unsigned long long get(int l, int r) const
    {
        // l、r 是原串下标；返回半开子串 [l,r) 的哈希。
        return red(h[r] + M - mul(h[l], p[r - l]));
    }
};
```

构造 `ShortHash h(s)` 后用 `h.get(l,r)` 查询 $s[l,r)$。需要反向查询时，把反串 `t` 另建为 `ShortHash rev(t)`，则反转 $s[l,r)$ 的哈希为 `rev.get(n-r,n-l)`。构造为 $O(n)$，查询为 $O(1)$，每次查询仍只做一次 128 位乘法与 M61 约简；短版每个字符串只保存一组前缀，额外空间为 $O(n)$。底数固定在 $[M/2, M-2]$ 的选定常数，哈希仍是概率判定；需要抵御构造碰撞时使用上面的运行时随机底数版本。性能数据只代表对应机器与数据分布。

4.1.002 KMP / 前缀函数

前缀函数 `pi[i]` 记录 $s[0..i]$ 的最长真 border（既是前缀又是后缀的最长长度），求它时若 `s[i]` 与当前候选位 `s[j]` 失配就沿 `pi` 链跳到次长 border 继续试，均摊线性。有了模式串的前缀函数，KMP 扫描文本时文本指针从不回退：失配就让模式指针沿 `pi` 回跳，匹配到串尾记录一次出现再回跳继续找重叠出现。`borders` 顺着 `pi.back()` 链枚举全部 border，`minPeriod` 用「最短周期 $= n - \pi_{n-1}$、当它整除 n 才是真整周期」判断循环节。

```cpp
vector<int> prefix(const string &s)
{
    // s 是原字符串；返回前缀函数，p[i] 是 s[0..i] 的最长真 border 长度。
    vector<int> p(s.size());
    for (int i = 1; i < (int)s.size(); i++)
    {
        int j = p[i - 1];
        while (j && s[i] != s[j])
        {
            j = p[j - 1];
        }
        if (s[i] == s[j])
        {
            j++;
        }
        p[i] = j;
    }
    return p;
}

vector<int> kmp(const string &s, const string &t)
{
    // s 是文本串，t 是模式串；返回全部匹配起点，按下标递增排列。
    if (t.empty())
    {
        vector<int> a(s.size() + 1);
        iota(a.begin(), a.end(), 0);
        return a;
    }
    auto p = prefix(t);
    vector<int> ans;
    int j = 0;
    for (int i = 0; i < (int)s.size(); i++)
    {
        while (j && s[i] != t[j])
```

```
        {
            j = p[j - 1];
        }
        if (s[i] == t[j])
        {
            j++;
        }
        if (j == (int)t.size())
        {
            ans.push_back(i - j + 1);
            j = p[j - 1];
        }
    }
    return ans;
}

vector<int> borders(const string &s)
{
    // s 是原字符串；返回全部非空真 border 的长度，按递增排列。
    auto p = prefix(s);
    vector<int> a;
    int x = s.empty() ? 0 : p.back();
    while (x)
    {
        a.push_back(x);
        x = p[x - 1];
    }
    reverse(a.begin(), a.end());
    return a;
}

int minPeriod(const string &s)
{
    // s 是原字符串；返回最短整周期长度，空串返回 0。
    if (s.empty())
    {
        return 0;
    }
    auto p = prefix(s);
    int x = s.size() - p.back();
    return (int)s.size() % x == 0 ? x : (int)s.size();
}
```

`prefix(s)` 返回前缀函数表；`kmp(s,t)` 返回 `t` 在 `s` 的全部出现起点（空模式返回 $0, 1, \dots, n$）；`borders(s)` 按长度递增给出全部非空真 border，不包含整串；`minPeriod(s)` 返回最短整周期，非周期串返回 n。四者均为线性时间和线性空间。相比 4.1.001 字符串哈希 的概率匹配，KMP 确定性正确；只关心某后缀与前缀的匹配长度时 4.1.003 扩展 KMP / Z 函数 更直接。周期、border、循环节的汇总应用另见 4.5.006 字符串周期、循环节与前后缀，它直接复用这里的前缀函数。

需要在自动机或数位 DP 中反复读取字符时，可把同一回退过程预计算成 DFA。`KMPAutomaton<A,F>` 的状态 $0 \dots |p|$ 表示当前后缀与模式 `pattern` 的最长匹配长度；完整匹配状态读入新字符时先沿 `pi` 回退，因

此重叠出现不会丢失。字符集必须是连续区间 $[F, F+A)$，空模式也有合法的唯一状态。

```
template <int A = 26, char F = 'a'> struct KMPAutomaton
{
    // pattern 保存模式串，pi 是其前缀函数，go[state][字符] 是 DFA 转移后的匹配长度。
    string pattern;
    vector<int> pi;
    vector<array<int, A>> go;

    KMPAutomaton() = default;

    explicit KMPAutomaton(const string &s)
    {
        // s 是模式串；构造其所有前缀匹配状态的自动机。
        build(s);
    }

    int id(char c) const
    {
        // c 是字符集 [F,F+A) 中的字符；返回从 0 开始的字符编号。
        int x = c - F;
        assert(0 <= x && x < A);
        return x;
    }

    void build(const string &s)
    {
        // s 是模式串；重建前缀函数和状态转移表，无返回值。
        pattern = s;
        int n = pattern.size();
        for (char c : pattern)
        {
            id(c);
        }
        pi.assign(n, 0);
        for (int i = 1, j = 0; i < n; i++)
        {
            while (j && pattern[i] != pattern[j])
            {
                j = pi[j - 1];
            }
            if (pattern[i] == pattern[j])
            {
                j++;
            }
            pi[i] = j;
        }
        go.assign(n + 1, array<int, A>());
        for (int state = 0; state <= n; state++)
        {
            for (int x = 0; x < A; x++)
            {
```

```
                int j = state;
                char c = (char)(F + x);
                while (j && (j == n || pattern[j] != c))
                {
                    j = pi[j - 1];
                }
                if (j < n && pattern[j] == c)
                {
                    j++;
                }
                go[state][x] = j;
            }
        }
    }

    int next(int state, int x) const
    {
        // state 是当前已匹配前缀长度，x 是字符编号；返回读入该字符后的匹配长度。
        assert(0 <= state && state <= (int)pattern.size() && 0 <= x && x < A);
        return go[state][x];
    }

    int next(int state, char c) const
    {
        // state 是当前已匹配前缀长度，c 是字符集内字符；返回读入该字符后的匹配长度。
        return next(state, id(c));
    }

    bool matched(int state) const
    {
        // state 是当前状态；返回是否已经匹配完整模式串。
        return state == (int)pattern.size();
    }
};
```

构造 `KMPAutomaton<26,'a'> autom("aba")` 后，状态从 `0` 开始，逐字符令 `state=autom.next(state,c)`，`autom.matched(state)` 为真时记录一次出现；完整状态仍可继续转移以保留重叠匹配。预处理时间和空间为 $O(A|p|)$，每次转移 $O(1)$；若只扫描一次文本，直接使用本条前面的 `kmp` 可省掉这张表。

4.1.003 扩展 KMP / Z 函数

Z 函数 `z[i]` 记录后缀 $s[i..n)$ 与整串前缀的最长公共前缀长度（约定 `z[0]=n`）。维护一个当前已知能匹配到前缀的最靠右区间 $[l,r)$（Z-box）：若 `i` 落在盒内，`z[i]` 至少是镜像位置 `z[i-l]` 与盒剩余长度 `r-i` 的较小值，再从这个下界向右暴力扩展；每次扩展都推进 `r`，因此总匹配字符数线性。它和 KMP 互补——KMP 看前缀函数，Z 看与整串前缀的对齐，用于模式匹配（把模式接文本前）、周期判断、前缀出现统计。

```
vector<int> zfunc(const string &s)
{
    // s 是原字符串；返回 z 数组，z[i]=lcp(s,s[i..n))。
```

```
    int n = s.size();
    vector<int> z(n);
    if (!n)
    {
        return z;
    }
    z[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; ++i)
    {
        if (i < r)
        {
            z[i] = min(r - i, z[i - l]);
        }
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
        {
            ++z[i];
        }
        if (i + z[i] > r)
        {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

`zfunc(s)` 返回长度 n 的 Z 数组，空串返回空数组，时间与空间均为 $O(n)$。要全部出现位置且确定性时它与 4.1.002 KMP / 前缀函数 等价；大量任意子串比较仍用 4.1.001 字符串哈希。下面四个固定串分别覆盖空串、单字符、整段留在 Z-box 内以及越过右端继续扩展：

```
signed main()
{
    assert(zfunc("").empty());
    assert(zfunc("a") == vector<int>{1});
    assert(zfunc("aaaaa") == vector<int>({5, 4, 3, 2, 1}));
    assert(zfunc("abacaba") == vector<int>({7, 0, 1, 0, 3, 0, 1}));
    cout << "7 0 1 0 3 0 1\n";
    return 0;
}
```

真实输出为 `7 0 1 0 3 0 1`。

4.1.004 Manacher

Manacher 在线性时间求出以每个中心的最长回文半径。奇数长度回文以字符为中心，偶数长度以字符缝为中心，本实现分两趟分别求 `odd[i]`、`even[i]`，各自维护当前最靠右回文区间 $[l, r]$：中心落在区间内时先用关于区间中心的镜像半径当下界，再向外扩展，扩展总量线性。`odd[i]` 对应长度 $2 \cdot odd[i] - 1$ 的回文，`even[i]` 对应中心缝 $(i-1, i)$ 的半径，因此 `isPal` 查询半开区间 $[l, r)$ 时要按长度奇偶分别取 `odd` 或 `even` 中点比较，混用会错。

```cpp
struct Manacher
{
    // odd、even 分别保存奇回文与偶回文半径。
    vector<int> odd, even;

    Manacher(const string &s)
    {
        // s 是要预处理回文半径的原字符串。
        int n = s.size();
        odd.resize(n);
        for (int i = 0, l = 0, r = -1; i < n; i++)
        {
            int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
            while (i - k >= 0 && i + k < n && s[i - k] == s[i + k])
            {
                k++;
            }
            odd[i] = k--;
            if (i + k > r)
            {
                l = i - k;
                r = i + k;
            }
        }
        even.resize(n);
        for (int i = 0, l = 0, r = -1; i < n; i++)
        {
            int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
            while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k])
            {
                k++;
            }
            even[i] = k--;
            if (i + k > r)
            {
                l = i - k - 1;
                r = i + k;
            }
        }
    }

    // odd[i] 对应长度 2*odd[i]-1；even[i] 对应中心缝 (i-1,i) 的半径。
    bool isPal(int l, int r) const
    {
        // l、r 是原串下标；返回半开区间 [l,r) 是否为回文串。
        assert(0 <= l && l <= r && r <= (int)odd.size()); // 调试检查，可删
        if (l == r)
        {
            return true;
        }
        int n = r - l;
        if (n & 1)
```

```
        {
            return odd[(l + r) / 2] > n / 2;
        }
        return even[(l + r) / 2] >= n / 2;
    }
};
```

构造 `Manacher m(s)` 后 `m.isPal(l,r)` $O(1)$ 判断 $[l,r)$ 是否回文；构造 $O(n)$ 时间与空间。它只回答连续回文，需要枚举本质不同回文串或其出现次数应改用 4.4.001 回文自动机 PAM / 回文树；只做一次「某子串是否回文」的概率判定也可用 4.1.001 字符串哈希 正反哈希比对。

4.1.005 最小表示法

求循环串所有旋转里字典序最小的那个起点，用于环形串的规范化与同构判定。用双指针 `i`、`j` 各代表一个候选起点，`k` 是已确认相等的公共长度：比较 $a[(i+k) \bmod n]$ 与 $a[(j+k) \bmod n]$，相等则 `k++`，否则字典序较大的那个起点直接跳过 `k+1` 个位置（这段区间内任何起点都不可能更优），两指针相遇时把落后的一个再推进一格；`k` 每次失配归零但两指针单调前进，总比较线性。

```
template <class T>
int minRepr(const vector<T> &a)
{
    // a 是循环序列；返回其字典序最小旋转的起点。
    int n = a.size();
    if (!n)
    {
        return 0;
    }
    int i = 0, j = 1, k = 0;
    while (i < n && j < n && k < n)
    {
        T x = a[(i + k) % n], y = a[(j + k) % n];
        if (x == y)
        {
            k++;
            continue;
        }
        if (x > y)
        {
            i += k + 1;
        }
        else
        {
            j += k + 1;
        }
        if (i == j)
        {
            j++;
        }
        k = 0;
```

```
    }
    return min(i, j);
}

template <class T>
vector<T> minRotation(const vector<T> &a)
{
    // a 是循环序列；返回其字典序最小的完整旋转。
    int p = minRepr(a);
    vector<T> b;
    for (int i = 0; i < (int)a.size(); i++)
    {
        b.push_back(a[(p + i) % a.size()]);
    }
    return b;
}
```

`minRepr(a)` 返回最小字典序旋转的起点下标，`minRotation(a)` 返回旋转后的完整序列；求起点 $O(n)$ 时间、$O(1)$ 额外空间，`minRotation` 另需 $O(n)$ 输出。模板参数 `T` 只需支持比较与拷贝，故字符串、整数序列通用。求唯一 Lyndon 分解或用它做后缀比较时见 4.1.006 Lyndon 分解，最小旋转恰是整串旋转的一个 Lyndon 相关副产品。

4.1.006 Lyndon 分解

Lyndon 串是严格小于自己所有真后缀的串；任何串都能唯一分解成字典序非增的若干 Lyndon 串（CFL 分解）。Duval 算法在线扫描：维护当前正在生长的近似 Lyndon 块，用 `k` 指向块内与新字符 `s[j]` 对应比较的位置——`s[k]<s[j]` 说明还能扩展，把 `k` 拉回块首；`s[k]==s[j]` 则 `k` 随 `j` 前进保持周期；一旦 `s[k]>s[j]` 就把已定型的整块（长度 `j-k`）逐段切出，`i` 前移。返回各段左端点，末尾补哨兵 `n`。

```
vector<int> lyndon(const string &s)
{
    // s 是待分解字符串；返回各段左端点，并在末尾附加 s.size()。
    int n = s.size();
    vector<int> a;
    for (int i = 0; i < n;)
    {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j])
        {
            if (s[k] < s[j])
            {
                k = i;
            }
            else
            {
                k++;
            }
            j++;
        }
```

```
        while (i <= k)
        {
            a.push_back(i);
            i += j - k;
        }
    }
    a.push_back(n);
    return a;
}
```

`lyndon(s)` 返回各 Lyndon 段的左端点并以 n 收尾，故第 i 段是 $[a_i, a_{i+1})$；$O(n)$ 时间、$O(n)$ 输出空间。它与 4.1.005 最小表示法 都利用字典序下的周期结构；4.1.007 Runs 使用两次 Lyndon 方向枚举，但维护独立的单调栈。

4.1.007 Runs

一个 run（极大重复）是指数至少为 2 的极大周期区间：其最小周期为 p，区间长度至少为 2p，且向左右都无法继续保持周期 p。当前实现采用两次 Lyndon 方向枚举：单调栈确定候选 Lyndon 根，再用前后缀哈希二分求 LCP/LCS，把根扩展成极大周期区间；第二次把字符序翻转后重复枚举，最后排序去重。

```
struct Run
{
    // p 是最小周期，[l,r) 是极大周期区间；32 位字段用于降低百万规模候选的内存占用。
    int p, l, r;
    auto operator<=>(const Run &) const = default;
};

struct RunHash
{
    // a 是 1 下标字符串值；h、pw 用于单模哈希 LCE。
    int n;
    vector<int> a, h, pw;

    explicit RunHash(const string &s) : n(s.size()), a(n + 1), h(n + 1), pw(n + 1, 1)
    {
        // s 是小写字母串；字符映射为 1..26 存入 a[1..n] 后建立前缀哈希。
        for (int i = 1; i <= n; i++)
        {
            a[i] = s[i - 1] - 'a' + 1;
        }
        build();
    }

    void build()
    {
        // 根据当前 a 建立前缀哈希。
        constexpr int mod = 998244353, bs = 31;
        h[0] = 0;
        pw[0] = 1;
        for (int i = 1; i <= n; i++)
```

```
        {
            pw[i] = 1LL * pw[i - 1] * bs % mod;
            h[i] = (1LL * h[i - 1] * bs + a[i]) % mod;
        }
    }

    int get(int l, int len) const
    {
        // 返回 1 下标区间 a[l..l+len) 的哈希值。
        constexpr int mod = 998244353;
        return (h[l + len - 1] - 1LL * h[l - 1] * pw[len] % mod + mod) % mod;
    }

    int lcp(int x, int y, int up) const
    {
        // 返回从 x、y 开始的最长公共前缀，长度不超过 up。
        if (x > n || y > n || a[x] != a[y])
        {
            return 0;
        }
        int l = 0, r = min({up, n - x + 1, n - y + 1});
        while (l < r)
        {
            int m = (l + r + 1) / 2;
            if (get(x, m) == get(y, m))
            {
                l = m;
            }
            else
            {
                r = m - 1;
            }
        }
        return l;
    }

    int lcs(int x, int y, int up) const
    {
        // 返回以 x、y 结尾的最长公共后缀，长度不超过 up。
        if (x <= 0 || y <= 0 || a[x] != a[y])
        {
            return 0;
        }
        int l = 0, r = min({up, x, y});
        while (l < r)
        {
            int m = (l + r + 1) / 2;
            if (get(x - m + 1, m) == get(y - m + 1, m))
            {
                l = m;
            }
            else
```

```
            {
                r = m - 1;
            }
        }
        return l;
    }

    void flip()
    {
        // 互补字符值，枚举另一种 Lyndon 方向。
        for (int i = 1; i <= n; i++)
        {
            a[i] = 27 - a[i];
        }
        build();
    }

    void getRuns(vector<Run> &ans)
    {
        // 用 Lyndon 单调栈枚举当前方向的全部 Runs。
        vector<int> st(n + 1);
        st[0] = n + 1;
        for (int i = n, top = 0, lt = 0; i >= 1; i--)
        {
            while (top)
            {
                int x = min(st[top] - i, st[top - 1] - st[top]);
                lt = lcp(i, st[top], x);
                if ((lt == x && st[top] - i < st[top - 1] - st[top]) ||
                    (lt < x && a[i + lt] < a[st[top] + lt]))
                {
                    top--;
                    lt = 0;
                }
                else
                {
                    break;
                }
            }
            int j = st[top];
            st[++top] = i;
            int x = lcs(i - 1, j - 1, j - i), y;
            if (x < j - i)
            {
                y = lt + lcp(i + lt, j + lt, n);
                if (x + y >= j - i)
                {
                    ans.push_back({j - i, i - x - 1, j + y - 1});
                }
            }
        }
    }
```

```
};

vector<Run> runs(const string &s)
{
    // s 是原字符串；返回全部极大周期区间及其最小周期。哈希碰撞概率极低。
    RunHash e(s);
    vector<Run> ans;
    e.getRuns(ans);
    e.flip();
    e.getRuns(ans);
    sort(ans.begin(), ans.end());
    ans.erase(unique(ans.begin(), ans.end()), ans.end());
    return ans;
}
```

`runs(s)` 返回全部 run，每个 `Run{p,l,r}` 给出最小周期 p 与半开区间 $[l,r)$，结果按 `(p,l,r)` 排序。单调栈枚举为线性次候选，每次 LCE 由哈希二分完成，时间 $O(n\log n)$、空间 $O(n)$；QOJ 21648 的百万长度提交 2830439 已通过。实现使用模 998244353、底数 31 的单模滚动哈希，因此概率正确，不能抵抗专门构造的碰撞；要求绝对正确时保留旧 Main-Lorentz/Z-LCE 实现或改用确定性 LCE。full 版使用 32 位字段保存字符串长度、周期和端点，打印版按模板规则展开为 `int`。

4.1.008 Karp-Rabin

Karp-Rabin 用滚动哈希筛选与模式等长的窗口，命中后按需逐字符复核。它与 4.1.001 字符串哈希 是同一份代码，这里不重复：直接依赖其中的 `StrHash::match(s,t)` 即得全部匹配起点，构造 $O(n)$、匹配 $O(n+m)$、空间 $O(n+m)$。Mersenne 单模与进程级随机底数仍是概率正确；要求绝对无误时对每个哈希命中的窗口再逐字符比对一次，或改用确定性的 4.1.002 KMP / 前缀函数、4.1.003 扩展 KMP / Z 函数。

4.1.009 Sunday Algorithm

Sunday 是启发式匹配：失配时不看当前窗口，而是看紧贴窗口右侧的下一个文本字符 `s[i+m]`，把模式中该字符最右出现位置对齐过来，一次可能跳过多达 $m+1$ 个位置。预处理 `sh[c]` 存「模式中字符 c 到末尾再加一的距离」，不含则为 $m+1$。它实现简单、随机数据上很快，但最坏仍是 $O(nm)$，不保证线性。

```
vector<int> sunday(string_view s, string_view p)
{
    // s 是文本串，p 是模式串；返回全部匹配起点，按下标递增排列。
    vector<int> ans;
    if (p.empty())
    {
        for (int i = 0; i <= (int)s.size(); i++)
        {
            ans.push_back(i);
        }
        return ans;
    }
```

```
    array<int, 256> sh;
    sh.fill(p.size() + 1);
    for (int i = 0; i < (int)p.size(); i++)
    {
        sh[(unsigned char)p[i]] = p.size() - i;
    }
    for (int i = 0; i + (int)p.size() <= (int)s.size();)
    {
        int j = 0;
        while (j < (int)p.size() && s[i + j] == p[j])
        {
            j++;
        }
        if (j == (int)p.size())
        {
            ans.push_back(i);
        }
        if (i + (int)p.size() == (int)s.size())
        {
            break;
        }
        i += sh[(unsigned char)(s[i + p.size()])];
    }
    return ans;
}
```

`sunday(s,p)` 返回 `p` 在 `s` 的全部匹配起点；空模式用 $O(n+1)$ 时间和输出空间返回所有位置，非空模式最坏 $O(nm)$ 时间、$O(|\Sigma|+occ)$ 空间，其中 occ 是匹配数。需要最坏线性保证时改用 4.1.002 KMP / 前缀函数 或 4.1.003 扩展 KMP / Z 函数；只找首次出现见 4.1.010 `strstr` 类匹配。

4.1.010 `strstr` 类匹配

只需要模式首次出现位置时，短模式直接用 `string::find` 即可。模式较长、字符分布较散，或要拿同一模式搜索多个文本时，可以用 C++17 的 `boyer_moore_horspool_searcher`：它先按模式建立坏字符位移表，再让 `search` 从模式末端比较，失配后一次跳过多个位置。

```
// s 是文本串，t 是模式串；pos 是首次匹配位置，找不到为 -1，空模式为 0。
boyer_moore_horspool_searcher bm(t.begin(), t.end());
auto it = search(s.begin(), s.end(), bm);
int pos = it == s.end() ? -1 : (int)(it - s.begin());
```

这里不能写 `boyer_moore_horspool_searcher<>`：它的首个模板参数没有默认值。C++17 起像代码中那样省略整个 `<>`，由 `t.begin()` 推导迭代器类型；需要显式写出时，普通 `string` 用 `boyer_moore_horspool_searcher<string::iterator>`，`const string` 用 `string::const_iterator`。模式串 `t` 及其迭代器必须在 `bm` 使用期间保持有效，期间不要修改 `t`；同一个 `bm` 可以传给多次 `search` 以复用预处理。该接口要求模式和文本都是随机访问序列，`string`、`vector`、`array` 均满足；`string` 中含 `\0` 也能正常匹配。

设文本长 n、模式长 m，建立搜索器通常需 $O(m)$ 时间和空间，搜索最坏为 $O((n-m+1)m)$；只有失

配能产生较大跳跃时常数才占优，重复字符等数据可能退化。因此单次短模式优先写 `s.find(t)`，需要确定性最坏 $O(n+m)$ 或全部匹配位置时改用 4.1.002 KMP / 前缀函数 或 4.1.003 扩展 KMP / Z 函数，批量子串比较用 4.1.001 字符串哈希。

4.1.011 通配符模式匹配

带单字符通配符（`s` 与 `t` 两侧的 `any` 都匹配任意一个字符）的匹配可化成卷积。把普通小写字母映成 $1 \dots 26$，另用 $0/1$ 掩码标记非通配符位置，则文本窗口与反转模式在位置 i 完全匹配当且仅当 $\sum_j (x_{i+j} - y_j)^2 p_{i+j} q_j = 0$。每项均非负：任一侧通配符都使对应项为 0，普通位置只有字符相等才为 0。展开后用三次卷积计算 $\sum x^2 pq - 2\sum xypq + \sum y^2 pq$。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.1.003 NTT
vector<bool> wildcard(const string &s, string t, char any = '*')
{
    // s、t 只含 a-z 与 any，any 在两侧都表示通配符；返回每个合法起点是否匹配。
    int n = s.size(), m = t.size();
    if (!m)
    {
        return vector<bool>(n + 1, true);
    }
    if (m > n)
    {
        return {};
    }
    reverse(t.begin(), t.end());
    vector<Z> sx(n), sx2(n), sm(n);
    vector<Z> ty(m), ty2(m), tm(m);
    for (int i = 0; i < n; i++)
    {
        if (s[i] != any)
        {
            int x = s[i] - 'a' + 1;
            assert(1 <= x && x <= 26); // 调试检查，可删。
            sx[i] = x;
            sx2[i] = x * x;
            sm[i] = 1;
        }
    }
    for (int i = 0; i < m; i++)
    {
        if (t[i] != any)
        {
            int y = t[i] - 'a' + 1;
            assert(1 <= y && y <= 26); // 调试检查，可删。
            ty[i] = y;
            ty2[i] = y * y;
            tm[i] = 1;
        }
    }
```

```
    auto c0 = conv(move(sx2), move(tm));
    auto c1 = conv(move(sx), move(ty));
    auto c2 = conv(move(sm), move(ty2));
    vector<bool> ans(n - m + 1);
    for (int i = 0; i + m <= n; i++)
    {
        int k = i + m - 1;
        ans[i] = c0[k] - c1[k] * 2 + c2[k] == Z(0);
    }
    return ans;
}
```

`wildcard(s,t,any)` 把两侧的 `any` 都视为通配符，返回长度 $n-m+1$ 的布尔向量，`ans[i]` 表示窗口在起点 i 是否匹配；接口按 Library Checker 原题约束要求普通字符属于 `a-z`。代码直接复用 10.1.003 NTT 的 `Z` 与 `conv`，这里不重复写 NTT。原题 $m \le 524288$，每项至多 25^2，故真实总和至多 $625m \le 327680000 < 998244353$；单模结果不会绕回 0，判定是确定性的。设补齐后的变换长度为 N，三次卷积的时间为 $O(N \log N)$、空间为 $O(N)$；相较原浮点 FFT 版消除了舍入风险，公共 NTT 的四进制蝶形也更适合这里的整数卷积。无通配符时首选 4.1.002 KMP / 前缀函数。

4.1.012 随机单卷积通配符匹配

确定性版展开平方恒等式要做三次卷积；随机版给每个字节独立取正实数 r_c，文本写 r_c，反转模式的普通字符写 $1/r_c$、通配符写 0。若窗口完全匹配，卷积值必为模式普通字符数 k；错配窗口只有在随机比值恰好抵消到 k 时才会误判。`wildcardMC` 从 2^{48} 个离散实数中取权，理想精确运算模型下可把单轮代数碰撞粗略估为 $O(m/2^{48})$；浮点 FFT 的舍入误差不服从该界，阈值附近仍可能误判，独立换 `seed` 重跑可降低随机碰撞但不能消除数值风险。浮点卷积直接复用 10.1.002 FFT 的 `convolution`；需要确定性结果时使用 4.1.011 通配符模式匹配。

```
// 依赖 10.1.002 FFT
vector<bool> wildcardMC(const string &s, string t, unsigned long long seed, char any = '*')
{
    // s 是文本串，t 是模式串，seed 是本轮随机种子；返回每个合法起点是否匹配。
    int n = s.size();
    int m = t.size();
    if (!m)
    {
        return vector<bool>(n + 1, true);
    }
    if (m > n)
    {
        return {};
    }
    mt19937_64 rng(seed);
    constexpr unsigned long long mask = (1ULL << 48) - 1;
    constexpr long double scale = 1ULL << 48;
    array<long double, 256> w;
    for (long double &x : w)
    {
```

```
        x = 1 + ((rng() & mask) + 0.5L) / scale;
    }
    vector<long double> a(n), b(m);
    for (int i = 0; i < n; i++)
    {
        a[i] = w[(unsigned char)s[i]];
    }
    int fixed = 0;
    for (int i = 0; i < m; i++)
    {
        char c = t[m - 1 - i];
        if (c != any)
        {
            b[i] = 1 / w[(unsigned char)c];
            fixed++;
        }
    }
    auto c = convolution(a, b);
    long double tol = 1E-9L * max<int>(1, fixed);
    vector<bool> ans(n - m + 1);
    for (int i = 0; i + m <= n; i++)
    {
        ans[i] = fabsl(c[i + m - 1] - fixed) <= tol;
    }
    return ans;
}
```

`wildcardMC(s,t,seed,any)` 在普通分支只做一次浮点卷积，单项在 $(1/2, 2)$ 内，卷积绝对值小于 $2m$；时间、峰值空间为 $O(N \log N)$、$O(N)$。它比 4.1.011 通配符模式匹配 少两次卷积，代价是同时承担 Monte Carlo 碰撞和平台相关的浮点阈值误判。`seed` 应由调用方控制；调试时固定种子可复现，多轮结果取交能压低代数假阳性，却会放大浮点假阴性的风险。下面的固定例检查空模式、模式过长、任意字节，以及文本中的 `any` 会按普通字节参与：

```
signed main()
{
    assert(wildcardMC("abc", "", 1).size() == 4);
    assert(wildcardMC("a", "aa", 1).empty());
    assert(!wildcardMC("*b", "ab", 1, '*')[0]);
    assert(wildcardMC("*b", "*b", 1, '*')[0]);
    auto a = wildcardMC("ababa", "a*a", 0);
    assert(a.size() == 3 && a[0] && !a[1] && a[2]);
    string x;
    x.push_back((char)0x00);
    x.push_back((char)0xFF);
    assert(wildcardMC(x, x, 7, '*')[0]);
}
```

4.1.013 编辑距离与 Myers 位并行

编辑距离（Levenshtein 距离）是把 a 改成 b 所需的最少单字符插入、删除、替换次数。设 $D_{i,j}$ 为 a 的前 i 个字符与 b 的前 j 个字符的距离，则 $D_{i,0}=i$、$D_{0,j}=j$，

$$D_{i,j}=\min\bigl(D_{i-1,j}+1,\ D_{i,j-1}+1,\ D_{i-1,j-1}+[a_i\neq b_j]\bigr).$$

`editDist` 按行滚动，只保存一行。

Myers 位并行算法按 t 的字符逐列推进 DP，利用同列相邻两行之差 $D_{i,j}-D_{i-1,j}\in\{-1,0,1\}$：用两个位集 `vp`、`vn` 记录差为 $+1$、-1 的行，`eq[c]` 记录模式中等于 c 的位置。每读入一个字符，用常数次字运算（含一次加法进位）算出新一列的差分，并根据最高位的横向差更新最后一行的值 `ans`。模式长不超过 64 时一整列放进一个机器字。

```cpp
int editDist(const string &a, const string &b)
{
    // a、b 是字节串；返回把 a 改成 b 的最少插入、删除、替换次数。
    // row[j] 滚动保存 a 前 i 个字节与 b 前 j 个字节的编辑距离。
    vector<int> row(b.size() + 1);
    iota(row.begin(), row.end(), 0);
    for (int i = 0; i < (int)a.size(); i++)
    {
        int pre = row[0]; // pre 是上一行的 row[j]，即左上角的值。
        row[0] = i + 1;
        for (int j = 0; j < (int)b.size(); j++)
        {
            int old = row[j + 1];
            row[j + 1] = min({row[j + 1] + 1, row[j] + 1, pre + (a[i] != b[j])});
            pre = old;
        }
    }
    return row.back();
}

int myersDist(const string &p, const string &t)
{
    // p 是至多 64 字节的串，t 是任意字节串；返回二者的编辑距离，与 editDist(p,t) 相同。
    int n = p.size();
    if (!n)
    {
        return t.size();
    }
    assert(n <= 64); // 调试检查，可删。
    // eq[c] 的第 i 位表示 p[i]==c；vp、vn 是 DP 当前列竖向差为 +1、-1 的位集。
    array<unsigned long long, 256> eq{};
    for (int i = 0; i < n; i++)
    {
        eq[(unsigned char)p[i]] |= 1ULL << i;
    }
    unsigned long long vp = ~0ULL, vn = 0, top = 1ULL << (n - 1);
    int ans = n; // ans 是当前列最后一行的值，即 p 与 t 已读前缀的编辑距离。
    for (unsigned char c : t)
```

```
    {
        unsigned long long x = eq[c] | vn;
        unsigned long long d = (((eq[c] & vp) + vp) ^ vp) | eq[c] | vn;
        unsigned long long hp = vn | ~(d | vp), hn = vp & d;
        if (hp & top)
        {
            ans++;
        }
        if (hn & top)
        {
            ans--;
        }
        hp = (hp << 1) | 1;
        hn <<= 1;
        vp = hn | ~(x | hp);
        vn = d & hp;
    }
    return ans;
}
```

`editDist(a,b)` 的时间 $O(|a||b|)$、额外空间 $O(|b|)$。`myersDist(p,t)` 要求 $|p| \le 64$，结果与 `editDist(p,t)` 相同，时间 $O(|t|)$、空间 $O(\sigma)$（$\sigma = 256$ 为字节表）。这里的第 0 行是 $D_{0,j} = j$，对应代码中 `hp = (hp << 1) | 1`；改成 `hp <<= 1` 即第 0 行全为 0，读入 t 的第 j 个字符后 `ans` 变成 p 与以 t_j 结尾的某个子串的最小编辑距离，可用于近似匹配。模式更长时要把位集拆成多个字并逐字传递进位，此处未实现。

只允许插入删除时编辑距离是 $|a| + |b| - 2\,\mathrm{LCS}(a, b)$；匹配含通配符时见 4.1.011 通配符模式匹配。下面先核对经典例子，再用固定种子的 64 字节模式和 10^5 字节文本核对结果：

```
signed main()
{
    assert(editDist("kitten", "sitting") == 3 && myersDist("kitten", "sitting") == 3);
    assert(editDist("", "abc") == 3 && myersDist("", "abc") == 3);
    mt19937 rng(20261015);
    string p(64, 'a'), t(100000, 'a');
    for (auto &c : p)
    {
        c = 'a' + rng() % 4;
    }
    for (auto &c : t)
    {
        c = 'a' + rng() % 4;
    }
    cout << myersDist(p, t) << ' ' << editDist(p, t.substr(0, 200)) << '\n';
    // 期望输出：99936 140
}
```

4.2 Trie 与多模式匹配

4.2.001 Trie

Trie 把一组串按字符建成前缀树：根是空前缀，每条边一个字符，一个节点唯一对应从根到它的前缀，公共前缀只存一次。`cnt[u]` 记录有多少次插入经过 `u`（即以该前缀开头的串数），`end[u]` 记录有多少完整串在此结束，二者语义不同不能混用——前缀计数查 `cnt`、整串计数查 `end`。模板参数 `A` 是字符集大小、`S` 是最小字符，插入的每个字符须落在 $[S, S+A)$。本条与第 3 章 3.6.001 都完整讲字符 Trie，二者都有独立阅读价值，故此处按规范保留完整实现。

```
template <int A = 26, char S = 'a'> struct Trie
{
    // ch[u][c] 是节点 u 沿字符 c 的儿子编号，0 表示不存在。
    vector<array<int, A>> ch;
    // cnt[u] 是经过节点 u 的插入次数，end[u] 是在 u 结束的字符串次数。
    vector<int> cnt, end;

    Trie()
    {
        // 建立只含根节点的空 Trie。
        newNode();
    }

    int newNode()
    {
        // 分配并返回一个清空后的节点编号。
        ch.push_back({});
        cnt.push_back(0);
        end.push_back(0);
        return (int)ch.size() - 1;
    }

    int insert(const string &s)
    {
        // s 是待插入字符串；返回它在 Trie 中的终止节点编号。
        int p = 0;
        cnt[p]++;
        for (char c : s)
        {
            int x = c - S;
            assert(0 <= x && x < A); // 调试检查，可删
            if (!ch[p][x])
            {
                ch[p][x] = newNode();
            }
            p = ch[p][x];
            cnt[p]++;
        }
        end[p]++;
        return p;
    }
```

```
42
43      int find(const string &s) const
44      {
45          // s 是查询串；返回它的终止节点编号，不存在时返回 -1。
46          int p = 0;
47          for (char c : s)
48          {
49              int x = c - S;
50              if (x < 0 || x >= A || !ch[p][x])
51              {
52                  return -1;
53              }
54              p = ch[p][x];
55          }
56          return p;
57      }
58
59      int lcp(const string &s) const
60      {
61          // s 是查询串；返回它与任意已插入字符串的最长公共前缀长度。
62          int p = 0, n = 0;
63          for (char c : s)
64          {
65              int x = c - S;
66              if (x < 0 || x >= A || !ch[p][x])
67              {
68                  break;
69              }
70              p = ch[p][x];
71              n++;
72          }
73          return n;
74      }
75
76      int prefixCnt(const string &s) const
77      {
78          // s 是查询前缀；返回以 s 为前缀的已插入字符串数量。
79          int p = find(s);
80          return p < 0 ? 0 : cnt[p];
81      }
82      int stringCnt(const string &s) const
83      {
84          // s 是查询串；返回 s 被完整插入的次数。
85          int p = find(s);
86          return p < 0 ? 0 : end[p];
87      }
88  };
```

构造 `Trie<A,S> tr`；`insert(s)` 返回终点节点，`find(s)` 返回节点或 `-1`，`prefixCnt(s)`、`stringCnt(s)` 分别查前缀出现次数与整串出现次数，`lcp(s)` 查与字典内任意串的最长公共前缀长度。各操作最坏 $O(|s|)$，空间 $O((1+\sum_i |s_i|)A)$。依次插入 `ab`、`abc`、`ab` 后 `stringCnt("ab")==2`、`prefixCnt("ab")==3`、`lcp("abd")==2`、`find("ac")==-1`。要在长文本里同时匹配多个模式需在其上加 fail 指针，见 4.2.002 AC 自动机；判断子

序列而非前缀用 4.2.004 子序列自动机。

4.2.002 AC 自动机

AC 自动机是在 Trie 上补 fail 指针的多模式匹配机：`fail` 指向当前状态所代表串的最长真后缀状态，`nxt[c]` 是补全后的自动机转移（无真实儿子时顺着 fail 走）。BFS 建图时先令根的儿子以根为 fail，再逐层用父亲的 `nxt` 填儿子的 fail 与自身 `nxt`，使一次文本扫描每步 $O(1)$ 转移即可同时命中所有以当前状态结尾、且在其 fail 链上的模式。累计每个模式出现次数时把文本经过的状态在 fail 树上向上求和。

```cpp
template <int A = 26> class AC
{
  public:
    struct Node
    {
        // ch 是 Trie 实边，nxt 是补全失配后的自动机转移。
        array<int, A> ch, nxt;
        // fail 指向当前状态的最长真后缀状态。
        int fail = 0;
        Node()
        {
            // 无参数；Trie 实边置为 -1（不存在），补全转移默认指向根 0。
            ch.fill(-1);
            nxt.fill(0);
        }
    };

  private:
    // t 是状态表，pos 按插入顺序保存各模式终态。
    vector<Node> t{1};
    vector<int> pos;

  public:
    int add(const string &s, char fst = 'a')
    {
        // s 是非空模式串，fst 是字符集起始字节；返回模式终态。
        assert(!s.empty()); // 调试检查，可删
        int u = 0;
        for (char c : s)
        {
            int x = (unsigned char)c - (unsigned char)fst;
            assert(0 <= x && x < A); // 调试检查，可删
            if (t[u].ch[x] == -1)
            {
                t[u].ch[x] = t.size();
                t.push_back({});
            }
            u = t[u].ch[x];
        }
        pos.push_back(u);
        return u;
```

```
    }

    void build()
    {
        // 补全全部失配链接和自动机转移。
        queue<int> q;
        for (int c = 0; c < A; c++)
        {
            int v = t[0].ch[c];
            if (v == -1)
            {
                continue;
            }
            t[0].nxt[c] = v;
            q.push(v);
        }
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            t[u].nxt = t[t[u].fail].nxt;
            for (int c = 0; c < A; c++)
            {
                int v = t[u].ch[c];
                if (v == -1)
                {
                    continue;
                }
                t[v].fail = t[u].nxt[c];
                t[u].nxt[c] = v;
                q.push(v);
            }
        }
    }

    int step(int u, int c) const
    {
        // u 是当前状态，c 是 [0,A) 内字符编号；返回下一状态。
        assert(0 <= u && u < (int)t.size()); // 调试检查，可删
        assert(0 <= c && c < A); // 调试检查，可删
        return t[u].nxt[c];
    }
    const vector<Node> &nodes() const
    {
        // 返回按编号排列的状态表。
        return t;
    }
    const vector<int> &ends() const
    {
        // 返回各模式按插入顺序对应的终态。
        return pos;
    }
```

```
};
```

`add(s,fst)` 插入非空模式并返回终态；所有模式须使用同一个 `fst`，字符均落在 $[fst, fst + A)$，并在全部插入后只调用一次 `build()`。`step(u,c)` 用已经离散到 $[0, A)$ 的字符走一步；扫描原始文本遇到域外字符时把状态重置为根。插入总长 L 为 $O(L)$，`build` 的时间与空间均为 $O(SA)$（S 为状态数）。空模式不属于本接口；若题目允许空模式，它在长度 n 的文本中出现 $n + 1$ 次，应在自动机外单独统计。下面用经典 fail 链样例核对四个模式的出现次数：

```
signed main()
{
    AC<26> ac;
    vector<string> pat{"he", "she", "his", "hers"};
    for (const string &s : pat)
    {
        ac.add(s);
    }
    ac.build();
    const auto &nd = ac.nodes();
    vector<int> hit(nd.size());
    int u = 0;
    for (char c : string("ahishers"))
    {
        u = ac.step(u, c - 'a');
        hit[u]++;
    }
    vector<int> cnt(nd.size());
    for (int s = 1; s < (int)nd.size(); s++)
    {
        for (int p = s; p > 0; p = nd[p].fail)
        {
            cnt[p] += hit[s];
        }
    }
    vector<int> ans;
    for (int p : ac.ends())
    {
        ans.push_back(cnt[p]);
    }
    assert(ans == vector<int>({1, 1, 1, 1}));
    cout << "1 1 1 1\n";
    return 0;
}
```

真实输出为 `1 1 1 1`。

4.2.003 AC 自动机 fail 树应用

把每个状态向它的 `fail` 连边（反向）就得到 fail 树：一个状态的所有失配后缀都成为它在树上的祖先。于是「文本扫描经过某状态」这一贡献，对该状态到根路径上的每个模式终态都有效——转成子树求和或 DFS 序上的区间操作，即可离线统计每个模式的出现次数、或做子树加 / 祖先查。

```cpp
// 依赖 4.2.002 AC 自动机
template <int A> vector<vector<int>> failTree(const AC<A> &ac)
{
    // ac 是已经 build 的 AC 自动机；返回 fail 边反向后的树形邻接表。
    const auto &t = ac.nodes();
    vector<vector<int>> g(t.size());
    for (int u = 1; u < (int)t.size(); u++)
    {
        g[t[u].fail].push_back(u);
    }
    return g;
}
```

先对 4.2.002 AC 自动机 调用 `build()`，再 `failTree(ac)` 得反向邻接表，`g[u]` 是 fail 树上 `u` 的孩子；建树 $O(S)$，其后可求 DFS 序做子树加与祖先统计。它只重排 4.2.002 AC 自动机 已算好的 fail 指针，不新建自动机；在线逐字符统计而非离线子树和时直接沿 `fail` 上跳即可。

4.2.004 子序列自动机

子序列自动机为「判断某串是否为文本的子序列」而建：`nxt[i][c]` 存位置 i（含）之后字符 c 第一次出现的下标，从右往左递推一遍即可。匹配时从位置 0 起，对模式每个字符跳到它的下一出现位置再加一，任一步越界即非子序列，查询只与模式长度有关。它按文本下标离线建整张表，和回文小节的 4.4.004 SqAM 是不同结构——后者在线维护、这里离线预处理。

```cpp
template <int A = 26> class SubseqAM
{
    // nxt[i][c] 是位置 i（含）之后字符 c 的首次出现位置。
    vector<array<int, A>> nxt;

  public:
    explicit SubseqAM(const vector<int> &s) : nxt(s.size() + 1)
    {
        // s 是 [0,A) 内的文本序列；从右向左建立跳转表。
        int n = s.size();
        nxt.back().fill(n);
        for (int i = n - 1; i >= 0; --i)
        {
            assert(0 <= s[i] && s[i] < A); // 调试检查，可删
            nxt[i] = nxt[i + 1];
            nxt[i][s[i]] = i;
        }
    }

    bool contains(const vector<int> &t) const
    {
        // t 是 [0,A) 内的模式序列；返回它是否为文本的子序列。
        int p = 0, n = nxt.size() - 1;
        for (int c : t)
        {
```

```
            assert(0 <= c && c < A); // 调试检查，可删
            if (p == n || nxt[p][c] == n)
            {
                return false;
            }
            p = nxt[p][c] + 1;
        }
        return true;
    }
};
```

构造 `SubseqAM<A> am(s)`（`s` 为 $[0, A)$ 的整数序列），`am.contains(t)` 判断 `t` 是否为子序列；构造的时间、空间均为 $O((n+1)A)$，长度 m 查询 $O(m)$。它只判子序列，连续模式匹配用 4.2.002 AC 自动机或后缀结构；在线加字符维护同样功能见 4.4.004 SqAM。

4.3 后缀结构

4.3.001 后缀数组：倍增法

后缀数组 `sa` 按字典序给出全部后缀的起点。倍增法先按首字符给每个后缀一个排名，再反复以「长度 2^k 前缀的排名对 (rk_i, rk_{i+2^k})」为关键字排序并重新编号，k 每轮翻倍，$O(\log n)$ 轮后排名互异即得 `sa`。`saNaive` 是直接对后缀 `sort` 的 $O(n^2 \log n)$ 朴素版，只用于很短输入的内部回退；`saDouble` 是上述倍增，每轮用 `sort` 故 $O(n \log^2 n)$。

```
vector<int> saNaive(const vector<int> &s)
{
    // s 是整数序列；返回按字典序排列的全部后缀起点。
    int n = s.size();
    vector<int> sa(n);
    iota(sa.begin(), sa.end(), 0);
    sort(sa.begin(),
         sa.end(),
         [&](int x, int y)
         {
             while (x < n && y < n && s[x] == s[y])
             {
                 x++;
                 y++;
             }
             return y < n && (x == n || s[x] < s[y]);
         });
    return sa;
}

vector<int> saDouble(const vector<int> &s)
{
    // s 是整数序列；返回倍增法求得的后缀数组。
    int n = s.size();
    vector<int> sa(n), rk(n), nrk(n), v = s;
```

```
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());
    for (int i = 0; i < n; i++)
    {
        rk[i] = lower_bound(v.begin(), v.end(), s[i]) - v.begin();
    }
    iota(sa.begin(), sa.end(), 0);
    for (int k = 1; k < n; k *= 2)
    {
        auto cmp = [&](int x, int y)
        {
            if (rk[x] != rk[y])
            {
                return rk[x] < rk[y];
            }
            int a = x + k < n ? rk[x + k] : -1;
            int b = y + k < n ? rk[y + k] : -1;
            return a < b;
        };
        sort(sa.begin(), sa.end(), cmp);
        nrk[sa[0]] = 0;
        for (int i = 1; i < n; i++)
        {
            nrk[sa[i]] = nrk[sa[i - 1]] + cmp(sa[i - 1], sa[i]);
        }
        swap(rk, nrk);
    }
    return sa;
}
```

`saNaive`、`saDouble` 都接受整数序列返回后缀数组下标。朴素版 $O(n^2 \log n)$、倍增版 $O(n \log^2 n)$、空间 $O(n)$。数据大时换线性的 4.3.002 后缀数组：SA-IS（`suffixArray` 综合入口内部对小串就回退到这两者），求两后缀 LCP 再接 4.3.003 LCP / Height / RMQ 应用。统计本质不同子串与出现次数往往 4.3.004 后缀自动机 SAM 更省心。

4.3.002 后缀数组：SA-IS

SA-IS 用诱导排序在线性时间构造后缀数组。把每个位置按其后缀比右邻后缀小 / 大分成 S 型与 L 型，两个相邻 LMS 位置（L 转 S 的边界）之间的 LMS 子串先被诱导排序、命名，再对命名后的缩短串递归求 SA，最后由已排好的 LMS 顺序两趟诱导（先 L 型从左入桶、再 S 型从右入桶）得到全部后缀顺序。实现必须正确处理哨兵、字符离散化与 LMS 命名，故 `saIs` 对小规模直接退回朴素版。`suffixArray` 是对外入口：字符串版按字节离散，整数版先坐标压缩再调 `saIs`。

```
// 依赖 4.3.001 后缀数组：倍增法
vector<int> saIs(const vector<int> &s, int m)
{
    // s 是整数序列，元素须在 [0,m] 内，m 是最大符号值；返回后缀数组。
    int n = s.size();
    if (n == 0)
```

```
    {
        return {};
    }
    if (n == 1)
    {
        return {0};
    }
    if (n == 2)
    {
        return s[0] < s[1] ? vector<int>{0, 1} : vector<int>{1, 0};
    }
    if (n < 10)
    {
        return saNaive(s);
    }
    if (n < 40)
    {
        return saDouble(s);
    }

    vector<int> sa(n, -1), sl(m + 1), ss(m + 1);
    vector<bool> ls(n);
    for (int i = n - 2; i >= 0; i--)
    {
        ls[i] = s[i] == s[i + 1] ? ls[i + 1] : s[i] < s[i + 1];
    }
    for (int i = 0; i < n; i++)
    {
        if (ls[i])
        {
            sl[s[i] + 1]++;
        }
        else
        {
            ss[s[i]]++;
        }
    }
    for (int i = 0; i <= m; i++)
    {
        ss[i] += sl[i];
        if (i < m)
        {
            sl[i + 1] += ss[i];
        }
    }

    auto induce = [&](const vector<int> &lms)
    {
        fill(sa.begin(), sa.end(), -1);
        vector<int> b = ss;
        for (int x : lms)
        {
```

```
            if (x < n)
            {
                sa[b[s[x]]++] = x;
            }
        }
        b = sl;
        sa[b[s[n - 1]]++] = n - 1;
        for (int i = 0; i < n; i++)
        {
            int x = sa[i];
            if (x > 0 && !ls[x - 1])
            {
                sa[b[s[x - 1]]++] = x - 1;
            }
        }
        b = sl;
        for (int i = n - 1; i >= 0; i--)
        {
            int x = sa[i];
            if (x > 0 && ls[x - 1])
            {
                sa[--b[s[x - 1] + 1]] = x - 1;
            }
        }
    };

    vector<int> id(n + 1, -1), lms;
    for (int i = 1; i < n; i++)
    {
        if (!ls[i - 1] && ls[i])
        {
            id[i] = lms.size();
            lms.push_back(i);
        }
    }
    induce(lms);
    int z = lms.size();
    if (z)
    {
        vector<int> ord;
        for (int x : sa)
        {
            if (x >= 0 && id[x] != -1)
            {
                ord.push_back(x);
            }
        }
        vector<int> ns(z);
        int nm = 0;
        ns[id[ord[0]]] = 0;
        for (int i = 1; i < z; i++)
        {
```

```
            int x = ord[i - 1], y = ord[i];
            int ex = id[x] + 1 < z ? lms[id[x] + 1] : n;
            int ey = id[y] + 1 < z ? lms[id[y] + 1] : n;
            bool same = ex - x == ey - y;
            while (same && x < ex)
            {
                if (s[x] != s[y])
                {
                    same = false;
                }
                x++;
                y++;
            }
            if (same && (x == n || s[x] != s[y]))
            {
                same = false;
            }
            if (!same)
            {
                nm++;
            }
            ns[id[ord[i]]] = nm;
        }
        vector<int> rsa = saIs(ns, nm);
        for (int i = 0; i < z; i++)
        {
            ord[i] = lms[rsa[i]];
        }
        induce(ord);
    }
    return sa;
}

template <class T> vector<int> suffixArray(const vector<T> &s)
{
    // s 是可比较序列；先离散化，再返回按字典序排列的全部后缀起点。
    int n = s.size();
    vector<int> p(n), a(n);
    iota(p.begin(), p.end(), 0);
    sort(p.begin(),
         p.end(),
         [&](int x, int y)
         {
             return s[x] < s[y];
         });
    int m = 0;
    for (int i = 0; i < n; i++)
    {
        if (i && s[p[i - 1]] != s[p[i]])
        {
            m++;
        }
```

```
        a[p[i]] = m;
    }
    return saIs(a, m);
}

vector<int> suffixArray(const string &s)
{
    // s 按无符号字节比较；返回按后缀字典序排列的起点下标。
    vector<int> a;
    a.reserve(s.size());
    for (unsigned char c : s)
    {
        a.push_back(c);
    }
    return saIs(a, 255);
}
```

字符串入口按无符号字节比较，SA-IS 为 $O(n+256)$ 时间、$O(n+256)$ 空间；泛型 `vector` 入口先排序离散化，故为 $O(n\log n)$ 时间、$O(n)$ 空间。得到 `sa` 后用 4.3.003 LCP / Height / RMQ 应用 求任意两后缀 LCP；需要压缩后缀树边才用 4.3.008 隐式后缀树 Ukkonen。下面检查空串、重复字符、一般字符串与无符号字节顺序：

```
signed main()
{
    assert(suffixArray("").empty());
    assert(suffixArray("aaaa") == vector<int>({3, 2, 1, 0}));
    assert(suffixArray("banana") == vector<int>({5, 3, 1, 0, 4, 2}));
    string bytes;
    bytes.push_back((char)0xFF);
    bytes.push_back((char)0x00);
    bytes.push_back((char)0x80);
    assert(suffixArray(bytes) == vector<int>({1, 2, 0}));
}
```

4.3.003 LCP / Height / RMQ 应用

给定后缀数组，Kasai 算法线性求 Height：`ht[i]` 是排名相邻的 `sa[i]`、`sa[i+1]` 两后缀的 LCP。关键性质是删去首字符后 LCP 至多减一，故按原串下标顺序处理、`h` 只减不重置能均摊 $O(n)$。任意两后缀 $s[i..)$、$s[j..)$ 的 LCP 等于它们排名区间内 Height 的最小值，于是对 `ht` 建 ST 表即可 $O(1)$ 回答，`query` 先把下标换成排名再取区间最小。它不构造 SA，`sa` 必须是合法排列、空串不可查。

```
namespace suffix_lcp
{
vector<int> lcpArray(const string &s, const vector<int> &sa)
{
    // s 是原字符串，sa 是全部后缀按字典序排列的起点；返回相邻后缀 LCP，ans[i]=lcp(sa[i],sa[i+1])。
    auto n = (int)s.size();
    assert((int)sa.size() == n); // 调试检查，可删
    vector<int> rk(n), ht(max<int>(0, n - 1));
```

```
    vector<char> seen(n);
    for (int i = 0; i < n; i++)
    {
        assert(0 <= sa[i] && sa[i] < n && !seen[sa[i]]); // 调试检查，可删
        seen[sa[i]] = true;
        rk[sa[i]] = i;
    }
    int h = 0;
    for (int i = 0; i < n; i++)
    {
        if (rk[i] == n - 1)
        {
            h = 0;
            continue;
        }
        auto j = sa[rk[i] + 1];
        while (i + h < n && j + h < n && s[i + h] == s[j + h])
        {
            h++;
        }
        ht[rk[i]] = h;
        if (h)
        {
            h--;
        }
    }
    return ht;
}

struct LcpRMQ
{
    int n = 0;              // n 是字符串长度。
    vector<int> sa;         // sa[r] 是排名 r 的后缀起点。
    vector<int> rk;         // rk[i] 是后缀 s[i..n) 的排名。
    vector<int> ht;         // ht[i] 是 sa[i] 与 sa[i+1] 的最长公共前缀长度。
    vector<int> lg;         // lg[len] 是不超过 len 的最大二次幂指数。
    vector<vector<int>> st; // st[k][i] 是 ht[i..i+2^k) 的最小值。

    LcpRMQ(const string &s, vector<int> order)
        : n((int)s.size()), sa(move(order)), rk(n), ht(lcpArray(s, sa)), lg(n + 1)
    {
        // s 是原字符串，order 是后缀数组；建立 Height 的 ST 表。
        for (int i = 0; i < n; i++)
        {
            rk[sa[i]] = i;
        }
        for (int i = 2; i <= n; i++)
        {
            lg[i] = lg[i / 2] + 1;
        }
        if (n <= 1)
        {
```

```
            return;
        }
        st.assign(lg[n - 1] + 1, {});
        st[0] = ht;
        for (int k = 1; k < (int)st.size(); k++)
        {
            auto len = 1LL << k;
            st[k].resize(n - len);
            for (int i = 0; i + len <= n - 1; i++)
            {
                st[k][i] = min(st[k - 1][i], st[k - 1][i + len / 2]);
            }
        }
    }

    int queryRank(int l, int r) const
    {
        // l、r 是后缀排名且 l<r；返回 sa[l] 与 sa[r] 的最长公共前缀长度。
        assert(0 <= l && l < r && r < n); // 调试检查，可删
        auto k = lg[r - l];
        return min(st[k][l], st[k][r - (1LL << k)]);
    }

    int query(int i, int j) const
    {
        // i、j 是两个后缀起点；返回 s[i..n) 与 s[j..n) 的最长公共前缀长度。
        assert(0 <= i && i < n && 0 <= j && j < n); // 调试检查，可删
        if (i == j)
        {
            return n - i;
        }
        auto x = rk[i];
        auto y = rk[j];
        if (x > y)
        {
            swap(x, y);
        }
        return queryRank(x, y);
    }
};
}
```

实现放在命名空间 `suffix_lcp`。只要 Height 用 `suffix_lcp::lcpArray(s,sa)`；要多次询问就建 `suffix_lcp::LcpRMQ q(s,sa)` 再 `q.query(i,j)` 查后缀 $s[i..n)$ 与 $s[j..n)$ 的 LCP；构造 Height 与 ST 表 $O(n\log n)$ 时间与空间,单次查询 $O(1)$。以 `s="banana"`、`sa={5,3,1,0,4,2}` 为例,Height 为 `{1,3,0,0,2}`，`query(1,3)=3`、`query(0,2)=0`。SA 由 4.3.001 后缀数组：倍增法 或 4.3.002 后缀数组：SA-IS 提供；区间受限的本质不同子串不能套整串公式，见 4.3.010 区间本质不同子串。

4.3.004 后缀自动机 SAM

后缀自动机把一个串的全部子串压进至多 $2n-1$ 个状态：每个状态代表一组 endpos（结束位置集合）相同的子串，这组子串长度连续，`link` 指向其最长真后缀所在的（endpos 更大的）状态。在线加字符 `add`：新建末状态 `q`，沿 `last` 的 `link` 链把没有 `x` 转移的状态都指向 `q`；遇到已有转移 `v` 时，若 `len[v]==len[p]+1` 直接令 `link[q]=v`，否则克隆 `v` 为 `z`（复制转移、`len` 设为 `len[p]+1`）分裂出正确长度区间。不同子串数为 $\sum len[u]-len[link[u]]$；`count` 按 `len` 基数排序后沿 `link` 累加初始 `cnt=1`（只在非克隆末状态）得到每个状态代表子串的出现次数。

```cpp
template <int A = 26, char S = 'a'> struct SAM
{
    struct Node
    {
        array<int, A> next{}; // next[c] 是字符 c 的转移，0 表示不存在（根没有入边）。
        // link 是后缀链接，len 是最长长度，cnt 是当前出现次数，first 是一个最长串的末位置。
        int link = -1, len = 0, cnt = 0, first = -1;
    };

    vector<Node> t{{}};
    // last 是当前整串对应的末状态。
    int last = 0;
    // counted 表示出现次数是否已经沿后缀链接上传。
    bool counted = false;

    SAM() = default;
    SAM(const string &s)
    {
        // s 是原字符串；按顺序插入其全部字符。
        for (char c : s)
        {
            add(c);
        }
    }

    void add(char c)
    {
        // c 是要追加到原串末尾的字符。
        assert(!counted); // 调试检查，可删
        int x = c - S;
        assert(0 <= x && x < A); // 调试检查，可删
        int p = last, q = t.size();
        t.push_back({});
        t[q].len = t[p].len + 1;
        t[q].cnt = 1;
        t[q].first = t[q].len - 1;
        while (p != -1 && !t[p].next[x])
        {
            t[p].next[x] = q;
            p = t[p].link;
        }
        if (p == -1)
```

```
        {
            t[q].link = 0;
        }
        else
        {
            int v = t[p].next[x];
            if (t[v].len == t[p].len + 1)
            {
                t[q].link = v;
            }
            else
            {
                int z = t.size();
                t.push_back(t[v]);
                t[z].len = t[p].len + 1;
                t[z].cnt = 0;
                while (p ≠ -1 && t[p].next[x] == v)
                {
                    t[p].next[x] = z;
                    p = t[p].link;
                }
                t[v].link = t[q].link = z;
            }
        }
        last = q;
    }

    int find(const string &s) const
    {
        // s 是查询串；返回对应状态，不存在或含域外字符时返回 -1。
        int p = 0;
        for (char c : s)
        {
            int x = c - S;
            if (x < 0 || x >= A || !t[p].next[x])
            {
                return -1;
            }
            p = t[p].next[x];
        }
        return p;
    }

    long long distinct() const
    {
        // 返回当前自动机表示的本质不同非空子串数。
        long long ans = 0;
        for (int i = 1; i < (int)t.size(); i++)
        {
            ans += t[i].len - t[t[i].link].len;
        }
        return ans;
```

```
    }

    void count()
    {
        // 把各末状态的计数沿后缀链接上传；重复调用不再二次累计。
        if (counted)
        {
            return;
        }
        int n = 0;
        for (auto &v : t)
        {
            chmax(n, v.len);
        }
        vector<int> c(n + 1), p(t.size());
        for (auto &v : t)
        {
            c[v.len]++;
        }
        partial_sum(c.begin(), c.end(), c.begin());
        for (int i = (int)t.size() - 1; i >= 0; i--)
        {
            p[--c[t[i].len]] = i;
        }
        for (int i = (int)p.size() - 1; i; i--)
        {
            int x = p[i];
            t[t[x].link].cnt += t[x].cnt;
        }
        counted = true;
    }

    int occ(const string &s)
    {
        // s 是查询串；返回它在原串中的出现次数，空串返回原串长度加一。
        if (s.empty())
        {
            return t[last].len + 1;
        }
        int p = find(s);
        if (p < 0)
        {
            return 0;
        }
        count();
        return t[p].cnt;
    }
};

array<int, 4> commonSubstr(const string &a, const string &b)
{
    // a、b 是小写字符串；返回 {a_l,a_r,b_l,b_r} 的一组最长公共子串半开区间。
```

```
    SAM<26, 'a'> sam(a);
    int p = 0, len = 0, best = 0;
    array<int, 4> ans{};
    for (int i = 0; i < (int)b.size(); i++)
    {
        int x = b[i] - 'a';
        while (p && !sam.t[p].next[x])
        {
            p = sam.t[p].link;
            len = min(len, sam.t[p].len);
        }
        if (sam.t[p].next[x])
        {
            p = sam.t[p].next[x];
            len++;
        }
        else
        {
            len = 0;
        }
        if (len > best)
        {
            best = len;
            int end = sam.t[p].first;
            ans = {end - len + 1, end + 1, i - len + 1, i + 1};
        }
    }
    return ans;
}
```

构造 `SAM<A,S> sam(s)`；`find(s)` 定位状态，`distinct()` 返回本质不同子串数，`occ(t)` 返回 `t` 在原串中的出现次数，空串按出现位置定义返回 $n+1$。`commonSubstr(a,b)` 返回一组 `{a_l,a_r,b_l,b_r}`，两段半开区间长度相等且为最长公共子串长度；无公共字符时返回全零区间。首次非空查询触发一次 $O(n)$ 的 `count()`；调用 `count()` 或首次查询非空串后不得再 `add`。构造时间和空间均为 $O(nA)$；非空串的状态数不超过 $2n-1$，空串只有根状态；查长度 m 的串需 $O(m)$。最长公共子串在扫描第二串时也只沿后缀链接回退，故为 $O(|a|+|b|)$ 时间与 $O(|a|)$ 空间。下面用会产生状态分裂的 `ababa` 直接核对本质不同子串数和各节点次数：

```
signed main()
{
    SAM<3, 'a'> sam("ababa");
    assert(sam.distinct() == 9);
    assert(sam.occ("a") == 3);
    assert(sam.occ("b") == 2);
    assert(sam.occ("ab") == 2);
    assert(sam.occ("aba") == 2);
    assert(sam.occ("bab") == 1);
    assert(sam.occ("c") == 0);
    assert(sam.occ("") == 6);
    cout << "9 3 2 2 2 1 0 6\n";
    return 0;
```

```
}
```

真实输出为 `9 3 2 2 2 1 0 6`。

4.3.005 广义 SAM / ExSAM

广义 SAM 把多条串的全部子串压进一个自动机。离线版先把所有串插进公共 Trie（`pass` 记录每个 Trie 节点被多少串经过），再按 BFS 顺序对 Trie 的每条边调用 `extend(p,c)` 扩展 SAM：相同前缀天然共享状态，且不依赖插入顺序。`extend` 处理已有转移时同样按 `len[q]==len[p]+1` 决定直接复用还是克隆分裂。建完后把 Trie 节点的 `pass` 加到对应 SAM 状态，再按 `len` 降序沿 `link` 上推，得到每个子串跨全部串的出现次数。

```
template <int A = 26, char B = 'a'> struct ExSAM
{
    struct Node
    {
        array<int, A> ch{}; // ch[c] 是 SAM 中字符 c 的转移，0 表示不存在。
        // link 是后缀链接，len 是该状态的最长长度。
        int link = -1, len = 0;
        // cnt 是该状态所代表子串在全部输入串中的出现次数。
        long long cnt = 0;
    };

    struct TrieNode
    {
        array<int, A> ch{}; // ch[c] 是 Trie 中字符 c 的儿子，0 表示不存在。
        // pass 是经过该 Trie 节点的输入串数，重复插入重复计算。
        long long pass = 0;
    };

    vector<Node> t{{}};
    vector<TrieNode> tr{{}};
    // done 记录是否已经 build，emptyCnt 是空串在全部输入串中的出现位置数。
    bool done = false;
    long long emptyCnt = 0;

    void insert(const string &s)
    {
        // s 是待加入集合的整条字符串；相邻字符串之间不产生跨边界子串。
        assert(!done); // 调试检查，可删
        emptyCnt += (int)s.size() + 1;
        int u = 0;
        for (char c : s)
        {
            int x = c - B;
            assert(0 <= x && x < A); // 调试检查，可删
            if (!tr[u].ch[x])
            {
                tr[u].ch[x] = tr.size();
                tr.push_back({});
```

```
            }
            u = tr[u].ch[x];
            tr[u].pass++;
        }
    }

    int extend(int p, int c)
    {
        // p 是当前 SAM 状态，c 是离散字符；返回扩展后的状态。
        int q = t[p].ch[c];
        if (q)
        {
            if (t[q].len == t[p].len + 1)
            {
                return q;
            }
            int nq = t.size();
            t.push_back(t[q]);
            t[nq].len = t[p].len + 1;
            t[nq].cnt = 0;
            while (p != -1 && t[p].ch[c] == q)
            {
                t[p].ch[c] = nq;
                p = t[p].link;
            }
            t[q].link = nq;
            return nq;
        }

        int u = t.size();
        t.push_back({});
        t[u].len = t[p].len + 1;
        while (p != -1 && !t[p].ch[c])
        {
            t[p].ch[c] = u;
            p = t[p].link;
        }
        if (p == -1)
        {
            t[u].link = 0;
            return u;
        }
        q = t[p].ch[c];
        if (t[q].len == t[p].len + 1)
        {
            t[u].link = q;
            return u;
        }
        int nq = t.size();
        t.push_back(t[q]);
        t[nq].len = t[p].len + 1;
        t[nq].cnt = 0;
```

```
        while (p != -1 && t[p].ch[c] == q)
        {
            t[p].ch[c] = nq;
            p = t[p].link;
        }
        t[q].link = t[u].link = nq;
        return u;
    }

    void build()
    {
        // 从公共 Trie 构造广义 SAM，并上传各状态的跨串出现次数。
        assert(!done); // 调试检查，可删
        done = true;
        vector<int> pos(tr.size());
        queue<int> q;
        q.push(0);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int c = 0; c < A; c++)
            {
                int v = tr[u].ch[c];
                if (!v)
                {
                    continue;
                }
                pos[v] = extend(pos[u], c);
                t[pos[v]].cnt += tr[v].pass;
                q.push(v);
            }
        }
        int mx = 0;
        for (const Node &v : t)
        {
            mx = max(mx, v.len);
        }
        vector<int> c(mx + 1), p(t.size());
        for (const Node &v : t)
        {
            c[v.len]++;
        }
        partial_sum(c.begin(), c.end(), c.begin());
        for (int u = (int)t.size() - 1; u >= 0; u--)
        {
            p[--c[t[u].len]] = u;
        }
        for (int i = (int)p.size() - 1; i > 0; i--)
        {
            int u = p[i];
            if (t[u].link >= 0)
```

```
            {
                t[t[u].link].cnt += t[u].cnt;
            }
        }
    }

    int find(const string &s) const
    {
        // s 是查询串；返回对应状态，不存在或含域外字符时返回 -1。
        int u = 0;
        for (char c : s)
        {
            int x = c - B;
            if (x < 0 || x >= A)
            {
                return -1;
            }
            u = t[u].ch[x];
            if (!u)
            {
                return -1;
            }
        }
        return u;
    }

    long long count(const string &s) const
    {
        // s 是查询串；返回它在全部输入串中的出现次数，空串按各串分别计位置。
        assert(done); // 调试检查，可删
        if (s.empty())
        {
            return emptyCnt;
        }
        int u = find(s);
        return u < 0 ? 0 : t[u].cnt;
    }

    long long distinct() const
    {
        // 返回全部输入串并集中的本质不同非空子串数。
        assert(done); // 调试检查，可删
        long long ans = 0;
        for (int i = 1; i < t.size(); i++)
        {
            ans += t[i].len - t[t[i].link].len;
        }
        return ans;
    }
};
```

逐串 `insert(s)` 建 Trie，只能调用一次 `build()`；之后 `find`、`count(s)` 和 `distinct()` 可查。`count("")`

返回 $\sum_i(|s_i|+1)$，域外查询字符直接视为不存在。按长度计数排序上传状态后，总长 L 的构造时间和空间均为 $O((L+1)A)$，状态数不超过 $2L+1$，单次查询为 $O(|s|)$。要边到达边在线加串而非先收齐用 4.3.006 在线广义 SAM；单串场景直接 4.3.004 后缀自动机 SAM。

4.3.006 在线广义 SAM

在线广义 SAM 不预先收集全部文本：每来一条新串就从根开始逐字符 `extend`。与离线版的区别只在于没有 Trie 前置——`extend(p,c)` 若发现转移已存在但 `len[q]≠len[p]+1` 就克隆分裂到正确长度并沿 `link` 改边，从而在任意插入顺序下都维持广义 SAM 的正确结构，适合字典串陆续到达、需要随时查询的场景。

```cpp
template <int A = 26, char B = 'a'> struct OnlineExSAM
{
    struct Node
    {
        // ch 是字符转移，link 是后缀链接，len 是该状态的最长长度。
        array<int, A> ch{};
        int link = -1, len = 0;
    };

    // tr 是状态池。
    vector<Node> tr{{}};

    int extend(int p, int c)
    {
        // p 是当前状态，c 是离散字符；返回扩展后的状态。
        int q = tr[p].ch[c];
        if (q)
        {
            if (tr[q].len == tr[p].len + 1)
            {
                return q;
            }
            int nq = tr.size();
            tr.push_back(tr[q]);
            tr[nq].len = tr[p].len + 1;
            while (p != -1 && tr[p].ch[c] == q)
            {
                tr[p].ch[c] = nq;
                p = tr[p].link;
            }
            tr[q].link = nq;
            return nq;
        }

        int u = tr.size();
        tr.push_back({});
        tr[u].len = tr[p].len + 1;
        while (p != -1 && !tr[p].ch[c])
        {
            tr[p].ch[c] = u;
```

```
            p = tr[p].link;
        }
        if (p == -1)
        {
            tr[u].link = 0;
            return u;
        }
        q = tr[p].ch[c];
        if (tr[q].len == tr[p].len + 1)
        {
            tr[u].link = q;
            return u;
        }
        int nq = tr.size();
        tr.push_back(tr[q]);
        tr[nq].len = tr[p].len + 1;
        while (p != -1 && tr[p].ch[c] == q)
        {
            tr[p].ch[c] = nq;
            p = tr[p].link;
        }
        tr[q].link = tr[u].link = nq;
        return u;
    }

    void insert(const string &s)
    {
        // s 是新字符串；每条串从根开始，不产生跨边界子串。
        int p = 0;
        for (char c : s)
        {
            int x = c - B;
            assert(0 <= x && x < A); // 调试检查，可删
            p = extend(p, x);
        }
    }

    int find(const string &s) const
    {
        // s 是查询串；返回对应状态，不存在或含域外字符时返回 -1。
        int p = 0;
        for (char c : s)
        {
            int x = c - B;
            if (x < 0 || x >= A)
            {
                return -1;
            }
            p = tr[p].ch[x];
            if (!p)
            {
                return -1;
```

```
            }
        }
        return p;
    }

    long long distinct() const
    {
        // 返回当前所有输入串的本质不同非空子串数。
        long long ans = 0;
        for (int p = 1; p < (int)tr.size(); p++)
        {
            ans += tr[p].len - tr[tr[p].link].len;
        }
        return ans;
    }
};
```

构造 `OnlineExSAM<> sam`，每条新串 `insert(s)`，`distinct()` 随时返回当前串集合的本质不同子串总数；当前接口不分别记录各串出现次数。设总长 L，构造和空间均为 $O((L+1)A)$，状态数 $\le 2L+1$，`find` 为 $O(|s|)$。依次插入 `ababa`、`bcb` 后子串并集为 13 个，`distinct()==13`、`find("aba")>=0`、`find("ac")==-1`。需要跨串出现次数用离线的 4.3.005 广义 SAM / ExSAM。

4.3.007 SAMSA / CDAWG

把 SAM 转移 DAG 中入度、出度都为 1 的非关键状态压进边标签，得到紧凑的 C-SAM 视图：保留根以及入度或出度不为 1 的状态，每条保留边沿唯一出链走到下一个关键状态、把途经字符拼成标签。它减少显式节点而仍能判子串——SAM 路径的任意前缀都是合法子串，故查询允许停在标签中间。注意若题目要求接受全部后缀的严格 CDAWG，还须另标后缀终止态，本条只做子串判定视图。

```
// 依赖 4.3.004 后缀自动机 SAM
template <int A = 26, char B = 'a'> struct CSAM
{
    struct Edge
    {
        // to 是目标关键状态，s 是压缩路径的非空标签。
        int to = 0;
        string s;
    };

    // g 是压缩后的 DAG；id 把 SAM 状态映射到关键状态，非关键状态为 -1。
    vector<vector<Edge>> g;
    vector<int> id;

    CSAM(const SAM<A, B> &sam)
    {
        // sam 是已经建好的后缀自动机；压缩入度、出度均为 1 的路径。
        int n = sam.t.size();
        vector<int> in(n), out(n);
        for (int u = 0; u < n; u++)
```

```
        {
            for (int c = 0; c < A; c++)
            {
                int v = sam.t[u].next[c];
                if (v)
                {
                    in[v]++, out[u]++;
                }
            }
        }
        id.assign(n, -1);
        for (int u = 0; u < n; u++)
        {
            if (u == 0 || in[u] != 1 || out[u] != 1)
            {
                id[u] = g.size();
                g.push_back({});
            }
        }
        for (int u = 0; u < n; u++)
        {
            if (id[u] < 0)
            {
                continue;
            }
            for (int c = 0; c < A; c++)
            {
                int v = sam.t[u].next[c];
                if (!v)
                {
                    continue;
                }
                string s(1, (char)(B + c));
                while (id[v] < 0)
                {
                    for (int d = 0; d < A; d++)
                    {
                        int w = sam.t[v].next[d];
                        if (w)
                        {
                            s += (char)(B + d);
                            v = w;
                            break;
                        }
                    }
                }
                g[id[u]].push_back({id[v], move(s)});
            }
        }
    }

    bool contains(const string &t) const
```

```
    {
        // t 是查询串；返回它是否对应根出发的一条路径前缀，空串返回 true。
        int u = 0, p = 0;
        while (p < (int)t.size())
        {
            const Edge *e = nullptr;
            for (const auto &cur : g[u])
            {
                if (cur.s[0] == t[p])
                {
                    e = &cur;
                    break;
                }
            }
            if (!e)
            {
                return false;
            }
            for (char c : e->s)
            {
                if (p == (int)t.size())
                {
                    return true;
                }
                if (t[p++] != c)
                {
                    return false;
                }
            }
            u = e->to;
        }
        return true;
    }
};
```

先 `SAM<> sam(s)` 再 `CSAM<> cs(sam)`，`cs.contains(t)` 判断 `t` 是否为 `s` 的子串；边标签可能多字符，匹配时必须逐字符核对而非只看首字符。构造与 4.3.004 后缀自动机 SAM 同量级，时间、空间均为 $O((n+1)A)$。原串 `ababa` 上 `contains("")`、`contains("aba")`、`contains("bab")` 为真，`contains("abb")`、`contains("abac")` 为假。只判子串不需压缩时直接用 4.3.004 后缀自动机 SAM 的 `find`。

4.3.008 隐式后缀树 Ukkonen

Ukkonen 在线构造压缩后缀 Trie：边用原串区间 $[left, right)$ 表示，逐字符加入时维护活动点，借后缀链接从一个后缀跳到下一个，内部节点只在边分裂时创建。直接传入原串得到隐式后缀树，部分后缀会结束在边内而没有独立叶子；若每个后缀都要对应显式叶子，先在原串末尾追加一个未出现且小于原字母表所有字符的哨兵。

```
class SuffixTree
{
```

```
public:
  struct Node
  {
      // [left,right) 是父边在原串中的标签区间，parent 是父节点，suffixLink 是后缀链接。
      int left = 0, right = 0, parent = -1, suffixLink = -1;
      // children 把出边首字符映射到儿子节点。
      map<char, int> children;
      int length() const
      {
          // 返回当前压缩边长度。
          return right - left;
      }
  };

private:
  struct State
  {
      // node 是当前边的下端节点，position 是从该边起点走过的字符数。
      int node, position;
  };

  // text 是建树所用原串，nodes 是后缀树节点池。
  string text;
  vector<Node> nodes;
  State active{0, 0};

  State go(State state, int left, int right)
  {
      // state 是活动点，[left,right) 是待匹配文本；返回匹配后的活动点，失败返回 {-1,-1}。
      while (left < right)
      {
          if (state.position == nodes[state.node].length())
          {
              auto it = nodes[state.node].children.find(text[left]);
              if (it == nodes[state.node].children.end())
              {
                  return {-1, -1};
              }
              state = {it->second, 0};
          }
          else
          {
              if (text[nodes[state.node].left + state.position] != text[left])
              {
                  return {-1, -1};
              }
              int length = min(right - left, nodes[state.node].length() - state.position);
              state.position += length;
              left += length;
          }
      }
      return state;
```

```
    }

    int split(State state)
    {
        // state 是活动点；若它在边内则分裂该边，返回活动点对应的显式节点。
        if (state.position == nodes[state.node].length())
        {
            return state.node;
        }
        if (!state.position)
        {
            return nodes[state.node].parent;
        }
        Node original = nodes[state.node];
        int middle = nodes.size();
        nodes.push_back({original.left, original.left + state.position, original.parent, -1,
↪   {}});
        nodes[original.parent].children[text[original.left]] = middle;
        nodes[middle].children[text[original.left + state.position]] = state.node;
        nodes[state.node].parent = middle;
        nodes[state.node].left += state.position;
        return middle;
    }

    int suffixLink(int node)
    {
        // node 是显式后缀树节点；惰性计算并返回它的后缀链接。
        if (nodes[node].suffixLink != -1)
        {
            return nodes[node].suffixLink;
        }
        if (nodes[node].parent == -1)
        {
            return 0;
        }
        int parLink = suffixLink(nodes[node].parent);
        int skip = nodes[node].parent == 0;
        return nodes[node].suffixLink =
                    split(go({parLink, nodes[parLink].length()}, nodes[node].left + skip,
↪   nodes[node].right));
    }

    void extend(int position)
    {
        // position 是本次追加字符在 text 中的下标；补齐所有新后缀。
        while (true)
        {
            State next = go(active, position, position + 1);
            if (next.node != -1)
            {
                active = next;
                return;
```

```
            }
            int middle = split(active), leaf = nodes.size();
            nodes.push_back({position, (int)text.size(), middle, -1, {}});
            nodes[middle].children[text[position]] = leaf;
            active.node = suffixLink(middle);
            active.position = nodes[active.node].length();
            if (!middle)
            {
                break;
            }
        }
    }

  public:
    // 需要显式叶子时，在末尾加入一个小于字母表中所有字符的哨兵。
    explicit SuffixTree(string text) : text(move(text))
    {
        // text 是原字符串；按顺序完成 Ukkonen 在线构造。
        nodes.reserve(this->text.size() * 2 + 1);
        nodes.push_back({0, 0, -1, 0, {}});
        for (int i = 0; i < this->text.size(); ++i)
        {
            extend(i);
        }
    }

    const vector<Node> &getNodes() const
    {
        // 返回后缀树节点池的只读引用，顺序与节点编号一致。
        return nodes;
    }
    const string &getText() const
    {
        // 返回建树所用的完整文本。
        return text;
    }
};
```

构造 `SuffixTree st(s)` 得到隐式树；若 `s` 不含 `\0`，可用 `SuffixTree st(s + '\0')` 得到每个后缀均有独立叶子的显式树。`getNodes()`、`getText()` 分别返回节点池与实际建树文本，边 $[left, right)$ 与 `children` 描述压缩边；字符集大小为 A 时构造摊还 $O(n\log(A+1)+1)$，空间 $O(n+1)$。只需统计子串时，4.3.004 后缀自动机 SAM 更易写。

4.3.009 后缀平衡树（前插、回滚与动态排名）

后缀平衡树维护只在串首加入字符的全部后缀。把当前串反向保存为 `r` 后，每个后缀对应 `r` 的一个前缀终点；继续前插只会在 `r` 末尾追加字符，旧前缀代表的字符串不变，因而旧结点之间的字典序不会改变。Treap 用反向前缀的 LCP 比较键，子树大小支持排名和第 k 后缀；新后缀只与排序后的前驱、后继比较 LCP，即可累计本质不同子串数。哈希采用自然溢出的 64 位多项式哈希，是速度优先的 Monte Carlo 实现。

```cpp
struct SuffixBST
{
    struct Node
    {
        int l = 0;                  // l 保存 Treap 左儿子编号。
        int r = 0;                  // r 保存 Treap 右儿子编号。
        int sz = 1;                 // sz 保存子树结点数。
        int key = 0;                // key 是反向串前缀终点，也是后缀长度。
        unsigned long long pri = 0; // pri 保存固定随机优先级。
    };

    static constexpr unsigned long long base = 11995408973635179863ULL; // base 是多项式哈希的基数，哈希在 2^64 下自然溢出。
    string r;                       // r 保存当前字符串的反串。
    vector<unsigned long long> h{0};  // h 保存 r 的 64 位前缀哈希。
    vector<unsigned long long> pw{1}; // pw[i] 保存 base 的 i 次幂。
    vector<Node> t{{}};             // t[0] 是空结点，其余结点编号等于 key。
    vector<int> ways{0};            // ways[n] 保存长度为 n 时的不同子串数。
    int root = 0;                   // root 保存 Treap 根编号。

    static unsigned long long priority(unsigned long long x)
    {
        // x 是后缀长度；返回 SplitMix64 生成的固定优先级。
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }

    int size(int u) const
    {
        // u 是 Treap 结点编号；返回其子树大小，空结点返回 0。
        return u ? t[u].sz : 0;
    }

    void pull(int u)
    {
        // u 是非空结点；由两个儿子重算子树大小。
        t[u].sz = size(t[u].l) + size(t[u].r) + 1;
    }

    unsigned long long hash(int l, int rr) const
    {
        // l、rr 是 r 的半开区间端点；返回 r[l,rr) 的 64 位哈希。
        return h[rr] - h[l] * pw[rr - l];
    }

    int lcpKey(int a, int b) const
    {
        // a、b 是两个反向前缀终点；返回对应原串后缀的 LCP 长度。
        int l = 0, rr = min(a, b) + 1;
        while (l + 1 < rr)
```

```
        {
            int m = (l + rr) / 2;
            if (hash(a - m, a) == hash(b - m, b))
            {
                l = m;
            }
            else
            {
                rr = m;
            }
        }
        return l;
    }

    bool lessKey(int a, int b) const
    {
        // a、b 是后缀长度；返回长度 a 对应后缀是否字典序更小。
        if (a == b)
        {
            return false;
        }
        int p = lcpKey(a, b);
        if (p == min(a, b))
        {
            return a < b;
        }
        return (unsigned char)r[a - p - 1] < (unsigned char)r[b - p - 1];
    }

    void split(int u, int key, int &x, int &y)
    {
        // u 是原树，key 是分界后缀；x、y 分别接收小于 key 和不小于 key 的树。
        if (!u)
        {
            x = y = 0;
        }
        else if (lessKey(t[u].key, key))
        {
            x = u;
            split(t[u].r, key, t[u].r, y);
            pull(u);
        }
        else
        {
            y = u;
            split(t[u].l, key, x, t[u].l);
            pull(u);
        }
    }

    int merge(int x, int y)
    {
```

```
104         // x 的全部键小于 y；返回合并后的 Treap 根。
105         if (!x || !y)
106         {
107             return x | y;
108         }
109         if (t[x].pri > t[y].pri)
110         {
111             t[x].r = merge(t[x].r, y);
112             pull(x);
113             return x;
114         }
115         t[y].l = merge(x, t[y].l);
116         pull(y);
117         return y;
118     }
119 
120     void insert(int u)
121     {
122         // u 是尚未入树的新结点；按其 key 插入 Treap。
123         int x, y;
124         split(root, t[u].key, x, y);
125         root = merge(merge(x, u), y);
126     }
127 
128     int erase(int u, int key)
129     {
130         // u 是当前根，key 是已存在后缀；删除后返回新根。
131         if (t[u].key == key)
132         {
133             return merge(t[u].l, t[u].r);
134         }
135         if (lessKey(key, t[u].key))
136         {
137             t[u].l = erase(t[u].l, key);
138         }
139         else
140         {
141             t[u].r = erase(t[u].r, key);
142         }
143         pull(u);
144         return u;
145     }
146 
147     pair<int, int> neighbors(int key) const
148     {
149         // key 是尚未入树的后缀；返回其前驱、后继的后缀长度，不存在时为 0。
150         int u = root, pre = 0, suc = 0;
151         while (u)
152         {
153             if (lessKey(t[u].key, key))
154             {
155                 pre = t[u].key;
```

```
156                 u = t[u].r;
157             }
158             else
159             {
160                 suc = t[u].key;
161                 u = t[u].l;
162             }
163         }
164         return {pre, suc};
165     }
166 
167     void pushFront(char c)
168     {
169         // c 是新字符；把 c 加到当前串首，并插入由整个新串形成的后缀。
170         r.push_back(c);
171         h.push_back(h.back() * base + (unsigned char)c + 1);
172         pw.push_back(pw.back() * base);
173         int key = r.size();
174         auto [pre, suc] = neighbors(key);
175         int same = 0;
176         if (pre)
177         {
178             same = max(same, lcpKey(key, pre));
179         }
180         if (suc)
181         {
182             same = max(same, lcpKey(key, suc));
183         }
184         ways.push_back(ways.back() + key - same);
185         t.push_back({0, 0, 1, key, priority(key)});
186         insert(key);
187     }
188 
189     void popFront()
190     {
191         // 无参数；删除最近一次加入串首的字符及其完整后缀。
192         assert(!r.empty()); // 调试检查，可删
193         int key = r.size();
194         root = erase(root, key);
195         r.pop_back();
196         h.pop_back();
197         pw.pop_back();
198         ways.pop_back();
199         t.pop_back();
200     }
201 
202     int rank(int i) const
203     {
204         // i 是当前串的后缀起点；返回该后缀从 0 开始的字典序排名。
205         int key = (int)r.size() - i;
206         assert(1 <= key && key <= (int)r.size()); // 调试检查，可删
207         int u = root, ans = 0;
```

```
        while (t[u].key ≠ key)
        {
            if (lessKey(key, t[u].key))
            {
                u = t[u].l;
            }
            else
            {
                ans += size(t[u].l) + 1;
                u = t[u].r;
            }
        }
        return ans + size(t[u].l);
    }

    int kth(int k) const
    {
        // k 是从 0 开始的字典序排名；返回当前串中对应后缀的起点。
        assert(0 <= k && k < size(root)); // 调试检查，可删
        int u = root;
        while (true)
        {
            int z = size(t[u].l);
            if (k < z)
            {
                u = t[u].l;
            }
            else if (k == z)
            {
                return (int)r.size() - t[u].key;
            }
            else
            {
                k -= z + 1;
                u = t[u].r;
            }
        }
    }

    int distinct() const
    {
        // 无参数；返回当前串本质不同子串个数。
        return ways.back();
    }
};
```

`pushFront(c)`、`popFront()`、`rank(i)` 和 `kth(k)` 的期望时间均为 $O(\log^2 n)$，其中一次 Treap 路径含 $O(\log n)$ 次比较，每次比较二分 LCP；`distinct()` 为 $O(1)$，总空间 $O(n)$。只允许撤销最近一次前插，不能删除中间后缀；64 位哈希存在极低碰撞概率，需要确定性结果时改用动态 LCP 结构。静态字符串仍优先使用 4.3.002 后缀数组：SA-IS。

```
signed main()
{
    SuffixBST s;
    for (char c : string("ananab"))
    {
        s.pushFront(c);
    }
    vector<int> sa{5, 3, 1, 0, 4, 2};
    for (int k = 0; k < 6; k++)
    {
        assert(s.kth(k) == sa[k]);
        assert(s.rank(sa[k]) == k);
    }
    assert(s.distinct() == 15);
}
```

4.3.010 区间本质不同子串

限定下标区间 $[l, r)$ 内的本质不同子串数不能直接套整串 SAM 的 $\sum len - len_{link}$。本条给出最直接的做法：把子串 `s[l,r)` 取出，对它单独建后缀数组与 Height，答案就是所有长度之和 $\frac{L(L+1)}{2}$ 减去相邻 Height 之和。它对单次询问简单可靠；大量区间询问才需要持久化或离线的可持久化 SAM / SA 结构。

```
// 依赖 3.3.001 单点修改线段树与摘要二分
// 依赖 3.3.002 懒标记线段树
// 依赖 4.3.003 LCP / Height / RMQ 应用
// 依赖 4.3.004 后缀自动机 SAM
inline long long subDistinct(const string &s, int l, int r)
{
    // s 是原字符串，l、r 是半开端点；返回 s[l,r) 的本质不同子串数量。
    assert(0 <= l && l <= r && r <= (int)s.size()); // 调试检查，可删
    string t = s.substr(l, r - l);
    auto sa = suffixArray(t);
    auto h = suffix_lcp::lcpArray(t, sa);
    long long n = t.size();
    return n * (n + 1) / 2 - accumulate(h.begin(), h.end(), 0LL);
}

struct RangeSubMax
{
    // n、z 是叶子数和补齐后的二次幂；tr 保存区间最大值。
    int n, z = 1;
    vector<int> tr;

    RangeSubMax(int n) : n(n)
    {
        // n 是叶子数；初始各位置均为 0。
        while (z < n)
        {
            z <<= 1;
        }
```

```
        tr.assign(2 * z, 0);
    }

    void setMax(int p, int v)
    {
        // p 是位置，v 是候选值；执行单点取最大。
        for (p += z, tr[p] = max(tr[p], v); p >>= 1;)
        {
            tr[p] = max(tr[2 * p], tr[2 * p + 1]);
        }
    }

    int query(int l, int r) const
    {
        // l、r 是半开端点；返回 [l,r) 的最大值。
        int ans = 0;
        for (l += z, r += z; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1) ans = max(ans, tr[l++]);
            if (r & 1) ans = max(ans, tr[--r]);
        }
        return ans;
    }
};

inline vector<int> rangeSubs(const string &s,
                             const vector<pair<int, int>> &qs)
{
    // s 是小写字母串；qs 是 0 下标半开区间；返回各区间的本质不同子串数。
    int n = s.size(), q = qs.size();
    assert(n > 0); // 调试检查，可删
    vector<vector<pair<int, int>>> at(n + 1);
    for (int i = 0; i < q; i++)
    {
        auto [l, r] = qs[i];
        assert(0 <= l && l < r && r <= n); // 调试检查，可删
        at[r].push_back({l + 1, i});
    }
    SAM<26, 'a'> sam(s);
    int states = sam.t.size();
    vector<vector<int>> son(states);
    for (int i = 1; i < states; i++)
    {
        son[sam.t[i].link].push_back(i);
    }
    vector<int> in(states), out(states), it(states), st{0};
    int tim = 0;
    while (!st.empty())
    {
        int u = st.back();
        if (!in[u]) in[u] = ++tim;
        if (it[u] < (int)son[u].size()) st.push_back(son[u][it[u]++]);
```

```
        else out[u] = tim, st.pop_back();
    }
    LazySeg<Info, Tag> seg(n);
    RangeSubMax mx(states + 1);
    vector<int> top(n + 1), ans(q);
    vector<unsigned char> seen(states);
    int u = 0;
    for (int r = 1; r <= n; r++)
    {
        u = sam.t[u].next[s[r - 1] - 'a'];
        int v = u;
        while (v && !seen[v])
        {
            seen[v] = 1, v = sam.t[v].link;
        }
        while (v)
        {
            int p = mx.query(in[v] - 1, out[v]);
            int old = top[p];
            seg.apply(p - sam.t[v].len, p - sam.t[old].len, Tag{-1});
            swap(top[p], v);
        }
        mx.setMax(in[u] - 1, r);
        top[r] = 0;
        for (auto [l, id] : at[r])
        {
            int len = r - l + 1;
            ans[id] = len * (len + 1) / 2 + seg.query(l - 1, r).sum;
        }
    }
    return ans;
}
```

`subDistinct(s,l,r)` 返回子串 $[l,r)$ 的本质不同子串数，单次 $O(L+1)$（$L=r-l$）时间与空间，依赖 4.3.002 后缀数组：SA-IS 与 4.3.003 LCP / Height / RMQ 应用。批量区间询问可调用 `rangeSubs(s,qs)`：离线扫描右端点，在 SAM 后缀链接树上维护贡献，复杂度 $O(n\log^2 n+q\log n)$，实现见 `include/full/4_3_010.hpp`。

4.3.011 Dictionary of Basic Factors

Dictionary of Basic Factors 把所有长度为 2 的幂的子串（基本因子）分层编号：第 k 层长度 2^k 的子串由第 $k-1$ 层的前、后两个 2^{k-1} 块的编号对决定，逐层排序压缩得离散排名 `rk[k][i]`。有了它，任意等长子串比较可拆成常数个基本因子——`compare(l,r,len)` 取覆盖 `len` 的两个 2^k 块（$k=\lfloor\log_2 len\rfloor$，右块从 `len` 减 2^k 起）比较排名对。

```
struct BasicFactors
{
    // s 是原字符串，rk[k][i] 是长 2^k 子串 s[i..i+2^k) 的排名。
    string s;
```

```
    vector<vector<int>> rk;

    BasicFactors(string s) : s(move(s))
    {
        // s 按无符号字节字典序比较；建立所有二次幂长度子串的分层排名。
        int n = this->s.size();
        vector<int> first;
        for (unsigned char c : this->s)
        {
            first.push_back(c);
        }
        rk.push_back(move(first));
        for (int k = 1; (1LL << k) <= n; k++)
        {
            int len = 1LL << (k - 1);
            vector<int> id(n - 2 * len + 1), next(id.size());
            iota(id.begin(), id.end(), 0);
            sort(id.begin(), id.end(), [&](int i, int j)
            {
                return pair{rk[k - 1][i], rk[k - 1][i + len]} <
                       pair{rk[k - 1][j], rk[k - 1][j + len]};
            });
            int cls = 0;
            for (int i = 0; i < (int)id.size(); i++)
            {
                if (i && pair{rk[k - 1][id[i - 1]], rk[k - 1][id[i - 1] + len]} !=
                         pair{rk[k - 1][id[i]], rk[k - 1][id[i] + len]}) cls++;
                next[id[i]] = cls;
            }
            rk.push_back(move(next));
        }
    }

    int compare(int l, int r, int len) const
    {
        // l、r 是两个子串起点，len 是共同长度；按字典序返回 -1、0 或 1。
        assert(0 <= l && 0 <= r && 0 <= len && l + len <= (int)s.size() && r + len <=
        ↪  (int)s.size()); // 调试检查，可删
        if (!len) return 0;
        int k = bit_width((unsigned long long)len) - 1, d = len - (1LL << k);
        pair<int, int> a{rk[k][l], rk[k][l + d]}, b{rk[k][r], rk[k][r + d]};
        return (a > b) - (a < b);
    }
};

struct BorderInfo
{
    // mn、mx 是最短和最长真 border 长度，cnt 是真 border 数量。
    int mn = 0, mx = 0, cnt = 0;
};

struct BasicDictQuery
```

```cpp
{
    int l, r; // l、r 是询问子串的 1 起闭区间端点。
};

inline vector<BorderInfo> basicDict(const string &s,
                                    const vector<pair<int, int>> &qs)
{
    // s 是字符串；qs 是 0 下标闭区间；返回每个区间的真 border 统计。
    int n = s.size(), q = qs.size();
    vector<BorderInfo> ans(q);
    if (!n || !q) return ans;
    vector<BasicDictQuery> que(q);
    vector<vector<int>> atL(n + 2), atR(n + 2);
    for (int i = 0; i < q; i++)
    {
        int l = qs[i].first + 1, r = qs[i].second + 1;
        assert(1 <= l && l <= r && r <= n);
        que[i] = {l, r};
        atL[l].push_back(i);
        atR[r].push_back(i);
    }
    vector<int> sa(n + 1), rk(n + 1), id(2 * n + 3), cnt(256);
    for (int i = 1; i <= n; i++)
    {
        cnt[(unsigned char)s[i - 1]]++;
    }
    for (int i = 1; i < 256; i++)
    {
        cnt[i] += cnt[i - 1];
    }
    for (int i = n; i; i--)
    {
        sa[cnt[(unsigned char)s[i - 1]]--] = i;
    }
    int cls = 0;
    for (int i = 1; i <= n; i++)
    {
        if (i == 1 || s[sa[i] - 1] != s[sa[i - 1] - 1]) cls++;
        rk[sa[i]] = cls;
    }
    vector<int> nxt(n + 2), leftMost(q, n + 1), rightMost(q),
        leftRev(q, n + 1), rightRev(q), low(q, n + 1), high(q), number(q),
        buf(n + 3);
    auto addOne = [&](int idq, int x, int l, int r, int step)
    {
        if (l <= x && x <= r && (x - l) % step == 0)
        {
            low[idq] = min(low[idq], x);
            high[idq] = max(high[idq], x);
            number[idq]++;
        }
    };
```

```
    auto addTwo = [&](int idq, int l1, int r1, int d1,
                      int l2, int r2, int d2)
    {
        if (r1 - l1 <= d1)
        {
            for (int x = l1; x <= r1; x += d1)
            {
                addOne(idq, x, l2, r2, d2);
            }
            return;
        }
        if (r2 - l2 <= d2)
        {
            for (int x = l2; x <= r2; x += d2)
            {
                addOne(idq, x, l1, r1, d1);
            }
            return;
        }
        if ((l1 - l2) % d1) return;
        l1 = max(l1, l2), r1 = min(r1, r2);
        if (l1 <= r1)
        {
            low[idq] = min(low[idq], l1);
            high[idq] = max(high[idq], r1);
            number[idq] += (r1 - l1) / d1 + 1;
        }
    };
    auto work = [&](int len)
    {
        fill(buf.begin(), buf.begin() + cls + 1, n + 1);
        for (int i = n - len + 1; i >= 1; i--)
        {
            nxt[i] = buf[rk[i]], buf[rk[i]] = i;
        }
        fill(buf.begin(), buf.begin() + cls + 1, -1);
        for (int i = 2 - len; i <= n; i++)
        {
            if (1 <= i && i <= n) buf[rk[i]] = i;
            int j = i + len - 1;
            if (1 <= j && j <= n)
            {
                for (int idq : atR[j])
                {
                    if (que[idq].r - que[idq].l + 1 > len)
                    {
                        rightMost[idq] = buf[rk[que[idq].l]];
                    }
                }
            }
            j = i - len + 1;
            if (1 <= j && j <= n)
```

```
            {
                for (int idq : atL[j])
                {
                    if (que[idq].r - que[idq].l + 1 > len)
                    {
                      rightRev[idq] = min(buf[rk[que[idq].r - len + 1]], que[idq].r - len + 1);
                    }
                }
            }
        }
        fill(buf.begin(), buf.begin() + cls + 1, n + 1);
        for (int i = n; i + len - 1; i--)
        {
            if (1 <= i && i <= n) buf[rk[i]] = i;
            int j = i + 2 * len - 2;
            if (1 <= j && j <= n)
            {
                for (int idq : atR[j])
                {
                    if (que[idq].r - que[idq].l + 1 > len)
                    {
                        leftMost[idq] = max(buf[rk[que[idq].l]], nxt[que[idq].l]);
                    }
                }
            }
            if (1 <= i && i <= n)
            {
                for (int idq : atL[i])
                {
                    if (que[idq].r - que[idq].l + 1 > len)
                    {
                        leftRev[idq] = buf[rk[que[idq].r - len + 1]];
                    }
                }
            }
        }
        for (int i = 0; i < q; i++)
        {
            if (que[i].r - que[i].l + 1 <= len || leftMost[i] > rightMost[i] || leftRev[i] >
            ↪  rightRev[i]) continue;
            int d1 = nxt[leftMost[i]] - leftMost[i], d2 = nxt[leftRev[i]] - leftRev[i];
            swap(leftMost[i], rightMost[i]);
            leftRev[i] += len - que[i].l;
            rightRev[i] += len - que[i].l;
            leftMost[i] = que[i].r - leftMost[i] + 1;
            rightMost[i] = que[i].r - rightMost[i] + 1;
            addTwo(i, leftMost[i], rightMost[i], d1, leftRev[i], rightRev[i], d2);
        }
        if (cls == n) return;
        fill(buf.begin(), buf.begin() + cls + 1, 0);
        int z = 0;
        for (int i = n; i > n - len; i--)
```

```
        {
            id[++z] = i;
        }
        for (int i = 1; i <= n; i++)
        {
            if (sa[i] > len) id[++z] = sa[i] - len;
        }
        for (int i = 1; i <= n; i++)
        {
            buf[rk[i]]++;
        }
        for (int i = 2; i <= cls; i++)
        {
            buf[i] += buf[i - 1];
        }
        for (int i = n; i; i--)
        {
            sa[buf[rk[id[i]]]--] = id[i], id[i] = rk[i];
        }
        cls = 0;
        for (int i = 1; i <= n; i++)
        {
            int x = sa[i], y = sa[i - 1];
            if (i == 1 || id[x] != id[y] || id[x + len] != id[y + len]) cls++;
            rk[x] = cls;
        }
    };
    for (int len = 1; len <= n; len <<= 1)
    {
        work(len);
    }
    for (int i = 0; i < q; i++)
    {
        if (number[i]) ans[i] = {low[i], high[i], number[i]};
    }
    return ans;
}
```

构造 `BasicFactors bf(s)` 后 `bf.compare(l,r,len)` 按字典序比较两个等长子串返回 $-1/0/1$；非空串构造 $O(n\log^2 n)$ 时间、$O(n\log n)$ 空间，空串构造 $O(1)$，定长比较 $O(1)$。批量 border 统计调用 `basicDict(s,qs)`，其中 `qs` 为 0 下标闭区间，返回真 border 的最短长度、最长长度和数量，复杂度 $O((n+q)\log n)$，实现见 `include/full/4_3_011.hpp`。

4.3.012 BWT / FM-index

在文本末尾加入比所有字符都小的终止符 \$，把它的全部循环移位排序，排序结果的最后一列就是 BWT：第 i 行对应后缀数组中第 i 小的后缀，$L_i = s_{sa_i-1}$（$sa_i = 0$ 时为 \$）。记 $C[x]$ 为 L 中小于 x 的符号个数，$\mathrm{rank}(x, r)$ 为 $L_0 \dots L_{r-1}$ 中 x 的个数，则：

- LF 映射：第 i 行循环右移一位后位于第 $\mathrm{LF}(i) = C[L_i] + \mathrm{rank}(L_i, i)$ 行，因为同一字符在首列和

末列的相对顺序相同。

- 反向搜索：以模式 p 为前缀的行构成一段区间 $[l, r)$。从 p 的最后一个字符向前，每加一个字符 x 更新为 $l \leftarrow C[x] + \mathrm{rank}(x, l)$、$r \leftarrow C[x] + \mathrm{rank}(x, r)$。
- 还原：从 $L_i = \$$ 的那一行（即整个原文所在行）出发反复做 LF，依次得到原文从后往前的字符。

实现用 4.3.002 后缀数组：SA-IS 建后缀数组；字节 x 编码为 $x + 1$，终止符编码为 0；`rank` 用每个符号的出现位置表二分，不做压缩。

```cpp
// 依赖 4.3.002 后缀数组：SA-IS
struct FMIndex
{
    // sa 是带终止符文本的后缀数组；bwt[i] 是第 i 小后缀前面的那个符号。
    // 字节 x 编码为 x+1，唯一的终止符编码为 0。
    vector<int> sa, bwt;
    // c[x] 是 bwt 中小于 x 的符号个数；at[x] 是符号 x 在 bwt 中出现的行号，升序。
    array<int, 258> c{};
    array<vector<int>, 257> at;

    FMIndex(const string &text)
    {
        // text 是任意字节串；建立后缀数组、BWT 与按符号分组的出现位置。
        int n = text.size() + 1;
        vector<int> s(n);
        for (int i = 0; i + 1 < n; i++)
        {
            s[i] = (unsigned char)text[i] + 1;
        }
        sa = suffixArray(s);
        bwt.resize(n);
        for (int i = 0; i < n; i++)
        {
            bwt[i] = s[(sa[i] + n - 1) % n];
            at[bwt[i]].push_back(i);
            c[bwt[i] + 1]++;
        }
        for (int x = 1; x < 258; x++)
        {
            c[x] += c[x - 1];
        }
    }

    int rank(int x, int r) const
    {
        // x 是符号编码，r 是行号上界；返回 bwt[0..r) 中 x 的出现次数。
        return lower_bound(at[x].begin(), at[x].end(), r) - at[x].begin();
    }

    int lf(int i) const
    {
        // i 是行号；返回把该行循环右移一位后所在的行（LF 映射）。
        return c[bwt[i]] + rank(bwt[i], i);
```

```
    }

    pair<int, int> search(const string &p) const
    {
        // p 是模式串；返回后缀数组中以 p 为前缀的行区间 [l,r)，不出现时 l==r。
        int l = 0, r = sa.size();
        for (int i = (int)p.size() - 1; i >= 0 && l < r; i--)
        {
            int x = (unsigned char)p[i] + 1;
            l = c[x] + rank(x, l);
            r = c[x] + rank(x, r);
        }
        return {l, r};
    }

    vector<int> locate(const string &p) const
    {
        // p 是模式串；返回 p 在原文中全部出现的起点，升序；空模式返回 0..|text|。
        auto [l, r] = search(p);
        vector<int> ans(sa.begin() + l, sa.begin() + r);
        sort(ans.begin(), ans.end());
        return ans;
    }

    string restore() const
    {
        // 只用 bwt 沿 LF 映射倒推，返回原文。
        int n = bwt.size(), i = at[0][0];
        string s(n - 1, '\0');
        for (int k = n - 2; k >= 0; k--)
        {
            i = lf(i);
            s[k] = (char)(bwt[i] - 1);
        }
        return s;
    }
};
```

`FMIndex fm(text)` 接受任意字节串。`search(p)` 返回以 p 为前缀的后缀数组行区间，$r-l$ 是出现次数；`locate(p)` 返回全部出现起点并升序排序，空模式返回 $0 \dots |text|$ 全部位置；`restore()` 只用 BWT 还原原文。构造 $O(n)$，`search` 为 $O(|p| \log n)$，`locate` 另加 $O(occ \log occ)$ 排序；保存完整的 `sa`、`bwt` 与出现位置表，空间 $O(n)$ 个整数，不是压缩索引。

只需要子串查找时，直接在后缀数组上二分（4.3.002 后缀数组：SA-IS）或用 4.3.004 后缀自动机 SAM 更直接；FM-index 的价值在于逐字符反向扩展区间以及由 BWT 还原原串。下面核对 `abracadabra`：

```
signed main()
{
    FMIndex fm("abracadabra");
    auto [l, r] = fm.search("abra");
    assert(r - l == 2);
```

```
    assert(fm.locate("a") == vector<int>({0, 3, 5, 7, 10}));
    assert(fm.locate("cad") == vector<int>({4}) && fm.locate("zzz").empty());
    assert(fm.restore() == "abracadabra");
}
```

4.4 回文结构

4.4.001 回文自动机 PAM / 回文树

回文自动机（回文树）每个节点恰对应一种本质不同回文子串，`link` 指向该回文的最长真回文后缀。有两个根：长度 -1 的奇根与长度 0 的偶根。在线 `add(c)` 时先沿 `getLink` 从 `last` 找到能左右扩展当前字符的回文后缀 `p`（即 $s[n-len_p-1]=s[n]$），若 `p` 没有 `x` 边就新建长度 len_p+2 的回文节点，其 `link` 在长度为 1 时接偶根、否则接 `getLink(link_p)` 的 `x` 边；至多新建一个节点。节点数减 2 即不同回文串数，`count` 沿 `link` 逆序上推 `cnt` 得每种回文的出现次数。

```
template <int A = 26, char S = 'a'> struct PAM
{
    struct Node
    {
        array<int, A> ch{}; // ch[c] 是两侧各加字符 c 得到的回文节点，0 表示不存在。
        // len 是回文长度，link 是回文后缀链接，cnt 是出现次数。
        int len = 0, link = 0, cnt = 0;
    };

    vector<Node> t{{{}, -1, 0, 0}, {{}, 0, 0, 0}};
    vector<int> s{-1}; // s 是已加入的字符，s[0]=-1 作为哨兵。
    // last 是当前最长回文后缀节点，counted 记录出现次数是否已经上传。
    int last = 1;
    bool counted = false;

    int getLink(int p) const
    {
        // p 是回文节点；返回可被当前末字符继续扩展的后缀回文节点。
        int n = s.size() - 1;
        while (n - t[p].len - 1 < 0 || s[n - t[p].len - 1] != s[n])
        {
            p = t[p].link;
        }
        return p;
    }

    int add(char c)
    {
        // c 是追加到原串末尾的字符；返回新的最长回文后缀节点。
        assert(!counted); // 调试检查，可删
        int x = c - S;
        assert(0 <= x && x < A); // 调试检查，可删
        s.push_back(x);
        int p = getLink(last);
```

```
        if (!t[p].ch[x])
        {
            int q = t.size();
            t.push_back({{}, t[p].len + 2, 0, 0});
            if (t[q].len == 1)
            {
                t[q].link = 1;
            }
            else
            {
                t[q].link = t[getLink(t[p].link)].ch[x];
            }
            t[p].ch[x] = q;
        }
        last = t[p].ch[x];
        t[last].cnt++;
        return last;
    }

    void build(const string &x)
    {
        // x 是要继续追加的字符串。
        for (char c : x)
        {
            add(c);
        }
    }

    void count()
    {
        // 沿后缀链接上传出现次数；重复调用不再二次累计。
        if (counted)
        {
            return;
        }
        for (int p = (int)t.size() - 1; p >= 2; p--)
        {
            t[t[p].link].cnt += t[p].cnt;
        }
        counted = true;
    }
};
```

`build(s)` 逐字符加入，`add(c)` 返回当前最长回文后缀节点，`count()` 幂等地上传一次出现次数；上传后不能再追加字符。节点数减二是本质不同回文数。构造 $O(n+1)$、计数 $O(p+1)$、空间 $O((p+2)A+n+1)$，其中 $p \le n$ 是本质不同回文数。需要在两端插入和删除时改用 4.4.002 双端可撤销回文树，回文划分 DP 的等差链优化见 4.4.003 Palindrome Series。下面加入 `ababa`，记录每次新增长后缀对应的节点，再逐节点核对五种回文的出现次数：

```
signed main()
{
```

```
    PAM<3, 'a'> pam;
    vector<int> id;
    for (char c : string("ababa"))
    {
        id.push_back(pam.add(c));
    }
    pam.count();
    pam.count();
    assert((int)pam.t.size() - 2 == 5);
    assert(pam.t[id[0]].cnt == 3);
    assert(pam.t[id[1]].cnt == 2);
    assert(pam.t[id[2]].cnt == 2);
    assert(pam.t[id[3]].cnt == 1);
    assert(pam.t[id[4]].cnt == 1);
    cout << "5 3 2 2 1 1\n";
    return 0;
}
```

真实输出为 `5 3 2 2 1 1`。

4.4.002 双端可撤销回文树

普通 PAM 只支持在右端追加；`DequePAM` 额外维护各位置对应的最长回文前缀 `pre` 和最长回文后缀 `suf`，从而能在 deque 两端插入和撤销。节点的 `cnt` 是当前 `pre`、`suf` 记录对它的引用数，`sons` 是后缀链接孩子数；二者都归零时，该回文已不在当前串中，可删除父转移并回收编号。`quick` 跳过一批不可能被新字符扩展的后缀回文，前后端使用对称的 `frontCan`、`backCan` 查找可扩展节点。字符必须落在 $[S, S+A)$，删除只允许作用于非空串。

```
template <int A = 26, char S = 'a'>
struct DequePAM
{
  private:
    struct Node
    {
        // ch 是回文树转移；par、link、quick 分别是转移父亲、后缀链接和快速跳转。
        array<int, A> ch{};
        int par = 0;
        int link = 0;
        int quick = 0;
        // len 是回文长度，cnt 是边界记录引用数，sons 是后缀链接孩子数。
        int len = 0;
        int cnt = 0;
        int sons = 0;
    };

    struct Pos
    {
        // c 是当前位置字符编号；pre、suf 记录当前最长回文前缀、后缀节点。
        int c = 0;
        int pre = 1;
```

```
        int suf = 1;
    };

    // t 是节点池，spare 是可复用编号，d 是当前字符串，kinds 是本质不同回文数。
    vector<Node> t;
    vector<int> spare;
    deque<Pos> d;
    int kinds = 0;

    int newNode()
    {
        // 返回一个已经清空的节点编号，优先复用回收节点。
        if (spare.empty())
        {
            t.push_back({});
            return (int)t.size() - 1;
        }
        int p = spare.back();
        spare.pop_back();
        t[p] = Node{};
        return p;
    }

    void drop(int p)
    {
        // p 是已经脱离所有转移和边界记录的节点；清空后回收其编号。
        t[p] = Node{};
        spare.push_back(p);
    }

    int backCan(int c, int p) const
    {
        // c 是待加字符，p 是当前最长回文后缀；返回最先能从右端扩展的后缀回文。
        int n = d.size();
        while (true)
        {
            if (t[p].len == -1 || (t[p].len < n && d[n - t[p].len - 1].c == c))
            {
                return p;
            }
            int q = t[p].link;
            if (t[q].len == -1 || (t[q].len < n && d[n - t[q].len - 1].c == c))
            {
                return q;
            }
            p = t[p].quick;
        }
    }

    int frontCan(int c, int p) const
    {
        // c 是待加字符，p 是当前最长回文前缀；返回最先能从左端扩展的前缀回文。
```

```
        int n = d.size();
        while (true)
        {
            if (t[p].len == -1 || (t[p].len < n && d[t[p].len].c == c))
            {
                return p;
            }
            int q = t[p].link;
            if (t[q].len == -1 || (t[q].len < n && d[t[q].len].c == c))
            {
                return q;
            }
            p = t[p].quick;
        }
    }

public:
    DequePAM() : t(2)
    {
        // 建立长度 -1 的奇根 0 和长度 0 的偶根 1。
        t[0].len = -1;
        t[0].par = 0;
        t[0].link = 0;
        t[0].quick = 0;
        t[1].len = 0;
        t[1].par = 0;
        t[1].link = 0;
        t[1].quick = 0;
    }

    void pushBack(char c)
    {
        // c 是新字符；加入右端并维护最长回文前后缀。
        int x = (unsigned char)c - (unsigned char)S;
        assert(0 <= x && x < A); // 调试检查，可删
        int par = d.empty() ? 0 : backCan(x, d.back().suf);
        int n = d.size();
        int v = t[par].ch[x];
        int w = 1;
        if (!v)
        {
            v = newNode();
            t[v].par = par;
            t[v].len = t[par].len + 2;
            if (par != 0)
            {
                w = t[backCan(x, t[par].link)].ch[x];
                assert(w); // 调试检查，可删
            }
            t[v].link = w;
            t[w].sons++;
            d.push_back({x, 1, 1});
```

```
            n++;
            if (t[w].link != 0 && d[n - t[w].len - 1].c == d[n - t[t[w].link].len - 1].c)
            {
                t[v].quick = t[w].quick;
            }
            else
            {
                t[v].quick = t[w].link;
            }
            t[par].ch[x] = v;
            kinds++;
        }
        else
        {
            d.push_back({x, 1, 1});
            n++;
            w = t[v].link;
        }
        d[n - 1].suf = v;
        d[n - t[v].len].pre = v;
        if (t[w].len >= 1 && d[n - t[v].len + t[w].len - 1].suf == w)
        {
            d[n - t[v].len + t[w].len - 1].suf = 1;
        }
        t[v].cnt++;
    }

    void pushFront(char c)
    {
        // c 是新字符；加入左端并维护最长回文前后缀。
        int x = (unsigned char)c - (unsigned char)S;
        assert(0 <= x && x < A); // 调试检查，可删
        int par = d.empty() ? 0 : frontCan(x, d.front().pre);
        int v = t[par].ch[x];
        int w = 1;
        if (!v)
        {
            v = newNode();
            t[v].par = par;
            t[v].len = t[par].len + 2;
            if (par != 0)
            {
                w = t[frontCan(x, t[par].link)].ch[x];
                assert(w); // 调试检查，可删
            }
            t[v].link = w;
            t[w].sons++;
            d.push_front({x, 1, 1});
            if (t[w].link != 0 && d[t[w].len].c == d[t[t[w].link].len].c)
            {
                t[v].quick = t[w].quick;
            }
```

```
            else
            {
                t[v].quick = t[w].link;
            }
            t[par].ch[x] = v;
            kinds++;
        }
        else
        {
            d.push_front({x, 1, 1});
            w = t[v].link;
        }
        d[0].pre = v;
        d[t[v].len - 1].suf = v;
        if (t[w].len >= 1 && d[t[v].len - t[w].len].pre == w)
        {
            d[t[v].len - t[w].len].pre = 1;
        }
        t[v].cnt++;
    }

    void popBack()
    {
        // 删除右端字符；当前串必须非空。
        assert(!d.empty()); // 调试检查，可删
        int n = d.size();
        int v = d.back().suf;
        int x = d.back().c;
        int w = t[v].link;
        if (t[v].len >= 2 && t[d[n - t[v].len + t[w].len - 1].suf].len < t[w].len)
        {
            d[n - t[v].len + t[w].len - 1].suf = w;
            d[n - t[v].len].pre = w;
        }
        else
        {
            d[n - t[v].len].pre = 1;
        }
        t[v].cnt--;
        if (!t[v].sons && !t[v].cnt)
        {
            t[t[v].par].ch[x] = 0;
            t[w].sons--;
            drop(v);
            kinds--;
        }
        d.pop_back();
    }

    void popFront()
    {
        // 删除左端字符；当前串必须非空。
```

```
        assert(!d.empty()); // 调试检查，可删
        int v = d.front().pre;
        int x = d.front().c;
        int w = t[v].link;
        if (t[v].len >= 2 && t[d[t[v].len - t[w].len].pre].len < t[w].len)
        {
            d[t[v].len - t[w].len].pre = w;
            d[t[v].len - 1].suf = w;
        }
        else
        {
            d[t[v].len - 1].suf = 1;
        }
        t[v].cnt--;
        if (!t[v].sons && !t[v].cnt)
        {
            t[t[v].par].ch[x] = 0;
            t[w].sons--;
            drop(v);
            kinds--;
        }
        d.pop_front();
    }

    int size() const
    {
        // 返回当前字符串长度。
        return d.size();
    }

    int distinct() const
    {
        // 返回当前字符串的本质不同非空回文子串数。
        return kinds;
    }

    int longPrefix() const
    {
        // 返回当前最长回文前缀长度，空串返回 0。
        return d.empty() ? 0 : t[d.front().pre].len;
    }

    int longSuffix() const
    {
        // 返回当前最长回文后缀长度，空串返回 0。
        return d.empty() ? 0 : t[d.back().suf].len;
    }
};
```

`pushFront`、`pushBack`、`popFront`、`popBack` 分别修改两端；`distinct()`、`longPrefix()`、`longSuffix()` 均为 $O(1)$ 查询。数组转移为 $O(1)$，一次修改沿 `quick` 链查找可扩展回文，保守最坏为相关回文链接链长。设操作过程中最大串长为 N、最多同时存在 P 种非空回文，节点编号由 `spare` 复用，峰值空间为

$O((P+2)A+N)$。下面从 abac 出发依次在两端增删，检查四种修改和节点回收：

```
signed main()
{
    DequePAM<3, 'a'> pam;
    for (char c : string("abac"))
    {
        pam.pushBack(c);
    }
    assert((array{pam.distinct(), pam.longPrefix(), pam.longSuffix()} == array{4LL, 3LL,
    ↪ 1LL}));
    pam.pushFront('c');
    assert((array{pam.distinct(), pam.longPrefix(), pam.longSuffix()} == array{5LL, 5LL,
    ↪ 5LL}));
    pam.pushBack('a');
    assert((array{pam.distinct(), pam.longPrefix(), pam.longSuffix()} == array{6LL, 5LL,
    ↪ 3LL}));
    pam.popFront();
    assert((array{pam.distinct(), pam.longPrefix(), pam.longSuffix()} == array{5LL, 3LL,
    ↪ 3LL}));
    pam.popBack();
    assert((array{pam.distinct(), pam.longPrefix(), pam.longSuffix()} == array{4LL, 3LL,
    ↪ 1LL}));
}
```

4.4.003 Palindrome Series

把回文树的后缀链接链按 diff=len[u]-len[link[u]] 分成等差系列，并维护 series 指针跳过同 diff 段。这样每个前缀的全部回文后缀被压成 $O(\log n)$ 个等差系列，回文划分类 DP（如把串划分成最少 / 方案数个回文）可按系列整体转移而非逐个回文，均摊 $O(n \log n)$。add 在建回文节点时顺带算出 diff 与 series，eachSeries 沿 series 链枚举关键状态。

```
template <int A = 26, char S = 'a'> struct PamSeries
{
    struct Node
    {
        array<int, A> ch{}; // ch[c] 是两侧各加字符 c 得到的回文节点，0 表示不存在。
        // diff=len-len[link]，series 指向上一条不同公差的系列。
        int len = 0, link = 0, diff = 0, series = 0;
    };

    vector<Node> t{{{}, -1, 0, 0, 0}, {{}, 0, 0, 0, 0}};
    vector<int> s{-1}; // s 是已加入的字符，s[0]=-1 作为哨兵。
    // last 是当前最长回文后缀节点。
    int last = 1;

    int get(int p) const
    {
        // p 是一个回文节点；返回能被当前末字符继续扩展的后缀回文节点。
        int n = s.size() - 1;
```

```
        while (s[n - t[p].len - 1] != s[n])
        {
            p = t[p].link;
        }
        return p;
    }

    int add(char c)
    {
        // c 是追加到原串末尾的字符；返回新的最长回文后缀节点。
        int x = c - S;
        assert(0 <= x && x < A); // 调试检查，可删
        s.push_back(x);
        int p = get(last);
        if (!t[p].ch[x])
        {
            int q = t.size();
            t.push_back({{}, t[p].len + 2});
            t[q].link = t[q].len == 1 ? 1 : t[get(t[p].link)].ch[x];
            t[q].diff = t[q].len - t[t[q].link].len;
            t[q].series = t[q].diff == t[t[q].link].diff ? t[t[q].link].series : t[q].link;
            t[p].ch[x] = q;
        }
        return last = t[p].ch[x];
    }

    template <class F> void eachSeries(int p, F f) const
    {
        // p 是当前最长回文后缀，f 接收每条等差系列的首节点。
        for (; t[p].len > 0; p = t[p].series)
        {
            f(p);
        }
    }
};
```

`add(c)` 在线扩展并维护 `diff`、`series`；`eachSeries(p,f)` 沿等差链对每个关键状态调用回调 `f`。每次追加摊还 $O(1)$，状态数 $\le n+2$、空间 $O((n+2)A+n+1)$；一个前缀的系列链长为 $O(\log(n+1))$，故配合系列 DP 时每次枚举 $O(\log(n+1))$。只统计本质不同回文或其出现次数用 4.4.001 回文自动机 PAM / 回文树；本条专为回文划分 DP 的等差优化。

例如把当前前缀的每条系列首节点收集起来：

```
signed main()
{
    PamSeries<> pam;
    int p = 0;
    for (char c : string("abacaba"))
    {
        p = pam.add(c);
    }
```

```
    vector<int> head;
    pam.eachSeries(p, [&](int u)
    {
        head.push_back(u);
    });
    assert(!head.empty());
}
```

4.4.004 SqAM

SqAM 是在线子序列自动机：每加入一个文本字符建一个状态，`go[u][c]` 指向位置 `u` 之后字符 `c` 首次出现的状态。插入新字符 `x` 时沿每种字符 `c` 的「上一同字符位置」链 `pre` 补齐尚不存在的 `go[..][x]` 转移，每个状态与字符对最多被写一次，故总代价 $O(nA)$。沿 `go` 读完模式当且仅当它是文本子序列。

```
template <int A = 26> class SqAM
{
    // go[u][c] 是状态 u 读字符 c 后到达的最早后继，-1 表示不存在。
    vector<array<int, A>> go;
    // pre[u] 是与状态 u 字符相同的上一次出现位置，-1 表示此前未出现。
    vector<int> pre;
    // last[c] 是字符 c 最近一次出现的状态，-1 表示尚未出现。
    array<int, A> last;

  public:
    SqAM()
    {
        // 建立只含空文本状态的子序列自动机。
        last.fill(-1);
        go.push_back({});
        go[0].fill(-1);
        pre.push_back(-1);
    }

    void add(int x)
    {
        // x 是 [0,A) 内的新字符编号；把它追加到文本末尾。
        assert(0 <= x && x < A); // 调试检查，可删
        int v = go.size();
        go.push_back({});
        go.back().fill(-1);
        pre.push_back(last[x]);
        for (int c = 0; c < A; c++)
        {
            for (int u = last[c]; u != -1 && go[u][x] == -1; u = pre[u])
            {
                go[u][x] = v;
            }
        }
        if (last[x] == -1)
        {
```

```
            go[0][x] = v;
        }
        last[x] = v;
    }

    bool contains(const vector<int> &s) const
    {
        // s 是 [0,A) 内的模式序列；返回 s 是否为当前文本的子序列。
        int u = 0;
        for (int x : s)
        {
            if (x < 0 || x >= A || go[u][x] == -1)
            {
                return false;
            }
            u = go[u][x];
        }
        return true;
    }

    int size() const
    {
        // 返回当前文本长度。
        return go.size() - 1;
    }
};
```

构造 `SqAM<A> q`，`add(x)` 追加 $[0,A)$ 内字符编号，`contains(s)` 判断编号序列是否为当前文本子序列，`size()` 返回文本长度；只支持末尾追加、不支持删除，也不判连续子串。加入总长 n 的时间、空间均为 $O((n+1)A)$，长度 m 查询 $O(m)$。依次加入 `{0,1,0,2}` 后 `{0,0,2}` 与空序列是子序列，`{1,1}`、`{2,0}` 不是。离线一次建表的等价结构见 4.2.004 子序列自动机。

4.5 子序列与高级问题

4.5.001 最长公共子序列 LCS

最长公共子序列求两序列中不要求连续、但保持相对顺序的最长公共子列（勿与最长公共子串混淆）。令 $f_{i,j}$ 为 $a[0,i)$、$b[0,j)$ 的 LCS 长度：末字符相等时 $f_{i,j}=f_{i-1,j-1}+1$，否则 $f_{i,j}=\max(f_{i-1,j},f_{i,j-1})$。本实现存整张 `f` 表以便回溯：从 (n,m) 出发，相等就取该字符并双向回退，否则往 `f` 更大的一侧走，最后反转得到一条 LCS。

```
template <class T> vector<T> lcs(const vector<T> &a, const vector<T> &b)
{
    // a、b 是两个序列；返回二者的一条最长公共子序列。
    int n = a.size(), m = b.size();
    vector<vector<int>> f(n + 1, vector<int>(m + 1));
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
```

```
        {
            if (a[i] == b[j])
            {
                f[i + 1][j + 1] = f[i][j] + 1;
            }
            else
            {
                f[i + 1][j + 1] = max(f[i][j + 1], f[i + 1][j]);
            }
        }
    }
    vector<T> c;
    for (int i = n, j = m; i && j;)
    {
        if (a[i - 1] == b[j - 1])
        {
            i--;
            j--;
            c.push_back(a[i]);
        }
        else if (f[i - 1][j] >= f[i][j - 1])
        {
            i--;
        }
        else
        {
            j--;
        }
    }
    reverse(c.begin(), c.end());
    return c;
}
```

`lcs(a,b)` 返回一条具体的最长公共子序列（元素只需支持判等）；构造与回溯的时间、空间均为 $O((n+1)(m+1))$。只要长度且串长时用位并行的 4.5.002 Bitset 优化 LCS；连续公共片段是另一问题，见 4.5.004 多串最长公共子串/子序列。最短公共超序列 4.5.005 最短公共超序列 直接建在这里恢复出的 LCS 上。

4.5.002 Bitset 优化 LCS

只要 LCS 长度、且串较长时可用位并行。把第二串每个字符的出现位置编码成位集 `mask[c]`，用经典公式 `x=mask[c]|s; s=x&~(x-((s<<1)|1))` 逐个 `a` 中字符批量推进整行（跨多机器字用手写借位减法处理进位），最后各字长的 `popcount` 之和即 LCS 长度。它把一维转移加速约一个字长倍。

```
int lcsBitset(const string &a, const string &b)
{
    // a、b 是两个字节串；返回二者的最长公共子序列长度。
    int w = (b.size() + 63) / 64;
    vector<vector<unsigned long long>> mask(256, vector<unsigned long long>(w));
    vector<unsigned long long> s(w), x(w), z(w);
    for (int i = 0; i < (int)b.size(); i++)
```

```
    {
        mask[(unsigned char)b[i]][i / 64] |= 1ULL << (i % 64);
    }
    for (unsigned char c : a)
    {
        unsigned long long shift = 1, borrow = 0;
        for (int i = 0; i < w; i++)
        {
            x[i] = mask[c][i] | s[i];
            unsigned long long y = s[i] << 1 | shift;
            shift = s[i] >> 63;
            unsigned long long q = y + borrow;
            unsigned long long borrow = q < y || x[i] < q;
            z[i] = x[i] - q;
            borrow = borrow;
        }
        for (int i = 0; i < w; i++)
        {
            s[i] = x[i] & ~z[i];
        }
    }
    int ans = 0;
    for (unsigned long long x : s)
    {
        ans += popcount(x);
    }
    return ans;
}
```

`lcsBitset(a,b)` 只返回 LCS 长度，字符按 `unsigned char` 映射到 256 个掩码。答案不超过 $\min(n,m)$；两个串长和 $W=\lceil m/64\rceil$ 都须能由模板中的 64 位 `int` 表示。时间为 $O(nW+m+256W)$，空间为 $O(256(W+1))$；需要恢复具体序列时改用 4.5.001 最长公共子序列 LCS 的普通 DP。

4.5.003 区间 LCS / 区间 LIS

两个不同问题放在一起对比。`rangeLCS` 是限定两串各自下标区间 $[l,r)$、$[x,y)$ 的 LCS，用滚动一维 `f` 数组做标准转移，`pre` 暂存左上角值。`rangeLIS` 是区间内严格递增子序列长度，用经典的 `lower_bound` 维护「长度 k 的 LIS 的最小末元素」贪心数组。二者状态含义、答案都不同，不能相互套用；LCS 只有在一串元素互异时才能映射成另一串的位置序列转成 LIS。

```
int rangeLCS(const string &a, int l, int r, const string &b, int x, int y)
{
    // a、b 是两个字符串；[l,r)、[x,y) 是参与计算的半开区间；返回区间 LCS 长度。
    vector<int> f(y - x + 1);
    for (int i = l; i < r; i++)
    {
        int pre = 0;
        for (int j = x; j < y; j++)
        {
```

```
            int old = f[j - x + 1];
            if (a[i] == b[j])
            {
                f[j - x + 1] = pre + 1;
            }
            else
            {
                f[j - x + 1] = max(f[j - x + 1], f[j - x]);
            }
            pre = old;
        }
    }
    return f.back();
}

template <class T> int rangeLIS(const vector<T> &a, int l, int r)
{
    // a 是输入序列，[l,r) 是查询区间；返回严格递增子序列长度。
    vector<T> d;
    for (int i = l; i < r; i++)
    {
        auto it = lower_bound(d.begin(), d.end(), a[i]);
        if (it == d.end())
        {
            d.push_back(a[i]);
        }
        else
        {
            *it = a[i];
        }
    }
    return d.size();
}
```

`rangeLCS(a,l,r,b,x,y)` 查两个半开子串的 LCS 长度；令 $n=r-l$、$m=y-x$，时间为 $O((n+1)(m+1))$，空间为 $O(m+1)$。`rangeLIS(a,l,r)` 查半开区间内严格 LIS 长度，令 $L=r-l$，时间为 $O(1+L\log(L+1))$，空间为 $O(L+1)$。求可恢复的整串 LCS 用 4.5.001 最长公共子序列 LCS；连续公共片段用 4.5.004 多串最长公共子串/子序列。

4.5.004 多串最长公共子串/子序列

公共子串（连续）与公共子序列（可跳字符）是两个问题。`multiSubstr` 对首串建 SAM，再让其余每串在自动机上跑匹配、用 `best` 记录每状态能匹配到的最长长度并沿 link 上推，`mn` 取各串在每状态的最小匹配长度，最终最大值即多串最长公共子串。`multiSubseq` 是 k 维 DP：把各串下标打包成混合进制状态 `s`，同层字符全相等则从「全体减一」转移 $+1$，否则从某一维减一取最大。二者状态与答案不能混用。

```
int multiSubstr(const vector<string> &a)
{
    // a 是全部输入串；返回它们的最长公共子串长度。
```

```
    if (a.empty())
    {
        return 0;
    }
    struct N
    {
        // len 是该状态的最长长度，link 是后缀链接。
        int len = 0, link = -1;
        // to 按字节保存转移目标状态。
        unordered_map<unsigned char, int> to;
    };
    vector<N> t(1);
    int last = 0;
    for (unsigned char c : a[0])
    {
        int u = t.size();
        t.push_back({t[last].len + 1});
        int p = last;
        while (p != -1 && !t[p].to.contains(c))
        {
            t[p].to[c] = u;
            p = t[p].link;
        }
        if (p == -1)
        {
            t[u].link = 0;
        }
        else
        {
            int q = t[p].to[c];
            if (t[p].len + 1 == t[q].len)
            {
                t[u].link = q;
            }
            else
            {
                int v = t.size();
                t.push_back(t[q]);
                t[v].len = t[p].len + 1;
                while (p != -1)
                {
                    auto it = t[p].to.find(c);
                    if (it == t[p].to.end() || it->second != q)
                    {
                        break;
                    }
                    it->second = v;
                    p = t[p].link;
                }
                t[q].link = t[u].link = v;
            }
        }
```

```
        last = u;
    }
    vector<int> ord(t.size()), cnt(a[0].size() + 1), mn(t.size());
    for (const N &v : t)
    {
        cnt[v.len]++;
    }
    partial_sum(cnt.begin(), cnt.end(), cnt.begin());
    for (int u = (int)t.size() - 1; u >= 0; u--)
    {
        ord[--cnt[t[u].len]] = u;
    }
    reverse(ord.begin(), ord.end());
    for (int i = 0; i < (int)t.size(); i++)
    {
        mn[i] = t[i].len;
    }
    for (int k = 1; k < (int)a.size(); k++)
    {
        vector<int> best(t.size());
        int v = 0, l = 0;
        for (unsigned char c : a[k])
        {
            while (v && !t[v].to.contains(c))
            {
                v = t[v].link;
                l = min(l, t[v].len);
            }
            if (t[v].to.contains(c))
            {
                v = t[v].to[c];
                l++;
            }
            else
            {
                l = 0;
            }
            best[v] = max(best[v], l);
        }
        for (int u : ord)
        {
            if (t[u].link >= 0)
            {
                best[t[u].link] = max(best[t[u].link], min(best[u], t[t[u].link].len));
            }
        }
        for (int i = 0; i < (int)t.size(); i++)
        {
            mn[i] = min(mn[i], best[i]);
        }
    }
    return *max_element(mn.begin(), mn.end());
```

```cpp
}

optional<int> multiSubseq(const vector<string> &a)
{
    // a 是全部输入串；返回多串 LCS 长度，状态数无法用 size_t 表示时返回空。
    if (a.empty())
    {
        return 0;
    }
    int k = a.size();
    size_t states = 1;
    vector<size_t> st(k), base(k);
    for (int i = 0; i < k; i++)
    {
        size_t len = a[i].size() + 1;
        if (len > numeric_limits<size_t>::max() / states)
        {
            return nullopt;
        }
        base[i] = states;
        states *= len;
    }
    if (states > vector<int>().max_size())
    {
        return nullopt;
    }
    vector<int> f(states);
    for (size_t s = 1; s < states; s++)
    {
        size_t q = s;
        bool zero = false, same = true;
        int c = -1;
        size_t back = 0;
        for (int i = k - 1; i >= 0; i--)
        {
            st[i] = q / base[i];
            q %= base[i];
            if (!st[i])
            {
                zero = true;
            }
            else
            {
                int cur = (unsigned char)a[i][st[i] - 1];
                if (c == -1)
                {
                    c = cur;
                }
                else if (c != cur)
                {
                    same = false;
                }
```

```
            }
            back += base[i];
        }
        if (!zero && same)
        {
            f[s] = f[s - back] + 1;
        }
        else
        {
            for (int i = 0; i < k; i++)
            {
                if (st[i])
                {
                    f[s] = max(f[s], f[s - base[i]]);
                }
            }
        }
    }
    return f.back();
}
```

`multiSubstr(a)` 返回多串最长公共连续子串长度，答案不超过最短串长；首串产生至多 $2|a_0|-1$ 个状态，串长和状态编号须能由模板中的 64 位 `int` 表示。转移使用哈希表，总长 L 下为期望 $O(L+1)$ 时间、$O(|a_0|+1)$ 额外空间。`multiSubseq(a)` 的答案同样不超过最短串长；它在混合进制状态数不能由 `size_t` 或容器表示时返回空，非空输入需要 $O(k\prod_i(|a_i|+1))$ 时间和 $O(k+\prod_i(|a_i|+1))$ 空间，只适合串少且短。

4.5.005 最短公共超序列

最短公共超序列是同时以 `a`、`b` 为子序列的最短串，长度为 $|a|+|b|-\mathrm{LCS}(a,b)$。构造时先取一条 LCS，再沿 LCS 逐个对齐：把两串在下一个公共字符前的未匹配字符依次补入，公共字符只加一次，末尾补上各自剩余。

```
// 依赖 4.5.001 最长公共子序列 LCS
string scs(const string &a, const string &b)
{
    // a、b 是两个字符串；返回二者的一条最短公共超序列。
    vector<char> x(a.begin(), a.end()), y(b.begin(), b.end());
    auto c = lcs(x, y);
    string ans;
    int i = 0, j = 0;
    for (char z : c)
    {
        while (a[i] != z)
        {
            ans += a[i++];
        }
        while (b[j] != z)
        {
```

```
            ans += b[j++];
        }
        ans += z;
        i++;
        j++;
    }
    return ans + a.substr(i) + b.substr(j);
}
```

`scs(a,b)` 复用 4.5.001 最长公共子序列 LCS 恢复的 LCS 交错拼出一条最短公共超序列，时间、空间均为 $O((n+1)(m+1))$。它必须用能恢复序列的普通 LCS，位并行的 4.5.002 Bitset 优化 LCS 只给长度、无法回溯故不适用。

4.5.006 字符串周期、循环节与前后缀

字符串的周期、最小循环节、border、前后缀关系都能由前缀函数一次性得到，代码与 4.1.002 KMP / 前缀函数 完全相同，这里不重复：直接依赖其中的 `prefix`、`kmp`、`borders`、`minPeriod`。对非空串，前缀函数末端 π_{n-1} 的失配链枚举全部真 border，而 $n-\pi_{n-1}$ 是最短周期；它整除 n 时才是最小循环节，否则整串自身才是循环节。空串没有非空 border，`minPeriod("")` 返回 `0`。`prefix`、`borders`、`minPeriod` 为 $O(n+1)$ 时间与空间；`kmp` 对长度 n 的文本和长度 m 的模式为 $O(n+m+1)$ 时间、$O(m+occ+1)$ 空间，其中 occ 是匹配数。需要在文本中定位循环出现用 `kmp`，判前缀对齐用 4.1.003 扩展 KMP / Z 函数。

4.5.007 前缀-子串 LCS 批量查询

给定固定的首串 s、次串 t，每个查询要求 $s[0,a)$ 与 $t[l,r)$ 的 LCS 长度。逐个调用 4.5.003 区间 LCS / 区间 LIS 的动态规划要 $O(Q|s||t|)$，无法承受 $Q=5\times10^5$。对每个首串前缀维护 seaweed 边界的置换表示 `h`：扫描新字符时，交换恰能继续匹配或已经跨过当前边界的端点；边界值落在 $[0,l)$ 的数量正好给出查询矩形中没有被 LCS 使用的位置数。按右端点离线并用 Fenwick 计数，即可一次扫描回答该前缀的全部查询。

```
struct PrefixSubLCS
{
    struct Query
    {
        int a, l, r, id; // a 是首串前缀长度，l、r 是次串半开区间，id 是原查询编号。
    };

    static vector<int> solve(const string &s, const string &t, const vector<array<int, 3>>
    ↪ &ask)
    {
        // s、t 是小写字符串；ask 每项为 {前缀长度，左端，右端}，返回对应 LCS 长度。
        int m = s.size(), n = t.size(), q = ask.size();
        vector<int> ans(q);
        vector<vector<Query>> byA(m);
        for (int i = 0; i < q; i++)
        {
            auto [a, l, r] = ask[i];
```

```
            assert(0 <= a && a <= m && 0 <= l && l <= r && r <= n);
            if (a && l < r)
            {
                byA[a - 1].push_back({a, l, r, i});
            }
        }
        vector<int> h(n);
        iota(h.begin(), h.end(), 0);
        for (int a = 0; a < m; a++)
        {
            int cur = -1;
            for (int j = 0; j < n; j++)
            {
                if (s[a] == t[j] || h[j] < cur)
                {
                    swap(h[j], cur);
                }
            }
            vector<vector<Query>> byR(n);
            for (auto x : byA[a])
            {
                byR[x.r - 1].push_back(x);
            }
            vector<int> bit(n + 1);
            auto add = [&](int p)
            {
                for (p++; p <= n; p += p & -p)
                {
                    bit[p]++;
                }
            };
            auto sum = [&](int p)
            {
                int ret = 0;
                for (; p; p &= p - 1)
                {
                    ret += bit[p];
                }
                return ret;
            };
            int cnt = 0;
            for (int r = 0; r < n; r++)
            {
                if (h[r] != -1)
                {
                    add(h[r]);
                    cnt++;
                }
                for (auto x : byR[r])
                {
                    ans[x.id] = (x.r - x.l) - (cnt - sum(x.l));
                }
```

```
            }
        }
        return ans;
    }
};
```

`PrefixSubLCS::solve(s,t,ask)` 中每个 `ask[i]={a,l,r}` 使用半开下标，返回同顺序的 LCS 长度；允许空前缀和空子串。设 $m=|s|$、$n=|t|$，总时间为 $O(mn+Q\log n)$，额外空间为 $O(n+Q)$；题目范围 $m,n\le 1000$、$Q\le 5\times 10^5$。单个查询或需要恢复子序列时仍选 4.5.001 最长公共子序列 LCS 或 4.5.003 区间 LCS / 区间 LIS，本条只输出批量长度。